$$\text{Latency} = \text{Sending overhead} + T_{\text{LinkProp}} \times (d+1) + (T_{\text{r}} + \tau_{\text{a}} + T_{\text{S}}) \times d + \frac{(\text{Packet} + (d \times \text{Header}))}{\text{Bandwidth}} + \text{Receiving overhead}$$

The effective bandwidth is impacted by how efficiently routing, arbitration, and switching allow network bandwidth to be used. The routing algorithm can distribute traffic more evenly across a loaded network to increase the utilization of the aggregate bandwidth provided by the topology—particularly, by the bisection links. The arbitration algorithm can maximize the number of switch output ports that accept packets, which also increases the utilization of network bandwidth. The switching technique can increase the degree of resource sharing by packets, which further increases bandwidth utilization. These combine to affect network bandwidth, $\text{BW}_{\text{Network}}$, by an *efficiency factor*, ρ, where $0 < \rho \leq 1$:

$$\text{BW}_{\text{Network}} = \rho \times \frac{\text{BW}_{\text{Bisection}}}{\gamma}$$

The efficiency factor, ρ, is difficult to calculate or to quantify by means other than simulation. Nevertheless, with this parameter we can estimate the best-case upper-bound effective bandwidth by using the following expression that takes into account the effects of routing, arbitration, and switching:

$$\text{Effective bandwidth} = \min\left(N \times \text{BW}_{\text{LinkInjection}},\ \rho \times \frac{\text{BW}_{\text{Bisection}}}{\gamma}, \sigma \times N \times \text{BW}_{\text{LinkReception}}\right)$$

We note that ρ also depends on how well the network handles the traffic generated by applications. For instance, ρ could be higher for circuit switching than for cut-through switching if large streams of packets are continually transmitted between a source-destination pair, whereas the converse could be true if packets are transmitted intermittently.

Example Compare the performance of deterministic routing versus adaptive routing for a 3D torus network interconnecting 4096 nodes. Do so by plotting latency versus applied load and throughput versus applied load. Also compare the efficiency of the best and worst of these networks. Assume that virtual cut-through switching, three-phase arbitration, and virtual channels are implemented. Consider separately the cases for two and four virtual channels, respectively. Assume that one of the virtual channels uses bubble flow control in dimension order so as to avoid deadlock; the other virtual channels are used either in dimension order (for deterministic routing) or minimally along shortest paths (for adaptive routing), as is done in the IBM Blue Gene/L torus network.

Answer It is very difficult to compute analytically the performance of routing algorithms given that their behavior depends on several network design parameters with complex interdependences among them. As a consequence, designers typically resort to cycle-accurate simulators to evaluate performance. One way to evaluate the effect of a certain design decision is to run sets of simulations over a range of network loads, each time modifying one of the design parameters of interest while

keeping the remaining ones fixed. The use of synthetic traffic loads is quite frequent in these evaluations as it allows the network to stabilize at a certain working point and for behavior to be analyzed in detail. This is the method we use here (alternatively, trace-driven or execution-driven simulation can be used).

Figure F.19 shows the typical interconnection network performance plots. On the left, average packet latency (expressed in network cycles) is plotted as a function of applied load (traffic generation rate) for the two routing algorithms with two and four virtual channels each; on the right, throughput (traffic delivery rate) is similarly plotted. Applied load is normalized by dividing it by the number of nodes in the network (i.e., bytes per cycle per node). Simulations are run under the assumption of uniformly distributed traffic consisting of 256-byte packets, where flits are byte sized. Routing, arbitration, and switching delays are assumed to sum to 1 network cycle per hop while the time-of-flight delay over each link is assumed to be 10 cycles. Link bandwidth is 1 byte per cycle, thus providing results that are independent of network clock frequency.

As can be seen, the plots within each graph have similar characteristic shapes, but they have different values. For the latency graph, all start at the no-load latency

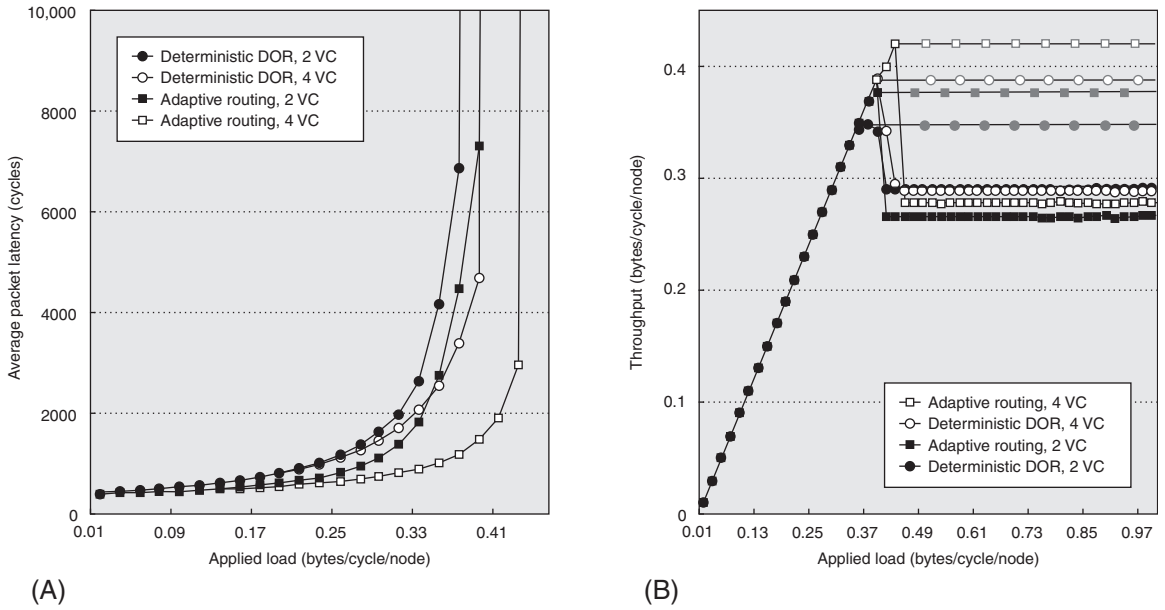


Figure F.19 Deterministic routing is compared against adaptive routing, both with either two or four virtual channels, assuming uniformly distributed traffic on a 4 K node 3D torus network with virtual cut-through switching and bubble flow control to avoid deadlock. (a) Average latency is plotted versus applied load, and (b) throughput is plotted versus applied load (the upper grayish plots show peak throughput, and the lower black plots show sustained throughput). Simulation data were collected by P. Gilabert and J. Flich at the Universidad Politècnica de València, Spain (2006).

as predicted by the latency expression given above, then slightly increase with traffic load as contention for network resources increases. At higher applied loads, latency increases exponentially, and the network approaches its saturation point as it is unable to absorb the applied load, = causing packets to queue up at their source nodes awaiting injection. In these simulations, the queues keep growing over time, making latency tend toward infinity. However, in practice, queues reach their capacity and trigger the application to stall further packet generation, or the application throttles itself waiting for acknowledgments/responses to outstanding packets. Nevertheless, latency grows at a slower rate for adaptive routing as alternative paths are provided to packets along congested resources.

For this same reason, adaptive routing allows the network to reach a higher peak throughput for the same number of virtual channels as compared to deterministic routing. At nonsaturation loads, throughput increases fairly linearly with applied load. When the network reaches its saturation point, however, it is unable to deliver traffic at the same rate at which traffic is generated. The saturation point, therefore, indicates the maximum achievable or "peak" throughput, which would be no more than that predicted by the effective bandwidth expression given above. Beyond saturation, throughput tends to drop as a consequence of massive head-of-line blocking across the network (as will be explained further in Section F.6), very much like cars tend to advance more slowly at rush hour. This is an important region of the throughput graph as it shows how significant of a performance drop the routing algorithm can cause if congestion management techniques (discussed briefly in Section F.7) are not used effectively. In this case, adaptive routing has more of a performance drop after saturation than deterministic routing, as measured by the postsaturation sustained throughput.

For both routing algorithms, more virtual channels (i.e., four) give packets a greater ability to pass over blocked packets ahead, allowing for a higher peak throughput as compared to fewer virtual channels (i.e., two). For adaptive routing with four virtual channels, the peak throughput of 0.43 bytes/cycle/node is near the maximum of 0.5 bytes/cycle/node that can be obtained with 100% efficiency (i.e., $\rho = 100\%$), assuming there is enough injection and reception bandwidth to make the network bisection the bottlenecking point. In that case, the network bandwidth is simply 100% times the network bisection bandwidth ($BW_{Bisection}$) divided by the fraction of traffic crossing the bisection (γ), as given by the expression above. Taking into account that the bisection splits the torus into two equally sized halves, γ is equal to 0.5 for uniform traffic as only half the injected traffic is destined to a node at the other side of the bisection. The $BW_{Bisection}$ for a 4096-node 3D torus network is $16 \times 16 \times 4$ unidirectional links times the link bandwidth (i.e., 1 byte/cycle). If we normalize the bisection bandwidth by dividing it by the number of nodes (as we did with network bandwidth), the $BW_{Bisection}$ is 0.25 bytes/cycle/node. Dividing this by γ gives the ideal maximally obtainable network bandwidth of 0.5 bytes/cycle/node.

We can find the efficiency factor, ρ, of the simulated network simply by dividing the measured peak throughput by the ideal throughput. The efficiency factor for

the network with fully adaptive routing and four virtual channels is 0.43/(0.25/0.5) = 86%, whereas for the network with deterministic routing and two virtual channels it is 0.37/(0.25/0.5) = 74%. Besides the 12% difference in efficiency between the two, another 14% gain in efficiency might be obtained with even better routing, arbitration, switching, and virtual channel designs.

To put this discussion on routing, arbitration, and switching in perspective, Figure F.20 lists the techniques used in SANs designed for commercial high-performance computers. In addition to being applied to the SANs as shown in the figure, the issues discussed in this section also apply to other interconnect domains: from OCNs to WANs.

F.6 Switch Microarchitecture

Network switches implement the routing, arbitration, and switching functions of switched-media networks. Switches also implement buffer management mechanisms and, in the case of lossless networks, the associated flow control. For some networks, switches also implement part of the network management functions that explore, configure, and reconfigure the network topology in response to boot-up and failures. Here, we reveal the internal structure of network switches by describing a basic switch microarchitecture and various alternatives suitable for different routing, arbitration, and switching techniques presented previously.

Basic Switch Microarchitecture

The internal data path of a switch provides connectivity among the input and output ports. Although a shared bus or a multiported central memory could be used, these solutions are insufficient or too expensive, respectively, when the required aggregate switch bandwidth is high. Most high-performance switches implement an internal crossbar to provide nonblocking connectivity within the switch, thus allowing concurrent connections between multiple input-output port pairs. Buffering of blocked packets can be done using first in, first out (FIFO) or circular queues, which can be implemented as *dynamically allocatable multi-queues* (DAMQs) in static RAM to provide high capacity and flexibility. These queues can be placed at input ports (i.e., *input buffered switch*), output ports (i.e., *output buffered switch*), centrally within the switch (i.e., *centrally buffered switch*), or at both the input and output ports of the switch (i.e., *input-output-buffered switch*). Figure F.21 shows a block diagram of an input-output-buffered switch.

Routing can be implemented using a finite-state machine or forwarding table within the routing control unit of switches. In the former case, the routing information given in the packet header is processed by a finite-state machine that determines the allowed switch output port (or ports if routing is adaptive), according to the routing algorithm. Portions of the routing information in the header are usually

Company	System [network] name	Max. number of nodes [× # CPUs]	Basic network topology	Switch queuing (buffers)	Network routing algorithm	Switch arbitration technique	Network switching technique
Intel	ASCI Red Paragon	4510 [×2]	2D mesh (64 × 64)	Input buffered (1 flit)	Distributed dimension-order routing	2-phased RR, distributed across switch	Wormhole with no virtual channels
IBM	ASCI White SP Power3 [Colony]	512 [×16]	Bidirectional MIN with 8-port bidirectional switches (typically a fat tree or Omega)	Input and central buffer with output queuing (8-way speedup)	Source-based LCA adaptive, shortest-path routing, and table-based multicast routing	2-phased RR, centralized and distributed at outputs for bypass paths	Buffered wormhole and virtual cut-through for multicasting, no virtual channels
Intel	Thunder Itanium2 Tiger4 [QsNet^{II}]	1024 [×4]	Fat tree with 8-port bidirectional switches	Input buffered	Source-based LCA adaptive, shortest-path routing	2-phased RR, priority, aging, distributed at output ports	Wormhole with 2 virtual channels
Cray	XT3 [SeaStar]	30,508 [×1]	3D torus (40 × 32 × 24)	Input with staging output	Distributed table-based dimension-order routing	2-phased RR, distributed at output ports	Virtual cut-through with 4 virtual channels
Cray	X1E	1024 [×1]	4-way bristled 2D torus (∼23 × 11) with express links	Input with virtual output queuing	Distributed table-based dimension-order routing	2-phased wavefront (pipelined) global arbiter	Virtual cut-through with 4 virtual channels
IBM	ASC Purple pSeries 575 [Federation]	>1280 [×8]	Bidirectional MIN with 8-port bidirectional switches (typically a fat tree or Omega)	Input and central buffer with output queuing (8-way speedup)	Source and distributed table-based LCA adaptive, shortest-path routing, and multicast	2-phased RR, centralized and distributed at outputs for bypass paths	Buffered wormhole and virtual cut-through for multicasting with 8 virtual channels
IBM	Blue Gene/L eServer Solution [Torus Net.]	65,536 [×2]	3D torus (32 × 32 × 64)	Input-output buffered	Distributed, adaptive with bubble escape virtual channel	2-phased SLQ, distributed at input and output	Virtual cut-through with 4 virtual channels

Figure F.20 Routing, arbitration, and switching characteristics of interconnections networks in commercial machines.

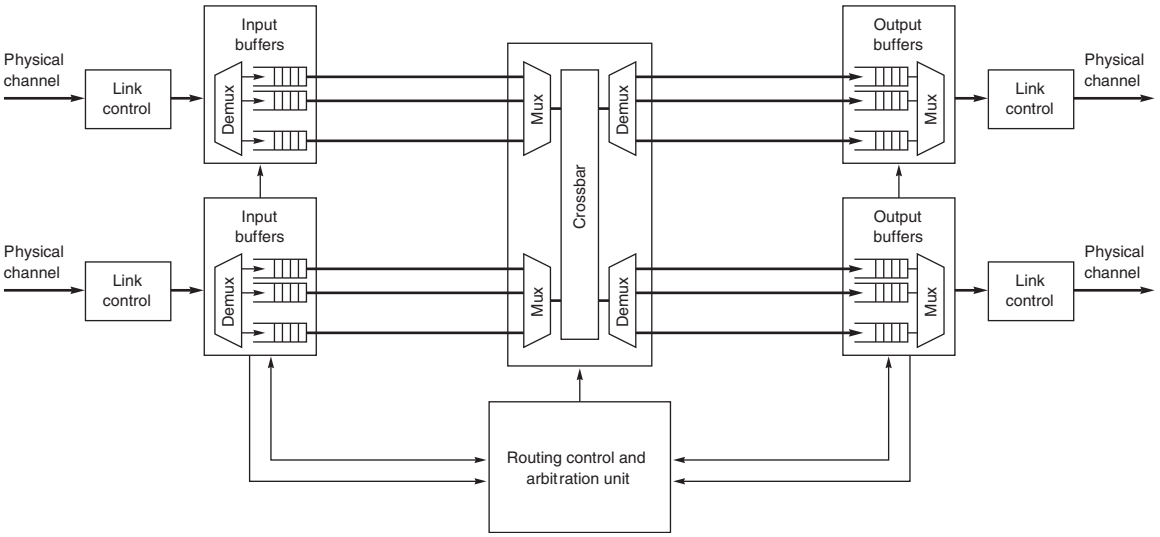


Figure F.21 Basic microarchitectural components of an input-output-buffered switch.

stripped off or modified by the routing control unit after use to simplify processing at the next switch along the path. When routing is implemented using forwarding tables, the routing information given in the packet header is used as an address to access a forwarding table entry that contains the allowed switch output port(s) provided by the routing algorithm. Forwarding tables must be preloaded into the switches at the outset of network operation. Hybrid approaches also exist where the forwarding table is reduced to a small set of routing bits and combined with a small logic block. Those routing bits are used by the routing control unit to know what paths are allowed and decide the output ports the packets need to take. The goal with those approaches is to build flexible yet compact routing control units, eliminating the area and power wastage of a large forwarding table and thus being suitable for OCNs. The routing control unit is usually implemented as a centralized resource, although it could be replicated at every input port so as not to become a bottleneck. Routing is done only once for every packet, and packets typically are large enough to take several cycles to flow through the switch, so a centralized routing control unit rarely becomes a bottleneck. Figure F.21 assumes a centralized routing control unit within the switch.

Arbitration is required when two or more packets concurrently request the same output port, as described in the previous section. Switch arbitration can be implemented in a centralized or distributed way. In the former case, all of the requests and status information are transmitted to the central switch arbitration unit; in the latter case, the arbiter is distributed across the switch, usually among the input and/or output ports. Arbitration may be performed multiple times on packets, and there may be multiple queues associated with each input port,

increasing the number of arbitration requests that must be processed. Thus, many implementations use a hierarchical arbitration approach, where arbitration is first performed locally at every input port to select just one request among the corresponding packets and queues, and later arbitration is performed globally to process the requests made by each of the local input port arbiters. Figure F.21 assumes a centralized arbitration unit within the switch.

The basic switch microarchitecture depicted in Figure F.21 functions in the following way. When a packet starts to arrive at a switch input port, the link controller decodes the incoming signal and generates a sequence of bits, possibly deserializing data to adapt them to the width of the internal data path if different from the external link width. Information is also extracted from the packet header or link control signals to determine the queue to which the packet should be buffered. As the packet is being received and buffered (or after the entire packet has been buffered, depending on the switching technique), the header is sent to the routing unit. This unit supplies a request for one or more output ports to the arbitration unit. Arbitration for the requested output port succeeds if the port is free and has enough space to buffer the entire packet or flit, depending on the switching technique. If wormhole switching with virtual channels is implemented, additional arbitration and allocation steps may be required for the transmission of each individual flit. Once the resources are allocated, the packet is transferred across the internal crossbar to the corresponding output buffer and link if no other packets are ahead of it and the link is free. Link-level flow control implemented by the link controller prevents input queue overflow at the neighboring switch on the other end of the link. If virtual channel switching is implemented, several packets may be time-multiplexed across the link on a flit-by-flit basis. As the various input and output ports operate independently, several incoming packets may be processed concurrently in the absence of contention.

Buffer Organizations

As mentioned above, queues can be located at the switch input, output, or both sides. Output-buffered switches have the advantage of completely eliminating *head-of-line blocking*. Head-of-line (HOL) blocking occurs when two or more packets are buffered in a queue, and a blocked packet at the head of the queue blocks other packets in the queue that would otherwise be able to advance if they were at the queue head. This cannot occur in output-buffered switches as all the packets in a given queue have the same status; they require the same output port. However, it may be the case that all the switch input ports simultaneously receive a packet for the same output port. As there are no buffers at the input side, output buffers must be able to store all those incoming packets at the same time. This requires implementing output queues with an internal switch *speedup* of k. That is, output queues must have a write bandwidth k times the link bandwidth, where k is the number of switch ports. This oftentimes is too expensive. Hence, this solution by itself has rarely been implemented in lossless networks. As the probability of concurrently receiving many packets for the same output port is usually small,

commercial systems that use output-buffered switches typically implement only moderate switch speedup, dropping packets on rare buffer overflow.

Switches with buffers on the input side are able to receive packets without having any switch speedup; however, HOL blocking can occur within input port queues, as illustrated in Figure F.22(a). This can reduce switch output port utilization to less than 60% even when packet destinations are uniformly distributed. As shown in Figure F.22(b), the use of virtual channels (two in this case) can mitigate HOL blocking but does not eliminate it. A more effective solution is to organize the input queues as *virtual output queues* (VOQs), shown in Figure F.22(c). With this, each input port implements as many queues as there are output ports, thus providing separate buffers for packets destined to different output ports. This is a popular technique widely used in ATM switches and IP routers. The main drawbacks of

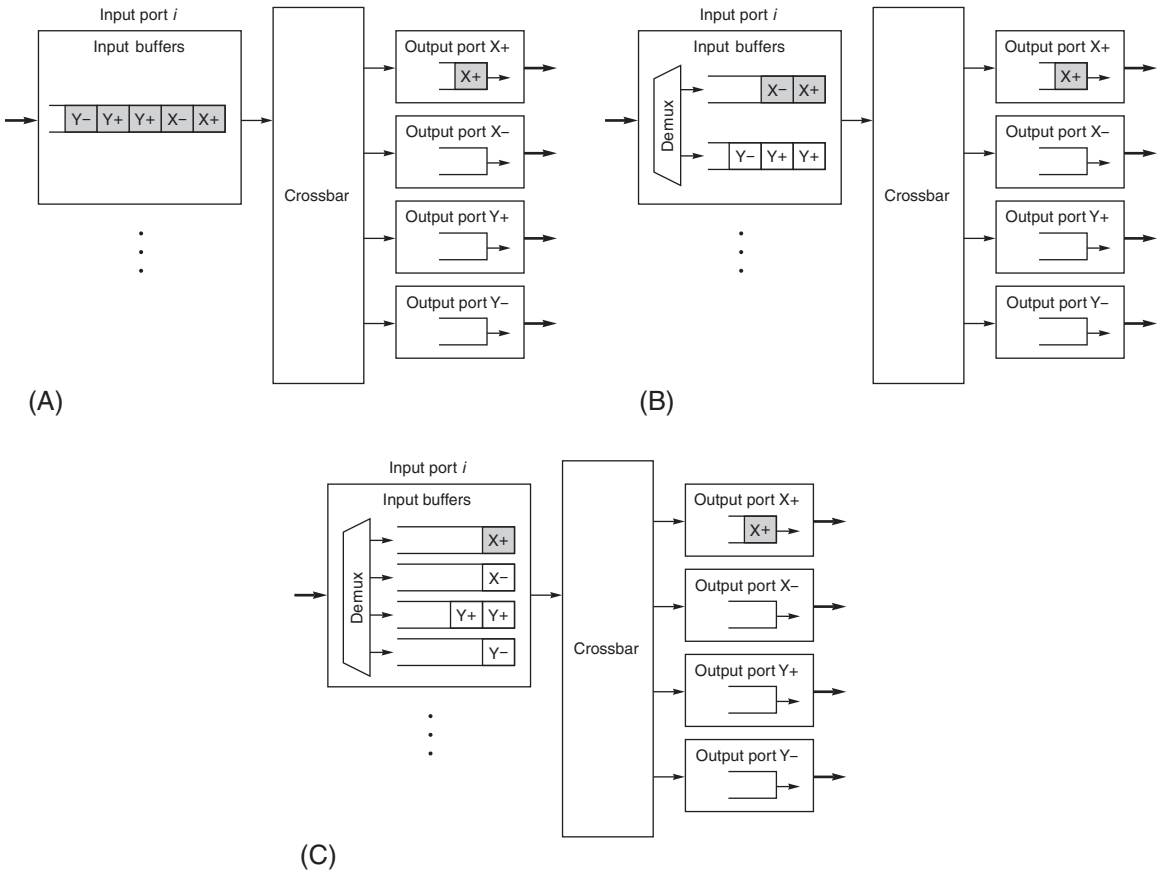


Figure F.22 (a) Head-of-line blocking in an input buffer, (b) the use of two virtual channels to reduce HOL blocking, and (c) the use of virtual output queuing to eliminate HOL blocking within a switch. The shaded input buffer is the one to which the crossbar is currently allocated. This assumes each input port has only one access port to the switch's internal crossbar.

VOQs, however, are cost and lack of scalability: The number of VOQs grows quadratically with switch ports. Moreover, although VOQs eliminate HOL blocking within a switch, HOL blocking occurring at the network level end-to-end is not solved. Of course, it is possible to design a switch with VOQ support at the network level also—that is, to implement as many queues per switch input port as there are output ports across the entire network—but this is extremely expensive. An alternative is to dynamically assign only a fraction of the queues to store (cache) separately only those packets headed for congested destinations.

Combined input-output-buffered switches minimize HOL blocking when there is sufficient buffer space at the output side to buffer packets, and they minimize the switch speedup required due to buffers being at the input side. This solution has the further benefit of decoupling packet transmission through the internal crossbar of the switch from transmission through the external links. This is especially useful for cut-through switching implementations that use virtual channels, where flit transmissions are time-multiplexed over the links. Many designs used in commercial systems implement input-output-buffered switches.

Routing Algorithm Implementation

It is important to distinguish between the routing algorithm and its implementation. While the routing algorithm describes the rules to forward packets across the network and affects packet latency and network throughput, its implementation affects the delay suffered by packets when reaching a node, the required silicon area, and the power consumption associated with the routing computation. Several techniques have been proposed to pre-compute the routing algorithm and/or hide the routing computation delay. However, significantly less effort has been devoted to reduce silicon area and power consumption without significantly affecting routing flexibility. Both issues have become very important, particularly for OCNs. Many existing designs address these issues by implementing relatively simple routing algorithms, but more sophisticated routing algorithms will likely be needed in the future to deal with increasing manufacturing defects, process variability, and other complications arising from continued technology scaling, as discussed briefly below.

As mentioned in a previous section, depending on where the routing algorithm is computed, two basic forms of routing exist: source and distributed routing. In source routing, the complexity of implementation is moved to the end nodes where paths need to be stored in tables, and the path for a given packet is selected based on the destination end node identifier. In distributed routing, however, the complexity is moved to the switches where, at each hop along the path of a packet, a selection of the output port to take is performed. In distributed routing, two basic implementations exist. The first one consists of using a logic block that implements a fixed routing algorithm for a particular topology. The most common example of such an implementation is dimension-order routing, where dimensions are offset in an established order. Alternatively, distributed routing can be implemented with forwarding tables, where each entry encodes the output port to be used for a particular

destination. Therefore, in the worst case, as many entries as destination nodes are required.

Both methods for implementing distributed routing have their benefits and drawbacks. Logic-based routing features a very short computation delay, usually requires a small silicon area, and has low power consumption. However, logic-based routing needs to be designed with a specific topology in mind and, therefore, is restricted to that topology. Table-based distributed routing is quite flexible and supports any topology and routing algorithm. Simply, tables need to be filled with the proper contents based on the applied routing algorithm (e.g., the up*/down* routing algorithm can be defined for any irregular topology). However, the down side of table-based distributed routing is its non-negligible area and power cost. Also, scalability is problematic in table-based solutions as, in the worst case, a system with N end nodes (and switches) requires as many as N tables each with N entries, thus having quadratic cost.

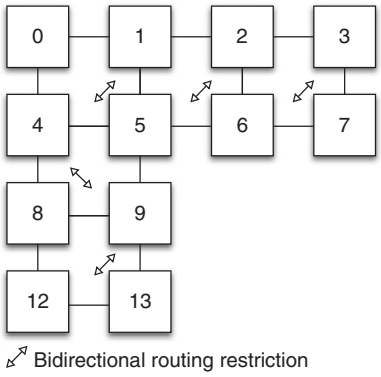
Depending on the network domain, one solution is more suitable than the other. For instance, in SANs, it is usual to find table-based solutions as is the case with InfiniBand. In other environments, like OCNs, table-based implementations are avoided due to the aforementioned costs in power and silicon area. In such environments, it is more advisable to rely on logic-based implementations. Herein lies some of the challenges OCN designers face: ever continuing technology scaling through device miniaturization leads to increases in the number of manufacturing defects, higher failure rates (either transient or permanent), significant process variations (transistors behaving differently from design specs), the need for different clock frequency and voltage domains, and tight power and energy budgets. All of these challenges translate to the network needing support for heterogeneity. Different—possibly irregular—regions of the network will be created owing to failed components, powered down switches and links, disabled components (due to unacceptable variations in performance) and so on. Hence, heterogeneous systems may emerge from a homogeneous design. In this framework, it is important to efficiently implement routing algorithms designed to provide enough flexibility to address these new challenges.

A well-known solution for providing a certain degree of flexibility while being much more compact than traditional table-based approaches is interval routing [Leeuwen 1987], where a range of destinations is defined for each output port. Although this approach is not flexible enough, it provides a clue on how to address emerging challenges. A more recent approach provides a plausible implementation design point that lies between logic-based implementation (efficiency) and table-based implementation (flexibility). Logic-Based Distributed Routing (LBDR) is a hybrid approach that takes as a reference a regular 2D mesh but allows an irregular network to be derived from it due to changes in topology induced by manufacturing defects, failures, and other anomalies. Due to the faulty, disabled, and powered-down components, regularity is compromised and the dimension-order routing algorithm can no longer be used. To support such topologies, LBDR defines a set of configuration bits at each switch. Four connectivity bits are used at each switch to indicate the connectivity of the switch to the neighbor switches in the

topology. Thus, one connectivity bit per port is used. Those connectivity bits are used, for instance, to disable an output port leading to a faulty component. Additionally, eight routing bits are used, two per output port, to define the available routing options. The value of the routing bits is set at power-on and is computed from the routing algorithm to be implemented in the network. Basically, when a routing bit is set, it indicates that a packet can leave the switch through the associated output port and is allowed to perform a certain turn at the next switch. In this respect, LBDR is similar to interval routing, but it defines geographical areas instead of ranges of destinations. Figure F.23 shows an example where a topology-agnostic routing algorithm is implemented with LBDR on an irregular topology. The figure shows the computed configuration bits.

The connectivity and routing bits are used to implement the routing algorithm. For that purpose, a small set of logic gates are used in combination with the configuration bits. Basically, the LBDR approach takes as a reference the initial topology (a 2D mesh), and makes a decision based on the current coordinates of the router, the coordinates of the destination router, and the configuration bits. Figure F.24 shows the required logic, and Figure F.25 shows an example of where a packet is forwarded from its source to its destination with the use of the configuration bits. As can be noticed, routing restrictions are enforced by preventing the use of the west port at switch 10.

LBDR represents a method for efficient routing implementation in OCNs. This mechanism has been recently extended to support non-minimal paths, collective communication operations, and traffic isolation. All of these improvements have been made while maintaining a compact and efficient implementation with the use of a small set of configuration bits. A detailed description of LBDR and its extensions, and the current research on OCNs can be found in Flich [2010].



Router	Cn	Ce	Cw	Cs
0	0	1	0	1
1	0	1	1	1
2	0	1	1	1
3	0	0	1	1
4	1	1	0	1
5	1	1	1	1
6	1	1	1	0
7	1	0	1	0
8	1	1	0	1
9	1	0	1	1
10	-	-	-	-
11	-	-	-	-
12	1	1	0	0
13	1	0	1	0
14	-	-	-	-
15	-	-	-	-

Rne	Rnw
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
-	-
-	-
1	1
1	1
-	-
-	-

Ren	Res
1	1
1	1
1	1
1	1
0	1
0	1
0	1
1	1
1	1
1	1
-	-
-	-
0	1
1	1
-	-
-	-

Rwn	Rws
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
-	-
-	-
1	1
1	1
-	-
-	-

Rse	Rsw
1	1
1	0
1	0
1	0
0	1
1	1
1	1
1	1
1	1
1	0
-	-
-	-
1	1
1	1
-	-
-	-

Figure F.23 Shown is an example of an irregular network that uses LBDR to implement the routing algorithm. For each router, connectivity and routing bits are defined.

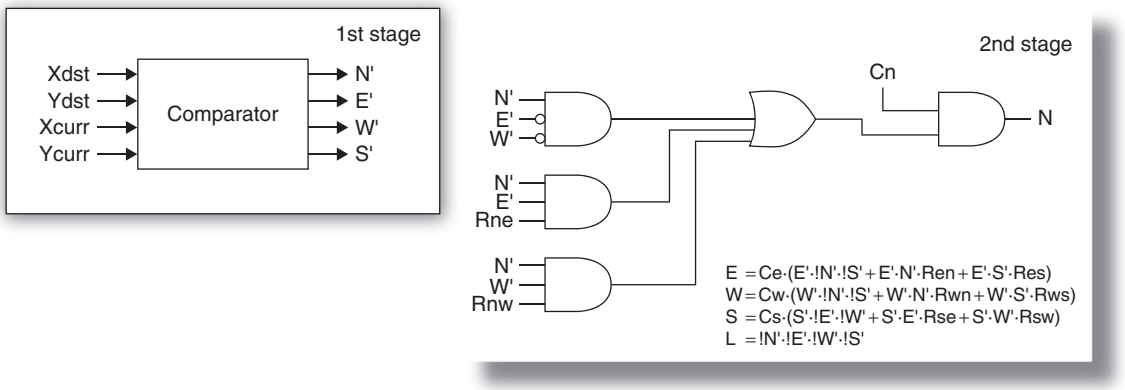


Figure F.24 LBDR logic at each input port of the router.

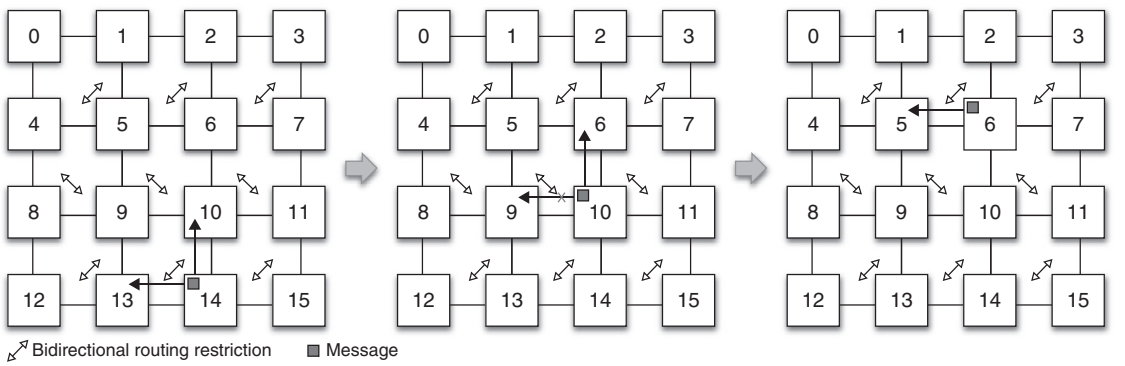


Figure F.25 Example of routing a message from Router 14 to Router 5 using LBDR at each router.

Pipelining the Switch Microarchitecture

Performance can be enhanced by pipelining the switch microarchitecture. Pipelined processing of packets in a switch has similarities with pipelined execution of instructions in a vector processor. In a vector pipeline, a single instruction indicates what operation to apply to all the vector elements executed in a pipelined way. Similarly, in a switch pipeline, a single packet header indicates how to process all of the internal data path physical transfer units (or *phits*) of a packet, which are processed in a pipelined fashion. Also, as packets at different input ports are independent of each other, they can be processed in parallel similar to the way multiple independent instructions or threads of pipelined instructions can be executed in parallel.

The switch microarchitecture can be pipelined by analyzing the basic functions performed within the switch and organizing them into several stages. Figure F.26 shows a block diagram of a five-stage pipelined organization for the basic switch microarchitecture given in Figure F.21, assuming cut-through switching and the use of a forwarding table to implement routing. After receiving the header portion of the packet in the first stage, the routing information (i.e., destination address) is used in the second stage to look up the allowed routing option(s) in the forwarding table. Concurrent with this, other portions of the packet are received and buffered in the input port queue at the first stage. Arbitration is performed in the third stage. The crossbar is configured to allocate the granted output port for the packet in the fourth stage, and the packet header is buffered in the switch output port and ready for transmission over the external link in the fifth stage. Note that the second and

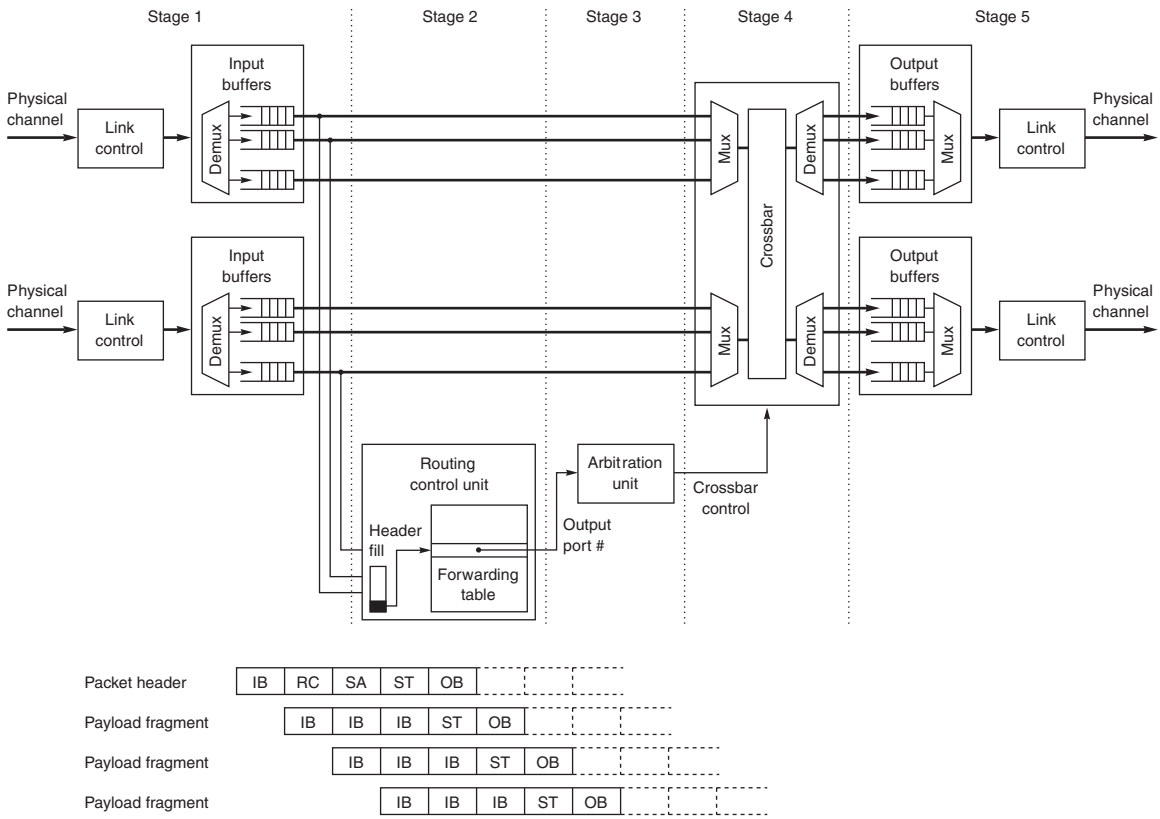


Figure F.26 Pipelined version of the basic input-output-buffered switch. The notation in the figure is as follows: IB is the input link control and buffer stage, RC is the route computation stage, SA is the crossbar switch arbitration stage, ST is the crossbar switch traversal stage, and OB is the output buffer and link control stage. Packet fragments (flits) coming after the header remain in the IB stage until the header is processed and the crossbar switch resources are provided.

third stages are used only by the packet header; the payload and trailer portions of the packet use only three of the stages—those used for data flow-thru once the internal data path of the switch is set up.

A virtual channel switch usually requires an additional stage for virtual channel allocation. Moreover, arbitration is required for every flit before transmission through the crossbar. Finally, depending on the complexity of the routing and arbitration algorithms, several clock cycles may be required for these operations.

Other Switch Microarchitecture Enhancements

As mentioned earlier, internal switch speedup is sometimes implemented to increase switch output port utilization. This speedup is usually implemented by increasing the clock frequency and/or the internal data path width (i.e., phit size) of the switch. An alternative solution consists of implementing several parallel data paths from each input port's set of queues to the output ports. One way of doing this is by increasing the number of crossbar input ports. When implementing several physical queues per input port, this can be achieved by devoting a separate crossbar port to each input queue. For example, the IBM Blue Gene/L implements two crossbar access ports and two read ports per switch input port.

Another way of implementing parallel data paths between input and output ports is to move the buffers to the crossbar crosspoints. This switch architecture is usually referred to as a *buffered crossbar switch*. A buffered crossbar provides independent data paths from each input port to the different output ports, thus making it possible to send up to k packets at a time from a given input port to k different output ports. By implementing independent crosspoint memories for each input-output port pair, HOL blocking is eliminated at the switch level. Moreover, arbitration is significantly simpler than in other switch architectures. Effectively, each output port can receive packets from only a disjoint subset of the crosspoint memories. Thus, a completely independent arbiter can be implemented at each switch output port, each of those arbiters being very simple.

A buffered crossbar would be the ideal switch architecture if it were not so expensive. The number of crosspoint memories increases quadratically with the number of switch ports, dramatically increasing its cost and reducing its scalability with respect to the basic switch architecture. In addition, each crosspoint memory must be large enough to efficiently implement link-level flow control. To reduce cost, most designers prefer input-buffered or combined input-output-buffered switches enhanced with some of the mechanisms described previously.

F.7 Practical Issues for Commercial Interconnection Networks

There are practical issues in addition to the technical issues described thus far that are important considerations for interconnection networks within certain domains. We mention a few of these below.

Connectivity

The type and number of devices that communicate and their communication requirements affect the complexity of the interconnection network and its protocols. The protocols must target the largest network size and handle the types of anomalous systemwide events that might occur. Among some of the issues are the following: How lightweight should the network interface hardware/software be? Should it attach to the memory network or the I/O network? Should it support cache coherence? If the operating system must get involved for every network transaction, the sending and receiving overhead becomes quite large. If the network interface attaches to the I/O network (PCI-Express or HyperTransport interconnect), the injection and reception bandwidth will be limited to that of the I/O network. This is the case for the Cray XT3 SeaStar, Intel Thunder Tiger 4 QsNet^{II}, and many other supercomputer and cluster networks. To support coherence, the sender may have to flush the cache before each send, and the receiver may have to flush its cache before each receive to prevent the stale-data problem. Such flushes further increase sending and receiving overhead, often causing the network interface to be the network bottleneck.

Computer systems typically have a multiplicity of interconnects with different functions and cost-performance objectives. For example, processor-memory interconnects usually provide higher bandwidth and lower latency than I/O interconnects and are more likely to support cache coherence, but they are less likely to follow or become standards. Personal computers typically have a processormemory interconnect and an I/O interconnect (e.g., PCI-X 2.0, PCIe or Hyper-Transport) designed to connect both fast and slow devices (e.g., USB 2.0, Gigabit Ethernet LAN, Firewire 800). The Blue Gene/L supercomputer uses five interconnection networks, only one of which is the 3D torus used for most of the interprocessor application traffic. The others include a tree-based collective communication network for broadcast and multicast; a tree-based barrier network for combining results (scatter, gather); a control network for diagnostics, debugging, and initialization; and a Gigabit Ethernet network for I/O between the nodes and disk. The University of Texas at Austin's TRIPS Edge processor has eight specialized on-chip networks—some with bidirectional channels as wide as 128 bits and some with 168 bits in each direction—to interconnect the 106 heterogeneous tiles composing the two processor cores with L2 on-chip cache. It also has a chip-to-chip switched network to interconnect multiple chips in a multiprocessor configuration. Two of the on-chip networks are switched networks: One is used for operand transport and the other is used for on-chip memory communication. The others are essentially fan-out trees or recombination dedicated link networks used for status and control. The portion of chip area allocated to the interconnect is substantial, with five of the seven metal layers used for global network wiring.

Standardization: Cross-Company Interoperability

Standards are useful in many places in computer design, including interconnection networks. Advantages of successful standards include low cost and stability.

The customer has many vendors to choose from, which keeps price close to cost due to competition. It makes the viability of the interconnection independent of the stability of a single company. Components designed for a standard interconnection may also have a larger market, and this higher volume can reduce the vendors' costs, further benefiting the customer. Finally, a standard allows many companies to build products with interfaces to the standard, so the customer does not have to wait for a single company to develop interfaces to all the products of interest.

One drawback of standards is the time it takes for committees and special-interest groups to agree on the definition of standards, which is a problem when technology is changing rapidly. Another problem is *when* to standardize: On the one hand, designers would like to have a standard before anything is built; on the other hand, it would be better if something were built before standardization to avoid legislating useless features or omitting important ones. When done too early, it is often done entirely by committee, which is like asking all of the chefs in France to prepare a single dish of food—masterpieces are rarely served. Standards can also suppress innovation at that level, since standards fix the interfaces—at least until the next version of the standards surface, which can be every few years or longer. More often, we are seeing consortiums of companies getting together to define and agree on technology that serve as "*de facto*" industry standards. This was the case for InfiniBand.

LANs and WANs use standards and interoperate effectively. WANs involve many types of companies and must connect to many brands of computers, so it is difficult to imagine a proprietary WAN ever being successful. The ubiquitous nature of the Ethernet shows the popularity of standards for LANs as well as WANs, and it seems unlikely that many customers would tie the viability of their LAN to the stability of a single company. Some SANs are standardized such as Fibre Channel, but most are proprietary. OCNs for the most part are proprietary designs, with a few gaining widespread commercial use in system-on-chip (SoC) applications, such as IBM's CoreConnect and ARM's AMBA.

Congestion Management

Congestion arises when too many packets try to use the same link or set of links. This leads to a situation in which the bandwidth required exceeds the bandwidth supplied. Congestion by itself does not degrade network performance: simply, the congested links are running at their maximum capacity. Performance degradation occurs in the presence of HOL blocking where, as a consequence of packets going to noncongested destinations getting blocked by packets going to congested destinations, some link bandwidth is wasted and network throughput drops, as illustrated in the example given at the end of Section F.4. *Congestion control* refers to schemes that reduce traffic when the collective traffic of all nodes is too large for the network to handle.

One advantage of a circuit-switched network is that, once a circuit is established, it ensures that there is sufficient bandwidth to deliver all the information

sent along that circuit. Interconnection bandwidth is reserved as circuits are established, and if the network is full, no more circuits can be established. Other switching techniques generally do not reserve interconnect bandwidth in advance, so the interconnection network can become clogged with too many packets. Just as with poor rush-hour commuters, a traffic jam of packets increases packet latency and, in extreme cases, fewer packets per second get delivered by the interconnect. In order to handle congestion in packet-switched networks, some form of *congestion management* must be implemented. The two kinds of mechanisms used are those that control congestion and those that eliminate the performance degradation introduced by congestion.

There are three basic schemes used for congestion control in interconnection networks, each with its own weaknesses: packet discarding, flow control, and choke packets. The simplest scheme is *packet discarding*, which we discussed briefly in Section F.2. If a packet arrives at a switch and there is no room in the buffer, the packet is discarded. This scheme relies on higher-level software that handles errors in transmission to resend lost packets. This leads to significant bandwidth wastage due to (re)transmitted packets that are later discarded and, therefore, is typically used only in lossy networks like the Internet.

The second scheme relies on *flow control*, also discussed previously. When buffers become full, link-level flow control provides feedback that prevents the transmission of additional packets. This *backpressure* feedback rapidly propagates backward until it reaches the sender(s) of the packets producing congestion, forcing a reduction in the injection rate of packets into the network. The main drawbacks of this scheme are that sources become aware of congestion too late when the network is already congested, and nothing is done to alleviate congestion. Backpressure flow control is common in lossless networks like SANs used in supercomputers and enterprise systems.

A more elaborate way of using flow control is by implementing it directly between the sender and the receiver end nodes, generically called *end-to-end flow control*. *Windowing* is one version of end-to-end credit-based flow control where the window size should be large enough to efficiently pipeline packets through the network. The goal of the window is to limit the number of unacknowledged packets, thus bounding the contribution of each source to congestion, should it arise. The TCP protocol uses a sliding window. Note that end-to-end flow control describes the interaction between just two nodes of the interconnection network, not the entire interconnection network between all end nodes. Hence, flow control helps congestion control, but it is not a global solution.

Choke packets are used in the third scheme, which is built upon the premise that traffic injection should be throttled only when congestion exists across the network. The idea is for each switch to see how busy it is and to enter into a warning state when it passes a threshold. Each packet received by a switch in the warning state is sent back to the source via a choke packet that includes the intended destination. The source is expected to reduce traffic to that destination by a fixed percentage. Since it likely will have already sent other packets along that path, the source node waits for all the packets in transit to be returned before acting on

the choke packets. In this scheme, congestion is controlled by reducing the packet injection rate until traffic reduces, just as metering lights that guard on-ramps control the rate of cars entering a freeway. This scheme works efficiently when the feedback delay is short. When congestion notification takes a long time, usually due to long time of flight, this congestion control scheme may become unstable—reacting too slowly or producing oscillations in packet injection rate, both of which lead to poor network bandwidth utilization.

An alternative to congestion control consists of eliminating the negative consequences of congestion. This can be done by eliminating HOL blocking at every switch in the network as discussed previously. Virtual output queues can be used for this purpose; however, it would be necessary to implement as many queues at every switch input port as devices attached to the network. This solution is very expensive, and not scalable at all. Fortunately, it is possible to achieve good results by dynamically assigning a few set-aside queues to store only the congested packets that travel through some hot-spot regions of the network, very much like caches are intended to store only the more frequently accessed memory locations. This strategy is referred to as *regional explicit congestion notification* (RECN).

Fault Tolerance

The probability of system failures increases as transistor integration density and the number of devices in the system increases. Consequently, system reliability and availability have become major concerns and will be even more important in future systems with the proliferation of interconnected devices. A practical issue arises, therefore, as to whether or not the interconnection network relies on all the devices being operational in order for the network to work properly. Since software failures are generally much more frequent than hardware failures, another question surfaces as to whether a software crash on a single device can prevent the rest of the devices from communicating. Although some hardware designers try to build fault-free networks, in practice, it is only a question of the rate of failures, not whether they can be prevented. Thus, the communication subsystem must have mechanisms for dealing with faults when—not if—they occur.

There are two main kinds of failure in an interconnection network: *transient* and *permanent*. Transient failures are usually produced by electromagnetic interference and can be detected and corrected using the techniques described in Section F.2. Oftentimes, these can be dealt with simply by retransmitting the packet either at the link level or end-to-end. Permanent failures occur when some component stops working within specifications. Typically, these are produced by overheating, overbiasing, overuse, aging, and so on and cannot be recovered from simply by retransmitting packets with the help of some higher-layer software protocol. Either an alternative physical path must exist in the network and be supplied by the routing algorithm to circumvent the fault or the network will be crippled, unable to deliver packets whose only paths are through faulty resources.

Three major categories of techniques are used to deal with permanent failures: *resource sparing, fault-tolerant routing*, and *network reconfiguration*. In the first

technique, faulty resources are switched off or bypassed, and some spare resources are switched in to replace the faulty ones. As an example, the ServerNet interconnection network is designed with two identical switch fabrics, only one of which is usable at any given time. In case of failure in one fabric, the other is used. This technique can also be implemented without switching in spare resources, leading to a degraded mode of operation after a failure. The IBM Blue Gene/L supercomputer, for instance, has the facility to bypass failed network resources while retaining its base topological structure and routing algorithm. The main drawback of this technique is the relatively large number of healthy resources (e.g., midplane node boards) that may need to be switched off after a failure in order to retain the base topological structure (e.g., a 3D torus).

Fault-tolerant routing, on the other hand, takes advantage of the multiple paths already existing in the network topology to route messages in the presence of failures without requiring spare resources. Alternative paths for each supported fault combination are identified at design time and incorporated into the routing algorithm. When a fault is detected, a suitable alternative path is used. The main difficulty when using this technique is guaranteeing that the routing algorithm will remain deadlock-free when using the alternative paths, given that arbitrary fault patterns may occur. This is especially difficult in direct networks whose regularity can be compromised by the fault pattern. The Cray T3E is an example system that successfully applies this technique on its 3D torus direct network. There are many examples of this technique in systems using indirect networks, such as with the bidirectional multistage networks in the ASCI White and ASC Purple. Those networks provide multiple minimal paths between end nodes and, inherently, have no routing deadlock problems (see Section F.5). In these networks, alternative paths are selected at the source node in case of failure.

Network reconfiguration is yet another, more general technique to handle voluntary and involuntary changes in the network topology due either to failures or to some other cause. In order for the network to be reconfigured, the nonfaulty portions of the topology must first be discovered, followed by computation of the new routing tables and distribution of the routing tables to the corresponding network locations (i.e., switches and/or end node devices). Network reconfiguration requires the use of programmable switches and/or network interfaces, depending on how routing is performed. It may also make use of generic routing algorithms (e.g., up*/down* routing) that can be configured for all the possible network topologies that may result after faults. This strategy relieves the designer from having to supply alternative paths for each possible fault combination at design time. Programmable network components provide a high degree of flexibility but at the expense of higher cost and latency. Most standard and proprietary interconnection networks for clusters and SANs—including Myrinet, Quadrics, InfiniBand, Advanced Switching, and Fibre Channel—incorporate software for (re)configuring the network routing in accordance with the prevailing topology.

Another practical issue ties to node failure tolerance. If an interconnection network can survive a failure, can it also continue operation while a new node is added to or removed from the network, usually referred to as *hot swapping*? If not, each addition or removal of a new node disables the interconnection network, which is

impractical for WANs and LANs and is usually intolerable for most SANs. Online system expansion requires hot swapping, so most networks allow for it. Hot swapping is usually supported by implementing *dynamic network reconfiguration*, in which the network is reconfigured without having to stop user traffic. The main difficulty with this is guaranteeing deadlock-free routing while routing tables for switches and/or end node devices are dynamically and asynchronously updated as more than one routing algorithm may be alive (and, perhaps, clashing) in the network at the same time. Most WANs solve this problem by dropping packets whenever required, but dynamic network reconfiguration is much more complex in lossless networks. Several theories and practical techniques have recently been developed to address this problem efficiently.

Example Figure F.27 shows the number of failures of 58 desktop computers on a local area network for a period of just over one year. Suppose that one local area network is based on a network that requires all machines to be operational for the interconnection network to send data; if a node crashes, it cannot accept messages, so the interconnection becomes choked with data waiting to be delivered. An alternative is the traditional local area network, which can operate in the presence of node failures; the interconnection simply discards messages for a node that decides not to accept them. Assuming that you need to have both your workstation and the connecting LAN to get your work done, how much greater are your chances of being prevented from getting your work done using the failure-intolerant LAN versus traditional LANs? Assume the downtime for a crash is less than 30 minutes. Calculate using the one-hour intervals from this figure.

Answer Assuming the numbers for Figure F.27, the percentage of hours that you can't get your work done using the failure-intolerant network is

$$\frac{\text{Intervals with failures}}{\text{Total intervals}} = \frac{\text{Total intervals} - \text{Intervals with no failures}}{\text{Total intervals}}$$

$$= \frac{8974 - 8605}{8974} = \frac{369}{8974} = 4.1\%$$

The percentage of hours that you can't get your work done using the traditional network is just the time your workstation has crashed. If these failures are equally distributed among workstations, the percentage is

$$\frac{\text{Failures/Machines}}{\text{Total intervals}} = \frac{654/58}{8974} = \frac{11.28}{8974} = 0.13\%$$

Hence, you are more than 30 times more likely to be prevented from getting your work done with the failure-intolerant LAN than with the traditional LAN, according to the failure statistics in Figure F.27. Stated alternatively, the person responsible for maintaining the LAN would receive a 30-fold increase in phone calls from irate users!

Failed machines per time interval	One-hour intervals with number of failed machines in first column	Total failures per one-hour interval	One-day intervals with number of failed machines in first column	Total failures per one-day interval
0	8605	0	184	0
1	264	264	105	105
2	50	100	35	70
3	25	75	11	33
4	10	40	6	24
5	7	35	9	45
6	3	18	6	36
7	1	7	4	28
8	1	8	4	32
9	2	18	2	18
10	2	20		
11	1	11	2	22
12			1	12
17	1	17		
20	1	20		
21	1	21	1	21
31			1	31
38			1	38
58			1	58
Total	**8974**	**654**	**373**	**573**

Figure F.27 Measurement of reboots of 58 DECstation 5000 s running Ultrix over a 373-day period. These reboots are distributed into time intervals of one hour and one day. The first column sorts the intervals according to the number of machines that failed in that interval. The next two columns concern one-hour intervals, and the last two columns concern one-day intervals. The second and fourth columns show the number of intervals for each number of failed machines. The third and fifth columns are just the product of the number of failed machines and the number of intervals. For example, there were 50 occurrences of one-hour intervals with 2 failed machines, for a total of 100 failed machines, and there were 35 days with 2 failed machines, for a total of 70 failures. As we would expect, the number of failures per interval changes with the size of the interval. For example, the day with 31 failures might include one hour with 11 failures and one hour with 20 failures. The last row shows the total number of each column; the number of failures doesn't agree because multiple reboots of the same machine in the same interval do not result in separate entries. (Randy Wang of the University of California–Berkeley collected these data.)

F.8 Examples of Interconnection Networks

To further provide mass to the concepts described in the previous sections, we look at five example networks from the four interconnection network domains considered in this appendix. In addition to one for each of the OCN, LAN, and WAN areas, we look at two examples from the SAN area: one for system area networks

and one for system/storage area networks. The first two examples are proprietary networks used in high-performance systems; the latter three examples are network standards widely used in commercial systems.

On-Chip Network: Intel Single-Chip Cloud Computer

With continued increases in transistor integration as predicted by Moore's law, processor designers are under the gun to find ways of combating chip-crossing wire delay and other problems associated with deep submicron technology scaling. Multicore microarchitectures have gained popularity, given their advantages of simplicity, modularity, and ability to exploit parallelism beyond that which can be achieved through aggressive pipelining and multiple instruction/data issuing on a single core. No matter whether the processor consists of a single core or multiple cores, higher and higher demands are being placed on intrachip communication bandwidth to keep pace—not to mention interchip bandwidth. This has spurred a great amount of interest in OCN designs that efficiently support communication of instructions, register operands, memory, and I/O data within and between processor cores both on and off the chip. Here we focus on one such on-chip network: The Intel Single-chip Cloud Computer prototype.

The Single-chip Cloud Computer (SCC) is a prototype chip multiprocessor with 48 Intel IA-32 architecture cores. Cores are laid out (see Figure F.28) on a network with a 2D mesh topology (6×4). The network connects 24 tiles, 4 on-die memory controllers, a voltage regulator controller (VRC), and an external system interface controller (SIF). In each tile two cores are connected to a router. The four memory controllers are connected at the boundaries of the mesh, two on each side, while the VRC and SIF controllers are connected at the bottom border of the mesh.

Each memory controller can address two DDR3 DIMMS, each up to 8 GB of memory, thus resulting in a maximum of 64 GB of memory. The VRC controller allows any core or the system interface to adjust the voltage in any of the six predefined regions configuring the network (two 2-tile regions). The clock can also be adjusted at a finer granularity with each tile having its own operating frequency. These regions can be turned off or scaled down for large power savings. This method allows full application control of the power state of the cores. Indeed, applications have an API available to define the voltage and the frequency of each region. The SIF controller is used to communicate the network from outside the chip.

Each of the tiles includes two processor cores (P54C-based IA) with associated L1 16 KB data cache and 16 KB instruction cache and a 256 KB L2 cache (with the associated controller), a 5-port router, traffic generator (for testing purposes only), a mesh interface unit (MIU) handling all message passing requests, memory look-up tables (with configuration registers to set the mapping of a core's physical addresses to the extended memory map of the system), a message-passing buffer, and circuitry for the clock generation and synchronization for crossing asynchronous boundaries.

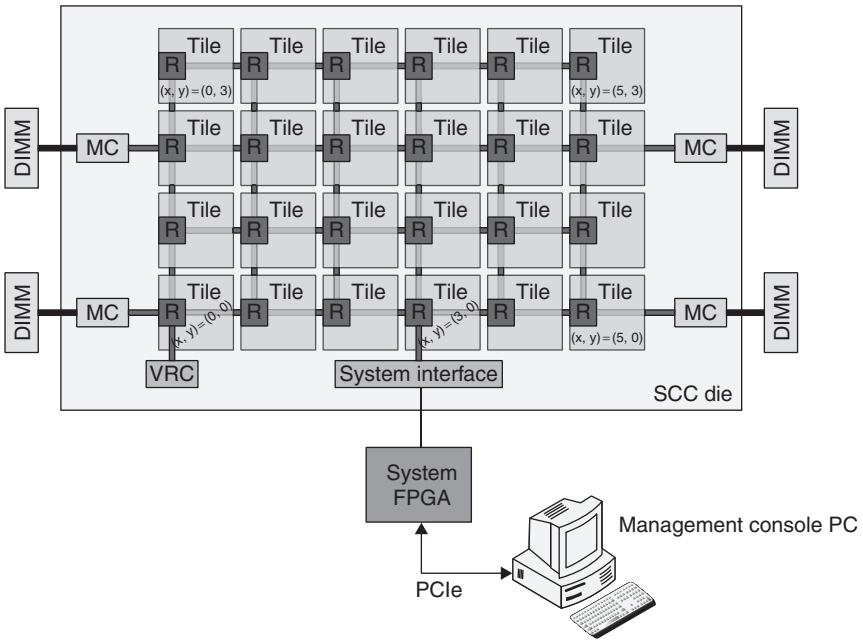


Figure F.28 SCC Top-level architecture. From Howard, J. et al., *IEEE International Solid-State Circuits Conference Digest of Technical Papers*, pp. 58–59.

Focusing on the OCN, the MIU unit is in charge of interfacing the cores to the network, including the packetization and de-packetization of large messages; command translation and address decoding/lookup; link-level flow control and credit management; and arbiter decisions following a round-robin scheme. A credit-based flow control mechanism is used together with virtual cut-through switching (thus making it necessary to split long messages into packets). The routers are connected in a 2D mesh layout, each on its own power supply and clock source. Links connecting routers have 16B + 2B side bands running at 2 GHz. Zero-load latency is set to 4 cycles, including link traversal. Eight virtual channels are used for performance (6 VCs) and protocol-level deadlock handling (2 VCs). A message-level arbitration is implemented by a wrapped wave-front arbiter. The dimension-order XY routing algorithm is used and pre-computation of the output port is performed at every router.

Besides the tiles having regions defined for voltage and frequency, the network (made of routers and links) has its own single region. Thus, all the network components run at the same speed and use the same power supply. An asynchronous clock transition is required between the router and the tile.

One of the distinctive features of the SCC architecture is the support for a messaging-based communication protocol rather than hardware cache-coherent

memory for inter-core communication. Message passing buffers are located on every router and APIs are provided to take full control of MPI structures. Cache coherency can be implemented by software.

The SCC router represents a significant improvement over the Teraflops processor chip in the implementation of a 2D on-chip interconnect. Contrasted with the 2D mesh implemented in the Teraflops processor, this implementation is tuned for a wider data path in a multiprocessor interconnect and is more latency, area, and power optimized for such a width. It targets a lower 2-GHz frequency of operation compared to the 5 GHz of its predecessor Teraflops processor, yet with a higher-performance interconnect architecture.

System Area Network: IBM Blue Gene/L 3D Torus Network

The IBM BlueGene/L was the largest-scaled, highest-performing computer system in the world in 2005, according to *www.top500.org*. With 65,536 dual-processor compute nodes and 1024 I/O nodes, this 360 TFLOPS (peak) supercomputer has a system footprint of approximately 2500 square feet. Both processors at each node can be used for computation and can handle their own communication protocol processing in virtual mode or, alternatively, one of the processors can be used for computation and the other for network interface processing. Packets range in size from 32 bytes to a maximum of 256 bytes, and 8 bytes are used for the header. The header includes routing, virtual channel, link-level flow control, packet size, and other such information, along with 1 byte for CRC to protect the header. Three bytes are used for CRC at the packet level, and 1 byte serves as a valid indicator.

The main interconnection network is a proprietary $32 \times 32 \times 64$ 3D torus SAN that interconnects all 64 K nodes. Each node switch has six 350 MB/sec bidirectional links to neighboring torus nodes, an injection bandwidth of 612.5 MB/sec from the two node processors, and a reception bandwidth of 1050 MB/sec to the two node processors. The reception bandwidth from the network equals the inbound bandwidth across all switch ports, which prevents reception links from bottlenecking network performance. Multiple packets can be sunk concurrently at each destination node because of the higher reception link bandwidth.

Two nodes are implemented on a $2 \times 1 \times 1$ compute card, 16 compute cards and 2 I/O cards are implemented on a $4 \times 4 \times 2$ node board, 16 node boards are implemented on an $8 \times 8 \times 8$ midplane, and 2 midplanes form a 1024-node rack with physical dimensions of $0.9 \times 0.9 \times 1.9$ cubic meters. Links have a maximum physical length of 8.6 meters, thus enabling efficient link-level flow control with reasonably low buffering requirements. Low latency is achieved by implementing virtual cut-through switching, distributing arbitration at switch input and output ports, and precomputing the current routing path at the previous switch using a finite-state machine so that part of the routing delay is removed from the critical path in switches. High effective bandwidth is achieved using input-buffered

switches with dual read ports, virtual cut-through switching with four virtual channels, and fully adaptive deadlock-free routing based on bubble flow control.

A key feature in networks of this size is fault tolerance. Failure rate is reduced by using a relatively low link clock frequency of 700 MHz (same as processor clock) on which both edges of the clock are used (i.e., 1.4 Gbps or 175 MB/sec transfer rate is supported for each bit-serial network link in each direction), but failures may still occur in the network. In case of failure, the midplane node boards containing the fault(s) are switched off and bypassed to isolate the fault, and computation resumes from the last checkpoint. Bypassing is done using separate bypass switch boards associated with each midplane that are additional to the set of torus node boards. Each bypass switch board can be configured to connect either to the corresponding links in the midplane node boards or to the next bypass board, effectively removing the corresponding set of midplane node boards. Although the number of processing nodes is reduced to some degree in some network dimensions, the machine retains its topological structure and routing algorithm.

Some collective communication operations such as barrier synchronization, broadcast/multicast, reduction, and so on are not performed well on the 3D torus as the network would be flooded with traffic. To remedy this, two separate tree networks with higher per-link bandwidth are used to implement collective and combining operations more efficiently. In addition to providing support for efficient synchronization and broadcast/multicast, hardware is used to perform some arithmetic reduction operations in an efficient way (e.g., to compute the sum or the maximum value of a set of values, one from each processing node). In addition to the 3D torus and the two tree networks, the Blue Gene/L implements an I/O Gigabit Ethernet network and a control system Fast Ethernet network of lower bandwidth to provide for parallel I/O, configuration, debugging, and maintenance.

System/Storage Area Network: InfiniBand

InfiniBand is an industrywide *de facto* networking standard developed in October 2000 by a consortium of companies belonging to the InfiniBand Trade Association. InfiniBand can be used as a system area network for interprocessor communication or as a storage area network for server I/O. It is a switch-based interconnect technology that provides flexibility in the topology, routing algorithm, and arbitration technique implemented by vendors and users. InfiniBand supports data transmission rates of 2 to 120 Gbp/link per direction across distances of 300 meters. It uses cut-through switching, 16 virtual channels and service levels, credit-based link-level flow control, and weighted round-robin fair scheduling and implements programmable forwarding tables. It also includes features useful for increasing reliability and system availability, such as communication subnet management, end-to-end path establishment, and virtual destination naming.

Institution and processor [network] name	Year built	Number of network ports [cores or tiles + other ports]	Basic network topology	# of data bits per link per direction	Link bandwidth [link clock speed]	Routing; arbitration; switching	# of chip metal layers; flow control; #virtual channels
MIT Raw [General Dynamic Network]	2002	16 ports [16 tiles]	2D mesh (4 × 4)	32 bits	0.9 GB/sec [225 MHz, clocked at proc speed]	XY DOR with request-reply deadlock recovery; RR arbitration; wormhole	6 layers; credit-based no virtual channels
IBM Power5	2004	7 ports [2 PE cores + 5 other ports]	Crossbar	256 bits Inst fetch; 64 bits for stores; 256 bits LDs	[1.9 GHz, clocked at proc speed]	Shortest-path; nonblocking; circuit switch	7 layers; handshaking; no virtual channels
U.T. Austin TRIP Edge [Operand Network]	2005	25 ports [25 execution unit tiles]	2D mesh (5 × 5)	110 bits	5.86 GB/sec [533 MHz clock scaled by 80%]	YX DOR; distributed RR arbitration; wormhole	7 layers; on/off flow control; no virtual channels
U.T. Austin TRIP Edge [On-Chip Network]	2005	40 ports [16 L2 tiles + 24 network interface tile]	2D mesh (10 × 4)	128 bits	6.8 GB/sec [533 MHz clock scaled by 80%]	YX DOR; distributed RR arbitration; VCT switched	7 layers; credit-based flow control; 4 virtual channels
Sony, IBM, Toshiba Cell BE [Element Interconnect Bus]	2005	12 ports [1 PPE and 8 SPEs + 3 other ports for memory, I/O interface]	Ring (4 total, 2 in each direction)	128 bits data (+16 bits tag)	25.6 GB/sec [1.6 GHz, clocked at half the proc speed]	Shortest-path; tree-based RR arbitration (centralized); pipelined circuit switch	8 layers; credit-based flow control; no virtual channels
Sun UltraSPARC T1 processor	2005	Up to 13 ports [8 PE cores + 4 L2 banks + 1 shared I/O]	Crossbar	128 bits both for the 8 cores and the 4 L2 banks	19.2 GB/sec [1.2 GHz, clocked at proc speed]	Shortest-path; age-based arbitration; VCT switched	9 layers; handshaking; no virtual channels

Figure F.29 Characteristics of on-chip networks implemented in recent research and commercial processors. Some processors implement multiple on-chip networks (not all shown)—for example, two in the MIT Raw and eight in the TRIP Edge.

Figure F.30 shows the packet format for InfiniBand juxtaposed with two other network standards from the LAN and WAN areas. Figure F.31 compares various characteristics of the InfiniBand standard with two proprietary system area networks widely used in research and commercial high-performance computer systems.

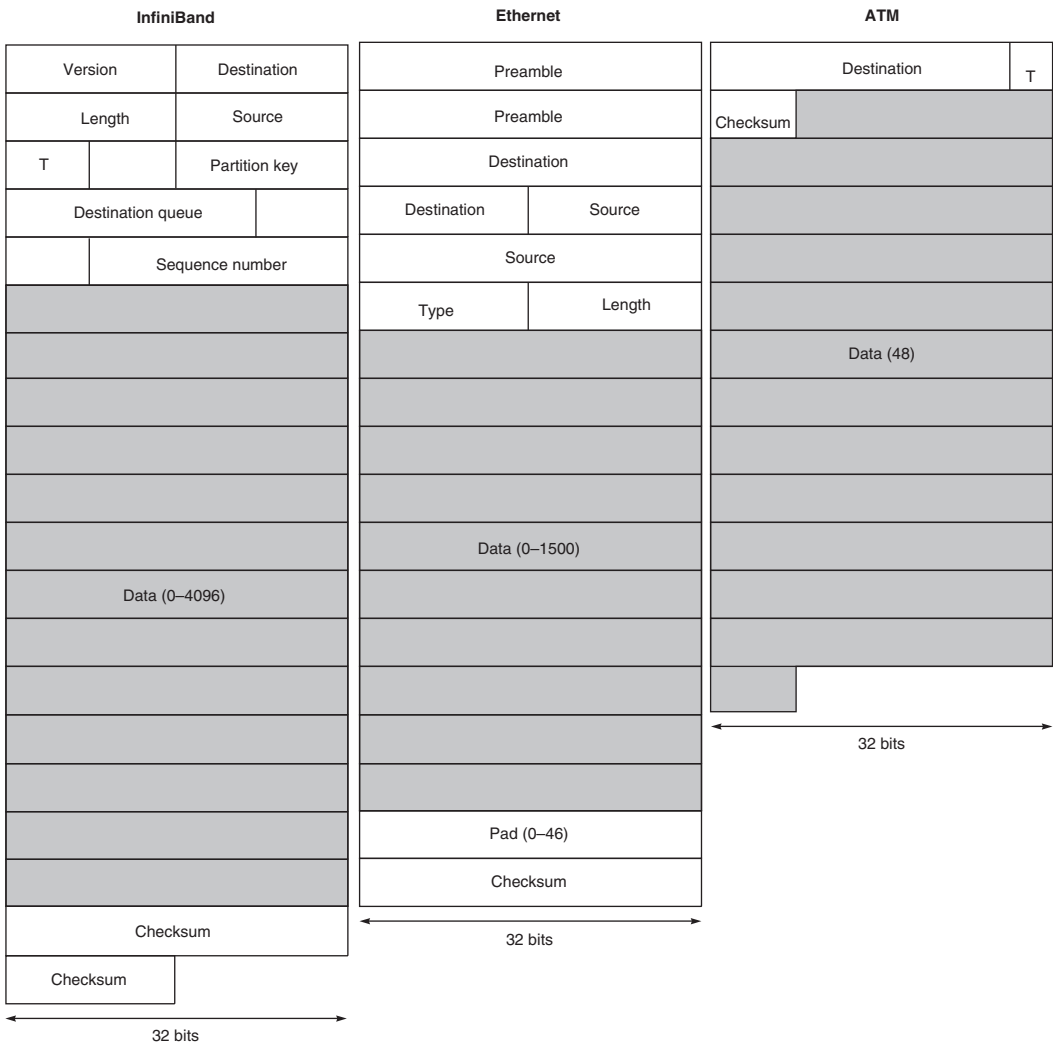


Figure F.30 Packet format for InfiniBand, Ethernet, and ATM. ATM calls their messages "cells" instead of packets, so the proper name is ATM cell format. The width of each drawing is 32 bits. All three formats have destination addressing fields, encoded differently for each situation. All three also have a checksum field to catch transmission errors, although the ATM checksum field is calculated only over the header; ATM relies on higher-level protocols to catch errors in the data. Both InfiniBand and Ethernet have a length field, since the packets hold a variable amount of data, with the former counted in 32-bit words and the latter in bytes. InfiniBand and ATM headers have a type field (T) that gives the type of packet. The remaining Ethernet fields are a preamble to allow the receiver to recover the clock from the self-clocking code used on the Ethernet, the source address, and a pad field to make sure the smallest packet is 64 bytes (including the header). InfiniBand includes a version field for protocol version, a sequence number to allow in-order delivery, a field to select the destination queue, and a partition key field. Infiniband has many more small fields not shown and many other packet formats; above is a simplified view. ATM's short, fixed packet is a good match to real-time demand of digital voice.

Network name [vendors]	Used in top 10 supercomputer clusters (2005)	Number of nodes	Basic network topology	Raw link bidirectional BW	Routing algorithm	Arbitration technique	Switching technique; flow control
InfiniBand [Mellanox, Voltair]	SGI Altrix and Dell Poweredge Thunderbird	>Millions (2^{128} GUID addresses, like IPv6)	Completely configurable (arbitrary)	4–240 Gbps	Arbitrary (table-driven), typically up*/down*	Weighted RR fair scheduling (2-level priority)	Cut-through, 16 virtual channels (15 for data); credit-based
Myrinet-2000 [Myricom]	Barcelona Supercomputer Center in Spain	8192 nodes	Bidirectional MIN with 16-port bidirectional switches (Clos net.)	4 Gbps	Source-based dispersive (adaptive) minimal routing	Round-robin arbitration	Cut-through switching with no virtual channels; Xon/Xoff flow control
$QsNet^{II}$ [Quadrics]	Intel Thunder Itanium2 Tiger4	>Tens of thousands	Fat tree with 8-port bidirectional switches	21.3 Gbps	Source-based LCA adaptive shortest-path routing	2-phased RR, priority, aging, distributed at output ports	Wormhole with 2 virtual channels; credit-based

Figure F.31 Characteristics of system area networks implemented in various top 10 supercomputer clusters in 2005.

InfiniBand offers two basic mechanisms to support user-level communication: send/receive and remote DMA (RDMA). With send/receive, the receiver has to explicitly post a receive buffer (i.e., allocate space in its channel adapter network interface) before the sender can transmit data. With RDMA, the sender can remotely DMA data directly into the receiver device's memory. For example, for a nominal packet size of 4 bytes measured on a Mellanox MHEA28-XT channel adapter connected to a 3.4 GHz Intel Xeon host device, sending and receiving overhead is 0.946 and 1.423 μs, respectively, for the send/receive mechanism, whereas it is 0.910 and 0.323 μs, respectively, for the RDMA mechanism.

As discussed in Section F.2, the packet size is important in getting full benefit of the network bandwidth. One might ask, "What is the natural size of messages?" Figure F.32(a) shows the size of messages for a commercial fluid dynamics simulation application, called Fluent, collected on an InfiniBand network at The Ohio State University's Network-Based Computer Laboratory. One plot is cumulative in messages sent and the other is cumulative in data bytes sent. Messages in this graph are message passing interface (MPI) units of information, which gets divided into InfiniBand maximum transfer units (packets) transferred over the network. As shown, the maximum message size is over 512 KB, but approximately 90% of the messages are less than 512 bytes. Messages of 2 KB represent approximately 50% of the bytes transferred. An Integer Sort application kernel in the NAS Parallel

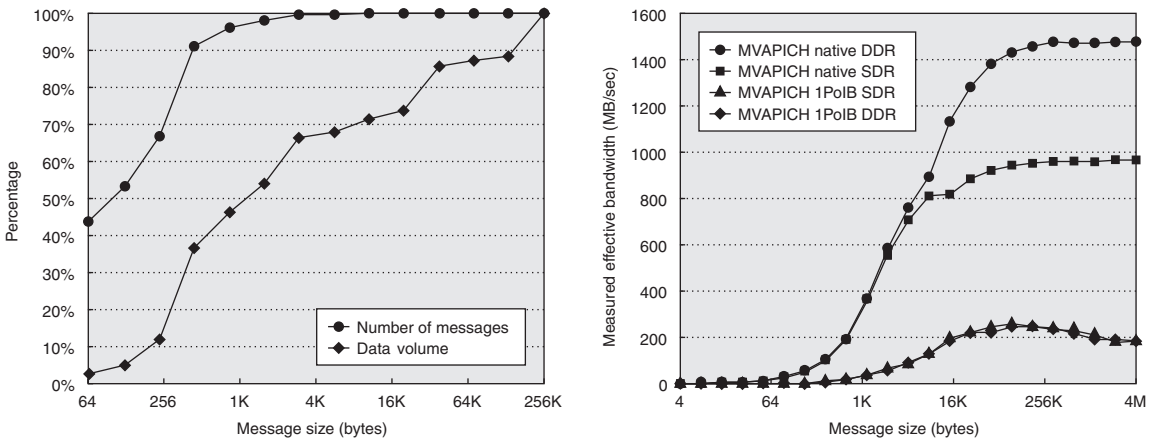


Figure F.32 Data collected by D.K. Panda, S. Sur, and L. Chai (2005) in the Network-Based Computing Laboratory at The Ohio State University. (a) Cumulative percentage of messages and volume of data transferred as message size varies for the Fluent application (*www.fluent.com*). Each *x*-axis entry includes all bytes up to the next one; for example, 128 represents 1 byte to 128 bytes. About 90% of the messages are less than 512 bytes, which represents about 40% of the total bytes transferred. (b) Effective bandwidth versus message size measured on SDR and DDR InfiniBand networks running MVAPICH (*http://nowlab.cse.ohio-state.edu/projects/mpi-iba*) with OS bypass (native) and without (IPoIB).

Benchmark suite is also measured to have about 75% of its messages below 512 bytes (plots not shown). Many applications send far more small messages than large ones, particularly since requests and acknowledgments are more frequent than data responses and block writes.

InfiniBand reduces protocol processing overhead by allowing it to be offloaded from the host computer to a controller on the InfiniBand network interface card. The benefits of protocol offloading and bypassing the operating system are shown in Figure F.32(b) for MVAPICH, a widely used implementation of MPI over InfiniBand. Effective bandwidth is plotted against message size for MVAPICH configured in two modes and two network speeds. One mode runs IPoIB, in which InfiniBand communication is handled by the IP layer implemented by the host's operating system (i.e., no OS bypass). The other mode runs MVAPICH directly over VAPI, which is the native Mellanox InfiniBand interface that offloads transport protocol processing to the channel adapter hardware (i.e., OS bypass). Results are shown for 10 Gbps single data rate (SDR) and 20 Gbps double data rate (DDR) InfiniBand networks. The results clearly show that offloading the protocol processing and bypassing the OS significantly reduce sending and receiving overhead to allow near wire-speed effective bandwidth to be achieved.

Ethernet: *The* Local Area Network

Ethernet has been extraordinarily successful as a LAN—from the 10 Mbit/sec standard proposed in 1978 used practically everywhere today to the more recent 10 Gbit/sec standard that will likely be widely used. Many classes of computers include Ethernet as a standard communication interface. Ethernet, codified as IEEE standard 802.3, is a packet-switched network that routes packets using the destination address. It was originally designed for coaxial cable but today uses primarily Cat5E copper wire, with optical fiber reserved for longer distances and higher bandwidths. There is even a wireless version (802.11), which is testimony to its ubiquity.

Over a 20-year span, computers became thousands of times faster than they were in 1978, but the shared media Ethernet network remained the same. Hence, engineers had to invent temporary solutions until a faster, higher-bandwidth network became available. One solution was to use multiple Ethernets to interconnect machines and to connect those Ethernets with internetworking devices that could transfer traffic from one Ethernet to another, as needed. Such devices allow individual Ethernets to operate in parallel, thereby increasing the aggregate interconnection bandwidth of a collection of computers. In effect, these devices provide similar functionality to the switches described previously for point-to-point networks.

Figure F.33 shows the potential parallelism that can be gained. Depending on how they pass traffic and what kinds of interconnections they can join together, these devices have different names:

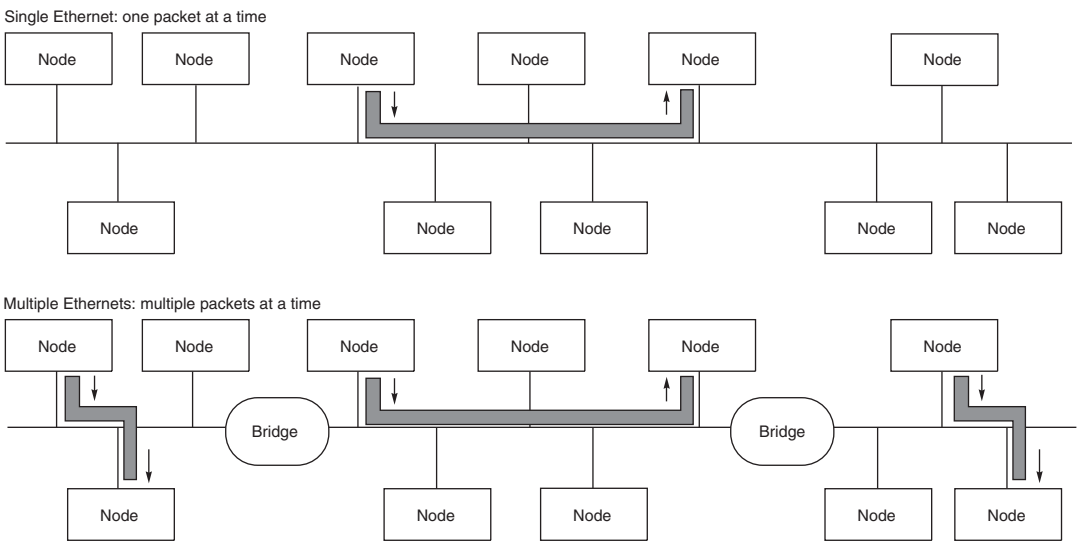


Figure F.33 The potential increased bandwidth of using many Ethernets and bridges.

- *Bridges*—These devices connect LANs together, passing traffic from one side to another depending on the addresses in the packet. Bridges operate at the Ethernet protocol level and are usually simpler and cheaper than routers, discussed next. Using the notation of the OSI model described in the next section (see Figure F.36 on page F-85), bridges operate at layer 2, the data link layer.
- Routers or *gateways*—These devices connect LANs to WANs, or WANs to WANs, and resolve incompatible addressing. Generally slower than bridges, they operate at OSI layer 3, the network layer. WAN routers divide the network into separate smaller subnets, which simplifies manageability and improves security.

The final internetworking devices are *hubs*, but they merely extend multiple segments into a single LAN. Thus, hubs do not help with performance, as only one message can transmit at a time. Hubs operate at OSI layer 1, called the physical

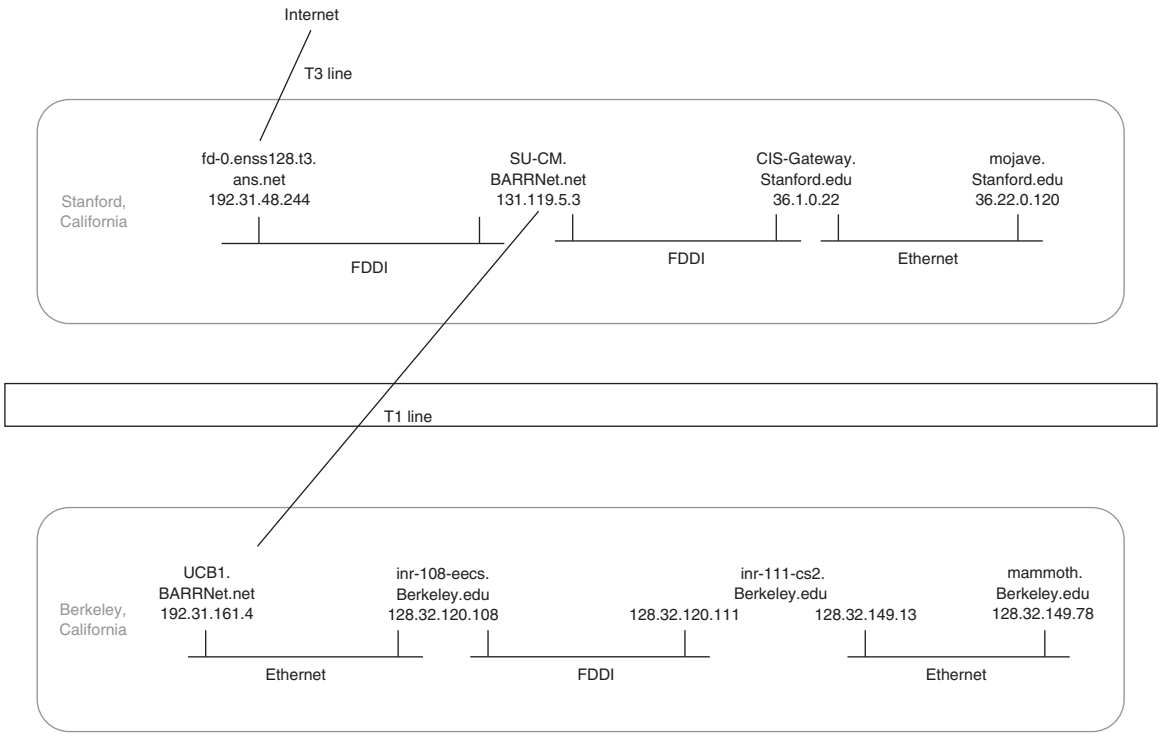


Figure F.34 The connection established between mojave.stanford.edu and mammoth.berkeley.edu (1995). FDDI is a 100 Mbit/sec LAN, while a T1 line is a 1.5 Mbit/sec telecommunications line and a T3 is a 45 Mbit/sec telecommunications line. BARRNet stands for Bay Area Research Network. Note that inr-111-cs2.Berkeley.edu is a router with two Internet addresses, one for each port.

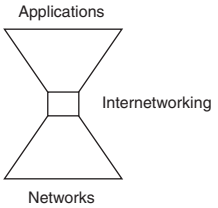


Figure F.35 The role of internetworking. The width indicates the relative number of items at each level.

Layer number	Layer name	Main function	Example protocol	Network component
7	Application	Used for applications specifically written to run over the network	FTP, DNS, NFS, http	Gateway, smart switch
6	Presentation	Translates from application to network format, and *vice versa*		Gateway
5	Session	Establishes, maintains, and ends sessions across the network	Named pipes, RPC	Gateway
4	Transport	Additional connection below the session layer	TCP	Gateway
3	Network	Translates logical network address and names to their physical address (e.g., computer name to MAC address)	IP	Router, ATM switch
2	Data Link	Turns packets into raw bits and at the receiving end turns bits into packets	Ethernet	Bridge, network interface card
1	Physical	Transmits raw bit stream over physical cable	IEEE 802	Hub

Figure F.36 The OSI model layers. Based on *www.geocities.com/SiliconValley/Monitor/3131/ne/osimodel.html*.

layer. Since these devices were not planned as part of the Ethernet standard, their ad hoc nature has added to the difficulty and cost of maintaining LANs.

As of 2011, Ethernet link speeds are available at 10, 100, 10,000, and 100,000 Mbits/sec. Although 10 and 100 Mbits/sec Ethernets share the media with multiple devices, 1000 Mbits/sec and above Ethernets rely on point-to-point links and switches. Ethernet switches normally use some form of store-and-forward.

Ethernet has no real flow control, dating back to its first instantiation. It originally used carrier sensing with exponential back-off (see page F-23) to arbitrate for the shared media. Some switches try to use that interface to retrofit their version of flow control, but flow control is not part of the Ethernet standard.

Wide Area Network: ATM

Asynchronous Transfer Mode (ATM) is a wide area networking standard set by the telecommunications industry. Although it flirted as competition to Ethernet as a LAN in the 1990s, ATM has since retreated to its WAN stronghold.

The telecommunications standard has scalable bandwidth built in. It starts at 155 Mbits/sec and scales by factors of 4 to 620 Mbits/sec, 2480 Mbits/sec, and so on. Since it is a WAN, ATM's medium is fiber, both single mode and multimode. Although it is a switched medium, unlike the other examples it relies on virtual connections for communication. ATM uses virtual channels for routing to multiplex different connections on a single network segment, thereby avoiding the inefficiencies of conventional connection-based networking. The WAN focus also led to store-and-forward switching. Unlike the other protocols, Figure F.30 shows ATM has a small, fixed-sized packet with 48 bytes of payload. It uses a credit-based flow control scheme as opposed to IP routers that do not implement flow control.

The reason for virtual connections and small packets is quality of service. Since the telecommunications industry is concerned about voice traffic, predictability matters as well as bandwidth. Establishing a virtual connection has less variability than connectionless networking, and it simplifies store-and-forward switching. The small, fixed packet also makes it simpler to have fast routers and switches. Toward that goal, ATM even offers its own protocol stack to compete with TCP/IP. Surprisingly, even though the switches are simple, the ATM suite of protocols is large and complex. The dream was a seamless infrastructure from LAN to WAN, avoiding the hodgepodge of routers common today. That dream has faded from inspiration to nostalgia.

F.9 Internetworking

Undoubtedly one of the most important innovations in the communications community has been internetworking. It allows computers on independent and incompatible networks to communicate reliably and efficiently. Figure F.34 illustrates the need to traverse between networks. It shows the networks and machines involved in transferring a file from Stanford University to the University of California at Berkeley, a distance of about 75 km.

The low cost of internetworking is remarkable. For example, it is vastly less expensive to send electronic mail than to make a coast-to-coast telephone call and leave a message on an answering machine. This dramatic cost improvement is achieved using the same long-haul communication lines as the telephone call, which makes the improvement even more impressive.

The enabling technologies for internetworking are software standards that allow reliable communication without demanding reliable networks. The underlying principle of these successful standards is that they were composed as a hierarchy of layers, each layer taking responsibility for a portion of the overall communication task. Each computer, network, and switch implements its layer of the standards, relying on the other components to faithfully fulfill their responsibilities. These layered software standards are called protocol families or protocol suites. They enable applications to work with any interconnection without extra work by the application programmer. Figure F.35 suggests the hierarchical model of communication.

The most popular internetworking standard is TCP/IP (Transmission Control Protocol/Internet Protocol). This protocol family is the basis of the humbly named Internet, which connects hundreds of millions of computers around the world. This popularity means TCP/IP is used even when communicating locally across compatible networks; for example, the network file system (NFS) uses IP even though it is very likely to be communicating across a homogenous LAN such as Ethernet. We use TCP/IP as our protocol family example; other protocol families follow similar lines. Section F.13 gives the history of TCP/IP.

The goal of a family of protocols is to simplify the standard by dividing responsibilities hierarchically among layers, with each layer offering services needed by the layer above. The application program is at the top, and at the bottom is the physical communication medium, which sends the bits. Just as abstract data types simplify the programmer's task by shielding the programmer from details of the implementation of the data type, this layered strategy makes the standard easier to understand.

There were many efforts at network protocols, which led to confusion in terms. Hence, Open Systems Interconnect (OSI) developed a model that popularized describing networks as a series of layers. Figure F.36 shows the model. Although all protocols do not exactly follow this layering, the nomenclature for the different layers is widely used. Thus, you can hear discussions about a simple layer 3 switch versus a layer 7 smart switch.

The key to protocol families is that communication occurs logically at the same level of the protocol in both sender and receiver, but services of the lower level implement it. This style of communication is called *peer-to-peer*. As an analogy, imagine that General A needs to send a message to General B on the battlefield. General A writes the message, puts it in an envelope addressed to General B, and gives it to a colonel with orders to deliver it. This colonel puts it in an envelope, and writes the name of the corresponding colonel who reports to General B, and gives it to a major with instructions for delivery. The major does the same thing and gives it to a captain, who gives it to a lieutenant, who gives it to a sergeant. The sergeant takes the envelope from the lieutenant, puts it into an envelope with the name of a sergeant who is in General B's division, and finds a private with orders to take the large envelope. The private borrows a motorcycle and delivers the envelope to the other sergeant. Once it arrives, it is passed up the chain of command, with each person removing an outer envelope with his name on it and passing on the inner envelope to his superior. As far as General B can tell, the note is from another general. Neither general knows who was involved in transmitting the envelope, nor how it was transported from one division to the other.

Protocol families follow this analogy more closely than you might think, as Figure F.37 shows. The original message includes a header and possibly a trailer sent by the lower-level protocol. The next-lower protocol in turn adds its own header to the message, possibly breaking it up into smaller messages if it is too large for this layer. Reusing our analogy, a long message from the general is divided and placed in several envelopes if it could not fit in one. This division of the message and appending of headers and trailers continues until the message

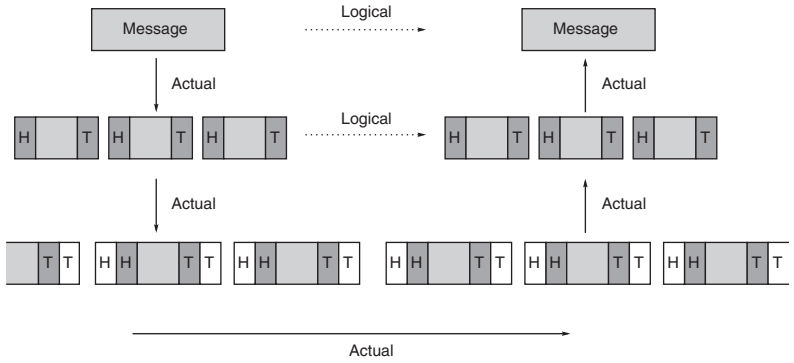


Figure F.37 A generic protocol stack with two layers. Note that communication is peer-to-peer, with headers and trailers for the peer added at each sending layer and removed by each receiving layer. Each layer offers services to the one above to shield it from unnecessary details.

descends to the physical transmission medium. The message is then sent to the destination. Each level of the protocol family on the receiving end will check the message at its level and peel off its headers and trailers, passing it on to the next higher level and putting the pieces back together. This nesting of protocol layers for a specific message is called a *protocol stack*, reflecting the last in, first out nature of the addition and removal of headers and trailers.

As in our analogy, the danger in this layered approach is the considerable latency added to message delivery. Clearly, one way to reduce latency is to reduce the number of layers, but keep in mind that protocol families define a standard but do not force how to implement the standard. Just as there are many ways to implement an instruction set architecture, there are many ways to implement a protocol family.

Our protocol stack example is TCP/IP. Let's assume that the bottom protocol layer is Ethernet. The next level up is the Internet Protocol or IP layer; the official term for an IP packet is a datagram. The IP layer routes the datagram to the destination machine, which may involve many intermediate machines or switches. IP makes a best effort to deliver the packets but does not guarantee delivery, content, or order of datagrams. The TCP layer above IP makes the guarantee of reliable, in-order delivery and prevents corruption of datagrams.

Following the example in Figure F.37, assume an application program wants to send a message to a machine via an Ethernet. It starts with TCP. The largest number of bytes that can be sent at once is 64 KB. Since the data may be much larger than 64 KB, TCP must divide them into smaller segments and reassemble them in proper order upon arrival. TCP adds a 20-byte header (Figure F.38) to every datagram and passes them down to IP. The IP layer above the physical layer adds a 20-byte header, also shown in Figure F.38. The data sent down from the IP level

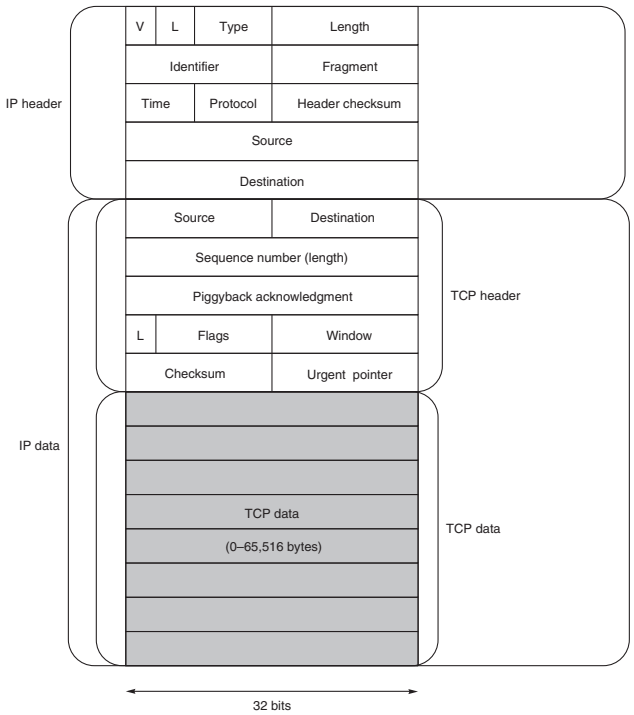


Figure F.38 The headers for IP and TCP. This drawing is 32 bits wide. The standard headers for both are 20 bytes, but both allow the headers to optionally lengthen for rarely transmitted information. Both headers have a length of header field (L) to accommodate the optional fields, as well as source and destination fields. The length field of the whole datagram is in a separate length field in IP, while TCP combines the length of the datagram with the sequence number of the datagram by giving the sequence number in bytes. TCP uses the checksum field to be sure that the datagram is not corrupted, and the sequence number field to be sure the datagrams are assembled into the proper order when they arrive. IP provides checksum error detection only for the header, since TCP has protected the rest of the packet. One optimization is that TCP can send a sequence of datagrams before waiting for permission to send more. The number of datagrams that can be sent without waiting for approval is called the *window*, and the window field tells how many bytes may be sent beyond the byte being acknowledged by this datagram. TCP will adjust the size of the window depending on the success of the IP layer in sending datagrams; the more reliable and faster it is, the larger TCP makes the window. Since the window slides forward as the data arrive and are acknowledged, this technique is called a *sliding window protocol.* The piggyback acknowledgment field of TCP is another optimization. Since some applications send data back and forth over the same connection, it seems wasteful to send a datagram containing only an acknowledgment. This piggyback field allows a datagram carrying data to also carry the acknowledgment for a previous transmission, "piggybacking" on top of a data transmission. The urgent pointer field of TCP gives the address within the datagram of an important byte, such as a break character. This pointer allows the application software to skip over data so that the user doesn't have to wait for all prior data to be processed before seeing a character that tells the software to stop. The identifier field and fragment field of IP allow intermediary machines to break the original datagram into many smaller datagrams. A unique identifier is associated with the original datagram and placed in every fragment, with the fragment field saying which piece is which. The time-to-live field allows a datagram to be killed off after going through a maximum number of intermediate switches no matter where it is in the network. Knowing the maximum number of hops that it will take for a datagram to arrive—if it ever arrives—simplifies the protocol software. The protocol field identifies which possible upper layer protocol sent the IP datagram; in our case, it is TCP. The V (for version) and type fields allow different versions of the IP protocol software for the network. Explicit version numbering is included so that software can be upgraded gracefully machine by machine, without shutting down the entire network. Nowadays, version six of the Internet protocol (IPv6) was widely used.

to the Ethernet are sent in packets with the format shown in Figure F.30. Note that the TCP packet appears inside the data portion of the IP datagram, just as Figure F.37 suggests.

F.10 Crosscutting Issues for Interconnection Networks

This section describes five topics discussed in other chapters that are fundamentally impacted by interconnection networks, and *vice versa*.

Density-Optimized Processors versus SPEC-Optimized Processors

Given that people all over the world are accessing Web sites, it doesn't really matter where servers are located. Hence, many servers are kept at *collocation sites*, which charge by network bandwidth reserved and used and by space occupied and power consumed. Desktop microprocessors in the past have been designed to be as fast as possible at whatever heat could be dissipated, with little regard for the size of the package and surrounding chips. In fact, some desktop microprocessors from Intel and AMD as recently as 2006 burned as much as 130 watts! Floor space efficiency was also largely ignored. As a result of these priorities, power is a major cost for collocation sites, and processor density is limited by the power consumed and dissipated, including within the interconnect!

With the proliferation of portable computers (notebook sales exceeded desktop sales for the first time in 2005) and their reduced power consumption and cooling demands, the opportunity exists for using this technology to create considerably denser computation. For instance, the power consumption for the Intel Pentium M in 2006 was 25 watts, yet it delivered performance close to that of a desktop microprocessor for a wide set of applications. It is therefore conceivable that performance per watt or performance per cubic foot could replace performance per microprocessor as the important figure of merit. The key is that many applications already make use of large clusters, so it is possible that replacing 64 power-hungry processors with, say, 256 power-efficient processors could be cheaper yet be software compatible. This places greater importance on power- and performance-efficient interconnection network design.

The Google cluster is a prime example of this migration to many "cooler" processors versus fewer "hotter" processors. It uses racks of up to 80 Intel Pentium III 1 GHz processors instead of more power-hungry high-end processors. Other examples include blade servers consisting of 1-inch-wide by 7-inch-high rack unit blades designed based on mobile processors. The HP ProLiant BL10e G2 blade server supports up to 20 1-GHz ultra-low-voltage Intel Pentium M processors with a 400-MHz front-side bus, 1-MB L2 cache, and up to 1 GB memory. The Fujitsu Primergy BX300 blade server supports up to 20 1.4- or 1.6-GHz Intel Pentium M processors, each with 512 MB of memory expandable to 4 GB.

Smart Switches versus Smart Interface Cards

Figure F.39 shows a trade-off as to where intelligence can be located within a network. Generally, the question is whether to have either smarter network interfaces or smarter switches. Making one smarter generally makes the other simpler and less expensive. By having an inexpensive interface, it was possible for Ethernet to become standard as part of most desktop and server computers. Lower-cost switches were made available for people with small configurations, not needing sophisticated forwarding tables and spanning-tree protocols of larger Ethernet switches.

Myrinet followed the opposite approach. Its switches are dumb components that, other than implementing flow control and arbitration, simply extract the first byte from the packet header and use it to directly select the output port. No routing tables are implemented, so the intelligence is in the network interface cards (NICs). The NICs are responsible for providing support for efficient communication and for implementing a distributed protocol for network (re)configuration. InfiniBand takes a hybrid approach by offering lower-cost, less sophisticated interface cards called target channel adapters (or TCAs) for less demanding devices such as disks—in the hope that it can be included within some I/O devices—and by offering more expensive, powerful interface cards for hosts called host channel adapters (or HCAs). The switches implement routing tables.

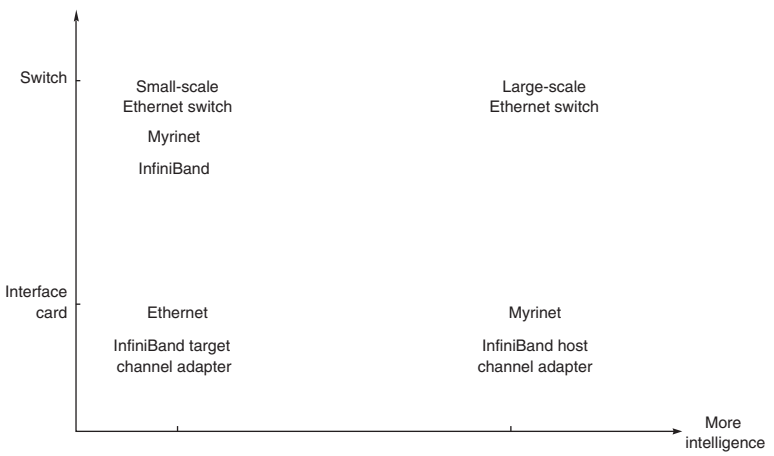


Figure F.39 Intelligence in a network: switch versus network interface card. Note that Ethernet switches come in two styles, depending on the size of the network, and that InfiniBand network interfaces come in two styles, depending on whether they are attached to a computer or to a storage device. Myrinet is a proprietary system area network.

Protection and User Access to the Network

A challenge is to ensure safe communication across a network without invoking the operating system in the common case. The Cray Research T3D supercomputer offers an interesting case study. Like the more recent Cray X1E, the T3D supports a global address space, so loads and stores can access memory across the network. Protection is ensured because each access is checked by the TLB. To support transfer of larger objects, a block transfer engine (BLT) was added to the hardware. Protection of access requires invoking the operating system before using the BLT to check the range of accesses to be sure there will be no protection violations.

Figure F.40 compares the bandwidth delivered as the size of the object varies for reads and writes. For very large reads (e.g., 512 KB), the BLT achieves the highest performance: 140 MB/sec. But simple loads get higher performance for 8 KB or less. For the write case, both achieve a peak of 90 MB/sec, presumably because of the limitations of the memory bus. But, for writes, the BLT can only match the performance of simple stores for transfers of 2 MB; anything smaller and it's faster to send stores. Clearly, a BLT that can avoid invoking the operating system in the common case would be more useful.

Efficient Interface to the Memory Hierarchy versus the Network

Traditional evaluations of processor performance, such as SPECint and SPECfp, encourage integration of the memory hierarchy with the processor as the efficiency of the memory hierarchy translates directly into processor performance. Hence,

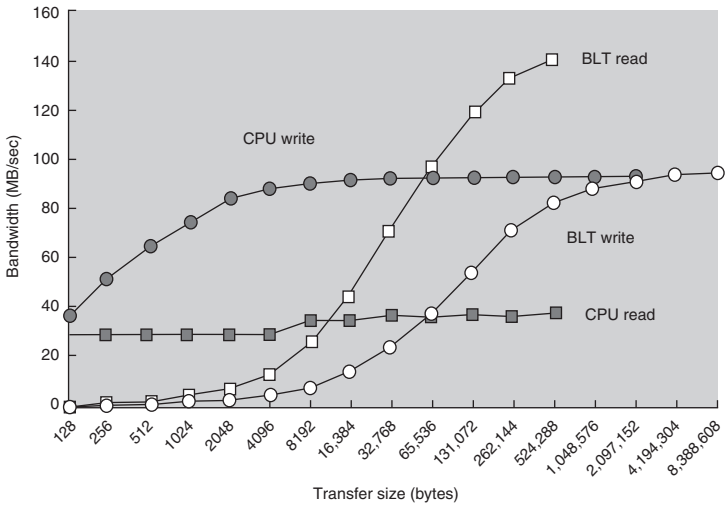


Figure F.40 Bandwidth versus transfer size for simple memory access instructions versus a block transfer device on the Cray Research T3D. (From Arpaci et al. [1995].)

microprocessors have multiple levels of caches on chip along with buffers for writes. Because benchmarks such as SPECint and SPECfp do not reward good interfaces to interconnection networks, many machines make the access time to the network delayed by the full memory hierarchy. Writes must lumber their way through full write buffers, and reads must go through the cycles of first-, second-, and often third-level cache misses before reaching the interconnection network. This hierarchy results in newer systems having higher latencies to the interconnect than older machines.

Let's compare three machines from the past: a 40-MHz SPARCstation-2, a 50-MHz SPARCstation-20 without an external cache, and a 50-MHz SPARCstation-20 with an external cache. According to SPECint95, this list is in order of increasing performance. The time to access the I/O bus (S-bus), however, increases in this sequence: 200 ns, 500 ns, and 1000 ns. The SPARCstation-2 is fastest because it has a single bus for memory and I/O, and there is only one level to the cache. The SPARCstation-20 memory access must first go over the memory bus (M-bus) and then to the I/O bus, adding 300 ns. Machines with a second-level cache pay an extra penalty of 500 ns before accessing the I/O bus.

Compute-Optimized Processors versus Receiver Overhead

The overhead to receive a message likely involves an interrupt, which bears the cost of flushing and then restarting the processor pipeline, if not offloaded. As mentioned earlier, reading network status and receiving data from the network interface likely operate at cache miss speeds. If microprocessors become more superscalar and go to even faster clock rates, the number of missed instruction issue opportunities per message reception will likely rise to unacceptable levels.

F.11 Fallacies and Pitfalls

Myths and hazards are widespread with interconnection networks. This section mentions several warnings, so proceed carefully.

Fallacy *The interconnection network is very fast and does not need to be improved*

The interconnection network provides certain functionality to the system, very much like the memory and I/O subsystems. It should be designed to allow processors to execute instructions at the maximum rate. The interconnection network subsystem should provide high enough bandwidth to keep from continuously entering saturation and becoming an overall system bottleneck.

In the 1980s, when wormhole switching was introduced, it became feasible to design large-diameter topologies with single-chip switches so that the bandwidth capacity of the network was not the limiting factor. This led to the flawed belief that interconnection networks need no further improvement.

Since the 1980s, much attention has been placed on improving processor performance, but comparatively less has been focused on interconnection networks. As technology advances, the interconnection network tends to represent an increasing fraction of system resources, cost, power consumption, and various other attributes that impact functionality and performance. Scaling the bandwidth simply by overdimensioning certain network parameters is no longer a cost-viable option. Designers must carefully consider the end-to-end interconnection network design in concert with the processor, memory, and I/O subsystems in order to achieve the required cost, power, functionality, and performance objectives of the entire system. An obvious case in point is multicore processors with on-chip networks.

Fallacy *Bisection bandwidth is an accurate cost constraint of a network*

Despite being very popular, bisection bandwidth has never been a practical constraint on the implementation of an interconnection network, although it may be one in future designs. It is more useful as a performance measure than as a cost measure. Chip pin-outs are the more realistic bandwidth constraint.

Pitfall *Using bandwidth (in particular, bisection bandwidth) as the only measure of network performance*

It seldom is the case that aggregate network bandwidth (likewise, network bisection bandwidth) is the end-to-end bottlenecking point across the network. Even if it were the case, networks are almost never 100% efficient in transporting packets across the bisection (i.e., $\rho < 100\%$) nor at receiving them at network endpoints (i.e., $\sigma < 100\%$). The former is highly dependent upon routing, switching, arbitration, and other such factors while both the former and the latter are highly dependent upon traffic characteristics. Ignoring these important factors and concentrating only on raw bandwidth can give very misleading performance predictions. For example, it is perfectly conceivable that a network could have higher aggregate bandwidth and/or bisection bandwidth relative to another network but also have lower measured performance!

Apparently, given sophisticated protocols like TCP/IP that maximize delivered bandwidth, many network companies believe that there is only one figure of merit for networks. This may be true for some applications, such as video streaming, where there is little interaction between the sender and the receiver. Many applications, however, are of a request-response nature, and so for every large message there must be one or more small messages. One example is NFS.

Figure F.41 compares a shared 10-Mbit/sec Ethernet LAN to a switched 155-Mbit/sec ATM LAN for NFS traffic. Ethernet drivers were better tuned than the ATM drivers, such that 10-Mbit/sec Ethernet was faster than 155-Mbit/sec ATM for payloads of 512 bytes or less. Figure F.41 shows the overhead time, transmission time, and total time to send all the NFS messages over Ethernet and ATM. The peak link speed of ATM is 15 times faster, and the measured link speed for 8-KB messages is almost 9 times faster. Yet, the higher overheads offset the benefits so that ATM would transmit NFS traffic only 1.2 times faster.

Size	Number of messages	Overhead (sec)		Number of data bytes	Transmission (sec)		Total time (sec)	
		ATM	Ethernet		ATM	Ethernet	ATM	Ethernet
32	771,060	532	389	33,817,052	4	48	536	436
64	56,923	39	29	4,101,088	0	5	40	34
96	4,082,014	2817	2057	428,346,316	46	475	2863	2532
128	5,574,092	3846	2809	779,600,736	83	822	3929	3631
160	328,439	227	166	54,860,484	6	56	232	222
192	16,313	11	8	3,316,416	0	3	12	12
224	4820	3	2	1,135,380	0	1	3	4
256	24,766	17	12	9,150,720	1	9	18	21
512	32,159	22	16	25,494,920	3	23	25	40
1024	69,834	48	35	70,578,564	8	72	56	108
1536	8842	6	4	15,762,180	2	14	8	19
2048	9170	6	5	20,621,760	2	19	8	23
2560	20,206	14	10	56,319,740	6	51	20	61
3072	13,549	9	7	43,184,992	4	39	14	46
3584	4200	3	2	16,152,228	2	14	5	17
4096	67,808	47	34	285,606,596	29	255	76	290
5120	6143	4	3	35,434,680	4	32	8	35
6144	5858	4	3	37,934,684	4	34	8	37
7168	4140	3	2	31,769,300	3	28	6	30
8192	287,577	198	145	2,390,688,480	245	2132	444	2277
Total	11,387,913	7858	5740	4,352,876,316	452	4132	8310	9872

Figure F.41 Total time on a 10-Mbit Ethernet and a 155-Mbit ATM, calculating the total overhead and transmission time separately. Note that the size of the headers needs to be added to the data bytes to calculate transmission time. The higher overhead of the software driver for ATM offsets the higher bandwidth of the network. These measurements were performed in 1994 using SPARCstation 10s, the ForeSystems SBA-200 ATM interface card, and the Fore Systems ASX-200 switch. (NFS measurements taken by Mike Dahlin of the University of California–Berkeley.)

Pitfall *Not providing sufficient reception link bandwidth, which causes the network end nodes to become even more of a bottleneck to performance*

Unless the traffic pattern is a permutation, several packets will concurrently arrive at some destinations when most source devices inject traffic, thus producing contention. If this problem is not addressed, contention may turn into congestion that will spread across the network. This can be dealt with by analyzing traffic patterns and providing extra reception bandwidth. For example, it is possible to implement more reception bandwidth than injection bandwidth. The IBM Blue Gene/L, for example, implements an on-chip switch with 7-bit

injection and 12-bit reception links, where the reception BW equals the aggregate switch input link BW.

Pitfall *Using high-performance network interface cards but forgetting about the I/O subsystem that sits between the network interface and the host processor*

This issue is related to the previous one. Messages are usually composed in user space buffers and later sent by calling a send function from the communications library. Alternatively, a cache controller implementing a cache coherence protocol may compose a message in some SANs and in OCNs. In both cases, messages have to be copied to the network interface memory before transmission. If the I/O bandwidth is lower than the link bandwidth or introduces significant overhead, this is going to affect communication performance significantly. As an example, the first 10-Gigabit Ethernet cards in the market had a PCI-X bus interface for the system with a significantly lower bandwidth than 10 Gbps.

Fallacy *Zero-copy protocols do not require copying messages or fragments from one buffer to another*

Traditional communication protocols for computer networks allow access to communication devices only through system calls in supervisor mode. As a consequence of this, communication routines need to copy the corresponding message from the user buffer to a kernel buffer when sending a message. Note that the communication protocol may need to keep a copy of the message for retransmission in case of error, and the application may modify the contents of the user buffer once the system call returns control to the application. This buffer-to-buffer copy is eliminated in zero-copy protocols because the communication routines are executed in user space and protocols are much simpler.

However, messages still need to be copied from the application buffer to the memory in the network interface card (NIC) so that the card hardware can transmit it from there through to the network. Although it is feasible to eliminate this copy by allocating application message buffers directly in the NIC memory (and, indeed, this is done in some protocols), this may not be convenient in current systems because access to the NIC memory is usually performed through the I/O subsystem, which usually is much slower than accessing main memory. Thus, it is generally more efficient to compose the message in main memory and let DMA devices take care of the transfer to the NIC memory.

Moreover, what few people count is the copy from where the message fragments are computed (usually the ALU, with results stored in some processor register) to main memory. Some systolic-like architectures in the 1980s, like the iWarp, were able to directly transmit message fragments from the processor to the network, effectively eliminating all the message copies. This is the approach taken in the Cray X1E shared-memory multiprocessor supercomputer.

Similar comments can be made regarding the reception side; however, this does not mean that zero-copy protocols are inefficient. These protocols represent the most efficient kind of implementation used in current systems.

Pitfall *Ignoring software overhead when determining performance*

Low software overhead requires cooperation with the operating system as well as with the communication libraries, but even with protocol offloading it continues to dominate the hardware overhead and must not be ignored. Figures F.32 and F.41 give two examples, one for a SAN standard and the other for a WAN standard. Other examples come from proprietary SANs for supercomputers. The Connection Machine CM-5 supercomputer in the early 1990s had a software overhead of 20 μs to send a message and a hardware overhead of only 0.5 μs. The first Intel Paragon supercomputer built in the early 1990s had a hardware overhead of just 0.2 μs, but the initial release of the software had an overhead of 250 μs. Later releases reduced this overhead down to 25 μs and, more recently, down to only a few microseconds, but this still dominates the hardware overhead. The IBM Blue Gene/L has an MPI sending/receiving overhead of approximately 3 μs, only a third of which (at most) is attributed to the hardware.

This pitfall is simply Amdahl's law applied to networks: Faster network hardware is superfluous if there is not a corresponding decrease in software overhead. The software overhead is much reduced these days with OS bypass, lightweight protocols, and protocol offloading down to a few microseconds or less, typically, but it remains a significant factor in determining performance.

Fallacy *MINs are more cost-effective than direct networks*

A MIN is usually implemented using significantly fewer switches than the number of devices that need to be connected. On the other hand, direct networks usually include a switch as an integral part of each node, thus requiring as many switches as nodes to interconnect. However, nothing prevents the implementation of nodes with multiple computing devices on it (e.g., a multicore processor with an on-chip switch) or with several devices attached to each switch (i.e., bristling). In these cases, a direct network may be as (or even more) cost-effective as a MIN. Note that, for a MIN, several network interfaces may be required at each node to match the bandwidth delivered by the multiple links per node provided by the direct network.

Fallacy *Low-dimensional direct networks achieve higher performance than high-dimensional networks such as hypercubes*

This conclusion was drawn by several studies that analyzed the optimal number of dimensions under the main physical constraint of bisection bandwidth. However, most of those studies did not consider link pipelining, considered only very short links, and/or did not consider switch architecture design constraints. The misplaced assumption that bisection bandwidth serves as the main limit did not help matters. Nowadays, most researchers and designers believe that high-radix switches are more cost-effective than low-radix switches, including some who concluded the opposite before.

Fallacy *Wormhole switching achieves better performance than other switching techniques*

Wormhole switching delivers the same no-load latency as other pipelined switching techniques, like virtual cut-through switching. The introduction of wormhole switches in the late 1980s coinciding with a dramatic increase in network bandwidth led many to believe that wormhole switching was the main reason for the performance boost. Instead, most of the performance increase came from a drastic increase in link bandwidth, which, in turn, was enabled by the ability of wormhole switching to buffer packet fragments using on-chip buffers, instead of using the node's main memory or some other off-chip source for that task. More recently, much larger on-chip buffers have become feasible, and virtual cutthrough achieved the same no-load latency as wormhole while delivering much higher throughput. This did not mean that wormhole switching was dead. It continues to be the switching technique of choice for applications in which only small buffers should be used (e.g., perhaps for on-chip networks).

Fallacy *Implementing a few virtual channels always increases throughput by allowing packets to pass through blocked packets ahead*

In general, implementing a few virtual channels in a wormhole switch is a good idea because packets are likely to pass blocked packets ahead of them, thus reducing latency and significantly increasing throughput. However, the improvements are not as dramatic for virtual cut-through switches. In virtual cut-through, buffers should be large enough to store several packets. As a consequence, each virtual channel may introduce HOL blocking, possibly degrading performance at high loads. Adding virtual channels increases cost, but it may deliver little additional performance unless there are as many virtual channels as switch ports and packets are mapped to virtual channels according to their destination (i.e., virtual output queueing). It is certainly the case that virtual channels can be useful in virtual cut-through networks to segregate different traffic classes, which can be very beneficial. However, multiplexing the packets over a physical link on a flit-by-flit basis causes all the packets from different virtual channels to get delayed. The average packet delay is significantly shorter if multiplexing takes place on a packet-by-packet basis, but in this case packet size should be bounded to prevent any one packet from monopolizing the majority of link bandwidth.

Fallacy *Adaptive routing causes out-of-order packet delivery, thus introducing too much overhead needed to reorder packets at the destination device*

Adaptive routing allows packets to follow alternative paths through the network depending on network traffic; therefore, adaptive routing usually introduces outof-order packet delivery. However, this does not necessarily imply that reordering packets at the destination device is going to introduce a large overhead, making adaptive routing not useful. For example, the most efficient adaptive routing algorithms to date support fully adaptive routing in some virtual channels but required

deterministic routing to be implemented in some other virtual channels in order to prevent deadlocks (à la the IBM Blue Gene/L). In this case, it is very easy to select between adaptive and deterministic routing for each individual packet. A single bit in the packet header can indicate to the switches whether all the virtual channels can be used or only those implementing deterministic routing. This hardware support can be used as indicated below to eliminate packet reordering overhead at the destination.

Most communication protocols for parallel computers and clusters implement two different protocols depending on message size. For short messages, an eager protocol is used in which messages are directly transmitted, and the receiving nodes use some preallocated buffer to temporarily store the incoming message. On the other hand, for long messages, a rendezvous protocol is used. In this case, a control message is sent first, requesting the destination node to allocate a buffer large enough to store the entire message. The destination node confirms buffer allocation by returning an acknowledgment, and the sender can proceed with fragmenting the message into bounded-size packets, transmitting them to the destination.

If eager messages use only deterministic routing, it is obvious that they do not introduce any reordering overhead at the destination. On the other hand, packets belonging to a long message can be transmitted using adaptive routing. As every packet contains the sequence number within the message (or the offset from the beginning of the message), the destination node can store every incoming packet directly in its correct location within the message buffer, thus incurring no overhead with respect to using deterministic routing. The only thing that differs is the completion condition. Instead of checking that the last packet in the message has arrived, it is now necessary to count the arrived packets, notifying the end of reception when the count equals the message size. Taking into account that long messages, even if not frequent, usually consume most of the network bandwidth, it is clear that most packets can benefit from adaptive routing without introducing reordering overhead when using the protocol described above.

Fallacy *Adaptive routing by itself always improves network fault tolerance because it allows packets to follow alternative paths*

Adaptive routing by itself is not enough to tolerate link and/or switch failures. Some mechanism is required to detect failures and notify them, so that the routing logic could exclude faulty paths and use the remaining ones. Moreover, while a given link or switch failure affects a certain number of paths when using deterministic routing, many more source/destination pairs could be affected by the same failure when using adaptive routing. As a consequence of this, some switches implementing adaptive routing transition to deterministic routing in the presence of failures. In this case, failures are usually tolerated by sending messages through alternative paths from the source node. As an example, the Cray T3E implements direction-order routing to tolerate a few failures. This fault-tolerant routing technique avoids cycles in the use of resources by crossing directions in order

(e.g., $X+$, $Y+$, $Z+$, $Z-$, $Y-$, then $X-$). At the same time, it provides an easy way to send packets through nonminimal paths, if necessary, to avoid crossing faulty components. For instance, a packet can be initially forwarded a few hops in the $X+$ direction even if it has to go in the $X-$ direction at some point later.

Pitfall *Trying to provide features only within the network versus end-to-end*

The concern is that of providing at a lower level the features that can only be accomplished at the highest level, thus only partially satisfying the communication demand. Saltzer, Reed, and Clark [1984] gave the end-to-end argument as follows:

> The function in question can completely and correctly be specified only with the knowledge and help of the application standing at the endpoints of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. [page 278]

Their example of the pitfall was a network at MIT that used several gateways, each of which added a checksum from one gateway to the next. The programmers of the application assumed that the checksum guaranteed accuracy, incorrectly believing that the message was protected while stored in the memory of each gateway. One gateway developed a transient failure that swapped one pair of bytes per million bytes transferred. Over time, the source code of one operating system was repeatedly passed through the gateway, thereby corrupting the code. The only solution was to correct infected source files by comparing them to paper listings and repairing code by hand! Had the checksums been calculated and checked by the application running on the end systems, safety would have been ensured.

There is a useful role for intermediate checks at the link level, however, provided that end-to-end checking is available. End-to-end checking may show that something is broken between two nodes, but it doesn't point to where the problem is. Intermediate checks can discover the broken component.

A second issue regards performance using intermediate checks. Although it is sufficient to retransmit the whole in case of failures from the end point, it can be much faster to retransmit a portion of the message at an intermediate point rather than wait for a time-out and a full message retransmit at the end point.

Pitfall *Relying on TCP/IP for all networks, regardless of latency, bandwidth, or software requirements*

The network designers on the first workstations decided it would be elegant to use a single protocol stack no matter where the destination of the message: Across a room or across an ocean, the TCP/IP overhead must be paid. This might have been a wise decision back then, especially given the unreliability of early Ethernet hardware, but it sets a high software overhead barrier for commercial systems of today. Such an obstacle lowers the enthusiasm for low-latency network interface hardware and low-latency interconnection networks if the software is just going to waste hundreds of microseconds when the message must travel only dozens of meters or less. It also can use significant processor resources. One rough rule of

thumb is that each Mbit/sec of TCP/IP bandwidth needs about 1 MHz of processor speed, so a 1000-Mbit/sec link could saturate a processor with an 800- to 1000-MHz clock.

The flip side is that, from a software perspective, TCP/IP is the most desirable target since it is the most connected and, hence, provides the largest number of opportunities. The downside of using software optimized to a particular LAN or SAN is that it is limited. For example, communication from a Java program depends on TCP/IP, so optimization for another protocol would require creation of glue software to interface Java to it.

TCP/IP advocates point out that the protocol itself is theoretically not as burdensome as current implementations, but progress has been modest in commercial systems. There are also TCP/IP offloading engines in the market, with the hope of preserving the universal software model while reducing processor utilization and message latency. If processors continue to improve much faster than network speeds, or if multiple processors become ubiquitous, software TCP/IP may become less significant for processor utilization and message latency.

F.12 Concluding Remarks

Interconnection network design is one of the most exciting areas of computer architecture development today. With the advent of new multicore processor paradigms and advances in traditional multiprocessor/cluster systems and the Internet, many challenges and opportunities exist for interconnect architecture innovation. These apply to all levels of computer systems: communication between cores on a chip, between chips on a board, between boards in a system, and between computers in a machine room, over a local area and across the globe. Irrespective of their domain of application, interconnection networks should transfer the maximum amount of information within the least amount of time for given cost and power constraints so as not to bottleneck the system. Topology, routing, arbitration, switching, and flow control are among some of the key concepts in realizing such high-performance designs.

The design of interconnection networks is end-to-end: It includes injection links, reception links, and the interfaces at network end points as much as it does the topology, switches, and links within the network fabric. It is often the case that the bandwidth and overhead at the end node interfaces are the bottleneck, yet many mistakenly think of the interconnection network to mean only the network fabric. This is as bad as processor designers thinking of computer architecture to mean only the instruction set architecture or only the microarchitecture! End-to-end issues and understanding of the traffic characteristics make the design of interconnection networks challenging and very much relevant even today. For instance, the need for low end-to-end latency is driving the development of efficient network interfaces located closer to the processor/memory controller. We may soon see most multicore processors used in multiprocessor systems implementing network interfaces on-chip,

devoting some core(s) to execute communication tasks. This is already the case for the IBM Blue Gene/L supercomputer, which uses one of its two cores on each processor chip for this purpose.

Networking has a long way to go from its humble shared-media beginnings. It is in "catch-up" mode, with switched-media point-to-point networks only recently displacing traditional bus-based networks in many networking domains, including on chip, I/O, and the local area. We are not near any performance plateaus, so we expect rapid advancement of WANs, LANs, SANs, and especially OCNs in the near future. Greater interconnection network performance is key to the information- and communication-centric vision of the future of our field, which, so far, has benefited many millions of people around the world in various ways. As the quotes at the beginning of this appendix suggest, this revolution in *two-way* communication is at the heart of changes in the form of our human associations and actions.

Acknowledgments

We express our sincere thanks to the following persons who, in some way, have contributed to the contents of the previous edition of the appendix: Lei Chai, Scott Clark, José Flich, Jose Manuel Garcia, Paco Gilabert, Rama Govindaraju, Manish Gupta, Wai Hong Ho, Siao Jer, Steven Keckler, Dhabaleswar (D.K.) Panda, Fabrizio Petrini, Steve Scott, Jeonghee Shin, Craig Stunkel, Sayantan Sur, Michael B. Taylor, and Bilal Zafar. We especially appreciate the new contributions of Jose Flich to this edition of the appendix.

F.13 Historical Perspective and References

This appendix has taken the perspective that interconnection networks for very different domains—from on-chip networks within a processor chip to wide area networks connecting computers across the globe—share many of the same concerns. With this, interconnection network concepts are presented in a unified way, irrespective of their application; however, their histories are vastly different, as evidenced by the different solutions adopted to address similar problems. The lack of significant interaction between research communities from the different domains certainly contributed to the diversity of implemented solutions. Highlighted below are relevant readings on each topic. In addition, good general texts featuring WAN and LAN networking have been written by Davie, Peterson, and Clark [1999] and by Kurose and Ross [2001]. Good texts focused on SANs for multiprocessors and clusters have been written by Duato, Yalamanchili, and Ni [2003] and by Dally and Towles [2004]. An informative chapter devoted to dead-lock resolution in interconnection networks was written by Pinkston [2004]. Finally, an edited work by Jantsch and Tenhunen [2003] on OCNs for multicore processors and system-on-chips is also interesting reading.

Wide Area Networks

Wide area networks are the earliest of the data interconnection networks. The forerunner of the Internet is the ARPANET, which in 1969 connected computer science departments across the United States that had research grants funded by the Advanced Research Project Agency (ARPA), a U.S. government agency. It was originally envisioned as using reliable communications at lower levels. Practical experience with failures of the underlying technology led to the failure-tolerant TCP/IP, which is the basis for the Internet today. Vint Cerf and Robert Kahn are credited with developing the TCP/IP protocols in the mid-1970s, winning the ACM Software Award in recognition of that achievement. Kahn [1972] is an early reference on the ideas of ARPANET. For those interested in learning more about TPC/IP, Stevens [1994–1996] has written classic books on the topic.

In 1975, there were roughly 100 networks in the ARPANET; in 1983, only 200. In 1995, the Internet encompassed 50,000 networks worldwide, about half of which were in the United States. That number is hard to calculate now, but the number of IP hosts grew by a factor of 15 from 1995 to 2000, reaching 100 million Internet hosts by the end of 2000. It has grown much faster since then. With most service providers assigning dynamic IP addresses, many local area networks using private IP addresses, and with most networks allowing wireless connections, the total number of hosts in the Internet is nearly impossible to compute. In July 2005, the Internet Systems Consortium (*www.isc.org*) estimated more than 350 million Internet hosts, with an annual increase of about 25% projected. Although key government networks made the Internet possible (i.e., ARPANET and NSFNET), these networks have been taken over by the commercial sector, allowing the Internet to thrive. But major innovations to the Internet are still likely to come from government-sponsored research projects rather than from the commercial sector. The National Science Foundation's Global Environment for Network Innovation (GENI) initiative is an example of this.

The most exciting application of the Internet is the World Wide Web, developed in 1989 by Tim Berners-Lee, a programmer at the European Center for Particle Research (CERN), for information access. In 1992, a young programmer at the University of Illinois, Marc Andreessen, developed a graphical interface for the Web called Mosaic. It became immensely popular. He later became a founder of Netscape, which popularized commercial browsers. In May 1995, at the time of the second edition of this book, there were over 30,000 Web pages, and the number was doubling every two months. During the writing of the third edition of this text, there were more than 1.3 billion Web pages. In December 2005, the number of Web servers approached 75 million, having increased by 30% during that same year.

Asynchronous Transfer Mode (ATM) was an attempt to design the definitive communication standard. It provided good support for data transmission as well as digital voice transmission (i.e., phone calls). From a technical point of view, it combined the best from packet switching and circuit switching, also providing excellent support for providing quality of service (QoS). Alles [1995] offers a good

survey on ATM. In 1995, no one doubted that ATM was going to be the future for this community. Ten years later, the high equipment and personnel training costs basically killed ATM, and we returned back to the simplicity of TCP/IP. Another important blow to ATM was its defeat by the Ethernet family in the LAN domain, where packet switching achieved significantly lower latencies than ATM, which required establishing a connection before data transmission. ATM connectionless servers were later introduced in an attempt to fix this problem, but they were expensive and represented a central bottleneck in the LAN.

Finally, WANs today rely on optical fiber. Fiber technology has made so many advances that today WAN fiber bandwidth is often underutilized. The main reason for this is the commercial introduction of wavelength division multiplexing (WDM), which allows each fiber to transmit many data streams simultaneously over different wavelengths, thus allowing three orders of magnitude bandwidth increase in just one generation, that is, 3 to 5 years (a good text by Senior [1993] discusses optical fiber communications). However, IP routers may still become a bottleneck. At 10- to 40-Gbps link rates, and with thousands of ports in large core IP routers, packets must be processed very quickly—that is, within a few tens of nanoseconds. The most time-consuming operation is routing. The way IP addresses have been defined and assigned to Internet hosts makes routing very complicated, usually requiring a complex search in a tree structure for every packet. Network processors have become popular as a cost-effective solution for implementing routing and other packet-filtering operations. They usually are RISC-like and highly multithreaded and implement local stores instead of caches.

Local Area Networks

ARPA's success with wide area networks led directly to the most popular local area networks. Many researchers at Xerox Palo Alto Research Center had been funded by ARPA while working at universities, so they all knew the value of networking. In 1974, this group invented the Alto, the forerunner of today's desktop computers [Thacker et al. 1982], and the Ethernet [Metcalfe and Boggs 1976], today's LAN. This group—David Boggs, Butler Lampson, Ed McCreight, Bob Sprowl, and Chuck Thacker—became luminaries in computer science and engineering, collecting a treasure chest of awards among them.

This first Ethernet provided a 3-Mbit/sec interconnection, which seemed like an unlimited amount of communication bandwidth with computers of that era. It relied on the interconnect technology developed for the cable television industry. Special microcode support gave a round-trip time of 50 μs for the Alto over Ethernet, which is still a respectable latency. It was Boggs' experience as a ham radio operator that led to a design that did not need a central arbiter, but instead listened before use and then varied back-off times in case of conflicts.

The announcement by Digital Equipment Corporation, Intel, and Xerox of a standard for 10-Mbit/sec Ethernet was critical to the commercial success of

Ethernet. This announcement short-circuited a lengthy IEEE standards effort, which eventually did publish IEEE 802.3 as a standard for Ethernet.

There have been several unsuccessful candidates that have tried to replace the Ethernet. The Fiber Data Distribution Interconnect (FDDI) committee, unfortunately, took a very long time to agree on the standard, and the resulting interfaces were expensive. It was also a shared medium when switches were becoming affordable. ATM also missed the opportunity in part because of the long time to standardize the LAN version of ATM, and in part because of the high latency and poor behavior of ATM connectionless servers, as mentioned above. InfiniBand for the reasons discussed below has also faltered. As a result, Ethernet continues to be the absolute leader in the LAN environment, and it remains a strong opponent in the high-performance computing market as well, competing against the SANs by delivering high bandwidth at low cost. The main drawback of Ethernet for high-end systems is its relatively high latency and lack of support in most interface cards to implement the necessary protocols.

Because of failures of the past, LAN modernization efforts have been centered on extending Ethernet to lower-cost media such as unshielded twisted pair (UTP), switched interconnects, and higher link speeds as well as to new domains such as wireless communication. Practically all new PC motherboards and laptops implement a Fast/Gigabit Ethernet port (100/1000 Mbps), and most laptops implement a 54 Mbps Wireless Ethernet connection. Also, home wired or wireless LANs connecting all the home appliances, set-top boxes, desktops, and laptops to a shared Internet connection are very common. Spurgeon [2006] has provided a nice online summary of Ethernet technology, including some of its history.

System Area Networks

One of the first nonblocking multistage interconnection networks was proposed by Clos [1953] for use in telephone exchange offices. Building on this, many early inventions for system area networks came from their use in massively parallel processors (MPPs). One of the first MPPs was the Illiac IV, a SIMD array built in the early 1970s with 64 processing elements ("massive" at that time) interconnected using a topology based on a 2D torus that provided neighbor-to-neighbor communication. Another representative of early MPP was the Cosmic Cube, which used Ethernet interface chips to connect 64 processors in a 6-cube. Communication between nonneighboring nodes was made possible by store-and-forwarding of packets at intermediate nodes toward their final destination. A much larger and truly "massive" MPP built in the mid-1980s was the Connection Machine, a SIMD multiprocessor consisting of 64 K 1-bit processing elements, which also used a hypercube with store-and-forwarding. Since these early MPP machines, interconnection networks have improved considerably.

In the 1970s through the 1990s, considerable research went into trying to optimize the topology and, later, the routing algorithm, switching, arbitration, and flow control techniques. Initially, research focused on maximizing performance with

little attention paid to implementation constraints or crosscutting issues. Many exotic topologies were proposed having very interesting properties, but most of them complicated the routing. Rising from the fray was the hypercube, a very popular network in the 1980s that has all but disappeared from MPPs since the 1990s. What contributed to this shift was a performance model by Dally [1990] that showed that if the implementation is wire limited, lower-dimensional topologies achieve better performance than higher-dimensional ones because of their wider links for a given wire budget. Many designers followed that trend assuming their designs to be wire limited, even though most implementations were (and still are) pin limited. Several supercomputers since the 1990s have implemented low-dimensional topologies, including the Intel Paragon, Cray T3D, Cray T3E, HP AlphaServer, Intel ASCI Red, and IBM Blue Gene/L.

Meanwhile, other designers followed a very different approach, implementing bidirectional MINs in order to reduce the number of required switches below the number of network nodes. The most popular bidirectional MIN was the fat tree topology, originally proposed by Leiserson [1985] and first used in the Connection Machine CM-5 supercomputer and, later, the IBM ASCI White and ASC Purple supercomputers. This indirect topology was also used in several European parallel computers based on the Transputer. The Quadrics network has inherited characteristics from some of those Transputer-based networks. Myrinet has also evolved significantly from its first version, with Myrinet 2000 incorporating the fat tree as its principal topology. Indeed, most current implementations of SANs, including Myrinet, InfiniBand, and Quadrics as well as future implementations such as PCI-Express Advanced Switching, are based on fat trees.

Although the topology is the most visible aspect of a network, other features also have a significant impact on performance. A seminal work that raised awareness of deadlock properties in computer systems was published by Holt [1972]. Early techniques for avoiding deadlock in store-and-forward networks were proposed by Merlin and Schweitzer [1980] and by Gunther [1981]. Pipelined switching techniques were first introduced by Kermani and Kleinrock [1979] (virtual cut-through) and improved upon by Dally and Seitz [1986] (wormhole), which significantly reduced low-load latency and the topology's impact on message latency over previously proposed techniques. Wormhole switching was initially better than virtual cut-through largely because flow control could be implemented at a granularity smaller than a packet, allowing high-bandwidth links that were not as constrained by available switch memory bandwidth. Today, virtual cut-through is usually preferred over wormhole because it achieves higher throughput due to less HOL blocking effects and is enabled by current integration technology that allows the implementation of many packet buffers per link.

Tamir and Frazier [1992] laid the groundwork for virtual output queuing with the notion of dynamically allocated multiqueues. Around this same time, Dally [1992] contributed the concept of virtual channels, which was key to the development of more efficient deadlock-free routing algorithms and congestion-reducing flow control techniques for improved network throughput. Another highly relevant contribution to routing was a new theory proposed by Duato [1993] that allowed

the implementation of fully adaptive routing with just one "escape" virtual channel to avoid deadlock. Previous to this, the required number of virtual channels to avoid deadlock increased exponentially with the number of network dimensions. Pinkston and Warnakulasuriya [1997] went on to show that deadlock actually can occur very infrequently, giving credence to deadlock recovery routing approaches. Scott and Goodman [1994] were among the first to analyze the usefulness of pipelined channels for making link bandwidth independent of the time of flight. These and many other innovations have become quite popular, finding use in most high-performance interconnection networks, both past and present. The IBM Blue Gene/L, for example, implements virtual cut-through switching, four virtual channels per link, fully adaptive routing with one escape channel, and pipelined links.

MPPs represent a very small (and currently shrinking) fraction of the information technology market, giving way to bladed servers and clusters. In the United States, government programs such as the Advanced Simulation and Computing (ASC) program (formerly known as the Accelerated Strategic Computing Initiative, or ASCI) have promoted the design of those machines, resulting in a series of increasingly powerful one-of-a-kind MPPs costing $50 million to $100 million. These days, many are basically lower-cost clusters of symmetric multiprocessors (SMPs) (see Pfister [1998] and Sterling [2001] for two perspectives on clustering). In fact, in 2005, nearly 75% of the TOP500 supercomputers were clusters. Nevertheless, the design of each generation of MPPs and even clusters pushes interconnection network research forward to confront new problems arising due to shear size and other scaling factors. For instance, source-based routing—the simplest form of routing—does not scale well to large systems. Likewise, fat trees require increasingly longer links as the network size increases, which led IBM Blue Gene/L designers to adopt a 3D torus network with distributed routing that can be implemented with bounded-length links.

Storage Area Networks

System area networks were originally designed for a single room or single floor (thus their distances are tens to hundreds of meters) and were for use in MPPs and clusters. In the intervening years, the acronym SAN has been co-opted to also mean storage area networks, whereby networking technology is used to connect storage devices to compute servers. Today, many refer to "storage" when they say SAN. The most widely used SAN example in 2006 was Fibre Channel (FC), which comes in many varieties, including various versions of Fibre Channel Arbitrated Loop (FC-AL) and Fibre Channel Switched (FC-SW). Not only are disk arrays attached to servers via FC links, but there are even some disks with FC links attached to switches so that storage area networks can enjoy the benefits of greater bandwidth and interconnectivity of switching.

In October 2000, the InfiniBand Trade Association announced the version 1.0 specification of InfiniBand [InfiniBand Trade Association 2001]. Led by Intel, HP, IBM, Sun, and other companies, it was targeted to the high-performance

computing market as a successor to the PCI bus by having point-to-point links and switches with its own set of protocols. Its characteristics are desirable potentially both for system area networks to connect clusters and for storage area networks to connect disk arrays to servers. Consequently, it has had strong competition from both fronts. On the storage area networking side, the chief competition for InfiniBand has been the rapidly improving Ethernet technology widely used in LANs. The Internet Engineering Task Force proposed a standard called iSCSI to send SCSI commands over IP networks [Satran et al. 2001]. Given the cost advantages of the higher-volume Ethernet switches and interface cards, Gigabit Ethernet dominates the low-end and medium range for this market. What's more, the slow introduction of InfiniBand and its small market share delayed the development of chip sets incorporating native support for InfiniBand. Therefore, network interface cards had to be plugged into the PCI or PCI-X bus, thus never delivering on the promise of replacing the PCI bus.

It was another I/O standard, PCI-Express, that finally replaced the PCI bus. Like InfiniBand, PCI-Express implements a switched network but with point-to-point serial links. To its credit, it maintains software compatibility with the PCI bus, drastically simplifying migration to the new I/O interface. Moreover, PCI-Express benefited significantly from mass market production and has found application in the desktop market for connecting one or more high-end graphics cards, making gamers very happy. Every PC motherboard now implements one or more 16x PCI-Express interfaces. PCI-Express absolutely dominates the I/O interface, but the current standard does not provide support for interprocessor communication.

Yet another standard, Advanced Switching Interconnect (ASI), may emerge as a complementary technology to PCI-Express. ASI is compatible with PCI-Express, thus linking directly to current motherboards, but it also implements support for interprocessor communication as well as I/O. Its defenders believe that it will eventually replace both SANs and LANs with a unified network in the data center market, but ironically this was also said of InfiniBand. The interested reader is referred to Pinkston et al. [2003] for a detailed discussion on this. There is also a new disk interface standard called Serial Advanced Technology Attachment (SATA) that is replacing parallel Integrated Device Electronics (IDE) with serial signaling technology to allow for increased bandwidth. Most disks in the market use this new interface, but keep in mind that Fibre Channel is still alive and well. Indeed, most of the promises made by InfiniBand in the SAN market were satisfied by Fibre Channel first, thus increasing their share of the market.

Some believe that Ethernet, PCI-Express, and SATA have the edge in the LAN, I/O interface, and disk interface areas, respectively. But the fate of the remaining storage area networking contenders depends on many factors. A wonderful characteristic of computer architecture is that such issues will not remain endless academic debates, unresolved as people rehash the same arguments repeatedly. Instead, the battle is fought in the marketplace, with well-funded and talented groups giving their best efforts at shaping the future. Moreover, constant changes to technology reward those who are either astute or lucky. The best combination of

technology and follow-through has often determined commercial success. Time will tell us who will win and who will lose, at least for the next round!

On-Chip Networks

Relative to the other network domains, on-chip networks are in their infancy. As recently as the late 1990s, the traditional way of interconnecting devices such as caches, register files, ALUs, and other functional units within a chip was to use dedicated links aimed at minimizing latency or shared buses aimed at simplicity. But with subsequent increases in the volume of interconnected devices on a single chip, the length and delay of wires to cross a chip, and chip power consumption, it has become important to share on-chip interconnect bandwidth in a more structured way, giving rise to the notion of a network on-chip. Among the first to recognize this were Agarwal [Waingold et al. 1997] and Dally [Dally 1999; Dally and Towles 2001]. They and others argued that on-chip networks that route packets allow efficient sharing of burgeoning wire resources between many communication flows and also facilitate modularity to mitigate chip-crossing wire delay problems identified by Ho, Mai, and Horowitz [2001]. Switched on-chip networks were also viewed as providing better fault isolation and tolerance. Challenges in designing these networks were later described by Taylor et al. [2005], who also proposed a 5-tuple model for characterizing the delay of OCNs. A design process for OCNs that provides a complete synthesis flow was proposed by Bertozzi et al. [2005]. Following these early works, much research and development has gone into on-chip network design, making this a very hot area of microarchitecture activity.

Multicore and tiled designs featuring on-chip networks have become very popular since the turn of the millennium. Pinkston and Shin [2005] provide a survey of on-chip networks used in early multicore/tiled systems. Most designs exploit the reduced wiring complexity of switched OCNs as the paths between cores/tiles can be precisely defined and optimized early in the design process, thus enabling improved power and performance characteristics. With typically tens of thousands of wires attached to the four edges of a core or tile as "pinouts," wire resources can be traded off for improved network performance by having very wide channels over which data can be sent broadside (and possibly scaled up or down according to the power management technique), as opposed to serializing the data over fixed narrow channels.

Rings, meshes, and crossbars are straightforward to implement in planar chip technology and routing is easily defined on them, so these were popular topological choices in early switched OCNs. It will be interesting to see if this trend continues in the future when several tens to hundreds of heterogeneous cores and tiles will likely be interconnected within a single chip, possibly using 3D integration technology. Considering that processor microarchitecture has evolved significantly from its early beginnings in response to application demands and technological advancements, we would expect to see vast architectural improvements to on-chip networks as well.

References

Agarwal, A., 1991. Limits on interconnection network performance. IEEE Trans. on Parallel and Distributed Systems 2 (4 (April)), 398–412.

Alles, A., 1995. "ATM internetworking" (May). www.cisco.com/warp/public/614/12.html.

Anderson, T.E., Culler, D.E., Patterson, D., 1995. A case for NOW (networks of workstations). IEEE Micro 15 (1 (February)), 54–64.

Anjan, K.V., Pinkston, T.M., 1995. An efficient, fully-adaptive deadlock recovery scheme: Disha. In: Proc. 22nd Annual Int'l. Symposium on Computer Architecture, June 22–24, 1995. Santa Margherita Ligure, Italy.

Arpaci, R.H., Culler, D.E., Krishnamurthy, A., Steinberg, S.G., Yelick, K., 1995. Empirical evaluation of the Cray-T3D: A compiler perspective. In: Proc. 22nd Annual Int'l. Symposium on Computer Architecture, June 22–24, 1995. Santa Margherita Ligure, Italy.

Bell, G., Gray, J., 2001. Crays, Clusters and Centers. Microsoft Corporation, Redmond, Wash. MSR-TR-2001-76.

Benes, V.E., 1962. Rearrangeable three stage connecting networks. Bell Syst. Tech. J. 41, 1481–1492.

Bertozzi, D., Jalabert, A., Murali, S., Tamhankar, R., Stergiou, S., Benini, L., De Micheli, G., 2005. NoC synthesis flow for customized domain specific multiprocessor systems-on-chip. IEEE Trans. on Parallel and Distributed Systems 16 (2 (February)), 113–130.

Bhuyan, L.N., Agrawal, D.P., 1984. Generalized hypercube and hyperbus structures for a computer network. IEEE Trans. on Computers 32 (4 (April)), 322–333.

Brewer, E.A., Kuszmaul, B.C., 1994. How to get good performance from the CM-5 data network. In: Proc. Eighth Int'l Parallel Processing Symposium, April 26–29, 1994. Cancun, Mexico.

Clos, C., 1953. A study of non-blocking switching networks. Bell Systems Technical Journal 32 (March), 406–424.

Dally, W.J., 1990. Performance analysis of k-ary n-cube interconnection networks. IEEE Trans. on Computers 39 (6 (June)), 775–785.

Dally, W.J., 1992. Virtual channel flow control. IEEE Trans. on Parallel and Distributed Systems 3 (2 (March)), 194–205.

Dally, W.J., 1999. Interconnect limited VLSI architecture. In: Proc. of the Int'l. Interconnect Technology Conference, May 24–26, 1999. San Francisco, Calif.

Dally, W.J., Seitz, C.I., 1986. The torus routing chip. Distributed Computing 1 (4), 187–196.

Dally, W.J., Towles, B., 2001. Route packets, not wires: On-chip interconnection networks. In: Proc. of the 38th Design Automation Conference, June 18–22, 2001. Las Vegas, Nev.

Dally, W.J., Towles, B., 2004. Principles and Practices of Interconnection Networks. Morgan Kaufmann Publishers, San Francisco.

Davie, B.S., Peterson, L.L., Clark, D., 1999. Computer Networks: A Systems Approach, second ed. Morgan Kaufmann Publishers, San Francisco.

Duato, J., 1993. A new theory of deadlock-free adaptive routing in wormhole networks. IEEE Trans. on Parallel and Distributed Systems 4 (12 (December)), 1320–1331.

Duato, J., Pinkston, T.M., 2001. A general theory for deadlock-free adaptive routing using a mixed set of resources. IEEE Trans. on Parallel and Distributed Systems 12 (12 (December)), 1219–1235.

Duato, J., Yalamanchili, S., Ni, L., 2003. Interconnection Networks: An Engineering Approach. Morgan Kaufmann Publishers, San Francisco. 2nd printing.

Duato, J., Johnson, I., Flich, J., Naven, F., Garcia, P., Nachiondo, T., 2005a. A new scalable and cost-effective congestion management strategy for lossless multistage interconnection networks. In: Proc. 11th Int'l. Symposium on High Performance Computer Architecture, February 12–16, 2005 San Francisco.

Duato, J., Lysne, O., Pang, R., Pinkston, T.M., 2005b. Part I: A theory for deadlock-free dynamic reconfiguration of interconnection networks. IEEE Trans. on Parallel and Distributed Systems 16 (5 (May)), 412–427.

Flich, J., Bertozzi, D., 2010. Designing Network-on-Chip Architectures in the Nanoscale Era. CRC Press, Boca Raton, FL.

Glass, C.J., Ni, L.M., 1992. The Turn Model for adaptive routing. In: Proc. 19th Int'l. Symposium on Computer Architecture. May, Gold Coast, Australia.

Gunther, K.D., 1981. Prevention of deadlocks in packet-switched data transport systems. IEEE Trans. on Communications, 512–524. COM–29:4 (April).

Ho, R., Mai, K.W., Horowitz, M.A., 2001. The future of wires. In: Proc. of the IEEE 89:4 (April), pp. 490–504.

Holt, R.C., 1972. Some deadlock properties of computer systems. ACM Computer Surveys 4 (3 (September)), 179–196.

Hoskote, Y., Vangal, S., Singh, A., Borkar, N., Borkar, S., 2007. A 5-ghz mesh interconnect for a teraflops processor. IEEE Micro 27 (5), 51–61.

Howard, J., Dighe, S., Hoskote, Y., Vangal, S., Finan, S., Ruhl, G., Jenkins, D., Wilson, H., Borka, N., Schrom, G., Pailet, F., Jain, S., Jacob, T., Yada, S., Marella, S., Salihundam, P., Erraguntla, V., Konow, M., Riepen, M., Droege, G., Lindemann, J., Gries, M., Apel, T., Henriss, K., Lund-Larsen, T., Steibl, S., Borkar, S., De, V., Van Der Wijngaart, R., Mattson, T., 2010. A 48-core IA-32 message-passing processor with DVFS in 45 nm CMOS. In: IEEE International Solid-State Circuits Conference Digest of Technical Papers, pp. 58–59.

InfiniBand Trade Association, 2001. InfiniBand Architecture Specifications Release 1.0.a. www.infinibandta.org.

Jantsch, A., Tenhunen, H. (Eds.), 2003. Networks on Chips. Kluwer Academic Publishers, The Netherlands.

Kahn, R.E., 1972. Resource-sharing computer communication networks. In: Proc. IEEE 60:11 (November), pp. 1397–1407.

Kermani, P., Kleinrock, L., 1979. Virtual cut-through: A new computer communication switching technique. Computer Networks 3 (January), 267–286.

Kurose, J.F., Ross, K.W., 2001. Computer Networking: A Top-Down Approach Featuring the Internet. Addison-Wesley, Boston.

Leiserson, C.E., 1985. Fat trees: Universal networks for hardware-efficient supercomputing. IEEE Trans. on Computers, 892–901. C–34:10 (October).

Merlin, P.M., Schweitzer, P.J., 1980. Deadlock avoidance in store-and-forward networks. I. Store-and-forward deadlock. IEEE Trans. on Communications, 345–354. COM–28:3 (March).

Metcalfe, R.M., 1993. Computer/network interface design: Lessons from Arpanet and Ethernet. IEEE J. on Selected Areas in Communications 11 (2 (February)), 173–180.

Metcalfe, R.M., Boggs, D.R., 1976. Ethernet: Distributed packet switching for local computer networks. Comm. ACM 19 (7 (July)), 395–404.

Partridge, C., 1994. Gigabit Networking. Addison-Wesley, Reading, Mass.

Peh, L.S., Dally, W.J., 2001. A delay model and speculative architecture for pipelined routers. In: Proc. 7th Int'l. Symposium on High Performance Computer Architecture, January 20–24, 2001. Monterrey, Mexico.

Pfister, G.F., 1998. In Search of Clusters, second ed. Prentice Hall, Upper Saddle River, N.J.

Pinkston, T.M., 2004. Deadlock characterization and resolution in interconnection networks. In: Zhu, M.C., Fanti, M.P. (Eds.), Deadlock Resolution in Computer-Integrated Systems. CRC Press, Boca Raton, Fl, pp. 445–492.

Pinkston, T.M., Shin, J., 2005. Trends toward on-chip networked microsystems. Int'l. J. of High Performance Computing and Networking 3 (1), 3–18.

Pinkston, T.M., Warnakulasuriya, S., 1997. On deadlocks in interconnection networks. In: Proc. 24th Int'l. Symposium on Computer Architecture, June 2–4, 1997. Denver, Colo.

Pinkston, T.M., Benner, A., Krause, M., Robinson, I., Sterling, T., 2003. InfiniBand: The 'de facto' future standard for system and local area networks or just a scalable replacement for PCI buses?" Special Issue on Communication Architecture for Clusters 6:2 (April). Cluster Computing, 95–104.

Puente, V., Beivide, R., Gregorio, J.A., Prellezo, J.M., Duato, J., Izu, C., 1999. Adaptive bubble router: A design to improve performance in torus networks. In: Proc. 28th Int'l. Conference on Parallel Processing, September 21–24, 1999. Aizu-Wakamatsu, Japan.

Rodrigo, S., Flich, J., Duato, J., Hummel, M., 2008. Efficient unicast and multicast support for CMPs. In: Proc. 41st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-41), November 8–12, 2008. Lake Como, Italy, pp. 364–375.

Saltzer, J.H., Reed, D.P., Clark, D.D., 1984. End-to-end arguments in system design. ACM Trans. on Computer Systems 2 (4 (November)), 277–288.

Satran, J., Smith, D., Meth, K., Sapuntzakis, C., Wakeley, M., Von Stamwitz, P., Haagens, R., Zeidner, E., Dalle Ore, L., Klein, Y., 2001. "iSCSI", IPS working group of IETF, Internet draft. www.ietf.org/internet-drafts/draft-ietf-ips-iscsi-07.txt.

Scott, S.L., Goodman, J., 1994. The impact of pipelined channels on k-ary n-cube networks. IEEE Trans. on Parallel and Distributed Systems 5 (1 (January)), 1–16.

Senior, J.M., 1993. Optical Fiber Commmunications: Principles and Practice, second ed. Prentice Hall, Hertfordshire, U.K..

Spurgeon, C., 2006. Charles Spurgeon's Ethernet Web Site. www.etherman-age.com/ethernet/ethernet.html.

Sterling, T., 2001. Beowulf PC Cluster Computing with Windows and Beowulf PC Cluster Computing with Linux. MIT Press, Cambridge, Mass.
Stevens, W.R., 1994–1996. *TCP/IP Illustrated* (three volumes). Addison-Wesley, Reading, Mass.
Tamir, Y., Frazier, G., 1992. Dynamically-allocated multi-queue buffers for VLSI communication switches. IEEE Trans. on Computers 41 (6 (June)), 725–734.
Tanenbaum, A.S., 1988. Computer Networks, second ed. Prentice Hall, Englewood Cliffs, N.J.
Taylor, M.B., Lee, W., Amarasinghe, S.P., Agarwal, A., 2005. Scalar operand networks. IEEE Trans. on Parallel and Distributed Systems 16 (2 (February)), 145–162.
Thacker, C.P., McCreight, E.M., Lampson, B.W., Sproull, R.F., Boggs, D.R., 1982. Alto: A personal computer. In: Siewiorek, D.P., Bell, C.G., Newell, A. (Eds.), Computer Structures: Principles and Examples. McGraw-Hill, New York, pp. 549–572.
TILE-GX, http://www.tilera.com/sites/default/files/productbriefs/PB025_TILE-Gx_Processor_A_v3.pdf.
Vaidya, A.S., Sivasubramaniam, A., Das, C.R., 1997. Performance benefits of virtual channels and adaptive routing: An application-driven study. In: Proc. 11th ACM Int'l Conference on Supercomputing, July 7–11, 1997. Vienna, Austria.
Van Leeuwen, J., Tan, R.B., 1987. Interval Routing. The Computer Journal 30 (4), 298–307.
von Eicken, T., Culler, D.E., Goldstein, S.C., Schauser, K.E., 1992. Active messages: A mechanism for integrated communication and computation. In: Proc. 19th Annual Int'l. Symposium on Computer Architecture, May 19–21, 1992. Gold Coast, Australia.
Waingold, E., Taylor, M., Srikrishna, D., Sarkar, V., Lee, W., Lee, V., Kim, J., Frank, M., Finch, P., Barua, R., Babb, J., Amarasinghe, S., Agarwal, A., 1997. Baring it all to software: Raw Machines. IEEE Computer 30 (September), 86–93.
Yang, Y., Mason, G., 1991. Nonblocking broadcast switching networks. IEEE Trans. on Computers 40 (9 (September)), 1005–1015.

Exercises

Solutions to "starred" exercises are available for instructors who register at *textbooks.elsevier.com.*

✪ F.1 [15] <F.2, F.3> Is electronic communication always faster than nonelectronic means for longer distances? Calculate the time to send 1000 GB using 25 8-mm tapes and an overnight delivery service versus sending 1000 GB by FTP over the Internet. Make the following four assumptions:

- The tapes are picked up at 4 P.M. Pacific time and delivered 4200 km away at 10 A.M. Eastern time (7 A.M. Pacific time).
- On one route the slowest link is a T3 line, which transfers at 45 Mbits/sec.
- On another route the slowest link is a 100-Mbit/sec Ethernet.
- You can use 50% of the slowest link between the two sites.

Will all the bytes sent by either Internet route arrive before the overnight delivery person arrives?

✪ F.2 [10] <F.2, F.3> For the same assumptions as Exercise F.1, what is the bandwidth of overnight delivery for a 1000-GB package?

✪ F.3 [10] <F.2, F.3> For the same assumptions as Exercise F.1, what is the minimum bandwidth of the slowest link to beat overnight delivery? What standard network options match that speed?

✪ F.4 [15] <F.2, F.3> The original Ethernet standard was for 10 Mbits/sec and a maximum distance of 2.5 km. How many bytes could be in flight in the original Ethernet? Assume you can use 90% of the peak bandwidth.

✪ F.5 [15] <F.2, F.3> Flow control is a problem for WANs due to the long time of flight, as the example on page F-14 illustrates. Ethernet did not include flow control when it was first standardized at 10 Mbits/sec. Calculate the number of bytes in flight for a 10-Gbit/sec Ethernet over a 100 meter link, assuming you can use 90% of peak bandwidth. What does your answer mean for network designers?

✪ F.6 [15] <F.2, F.3> Assume the total overhead to send a zero-length data packet on an Ethernet is 100 μs and that an unloaded network can transmit at 90% of the peak 1000-Mbit/sec rating. For the purposes of this question, assume that the size of the Ethernet header and trailer is 56 bytes. Assume a continuous stream of packets of the same size. Plot the delivered bandwidth of user data in Mbits/sec as the payload data size varies from 32 bytes to the maximum size of 1500 bytes in 32-byte increments.

✪ F.7 [10] <F.2, F.3> Exercise F.6 suggests that the delivered Ethernet bandwidth to a single user may be disappointing. Making the same assumptions as in that exercise, by how much would the maximum payload size have to be increased to deliver half of the peak bandwidth?

✪ F.8 [10] <F.2, F.3> One reason that ATM has a fixed transfer size is that when a short message is behind a long message, a node may need to wait for an entire transfer to complete. For applications that are time sensitive, such as when transmitting voice or video, the large transfer size may result in transmission delays that are too long for the application. On an unloaded interconnection, what is the worstcase delay in microseconds if a node must wait for one full-size Ethernet packet versus an ATM transfer? See Figure F.30 (page F-78) to find the packet sizes. For this question assume that you can transmit at 100% of the 622-Mbits/sec ATM network and 100% of the 1000-Mbit/sec Ethernet.

✪ F.9 [10] <F.2, F.3> Exercise F.7 suggests the need for expanding the maximum pay-load to increase the delivered bandwidth, but Exercise F.8 suggests the impact on worst-case latency of making it longer. What would be the impact on latency of increasing the maximum payload size by the answer to Exercise F.7?

✪ F.10 [12/12/20] <F.4> The Omega network shown in Figure F.11 on page F-31 consists of three columns of four switches, each with two inputs and two outputs. Each switch can be set to *straight*, which connects the upper switch input to the upper switch output and the lower input to the lower output, and to *exchange,* which connects the upper input to the lower output and *vice versa* for the lower input. For each column of switches, label the inputs and outputs 0, 1, …, 7 from top to bottom, to correspond with the numbering of the processors.

a. [12] <F.4> When a switch is set to exchange and a message passes through, what is the relationship between the label values for the switch input and output used by the message? (*Hint*: Think in terms of operations on the digits of the binary representation of the label number.)

b. [12] <F.4> Between any two switches in adjacent columns that are connected by a link, what is the relationship between the label of the output connected to the input?

c. [20] <F.4> Based on your results in parts (a) and (b), design and describe a simple routing scheme for distributed control of the Omega network. A message will carry a *routing tag* computed by the sending processor. Describe how the processor computes the tag and how each switch can set itself by examining a bit of the routing tag.

✪ F.11 [12/12/12/12/12/12] <F.4> Prove whether or not it is possible to realize the following permutations (i.e., communication patterns) on the eight-node Omega network shown in Figure F.11 on page F-31:

a. [12] <F.4> Bit-reversal permutation—the node with binary coordinates a_{n-1}, a_{n-2}, …, a_1, a_0 communicates with the node a_0, a_1, …, a_{n-2}, a_{n-1}.

b. [12] <F.4> Perfect shuffle permutation—the node with binary coordinates a_{n-1}, a_{n-2}, …, a_1, a_0 communicates with the node a_{n-2}, a_{n-3}, …, a_0, a_{n-1} (i.e., rotate left 1 bit).

c. [12] <F.4> Bit-complement permutation—the node with binary coordinates a_{n-1}, a_{n-2}, …, a_1, a_0 communicates with the node $\overline{a}_{n-1}$, $\overline{a}_{n-2}$, …, $\overline{a}_1$, $\overline{a}_0$ (i.e., complement each bit).

d. [12] <F.4> Butterfly permutation—the node with binary coordinates a_{n-1}, a_{n-2}, …, a_1, a_0 communicates with the node a_0, a_{n-2}, …, a_1, a_{n-1} (i.e., swap the most and least significant bits).

e. [12] <F.4> Matrix transpose permutation—the node with binary coordinates a_{n-1}, a_{n-2}, …, a_1, a_0 communicates with the node $a_{n/2-1}$, …, a_0, a_{n-1}, …, $a_{n/2}$ (i.e., transpose the bits in positions approximately halfway around).

f. [12] <F.4> Barrel-shift permutation—node i communicates with node $i+1$ modulo $N-1$, where N is the total number of nodes and $0 \leq i$.

✪ F.12 [12] <F.4> Design a network topology using 18-port crossbar switches that has the minimum number of switches to connect 64 nodes. Each switch port supports communication to and from one device.

✪ F.13 [15] <F.4> Design a network topology that has the minimum latency through the switches for 64 nodes using 18-port crossbar switches. Assume unit delay in the switches and zero delay for wires.

✪ F.14 [15] <F.4> Design a switch topology that balances the bandwidth required for all links for 64 nodes using 18-port crossbar switches. Assume a uniform traffic pattern.

✪ F.15 [15] <F.4> Compare the interconnection latency of a crossbar, Omega network, and fat tree with eight nodes. Use Figure F.11 on page F-31, Figure F.12 on page F-33, and Figure F.14 on page F-37. Assume that the fat tree is built entirely from two-input, two-output switches so that its hardware resources are more comparable to that of the Omega network. Assume that each switch costs a unit time delay. Assume that the fat tree randomly picks a path, so give the best case and worst case for each example. How long will it take to send a message from node 0 to node 6? How long will it take node 1 and node 7 to communicate?

✪ F.16 [15] <F.4> Draw the topology of a 6-cube after the same manner of the 4-cube in Figure F.14 on page F-37. What is the maximum and average number of hops needed by packets assuming a uniform distribution of packet destinations?

✪ F.17 [15] <F.4> Complete a table similar to Figure F.15 on page F-40 that captures the performance and cost of various network topologies, but do it for the general case of N nodes using $k \times k$ switches instead of the specific case of 64 nodes.

✪ F.18 [20] <F.4> Repeat the example given on page F-41, but use the bit-complement communication pattern given in Exercise F.11 instead of NEWS communication.

✪ F.19 [15] <F.5> Give the four specific conditions necessary for deadlock to exist in an interconnection network. Which of these are removed by dimension-order routing? Which of these are removed in adaptive routing with the use of "escape" routing paths? Which of these are removed in adaptive routing with the technique of deadlock recovery (regressive or progressive)? Explain your answer.

✪ F.20 [12/12/12/12] <F.5> Prove whether or not the following routing algorithms based on prohibiting dimensional turns are suitable to be used as escape paths for 2D meshes by analyzing whether they are both connected and deadlock-free. Explain your answer. (*Hint*: You may wish to refer to the Turn Model algorithm and/or to prove your answer by drawing a directed graph for a 4×4 mesh that depicts dependencies between channels and verifying the channel dependency graph is free of cycles.) The routing algorithms are expressed with the following abbreviations: W = west, E = east, N = north, and S = south.

a. [12] <F.5> Allowed turns are from W to N, E to N, S to W, and S to E.

b. [12] <F.5> Allowed turns are from W to S, E to S, N to E, and S to E.

c. [12] <F.5> Allowed turns are from W to S, E to S, N to W, S to E, W to N, and S to W.

d. [12] <F.5> Allowed turns are from S to E, E to S, S to W, N to W, N to E, and E to N.

✪ F.21 [15] <F.5> Compute and compare the upper bound for the efficiency factor, ρ, for dimension-order routing and up*/down* routing assuming uniformly distributed traffic on a 64-node 2D mesh network. For up*/down* routing, assume optimal placement of the root node (i.e., a node near the middle of the mesh). (*Hint*: You will have to find the loading of links across the network bisection that carries the global load as determined by the routing algorithm.)

✪ F.22 [15] <F.5> For the same assumptions as Exercise F.21, find the efficiency factor for up*/down* routing on a 64-node fat tree network using 4×4 switches. Compare this result with the ρ found for up*/down* routing on a 2D mesh. Explain.

✪ F.23 [15] <F.5> Calculate the probability of matching two-phased arbitration requests from all k input ports of a switch simultaneously to the k output ports assuming a uniform distribution of requests and grants to/from output ports. How does this compare to the matching probability for three-phased arbitration in which each of the k input ports can make two simultaneous requests (again, assuming a uniform random distribution of requests and grants)?

✪ F.24 [15] <F.5> The equation on page F-52 shows the value of cut-through switching. Ethernet switches used to build clusters often do not support cut-through switching. Compare the time to transfer 1500 bytes over a 1000-Mbit/sec Ethernet with and without cut-through switching for a 64-node cluster. Assume that each Ethernet switch takes 1.0 μs and that a message goes through seven intermediate switches.

✪ F.25 [15] <F.5> Making the same assumptions as in Exercise F.24, what is the difference between cut-through and store-and-forward switching for 32 bytes?

✪ F.26 [15] <F.5> One way to reduce latency is to use larger switches. Unlike Exercise F.24, let's assume we need only three intermediate switches to connect any two nodes in the cluster. Make the same assumptions as in Exercise F.24 for the remaining parameters. What is the difference between cut-through and store-and-forward switching for 1500 bytes? For 32 bytes?

✪ F.27 [20] <F.5> Using FlexSim 1.2 (*http://ceng.usc.edu/smart/FlexSim/flexsim.html*) or some other cycle-accurate network simulator, simulate a 256-node 2D torus network assuming wormhole routing, 32-flit packets, uniform (random) communication pattern, and four virtual channels. Compare the performance of deterministic routing using DOR, adaptive routing using escape paths (i.e., Duato's Protocol), and true fully adaptive routing using progressive deadlock recovery (i.e., Disha routing). Do so by plotting latency versus applied load and through-put versus applied load for each, as is done in Figure F.19 for the example on page F-53. Also run simulations and plot results for two and eight virtual channels for each. Compare and explain your results by addressing how/why the number and use of virtual channels by the various routing algorithms affect network performance. (*Hint*: Be sure to let the simulation reach steady state by allowing a warm-up period of a several thousand network cycles before gathering results.)

✪ F.28 [20] <F.5> Repeat Exercise F.27 using bit-reversal communication instead of the uniform random communication pattern. Compare and explain your results by addressing how/why the communication pattern affects network performance.

✪ F.29 [40] <F.5> Repeat Exercises F.27 and F.28 using 16-flit packets and 128-flit packets. Compare and explain your results by addressing how/why the packet size along with the other design parameters affect network performance.

F.30 [20] <F.2, F.4, F.5, F.8> Figures F.7, F.16, and F.20 show interconnection network characteristics of several of the top 500 supercomputers by machine type

as of the publication of the fourth edition. Update that figure to the most recent top 500. How have the systems and their networks changed since the data in the original figure? Do similar comparisons for OCNs used in microprocessors and SANs targeted for clusters using Figures F.29 and F.31.

✪ F.31 [12/12/12/15/15/18] <F.8> Use the M/M/1 queuing model to answer this exercise. Measurements of a network bridge show that packets arrive at 200 packets per second and that the gateway forwards them in about 2 ms.

a. [12] <F.8> What is the utilization of the gateway?

b. [12] <F.8> What is the mean number of packets in the gateway?

c. [12] <F.8> What is the mean time spent in the gateway?

d. [15] <F.8> Plot response time versus utilization as you vary the arrival rate.

e. [15] <F.8> For an M/M/1 queue, the probability of finding n or more tasks in the system is Utilization^n. What is the chance of an overflow of the FIFO if it can hold 10 messages?

f. [18] <F.8> How big must the gateway be to have packet loss due to FIFO overflow less than one packet per million?

✪ F.32 [20] <F.8> The imbalance between the time of sending and receiving can cause problems in network performance. Sending too fast can cause the network to back up and increase the latency of messages, since the receivers will not be able to pull out the message fast enough. A technique called *bandwidth matching* proposes a simple solution: Slow down the sender so that it matches the performance of the receiver [Brewer and Kuszmaul 1994]. If two machines exchange an equal number of messages using a protocol like UDP, one will get ahead of the other, causing it to send all its messages first. After the receiver puts all these messages away, it will then send its messages. Estimate the performance for this case versus a bandwidth-matched case. Assume that the send overhead is 200 μs, the receive overhead is 300 μs, time of flight is 5 μs, latency is 10 μs, and that the two machines want to exchange 100 messages.

F.33 [40] <F.8> Compare the performance of UDP with and without bandwidth matching by slowing down the UDP send code to match the receive code as advised by bandwidth matching [Brewer and Kuszmaul 1994]. Devise an experiment to see how much performance changes as a result. How should you change the send rate when two nodes send to the same destination? What if one sender sends to two destinations?

✪ F.34 [40] <F.6, F.8> If you have access to an SMP and a cluster, write a program to measure latency of communication and bandwidth of communication between processors, as was plotted in Figure F.32 on page F-80.

F.35 [20/20/20] <F.9> If you have access to a UNIX system, use `ping` to explore the Internet. First read the manual page. Then use `ping` without option flags to be sure you can reach the following sites. It should say that `X is alive`. Depending on your system, you may be able to see the path by setting the flags to verbose mode

(`-v`) and trace route mode (`-R`) to see the path between your machine and the example machine. Alternatively, you may need to use the program `traceroute` to see the path. If so, try its manual page. You may want to use the UNIX command `script` to make a record of your session.

a. [20] <F.9> Trace the route to another machine on the same local area network. What is the latency?

b. [20] <F.9> Trace the route to another machine on your campus that is *not* on the same local area network.What is the latency?

c. [20] <F.9> Trace the route to another machine *off campus*. For example, if you have a friend you send email to, try tracing that route. See if you can discover what types of networks are used along that route.What is the latency?

F.36 [15] <F.9> Use FTP to transfer a file from a remote site and then between local sites on the same LAN. What is the difference in bandwidth for each transfer? Try the transfer at different times of day or days of the week. Is the WAN or LAN the bottleneck?

✪ F.37 [10/10] <F.9, F.11> Figure F.41 on page F-93 compares latencies for a high-bandwidth network with high overhead and a low-bandwidth network with low overhead for different TCP/IP message sizes.

a. [10] <F.9, F.11> For what message sizes is the delivered bandwidth higher for the high-bandwidth network?

b. [10] <F.9, F.11> For your answer to part (a), what is the delivered bandwidth for each network?

✪ F.38 [15] <F.9, F.11> Using the statistics in Figure F.41 on page F-93, estimate the per-message overhead for each network.

✪ F.39 [15] <F.9, F.11> Exercise F.37 calculates which message sizes are faster for two networks with different overhead and peak bandwidth. Using the statistics in Figure F.41 on page F-93, what is the percentage of messages that are transmitted more quickly on the network with low overhead and bandwidth? What is the percentage of data transmitted more quickly on the network with high overhead and bandwidth?

✪ F.40 [15] <F.9, F.11> One interesting measure of the latency and bandwidth of an inter-connection is to calculate the size of a message needed to achieve one-half of the peak bandwidth. This halfway point is sometimes referred to as $n_{1/2}$, taken from the terminology of vector processing. Using Figure F.41 on page F-93, estimate $n_{1/2}$ for TCP/IP message using 155-Mbit/sec ATM and 10-Mbit/sec Ethernet.

F.41 [Discussion] <F.10> The Google cluster used to be constructed from 1 rack unit (RU) PCs, each with one processor and two disks. Today there are considerably denser options. How much less floor space would it take if we were to replace the 1 RU PCs with modern alternatives? Go to the Compaq or Dell Web sites to find the densest alternative. What would be the estimated impact on cost of the equipment? What would be the estimated impact on rental cost of floor space?

What would be the impact on interconnection network design for achieving power/performance efficiency?

F.42 [Discussion] < F.13 > At the time of the writing of the fourth edition, it was unclear what would happen with Ethernet versus InfiniBand versus Advanced Switching in the machine room. What are the technical advantages of each? What are the economic advantages of each? Why would people maintaining the system prefer one to the other? How popular is each network today? How do they compare to proprietary commercial networks such as Myrinet and Quadrics?

G.1	Introduction	G-2
G.2	Vector Performance in More Depth	G-2
G.3	Vector Memory Systems in More Depth	G-9
G.4	Enhancing Vector Performance	G-11
G.5	Effectiveness of Compiler Vectorization	G-14
G.6	Putting It All Together: Performance of Vector Processors	G-15
G.7	A Modern Vector Supercomputer: The Cray X1	G-21
G.8	Concluding Remarks	G-25
G.9	Historical Perspective and References	G-26
	Exercises	G-29

G

Vector Processors in More Depth

Revised by Krste Asanovic
Massachusetts Institute of Technology

I'm certainly not inventing vector processors. There are three kinds that I know of existing today. They are represented by the Illiac-IV, the (CDC) Star processor, and the TI (ASC) processor. Those three were all pioneering processors.…One of the problems of being a pioneer is you always make mistakes and I never, never want to be a pioneer. It's always best to come second when you can look at the mistakes the pioneers made.

Seymour Cray
Public lecture at Lawrence Livermore Laboratorieson on the introduction of the Cray-1 (1976)

G.1 Introduction

Chapter 4 introduces vector architectures and places Multimedia SIMD extensions and GPUs in proper context to vector architectures.

In this appendix, we go into more detail on vector architectures, including more accurate performance models and descriptions of previous vector architectures. Figure G.1 shows the characteristics of some typical vector processors, including the size and count of the registers, the number and types of functional units, the number of load-store units, and the number of lanes.

G.2 Vector Performance in More Depth

The chime approximation is reasonably accurate for long vectors. Another source of overhead is far more significant than the issue limitation.

The most important source of overhead ignored by the chime model is vector *start-up time*. The start-up time comes from the pipelining latency of the vector operation and is principally determined by how deep the pipeline is for the functional unit used. The start-up time increases the effective time to execute a convoy to more than one chime. Because of our assumption that convoys do not overlap in time, the start-up time delays the execution of subsequent convoys. Of course, the instructions in successive convoys either have structural conflicts for some functional unit or are data dependent, so the assumption of no overlap is reasonable. The actual time to complete a convoy is determined by the sum of the vector length and the start-up time. If vector lengths were infinite, this start-up overhead would be amortized, but finite vector lengths expose it, as the following example shows.

Example Assume that the start-up overhead for functional units is shown in Figure G.2.

Show the time that each convoy can begin and the total number of cycles needed. How does the time compare to the chime approximation for a vector of length 64?

Answer Figure G.3 provides the answer in convoys, assuming that the vector length is n. One tricky question is when we assume the vector sequence is done; this determines whether the start-up time of the SV is visible or not. We assume that the instructions following cannot fit in the same convoy, and we have already assumed that convoys do not overlap. Thus, the total time is given by the time until the last vector instruction in the last convoy completes. This is an approximation, and the start-up time of the last vector instruction may be seen in some sequences and not in others. For simplicity, we always include it.

The time per result for a vector of length 64 is $4 + (42/64) = 4.65$ clock cycles, while the chime approximation would be 4. The execution time with startup overhead is 1.16 times higher.

Processor (year)	Vector clock rate (MHz)	Vector registers	Elements per register (64-bit elements)	Vector arithmetic units	Vector load-store units	Lanes
Cray-1 (1976)	80	8	64	6: FP add, FP multiply, FP reciprocal, integer add, logical, shift	1	1
Cray X-MP (1983) Cray Y-MP (1988)	118 166	8	64	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	1
Cray-2 (1985)	244	8	64	5: FP add, FP multiply, FP reciprocal/sqrt, integer add/shift/population count, logical	1	1
Fujitsu VP100/ VP200 (1982)	133	8–256	32–1024	3: FP or integer add/logical, multiply, divide	2	1 (VP100) 2 (VP200)
Hitachi S810/S820 (1983)	71	32	256	4: FP multiply-add, FP multiply/divide-add unit, 2 integer add/logical	3 loads 1 store	1 (S810) 2 (S820)
Convex C-1 (1985)	10	8	128	2: FP or integer multiply/divide, add/logical	1	1 (64 bit) 2 (32 bit)
NEC SX/2 (1985)	167	8+32	256	4: FP multiply/divide, FP add, integer add/ logical, shift	1	4
Cray C90 (1991) Cray T90 (1995)	240 460	8	128	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	2
NEC SX/5 (1998)	312	8+64	512	4: FP or integer add/shift, multiply, divide, logical	1	16
Fujitsu VPP5000 (1999)	300	8–256	128–4096	3: FP or integer multiply, add/logical, divide	1 load 1 store	16
Cray SV1 (1998) SV1ex (2001)	300 500	8	64 (MSP)	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	1 load-store 1 load	2 8 (MSP)
VMIPS (2001)	500	8	64	5: FP multiply, FP divide, FP add, integer add/ shift, logical	1 load-store	1
NEC SX/6 (2001)	500	8+64	256	4: FP or integer add/shift, multiply, divide, logical	1	8
NEC SX/8 (2004)	2000	8+64	256	4: FP or integer add/shift, multiply, divide, logical	1	4
Cray X1 (2002) Cray XIE (2005)	800 1130	32	64 256 (MSP)	3: FP or integer, add/logical, multiply/shift, divide/square root/logical	1 load 1 store	2 8 (MSP)

Figure G.1 Characteristics of several vector-register architectures. If the machine is a multiprocessor, the entries correspond to the characteristics of one processor. Several of the machines have different clock rates in the vector and scalar units; the clock rates shown are for the vector units. The Fujitsu machines' vector registers are configurable: The size and count of the 8K 64-bit entries may be varied inversely to one another (e.g., on the VP200, from eight registers each 1K elements long to 256 registers each 32 elements long). The NEC machines have eight foreground vector registers connected to the arithmetic units plus 32 to 64 background vector registers connected between the memory system and the foreground vector registers. Add pipelines perform add and subtract. The multiply/divide-add unit on the Hitachi S810/820 performs an FP multiply or divide followed by an add or subtract (while the multiply-add unit performs a multiply followed by an add or subtract). Note that most processors use the vector FP multiply and divide units for vector integer multiply and divide, and several of the processors use the same units for FP scalar and FP vector operations. Each vector load-store unit represents the ability to do an independent, overlapped transfer to or from the vector registers. The number of lanes is the number of parallel pipelines in each of the functional units as described in Section G.4. For example, the NEC SX/5 can complete 16 multiplies per cycle in the multiply functional unit. Several machines can split a 64-bit lane into two 32-bit lanes to increase performance for applications that require only reduced precision. The Cray SV1 and Cray X1 can group four CPUs with two lanes each to act in unison as a single larger CPU with eight lanes, which Cray calls a Multi-Streaming Processor (MSP).

Unit	Start-up overhead (cycles)
Load and store unit	12
Multiply unit	7
Add unit	6

Figure G.2 Start-up overhead.

Convoy	Starting time	First-result time	Last-result time
1. LV	0	12	$11+n$
2. MULVS.D LV	$12+n$	$12+n+12$	$23+2n$
3. ADDV.D	$24+2n$	$24+2n+6$	$29+3n$
4. SV	$30+3n$	$30+3n+12$	$41+4n$

Figure G.3 Starting times and first- and last-result times for convoys 1 through 4. The vector length is n.

For simplicity, we will use the chime approximation for running time, incorporating start-up time effects only when we want performance that is more detailed or to illustrate the benefits of some enhancement. For long vectors, a typical situation, the overhead effect is not that large. Later in the appendix, we will explore ways to reduce start-up overhead.

Start-up time for an instruction comes from the pipeline depth for the functional unit implementing that instruction. If the initiation rate is to be kept at 1 clock cycle per result, then

$$\text{Pipeline depth} = \left\lceil \frac{\text{Total functional unit time}}{\text{Clock cycle time}} \right\rceil$$

For example, if an operation takes 10 clock cycles, it must be pipelined 10 deep to achieve an initiation rate of one per clock cycle. Pipeline depth, then, is determined by the complexity of the operation and the clock cycle time of the processor. The pipeline depths of functional units vary widely—2 to 20 stages are common—although the most heavily used units have pipeline depths of 4 to 8 clock cycles.

For VMIPS, we will use the same pipeline depths as the Cray-1, although latencies in more modern processors have tended to increase, especially for loads. All functional units are fully pipelined. From Chapter 4, pipeline depths are 6 clock cycles for floating-point add and 7 clock cycles for floating-point multiply. On VMIPS, as on most vector processors, independent vector operations using different functional units can issue in the same convoy.

In addition to the start-up overhead, we need to account for the overhead of executing the strip-mined loop. This strip-mining overhead, which arises from

Operation	Start-up penalty
Vector add	6
Vector multiply	7
Vector divide	20
Vector load	12

Figure G.4 Start-up penalties on VMIPS. These are the start-up penalties in clock cycles for VMIPS vector operations.

the need to reinitiate the vector sequence and set the Vector Length Register (VLR) effectively adds to the vector start-up time, assuming that a convoy does not overlap with other instructions. If that overhead for a convoy is 10 cycles, then the effective overhead per 64 elements increases by 10 cycles, or 0.15 cycles per element.

Two key factors contribute to the running time of a strip-mined loop consisting of a sequence of convoys:

1. The number of convoys in the loop, which determines the number of chimes. We use the notation T_{chime} for the execution time in chimes.
2. The overhead for each strip-mined sequence of convoys. This overhead consists of the cost of executing the scalar code for strip-mining each block, T_{loop}, plus the vector start-up cost for each convoy, T_{start}.

There may also be a fixed overhead associated with setting up the vector sequence the first time. In recent vector processors, this overhead has become quite small, so we ignore it.

The components can be used to state the total running time for a vector sequence operating on a vector of length n, which we will call T_n:

$$T_n = \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + n \times T_{chime}$$

The values of T_{start}, T_{loop}, and T_{chime} are compiler and processor dependent. The register allocation and scheduling of the instructions affect both what goes in a convoy and the start-up overhead of each convoy.

For simplicity, we will use a constant value for T_{loop} on VMIPS. Based on a variety of measurements of Cray-1 vector execution, the value chosen is 15 for T_{loop}. At first glance, you might think that this value is too small. The overhead in each loop requires setting up the vector starting addresses and the strides, incrementing counters, and executing a loop branch. In practice, these scalar instructions can be totally or partially overlapped with the vector instructions, minimizing the time spent on these overhead functions. The value of T_{loop} of course depends on the loop structure, but the dependence is slight compared with the connection between the vector code and the values of T_{chime} and T_{start}.

Example What is the execution time on VMIPS for the vector operation $A = B \times s$, where s is a scalar and the length of the vectors A and B is 200?

Answer Assume that the addresses of A and B are initially in `Ra` and `Rb`, `s` is in `Fs`, and recall that for MIPS (and VMIPS) R0 always holds 0. Since (200 mod 64) = 8, the first iteration of the strip-mined loop will execute for a vector length of 8 elements, and the following iterations will execute for a vector length of 64 elements. The starting byte addresses of the next segment of each vector is eight times the vector length. Since the vector length is either 8 or 64, we increment the address registers by $8 \times 8 = 64$ after the first segment and $8 \times 64 = 512$ for later segments. The total number of bytes in the vector is $8 \times 200 = 1600$, and we test for completion by comparing the address of the next vector segment to the initial address plus 1600. Here is the actual code:

```
      DADDUI   R2,R0,#1600 ;total # bytes in vector
      DADDU    R2,R2,Ra    ;address of the end of A vector
      DADDUI   R1,R0,#8    ;loads length of 1st segment
      MTC1     VLR,R1      ;load vector length in VLR
      DADDUI   R1,R0,#64   ;length in bytes of 1st segment
      DADDUI   R3,R0,#64   ;vector length of other segments
Loop: LV       V1,Rb       ;load B
      MULVS.D  V2,V1,Fs    ;vector * scalar
      SV       Ra,V2       ;store A
      DADDU    Ra,Ra,R1    ;address of next segment of A
      DADDU    Rb,Rb,R1    ;address of next segment of B
      DADDUI   R1,R0,#512  ;load byte offset next segment
      MTC1     VLR,R3      ;set length to 64 elements
      DSUBU    R4,R2,Ra    ;at the end of A?
      BNEZ     R4,Loop     ;if not, go back
```

The three vector instructions in the loop are dependent and must go into three convoys, hence $T_{chime} = 3$. Let's use our basic formula:

$$T_n = \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + n \times T_{chime}$$

$$T_{200} = 4 \times (15 + T_{start}) + 200 \times 3$$

$$T_{200} = 60 + (4 \times T_{start}) + 600 = 660 + (4 \times T_{start})$$

The value of T_{start} is the sum of:

- The vector load start-up of 12 clock cycles
- A 7-clock-cycle start-up for the multiply
- A 12-clock-cycle start-up for the store

Thus, the value of T_{start} is given by:

$$T_{start} = 12 + 7 + 12 = 31$$

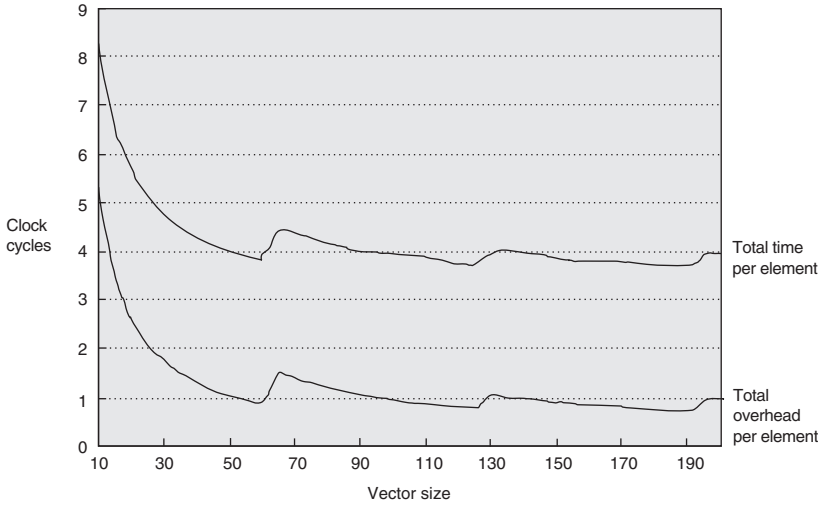


Figure G.5 The total execution time per element and the total overhead time per element versus the vector length for the example on page F-6. For short vectors, the total start-up time is more than one-half of the total time, while for long vectors it reduces to about one-third of the total time. The sudden jumps occur when the vector length crosses a multiple of 64, forcing another iteration of the strip-mining code and execution of a set of vector instructions. These operations increase T_n by $T_{loop} + T_{start}$.

So, the overall value becomes:

$$T_{200} = 660 + 4 \times 31 = 784$$

The execution time per element with all start-up costs is then $784/200 = 3.9$, compared with a chime approximation of three. In Section G.4, we will be more ambitious—allowing overlapping of separate convoys.

Figure G.5 shows the overhead and effective rates per element for the previous example ($A = B \times s$) with various vector lengths. A chime-counting model would lead to 3 clock cycles per element, while the two sources of overhead add 0.9 clock cycles per element in the limit.

Pipelined Instruction Start-Up and Multiple Lanes

Adding multiple lanes increases peak performance but does not change start-up latency, and so it becomes critical to reduce start-up overhead by allowing the start of one vector instruction to be overlapped with the completion of preceding vector instructions. The simplest case to consider is when two vector instructions access a different set of vector registers. For example, in the code sequence

```
ADDV.D V1,V2,V3
ADDV.D V4,V5,V6
```

An implementation can allow the first element of the second vector instruction to follow immediately the last element of the first vector instruction down the FP adder pipeline. To reduce the complexity of control logic, some vector machines require some *recovery time* or *dead time* in between two vector instructions dispatched to the same vector unit. Figure G.6 is a pipeline diagram that shows both start-up latency and dead time for a single vector pipeline.

The following example illustrates the impact of this dead time on achievable vector performance.

Example The Cray C90 has two lanes but requires 4 clock cycles of dead time between any two vector instructions to the same functional unit, even if they have no data dependences. For the maximum vector length of 128 elements, what is the reduction in achievable peak performance caused by the dead time? What would be the reduction if the number of lanes were increased to 16?

Answer A maximum length vector of 128 elements is divided over the two lanes and occupies a vector functional unit for 64 clock cycles. The dead time adds another 4 cycles of occupancy, reducing the peak performance to $64/(64+4) = 94.1\%$ of the value without dead time. If the number of lanes is increased to 16, maximum length vector instructions will occupy a functional unit for only $128/16 = 8$ cycles, and the dead time will reduce peak performance to $8/(8+4) = 66.6\%$ of the value without dead time. In this second case, the vector units can never be more than 2/3 busy!

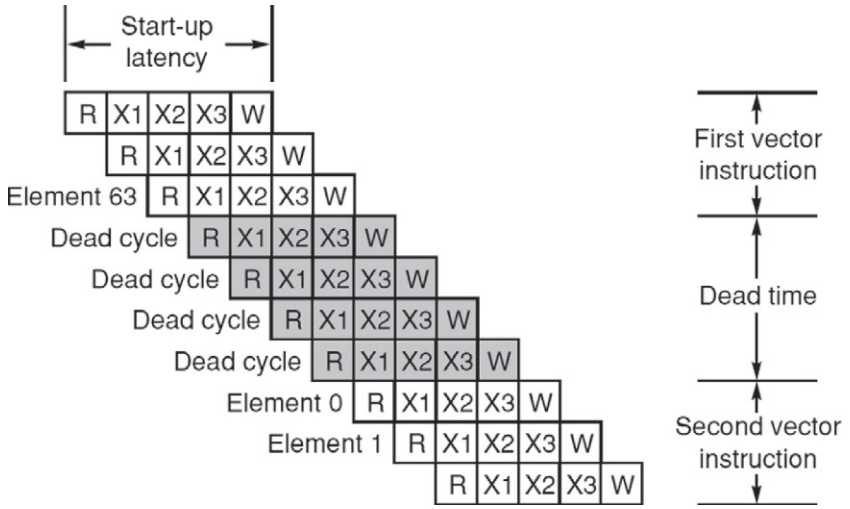


Figure G.6 Start-up latency and dead time for a single vector pipeline. Each element has a 5-cycle latency: 1 cycle to read the vector-register file, 3 cycles in execution, then 1 cycle to write the vector-register file. Elements from the same vector instruction can follow each other down the pipeline, but this machine inserts 4 cycles of dead time between two different vector instructions. The dead time can be eliminated with more complex control logic. (Reproduced with permission from Asanovic [1998].)

Pipelining instruction start-up becomes more complicated when multiple instructions can be reading and writing the same vector register and when some instructions may stall unpredictably—for example, a vector load encountering memory bank conflicts. However, as both the number of lanes and pipeline latencies increase, it becomes increasingly important to allow fully pipelined instruction start-up.

G.3 Vector Memory Systems in More Depth

To maintain an initiation rate of one word fetched or stored per clock, the memory system must be capable of producing or accepting this much data. As we saw in Chapter 4, this usually done by spreading accesses across multiple independent memory banks. Having significant numbers of banks is useful for dealing with vector loads or stores that access rows or columns of data.

The desired access rate and the bank access time determined how many banks were needed to access memory without stalls. This example shows how these timings work out in a vector processor.

Example Suppose we want to fetch a vector of 64 elements starting at byte address 136, and a memory access takes 6 clocks. How many memory banks must we have to support one fetch per clock cycle? With what addresses are the banks accessed? When will the various elements arrive at the CPU?

Answer Six clocks per access require at least 6 banks, but because we want the number of banks to be a power of 2, we choose to have 8 banks. Figure G.7 shows the timing for the first few sets of accesses for an 8-bank system with a 6-clock-cycle access latency.

The timing of real memory banks is usually split into two different components, the access latency and the bank cycle time (or *bank busy time*). The access latency is the time from when the address arrives at the bank until the bank returns a data value, while the busy time is the time the bank is occupied with one request. The access latency adds to the start-up cost of fetching a vector from memory (the total memory latency also includes time to traverse the pipelined interconnection networks that transfer addresses and data between the CPU and memory banks). The bank busy time governs the effective bandwidth of a memory system because a processor cannot issue a second request to the same bank until the bank busy time has elapsed.

For simple unpipelined SRAM banks as used in the previous examples, the access latency and busy time are approximately the same. For a pipelined SRAM bank, however, the access latency is larger than the busy time because each element access only occupies one stage in the memory bank pipeline. For a DRAM bank, the access latency is usually shorter than the busy time because a DRAM needs extra time to restore the read value after the destructive read operation. For memory systems that support multiple simultaneous vector accesses

	Bank							
Cycle no.	0	1	2	3	4	5	6	7
0		136						
1		Busy	144					
2		Busy	Busy	152				
3		Busy	Busy	Busy	160			
4		Busy	Busy	Busy	Busy	168		
5		Busy	Busy	Busy	Busy	Busy	176	
6			Busy	Busy	Busy	Busy	Busy	184
7	192			Busy	Busy	Busy	Busy	Busy
8	Busy	200			Busy	Busy	Busy	Busy
9	Busy	Busy	208			Busy	Busy	Busy
10	Busy	Busy	Busy	216			Busy	Busy
11	Busy	Busy	Busy	Busy	224			Busy
12	Busy	Busy	Busy	Busy	Busy	232		
13		Busy	Busy	Busy	Busy	Busy	240	
14			Busy	Busy	Busy	Busy	Busy	248
15	256			Busy	Busy	Busy	Busy	Busy
16	Busy	264			Busy	Busy	Busy	Busy

Figure G.7 Memory addresses (in bytes) by bank number and time slot at which access begins. Each memory bank latches the element address at the start of an access and is then busy for 6 clock cycles before returning a value to the CPU. Note that the CPU cannot keep all 8 banks busy all the time because it is limited to supplying one new address and receiving one data item each cycle.

or allow nonsequential accesses in vector loads or stores, the number of memory banks should be larger than the minimum; otherwise, memory bank conflicts will exist.

Memory bank conflicts will not occur within a single vector memory instruction if the stride and number of banks are relatively prime with respect to each other and there are enough banks to avoid conflicts in the unit stride case. When there are no bank conflicts, multiword and unit strides run at the same rates. Increasing the number of memory banks to a number greater than the minimum to prevent stalls with a stride of length 1 will decrease the stall frequency for some other strides. For example, with 64 banks, a stride of 32 will stall on every other access, rather than every access. If we originally had a stride of 8 and 16 banks, every other access would stall; with 64 banks, a stride of 8 will stall on every eighth access. If we have multiple memory pipelines and/or multiple processors sharing the same memory system, we will also need more banks to prevent conflicts. Even machines with a single memory pipeline can experience memory bank conflicts on unit stride

accesses between the last few elements of one instruction and the first few elements of the next instruction, and increasing the number of banks will reduce the probability of these inter-instruction conflicts. In 2011, most vector supercomputers spread the accesses from each CPU across hundreds of memory banks. Because bank conflicts can still occur in non-unit stride cases, programmers favor unit stride accesses whenever possible.

A modern supercomputer may have dozens of CPUs, each with multiple memory pipelines connected to thousands of memory banks. It would be impractical to provide a dedicated path between each memory pipeline and each memory bank, so, typically, a multistage switching network is used to connect memory pipelines to memory banks. Congestion can arise in this switching network as different vector accesses contend for the same circuit paths, causing additional stalls in the memory system.

G.4 Enhancing Vector Performance

In this section, we present techniques for improving the performance of a vector processor in more depth than we did in Chapter 4.

Chaining in More Depth

Early implementations of chaining worked like forwarding, but this restricted the timing of the source and destination instructions in the chain. Recent implementations use *flexible chaining*, which allows a vector instruction to chain to essentially any other active vector instruction, assuming that no structural hazard is generated. Flexible chaining requires simultaneous access to the same vector register by different vector instructions, which can be implemented either by adding more read and write ports or by organizing the vector-register file storage into interleaved banks in a similar way to the memory system. We assume this type of chaining throughout the rest of this appendix.

Even though a pair of operations depends on one another, chaining allows the operations to proceed in parallel on separate elements of the vector. This permits the operations to be scheduled in the same convoy and reduces the number of chimes required. For the previous sequence, a sustained rate (ignoring start-up) of two floating-point operations per clock cycle, or one chime, can be achieved, even though the operations are dependent! The total running time for the above sequence becomes:

$$\text{Vector length} + \text{Start-up time}_{\text{ADDV}} + \text{Start-up time}_{\text{MULV}}$$

Figure G.8 shows the timing of a chained and an unchained version of the above pair of vector instructions with a vector length of 64. This convoy requires one chime; however, because it uses chaining, the start-up overhead will be seen in the actual timing of the convoy. In Figure G.8, the total time for chained operation is 77 clock cycles, or 1.2 cycles per result. With 128 floating-point operations done in that time, 1.7 FLOPS per clock cycle are obtained. For the unchained version, there are 141 clock cycles, or 0.9 FLOPS per clock cycle.

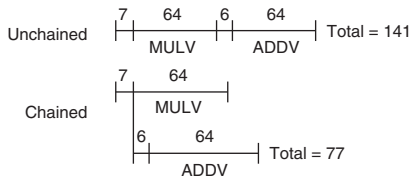


Figure G.8 Timings for a sequence of dependent vector operations `ADDV` and `MULV`, both unchained and chained. The 6- and 7-clock-cycle delays are the latency of the adder and multiplier.

Although chaining allows us to reduce the chime component of the execution time by putting two dependent instructions in the same convoy, it does not eliminate the start-up overhead. If we want an accurate running time estimate, we must count the start-up time both within and across convoys. With chaining, the number of chimes for a sequence is determined by the number of different vector functional units available in the processor and the number required by the application. In particular, no convoy can contain a structural hazard. This means, for example, that a sequence containing two vector memory instructions must take at least two convoys, and hence two chimes, on a processor like VMIPS with only one vector load-store unit.

Chaining is so important that every modern vector processor supports flexible chaining.

Sparse Matrices in More Depth

Chapter 4 shows techniques to allow programs with sparse matrices to execute in vector mode. Let's start with a quick review. In a sparse matrix, the elements of a vector are usually stored in some compacted form and then accessed indirectly. Assuming a simplified sparse structure, we might see code that looks like this:

```
          do      100 i = 1,n
100               A(K(i)) = A(K(i)) + C(M(i))
```

This code implements a sparse vector sum on the arrays `A` and `C`, using index vectors `K` and `M` to designate the nonzero elements of `A` and `C`. (`A` and `C` must have the same number of nonzero elements—`n` of them.) Another common representation for sparse matrices uses a bit vector to show which elements exist and a dense vector for the nonzero elements. Often both representations exist in the same program. Sparse matrices are found in many codes, and there are many ways to implement them, depending on the data structure used in the program.

A simple vectorizing compiler could not automatically vectorize the source code above because the compiler would not know that the elements of `K` are distinct values and thus that no dependences exist. Instead, a programmer directive would tell the compiler that it could run the loop in vector mode.

More sophisticated vectorizing compilers can vectorize the loop automatically without programmer annotations by inserting run time checks for data

dependences. These run time checks are implemented with a vectorized software version of the advanced load address table (ALAT) hardware described in Appendix H for the Itanium processor. The associative ALAT hardware is replaced with a software hash table that detects if two element accesses within the same stripmine iteration are to the same address. If no dependences are detected, the stripmine iteration can complete using the maximum vector length. If a dependence is detected, the vector length is reset to a smaller value that avoids all dependency violations, leaving the remaining elements to be handled on the next iteration of the stripmined loop. Although this scheme adds considerable software overhead to the loop, the overhead is mostly vectorized for the common case where there are no dependences; as a result, the loop still runs considerably faster than scalar code (although much slower than if a programmer directive was provided).

A scatter-gather capability is included on many of the recent supercomputers. These operations often run more slowly than strided accesses because they are more complex to implement and are more susceptible to bank conflicts, but they are still much faster than the alternative, which may be a scalar loop. If the sparsity properties of a matrix change, a new index vector must be computed. Many processors provide support for computing the index vector quickly. The `CVI` (create vector index) instruction in VMIPS creates an index vector given a stride (m), where the values in the index vector are 0, m, $2 \times m$, …, $63 \times m$. Some processors provide an instruction to create a compressed index vector whose entries correspond to the positions with a one in the mask register. Other vector architectures provide a method to compress a vector. In VMIPS, we define the `CVI` instruction to always create a compressed index vector using the vector mask. When the vector mask is all ones, a standard index vector will be created.

The indexed loads-stores and the `CVI` instruction provide an alternative method to support conditional vector execution. Let us first recall code from Chapter 4:

```
      low = 1
      VL = (n mod MVL) /*find the odd-size piece*/
      do 1 j = 0,(n/MVL) /*outer loop*/
           do 10 i = low, low + VL - 1 /*runs for length VL*/
                Y(i) = a * X(i) + Y(i) /*main operation*/
10         continue
           low = low + VL /*start of next vector*/
           VL = MVL /*reset the length to max*/
1     continue
```

Here is a vector sequence that implements that loop using CVI:

```
LV        V1,Ra         ;load vector A into V1
L.D       F0,#0         ;load FP zero into F0
SNEVS.D   V1,F0         ;sets the VM to 1 if V1(i)!=F0
CVI       V2,#8         ;generates indices in V2
POP       R1,VM         ;find the number of 1's in VM
MTC1      VLR,R1        ;load vector-length register
CVM                     ;clears the mask
```

```
LVI       V3,(Ra+V2)    ;load the nonzero A elements
LVI       V4,(Rb+V2)    ;load corresponding B elements
SUBV.D    V3,V3,V4      ;do the subtract
SVI       (Ra+V2),V3    ;store A back
```

Whether the implementation using scatter-gather is better than the conditionally executed version depends on the frequency with which the condition holds and the cost of the operations. Ignoring chaining, the running time of the original version is $5n + c_1$. The running time of the second version, using indexed loads and stores with a running time of one element per clock, is $4n + 4fn + c_2$, where f is the fraction of elements for which the condition is true (i.e., A(i) ¦ 0). If we assume that the values of c_1 and c_2 are comparable, or that they are much smaller than n, we can find when this second technique is better.

$$\text{Time}_1 = 5(n)$$
$$\text{Time}_2 = 4n + 4fn$$

We want $\text{Time}_1 > \text{Time}_2$, so

$$5n > 4n + 4fn$$
$$\frac{1}{4} > f$$

That is, the second method is faster if less than one-quarter of the elements are nonzero. In many cases, the frequency of execution is much lower. If the index vector can be reused, or if the number of vector statements within the if statement grows, the advantage of the scatter-gather approach will increase sharply.

G.5 Effectiveness of Compiler Vectorization

Two factors affect the success with which a program can be run in vector mode. The first factor is the structure of the program itself: Do the loops have true data dependences, or can they be restructured so as not to have such dependences? This factor is influenced by the algorithms chosen and, to some extent, by how they are coded. The second factor is the capability of the compiler. While no compiler can vectorize a loop where no parallelism among the loop iterations exists, there is tremendous variation in the ability of compilers to determine whether a loop can be vectorized. The techniques used to vectorize programs are the same as those discussed in Chapter 3 for uncovering ILP; here, we simply review how well these techniques work.

There is tremendous variation in how well different compilers do in vectorizing programs. As a summary of the state of vectorizing compilers, consider the data in Figure G.9, which shows the extent of vectorization for different processors using a test suite of 100 handwritten FORTRAN kernels. The kernels were designed to test vectorization capability and can all be vectorized by hand; we will see several examples of these loops in the exercises.

Processor	Compiler	Completely vectorized	Partially vectorized	Not vectorized
CDC CYBER 205	VAST-2 V2.21	62	5	33
Convex C-series	FC5.0	69	5	26
Cray X-MP	CFT77 V3.0	69	3	28
Cray X-MP	CFT V1.15	50	1	49
Cray-2	CFT2 V3.1a	27	1	72
ETA-10	FTN 77 V1.0	62	7	31
Hitachi S810/820	FORT77/HAP V20-2B	67	4	29
IBM 3090/VF	VS FORTRAN V2.4	52	4	44
NEC SX/2	FORTRAN77 / SX V.040	66	5	29

Figure G.9 Result of applying vectorizing compilers to the 100 FORTRAN test kernels. For each processor we indicate how many loops were completely vectorized, partially vectorized, and unvectorized. These loops were collected by Callahan, Dongarra, and Levine [1988]. Two different compilers for the Cray X-MP show the large dependence on compiler technology.

G.6 Putting It All Together: Performance of Vector Processors

In this section, we look at performance measures for vector processors and what they tell us about the processors. To determine the performance of a processor on a vector problem we must look at the start-up cost and the sustained rate. The simplest and best way to report the performance of a vector processor on a loop is to give the execution time of the vector loop. For vector loops, people often give the MFLOPS (millions of floating-point operations per second) rating rather than execution time. We use the notation R_n for the MFLOPS rating on a vector of length n. Using the measurements T_n (time) or R_n (rate) is equivalent if the number of FLOPS is agreed upon. In any event, either measurement should include the overhead.

In this section, we examine the performance of VMIPS on a DAXPY loop (see Chapter 4) by looking at performance from different viewpoints. We will continue to compute the execution time of a vector loop using the equation developed in Section G.2. At the same time, we will look at different ways to measure performance using the computed time. The constant values for T_{loop} used in this section introduce some small amount of error, which will be ignored.

Measures of Vector Performance

Because vector length is so important in establishing the performance of a processor, length-related measures are often applied in addition to time and MFLOPS. These length-related measures tend to vary dramatically across different processors

and are interesting to compare. (Remember, though, that *time* is always the measure of interest when comparing the relative speed of two processors.) Three of the most important length-related measures are

- R_∞—The MFLOPS rate on an infinite-length vector. Although this measure may be of interest when estimating peak performance, real problems have limited vector lengths, and the overhead penalties encountered in real problems will be larger.
- $N_{1/2}$—The vector length needed to reach one-half of R_∞. This is a good measure of the impact of overhead.
- N_v—The vector length needed to make vector mode faster than scalar mode. This measures both overhead and the speed of scalars relative to vectors.

Let's look at these measures for our DAXPY problem running on VMIPS. When chained, the inner loop of the DAXPY code in convoys looks like Figure G.10 (assuming that `Rx` and `Ry` hold starting addresses).

Recall our performance equation for the execution time of a vector loop with n elements, T_n:

$$T_n = \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + n \times T_{chime}$$

Chaining allows the loop to run in three chimes (and no less, since there is one memory pipeline); thus, $T_{chime} = 3$. If T_{chime} were a complete indication of performance, the loop would run at an MFLOPS rate of $2/3 \times$ clock rate (since there are 2 FLOPS per iteration). Thus, based only on the chime count, a 500 MHz VMIPS would run this loop at 333 MFLOPS assuming no strip-mining or start-up overhead. There are several ways to improve the performance: Add additional vector load-store units, allow convoys to overlap to reduce the impact of start-up overheads, and decrease the number of loads required by vector-register allocation. We will examine the first two extensions in this section. The last optimization is actually used for the Cray-1, VMIPS's cousin, to boost the performance by 50%. Reducing the number of loads requires an interprocedural optimization; we examine this transformation in Exercise G.6. Before we examine the first two extensions, let's see what the real performance, including overhead, is.

`LV V1,Rx`	`MULVS.D V2,V1,F0`	Convoy 1: chained load and multiply
`LV V3,Ry`	`ADDV.D V4,V2,V3`	Convoy 2: second load and add, chained
`SV Ry,V4`		Convoy 3: store the result

Figure G.10 The inner loop of the DAXPY code in chained convoys.

The Peak Performance of VMIPS on DAXPY

First, we should determine what the peak performance, R_∞, really is, since we know it must differ from the ideal 333 MFLOPS rate. For now, we continue to use the simplifying assumption that a convoy cannot start until all the instructions in an earlier convoy have completed; later we will remove this restriction. Using this simplification, the start-up overhead for the vector sequence is simply the sum of the start-up times of the instructions:

$$T_{start} = 12 + 7 + 12 + 6 + 12 = 49$$

Using MVL = 64, $T_{loop} = 15$, $T_{start} = 49$, and $T_{chime} = 3$ in the performance equation, and assuming that n is not an exact multiple of 64, the time for an n-element operation is

$$\begin{aligned} T_n &= \left\lceil \frac{n}{64} \right\rceil \times (15 + 49) + 3n \\ &\le (n + 64) + 3n \\ &= 4n + 64 \end{aligned}$$

The sustained rate is actually over 4 clock cycles per iteration, rather than the theoretical rate of 3 chimes, which ignores overhead. The major part of the difference is the cost of the start-up overhead for each block of 64 elements (49 cycles versus 15 for the loop overhead).

We can now compute R_∞ for a 500 MHz clock as:

$$R_\infty = \lim_{n \to \infty} \left(\frac{\text{Operations per iteration} \times \text{Clock rate}}{\text{Clock cycles per iteration}} \right)$$

The numerator is independent of n, hence

$$R_\infty = \frac{\text{Operations per iteration} \times \text{Clock rate}}{\lim\limits_{n \to \infty} (\text{Clock cycles per iteration})}$$

$$\lim_{n \to \infty} (\text{Clock cycles per iteration}) = \lim_{n \to \infty} \left(\frac{T_n}{n} \right) = \lim_{n \to \infty} \left(\frac{4n + 64}{n} \right) = 4$$

$$R_\infty = \frac{2 \times 500 \text{ MHz}}{4} = 250 \text{ MFLOPS}$$

The performance without the start-up overhead, which is the peak performance given the vector functional unit structure, is now 1.33 times higher. In actuality, the gap between peak and sustained performance for this benchmark is even larger!

Sustained Performance of VMIPS on the Linpack Benchmark

The Linpack benchmark is a Gaussian elimination on a 100×100 matrix. Thus, the vector element lengths range from 99 down to 1. A vector of length k is used k times. Thus, the average vector length is given by:

$$\frac{\sum_{i=1}^{99} i^2}{\sum_{i=1}^{99} i} = 66.3$$

Now we can obtain an accurate estimate of the performance of DAXPY using a vector length of 66:

$$T_{66} = 2 \times (15 + 49) + 66 \times 3 = 128 + 198 = 326$$

$$R_{66} = \frac{2 \times 66 \times 500}{326} \text{ MFLOPS} = 202 \text{ MFLOPS}$$

The peak number, ignoring start-up overhead, is 1.64 times higher than this estimate of sustained performance on the real vector lengths. In actual practice, the Linpack benchmark contains a nontrivial fraction of code that cannot be vectorized. Although this code accounts for less than 20% of the time before vectorization, it runs at less than one-tenth of the performance when counted as FLOPS. Thus, Amdahl's law tells us that the overall performance will be significantly lower than the performance estimated from analyzing the inner loop.

Since vector length has a significant impact on performance, the $N_{1/2}$ and N_v measures are often used in comparing vector machines.

Example What is $N_{1/2}$ for just the inner loop of DAXPY for VMIPS with a 500 MHz clock?

Answer Using R_∞ as the peak rate, we want to know the vector length that will achieve about 125 MFLOPS. We start with the formula for MFLOPS assuming that the measurement is made for $N_{1/2}$ elements:

$$\text{MFLOPS} = \frac{\text{FLOPS executed in } N_{1/2} \text{ iterations}}{\text{Clock cycles to execute } N_{1/2} \text{ iterations}} \times \frac{\text{Clock cycles}}{\text{Second}} \times 10^{-6}$$

$$125 = \frac{2 \times N_{1/2}}{T_{N_{1/2}}} \times 500$$

Simplifying this and then assuming $N_{1/2} < 64$, so that $T_{N_{1/2}<64} = 64 + 3 \times n$, yields:

$$T_{N_{1/2}} = 8 \times N_{1/2}$$

$$64 + 3 \times N_{1/2} = 8 \times N_{1/2}$$

$$5 \times N_{1/2} = 64$$

$$N_{1/2} = 12.8$$

So $N_{1/2} = 13$; that is, a vector of length 13 gives approximately one-half the peak performance for the DAXPY loop on VMIPS.

Example What is the vector length, N_v, such that the vector operation runs faster than the scalar?

Answer Again, we know that $N_v < 64$. The time to do one iteration in scalar mode can be estimated as $10 + 12 + 12 + 7 + 6 + 12 = 59$ clocks, where 10 is the estimate of the loop overhead, known to be somewhat less than the strip-mining loop overhead. In the last problem, we showed that this vector loop runs in vector mode in time $T_{n \le 64} = 64 + 3 \times n$ clock cycles. Therefore,

$$64 + 3N_v = 59N_v$$

$$N_v = \left\lceil \frac{64}{56} \right\rceil$$

$$N_v = 2$$

For the DAXPY loop, vector mode is faster than scalar as long as the vector has at least two elements. This number is surprisingly small.

DAXPY Performance on an Enhanced VMIPS

DAXPY, like many vector problems, is memory limited. Consequently, performance could be improved by adding more memory access pipelines. This is the major architectural difference between the Cray X-MP (and later processors) and the Cray-1. The Cray X-MP has three memory pipelines, compared with the Cray-1's single memory pipeline, and the X-MP has more flexible chaining. How does this affect performance?

Example What would be the value of T_{66} for DAXPY on VMIPS if we added two more memory pipelines?

Answer With three memory pipelines, all the instructions fit in one convoy and take one chime. The start-up overheads are the same, so

$$T_{66} = \left\lceil \frac{66}{64} \right\rceil \times (T_{loop} + T_{start}) + 66 \times T_{chime}$$

$$T_{66} = 2 \times (15 + 49) + 66 \times 1 = 194$$

With three memory pipelines, we have reduced the clock cycle count for sustained performance from 326 to 194, a factor of 1.7. Note the effect of Amdahl's law: We improved the theoretical peak rate as measured by the number of chimes by a factor of 3, but only achieved an overall improvement of a factor of 1.7 in sustained performance.

Another improvement could come from allowing different convoys to overlap and also allowing the scalar loop overhead to overlap with the vector instructions. This requires that one vector operation be allowed to begin using a functional unit before another operation has completed, which complicates the instruction issue logic. Allowing this overlap eliminates the separate start-up overhead for every convoy except the first and hides the loop overhead as well.

To achieve the maximum hiding of strip-mining overhead, we need to be able to overlap strip-mined instances of the loop, allowing two instances of a convoy as well as possibly two instances of the scalar code to be in execution simultaneously. This requires the same techniques we looked at in Chapter 3 to avoid WAR hazards, although because no overlapped read and write of a single vector element is possible, copying can be avoided. This technique, called *tailgating*, was used in the Cray-2. Alternatively, we could unroll the outer loop to create several instances of the vector sequence using different register sets (assuming sufficient registers), just as we did in Chapter 3. By allowing maximum overlap of the convoys and the scalar loop overhead, the start-up and loop overheads will only be seen *once* per vector sequence, independent of the number of convoys and the instructions in each convoy. In this way, a processor with vector registers can have both low start-up overhead for short vectors and high peak performance for very long vectors.

Example What would be the values of R_∞ and T_{66} for DAXPY on VMIPS if we added two more memory pipelines and allowed the strip-mining and start-up overheads to be fully overlapped?

Answer

$$R_\infty = \lim_{n\to\infty}\left(\frac{\text{Operations per iteration} \times \text{Clock rate}}{\text{Clock cycles per iteration}}\right)$$

$$\lim_{n\to\infty}(\text{Clock cycles per iteration}) = \lim_{n\to\infty}\left(\frac{T_n}{n}\right)$$

Since the overhead is only seen once, $T_n = n + 49 + 15 = n + 64$. Thus,

$$\lim_{n\to\infty}\left(\frac{T_n}{n}\right) = \lim_{n\to\infty}\left(\frac{n+64}{n}\right) = 1$$

$$R_\infty = \frac{2 \times 500 \text{ MHz}}{1} = 1000 \text{ MFLOPS}$$

Adding the extra memory pipelines and more flexible issue logic yields an improvement in peak performance of a factor of 4. However, $T_{66} = 130$, so for shorter vectors the sustained performance improvement is about $326/130 = 2.5$ times.

In summary, we have examined several measures of vector performance. Theoretical peak performance can be calculated based purely on the value of T_{chime} as:

$$\frac{\text{Number of FLOPS per iteration} \times \text{Clock rate}}{T_{chime}}$$

By including the loop overhead, we can calculate values for peak performance for an infinite-length vector (R_∞) and also for sustained performance, R_n for a vector of length n, which is computed as:

$$R_n = \frac{\text{Number of FLOPS per iteration} \times n \times \text{Clock rate}}{T_n}$$

Using these measures we also can find $N_{1/2}$ and N_v, which give us another way of looking at the start-up overhead for vectors and the ratio of vector to scalar speed. A wide variety of measures of performance of vector processors is useful in understanding the range of performance that applications may see on a vector processor.

G.7 A Modern Vector Supercomputer: The Cray X1

The Cray X1 was introduced in 2002, and, together with the NEC SX/8, represents the state of the art in modern vector supercomputers. The X1 system architecture supports thousands of powerful vector processors sharing a single global memory.

The Cray X1 has an unusual processor architecture, shown in Figure G.11. A large Multi-Streaming Processor (MSP) is formed by ganging together four Single-Streaming Processors (SSPs). Each SSP is a complete single-chip vector microprocessor, containing a scalar unit, scalar caches, and a two-lane vector unit. The SSP scalar unit is a dual-issue out-of-order superscalar processor with a 16 KB instruction cache and a 16 KB scalar write-through data cache, both two-way set associative with 32-byte cache lines. The SSP vector unit contains a vector register file, three vector arithmetic units, and one vector load-store unit. It is much easier to pipeline deeply a vector functional unit than a superscalar issue mechanism, so the X1 vector unit runs at twice the clock rate (800 MHz) of the scalar unit (400 MHz). Each lane can perform a 64-bit floating-point add and a 64-bit floating-point multiply each cycle, leading to a peak performance of 12.8 GFLOPS per MSP.

All previous Cray machines could trace their instruction set architecture (ISA) lineage back to the original Cray-1 design from 1976, with 8 primary registers each for addresses, scalar data, and vector data. For the X1, the ISA was redesigned from scratch to incorporate lessons learned over the last 30 years of compiler and microarchitecture research. The X1 ISA includes 64 64-bit scalar address registers and 64 64-bit scalar data registers, with 32 vector data registers (64 bits per element) and 8 vector mask registers (1 bit per element). The large increase in the number of registers allows the compiler to map more program variables into registers to reduce memory traffic and also allows better static scheduling of code to improve

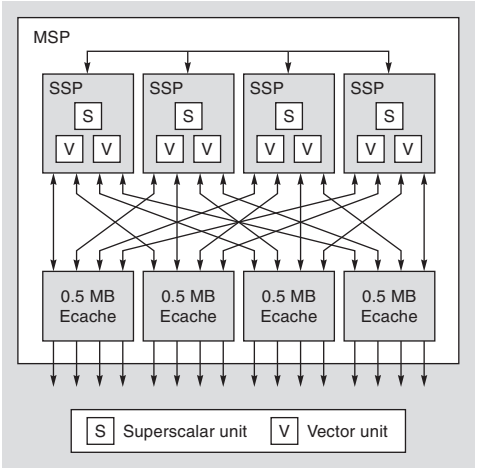


Figure G.11 Cray MSP module. (From Dunnigan et al. [2005].)

run time overlap of instruction execution. Earlier Crays had a compact variable-length instruction set, but the X1 ISA has fixedlength instructions to simplify superscalar fetch and decode.

Four SSP chips are packaged on a multichip module together with four cache chips implementing an external 2 MB cache (Ecache) shared by all the SSPs. The Ecache is two-way set associative with 32-byte lines and a write-back policy. The Ecache can be used to cache vectors, reducing memory traffic for codes that exhibit temporal locality. The ISA also provides vector load and store instruction variants that do not allocate in cache to avoid polluting the Ecache with data that is known to have low locality. The Ecache has sufficient bandwidth to supply one 64-bit word per lane per 800 MHz clock cycle, or over 50 GB/sec per MSP.

At the next level of the X1 packaging hierarchy, shown in Figure G.12, four MSPs are placed on a single printed circuit board together with 16 memory controller chips and DRAM to form an X1 node. Each memory controller chip has eight separate Rambus DRAM channels, where each channel provides 1.6 GB/sec of memory bandwidth. Across all 128 memory channels, the node has over 200 GB/sec of main memory bandwidth.

An X1 system can contain up to 1024 nodes (4096 MSPs or 16,384 SSPs), connected via a very high-bandwidth global network. The network connections are made via the memory controller chips, and all memory in the system is directly accessible from any processor using load and store instructions. This provides much faster global communication than the message-passing protocols used in cluster-based systems. Maintaining cache coherence across such a large number of high-bandwidth shared-memory nodes would be challenging. The approach taken in the X1 is to restrict each Ecache to cache data only from the local node DRAM. The memory controllers implement a directory scheme to maintain

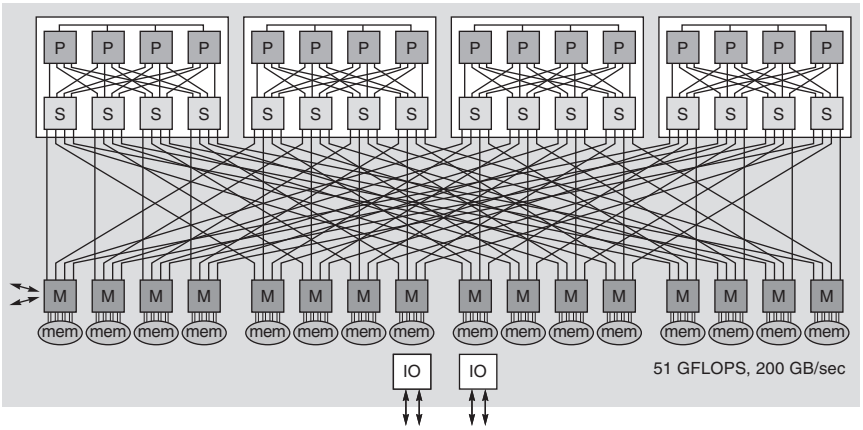


Figure G.12 Cray X1 node. (From Tanqueray [2002].)

coherency between the four Ecaches on a node. Accesses from remote nodes will obtain the most recent version of a location, and remote stores will invalidate local Ecaches before updating memory, but the remote node cannot cache these local locations.

Vector loads and stores are particularly useful in the presence of long-latency cache misses and global communications, as relatively simple vector hardware can generate and track a large number of in-flight memory requests. Contemporary superscalar microprocessors support only 8 to 16 outstanding cache misses, whereas each MSP processor can have up to 2048 outstanding memory requests (512 per SSP). To compensate, superscalar microprocessors have been moving to larger cache line sizes (128 bytes and above) to bring in more data with each cache miss, but this leads to significant wasted bandwidth on non-unit stride accesses over large datasets. The X1 design uses short 32-byte lines throughout to reduce bandwidth waste and instead relies on supporting many independent cache misses to sustain memory bandwidth. This latency tolerance together with the huge memory bandwidth for non-unit strides explains why vector machines can provide large speedups over superscalar microprocessors for certain codes.

Multi-Streaming Processors

The Multi-Streaming concept was first introduced by Cray in the SV1, but has been considerably enhanced in the X1. The four SSPs within an MSP share Ecache, and there is hardware support for barrier synchronization across the four SSPs within an MSP. Each X1 SSP has a two-lane vector unit with 32 vector registers each holding 64 elements. The compiler has several choices as to how to use the SSPs within an MSP.

The simplest use is to gang together four two-lane SSPs to emulate a single eight-lane vector processor. The X1 provides efficient barrier synchronization primitives between SSPs on a node, and the compiler is responsible for generating the MSP code. For example, for a vectorizable inner loop over 1000 elements, the compiler will allocate iterations 0–249 to SSP0, iterations 250–499 to SSP1, iterations 500–749 to SSP2, and iterations 750–999 to SSP3. Each SSP can process its loop iterations independently but must synchronize back with the other SSPs before moving to the next loop nest.

If inner loops do not have many iterations, the eight-lane MSP will have low efficiency, as each SSP will have only a few elements to process and execution time will be dominated by start-up time and synchronization overheads. Another way to use an MSP is for the compiler to parallelize across an outer loop, giving each SSP a different inner loop to process. For example, the following nested loops scale the upper triangle of a matrix by a constant:

```
/* Scale upper triangle by constant K. */
for (row = 0; row < MAX_ROWS; row++)
   for (col = row; col < MAX_COLS; col++)
      A[row][col] = A[row][col] * K;
```

Consider the case where `MAX_ROWS` and `MAX_COLS` are both 100 elements. The vector length of the inner loop steps down from 100 to 1 over the iterations of the outer loop. Even for the first inner loop, the loop length would be much less than the maximum vector length (256) of an eight-lane MSP, and the code would therefore be inefficient. Alternatively, the compiler can assign entire inner loops to a single SSP. For example, SSP0 might process rows 0, 4, 8, and so on, while SSP1 processes rows 1, 5, 9, and so on. Each SSP now sees a longer vector. In effect, this approach parallelizes the scalar overhead and makes use of the individual scalar units within each SSP.

Most application code uses MSPs, but it is also possible to compile code to use all the SSPs as individual processors where there is limited vector parallelism but significant thread-level parallelism.

Cray X1E

In 2004, Cray announced an upgrade to the original Cray X1 design. The X1E uses newer fabrication technology that allows two SSPs to be placed on a single chip, making the X1E the first multicore vector microprocessor. Each physical node now contains eight MSPs, but these are organized as two logical nodes of four MSPs each to retain the same programming model as the X1. In addition, the clock rates were raised from 400 MHz scalar and 800 MHz vector to 565 MHz scalar and 1130 MHz vector, giving an improved peak performance of 18 GFLOPS.

G.8 Concluding Remarks

During the 1980s and 1990s, rapid performance increases in pipelined scalar processors led to a dramatic closing of the gap between traditional vector supercomputers and fast, pipelined, superscalar VLSI microprocessors. In 2011, it is possible to buy a laptop computer for under $1000 that has a higher CPU clock rate than any available vector supercomputer, even those costing tens of millions of dollars. Although the vector supercomputers have lower clock rates, they support greater parallelism using multiple lanes (up to 16 in the Japanese designs) versus the limited multiple issue of the superscalar microprocessors. Nevertheless, the peak floating-point performance of the low-cost microprocessors is within a factor of two of the leading vector supercomputer CPUs. Of course, high clock rates and high peak performance do not necessarily translate into sustained application performance. Main memory bandwidth is the key distinguishing feature between vector supercomputers and superscalar microprocessor systems.

Providing this large non-unit stride memory bandwidth is one of the major expenses in a vector supercomputer, and traditionally SRAM was used as main memory to reduce the number of memory banks needed and to reduce vector start-up penalties. While SRAM has an access time several times lower than that of DRAM, it costs roughly 10 times as much per bit! To reduce main memory costs and to allow larger capacities, all modern vector supercomputers now use DRAM for main memory, taking advantage of new higher-bandwidth DRAM interfaces such as synchronous DRAM.

This adoption of DRAM for main memory (pioneered by Seymour Cray in the Cray-2) is one example of how vector supercomputers have adapted commodity technology to improve their price-performance. Another example is that vector supercomputers are now including vector data caches. Caches are not effective for all vector codes, however, so these vector caches are designed to allow high main memory bandwidth even in the presence of many cache misses. For example, the Cray X1 MSP can have 2048 outstanding memory loads; for microprocessors, 8 to 16 outstanding cache misses per CPU are more typical maximum numbers.

Another example is the demise of bipolar ECL or gallium arsenide as technologies of choice for supercomputer CPU logic. Because of the huge investment in CMOS technology made possible by the success of the desktop computer, CMOS now offers competitive transistor performance with much greater transistor density and much reduced power dissipation compared with these more exotic technologies. As a result, all leading vector supercomputers are now built with the same CMOS technology as superscalar microprocessors. The primary reason why vector supercomputers have lower clock rates than commodity microprocessors is that they are developed using standard cell ASIC techniques rather than full custom circuit design to reduce the engineering design cost. While a microprocessor design may sell tens of millions of copies and can amortize the design cost over this large number of units, a vector supercomputer is considered a success if over a hundred units are sold!

Conversely, via superscalar microprocessor designs have begun to absorb some of the techniques made popular in earlier vector computer systems, such as with the Multimedia SIMD extensions. As we showed in Chapter 4, the investment in hardware for SIMD performance is increasing rapidly, perhaps even more than for multiprocessors. If the even wider SIMD units of GPUs become well integrated with the scalar cores, including scatter-gather support, we may well conclude that vector architectures have won the architecture wars!

G.9 Historical Perspective and References

This historical perspective adds some details and references that were left out of the version in Chapter 4.

The CDC STAR processor and its descendant, the CYBER 205, were memory-memory vector processors. To keep the hardware simple and support the high bandwidth requirements (up to three memory references per floating-point operation), these processors did not efficiently handle non-unit stride. While most loops have unit stride, a non-unit stride loop had poor performance on these processors because memory-to-memory data movements were required to gather together (and scatter back) the nonadjacent vector elements; these operations used special scatter-gather instructions. In addition, there was special support for sparse vectors that used a bit vector to represent the zeros and nonzeros and a dense vector of nonzero values. These more complex vector operations were slow because of the long memory latency, and it was often faster to use scalar mode for sparse or non-unit stride operations. Schneck [1987] described several of the early pipelined processors (e.g., Stretch) through the first vector processors, including the 205 and Cray-1. Dongarra [1986] did another good survey, focusing on more recent processors.

The 1980s also saw the arrival of smaller-scale vector processors, called mini-supercomputers. Priced at roughly one-tenth the cost of a supercomputer ($0.5 to $1 million versus $5 to $10 million), these processors caught on quickly. Although many companies joined the market, the two companies that were most successful were Convex and Alliant. Convex started with the uniprocessor C-1 vector processor and then offered a series of small multiprocessors, ending with the C-4 announced in 1994. The keys to the success of Convex over this period were their emphasis on Cray software capability, the effectiveness of their compiler (see Figure G.9), and the quality of their UNIX OS implementation. The C-4 was the last vector machine Convex sold; they switched to making large-scale multiprocessors using Hewlett-Packard RISC microprocessors and were bought by HP in 1995. Alliant [1987] concentrated more on the multiprocessor aspects; they built an eight-processor computer, with each processor offering vector capability. Alliant ceased operation in the early 1990s.

In the early 1980s, CDC spun out a group, called ETA, to build a new supercomputer, the ETA-10, capable of 10 GFLOPS. The ETA processor was delivered in the late 1980s (see Fazio [1987]) and used low-temperature CMOS in a

configuration with up to 10 processors. Each processor retained the memory-memory architecture based on the CYBER 205. Although the ETA-10 achieved enormous peak performance, its scalar speed was not comparable. In 1989, CDC, the first supercomputer vendor, closed ETA and left the supercomputer design business.

In 1986, IBM introduced the System/370 vector architecture (see Moore et al. [1987]) and its first implementation in the 3090 Vector Facility. The architecture extended the System/370 architecture with 171 vector instructions. The 3090/VF was integrated into the 3090 CPU. Unlike most other vector processors of the time, the 3090/VF routed its vectors through the cache. The IBM 370 machines continued to evolve over time and are now called the IBM zSeries. The vector extensions have been removed from the architecture and some of the opcode space was reused to implement 64-bit address extensions.

In late 1989, Cray Research was split into two companies, both aimed at building high-end processors available in the early 1990s. Seymour Cray headed the spin-off, Cray Computer Corporation, until its demise in 1995. Their initial processor, the Cray-3, was to be implemented in gallium arsenide, but they were unable to develop a reliable and cost-effective implementation technology. A single Cray-3 prototype was delivered to the National Center for Atmospheric Research (NCAR) for evaluation purposes in 1993, but no paying customers were found for the design. The Cray-4 prototype, which was to have been the first processor to run at 1 GHz, was close to completion when the company filed for bankruptcy. Shortly before his tragic death in a car accident in 1996, Seymour Cray started yet another company, SRC Computers, to develop high-performance systems but this time using commodity components. In 2000, SRC announced the SRC-6 system, which combined 512 Intel microprocessors, 5 billion gates of reconfigurable logic, and a high-performance vector-style memory system.

Cray Research focused on the C90, a new high-end processor with up to 16 processors and a clock rate of 240 MHz. This processor was delivered in 1991. The J90 was a CMOS-based vector machine using DRAM memory starting at $250,000, but with typical configurations running about $1 million. In mid-1995, Cray Research was acquired by Silicon Graphics, and in 1998 released the SV1 system, which grafted considerably faster CMOS processors onto the J90 memory system, and which also added a data cache for vectors to each CPU to help meet the increased memory bandwidth demands. The SV1 also introduced the MSP concept, which was developed to provide competitive single-CPU performance by ganging together multiple slower CPUs. Silicon Graphics sold Cray Research to Tera Computer in 2000, and the joint company was renamed Cray Inc.

The basis for modern vectorizing compiler technology and the notion of data dependence was developed by Kuck and his colleagues [1974] at the University of Illinois. Banerjee [1979] developed the test named after him. Padua and Wolfe [1986] gave a good overview of vectorizing compiler technology.

Benchmark studies of various supercomputers, including attempts to understand the performance differences, have been undertaken by Lubeck, Moore,

and Mendez [1985], Bucher [1983], and Jordan [1987]. There are several benchmark suites aimed at scientific usage and often employed for supercomputer benchmarking, including Linpack and the Lawrence Livermore Laboratories FORTRAN kernels. The University of Illinois coordinated the collection of a set of benchmarks for supercomputers, called the Perfect Club. In 1993, the Perfect Club was integrated into SPEC, which released a set of benchmarks, SPEChpc96, aimed at high-end scientific processing in 1996. The NAS parallel benchmarks developed at the NASA Ames Research Center [Bailey et al. 1991] have become a popular set of kernels and applications used for supercomputer evaluation. A new benchmark suite, HPC Challenge, was introduced consisting of a few kernels that stress machine memory and interconnect bandwidths in addition to floating-point performance [Luszczek et al. 2005]. Although standard supercomputer benchmarks are useful as a rough measure of machine capabilities, large supercomputer purchases are generally preceded by a careful performance evaluation on the actual mix of applications required at the customer site.

References

Alliant Computer Systems Corp, 1987. Alliant FX/Series: Product Summary. Mass, Acton (June).

Asanovic, K., 1998. Vector microprocessors," Ph.D. thesis, Computer Science Division. University of California at Berkeley (May).

Bailey, D.H., Barszcz, E., Barton, J.T., Browning, D.S., Carter, R.L., Dagum, L., Fatoohi, R.A., Frederickson, P.O., Lasinski, T.A., Schreiber, R.S., Simon, H.D., Venkatakrishnan, V., Weeratunga, S.K., 1991. The NAS parallel benchmarks. Int'l. J. Supercomputing Applications 5, 63–73.

Banerjee, U., 1979. Speedup of ordinary programs," Ph.D. thesis, Department of Computer Science. University of Illinois at Urbana-Champaign (October).

Baskett, F., Keller, T.W., 1977. An Evaluation of the Cray-1 Processor. In: Kuck, D.J., Lawrie, D.H., Sameh, A.H. (Eds.), High Speed Computer and Algorithm Organization. Academic Press, San Diego, pp. 71–84.

Brandt, M., Brooks, J., Cahir, M., Hewitt, T., Lopez-Pineda, E., Sandness, D., 2000. The Benchmarker's Guide for Cray SV1 Systems. Cray Inc., Seattle, Wash.

Bucher, I.Y., 1983. The computational speed of supercomputers. In: Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems, August 29–31, 1983. Minneapolis, Minn, pp. 151–165.

Callahan, D., Dongarra, J., Levine, D., 1988. Vectorizing compilers: A test suite and results. In: Supercomputing '88: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing, November 12–17, pp. 98–105. Orlando, FL.

Chen, S., 1983. Large-scale and high-speed multiprocessor system for scientific applications. In: Hwang, K. (Ed.), Superprocessors: Design and applications. Proc. NATO Advanced Research Workshop on High-Speed Computing, June 20–22, 1983, Julich, Kernforschungsanlage, Federal Republic of Germany. IEEE, (August), 1984.

Dongarra, J.J., 1986. A survey of high performance processors. COMPCON, IEEE, 8–11 (March).

Dunnigan, T.H., Vetter, J.S., White III, J.B., Worley, P.H., 2005. Performance evaluation of the Cray X1 distributed shared-memory architecture. IEEE Micro 25 (1 (January–February)), 30–40.

Fazio, D., 1987. It's really much more fun building a supercomputer than it is simply inventing one. COMPCON, IEEE, 102–105 (February).

Flynn, M.J., 1966. Very high-speed computing systems. In: Proc. IEEE 54:12 (December), pp. 1901–1909.

Hintz, R.G., Tate, D.P., 1972. Control data STAR-100 processor design. COMPCON, IEEE 1–4 (September).

Jordan, K.E., 1987. Performance comparison of large-scale scientific processors: Scalar mainframes, mainframes with vector facilities, and supercomputers. Computer 20 (3 (March)), 10–23.

Kitagawa, K., Tagaya, S., Hagihara, Y., Kanoh, Y., 2003. A hardware overview of SX-6 and SX-7 supercomputer. NEC Research & Development J 44 (1 (January)), 2–7.
Kuck, D., Budnik, P.P., Chen, S.-C., Lawrie, D.H., Towle, R.A., Strebendt, R.E., Davis Jr., E.W., Han, J., Kraska, P.W., Muraoka, Y., 1974. Measurements of parallelism in ordinary FORTRAN programs. Computer 7 (1 (January)), 37–46.
Lincoln, N.R., 1982. Technology and design trade offs in the creation of a modern supercomputer. IEEE Trans. on Computers, 363–376. C-31:5 (May).
Lubeck, O., Moore, J., Mendez, R., 1985. A benchmark comparison of three supercomputers: Fujitsu VP-200, Hitachi S810/20, and Cray X-MP/2. Computer 18 (1 (January)), 10–29.
Luszczek, P., Dongarra, J.J., Koester, D., Rabenseifner, R., Lucas, B., Kepner, J., McCalpin, J., Bailey, D., Takahashi, D., 2005. In: Introduction to the HPC challenge benchmark suite," Lawrence Berkeley National Laboratory, Paper LBNL-57493 (April 25). http://repositories.cdlib.org/lbnl/LBNL-57493.
Miranker, G.S., Rubenstein, J., Sanguinetti, J., 1988. Squeezing a Cray-class supercomputer into a single-user package. COMPCON, IEEE, 452–456 (March).
Miura, K., Uchida, K., 1983. FACOM vector processing system: VP100/200. In: Proc. NATO Advanced Research Workshop on High-Speed Computing. June 20–22, 1983, Julich, Kernforschungsanlage, Federal Republic of Germany; also in K. Hwang, ed., "Superprocessors: Design and applications," *IEEE* (August), 1984, 59–73.
Moore, B., Padegs, A., Smith, R., Bucholz, W., 1987. Concepts of the System/370 vector architecture. In: Proc. 14th Int'l. Symposium on Computer Architecture, June 3–6, 1987. Pittsburgh, Penn, pp. 282–292.
Padua, D., Wolfe, M., 1986. Advanced compiler optimizations for supercomputers. Comm. ACM 29 (12 (December)), 1184–1201.
Russell, R.M., 1978. The Cray-1 processor system. Comm. of the ACM 21 (1 (January)), 63–72.
Schneck, P.B., 1987. Superprocessor Architecture. Kluwer Academic Publishers, Norwell, Mass.
Smith, B.J., 1981. Architecture and applications of the HEP multiprocessor system. Real-Time Signal Processing IV 298, 241–248. August.
Sporer, M., Moss, F.H., Mathais, C.J., 1988. An introduction to the architecture of the Stellar Graphics supercomputer. COMPCON, IEEE 464 (March).
Tanqueray, D., 2002. The Cray X1 and supercomputer road map. In: Proc. 13th Daresbury Machine Evaluation Workshop, December 11–12. Cheshire, England.
Vajapeyam, S., 1991. Instruction-level characterization of the Cray Y-MP processor. Ph.D. thesis, Computer Sciences Department. University of Wisconsin-Madison.
Watanabe, T., 1987. Architecture and performance of the NEC supercomputer SX system. Parallel Computing 5, 247–255.
Watson, W.J., 1972. The TI ASC—a highly modular and flexible super processor architecture. In: Proc. AFIPS Fall Joint Computer Conf, pp. 221–228.

Exercises

In these exercises assume VMIPS has a clock rate of 500 MHz and that $T_{loop} = 15$. Use the start-up times from Figure G.2, and assume that the store latency is always included in the running time.

G.1 [10] <G.1, G.2> Write a VMIPS vector sequence that achieves the peak MFLOPS performance of the processor (use the functional unit and instruction description in Section G.2). Assuming a 500-MHz clock rate, what is the peak MFLOPS?

G.2 [20/15/15] <G.1–G.6> Consider the following vector code run on a 500 MHz version of VMIPS for a fixed vector length of 64:

```
LV       V1,Ra
MULV.D   V2,V1,V3
ADDV.D   V4,V1,V3
SV       Rb,V2
SV       Rc,V4
```

Ignore all strip-mining overhead, but assume that the store latency must be included in the time to perform the loop. The entire sequence produces 64 results.

a. [20] <G.1–G.4> Assuming no chaining and a single memory pipeline, how many chimes are required? How many clock cycles per result (including both stores as one result) does this vector sequence require, including start-up overhead?

b. [15] <G.1–G.4> If the vector sequence is chained, how many clock cycles per result does this sequence require, including overhead?

c. [15] <G.1–G.6> Suppose VMIPS had three memory pipelines and chaining. If there were no bank conflicts in the accesses for the above loop, how many clock cycles are required per result for this sequence?

G.3 [20/20/15/15/20/20/20] <G.2–G.6> Consider the following FORTRAN code:

```
      do 10 i=1,n
            A(i)=A(i)+B(i)
            B(i)=x * B(i)
10    continue
```

Use the techniques of Section G.6 to estimate performance throughout this exercise, assuming a 500 MHz version of VMIPS.

a. [20] <G.2–G.6> Write the best VMIPS vector code for the inner portion of the loop. Assume `x` is in F0 and the addresses of `A` and `B` are in `Ra` and `Rb`, respectively.

b. [20] <G.2–G.6> Find the total time for this loop on VMIPS (T_{100}). What is the MFLOPS rating for the loop (R_{100})?

c. [15] <G.2–G.6> Find R_∞ for this loop.

d. [15] <G.2–G.6> Find $N_{1/2}$ for this loop.

e. [20] <G.2–G.6> Find N_v for this loop. Assume the scalar code has been pipeline scheduled so that each memory reference takes six cycles and each FP operation takes three cycles. Assume the scalar overhead is also T_{loop}.

f. [20] <G.2–G.6> Assume VMIPS has two memory pipelines. Write vector code that takes advantage of the second memory pipeline. Show the layout in convoys.

g. [20] <G.2–G.6> Compute T_{100} and R_{100} for VMIPS with two memory pipelines.

G.4 [20/10] <G.2> Suppose we have a version of VMIPS with eight memory banks (each a double word wide) and a memory access time of eight cycles.

a. [20] <G.2> If a load vector of length 64 is executed with a stride of 20 double words, how many cycles will the load take to complete?

b. [10] <G.2> What percentage of the memory bandwidth do you achieve on a 64-element load at stride 20 versus stride 1?

G.5 [12/12] <G.5–G.6> Consider the following loop:

```
      C=0.0
      do 10 i=1,64
            A(i)=A(i)+B(i)
            C=C+A(i)
10    continue
```

a. [12] <G.5–G.6> Split the loop into two loops: one with no dependence and one with a dependence. Write these loops in FORTRAN—as a source-to-source transformation. This optimization is called *loop fission.*

b. [12] <G.5–G.6> Write the VMIPS vector code for the loop without a dependence.

G.6 [20/15/20/20] <G.5–G.6> The compiled Linpack performance of the Cray-1 (designed in 1976) was almost doubled by a better compiler in 1989. Let's look at a simple example of how this might occur. Consider the DAXPY-like loop (where *k* is a parameter to the procedure containing the loop):

```
      do 10 i=1,64
         do 10 j=1,64
         Y(k,j)=a*X(i,j)+Y(k,j)
10    continue
```

a. [20] <G.5–G.6> Write the *straightforward* code sequence for just the inner loop in VMIPS vector instructions.

b. [15] <G.5–G.6> Using the techniques of Section G.6, estimate the performance of this code on VMIPS by finding T_{64} in clock cycles. You may assume that T_{loop} of overhead is incurred for each iteration of the outer loop. What limits the performance?

c. [20] <G.5–G.6> Rewrite the VMIPS code to reduce the performance limitation; show the resulting inner loop in VMIPS vector instructions. (*Hint:* Think about what establishes T_{chime}; can you affect it?) Find the total time for the resulting sequence.

d. [20] <G.5–G.6> Estimate the performance of your new version, using the techniques of Section G.6 and finding T_{64}.

G.7 [15/15/25] <G.4> Consider the following code:

```
      do 10 i=1,64
         if (B(i) .ne. 0) then
            A(i)=A(i)/B(i)
10    continue
```

Assume that the addresses of A and B are in Ra and Rb, respectively, and that F0 contains 0.

a. [15] <G.4> Write the VMIPS code for this loop using the vector-mask capability.

b. [15] <G.4> Write the VMIPS code for this loop using scatter-gather.

c. [25] <G.4> Estimate the performance (T_{100} in clock cycles) of these two vector loops, assuming a divide latency of 20 cycles. Assume that all vector instructions run at one result per clock, independent of the setting of the vector-mask register. Assume that 50% of the entries of B are 0. Considering hardware costs, which would you build if the above loop were typical?

G.8 [15/20/15/15] <G.1–G.6> The difference between peak and sustained performance can be large. For one problem, a Hitachi S810 had a peak speed twice as high as that of the Cray X-MP, while for another more realistic problem, the Cray X-MP was twice as fast as the Hitachi processor. Let's examine why this might occur using two versions of VMIPS and the following code sequences:

```
C           Code sequence 1
      do 10 i=1,10000
            A(i)=x * A(i)+y * A(i)
10    continue
C           Code sequence 2
      do 10 i=1,100
            A(i)=x * A(i)
10    continue
```

Assume there is a version of VMIPS (call it VMIPS-II) that has two copies of every floating-point functional unit with full chaining among them. Assume that both VMIPS and VMIPS-II have two load-store units. Because of the extra functional units and the increased complexity of assigning operations to units, all the overheads (T_{loop} and T_{start}) are doubled for VMIPS-II.

a. [15] <G.1–G.6> Find the number of clock cycles on code sequence 1 on VMIPS.

b. [20] <G.1–G.6> Find the number of clock cycles on code sequence 1 for VMIPS-II. How does this compare to VMIPS?

c. [15] <G.1–G.6> Find the number of clock cycles on code sequence 2 for VMIPS.

d. [15] <G.1–G.6> Find the number of clock cycles on code sequence 2 for VMIPS-II. How does this compare to VMIPS?

G.9 [20] <G.5> Here is a tricky piece of code with two-dimensional arrays. Does this loop have dependences? Can these loops be written so they are parallel? If so, how? Rewrite the *source* code so that it is clear that the loop can be vectorized, if possible.

```
      do 290 j=2,n
        do 290 i=2,j
          aa(i,j)=aa(i-1,j)*aa(i-1,j)+bb(i,j)
290   continue
```

G.10 [12/15] <G.5> Consider the following loop:

```
      do 10 i=2,n
         A(i)=B
10       C(i)=A(i - 1)
```

a. [12] <G.5> Show there is a loop-carried dependence in this code fragment.

b. [15] <G.5> Rewrite the code in FORTRAN so that it can be vectorized as two separate vector sequences.

G.11 [15/25/25] <G.5> As we saw in Section G.5, some loop structures are not easily vectorized. One common structure is a *reduction*—a loop that reduces an array to a single value by repeated application of an operation. This is a special case of a recurrence. A common example occurs in dot product:

```
      dot=0.0
      do 10 i=1,64
10          dot=dot+A(i) * B(i)
```

This loop has an obvious loop-carried dependence (on `dot`) and cannot be vectorized in a straightforward fashion. The first thing a good vectorizing compiler would do is split the loop to separate out the vectorizable portion and the recurrence and perhaps rewrite the loop as:

```
      do 10 i=1,64
10          dot(i)=A(i) * B(i)
      do 20 i=2,64
20          dot(1)=dot(1)+dot(i)
```

The variable `dot` has been expanded into a vector; this transformation is called *scalar expansion*. We can try to vectorize the second loop either relying strictly on the compiler (part (a)) or with hardware support as well (part (b)). There is an important caveat in the use of vector techniques for reduction. To make reduction work, we are relying on the associativity of the operator being used for the reduction. Because of rounding and finite range, however, floating-point arithmetic is not strictly associative. For this reason, most compilers require the programmer to indicate whether associativity can be used to more efficiently compile reductions.

a. [15] <G.5> One simple scheme for compiling the loop with the recurrence is to add sequences of progressively shorter vectors—two 32-element vectors, then two 16-element vectors, and so on. This technique has been called *recursive doubling*. It is faster than doing all the operations in scalar mode. Show how the FORTRAN code would look for execution of the second loop in the preceding code fragment using recursive doubling.

b. [25] <G.5> In some vector processors, the vector registers are addressable, and the operands to a vector operation may be two different parts of the same vector register. This allows another solution for the reduction, called *partial sums*.

The key idea in partial sums is to reduce the vector to *m* sums where *m* is the total latency through the vector functional unit, including the operand read and write times. Assume that the VMIPS vector registers are addressable (e.g., you can initiate a vector operation with the operand V1(16), indicating that the input operand began with element 16). Also, assume that the total latency for adds, including operand read and write, is eight cycles. Write a VMIPS code sequence that reduces the contents of V1 to eight partial sums. It can be done with one vector operation.

c. [25] < G.5 > Discuss how adding the extension in part (b) would affect a machine that had multiple lanes.

G.12 [40] < G.3–G.4 > Extend the MIPS simulator to be a VMIPS simulator, including the ability to count clock cycles. Write some short benchmark programs in MIPS and VMIPS assembly language. Measure the speedup on VMIPS, the percentage of vectorization, and usage of the functional units.

G.13 [50] < G.5 > Modify the MIPS compiler to include a dependence checker. Run some scientific code and loops through it and measure what percentage of the statements could be vectorized.

G.14 [Discussion] Some proponents of vector processors might argue that the vector processors have provided the best path to ever-increasing amounts of processor power by focusing their attention on boosting peak vector performance. Others would argue that the emphasis on peak performance is misplaced because an increasing percentage of the programs are dominated by nonvector performance. (Remember Amdahl's law?) The proponents would respond that programmers should work to make their programs vectorizable. What do you think about this argument?

H.1	Introduction: Exploiting Instruction-Level Parallelism Statically	H-2
H.2	Detecting and Enhancing Loop-Level Parallelism	H-2
H.3	Scheduling and Structuring Code for Parallelism	H-12
H.4	Hardware Support for Exposing Parallelism: Predicated Instructions	H-23
H.5	Hardware Support for Compiler Speculation	H-27
H.6	The Intel IA-64 Architecture and Itanium Processor	H-32
H.7	Concluding Remarks	H-43

H

Hardware and Software for VLIW and EPIC

The EPIC approach is based on the application of massive resources. These resources include more load-store, computational, and branch units, as well as larger, lower-latency caches than would be required for a superscalar processor. Thus, IA-64 gambles that, in the future, power will not be the critical limitation, and that massive resources, along with the machinery to exploit them, will not penalize performance with their adverse effect on clock speed, path length, or CPI factors.

M. Hopkins
in a commentary on the EPIC approach and the IA-64 architecture (2000)

H.1 Introduction: Exploiting Instruction-Level Parallelism Statically

In this chapter, we discuss compiler technology for increasing the amount of parallelism that we can exploit in a program as well as hardware support for these compiler techniques. The next section defines when a loop is parallel, how a dependence can prevent a loop from being parallel, and techniques for eliminating some types of dependences. The following section discusses the topic of scheduling code to improve parallelism. These two sections serve as an introduction to these techniques.

We do not attempt to explain the details of ILP-oriented compiler techniques, since that would take hundreds of pages, rather than the 20 we have allotted. Instead, we view this material as providing general background that will enable the reader to have a basic understanding of the compiler techniques used to exploit ILP in modern computers.

Hardware support for these compiler techniques can greatly increase their effectiveness, and Sections H.4 and H.5 explore such support. The IA-64 represents the culmination of the compiler and hardware ideas for exploiting parallelism statically and includes support for many of the concepts proposed by researchers during more than a decade of research into the area of compiler-based instruction-level parallelism. Section H.6 provides a description and performance analyses of the Intel IA-64 architecture and its second-generation implementation, Itanium 2.

The core concepts that we exploit in statically based techniques—finding parallelism, reducing control and data dependences, and using speculation—are the same techniques we saw exploited in Chapter 3 using dynamic techniques. The key difference is that the techniques in this appendix are applied at compile time by the compiler, rather than at runtime by the hardware. The advantages of compile time techniques are primarily two: They do not burden runtime execution with any inefficiency, and they can take into account a wider range of the program than a runtime approach might be able to incorporate. As an example of the latter, the next section shows how a compiler might determine that an entire loop can be executed in parallel; hardware techniques might or might not be able to find such parallelism. The major disadvantage of static approaches is that they can use only compile time information. Without runtime information, compile time techniques must often be conservative and assume the worst case.

H.2 Detecting and Enhancing Loop-Level Parallelism

Loop-level parallelism is normally analyzed at the source level or close to it, while most analysis of ILP is done once instructions have been generated by the compiler. Loop-level analysis involves determining what dependences exist among the operands in a loop across the iterations of that loop. For now, we will

consider only data dependences, which arise when an operand is written at some point and read at a later point. Name dependences also exist and may be removed by renaming techniques like those we explored in Chapter 3.

The analysis of loop-level parallelism focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations; such a dependence is called a *loop-carried dependence*. Most of the examples we considered in Section 3.2 have no loop-carried dependences and, thus, are loop-level parallel. To see that a loop is parallel, let us first look at the source representation:

```
for (i=1000; i>0; i=i-1)
      x[i] = x[i] + s;
```

In this loop, there is a dependence between the two uses of `x[i]`, but this dependence is within a single iteration and is not loop carried. There is a dependence between successive uses of `i` in different iterations, which is loop carried, but this dependence involves an induction variable and can be easily recognized and eliminated. We saw examples of how to eliminate dependences involving induction variables during loop unrolling in Section 3.2, and we will look at additional examples later in this section.

Because finding loop-level parallelism involves recognizing structures such as loops, array references, and induction variable computations, the compiler can do this analysis more easily at or near the source level, as opposed to the machine-code level. Let's look at a more complex example.

Example Consider a loop like this one:

```
for (i=1; i<=100; i=i+1) {
      A[i+1] = A[i] + C[i];   /* S1 */
      B[i+1] = B[i] + A[i+1]; /* S2 */
}
```

Assume that `A`, `B`, and `C` are distinct, nonoverlapping arrays. (In practice, the arrays may sometimes be the same or may overlap. Because the arrays may be passed as parameters to a procedure, which includes this loop, determining whether arrays overlap or are identical often requires sophisticated, interprocedural analysis of the program.) What are the data dependences among the statements S1 and S2 in the loop?

Answer There are two different dependences:

1. S1 uses a value computed by S1 in an earlier iteration, since iteration `i` computes `A[i+1]`, which is read in iteration `i+1`. The same is true of S2 for `B[i]` and `B[i+1]`.
2. S2 uses the value, `A[i+1]`, computed by S1 in the same iteration.

These two dependences are different and have different effects. To see how they differ, let's assume that only one of these dependences exists at a time. Because the dependence of statement S1 is on an earlier iteration of S1, this dependence is loop carried. This dependence forces successive iterations of this loop to execute in series.

The second dependence (S2 depending on S1) is within an iteration and is not loop carried. Thus, if this were the only dependence, multiple iterations of the loop could execute in parallel, as long as each pair of statements in an iteration were kept in order. We saw this type of dependence in an example in Section 3.2, where unrolling was able to expose the parallelism.

It is also possible to have a loop-carried dependence that does not prevent parallelism, as the next example shows.

Example Consider a loop like this one:

```
for (i=1; i<=100; i=i+1) {
      A[i] = A[i] + B[i];   /* S1 */
      B[i+1] = C[i] + D[i]; /* S2 */
}
```

What are the dependences between S1 and S2? Is this loop parallel? If not, show how to make it parallel.

Answer Statement S1 uses the value assigned in the previous iteration by statement S2, so there is a loop-carried dependence between S2 and S1. Despite this loop-carried dependence, this loop can be made parallel. Unlike the earlier loop, this dependence is not circular: Neither statement depends on itself, and, although S1 depends on S2, S2 does not depend on S1. A loop is parallel if it can be written without a cycle in the dependences, since the absence of a cycle means that the dependences give a partial ordering on the statements.

Although there are no circular dependences in the above loop, it must be transformed to conform to the partial ordering and expose the parallelism. Two observations are critical to this transformation:

1. There is no dependence from S1 to S2. If there were, then there would be a cycle in the dependences and the loop would not be parallel. Since this other dependence is absent, interchanging the two statements will not affect the execution of S2.
2. On the first iteration of the loop, statement S1 depends on the value of `B[1]` computed prior to initiating the loop.

These two observations allow us to replace the loop above with the following code sequence:

```
A[1] = A[1] + B[1];
for (i=1; i<=99; i=i+1) {
      B[i+1] = C[i] + D[i];
      A[i+1] = A[i+1] + B[i+1];
}
B[101] = C[100] + D[100];
```

The dependence between the two statements is no longer loop carried, so iterations of the loop may be overlapped, provided the statements in each iteration are kept in order.

Our analysis needs to begin by finding all loop-carried dependences. This dependence information is *inexact*, in the sense that it tells us that such a dependence *may* exist. Consider the following example:

```
for (i=1;i<=100;i=i+1) {
      A[i] = B[i] + C[i]
      D[i] = A[i] * E[i]
}
```

The second reference to `A` in this example need not be translated to a load instruction, since we know that the value is computed and stored by the previous statement; hence, the second reference to `A` can simply be a reference to the register into which `A` was computed. Performing this optimization requires knowing that the two references are *always* to the same memory address and that there is no intervening access to the same location. Normally, data dependence analysis only tells that one reference *may* depend on another; a more complex analysis is required to determine that two references *must be* to the exact same address. In the example above, a simple version of this analysis suffices, since the two references are in the same basic block.

Often loop-carried dependences are in the form of a *recurrence:*

```
for (i=2;i<=100;i=i+1) {
      Y[i] = Y[i-1] + Y[i];
}
```

A recurrence is when a variable is defined based on the value of that variable in an earlier iteration, often the one immediately preceding, as in the above fragment. Detecting a recurrence can be important for two reasons: Some architectures (especially vector computers) have special support for executing recurrences, and some recurrences can be the source of a reasonable amount of parallelism. To see how the latter can be true, consider this loop:

```
for (i=6;i<=100;i=i+1) {
      Y[i] = Y[i-5] + Y[i];
}
```

On the iteration *i,* the loop references element $i - 5$. The loop is said to have a *dependence distance* of 5. Many loops with carried dependences have a dependence distance of 1. The larger the distance, the more potential parallelism can be obtained by unrolling the loop. For example, if we unroll the first loop, with a dependence distance of 1, successive statements are dependent on one another; there is still some parallelism among the individual instructions, but not much. If we unroll the loop that has a dependence distance of 5, there is a sequence of five statements that have no dependences, and thus much more ILP. Although many loops with loop-carried dependences have a dependence distance of 1, cases with larger distances do arise, and the longer distance may well provide enough parallelism to keep a processor busy.

Finding Dependences

Finding the dependences in a program is an important part of three tasks: (1) good scheduling of code, (2) determining which loops might contain parallelism, and (3) eliminating name dependences. The complexity of dependence analysis arises because of the presence of arrays and pointers in languages like C or C++, or pass-by-reference parameter passing in FORTRAN. Since scalar variable references explicitly refer to a name, they can usually be analyzed quite easily, with aliasing because of pointers and reference parameters causing some complications and uncertainty in the analysis.

How does the compiler detect dependences in general? Nearly all dependence analysis algorithms work on the assumption that array indices are *affine*. In simplest terms, a one-dimensional array index is affine if it can be written in the form $a \times i + b$, where a and b are constants and i is the loop index variable. The index of a multidimensional array is affine if the index in each dimension is affine. Sparse array accesses, which typically have the form `x[y[i]]`, are one of the major examples of nonaffine accesses.

Determining whether there is a dependence between two references to the same array in a loop is thus equivalent to determining whether two affine functions can have the same value for different indices between the bounds of the loop. For example, suppose we have stored to an array element with index value $a \times i + b$ and loaded from the same array with index value $c \times i + d$, where i is the for-loop index variable that runs from m to n. A dependence exists if two conditions hold:

1. There are two iteration indices, j and k, both within the limits of the for loop. That is, $m \leq j \leq n$, $m \leq k \leq n$.
2. The loop stores into an array element indexed by $a \times j + b$ and later fetches from that *same* array element when it is indexed by $c \times k + d$. That is, $a \times j + b = c \times k + d$.

In general, we cannot determine whether a dependence exists at compile time. For example, the values of a, b, c, and d may not be known (they could be values in other arrays), making it impossible to tell if a dependence exists. In other cases, the dependence testing may be very expensive but decidable at compile time. For example, the accesses may depend on the iteration indices of multiple nested loops. Many programs, however, contain primarily simple indices where a, b, c, and d are all constants. For these cases, it is possible to devise reasonable compile time tests for dependence.

As an example, a simple and sufficient test for the absence of a dependence is the *greatest common divisor* (GCD) test. It is based on the observation that if a loop-carried dependence exists, then GCD (c,a) must divide ($d - b$). (Recall that an integer, x, *divides* another integer, y, if we get an integer quotient when we do the division y/x and there is no remainder.)

Example Use the GCD test to determine whether dependences exist in the following loop:

```
for (i=1; i<=100; i=i+1) {
      X[2*i+3] = X[2*i] * 5.0;
}
```

Answer Given the values $a = 2$, $b = 3$, $c = 2$, and $d = 0$, then GCD(a,c) $= 2$, and $d - b = -3$. Since 2 does not divide -3, no dependence is possible.

The GCD test is sufficient to guarantee that no dependence exists; however, there are cases where the GCD test succeeds but no dependence exists. This can arise, for example, because the GCD test does not take the loop bounds into account.

In general, determining whether a dependence actually exists is NP complete. In practice, however, many common cases can be analyzed precisely at low cost. Recently, approaches using a hierarchy of exact tests increasing in generality and cost have been shown to be both accurate and efficient. (A test is *exact* if it precisely determines whether a dependence exists. Although the general case is NP complete, there exist exact tests for restricted situations that are much cheaper.)

In addition to detecting the presence of a dependence, a compiler wants to classify the type of dependence. This classification allows a compiler to recognize name dependences and eliminate them at compile time by renaming and copying.

Example The following loop has multiple types of dependences. Find all the true dependences, output dependences, and antidependences, and eliminate the output dependences and antidependences by renaming.

```
for (i=1; i<=100; i=i+1) {
    Y[i] = X[i] / c; /* S1 */
    X[i] = X[i] + c; /* S2 */
    Z[i] = Y[i] + c; /* S3 */
    Y[i] = c - Y[i]; /* S4 */
}
```

Answer The following dependences exist among the four statements:

1. There are true dependences from S1 to S3 and from S1 to S4 because of `Y[i]`. These are not loop carried, so they do not prevent the loop from being considered parallel. These dependences will force S3 and S4 to wait for S1 to complete.
2. There is an antidependence from S1 to S2, based on `X[i]`.
3. There is an antidependence from S3 to S4 for `Y[i]`.
4. There is an output dependence from S1 to S4, based on `Y[i]`.

The following version of the loop eliminates these false (or pseudo) dependences:

```
for (i=1; i<=100; i=i+1 {
    /* Y renamed to T to remove output dependence */
    T[i] = X[i] / c;
    /* X renamed to X1 to remove antidependence */
    X1[i] = X[i] + c;
    /* Y renamed to T to remove antidependence */
    Z[i] = T[i] + c;
    Y[i] = c - T[i];
}
```

After the loop, the variable `X` has been renamed `X1`. In code that follows the loop, the compiler can simply replace the name `X` by `X1`. In this case, renaming does not require an actual copy operation but can be done by substituting names or by register allocation. In other cases, however, renaming will require copying.

Dependence analysis is a critical technology for exploiting parallelism. At the instruction level, it provides information needed to interchange memory references when scheduling, as well as to determine the benefits of unrolling a loop. For detecting loop-level parallelism, dependence analysis is the basic tool. Effectively compiling programs to either vector computers or multiprocessors depends critically on this analysis. The major drawback of dependence analysis is that it applies only under a limited set of circumstances—namely, among references within a single loop nest and using affine index functions. Thus, there is a wide variety of situations in which array-oriented dependence analysis *cannot* tell us what we might want to know, including the following:

- When objects are referenced via pointers rather than array indices (but see discussion below)
- When array indexing is indirect through another array, which happens with many representations of sparse arrays
- When a dependence may exist for some value of the inputs but does not exist in actuality when the code is run since the inputs never take on those values
- When an optimization depends on knowing more than just the possibility of a dependence but needs to know on *which* write of a variable does a read of that variable depend

To deal with the issue of analyzing programs with pointers, another type of analysis, often called *points-to* analysis, is required (see Wilson and Lam [1995]). The key question that we want answered from dependence analysis of pointers is whether two pointers can designate the same address. In the case of complex dynamic data structures, this problem is extremely difficult. For example, we may want to know whether two pointers can reference the *same* node in a list at a given point in a program, which in general is undecidable and in practice is extremely difficult to answer. We may, however, be able to answer a simpler question: Can two pointers designate nodes in the *same* list, even if they may be separate nodes? This more restricted analysis can still be quite useful in scheduling memory accesses performed through pointers.

The basic approach used in points-to analysis relies on information from three major sources:

1. Type information, which restricts what a pointer can point to.
2. Information derived when an object is allocated or when the address of an object is taken, which can be used to restrict what a pointer can point to. For example, if p always points to an object allocated in a given source line and q never points to that object, then p and q can never point to the same object.
3. Information derived from pointer assignments. For example, if p may be assigned the value of q, then p may point to anything q points to.

There are several cases where analyzing pointers has been successfully applied and is extremely useful:

- When pointers are used to pass the address of an object as a parameter, it is possible to use points-to analysis to determine the possible set of objects referenced by a pointer. One important use is to determine if two pointer parameters may designate the same object.
- When a pointer can point to one of several types, it is sometimes possible to determine the type of the data object that a pointer designates at different parts of the program.
- It is often possible to separate out pointers that may only point to a local object versus a global one.

There are two different types of limitations that affect our ability to do accurate dependence analysis for large programs. The first type of limitation arises from restrictions in the analysis algorithms. Often, we are limited by the lack of applicability of the analysis rather than a shortcoming in dependence analysis *per se*. For example, dependence analysis for pointers is essentially impossible for programs that use pointers in arbitrary fashion—such as by doing arithmetic on pointers.

The second limitation is the need to analyze behavior across procedure boundaries to get accurate information. For example, if a procedure accepts two parameters that are pointers, determining whether the values could be the same requires analyzing across procedure boundaries. This type of analysis, called *interprocedural analysis*, is much more difficult and complex than analysis within a single procedure. Unlike the case of analyzing array indices within a single loop nest, points-to analysis usually requires an interprocedural analysis. The reason for this is simple. Suppose we are analyzing a program segment with two pointers; if the analysis does not know anything about the two pointers at the start of the program segment, it must be conservative and assume the worst case. The worst case is that the two pointers *may* designate the same object, but they are not *guaranteed* to designate the same object. This worst case is likely to propagate through the analysis, producing useless information. In practice, getting fully accurate interprocedural information is usually too expensive for real programs. Instead, compilers usually use approximations in interprocedural analysis. The result is that the information may be too inaccurate to be useful.

Modern programming languages that use strong typing, such as Java, make the analysis of dependences easier. At the same time the extensive use of procedures to structure programs, as well as abstract data types, makes the analysis more difficult. Nonetheless, we expect that continued advances in analysis algorithms, combined with the increasing importance of pointer dependency analysis, will mean that there is continued progress on this important problem.

Eliminating Dependent Computations

Compilers can reduce the impact of dependent computations so as to achieve more instruction-level parallelism (ILP). The key technique is to eliminate or reduce a dependent computation by back substitution, which increases the amount of parallelism and sometimes increases the amount of computation required. These techniques can be applied both within a basic block and within loops, and we describe them differently.

Within a basic block, algebraic simplifications of expressions and an optimization called *copy propagation*, which eliminates operations that copy values, can be used to simplify sequences like the following:

```
DADDUI      R1,R2,#4
DADDUI      R1,R1,#4
```

to

```
DADDUI      R1,R2,#8
```

assuming this is the only use of R1. In fact, the techniques we used to reduce multiple increments of array indices during loop unrolling and to move the increments across memory addresses in Section 3.2 are examples of this type of optimization.

In these examples, computations are actually eliminated, but it is also possible that we may want to increase the parallelism of the code, possibly even increasing the number of operations. Such optimizations are called *tree height reduction* because they reduce the height of the tree structure representing a computation, making it wider but shorter. Consider the following code sequence:

```
ADD         R1,R2,R3
ADD         R4,R1,R6
ADD         R8,R4,R7
```

Notice that this sequence requires at least three execution cycles, since all the instructions depend on the immediate predecessor. By taking advantage of associativity, we can transform the code and rewrite it as

```
ADD         R1,R2,R3
ADD         R4,R6,R7
ADD         R8,R1,R4
```

This sequence can be computed in two execution cycles. When loop unrolling is used, opportunities for these types of optimizations occur frequently.

Although arithmetic with unlimited range and precision is associative, computer arithmetic is not associative, for either integer arithmetic, because of limited range, or floating-point arithmetic, because of both range and precision. Thus, using these restructuring techniques can sometimes lead to erroneous behavior, although such occurrences are rare. For this reason, most compilers require that optimizations that rely on associativity be explicitly enabled.

When loops are unrolled, this sort of algebraic optimization is important to reduce the impact of dependences arising from recurrences. *Recurrences* are expressions whose value on one iteration is given by a function that depends on the previous iterations. When a loop with a recurrence is unrolled, we may be able to algebraically optimize the unrolled loop, so that the recurrence need only be evaluated once per unrolled iteration. One common type of recurrence arises from an explicit program statement, such as:

```
sum = sum + x;
```

Assume we unroll a loop with this recurrence five times. If we let the value of `x` on these five iterations be given by `x1`, `x2`, `x3`, `x4`, and `x5`, then we can write the value of `sum` at the end of each unroll as:

```
sum = sum + x1 + x2 + x3 + x4 + x5;
```

If unoptimized, this expression requires five dependent operations, but it can be rewritten as:

```
sum = ((sum + x1) + (x2 + x3)) + (x4 + x5);
```

which can be evaluated in only three dependent operations.

Recurrences also arise from implicit calculations, such as those associated with array indexing. Each array index translates to an address that is computed based on the loop index variable. Again, with unrolling and algebraic optimization, the dependent computations can be minimized.

H.3 Scheduling and Structuring Code for Parallelism

We have already seen that one compiler technique, loop unrolling, is useful to uncover parallelism among instructions by creating longer sequences of straight-line code. There are two other important techniques that have been developed for this purpose: *software pipelining* and *trace scheduling*.

Software Pipelining: Symbolic Loop Unrolling

Software pipelining is a technique for reorganizing loops such that each iteration in the software-pipelined code is made from instructions chosen from different iterations of the original loop. This approach is most easily understood by looking at the scheduled code for the unrolled loop, which appeared in the example in Section 2.2. The scheduler in this example essentially interleaves instructions from different loop iterations, so as to separate the dependent instructions that occur within a single loop iteration. By choosing instructions from different iterations, dependent computations are separated from one another by an entire loop body, increasing the possibility that the unrolled loop can be scheduled without stalls.

A software-pipelined loop interleaves instructions from different iterations without unrolling the loop, as illustrated in Figure H.1. This technique is the software counterpart to what Tomasulo's algorithm does in hardware. The software-pipelined loop for the earlier example would contain one load, one add, and one store, each from a different iteration. There is also some start-up code that is needed before the loop begins as well as code to finish up after the loop is completed. We will ignore these in this discussion, for simplicity.

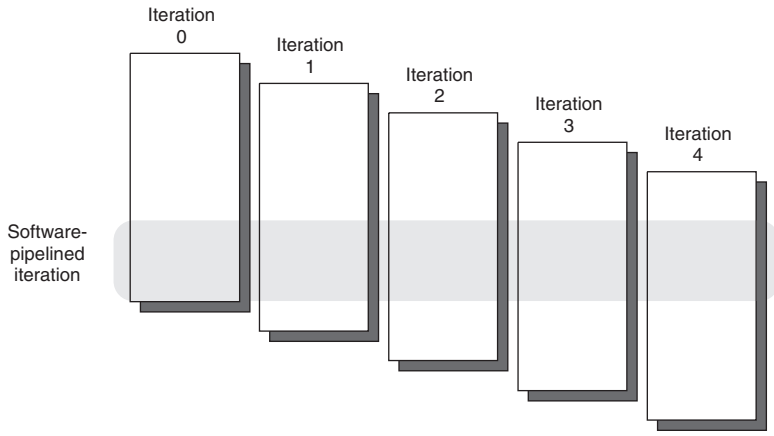


Figure H.1 A software-pipelined loop chooses instructions from different loop iterations, thus separating the dependent instructions within one iteration of the original loop. The start-up and finish-up code will correspond to the portions above and below the software-pipelined iteration.

Example Show a software-pipelined version of this loop, which increments all the elements of an array whose starting address is in R1 by the contents of F2:

```
Loop:  L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8
       BNE      R1,R2,Loop
```

You may omit the start-up and clean-up code.

Answer Software pipelining symbolically unrolls the loop and then selects instructions from each iteration. Since the unrolling is symbolic, the loop overhead instructions (the DADDUI and BNE) need not be replicated. Here's the body of the unrolled loop without overhead instructions, highlighting the instructions taken from each iteration:

```
Iteration i:    L.D      F0,0(R1)
                ADD.D    F4,F0,F2
                S.D      F4,0(R1)
Iteration i+1:  L.D      F0,0(R1)
                ADD.D    F4,F0,F2
                S.D      F4,0(R1)
Iteration i+2:  L.D      F0,0(R1)
                ADD.D    F4,F0,F2
                S.D      F4,0(R1)
```

The selected instructions from different iterations are then put together in the loop with the loop control instructions:

```
Loop:  S.D      F4,16(R1)      ;stores into M[i]
       ADD.D    F4,F0,F2       ;adds to M[i-1]
       L.D      F0,0(R1)       ;loads M[i-2]
       DADDUI   R1,R1,#-8
       BNE      R1,R2,Loop
```

This loop can be run at a rate of 5 cycles per result, ignoring the start-up and clean-up portions, and assuming that `DADDUI` is scheduled before the `ADD.D` and that the `L.D` instruction, with an adjusted offset, is placed in the branch delay slot. Because the load and store are separated by offsets of 16 (two iterations), the loop should run for two fewer iterations. Notice that the reuse of registers (e.g., F4, F0, and R1) requires the hardware to avoid the write after read (WAR) hazards that would occur in the loop. This hazard should not be a problem in this case, since no data-dependent stalls should occur.

By looking at the unrolled version we can see what the start-up code and finish-up code will need to be. For start-up, we will need to execute any instructions that correspond to iteration 1 and 2 that will not be executed. These instructions are the `L.D` for iterations 1 and 2 and the `ADD.D` for iteration 1. For the finish-up code, we need to execute any instructions that will not be executed in the final two iterations. These include the `ADD.D` for the last iteration and the `S.D` for the last two iterations.

Register management in software-pipelined loops can be tricky. The previous example is not too hard since the registers that are written on one loop iteration are read on the next. In other cases, we may need to increase the number of iterations between when we issue an instruction and when the result is used. This increase is required when there are a small number of instructions in the loop body and the latencies are large. In such cases, a combination of software pipelining and loop unrolling is needed.

Software pipelining can be thought of as *symbolic* loop unrolling. Indeed, some of the algorithms for software pipelining use loop-unrolling algorithms to figure out how to software-pipeline the loop. The major advantage of software pipelining over straight loop unrolling is that software pipelining consumes less code space. Software pipelining and loop unrolling, in addition to yielding a better scheduled inner loop, each reduce a different type of overhead. Loop unrolling reduces the overhead of the loop—the branch and counter update code. Software pipelining reduces the time when the loop is not running at peak speed to once per loop at the beginning and end. If we unroll a loop that does 100 iterations a constant number of times, say, 4, we pay the overhead 100/4 = 25 times—every time the inner unrolled loop is initiated. Figure H.2 shows this behavior graphically. Because these techniques attack two different types of overhead, the best performance can come from doing both. In practice, compilation using software pipelining is quite difficult for several reasons: Many loops require significant transformation

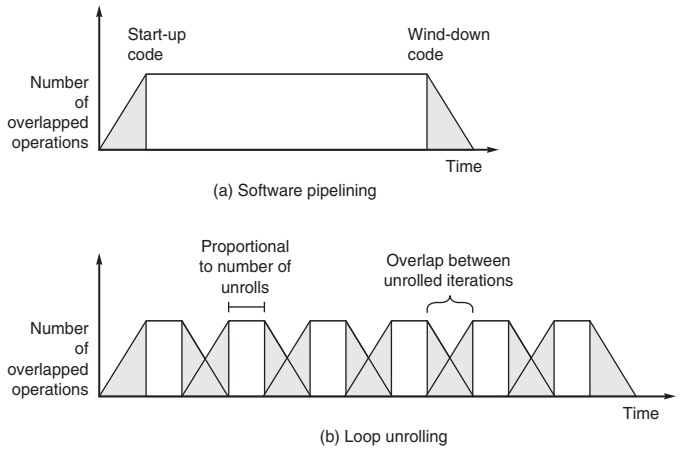


Figure H.2 The execution pattern for (a) a software-pipelined loop and (b) an unrolled loop. The shaded areas are the times when the loop is not running with maximum overlap or parallelism among instructions. This occurs once at the beginning and once at the end for the software-pipelined loop. For the unrolled loop it occurs *m/n* times if the loop has a total of *m* iterations and is unrolled *n* times. Each block represents an unroll of *n* iterations. Increasing the number of unrollings will reduce the start-up and clean-up overhead. The overhead of one iteration overlaps with the overhead of the next, thereby reducing the impact. The total area under the polygonal region in each case will be the same, since the total number of operations is just the execution rate multiplied by the time.

before they can be software pipelined, the trade-offs in terms of overhead versus efficiency of the software-pipelined loop are complex, and the issue of register management creates additional complexities. To help deal with the last two of these issues, the IA-64 added extensive hardware sport for software pipelining. Although this hardware can make it more efficient to apply software pipelining, it does not eliminate the need for complex compiler support, or the need to make difficult decisions about the best way to compile a loop.

Global Code Scheduling

In Section 3.2 we examined the use of loop unrolling and code scheduling to improve ILP. The techniques in Section 3.2 work well when the loop body is straight-line code, since the resulting unrolled loop looks like a single basic block. Similarly, software pipelining works well when the body is a single basic block, since it is easier to find the repeatable schedule. When the body of an unrolled loop contains internal control flow, however, scheduling the code is much more complex. In general, effective scheduling of a loop body with internal control flow will require moving instructions across branches, which is global code scheduling. In this section, we first examine the challenge and limitations of global code

scheduling. In Section H.4 we examine hardware support for eliminating control flow within an inner loop, then we examine two compiler techniques that can be used when eliminating the control flow is not a viable approach.

Global code scheduling aims to compact a code fragment with internal control structure into the shortest possible sequence that preserves the data and control dependences. The data dependences force a partial order on operations, while the control dependences dictate instructions across which code cannot be easily moved. Data dependences are overcome by unrolling and, in the case of memory operations, using dependence analysis to determine if two references refer to the same address. Finding the shortest possible sequence for a piece of code means finding the shortest sequence for the *critical path*, which is the longest sequence of dependent instructions.

Control dependences arising from loop branches are reduced by unrolling. Global code scheduling can reduce the effect of control dependences arising from conditional nonloop branches by moving code. Since moving code across branches will often affect the frequency of execution of such code, effectively using global code motion requires estimates of the relative frequency of different paths. Although global code motion cannot guarantee faster code, if the frequency information is accurate, the compiler can determine whether such code movement is likely to lead to faster code.

Global code motion is important since many inner loops contain conditional statements. Figure H.3 shows a typical code fragment, which may be thought of as an iteration of an unrolled loop, and highlights the more common control flow.

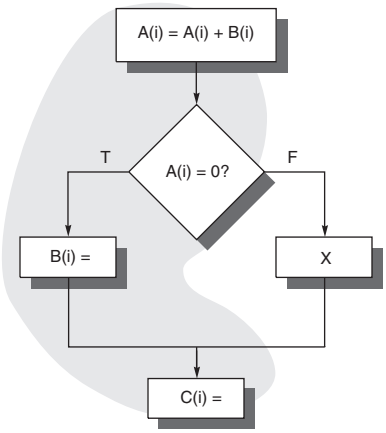


Figure H.3 A code fragment and the common path shaded with gray. Moving the assignments to B or C requires a more complex analysis than for straight-line code. In this section we focus on scheduling this code segment efficiently without hardware assistance. Predication or conditional instructions, which we discuss in the next section, provide another way to schedule this code.

Effectively scheduling this code could require that we move the assignments to B and C to earlier in the execution sequence, before the test of A. Such global code motion must satisfy a set of constraints to be legal. In addition, the movement of the code associated with B, unlike that associated with C, is speculative: It will speed the computation up only when the path containing the code would be taken.

To perform the movement of B, we must ensure that neither the data flow nor the exception behavior is changed. Compilers avoid changing the exception behavior by not moving certain classes of instructions, such as memory references, that can cause exceptions. In Section H.5, we will see how hardware support allows for more opportunities for speculative code motion and removes control dependences. Although such enhanced support for speculation can make it possible to explore more opportunities, the difficulty of choosing how to best compile the code remains complex.

How can the compiler ensure that the assignments to B and C can be moved without affecting the data flow? To see what's involved, let's look at a typical code generation sequence for the flowchart in Figure H.3. Assuming that the addresses for A, B, C are in R1, R2, and R3, respectively, here is such a sequence:

```
            LD      R4,0(R1)        ;load A
            LD      R5,0(R2)        ;load B
            DADDU   R4,R4,R5        ;Add to A
            SD      R4,0(R1)        ;Store A
            ...
            BNEZ    R4,elsepart     ;Test A
            ...                     ;then part
            SD      ...,0(R2)       ;Stores to B
            ...
            J       join            ;jump over else
elsepart:   ...                     ;else part
            X                       ;code for X
            ...
join:       ...                     ;after if
            SD      ...,0(R3)       ;store C[i]
```

Let's first consider the problem of moving the assignment to B to before the `BNEZ` instruction. Call the last instruction to assign to B before the if statement *i*. If B is referenced before it is assigned either in code segment X or after the if statement, call the referencing instruction *j*. If there is such an instruction *j*, then moving the assignment to B will change the data flow of the program. In particular, moving the assignment to B will cause *j* to become data dependent on the moved version of the assignment to B rather than on *i*, on which *j* originally depended. You could imagine more clever schemes to allow B to be moved even when the value is used: For example, in the first case, we could make a shadow copy of B before the if statement and use that shadow copy in X. Such schemes are usually avoided, both because they are complex to implement and because they will

slow down the program if the trace selected is not optimal and the operations end up requiring additional instructions to execute.

Moving the assignment to C up to before the first branch requires two steps. First, the assignment is moved over the join point of the else part into the portion corresponding to the then part. This movement makes the instructions for C control dependent on the branch and means that they will not execute if the else path, which is the infrequent path, is chosen. Hence, instructions that were data dependent on the assignment to C, and which execute after this code fragment, will be affected. To ensure the correct value is computed for such instructions, a copy is made of the instructions that compute and assign to C on the else path. Second, we can move C from the then part of the branch across the branch condition, if it does not affect any data flow into the branch condition. If C is moved to before the if test, the copy of C in the else branch can usually be eliminated, since it will be redundant.

We can see from this example that global code scheduling is subject to many constraints. This observation is what led designers to provide hardware support to make such code motion easier, and Sections H.4 and H.5 explores such support in detail.

Global code scheduling also requires complex trade-offs to make code motion decisions. For example, assuming that the assignment to B can be moved before the conditional branch (possibly with some compensation code on the alternative branch), will this movement make the code run faster? The answer is, possibly! Similarly, moving the copies of C into the if and else branches makes the code initially bigger! Only if the compiler can successfully move the computation across the if test will there be a likely benefit.

Consider the factors that the compiler would have to consider in moving the computation and assignment of B:

- What are the relative execution frequencies of the then case and the else case in the branch? If the then case is much more frequent, the code motion may be beneficial. If not, it is less likely, although not impossible, to consider moving the code.
- What is the cost of executing the computation and assignment to B above the branch? It may be that there are a number of empty instruction issue slots in the code above the branch and that the instructions for B can be placed into these slots that would otherwise go empty. This opportunity makes the computation of B "free" at least to first order.
- How will the movement of B change the execution time for the then case? If B is at the start of the critical path for the then case, moving it may be highly beneficial.
- Is B the best code fragment that can be moved above the branch? How does it compare with moving C or other statements within the then case?
- What is the cost of the compensation code that may be necessary for the else case? How effectively can this code be scheduled, and what is its impact on execution time?

As we can see from this *partial* list, global code scheduling is an extremely complex problem. The trade-offs depend on many factors, and individual decisions to globally schedule instructions are highly interdependent. Even choosing which instructions to start considering as candidates for global code motion is complex!

To try to simplify this process, several different methods for global code scheduling have been developed. The two methods we briefly explore here rely on a simple principle: Focus the attention of the compiler on a straight-line code segment representing what is estimated to be the most frequently executed code path. Unrolling is used to generate the straight-line code, but, of course, the complexity arises in how conditional branches are handled. In both cases, they are effectively straightened by choosing and scheduling the most frequent path.

Trace Scheduling: Focusing on the Critical Path

Trace scheduling is useful for processors with a large number of issues per clock, where conditional or predicated execution (see Section H.4) is inappropriate or unsupported, and where simple loop unrolling may not be sufficient by itself to uncover enough ILP to keep the processor busy. Trace scheduling is a way to organize the global code motion process, so as to simplify the code scheduling by incurring the costs of possible code motion on the less frequent paths. Because it can generate *significant* overheads on the designated infrequent path, it is best used where profile information indicates significant differences in frequency between different paths and where the profile information is highly indicative of program behavior independent of the input. Of course, this limits its effective applicability to certain classes of programs.

There are two steps to trace scheduling. The first step, called *trace selection,* tries to find a likely sequence of basic blocks whose operations will be put together into a smaller number of instructions; this sequence is called a *trace*. Loop unrolling is used to generate long traces, since loop branches are taken with high probability. Additionally, by using static branch prediction, other conditional branches are also chosen as taken or not taken, so that the resultant trace is a straight-line sequence resulting from concatenating many basic blocks. If, for example, the program fragment shown in Figure H.3 corresponds to an inner loop with the highlighted path being much more frequent, and the loop were unwound four times, the primary trace would consist of four copies of the shaded portion of the program, as shown in Figure H.4.

Once a trace is selected, the second process, called *trace compaction*, tries to squeeze the trace into a small number of wide instructions. Trace compaction is code scheduling; hence, it attempts to move operations as early as it can in a sequence (trace), packing the operations into as few wide instructions (or issue packets) as possible.

The advantage of the trace scheduling approach is that it simplifies the decisions concerning global code motion. In particular, branches are viewed as jumps into or out of the selected trace, which is assumed to be the most probable path.

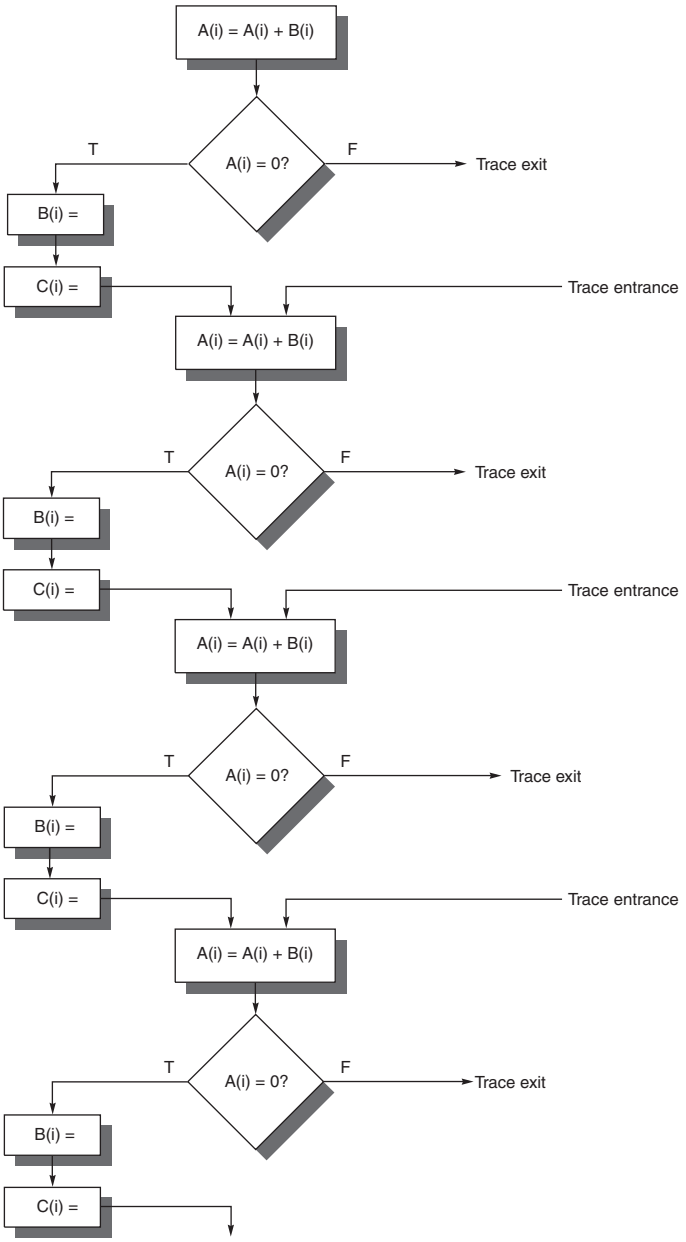


Figure H.4 This trace is obtained by assuming that the program fragment in Figure H.3 is the inner loop and unwinding it four times, treating the shaded portion in Figure H.3 as the likely path. The trace exits correspond to jumps off the frequent path, and the trace entrances correspond to returns to the trace.

When code is moved across such trace entry and exit points, additional bookkeeping code will often be needed on the entry or exit point. The key assumption is that the trace is so much more probable than the alternatives that the cost of the bookkeeping code need not be a deciding factor: If an instruction can be moved and thereby make the main trace execute faster, it is moved.

Although trace scheduling has been successfully applied to scientific code with its intensive loops and accurate profile data, it remains unclear whether this approach is suitable for programs that are less simply characterized and less loop intensive. In such programs, the significant overheads of compensation code may make trace scheduling an unattractive approach, or, at best, its effective use will be extremely complex for the compiler.

Superblocks

One of the major drawbacks of trace scheduling is that the entries and exits into the middle of the trace cause significant complications, requiring the compiler to generate and track the compensation code and often making it difficult to assess the cost of such code. *Superblocks* are formed by a process similar to that used for traces, but are a form of extended basic blocks, which are restricted to a single entry point but allow multiple exits.

Because superblocks have only a single entry point, compacting a superblock is easier than compacting a trace since only code motion across an exit need be considered. In our earlier example, we would form superblocks that contained only one entrance; hence, moving C would be easier. Furthermore, in loops that have a single loop exit based on a count (for example, a for loop with no loop exit other than the loop termination condition), the resulting superblocks have only one exit as well as one entrance. Such blocks can then be scheduled more easily.

How can a superblock with only one entrance be constructed? The answer is to use *tail duplication* to create a separate block that corresponds to the portion of the trace after the entry. In our previous example, each unrolling of the loop would create an exit from the superblock to a residual loop that handles the remaining iterations. Figure H.5 shows the superblock structure if the code fragment from Figure H.3 is treated as the body of an inner loop and unrolled four times. The residual loop handles any iterations that occur if the superblock is exited, which, in turn, occurs when the unpredicted path is selected. If the expected frequency of the residual loop were still high, a superblock could be created for that loop as well.

The superblock approach reduces the complexity of bookkeeping and scheduling versus the more general trace generation approach but may enlarge code size more than a trace-based approach. Like trace scheduling, superblock scheduling may be most appropriate when other techniques (e.g., if conversion) fail. Even in such cases, assessing the cost of code duplication may limit the usefulness of the approach and will certainly complicate the compilation process.

Loop unrolling, software pipelining, trace scheduling, and superblock scheduling all aim at trying to increase the amount of ILP that can be exploited by a processor issuing more than one instruction on every clock cycle. The effectiveness of

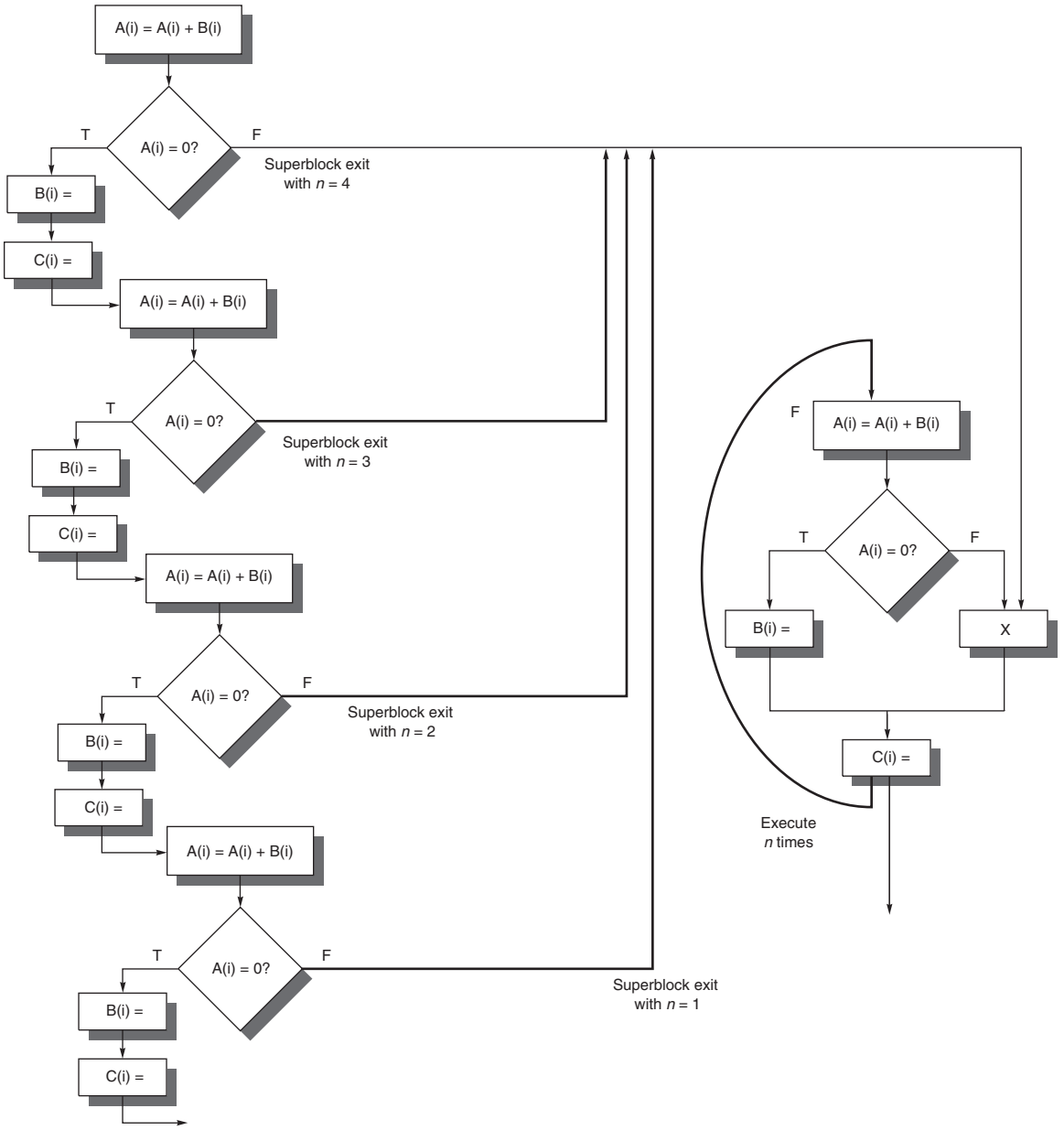


Figure H.5 This superblock results from unrolling the code in Figure H.3 four times and creating a superblock.

each of these techniques and their suitability for various architectural approaches are among the hottest topics being actively pursued by researchers and designers of high-speed processors.

H.4 Hardware Support for Exposing Parallelism: Predicated Instructions

Techniques such as loop unrolling, software pipelining, and trace scheduling can be used to increase the amount of parallelism available when the behavior of branches is fairly predictable at compile time. When the behavior of branches is not well known, compiler techniques alone may not be able to uncover much ILP. In such cases, the control dependences may severely limit the amount of parallelism that can be exploited. To overcome these problems, an architect can extend the instruction set to include *conditional* or *predicated instructions*. Such instructions can be used to eliminate branches, converting a control dependence into a data dependence and potentially improving performance. Such approaches are useful with either the hardware-intensive schemes in Chapter 3 or the software-intensive approaches discussed in this appendix, since in both cases predication can be used to eliminate branches.

The concept behind conditional instructions is quite simple: An instruction refers to a condition, which is evaluated as part of the instruction execution. If the condition is true, the instruction is executed normally; if the condition is false, the execution continues as if the instruction were a no-op. Many newer architectures include some form of conditional instructions. The most common example of such an instruction is conditional move, which moves a value from one register to another if the condition is true. Such an instruction can be used to completely eliminate a branch in simple code sequences.

Example Consider the following code:

```
if (A==0) {S=T;}
```

Assuming that registers R1, R2, and R3 hold the values of A, S, and T, respectively, show the code for this statement with the branch and with the conditional move.

Answer The straightforward code using a branch for this statement is (remember that we are assuming normal rather than delayed branches)

```
        BNEZ    R1,L
        ADDU    R2,R3,R0
L:
```

Using a conditional move that performs the move only if the third operand is equal to zero, we can implement this statement in one instruction:

```
        CMOVZ   R2,R3,R1
```

The conditional instruction allows us to convert the control dependence present in the branch-based code sequence to a data dependence. (This transformation is also used for vector computers, where it is called *if conversion*.) For a pipelined processor, this moves the place where the dependence must be resolved from near the front of the pipeline, where it is resolved for branches, to the end of the pipeline, where the register write occurs.

One obvious use for conditional move is to implement the absolute value function: `A = abs (B)`, which is implemented as `if (B<0) {A=-B;} else {A=B;}`. This if statement can be implemented as a pair of conditional moves, or as one unconditional move `(A=B)` and one conditional move `(A=-B)`.

In the example above or in the compilation of absolute value, conditional moves are used to change a control dependence into a data dependence. This enables us to eliminate the branch and possibly improve the pipeline behavior. As issue rates increase, designers are faced with one of two choices: execute multiple branches per clock cycle or find a method to eliminate branches to avoid this requirement. Handling multiple branches per clock is complex, since one branch must be control dependent on the other. The difficulty of accurately predicting two branch outcomes, updating the prediction tables, and executing the correct sequence has so far caused most designers to avoid processors that execute multiple branches per clock. Conditional moves and predicated instructions provide a way of reducing the branch pressure. In addition, a conditional move can often eliminate a branch that is hard to predict, increasing the potential gain.

Conditional moves are the simplest form of conditional or predicated instructions and, although useful for short sequences, have limitations. In particular, using conditional move to eliminate branches that guard the execution of large blocks of code can be inefficient, since many conditional moves may need to be introduced.

To remedy the inefficiency of using conditional moves, some architectures support full predication, whereby the execution of all instructions is controlled by a predicate. When the predicate is false, the instruction becomes a no-op. Full predication allows us to simply convert large blocks of code that are branch dependent. For example, an if-then-else statement within a loop can be entirely converted to predicated execution, so that the code in the then case executes only if the value of the condition is true and the code in the else case executes only if the value of the condition is false. Predication is particularly valuable with global code scheduling, since it can eliminate nonloop branches, which significantly complicate instruction scheduling.

Predicated instructions can also be used to speculatively move an instruction that is time critical, but may cause an exception if moved before a guarding branch. Although it is possible to do this with conditional move, it is more costly.

Example Here is a code sequence for a two-issue superscalar that can issue a combination of one memory reference and one ALU operation, or a branch by itself, every cycle:

First instruction slot		Second instruction slot
LW	R1,40 (R2)	ADD R3,R4,R5
		ADD R6,R3,R7
BEQZ	R10,L	
LW	R8,0 (R10)	
LW	R9,0(R8)	

This sequence wastes a memory operation slot in the second cycle and will incur a data dependence stall if the branch is not taken, since the second LW after the branch depends on the prior load. Show how the code can be improved using a predicated form of LW.

Answer Call the predicated version load word LWC and assume the load occurs unless the third operand is 0. The LW immediately following the branch can be converted to an LWC and moved up to the second issue slot:

First instruction slot		Second instruction slot
LW	R1,40(R2)	ADD R3,R4,R5
LWC	R8,0(R10),R10	ADD R6,R3,R7
BEQZ	R10,L	
LW	R9,0(R8)	

This improves the execution time by several cycles since it eliminates one instruction issue slot and reduces the pipeline stall for the last instruction in the sequence. Of course, if the compiler mispredicted the branch, the predicated instruction will have no effect and will not improve the running time. This is why the transformation is speculative.

If the sequence following the branch were short, the entire block of code might be converted to predicated execution and the branch eliminated.

When we convert an entire code segment to predicated execution or speculatively move an instruction and make it predicted, we remove a control dependence. Correct code generation and the conditional execution of predicated instructions ensure that we maintain the data flow enforced by the branch. To ensure that the exception behavior is also maintained, a predicated instruction must not generate an exception if the predicate is false. The property of not causing exceptions is

quite critical, as the previous example shows: If register R10 contains zero, the instruction LW R8,0(R10) executed unconditionally is likely to cause a protection exception, and this exception should not occur. Of course, if the condition is satisfied (i.e., R10 is not zero), the LW may still cause a legal and resumable exception (e.g., a page fault), and the hardware must take the exception when it knows that the controlling condition is true.

The major complication in implementing predicated instructions is deciding when to annul an instruction. Predicated instructions may either be annulled during instruction issue or later in the pipeline before they commit any results or raise an exception. Each choice has a disadvantage. If predicated instructions are annulled early in the pipeline, the value of the controlling condition must be known early to prevent a stall for a data hazard. Since data-dependent branch conditions, which tend to be less predictable, are candidates for conversion to predicated execution, this choice can lead to more pipeline stalls. Because of this potential for data hazard stalls, no design with predicated execution (or conditional move) annuls instructions early. Instead, all existing processors annul instructions later in the pipeline, which means that annulled instructions will consume functional unit resources and potentially have a negative impact on performance. A variety of other pipeline implementation techniques, such as forwarding, interact with predicated instructions, further complicating the implementation.

Predicated or conditional instructions are extremely useful for implementing short alternative control flows, for eliminating some unpredictable branches, and for reducing the overhead of global code scheduling. Nonetheless, the usefulness of conditional instructions is limited by several factors:

- Predicated instructions that are annulled (i.e., whose conditions are false) still take some processor resources. An annulled predicated instruction requires fetch resources at a minimum, and in most processors functional unit execution time. Therefore, moving an instruction across a branch and making it conditional will slow the program down whenever the moved instruction would not have been normally executed. Likewise, predicating a control-dependent portion of code and eliminating a branch may slow down the processor if that code would not have been executed. An important exception to these situations occurs when the cycles used by the moved instruction when it is not performed would have been idle anyway (as in the earlier superscalar example). Moving an instruction across a branch or converting a code segment to predicated execution is essentially speculating on the outcome of the branch. Conditional instructions make this easier but do not eliminate the execution time taken by an incorrect guess. In simple cases, where we trade a conditional move for a branch and a move, using conditional moves or predication is almost always better. When longer code sequences are made conditional, the benefits are more limited.
- Predicated instructions are most useful when the predicate can be evaluated early. If the condition evaluation and predicated instructions cannot be

separated (because of data dependences in determining the condition), then a conditional instruction may result in a stall for a data hazard. With branch prediction and speculation, such stalls can be avoided, at least when the branches are predicted accurately.

- The use of conditional instructions can be limited when the control flow involves more than a simple alternative sequence. For example, moving an instruction across multiple branches requires making it conditional on both branches, which requires two conditions to be specified or requires additional instructions to compute the controlling predicate. If such capabilities are not present, the overhead of if conversion will be larger, reducing its advantage.
- Conditional instructions may have some speed penalty compared with unconditional instructions. This may show up as a higher cycle count for such instructions or a slower clock rate overall. If conditional instructions are more expensive, they will need to be used judiciously.

For these reasons, many architectures have included a few simple conditional instructions (with conditional move being the most frequent), but only a few architectures include conditional versions for the majority of the instructions. The MIPS, Alpha, PowerPC, SPARC, and Intel x86 (as defined in the Pentium processor) all support conditional move. The IA-64 architecture supports full predication for all instructions, as we will see in Section H.6.

H.5 Hardware Support for Compiler Speculation

As we saw in Chapter 3, many programs have branches that can be accurately predicted at compile time either from the program structure or by using a profile. In such cases, the compiler may want to speculate either to improve the scheduling or to increase the issue rate. Predicated instructions provide one method to speculate, but they are really more useful when control dependences can be completely eliminated by if conversion. In many cases, we would like to move speculated instructions not only before the branch but also before the condition evaluation, and predication cannot achieve this.

To speculate ambitiously requires three capabilities:

1. The ability of the compiler to find instructions that, with the possible use of register renaming, can be speculatively moved and not affect the program data flow
2. The ability to ignore exceptions in speculated instructions, until we know that such exceptions should really occur
3. The ability to speculatively interchange loads and stores, or stores and stores, which may have address conflicts

The first of these is a compiler capability, while the last two require hardware support, which we explore next.

Hardware Support for Preserving Exception Behavior

To speculate ambitiously, we must be able to move any type of instruction and still preserve its exception behavior. The key to being able to do this is to observe that the results of a speculated sequence that is mispredicted will not be used in the final computation, and such a speculated instruction should not cause an exception.

There are four methods that have been investigated for supporting more ambitious speculation without introducing erroneous exception behavior:

1. The hardware and operating system cooperatively ignore exceptions for speculative instructions. As we will see later, this approach preserves exception behavior for correct programs, but not for incorrect ones. This approach may be viewed as unacceptable for some programs, but it has been used, under program control, as a "fast mode" in several processors.
2. Speculative instructions that never raise exceptions are used, and checks are introduced to determine when an exception should occur.
3. A set of status bits, called *poison bits*, are attached to the result registers written by speculated instructions when the instructions cause exceptions. The poison bits cause a fault when a normal instruction attempts to use the register.
4. A mechanism is provided to indicate that an instruction is speculative, and the hardware buffers the instruction result until it is certain that the instruction is no longer speculative.

To explain these schemes, we need to distinguish between exceptions that indicate a program error and would normally cause termination, such as a memory protection violation, and those that are handled and normally resumed, such as a page fault. Exceptions that can be resumed can be accepted and processed for speculative instructions just as if they were normal instructions. If the speculative instruction should not have been executed, handling the unneeded exception may have some negative performance effects, but it cannot cause incorrect execution. The cost of these exceptions may be high, however, and some processors use hardware support to avoid taking such exceptions, just as processors with hardware speculation may take some exceptions in speculative mode, while avoiding others until an instruction is known not to be speculative.

Exceptions that indicate a program error should not occur in correct programs, and the result of a program that gets such an exception is not well defined, except perhaps when the program is running in a debugging mode. If such exceptions arise in speculated instructions, we cannot take the exception until we know that the instruction is no longer speculative.

In the simplest method for preserving exceptions, the hardware and the operating system simply handle all resumable exceptions when the exception occurs and simply return an undefined value for any exception that would cause termination. If the instruction generating the terminating exception was not speculative, then the program is in error. Note that instead of terminating the program, the

program is allowed to continue, although it will almost certainly generate incorrect results. If the instruction generating the terminating exception is speculative, then the program may be correct and the speculative result will simply be unused; thus, returning an undefined value for the instruction cannot be harmful. This scheme can never cause a correct program to fail, no matter how much speculation is done. An incorrect program, which formerly might have received a terminating exception, will get an incorrect result. This is acceptable for some programs, assuming the compiler can also generate a normal version of the program, which does not speculate and can receive a terminating exception.

Example Consider that the following code fragment from an if-then-else statement of the form

```
if (A==0) A = B; else A = A+4;
```

where A is at 0(R3) and B is at 0(R2):

```
        LD      R1,0(R3)    ;load A
        BNEZ    R1,L1       ;test A
        LD      R1,0(R2)    ;then clause
        J       L2          ;skip else
L1:     DADDI   R1,R1,#4    ;else clause
L2:     SD      R1,0(R3)    ;store A
```

Assume that the then clause is *almost always* executed. Compile the code using compiler-based speculation. Assume R14 is unused and available.

Answer Here is the new code:

```
        LD      R1,0(R3)    ;load A
        LD      R14,0(R2)   ;speculative load B
        BEQZ    R1,L3       ;other branch of the if
        DADDI   R14,R1,#4   ;the else clause
L3:     SD      R14,0(R3)   ;nonspeculative store
```

The then clause is completely speculated. We introduce a temporary register to avoid destroying R1 when B is loaded; if the load is speculative, R14 will be useless. After the entire code segment is executed, A will be in R14. The else clause could have also been compiled speculatively with a conditional move, but if the branch is highly predictable and low cost, this might slow the code down, since two extra instructions would always be executed as opposed to one branch.

In such a scheme, it is not necessary to know that an instruction is speculative. Indeed, it is helpful only when a program is in error and receives a terminating exception on a normal instruction; in such cases, if the instruction were not marked as speculative, the program could be terminated.

In this method for handling speculation, as in the next one, renaming will often be needed to prevent speculative instructions from destroying live values. Renaming is usually restricted to register values. Because of this restriction, the targets of stores cannot be destroyed and stores cannot be speculative. The small number of registers and the cost of spilling will act as one constraint on the amount of speculation. Of course, the major constraint remains the cost of executing speculative instructions when the compiler's branch prediction is incorrect.

A second approach to preserving exception behavior when speculating introduces speculative versions of instructions that do not generate terminating exceptions and instructions to check for such exceptions. This combination preserves the exception behavior exactly.

Example Show how the previous example can be coded using a speculative load (`sLD`) and a speculation check instruction (`SPECCK`) to completely preserve exception behavior. Assume `R14` is unused and available.

Answer Here is the code that achieves this:

```
        LD      R1,0(R3)    ;load A
        sLD     R14,0(R2)   ;speculative, no termination
        BNEZ    R1,L1       ;test A
        SPECCK  0(R2)       ;perform speculation check
        J       L2          ;skip else
L1:     DADDI   R14,R1,#4   ;else clause
L2:     SD      R14,0(R3)   ;store A
```

Notice that the speculation check requires that we maintain a basic block for the then case. If we had speculated only a portion of the then case, then a basic block representing the then case would exist in any event. More importantly, notice that checking for a possible exception requires extra code.

A third approach for preserving exception behavior tracks exceptions as they occur but postpones any terminating exception until a value is actually used, preserving the occurrence of the exception, although not in a completely precise fashion. The scheme is simple: A poison bit is added to every register, and another bit is added to every instruction to indicate whether the instruction is speculative. The poison bit of the destination register is set whenever a speculative instruction results in a terminating exception; all other exceptions are handled immediately. If a speculative instruction uses a register with a poison bit turned on, the destination register of the instruction simply has its poison bit turned on. If a normal instruction attempts to use a register source with its poison bit turned on, the instruction causes a fault. In this way, any program that would have generated an exception still generates one, albeit at the first instance where a result is used by an instruction that is not speculative. Since poison bits exist only on register

values and not memory values, stores are never speculative and thus trap if either operand is "poison."

Example Consider the code fragment from page H-29 and show how it would be compiled with speculative instructions and poison bits. Show where an exception for the speculative memory reference would be recognized. Assume R14 is unused and available.

Answer Here is the code (an s preceding the opcode indicates a speculative instruction):

```
        LD      R1,0(R3)      ;load A
        sLD     R14,0(R2)     ;speculative load B
        BEQZ    R1,L3         ;
        DADDI   R14,R1,#4     ;
L3:     SD      R14,0(R3)     ;exception for speculative LW
```

If the speculative sLD generates a terminating exception, the poison bit of R14 will be turned on. When the nonspeculative SW instruction occurs, it will raise an exception if the poison bit for R14 is on.

One complication that must be overcome is how the OS saves the user registers on a context switch if the poison bit is set. A special instruction is needed to save and reset the state of the poison bits to avoid this problem.

The fourth and final approach listed earlier relies on a hardware mechanism that operates like a reorder buffer. In such an approach, instructions are marked by the compiler as speculative and include an indicator of how many branches the instruction was speculatively moved across and what branch action (taken/not taken) the compiler assumed. This last piece of information basically tells the hardware the location of the code block where the speculated instruction originally was. In practice, most of the benefit of speculation is gained by allowing movement across a single branch; thus, only 1 bit saying whether the speculated instruction came from the taken or not taken path is required. Alternatively, the original location of the speculative instruction is marked by a *sentinel*, which tells the hardware that the earlier speculative instruction is no longer speculative and values may be committed.

All instructions are placed in a reorder buffer when issued and are forced to commit in order, as in a hardware speculation approach. (Notice, though, that no actual speculative branch prediction or dynamic scheduling occurs.) The reorder buffer tracks when instructions are ready to commit and delays the "write-back" portion of any speculative instruction. Speculative instructions are not allowed to commit until the branches that have been speculatively moved over are also ready to commit, or, alternatively, until the corresponding sentinel is reached. At that point, we know whether the speculated instruction should have been executed or not. If it should have been executed and it generated a terminating

exception, then we know that the program should be terminated. If the instruction should not have been executed, then the exception can be ignored. Notice that the compiler, rather than the hardware, has the job of register renaming to ensure correct usage of the speculated result, as well as correct program execution.

Hardware Support for Memory Reference Speculation

Moving loads across stores is usually done when the compiler is certain the addresses do not conflict. As we saw with the examples in Section 3.2, such transformations are critical to reducing the critical path length of a code segment. To allow the compiler to undertake such code motion when it cannot be absolutely certain that such a movement is correct, a special instruction to check for address conflicts can be included in the architecture. The special instruction is left at the original location of the load instruction (and acts like a guardian), and the load is moved up across one or more stores.

When a speculated load is executed, the hardware saves the address of the accessed memory location. If a subsequent store changes the location before the check instruction, then the speculation has failed. If the location has not been touched, then the speculation is successful. Speculation failure can be handled in two ways. If only the load instruction was speculated, then it suffices to redo the load at the point of the check instruction (which could supply the target register in addition to the memory address). If additional instructions that depended on the load were also speculated, then a fix-up sequence that reexecutes all the speculated instructions starting with the load is needed. In this case, the check instruction specifies the address where the fix-up code is located.

In this section, we have seen a variety of hardware assist mechanisms. Such mechanisms are key to achieving good support with the compiler-intensive approaches of Chapter 3 and this appendix. In addition, several of them can be easily integrated in the hardware-intensive approaches of Chapter 3 and provide additional benefits.

H.6 The Intel IA-64 Architecture and Itanium Processor

This section is an overview of the Intel IA-64 architecture, the most advanced VLIW-style processor, and its implementation in the Itanium processor.

The Intel IA-64 Instruction Set Architecture

The IA-64 is a RISC-style, register-register instruction set, but with many novel features designed to support compiler-based exploitation of ILP. Our focus here is on the unique aspects of the IA-64 ISA. Most of these aspects have been discussed already in this appendix, including predication, compiler-based parallelism detection, and support for memory reference speculation.

When they announced the IA-64 architecture, HP and Intel introduced the term EPIC (Explicitly Parallel Instruction Computer) to distinguish this new architectural approach from the earlier VLIW architectures and from other RISC architectures. Although VLIW and EPIC architectures share many features, the EPIC approach includes several concepts that extend the earlier VLIW approach. These extensions fall into two main areas:

1. EPIC has greater flexibility in indicating parallelism among instructions and in instruction formats. Rather than relying on a fixed instruction format where all operations in the instruction must be capable of being executed in parallel and where the format is completely rigid, EPIC uses explicit indicators of possible instruction dependence as well as a variety of instruction formats. This EPIC approach can express parallelism more flexibly than the more rigid VLIW method and can reduce the increases in code size caused by the typically inflexible VLIW instruction format.
2. EPIC has more extensive support for software speculation than the earlier VLIW schemes that had only minimal support.

In addition, the IA-64 architecture includes a variety of features to improve performance, such as register windows and a rotating floating-point register (FPR) stack.

The IA-64 Register Model

The components of the IA-64 register state are

- 128 64-bit general-purpose registers, which as we will see shortly are actually 65 bits wide
- 128 82-bit floating-point registers, which provide two extra exponent bits over the standard 80-bit IEEE format
- 64 1-bit predicate registers
- 8 64-bit branch registers, which are used for indirect branches
- A variety of registers used for system control, memory mapping, performance counters, and communication with the OS

The integer registers are configured to help accelerate procedure calls using a register stack mechanism similar to that developed in the Berkeley RISC-I processor and used in the SPARC architecture. Registers 0 to 31 are always accessible and are addressed as 0 to 31. Registers 32 to 128 are used as a register stack, and each procedure is allocated a set of registers (from 0 to 96) for its use. The new register stack frame is created for a called procedure by renaming the registers in hardware; a special register called the current frame pointer (CFM) points to the set of registers to be used by a given procedure. The frame consists of two parts: the local area and the output area. The local area is used for local storage, while

the output area is used to pass values to any called procedure. The `alloc` instruction specifies the size of these areas. Only the integer registers have register stack support.

On a procedure call, the CFM pointer is updated so that R32 of the called procedure points to the first register of the output area of the called procedure. This update enables the parameters of the caller to be passed into the addressable registers of the callee. The callee executes an `alloc` instruction to allocate both the number of required local registers, which include the output registers of the caller, and the number of output registers needed for parameter passing to a called procedure. Special load and store instructions are available for saving and restoring the register stack, and special hardware (called the *register stack engine*) handles overflow of the register stack.

In addition to the integer registers, there are three other sets of registers: the floating-point registers, the predicate registers, and the branch registers. The floating-point registers are used for floating-point data, and the branch registers are used to hold branch destination addresses for indirect branches. The predication registers hold predicates, which control the execution of predicated instructions; we describe the predication mechanism later in this section.

Both the integer and floating-point registers support register rotation for registers 32 to 128. Register rotation is designed to ease the task of allocating registers in software-pipelined loops, a problem that we discussed in Section H.3. In addition, when combined with the use of predication, it is possible to avoid the need for unrolling and for separate prologue and epilogue code for a software-pipelined loop. This capability reduces the code expansion incurred to use software pipelining and makes the technique usable for loops with smaller numbers of iterations, where the overheads would traditionally negate many of the advantages.

Instruction Format and Support for Explicit Parallelism

The IA-64 architecture is designed to achieve the major benefits of a VLIW approach—implicit parallelism among operations in an instruction and fixed formatting of the operation fields—while maintaining greater flexibility than a VLIW normally allows. This combination is achieved by relying on the compiler to detect ILP and schedule instructions into parallel instruction slots, but adding flexibility in the formatting of instructions and allowing the compiler to indicate when an instruction cannot be executed in parallel with its successors.

The IA-64 architecture uses two different concepts to achieve the benefits of implicit parallelism and ease of instruction decode. Implicit parallelism is achieved by placing instructions into *instruction groups*, while the fixed formatting of multiple instructions is achieved through the introduction of a concept called a *bundle*, which contains three instructions. Let's start by defining an instruction group.

An instruction group is a sequence of consecutive instructions with no register data dependences among them (there are a few minor exceptions). All the instructions in a group could be executed in parallel, if sufficient hardware resources existed and if any dependences through memory were preserved. An instruction group can be arbitrarily long, but the compiler must *explicitly* indicate the

Execution unit slot	Instruction type	Instruction description	Example instructions
I-unit	A	Integer ALU	Add, subtract, and, or, compare
	I	Non-ALU	integer Integer and multimedia shifts, bit tests, moves
M-unit	A	Integer ALU	Add, subtract, and, or, compare
	M	Memory access	Loads and stores for integer/FP registers
F-unit	F	Floating point	Floating-point instructions
B-unit	B	Branches	Conditional branches, calls, loop branches
L + X	L + X	Extended	Extended immediates, stops and no-ops

Figure H.6 The five execution unit slots in the IA-64 architecture and what instructions types they may hold are shown. A-type instructions, which correspond to integer ALU instructions, may be placed in either an I-unit or M-unit slot. L + X slots are special, as they occupy two instruction slots; L + X instructions are used to encode 64-bit immediates and a few special instructions. L + X instructions are executed either by the I-unit or the B-unit.

boundary between one instruction group and another. This boundary is indicated by placing a *stop* between two instructions that belong to different groups. To understand how stops are indicated, we must first explain how instructions are placed into bundles.

IA-64 instructions are encoded in bundles, which are 128 bits wide. Each bundle consists of a 5-bit template field and three instructions, each 41 bits in length. (Actually, the 41-bit quantities are not truly instructions, since they can only be interpreted in conjunction with the template field. The name *syllable* is sometimes used for these operations. For simplicity, we will continue to use the term "instruction.") To simplify the decoding and instruction issue process, the template field of a bundle specifies what types of execution units each instruction in the bundle requires. Figure H.6 shows the five different execution unit types and describes what instruction classes they may hold, together with some examples.

The 5-bit template field within each bundle describes *both* the presence of any stops associated with the bundle and the execution unit type required by each instruction within the bundle. Figure H.7 shows the possible formats that the template field encodes and the position of any stops it specifies. The bundle formats can specify only a subset of all possible combinations of instruction types and stops. To see how the bundle works, let's consider an example.

Example Unroll the array increment example, `x[i] = x[i] + s`, seven times and place the instructions into bundles, first ignoring pipeline latencies (to minimize the number of bundles) and then scheduling the code to minimize stalls. In scheduling the code assume one bundle executes per clock and that any stalls cause the entire bundle to

Template	Slot 0	Slot 1	Slot 2
0	M	I	I
1	M	I	I
2	M	I	I
3	M	I	I
4	M	L	X
5	M	L	X
8	M	M	I
9	M	M	I
10	M	M	I
11	M	M	I
12	M	F	I
13	M	F	I
14	M	M	F
15	M	M	F
16	M	I	B
17	M	I	B
18	M	B	B
19	M	B	B
22	B	B	B
23	B	B	B
24	M	M	B
25	M	M	B
28	M	F	B
29	M	F	B

Figure H.7 The 24 possible template values (8 possible values are reserved) and the instruction slots and stops for each format. Stops are indicated by heavy lines and may appear within and/or at the end of the bundle. For example, template 9 specifies that the instruction slots are M, M, and I (in that order) and that the only stop is between this bundle and the next. Template 11 has the same type of instruction slots but also includes a stop after the first slot. The L + X format is used when slot 1 is L and slot 2 is X.

be stalled. Use the pipeline latencies from Figure 3.2. Use MIPS instruction mnemonics for simplicity.

Answer The two different versions are shown in Figure H.8. Although the latencies are different from those in Itanium, the most common bundle, MMF, must be issued by itself in Itanium, just as our example assumes.

Bundle template	Slot 0	Slot 1	Slot 2	Execute cycle (1 bundle/ cycle)
9: M M I	L.D F0,0(R1)	L.D F6,-8(R1)		1
14: M M F	L.D F10,-16(R1)	L.D F14,-24(R1)	ADD.D F4,F0,F2	3
15: M M F	L.D F18,-32(R1)	L.D F22,-40(R1)	ADD.D F8,F6,F2	4
15: M M F	L.D F26,-48(R1)	S.D F4,0(R1)	ADD.D F12,F10,F2	6
15: M M F	S.D F8,-8(R1)	S.D F12,-16(R1)	ADD.D F16,F14,F2	9
15: M M F	S.D F16,-24(R1)		ADD.D F20,F18,F2	12
15: M M F	S.D F20,-32(R1)		ADD.D F24,F22,F2	15
15: M M F	S.D F24,-40(R1)		ADD.D F28,F26,F2	18
16: M I B	S.D F28,-48(R1)	DADDUI R1,R1,#-56	BNE R1,R2,Loop	21

(a) The code scheduled to minimize the number of bundles

Bundle template	Slot 0	Slot 1	Slot 2	Execute cycle (1 bundle/ cycle)
8: M M I	L.D F0,0(R1)	L.D F6,-8(R1)		1
9: M M I	L.D F10,-16(R1)	L.D F14,-24(R1)		2
14: M M F	L.D F18,-32(R1)	L.D F22,-40(R1)	ADD.D F4,F0,F2	3
14: M M F	L.D F26,-48(R1)		ADD.D F8,F6,F2	4
15: M M F			ADD.D F12,F10,F2	5
14: M M F		S.D F4,0(R1)	ADD.D F16,F14,F2	6
14: M M F		S.D F8,-8(R1)	ADD.D F20,F18,F2	7
15: M M F		S.D F12,-16(R1)	ADD.D F24,F22,F2	8
14: M M F		S.D F16,-24(R1)	ADD.D F28,F26,F2	9
9: M M I	S.D F20,-32(R1)	S.D F24,-40(R1)		11
16: M I B	S.D F28,-48(R1)	DADDUI R1,R1,#-56	BNE R1,R2,Loop	12

(b) The code scheduled to minimize the number of cycles assuming one bundle executed per cycle

Figure H.8 The IA-64 instructions, including bundle bits and stops, for the unrolled version of x[i] = x[i] + s, when unrolled seven times and scheduled (a) to minimize the number of instruction bundles and (b) to minimize the number of cycles (assuming that a hazard stalls an entire bundle). Blank entries indicate unused slots, which are encoded as no-ops. The absence of stops indicates that some bundles could be executed in parallel. Minimizing the number of bundles yields 9 bundles versus the 11 needed to minimize the number of cycles. The scheduled version executes in just over half the number of cycles. Version (a) fills 85% of the instruction slots, while (b) fills 70%. The number of empty slots in the scheduled code and the use of bundles may lead to code sizes that are much larger than other RISC architectures. Note that the branch in the last bundle in both sequences depends on the DADD in the same bundle. In the IA-64 instruction set, this sequence would be coded as a setting of a predication register and a branch that would be predicated on that register. Normally, such dependent operations could not occur in the same bundle, but this case is one of the exceptions mentioned earlier.

Instruction Set Basics

Before turning to the special support for speculation, we briefly discuss the major instruction encodings and survey the instructions in each of the five primary instruction classes (A, I, M, F, and B). Each IA-64 instruction is 41 bits in length. The high-order 4 bits, together with the bundle bits that specify the execution unit slot, are used as the major opcode. (That is, the 4-bit opcode field is reused across the execution field slots, and it is appropriate to think of the opcode as being 4 bits plus the M, F, I, B, L + X designation.) The low-order 6 bits of every instruction are used for specifying the predicate register that guards the instruction (see the next subsection).

Figure H.9 summarizes most of the major instruction formats, other than the multimedia instructions, and gives examples of the instructions encoded for each format.

Predication and Speculation Support

The IA-64 architecture provides comprehensive support for predication: Nearly every instruction in the IA-64 architecture can be predicated. An instruction is predicated by specifying a predicate register, whose identity is placed in the lower 6 bits of each instruction field. Because nearly all instructions can be predicated, both if conversion and code motion have lower overhead than they would with only limited support for conditional instructions. One consequence of full predication is that a conditional branch is simply a branch with a guarding predicate!

Predicate registers are set using compare or test instructions. A compare instruction specifies one of ten different comparison tests and two predicate registers as destinations. The two predicate registers are written either with the result of the comparison (0 or 1) and the complement, or with some logical function that combines the two tests (such as `and`) `and` the complement. This capability allows multiple comparisons to be done in one instruction.

Speculation support in the IA-64 architecture consists of separate support for control speculation, which deals with deferring exception for speculated instructions, and memory reference speculation, which supports speculation of load instructions.

Deferred exception handling for speculative instructions is supported by providing the equivalent of poison bits. For the general-purpose registers (GPRs), these bits are called NaTs (Not a Thing), and this extra bit makes the GPRs effectively 65 bits wide. For the FP registers this capability is obtained using a special value, NaTVal (Not a Thing Value); this value is encoded using a significand of 0 and an exponent outside of the IEEE range. Only speculative load instructions generate such values, but all instructions that do not affect memory will cause a NaT or NaTVal to be propagated to the result register. (There are both speculative and non-speculative loads; the latter can only raise immediate exceptions and cannot defer them.) Floating-point exceptions are not handled through this mechanism but instead use floating-point status registers to record exceptions.

Instruction type	Number of formats	Representative instructions	Extra opcode bits	GPRs/ FPRs	Immediate bits	Other/comment
A	8	Add, subtract, and, or	9	3	0	
		Shift left and add	7	3	0	2-bit shift count
		ALU immediates	9	2	8	
		Add immediate	3	2	14	
		Add immediate	0	2	22	
		Compare	4	2	0	2 predicate register destinations
		Compare immediate	3	1	8	2 predicate register destinations
I	29	Shift R/L variable	9	3	0	Many multimedia instructions use this format.
		Test bit	6	3	6-bit field specifier	2 predicate register destinations
		Move to BR	6	1	9-bit branch predict	Branch register specifier
M	46	Integer/FP load and store, line prefetch	10	2	0	Speculative/ nonspeculative
		Integer/FP load and store, and line prefetch and post-increment by immediate	9	2	8	Speculative/ nonspeculative
		Integer/FP load prefetch and register postincrement	10	3		Speculative/ nonspeculative
		Integer/FP speculation check	3	1	21 in two fields	
B	9	PC-relative branch, counted branch	7	0	21	
		PC-relative call	4	0	21	1 branch register
F	15	FP arithmetic	2	4		
		FP compare	2	2		2 6-bit predicate regs
L + X	4	Move immediate long	2	1	64	

Figure H.9 A summary of some of the instruction formats of the IA-64 ISA. The major opcode bits and the guarding predication register specifier add 10 bits to every instruction. The number of formats indicated for each instruction class in the second column (a total of 111) is a strict interpretation: A different use of a field, even of the same size, is considered a different format. The number of formats that actually have *different field sizes* is one-third to one-half as large. Some instructions have unused bits that are reserved; we have not included those in this table. Immediate bits include the sign bit. The branch instructions include prediction bits, which are used when the predictor does not have a valid prediction. Only one of the many formats for the multimedia instructions is shown in this table.

A deferred exception can be resolved in two different ways. First, if a nonspeculative instruction, such as a store, receives a NaT or NaTVal as a source operand, it generates an immediate and unrecoverable exception. Alternatively, a `chk.s` instruction can be used to detect the presence of NaT or NaTVal and branch to a routine designed by the compiler to recover from the speculative operation. Such a recovery approach makes more sense for memory reference speculation.

The inability to store the contents of instructions with a NaT or NaTVal set would make it impossible for the OS to save the state of the processor. Thus, IA-64 includes special instructions to save and restore registers that do not cause an exception for a NaT or NaTVal and also save and restore the NaT bits.

Memory reference support in the IA-64 uses a concept called *advanced loads*. An advanced load is a load that has been speculatively moved above store instructions on which it is potentially dependent. To speculatively perform a load, the `ld.a` (for advanced load) instruction is used. Executing this instruction creates an entry in a special table, called the *ALAT*. The ALAT stores both the register destination of the load and the address of the accessed memory location. When a store is executed, an associative lookup against the active ALAT entries is performed. If there is an ALAT entry with the same memory address as the store, the ALAT entry is marked as invalid.

Before any nonspeculative instruction (i.e., a store) uses the value generated by an advanced load or a value derived from the result of an advanced load, an explicit check is required. The check specifies the destination register of the advanced load. If the ALAT for that register is still valid, the speculation was legal and the only effect of the check is to clear the ALAT entry. If the check fails, the action taken depends on which of two different types of checks was employed. The first type of check is an instruction `ld.c`, which simply causes the data to be reloaded from memory at that point. An `ld.c` instruction is used when *only* the load is advanced. The alternative form of a check, `chk.a`, specifies the address of a fix-up routine that is used to reexecute the load *and any other* speculated code that depended on the value of the load.

The Itanium 2 Processor

The Itanium 2 processor is the second implementation of the IA-64 architecture. The first version, Itanium 1, became available in 2001 with an 800 MHz clock. The Itanium 2, first delivered in 2003, had a maximum clock rate in 2005 of 1.6 GHz. The two processors are very similar, with some differences in the pipeline structure and greater differences in the memory hierarchies. The Itanium 2 is about four times faster than the Itanium 1. This performance improvement comes from a doubling of the clock rate, a more aggressive memory hierarchy, additional functional units that improve instruction throughput, more complete bypassing, a

shorter pipeline that reduces some stalls, and a more mature compiler system. During roughly the same period that elapsed from the Itanium 1 to Itanium 2, the Pentium processors improved by slightly more than a factor of three. The greater improvement for the Itanium is reasonable given the novelty of the architecture and software system versus the more established IA-32 implementations.

The Itanium 2 can fetch and issue two bundles, or up to six instructions, per clock. The Itanium 2 uses a three-level memory hierarchy all on-chip. The first level uses split instruction and data caches, each 16 KB; floating-point data are not placed in the first-level cache. The second and third levels are unified caches of 256 KB and of 3 MB to 9 MB, respectively.

Functional Units and Instruction Issue

There are 11 functional units in the Itanium 2 processor: two I-units, four M-units (two for loads and two for stores), three B-units, and two F-units. All the functional units are pipelined. Figure H.10 gives the pipeline latencies for some typical instructions. In addition, when a result is bypassed from one unit to another, there is usually at least one additional cycle of delay.

Itanium 2 can issue up to six instructions per clock from two bundles. In the worst case, if a bundle is split when it is issued, the hardware could see as few as four instructions: one from the first bundle to be executed and three from the second bundle. Instructions are allocated to functional units based on the bundle bits, ignoring the presence of no-ops or predicated instructions with untrue predicates. In addition, when issue to a functional unit is blocked because the next instruction to be issued needs an already committed unit, the resulting bundle is split. A split bundle still occupies one of the two bundle slots, even if it has only one instruction remaining.

Instruction	Latency
Integer load	1
Floating-point load	5–9
Correctly predicted taken branch	0–3
Mispredicted branch	6
Integer ALU operations	0
FP arithmetic	4

Figure H.10 The latency of some typical instructions on Itanium 2. The latency is defined as the smallest number of intervening instructions between two dependent instructions. Integer load latency assumes a hit in the first-level cache. FP loads always bypass the primary cache, so the latency is equal to the access time of the second-level cache. There are some minor restrictions for some of the functional units, but these primarily involve the execution of infrequent instructions.

The Itanium 2 processor uses an eight-stage pipeline divided into four major parts:

- *Front-end (stages IPG and Rotate)*—Prefetches up to 32 bytes per clock (two bundles) into a prefetch buffer, which can hold up to eight bundles (24 instructions). Branch prediction is done using a multilevel adaptive predictor like those described in Chapter 3.
- *Instruction delivery (stages EXP and REN)*—Distributes up to six instructions to the 11 functional units. Implements register renaming for both rotation and register stacking.
- *Operand delivery (REG)*—Accesses the register file, performs register bypassing, accesses and updates a register scoreboard, and checks predicate dependences. The scoreboard is used to detect when individual instructions can proceed, so that a stall of one instruction (for example, due to an unpredictable event like a cache miss) in a bundle need not cause the entire bundle to stall. (As we saw in Figure H.8, stalling the entire bundle leads to poor performance unless the instructions are carefully scheduled.)
- *Execution (EXE, DET, and WRB)*—Executes instructions through ALUs and load-store units, detects exceptions and posts NaTs, retires instructions, and performs write-back.

Both the Itanium 1 and the Itanium 2 have many of the features more commonly associated with the dynamically scheduled pipelines described in Chapter 3: dynamic branch prediction, register renaming, scoreboarding, a pipeline with a number of stages before execution (to handle instruction alignment, renaming, etc.), and several stages following execution to handle exception detection. Although these mechanisms are generally simpler than those in an advanced dynamically scheduled superscalar, the overall effect is that the Itanium processors, which rely much more on compiler technology, seem to be as complex as the dynamically scheduled processors we saw in Chapter 3!

One might ask why such features are included in a processor that relies primarily on compile time techniques for finding and exploiting parallelism. There are two main motivations. First, dynamic techniques are sometimes significantly better, and omitting them would hurt performance significantly. The inclusion of dynamic branch prediction is such a case.

Second, caches are absolutely necessary to achieve high performance, and with caches come cache misses, which are both unpredictable and which in current processors take a relatively long time. In the early VLIW processors, the entire processor would freeze when a cache miss occurred, retaining the lockstep parallelism initially specified by the compiler. Such an approach is totally unrealistic in a modern processor where cache misses can cost tens to hundreds of cycles. Allowing some instructions to continue while others are stalled, however, requires the introduction of some form of dynamic scheduling, in this case scoreboarding. In addition, if a stall is likely to be long, then antidependences are likely to prevent much

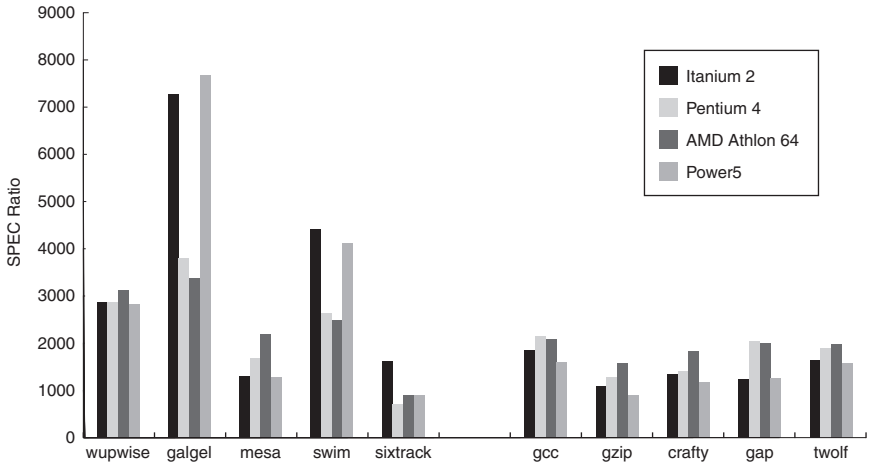


Figure H.11 The performance of four multiple-issue processors for five SPECfp and SPECint benchmarks. The clock rates of the four processors are Itanium 2 at 1.5 GHz, Pentium 4 Extreme Edition at 3.8 GHz, AMD Athlon 64 at 2.8 GHz, and the IBM Power5 at 1.9 GHz.

progress while waiting for the cache miss; hence, the Itanium implementations also introduce register renaming.

Itanium 2 Performance

Figure H.11 shows the performance of a 1.5 GHz Itanium 2 versus a Pentium 4, an AMD Athlon processor, and an IBM Power5 for five SPECint and five SPECfp benchmarks. Overall, the Itanium 2 is slightly slower than the Power5 for the full set of SPEC floating-point benchmarks and about 35% faster than the AMD Athlon or Pentium 4. On SPECint, the Itanium 2 is 15% faster than the Power5, while both the AMD Athlon and Pentium 4 are about 15% faster than the Itanium 2. The Itanium 2 and Power5 are much higher power and have larger die sizes. In fact, the Power5 contains two processors, only one of which is active during normal SPEC benchmarks, and still it has less than half the transistor count of the Itanium. If we were to reduce the die size, transistor count, and power of the Power5 by eliminating one of the processors, the Itanium would be by far the largest and highest-power processor.

H.7 Concluding Remarks

When the design of the IA-64 architecture began, it was a joint effort of Hewlett-Packard and Intel and many of the designers had benefited from experience with early VLIW processors as well of years of research building on the early concepts. The clear goal for the IA-64 architecture was to achieve levels of ILP as good or

better than what had been achieved with hardware-based approaches, while also allowing a much simpler hardware implementation. With a simpler hardware implementation, designers hoped that much higher clock rates could be achieved. Indeed, when the IA-64 architecture and the first Itanium were announced, they were announced as the successor to the RISC approaches with clearly superior advantages.

Unfortunately, the practical reality has been quite different. The IA-64 and Itanium implementations appear to be at least as complicated as the dynamically based speculative processors, and neither approach has a significant and consistent performance advantage. The fact that the Itanium designs have also not been more power efficient has led to a situation where the Itanium design has been adopted by only a small number of customers primarily interested in FP performance.

Intel had planned for IA-64 to be its new 64-bit architecture as well. But the combination of its mediocre integer performance (especially in Itanium 1) and large die size, together with AMD's introduction of a 64-bit version of the IA-32 architecture, forced Intel to extend the address space of IA-32. The availability of a larger address space IA-32 processor with strong integer performance has further reduced the interest in IA-64 and Itanium. Most recently, Intel has introduced the name IPF to replace IA-64, since the former name made less sense once the older x86 architecture was extended to 64 bits.

Reference

Wilson, R.P., Lam, M.S., 1995. Efficient context-sensitive pointer analysis for C programs. In: Proc. ACM SIGPLAN'95 Conf. on Programming Language Design and Implementation, June 18–21, 1995, La Jolla, Calif, pp. 1–12.

I.1	Introduction	I-2
I.2	Interprocessor Communication: The Critical Performance Issue	I-3
I.3	Characteristics of Scientific Applications	I-6
I.4	Synchronization: Scaling Up	I-12
I.5	Performance of Scientific Applications on Shared-Memory Multiprocessors	I-21
I.6	Performance Measurement of Parallel Processors with Scientific Applications	I-33
I.7	Implementing Cache Coherence	I-34
I.8	The Custom Cluster Approach: Blue Gene/L	I-41
I.9	Concluding Remarks	I-44

I

Large-Scale Multiprocessors and Scientific Applications

Hennessy and Patterson should move MPPs to Chapter 11.

Jim Gray, Microsoft Research
when asked about the coverage of massively parallel processors (MPPs) for the third edition in 2000

Unfortunately for companies in the MPP business, the third edition had only ten chapters and the MPP business did not grow as anticipated when the first and second edition were written.

I.1 Introduction

The primary application of large-scale multiprocessors is for true parallel programming, as opposed to multiprogramming or transaction-oriented computing where independent tasks are executed in parallel without much interaction. In true parallel computing, a set of tasks execute in a collaborative fashion on one application. The primary target of parallel computing is scientific and technical applications. In contrast, for loosely coupled commercial applications, such as Web servers and most transaction-processing applications, there is little communication among tasks. For such applications, loosely coupled clusters are generally adequate and most cost-effective, since intertask communication is rare.

Because true parallel computing involves cooperating tasks, the nature of communication between those tasks and how such communication is supported in the hardware is of vital importance in determining the performance of the application. The next section of this appendix examines such issues and the characteristics of different communication models.

In comparison to sequential programs, whose performance is largely dictated by the cache behavior and issues related to instruction-level parallelism, parallel programs have several additional characteristics that are important to performance, including the amount of parallelism, the size of parallel tasks, the frequency and nature of intertask communication, and the frequency and nature of synchronization. These aspects are affected both by the underlying nature of the application as well as by the programming style. Section I.3 reviews the important characteristics of several scientific applications to give a flavor of these issues.

As we saw in Chapter 5, synchronization can be quite important in achieving good performance. The larger number of parallel tasks that may need to synchronize makes contention involving synchronization a much more serious problem in large-scale multiprocessors. Section I.4 examines methods of scaling up the synchronization mechanisms of Chapter 5.

Section I.5 explores the detailed performance of shared-memory parallel applications executing on a moderate-scale shared-memory multiprocessor. As we will see, the behavior and performance characteristics are quite a bit more complicated than those in small-scale shared-memory multiprocessors. Section I.6 discusses the general issue of how to examine parallel performance for different sized multiprocessors. Section I.7 explores the implementation challenges of distributed shared-memory cache coherence, the key architectural approach used in moderate-scale multiprocessors. Sections I.7 and I.8 rely on a basic understanding of interconnection networks, and the reader should at least quickly review Appendix F before reading these sections.

Section I.8 explores the design of one of the newest and most exciting large-scale multiprocessors in recent times, Blue Gene. Blue Gene is a cluster-based multiprocessor, but it uses a custom, highly dense node designed specifically for this function, as opposed to the nodes of most earlier cluster multiprocessors that used a node architecture similar to those in a desktop or smaller-scale multiprocessor

node. By using a custom node design, Blue Gene achieves a significant reduction in the cost, physical size, and power consumption of a node. Blue Gene/L, a 64 K-node version, was the world's fastest computer in 2006, as measured by the linear algebra benchmark, Linpack.

I.2 Interprocessor Communication: The Critical Performance Issue

In multiprocessors with larger processor counts, interprocessor communication becomes more expensive, since the distance between processors increases. Furthermore, in truly parallel applications where the threads of the application must communicate, there is usually more communication than in a loosely coupled set of distinct processes or independent transactions, which characterize many commercial server applications. These factors combine to make efficient interprocessor communication one of the most important determinants of parallel performance, especially for the scientific market.

Unfortunately, characterizing the communication needs of an application and the capabilities of an architecture is complex. This section examines the key hardware characteristics that determine communication performance, while the next section looks at application behavior and communication needs.

Three performance metrics are critical in any hardware communication mechanism:

1. *Communication bandwidth*—Ideally, the communication bandwidth is limited by processor, memory, and interconnection bandwidths, rather than by some aspect of the communication mechanism. The interconnection network determines the maximum communication capacity of the system. The bandwidth in or out of a single node, which is often as important as total system bandwidth, is affected both by the architecture within the node and by the communication mechanism. How does the communication mechanism affect the communication bandwidth of a node? When communication occurs, resources within the nodes involved in the communication are tied up or occupied, preventing other outgoing or incoming communication. When this *occupancy* is incurred for each word of a message, it sets an absolute limit on the communication bandwidth. This limit is often lower than what the network or memory system can provide. Occupancy may also have a component that is incurred for each communication event, such as an incoming or outgoing request. In the latter case, the occupancy limits the communication rate, and the impact of the occupancy on overall communication bandwidth depends on the size of the messages.
2. *Communication latency*—Ideally, the latency is as low as possible. As Appendix F explains:

$$\begin{aligned}\text{Communication latency} = {} & \text{Sender overhead} + \text{Time of flight} \\ & + \text{Transmission time} + \text{Receiver overhead}\end{aligned}$$

assuming no contention. Time of flight is fixed and transmission time is determined by the interconnection network. The software and hardware overheads in sending and receiving messages are largely determined by the communication mechanism and its implementation. Why is latency crucial? Latency affects both performance and how easy it is to program a multiprocessor. Unless latency is hidden, it directly affects performance either by tying up processor resources or by causing the processor to wait.

Overhead and occupancy are closely related, since many forms of overhead also tie up some part of the node, incurring an occupancy cost, which in turn limits bandwidth. Key features of a communication mechanism may directly affect overhead and occupancy. For example, how is the destination address for a remote communication named, and how is protection implemented? When naming and protection mechanisms are provided by the processor, as in a shared address space, the additional overhead is small. Alternatively, if these mechanisms must be provided by the operating system for each communication, this increases the overhead and occupancy costs of communication, which in turn reduce bandwidth and increase latency.

3. *Communication latency hiding*—How well can the communication mechanism hide latency by overlapping communication with computation or with other communication? Although measuring this is not as simple as measuring the first two metrics, it is an important characteristic that can be quantified by measuring the running time on multiprocessors with the same communication latency but different support for latency hiding. Although hiding latency is certainly a good idea, it poses an additional burden on the software system and ultimately on the programmer. Furthermore, the amount of latency that can be hidden is application dependent. Thus, it is usually best to reduce latency wherever possible.

Each of these performance measures is affected by the characteristics of the communications needed in the application, as we will see in the next section. The size of the data items being communicated is the most obvious characteristic, since it affects both latency and bandwidth directly, as well as affecting the efficacy of different latency-hiding approaches. Similarly, the regularity in the communication patterns affects the cost of naming and protection, and hence the communication overhead. In general, mechanisms that perform well with smaller as well as larger data communication requests, and irregular as well as regular communication patterns, are more flexible and efficient for a wider class of applications. Of course, in considering any communication mechanism, designers must consider cost as well as performance.

Advantages of Different Communication Mechanisms

The two primary means of communicating data in a large-scale multiprocessor are message passing and shared memory. Each of these two primary communication

mechanisms has its advantages. For shared-memory communication, the advantages include

- Compatibility with the well-understood mechanisms in use in centralized multiprocessors, which all use shared-memory communication. The OpenMP consortium (see *www.openmp.org* for description) has proposed a standardized programming interface for shared-memory multiprocessors. Although message passing also uses a standard, MPI or Message Passing Interface, this standard is not used either in shared-memory multiprocessors or in loosely coupled clusters in use in throughput-oriented environments.
- Ease of programming when the communication patterns among processors are complex or vary dynamically during execution. Similar advantages simplify compiler design.
- The ability to develop applications using the familiar shared-memory model, focusing attention only on those accesses that are performance critical.
- Lower overhead for communication and better use of bandwidth when communicating small items. This arises from the implicit nature of communication and the use of memory mapping to implement protection in hardware, rather than through the I/O system.
- The ability to use hardware-controlled caching to reduce the frequency of remote communication by supporting automatic caching of all data, both shared and private. As we will see, caching reduces both latency and contention for accessing shared data. This advantage also comes with a disadvantage, which we mention below.

The major advantages for message-passing communication include the following:

- The hardware can be simpler, especially by comparison with a scalable shared-memory implementation that supports coherent caching of remote data.
- Communication is explicit, which means it is simpler to understand. In shared-memory models, it can be difficult to know when communication is occurring and when it is not, as well as how costly the communication is.
- Explicit communication focuses programmer attention on this costly aspect of parallel computation, sometimes leading to improved structure in a multiprocessor program.
- Synchronization is naturally associated with sending messages, reducing the possibility for errors introduced by incorrect synchronization.
- It makes it easier to use sender-initiated communication, which may have some advantages in performance.
- If the communication is less frequent and more structured, it is easier to improve fault tolerance by using a transaction-like structure. Furthermore,

the less tight coupling of nodes and explicit communication make fault isolation simpler.

- The very largest multiprocessors use a cluster structure, which is inherently based on message passing. Using two different communication models may introduce more complexity than is warranted.

Of course, the desired communication model can be created in software on top of a hardware model that supports either of these mechanisms. Supporting message passing on top of shared memory is considerably easier: Because messages essentially send data from one memory to another, sending a message can be implemented by doing a copy from one portion of the address space to another. The major difficulties arise from dealing with messages that may be misaligned and of arbitrary length in a memory system that is normally oriented toward transferring aligned blocks of data organized as cache blocks. These difficulties can be overcome either with small performance penalties in software or with essentially no penalties, using a small amount of hardware support.

Supporting shared memory efficiently on top of hardware for message passing is much more difficult. Without explicit hardware support for shared memory, all shared-memory references need to involve the operating system to provide address translation and memory protection, as well as to translate memory references into message sends and receives. Loads and stores usually move small amounts of data, so the high overhead of handling these communications in software severely limits the range of applications for which the performance of software-based shared memory is acceptable. For these reasons, it has never been practical to use message passing to implement shared memory for a commercial system.

I.3 Characteristics of Scientific Applications

The primary use of scalable shared-memory multiprocessors is for true parallel programming, as opposed to multiprogramming or transaction-oriented computing. The primary target of parallel computing is scientific and technical applications. Thus, understanding the design issues requires some insight into the behavior of such applications. This section provides such an introduction.

Characteristics of Scientific Applications

Our scientific/technical parallel workload consists of two applications and two computational kernels. The kernels are fast Fourier transformation (FFT) and an LU decomposition, which were chosen because they represent commonly used techniques in a wide variety of applications and have performance characteristics typical of many parallel scientific applications. In addition, the kernels have small code segments whose behavior we can understand and directly track to specific architectural characteristics. Like many scientific applications, I/O is essentially nonexistent in this workload.

The two applications that we use in this appendix are Barnes and Ocean, which represent two important but very different types of parallel computation. We briefly describe each of these applications and kernels and characterize their basic behavior in terms of parallelism and communication. We describe how the problem is decomposed for a distributed shared-memory multiprocessor; certain data decompositions that we describe are not necessary on multiprocessors that have a single, centralized memory.

The FFT Kernel

The FFT is the key kernel in applications that use spectral methods, which arise in fields ranging from signal processing to fluid flow to climate modeling. The FFT application we study here is a one-dimensional version of a parallel algorithm for a complex number FFT. It has a sequential execution time for n data points of $n \log n$. The algorithm uses a high radix (equal to $\sqrt{n}$) that minimizes communication. The measurements shown in this appendix are collected for a million-point input data set.

There are three primary data structures: the input and output arrays of the data being transformed and the roots of unity matrix, which is precomputed and only read during the execution. All arrays are organized as square matrices. The six steps in the algorithm are as follows:

1. Transpose data matrix.
2. Perform 1D FFT on each row of data matrix.
3. Multiply the roots of unity matrix by the data matrix and write the result in the data matrix.
4. Transpose data matrix.
5. Perform 1D FFT on each row of data matrix.
6. Transpose data matrix.

The data matrices and the roots of unity matrix are partitioned among processors in contiguous chunks of rows, so that each processor's partition falls in its own local memory. The first row of the roots of unity matrix is accessed heavily by all processors and is often replicated, as we do, during the first step of the algorithm just shown. The data transposes ensure good locality during the individual FFT steps, which would otherwise access nonlocal data.

The only communication is in the transpose phases, which require all-to-all communication of large amounts of data. Contiguous subcolumns in the rows assigned to a processor are grouped into blocks, which are transposed and placed into the proper location of the destination matrix. Every processor transposes one block locally and sends one block to each of the other processors in the system. Although there is no reuse of individual words in the transpose, with long cache blocks it makes sense to block the transpose to take advantage of the spatial locality afforded by long blocks in the source matrix.

The LU Kernel

LU is an LU factorization of a dense matrix and is representative of many dense linear algebra computations, such as QR factorization, Cholesky factorization, and eigenvalue methods. For a matrix of size $n \times n$ the running time is n^3 and the parallelism is proportional to n^2. Dense LU factorization can be performed efficiently by blocking the algorithm, using the techniques in Chapter 2, which leads to highly efficient cache behavior and low communication. After blocking the algorithm, the dominant computation is a dense matrix multiply that occurs in the innermost loop. The block size is chosen to be small enough to keep the cache miss rate low and large enough to reduce the time spent in the less parallel parts of the computation. Relatively small block sizes (8×8 or 16×16) tend to satisfy both criteria.

Two details are important for reducing interprocessor communication. First, the blocks of the matrix are assigned to processors using a 2D tiling: The $\frac{n}{B} \times \frac{n}{B}$ (where each block is $B \times B$) matrix of blocks is allocated by laying a grid of size $p \times p$ over the matrix of blocks in a cookie-cutter fashion until all the blocks are allocated to a processor. Second, the dense matrix multiplication is performed by the processor that owns the *destination* block. With this blocking and allocation scheme, communication during the reduction is both regular and predictable. For the measurements in this appendix, the input is a 512×512 matrix and a block of 16×16 is used.

A natural way to code the blocked LU factorization of a 2D matrix in a shared address space is to use a 2D array to represent the matrix. Because blocks are allocated in a tiled decomposition, and a block is not contiguous in the address space in a 2D array, it is very difficult to allocate blocks in the local memories of the processors that own them. The solution is to ensure that blocks assigned to a processor are allocated locally and contiguously by using a 4D array (with the first two dimensions specifying the block number in the 2D grid of blocks, and the next two specifying the element in the block).

The Barnes Application

Barnes is an implementation of the Barnes-Hut n-body algorithm solving a problem in galaxy evolution. *N-body algorithms* simulate the interaction among a large number of bodies that have forces interacting among them. In this instance, the bodies represent collections of stars and the force is gravity. To reduce the computational time required to model completely all the individual interactions among the bodies, which grow as n^2, n-body algorithms take advantage of the fact that the forces drop off with distance. (Gravity, for example, drops off as $1/d^2$, where d is the distance between the two bodies.) The Barnes-Hut algorithm takes advantage of this property by treating a collection of bodies that are "far away" from another body as a single point at the center of mass of the collection and with mass equal to the collection. If the body is far enough from any body in the collection, then the error introduced will be

negligible. The collections are structured in a hierarchical fashion, which can be represented in a tree. This algorithm yields an n log n running time with parallelism proportional to n.

The Barnes-Hut algorithm uses an octree (each node has up to eight children) to represent the eight cubes in a portion of space. Each node then represents the collection of bodies in the subtree rooted at that node, which we call a *cell*. Because the density of space varies and the leaves represent individual bodies, the depth of the tree varies. The tree is traversed once per body to compute the net force acting on that body. The force calculation algorithm for a body starts at the root of the tree. For every node in the tree it visits, the algorithm determines if the center of mass of the cell represented by the subtree rooted at the node is "far enough away" from the body. If so, the entire subtree under that node is approximated by a single point at the center of mass of the cell, and the force that this center of mass exerts on the body is computed. On the other hand, if the center of mass is not far enough away, the cell must be "opened" and each of its subtrees visited. The distance between the body and the cell, together with the error tolerances, determines which cells must be opened. This force calculation phase dominates the execution time. This appendix takes measurements using 16K bodies; the criterion for determining whether a cell needs to be opened is set to the middle of the range typically used in practice.

Obtaining effective parallel performance on Barnes-Hut is challenging because the distribution of bodies is nonuniform and changes over time, making partitioning the work among the processors and maintenance of good locality of reference difficult. We are helped by two properties: (1) the system evolves slowly, and (2) because gravitational forces fall off quickly, with high probability, each cell requires touching a small number of other cells, most of which were used on the last time step. The tree can be partitioned by allocating each processor a subtree. Many of the accesses needed to compute the force on a body in the subtree will be to other bodies in the subtree. Since the amount of work associated with a subtree varies (cells in dense portions of space will need to access more cells), the size of the subtree allocated to a processor is based on some measure of the work it has to do (e.g., how many other cells it needs to visit), rather than just on the number of nodes in the subtree. By partitioning the octree representation, we can obtain good load balance and good locality of reference, while keeping the partitioning cost low. Although this partitioning scheme results in good locality of reference, the resulting data references tend to be for small amounts of data and are unstructured. Thus, this scheme requires an efficient implementation of shared-memory communication.

The Ocean Application

Ocean simulates the influence of eddy and boundary currents on large-scale flow in the ocean. It uses a restricted red-black Gauss-Seidel multigrid technique to solve a set of elliptical partial differential equations. *Red-black Gauss-Seidel* is an iteration technique that colors the points in the grid so as to consistently update

each point based on previous values of the adjacent neighbors. *Multigrid methods* solve finite difference equations by iteration using hierarchical grids. Each grid in the hierarchy has fewer points than the grid below and is an approximation to the lower grid. A finer grid increases accuracy and thus the rate of convergence, while requiring more execution time, since it has more data points. Whether to move up or down in the hierarchy of grids used for the next iteration is determined by the rate of change of the data values. The estimate of the error at every time step is used to decide whether to stay at the same grid, move to a coarser grid, or move to a finer grid. When the iteration converges at the finest level, a solution has been reached. Each iteration has n^2 work for an $n \times n$ grid and the same amount of parallelism.

The arrays representing each grid are dynamically allocated and sized to the particular problem. The entire ocean basin is partitioned into square subgrids (as close as possible) that are allocated in the portion of the address space corresponding to the local memory of the individual processors, which are assigned responsibility for the subgrid. For the measurements in this appendix we use an input that has 130×130 grid points. There are five steps in a time iteration. Since data are exchanged between the steps, all the processors present synchronize at the end of each step before proceeding to the next. Communication occurs when the boundary points of a subgrid are accessed by the adjacent subgrid in nearest-neighbor fashion.

Computation/Communication for the Parallel Programs

A key characteristic in determining the performance of parallel programs is the ratio of computation to communication. If the ratio is high, it means the application has lots of computation for each datum communicated. As we saw in Section I.2, communication is the costly part of parallel computing; therefore, high computation-to-communication ratios are very beneficial. In a parallel processing environment, we are concerned with how the ratio of computation to communication changes as we increase either the number of processors, the size of the problem, or both. Knowing how the ratio changes as we increase the processor count sheds light on how well the application can be sped up. Because we are often interested in running larger problems, it is vital to understand how changing the data set size affects this ratio.

To understand what happens quantitatively to the computation-to-communication ratio as we add processors, consider what happens separately to computation and to communication as we either add processors or increase problem size. Figure I.1 shows that as we add processors, for these applications, the amount of computation per processor falls proportionately and the amount of communication per processor falls more slowly. As we increase the problem size, the computation scales as the $O(\ )$ complexity of the algorithm dictates. Communication scaling is more complex and depends on details of the algorithm; we describe the basic phenomena for each application in the caption of Figure I.1.

Application	Scaling of computation	Scaling of communication	Scaling of computation-to-communication
FFT	$\frac{n \log n}{p}$	$\frac{n}{p}$	$\log n$
LU	$\frac{n}{p}$	$\frac{\sqrt{n}}{\sqrt{p}}$	$\frac{\sqrt{n}}{\sqrt{p}}$
Barnes	$\frac{n \log n}{p}$	approximately $\frac{\sqrt{n}(\log n)}{\sqrt{p}}$	approximately $\frac{\sqrt{n}}{\sqrt{p}}$
Ocean	$\frac{n}{p}$	$\frac{\sqrt{n}}{\sqrt{p}}$	$\frac{\sqrt{n}}{\sqrt{p}}$

Figure I.1 Scaling of computation, of communication, and of the ratio are critical factors in determining performance on parallel multiprocessors. In this table, p is the increased processor count and n is the increased dataset size. Scaling is on a per-processor basis. The computation scales up with n at the rate given by $O(\ )$ analysis and scales down linearly as p is increased. Communication scaling is more complex. In FFT, all data points must interact, so communication increases with n and decreases with p. In LU and Ocean, communication is proportional to the boundary of a block, so it scales with dataset size at a rate proportional to the side of a square with n points, namely, $\sqrt{n}$; for the same reason communication in these two applications scales inversely to $\sqrt{p}$. Barnes has the most complex scaling properties. Because of the fall-off of interaction between bodies, the basic number of interactions among bodies that require communication scales as $\sqrt{n}$. An additional factor of $\log n$ is needed to maintain the relationships among the bodies. As processor count is increased, communication scales inversely to $\sqrt{p}$.

The overall computation-to-communication ratio is computed from the individual growth rate in computation and communication. In general, this ratio rises slowly with an increase in dataset size and decreases as we add processors. This reminds us that performing a fixed-size problem with more processors leads to increasing inefficiencies because the amount of communication among processors grows. It also tells us how quickly we must scale dataset size as we add processors to keep the fraction of time in communication fixed. The following example illustrates these trade-offs.

Example Suppose we know that for a given multiprocessor the Ocean application spends 20% of its execution time waiting for communication when run on four processors. Assume that the cost of each communication event is independent of processor count, which is not true in general, since communication costs rise with processor count. How much faster might we expect Ocean to run on a 32-processor machine with the same problem size? What fraction of the execution time is spent on communication in this case? How much larger a problem should we run if we want the fraction of time spent communicating to be the same?

Answer The computation-to-communication ratio for Ocean is $\sqrt{n}/\sqrt{p}$, so if the problem size is the same, the communication frequency scales by $\sqrt{p}$. This means that communication time increases by $\sqrt{8}$. We can use a variation on Amdahl's law,

recognizing that the computation is decreased but the communication time is increased. If T_4 is the total execution time for four processors, then the execution time for 32 processors is

$$\begin{aligned} T_{32} &= \text{Compute time} + \text{Communication time} \\ &= \frac{0.8 \times T_4}{8} + (0.2 \times T_4) \times \sqrt{8} \\ &= 0.1 \times T_4 + 0.57 \times T_4 = 0.67 \times T_4 \end{aligned}$$

Hence, the speedup is

$$\text{Speedup} = \frac{T_4}{T_{32}} = \frac{T_4}{0.67 \times T_4} = 1.49$$

and the fraction of time spent in communication goes from 20% to 0.57/0.67 = 85%.

For the fraction of the communication time to remain the same, we must keep the computation-to-communication ratio the same, so the problem size must scale at the same rate as the processor count. Notice that, because we have changed the problem size, we cannot fairly compare the speedup of the original problem and the scaled problem. We will return to the critical issue of scaling applications for multiprocessors in Section I.6.

I.4 Synchronization: Scaling Up

In this section, we focus first on synchronization performance problems in larger multiprocessors and then on solutions for those problems.

Synchronization Performance Challenges

To understand why the simple spin lock scheme presented in Chapter 5 does not scale well, imagine a large multiprocessor with all processors contending for the same lock. The directory or bus acts as a point of serialization for all the processors, leading to lots of contention, as well as traffic. The following example shows how bad things can be.

Example Suppose there are 10 processors on a bus and each tries to lock a variable simultaneously. Assume that each bus transaction (read miss or write miss) is 100 clock cycles long. You can ignore the time of the actual read or write of a lock held in the cache, as well as the time the lock is held (they won't matter much!). Determine the number of bus transactions required for all 10 processors to acquire the lock, assuming they are all spinning when the lock is released at time 0. About how long will it take to process the 10 requests? Assume that the bus is

totally fair so that every pending request is serviced before a new request and that the processors are equally fast.

Answer When i processes are contending for the lock, they perform the following sequence of actions, each of which generates a bus transaction:

i load linked operations to access the lock

i store conditional operations to try to lock the lock

1 store (to release the lock)

Thus, for i processes, there are a total of $2i+1$ bus transactions. Note that this assumes that the critical section time is negligible, so that the lock is released before any other processors whose store conditional failed attempt another load linked.

Thus, for n processes, the total number of bus operations is

$$\sum_{i=1}^{n}(2i+1) = n(n+1)+n = n^2+2n$$

For 10 processes there are 120 bus transactions requiring 12,000 clock cycles or 120 clock cycles per lock acquisition!

The difficulty in this example arises from contention for the lock and serialization of lock access, as well as the latency of the bus access. (The fairness property of the bus actually makes things worse, since it delays the processor that claims the lock from releasing it; unfortunately, for any bus arbitration scheme some worst-case scenario does exist.) The key advantages of spin locks—that they have low overhead in terms of bus or network cycles and offer good performance when locks are reused by the same processor—are both lost in this example. We will consider alternative implementations in the next section, but before we do that, let's consider the use of spin locks to implement another common high-level synchronization primitive.

Barrier Synchronization

One additional common synchronization operation in programs with parallel loops is a *barrier*. A barrier forces all processes to wait until all the processes reach the barrier and then releases all of the processes. A typical implementation of a barrier can be done with two spin locks: one to protect a counter that tallies the processes arriving at the barrier and one to hold the processes until the last process arrives at the barrier. To implement a barrier, we usually use the ability to spin on a variable until it satisfies a test; we use the notation `spin(condition)` to indicate this. Figure I.2 is a typical implementation, assuming that `lock` and `unlock` provide basic spin locks and `total` is the number of processes that must reach the barrier.

```
lock (counterlock);/* ensure update atomic */
if (count==0) release=0;/* first=>reset release */
count = count + 1;/* count arrivals */
unlock(counterlock);/* release lock */
if (count==total) {/* all arrived */
      count=0;/* reset counter */
      release=1;/* release processes */
}
else {/* more to come */
      spin (release==1);/* wait for arrivals */
}
```

Figure I.2 Code for a simple barrier. The lock `counterlock` protects the counter so that it can be atomically incremented. The variable `count` keeps the tally of how many processes have reached the barrier. The variable `release` is used to hold the processes until the last one reaches the barrier. The operation `spin (release==1)` causes a process to wait until all processes reach the barrier.

In practice, another complication makes barrier implementation slightly more complex. Frequently a barrier is used within a loop, so that processes released from the barrier would do some work and then reach the barrier again. Assume that one of the processes never actually leaves the barrier (it stays at the spin operation), which could happen if the OS scheduled another process, for example. Now it is possible that one process races ahead and gets to the barrier again before the last process has left. The "fast" process then traps the remaining "slow" process in the barrier by resetting the flag `release`. Now all the processes will wait infinitely at the next instance of this barrier because one process is trapped at the last instance, and the number of processes can never reach the value of total.

The important observation in this example is that the programmer did nothing wrong. Instead, the implementer of the barrier made some assumptions about forward progress that cannot be assumed. One obvious solution to this is to count the processes as they exit the barrier (just as we did on entry) and not to allow any process to reenter and reinitialize the barrier until all processes have left the prior instance of this barrier. This extra step would significantly increase the latency of the barrier and the contention, which as we will see shortly are already large. An alternative solution is a *sense-reversing barrier*, which makes use of a private per-process variable, `local_sense`, which is initialized to 1 for each process. Figure I.3 shows the code for the sense-reversing barrier. This version of a barrier is safely usable; as the next example shows, however, its performance can still be quite poor.

```
local_sense =! local_sense; /* toggle local_sense */
lock (counterlock);/* ensure update atomic */
count=count+1;/* count arrivals */
if (count==total) {/* all arrived */
      count=0;/* reset counter */
      release=local_sense;/* release processes */
}
unlock (counterlock);/* unlock */
spin (release==local_sense);/* wait for signal */
}
```

Figure I.3 Code for a sense-reversing barrier. The key to making the barrier reusable is the use of an alternating pattern of values for the flag `release`, which controls the exit from the barrier. If a process races ahead to the next instance of this barrier while some other processes are still in the barrier, the fast process cannot trap the other processes, since it does not reset the value of `release` as it did in Figure I.2.

Example Suppose there are 10 processors on a bus and each tries to execute a barrier simultaneously. Assume that each bus transaction is 100 clock cycles, as before. You can ignore the time of the actual read or write of a lock held in the cache as the time to execute other nonsynchronization operations in the barrier implementation. Determine the number of bus transactions required for all 10 processors to reach the barrier, be released from the barrier, and exit the barrier. Assume that the bus is totally fair, so that every pending request is serviced before a new request and that the processors are equally fast. Don't worry about counting the processors out of the barrier. How long will the entire process take?

Answer We assume that load linked and store conditional are used to implement lock and unlock. Figure I.4 shows the sequence of bus events for a processor to traverse the barrier, assuming that the first process to grab the bus does not have the lock. There is a slight difference for the last process to reach the barrier, as described in the caption.

For the ith process, the number of bus transactions is $3i+4$. The last process to reach the barrier requires one less. Thus, for n processes, the number of bus transactions is

$$\left(\sum_{i=1}^{n}(3i+4)\right)-1=\frac{3n^2+11n}{2}-1$$

For 10 processes, this is 204 bus cycles or 20,400 clock cycles! Our barrier operation takes almost twice as long as the 10-processor lock-unlock sequence.

Event	Number of times for process *i*	Corresponding source line	Comment
LL `counterlock`	*i*	`lock (counterlock);`	All processes try for lock.
Store conditional	*i*	`lock (counterlock);`	All processes try for lock.
LD `count`	1	`count = count + 1;`	Successful process.
Load linked	*i* − 1	`lock (counterlock);`	Unsuccessful process; try again.
SD `count`	1	`count = count + 1;`	Miss to get exclusive access.
SD `counterlock`	1	`unlock(counterlock);`	Miss to get the lock.
LD `release`	2	`spin (release==local_sense);/`	Read release: misses initially and when finally written.

Figure I.4 Here are the actions, which require a bus transaction, taken when the *i*th process reaches the barrier. The last process to reach the barrier requires one less bus transaction, since its read of release for the spin will hit in the cache!

As we can see from these examples, synchronization performance can be a real bottleneck when there is substantial contention among multiple processes. When there is little contention and synchronization operations are infrequent, we are primarily concerned about the latency of a synchronization primitive—that is, how long it takes an individual process to complete a synchronization operation. Our basic spin lock operation can do this in two bus cycles: one to initially read the lock and one to write it. We could improve this to a single bus cycle by a variety of methods. For example, we could simply spin on the swap operation. If the lock were almost always free, this could be better, but if the lock were not free, it would lead to lots of bus traffic, since each attempt to lock the variable would lead to a bus cycle. In practice, the latency of our spin lock is not quite as bad as we have seen in this example, since the write miss for a data item present in the cache is treated as an upgrade and will be cheaper than a true read miss.

The more serious problem in these examples is the serialization of each process's attempt to complete the synchronization. This serialization is a problem when there is contention because it greatly increases the time to complete the synchronization operation. For example, if the time to complete all 10 lock and unlock operations depended only on the latency in the uncontended case, then it would take 1000 rather than 15,000 cycles to complete the synchronization operations. The barrier situation is as bad, and in some ways worse, since it is highly likely to incur contention. The use of a bus interconnect exacerbates these problems, but serialization could be just as serious in a directory-based multiprocessor, where the latency would be large. The next subsection presents some solutions that are useful when either the contention is high or the processor count is large.

Synchronization Mechanisms for Larger-Scale Multiprocessors

What we would like are synchronization mechanisms that have low latency in uncontended cases and that minimize serialization in the case where contention is significant. We begin by showing how software implementations can improve the performance of locks and barriers when contention is high; we then explore two basic hardware primitives that reduce serialization while keeping latency low.

Software Implementations

The major difficulty with our spin lock implementation is the delay due to contention when many processes are spinning on the lock. One solution is to artificially delay processes when they fail to acquire the lock. The best performance is obtained by increasing the delay exponentially whenever the attempt to acquire the lock fails. Figure I.5 shows how a spin lock with *exponential back-off* is implemented. Exponential back-off is a common technique for reducing contention in shared resources, including access to shared networks and buses (see Sections F.4 to F.8). This implementation still attempts to preserve low latency when contention is small by not delaying the initial spin loop. The result is that if many processes are waiting, the back-off does not affect the processes on their first attempt to acquire the lock. We could also delay that process, but the result would be poorer

```
            DADDUI   R3,R0,#1    ;R3 = initial delay
lockit:     LL       R2,0(R1)    ;load linked
            BNEZ     R2,lockit   ;not available-spin
            DADDUI   R2,R2,#1    ;get locked value
            SC       R2,0(R1)    ;store conditional
            BNEZ     R2,gotit    ;branch if store succeeds
            DSLL     R3,R3,#1    ;increase delay by factor of 2
            PAUSE    R3          ;delays by value in R3
            J        lockit
gotit:      use data protected by lock
```

Figure I.5 A spin lock with exponential back-off. When the store conditional fails, the process delays itself by the value in R3. The delay can be implemented by decrementing a copy of the value in R3 until it reaches 0. The exact timing of the delay is multiprocessor dependent, although it should start with a value that is approximately the time to perform the critical section and release the lock. The statement `pause R3` should cause a delay of R3 of these time units. The value in R3 is increased by a factor of 2 every time the store conditional fails, which causes the process to wait twice as long before trying to acquire the lock again. The small variations in the rate at which competing processors execute instructions are usually sufficient to ensure that processes will not continually collide. If the natural perturbation in execution time was insufficient, R3 could be initialized with a small random value, increasing the variance in the successive delays and reducing the probability of successive collisions.

performance when the lock was in use by only two processes and the first one happened to find it locked.

Another technique for implementing locks is to use queuing locks. Queuing locks work by constructing a queue of waiting processors; whenever a processor frees up the lock, it causes the next processor in the queue to attempt access. This eliminates contention for a lock when it is freed. We show how queuing locks operate in the next section using a hardware implementation, but software implementations using arrays can achieve most of the same benefits. Before we look at hardware primitives, let's look at a better mechanism for barriers.

Our barrier implementation suffers from contention both during the *gather* stage, when we must atomically update the count, and at the *release* stage, when all the processes must read the release flag. The former is more serious because it requires exclusive access to the synchronization variable and thus creates much more serialization; in comparison, the latter generates only read contention. We can reduce the contention by using a *combining tree*, a structure where multiple requests are locally combined in tree fashion. The same combining tree can be used to implement the release process, reducing the contention there.

Our combining tree barrier uses a predetermined n-ary tree structure. We use the variable k to stand for the fan-in; in practice, $k = 4$ seems to work well. When the kth process arrives at a node in the tree, we signal the next level in the tree. When a process arrives at the root, we release all waiting processes. As in our earlier example, we use a sense-reversing technique. A tree-based barrier, as shown in Figure I.6, uses a tree to combine the processes and a single signal to release the barrier. Some MPPs (e.g., the T3D and CM-5) have also included hardware support for barriers, but more recent machines have relied on software libraries for this support.

Hardware Primitives

In this subsection, we look at two hardware synchronization primitives. The first primitive deals with locks, while the second is useful for barriers and a number of other user-level operations that require counting or supplying distinct indices. In both cases, we can create a hardware primitive where latency is essentially identical to our earlier version, but with much less serialization, leading to better scaling when there is contention.

The major problem with our original lock implementation is that it introduces a large amount of unneeded contention. For example, when the lock is released all processors generate both a read and a write miss, although at most one processor can successfully get the lock in the unlocked state. This sequence happens on each of the 10 lock/unlock sequences, as we saw in the example on page I-12.

We can improve this situation by explicitly handing the lock from one waiting processor to the next. Rather than simply allowing all processors to compete every time the lock is released, we keep a list of the waiting processors and hand the lock to one explicitly, when its turn comes. This sort of mechanism has been called a *queuing lock*. Queuing locks can be implemented either in hardware, which we

```
struct node{/* a node in the combining tree */
       int counterlock; /* lock for this node */
       int count; /* counter for this node */
       int parent; /* parent in the tree = 0..P-1 except for root */
};
struct node tree [0..P-1]; /* the tree of nodes */
int local_sense; /* private per processor */
int release; /* global release flag */

/* function to implement barrier */
barrier (int mynode, int local_sense) {
       lock (tree[mynode].counterlock); /* protect count */
       tree[mynode].count=tree[mynode].count+1;
              /* increment count */
       if (tree[mynode].count==k) {/* all arrived at mynode */
              if (tree[mynode].parent >=0) {
                     barrier(tree[mynode].parent);
              } else{
                     release = local_sense;
              };
              tree[mynode].count = 0; /* reset for the next time */
       unlock (tree[mynode].counterlock); /* unlock */
       spin (release==local_sense); /* wait */
};
/* code executed by a processor to join barrier */
local_sense =! local_sense;
barrier (mynode);
```

Figure I.6 An implementation of a tree-based barrier reduces contention considerably. The tree is assumed to be prebuilt statically using the nodes in the array tree. Each node in the tree combines *k* processes and provides a separate counter and lock, so that at most *k* processes contend at each node. When the *k*th process reaches a node in the tree, it goes up to the parent, incrementing the count at the parent. When the count in the parent node reaches *k*, the release flag is set. The count in each node is reset by the last process to arrive. Sense-reversing is used to avoid races as in the simple barrier. The value of `tree[root].parent` should be set to −1 when the tree is initially built.

describe here, or in software using an array to keep track of the waiting processes. The basic concepts are the same in either case. Our hardware implementation assumes a directory-based multiprocessor where the individual processor caches are addressable. In a bus-based multiprocessor, a software implementation would be more appropriate and would have each processor using a different address for the lock, permitting the explicit transfer of the lock from one process to another.

How does a queuing lock work? On the first miss to the lock variable, the miss is sent to a synchronization controller, which may be integrated with the memory controller (in a bus-based system) or with the directory controller. If the lock is free, it is simply returned to the processor. If the lock is unavailable,

the controller creates a record of the node's request (such as a bit in a vector) and sends the processor back a locked value for the variable, which the processor then spins on. When the lock is freed, the controller selects a processor to go ahead from the list of waiting processors. It can then either update the lock variable in the selected processor's cache or invalidate the copy, causing the processor to miss and fetch an available copy of the lock.

Example How many bus transactions and how long does it take to have 10 processors lock and unlock the variable using a queuing lock that updates the lock on a miss? Make the other assumptions about the system the same as those in the earlier example on page I-12.

Answer For n processors, each will initially attempt a lock access, generating a bus transaction; one will succeed and free up the lock, for a total of $n+1$ transactions for the first processor. Each subsequent processor requires two bus transactions, one to receive the lock and one to free it up. Thus, the total number of bus transactions is $(n+1)+2(n-1)=3n-1$. Note that the number of bus transactions is now linear in the number of processors contending for the lock, rather than quadratic, as it was with the spin lock we examined earlier. For 10 processors, this requires 29 bus cycles or 2900 clock cycles.

There are a couple of key insights in implementing such a queuing lock capability. First, we need to be able to distinguish the initial access to the lock, so we can perform the queuing operation, and also the lock release, so we can provide the lock to another processor. The queue of waiting processes can be implemented by a variety of mechanisms. In a directory-based multiprocessor, this queue is akin to the sharing set, and similar hardware can be used to implement the directory and queuing lock operations. One complication is that the hardware must be prepared to reclaim such locks, since the process that requested the lock may have been context-switched and may not even be scheduled again on the same processor.

Queuing locks can be used to improve the performance of our barrier operation. Alternatively, we can introduce a primitive that reduces the amount of time needed to increment the barrier count, thus reducing the serialization at this bottleneck, which should yield comparable performance to using queuing locks. One primitive that has been introduced for this and for building other synchronization operations is *fetch-and-increment*, which atomically fetches a variable and increments its value. The returned value can be either the incremented value or the fetched value. Using fetch-and-increment we can dramatically improve our barrier implementation, compared to the simple code-sensing barrier.

Example Write the code for the barrier using fetch-and-increment. Making the same assumptions as in our earlier example and also assuming that a fetch-and-increment operation, which returns the incremented value, takes 100 clock cycles, determine the time for 10 processors to traverse the barrier. How many bus cycles are required?

```
local_sense =! local_sense; /* toggle local_sense */
fetch_and_increment(count);/* atomic update */
if (count==total) {/* all arrived */
       count=0;/* reset counter */
       release=local_sense;/* release processes */
}
else {/* more to come */
       spin (release==local_sense);/* wait for signal */
}
```

Figure I.7 Code for a sense-reversing barrier using fetch-and-increment to do the counting.

Answer Figure I.7 shows the code for the barrier. For n processors, this implementation requires n fetch-and-increment operations, n cache misses to access the count, and n cache misses for the release operation, for a total of $3n$ bus transactions. For 10 processors, this is 30 bus transactions or 3000 clock cycles. Like the queueing lock, the time is linear in the number of processors. Of course, fetch-and-increment can also be used in implementing the combining tree barrier, reducing the serialization at each node in the tree.

As we have seen, synchronization problems can become quite acute in largerscale multiprocessors. When the challenges posed by synchronization are combined with the challenges posed by long memory latency and potential load imbalance in computations, we can see why getting efficient usage of large-scale parallel processors is very challenging.

I.5 Performance of Scientific Applications on Shared-Memory Multiprocessors

This section covers the performance of the scientific applications from Section I.3 on both symmetric shared-memory and distributed shared-memory multiprocessors.

Performance of a Scientific Workload on a Symmetric Shared-Memory Multiprocessor

We evaluate the performance of our four scientific applications on a symmetric shared-memory multiprocessor using the following problem sizes:

- *Barnes-Hut*—16 K bodies run for six time steps (the accuracy control is set to 1.0, a typical, realistic value)

- *FFT*—1 million complex data points
- *LU*—A 512×512 matrix is used with 16×16 blocks
- *Ocean*—A 130×130 grid with a typical error tolerance

In looking at the miss rates as we vary processor count, cache size, and block size, we decompose the total miss rate into *coherence misses* and normal uniprocessor misses. The normal uniprocessor misses consist of capacity, conflict, and compulsory misses. We label these misses as capacity misses because that is the dominant cause for these benchmarks. For these measurements, we include as a coherence miss any write misses needed to upgrade a block from shared to exclusive, even though no one is sharing the cache block. This measurement reflects a protocol that does not distinguish between a private and shared cache block.

Figure I.8 shows the data miss rates for our four applications, as we increase the number of processors from 1 to 16, while keeping the problem size constant. As we increase the number of processors, the total amount of cache increases, usually causing the capacity misses to drop. In contrast, increasing the processor count usually causes the amount of communication to increase, in turn causing the coherence misses to rise. The magnitude of these two effects differs by application.

In FFT, the capacity miss rate drops (from nearly 7% to just over 5%) but the coherence miss rate increases (from about 1% to about 2.7%), leading to a constant overall miss rate. Ocean shows a combination of effects, including some that relate to the partitioning of the grid and how grid boundaries map to cache blocks. For a typical 2D grid code the communication-generated misses are proportional to the boundary of each partition of the grid, while the capacity misses are proportional to the area of the grid. Therefore, increasing the total amount of cache while keeping the total problem size fixed will have a more significant effect on the capacity miss rate, at least until each subgrid fits within an individual processor's cache. The significant jump in miss rate between one and two processors occurs because of conflicts that arise from the way in which the multiple grids are mapped to the caches. This conflict is present for direct-mapped and two-way set associative caches, but fades at higher associativities. Such conflicts are not unusual in array-based applications, especially when there are multiple grids in use at once. In Barnes and LU, the increase in processor count has little effect on the miss rate, sometimes causing a slight increase and sometimes causing a slight decrease.

Increasing the cache size usually has a beneficial effect on performance, since it reduces the frequency of costly cache misses. Figure I.9 illustrates the change in miss rate as cache size is increased for 16 processors, showing the portion of the miss rate due to coherence misses and to uniprocessor capacity misses. Two effects can lead to a miss rate that does not decrease—at least not as quickly as we might expect—as cache size increases: inherent communication and plateaus in the miss rate. Inherent communication leads to a certain frequency of coherence misses that are not significantly affected by increasing cache size. Thus, if the cache size is increased while maintaining a fixed problem size, the coherence miss rate

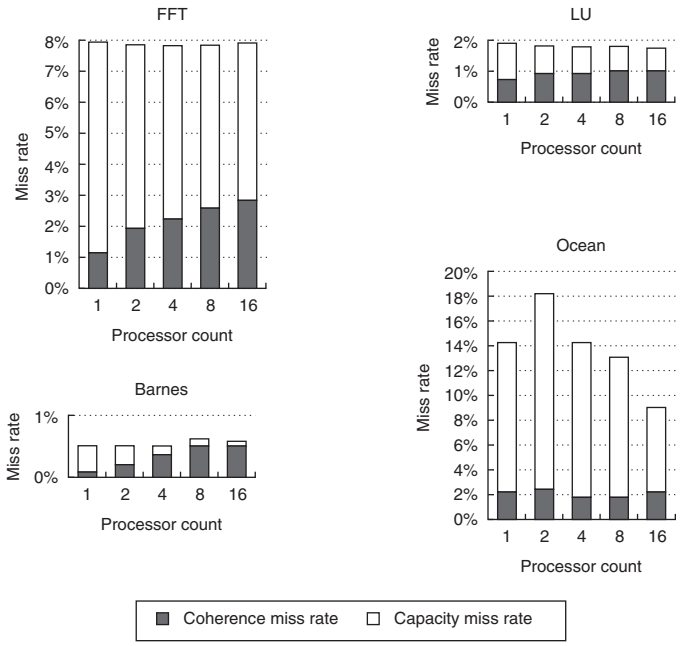


Figure I.8 Data miss rates can vary in nonobvious ways as the processor count is increased from 1 to 16. The miss rates include both coherence and capacity miss rates. The compulsory misses in these benchmarks are all very small and are included in the capacity misses. Most of the misses in these applications are generated by accesses to data that are potentially shared, although in the applications with larger miss rates (FFT and Ocean), it is the capacity misses rather than the coherence misses that comprise the majority of the miss rate. Data are potentially shared if they are allocated in a portion of the address space used for shared data. In all except Ocean, the potentially shared data are heavily shared, while in Ocean only the boundaries of the subgrids are actually shared, although the entire grid is treated as a potentially shared data object. Of course, since the boundaries change as we increase the processor count (for a fixed-size problem), different amounts of the grid become shared. The anomalous increase in capacity miss rate for Ocean in moving from 1 to 2 processors arises because of conflict misses in accessing the subgrids. In all cases except Ocean, the fraction of the cache misses caused by coherence transactions rises when a fixed-size problem is run on an increasing number of processors. In Ocean, the coherence misses initially fall as we add processors due to a large number of misses that are write ownership misses to data that are potentially, but not actually, shared. As the subgrids begin to fit in the aggregate cache (around 16 processors), this effect lessens. The single-processor numbers include write upgrade misses, which occur in this protocol even if the data are not actually shared, since they are in the shared state. For all these runs, the cache size is 64 KB, two-way set associative, with 32-byte blocks. Notice that the scale on the *y*-axis for each benchmark is different, so that the behavior of the individual benchmarks can be seen clearly.

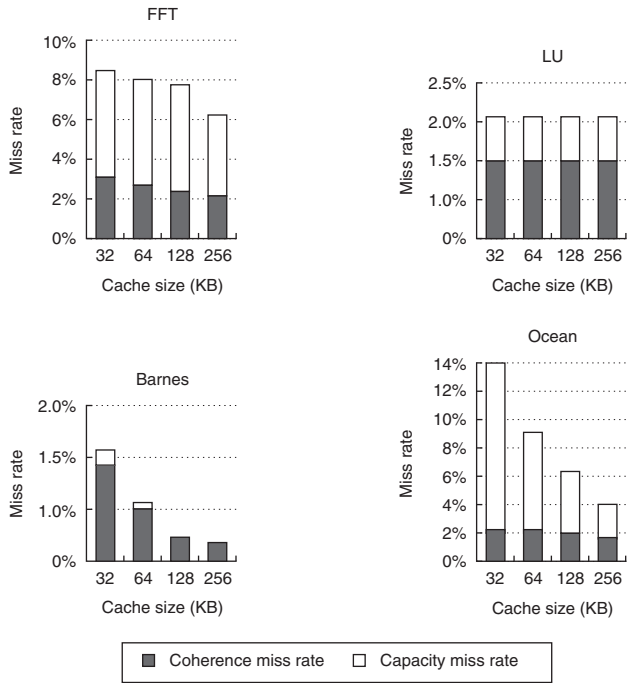


Figure I.9 The miss rate usually drops as the cache size is increased, although coherence misses dampen the effect. The block size is 32 bytes and the cache is two-way set associative. The processor count is fixed at 16 processors. Observe that the scale for each graph is different.

eventually limits the decrease in cache miss rate. This effect is most obvious in Barnes, where the coherence miss rate essentially becomes the entire miss rate.

A less important effect is a temporary plateau in the capacity miss rate that arises when the application has some fraction of its data present in cache but some significant portion of the dataset does not fit in the cache or in caches that are slightly bigger. In LU, a very small cache (about 4 KB) can capture the pair of 16×16 blocks used in the inner loop; beyond that, the next big improvement in capacity miss rate occurs when both matrices fit in the caches, which occurs when the total cache size is between 4 MB and 8 MB. This effect, sometimes called a *working set effect*, is partly at work between 32 KB and 128 KB for FFT, where the capacity miss rate drops only 0.3%. Beyond that cache size, a faster decrease in the capacity miss rate is seen, as a major data structure begins to reside in the cache. These plateaus are common in programs that deal with large arrays in a structured fashion.

Increasing the block size is another way to change the miss rate in a cache. In uniprocessors, larger block sizes are often optimal with larger caches. In

multiprocessors, two new effects come into play: a reduction in spatial locality for shared data and a potential increase in miss rate due to false sharing. Several studies have shown that shared data have lower spatial locality than unshared data. Poorer locality means that, for shared data, fetching larger blocks is less effective than in a uniprocessor because the probability is higher that the block will be replaced before all its contents are referenced. Likewise, increasing the basic size also increases the potential frequency of false sharing, increasing the miss rate.

Figure I.10 shows the miss rates as the cache block size is increased for a 16-processor run with a 64 KB cache. The most interesting behavior is in Barnes, where the miss rate initially declines and then rises due to an increase in the number of coherence misses, which probably occurs because of false sharing. In the other benchmarks, increasing the block size decreases the overall miss rate. In Ocean and LU, the block size increase affects both the coherence and capacity miss rates about equally. In FFT, the coherence miss rate is actually decreased at a faster rate than the capacity miss rate. This reduction occurs because the communication in FFT is structured to be very efficient. In less optimized programs, we would expect more

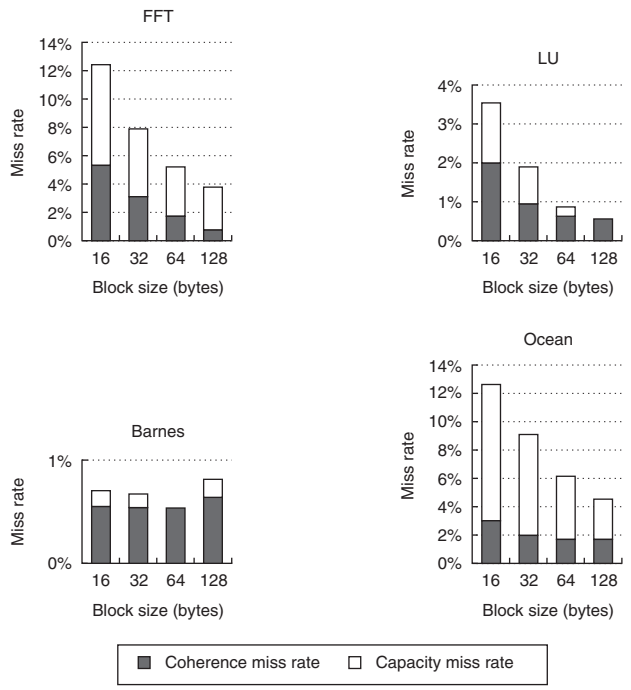


Figure I.10 The data miss rate drops as the cache block size is increased. All these results are for a 16-processor run with a 64 KB cache and two-way set associativity. Once again we use different scales for each benchmark.

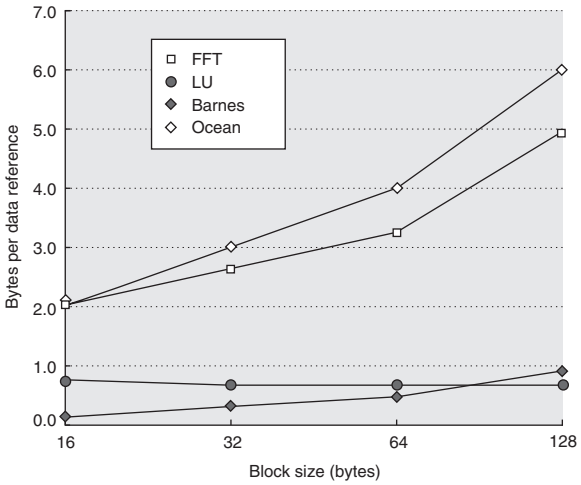


Figure I.11 Bus traffic for data misses climbs steadily as the block size in the data cache is increased. The factor of 3 increase in traffic for Ocean is the best argument against larger block sizes. Remember that our protocol treats ownership or upgrade misses the same as other misses, slightly increasing the penalty for large cache blocks; in both Ocean and FFT, this simplification accounts for less than 10% of the traffic.

false sharing and less spatial locality for shared data, resulting in more behavior like that of Barnes.

Although the drop in miss rates with longer blocks may lead you to believe that choosing a longer block size is the best decision, the bottleneck in bus-based multiprocessors is often the limited memory and bus bandwidth. Larger blocks mean more bytes on the bus per miss. Figure I.11 shows the growth in bus traffic as the block size is increased. This growth is most serious in the programs that have a high miss rate, especially Ocean. The growth in traffic can actually lead to performance slowdowns due both to longer miss penalties and to increased bus contention.

Performance of a Scientific Workload on a Distributed-Memory Multiprocessor

The performance of a directory-based multiprocessor depends on many of the same factors that influence the performance of bus-based multiprocessors (e.g., cache size, processor count, and block size), as well as the distribution of misses to various locations in the memory hierarchy. The location of a requested data item depends on both the initial allocation and the sharing patterns. We start by examining the basic cache performance of our scientific/technical workload and then look at the effect of different types of misses.

Because the multiprocessor is larger and has longer latencies than our snooping-based multiprocessor, we begin with a slightly larger cache (128 KB) and a larger block size of 64 bytes.

In distributed-memory architectures, the distribution of memory requests between local and remote is key to performance because it affects both the consumption of global bandwidth and the latency seen by requests. Therefore, for the figures in this section, we separate the cache misses into local and remote requests. In looking at the figures, keep in mind that, for these applications, most of the remote misses that arise are coherence misses, although some capacity misses can also be remote, and in some applications with poor data distribution such misses can be significant.

As Figure I.12 shows, the miss rates with these cache sizes are not affected much by changes in processor count, with the exception of Ocean, where the miss rate rises at 64 processors. This rise results from two factors: an increase in mapping conflicts in the cache that occur when the grid becomes small, which leads to a rise in local misses, and an increase in the number of the coherence misses, which are all remote.

Figure I.13 shows how the miss rates change as the cache size is increased, assuming a 64-processor execution and 64-byte blocks. These miss rates decrease at rates that we might expect, although the dampening effect caused by little or no reduction in coherence misses leads to a slower decrease in the remote misses than in the local misses. By the time we reach the largest cache size shown, 512 KB, the remote miss rate is equal to or greater than the local miss rate. Larger caches would amplify this trend.

We examine the effect of changing the block size in Figure I.14. Because these applications have good spatial locality, increases in block size reduce the miss rate, even for large blocks, although the performance benefits for going to the largest blocks are small. Furthermore, most of the improvement in miss rate comes from a reduction in the local misses.

Rather than plot the memory traffic, Figure I.15 plots the number of bytes required per data reference versus block size, breaking the requirement into local and global bandwidth. In the case of a bus, we can simply aggregate the demands of each processor to find the total demand for bus and memory bandwidth. For a scalable interconnect, we can use the data in Figure I.15 to compute the required per-node global bandwidth and the estimated bisection bandwidth, as the next example shows.

Example Assume a 64-processor multiprocessor with 1 GHz processors that sustain one memory reference per processor clock. For a 64-byte block size, the remote miss rate is 0.7%. Find the per-node and estimated bisection bandwidth for FFT. Assume that the processor does not stall for remote memory requests; this might be true if, for example, all remote data were prefetched. How do these bandwidth requirements compare to various interconnection technologies?

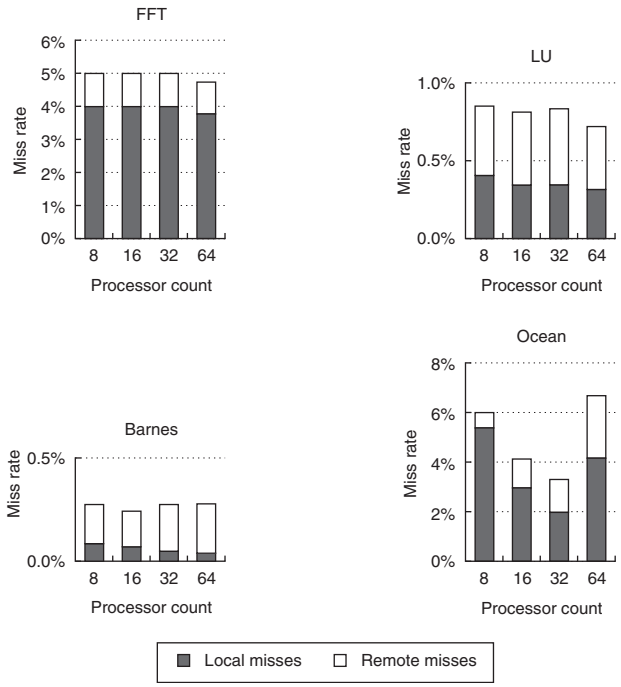


Figure I.12 The data miss rate is often steady as processors are added for these benchmarks. Because of its grid structure, Ocean has an initially decreasing miss rate, which rises when there are 64 processors. For Ocean, the local miss rate drops from 5% at 8 processors to 2% at 32, before rising to 4% at 64. The remote miss rate in Ocean, driven primarily by communication, rises monotonically from 1% to 2.5%. Note that, to show the detailed behavior of each benchmark, different scales are used on the *y*-axis. The cache for all these runs is 128 KB, two-way set associative, with 64-byte blocks. Remote misses include any misses that require communication with another node, whether to fetch the data or to deliver an invalidate. In particular, in this figure and other data in this section, the measurement of remote misses includes write upgrade misses where the data are up to date in the local memory but cached elsewhere and, therefore, require invalidations to be sent. Such invalidations do indeed generate remote traffic, but may or may not delay the write, depending on the consistency model.

FFT performs all-to-all communication, so the bisection bandwidth is equal to the number of processors times the per-node bandwidth, or about 64×448 MB/sec $= 28.7$ GB/sec. The SGI Origin 3000 with 64 processors has a bisection bandwidth of about 50 GB/sec. No standard networking technology comes close.

Answer The per-node bandwidth is simply the number of data bytes per reference times the reference rate: $0.7\% \times 1$ GB/sec $\times 64 = 448$ MB/sec. This rate is somewhat higher than the hardware sustainable transfer rate for the CrayT3E (using a block prefetch)

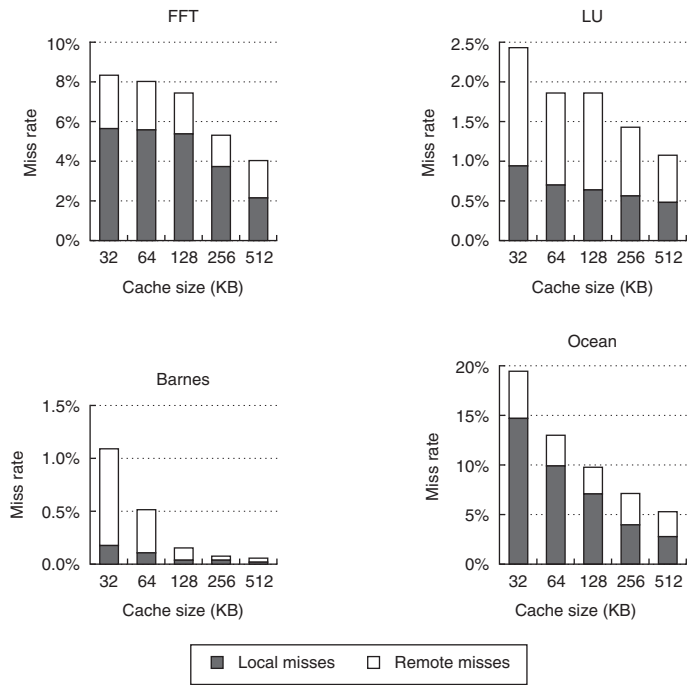


Figure I.13 Miss rates decrease as cache sizes grow. Steady decreases are seen in the local miss rate, while the remote miss rate declines to varying degrees, depending on whether the remote miss rate had a large capacity component or was driven primarily by communication misses. In all cases, the decrease in the local miss rate is larger than the decrease in the remote miss rate. The plateau in the miss rate of FFT, which we mentioned in the last section, ends once the cache exceeds 128 KB. These runs were done with 64 processors and 64-byte cache blocks.

and lower than that for an SGI Origin 3000 (1.6 GB/processor pair). The FFT per-node bandwidth demand exceeds the bandwidth sustainable from the fastest standard networks by more than a factor of 5.

The previous example looked at the bandwidth demands. The other key issue for a parallel program is remote memory access time, or latency. To get insight into this, we use a simple example of a directory-based multiprocessor. Figure I.16 shows the parameters we assume for our simple multiprocessor model. It assumes that the time to first word for a local memory access is 85 processor cycles and that the path to local memory is 16 bytes wide, while the network interconnect is 4 bytes wide. This model ignores the effects of contention, which are probably not too serious in the parallel benchmarks we examine, with the possible exception of FFT, which uses all-to-all communication. Contention could have a serious performance impact in other workloads.

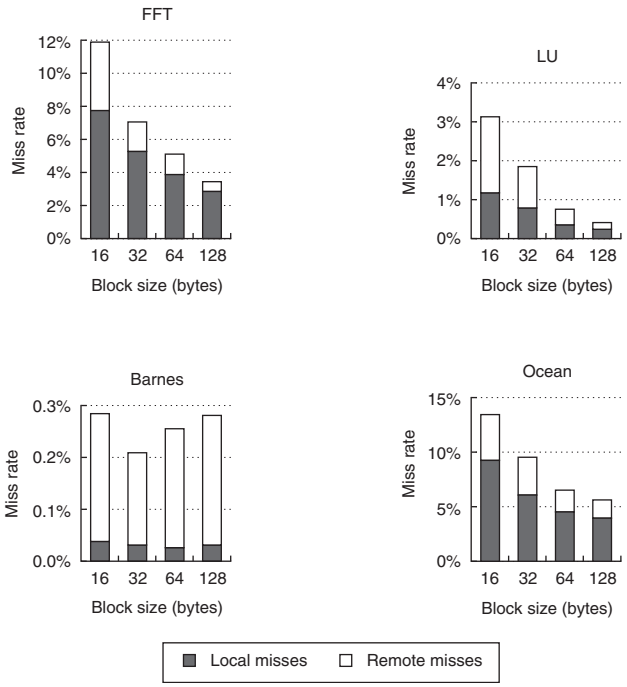


Figure I.14 Data miss rate versus block size assuming a 128 KB cache and 64 processors in total. Although difficult to see, the coherence miss rate in Barnes actually rises for the largest block size, just as in the last section.

Figure I.17 shows the cost in cycles for the average memory reference, assuming the parameters in Figure I.16. Only the latencies for each reference type are counted. Each bar indicates the contribution from cache hits, local misses, remote misses, and three-hop remote misses. The cost is influenced by the total frequency of cache misses and upgrades, as well as by the distribution of the location where the miss is satisfied. The cost for a remote memory reference is fairly steady as the processor count is increased, except for Ocean. The increasing miss rate in Ocean for 64 processors is clear in Figure I.12. As the miss rate increases, we should expect the time spent on memory references to increase also.

Although Figure I.17 shows the memory access cost, which is the dominant multiprocessor cost in these benchmarks, a complete performance model would need to consider the effect of contention in the memory system, as well as the losses arising from synchronization delays.

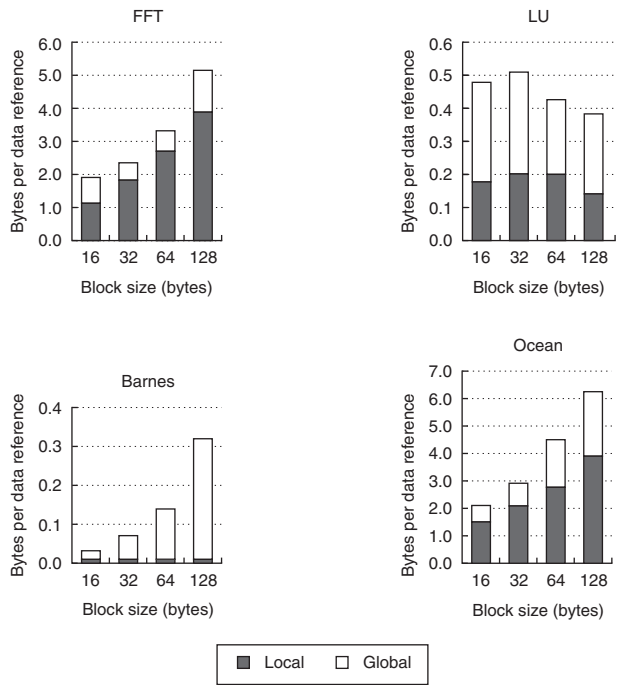


Figure I.15 The number of bytes per data reference climbs steadily as block size is increased. These data can be used to determine the bandwidth required per node both internally and globally. The data assume a 128 KB cache for each of 64 processors.

Characteristic	Processor clock cycles ≤16 processors	Processor clock cycles 17–64 processors
Cache hit	1	1
Cache miss to local memory	85	85
Cache miss to remote home directory	125	150
Cache miss to remotely cached data (three-hop miss)	140	170

Figure I.16 Characteristics of the example directory-based multiprocessor. Misses can be serviced locally (including from the local directory), at a remote home node, or using the services of both the home node and another remote node that is caching an exclusive copy. This last case is called a three-hop miss and has a higher cost because it requires interrogating both the home directory and a remote cache. Note that this simple model does not account for invalidation time but does include some factor for increasing interconnect time. These remote access latencies are based on those in an SGI Origin 3000, the fastest scalable interconnect system in 2001, and assume a 500 MHz processor.

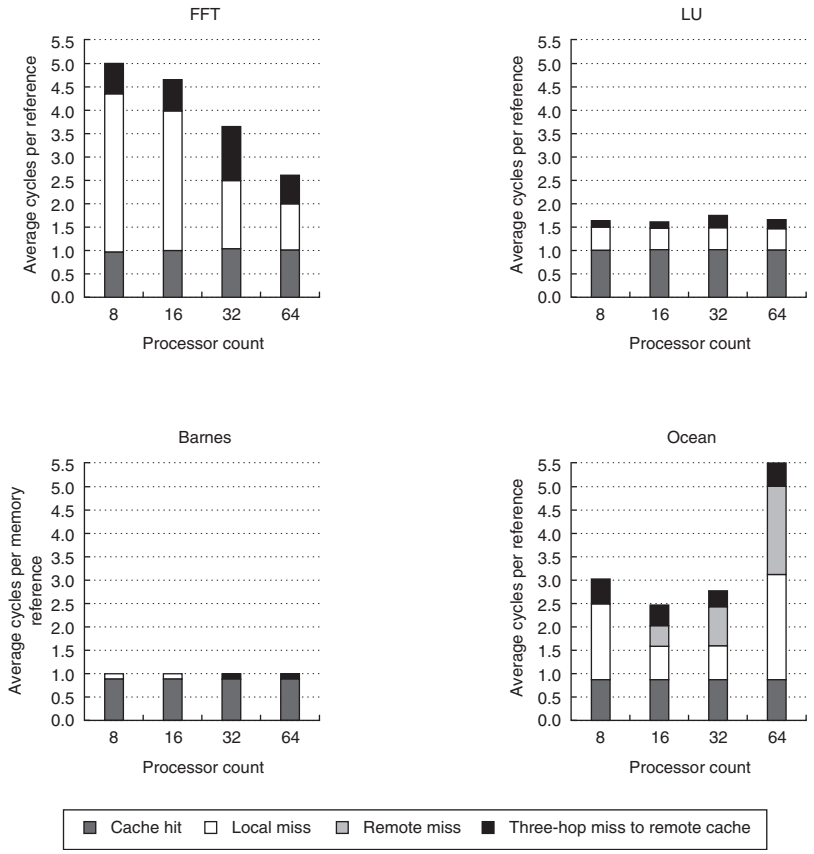


Figure I.17 The effective latency of memory references in a DSM multiprocessor depends both on the relative frequency of cache misses and on the location of the memory where the accesses are served. These plots show the memory access cost (a metric called average memory access time in Chapter 2) for each of the benchmarks for 8, 16, 32, and 64 processors, assuming a 512 KB data cache that is two-way set associative with 64-byte blocks. The average memory access cost is composed of four different types of accesses, with the cost of each type given in Figure I.16. For the Barnes and LU benchmarks, the low miss rates lead to low overall access times. In FFT, the higher access cost is determined by a higher local miss rate (1–4%) and a significant three-hop miss rate (1%). The improvement in FFT comes from the reduction in local miss rate from 4% to 1%, as the aggregate cache increases. Ocean shows the biggest change in the cost of memory accesses, and the highest overall cost at 64 processors. The high cost is driven primarily by a high local miss rate (average 1.6%). The memory access cost drops from 8 to 16 processors as the grids more easily fit in the individual caches. At 64 processors, the dataset size is too small to map properly and both local misses and coherence misses rise, as we saw in Figure I.12.

I.6 Performance Measurement of Parallel Processors with Scientific Applications

One of the most controversial issues in parallel processing has been how to measure the performance of parallel processors. Of course, the straightforward answer is to measure a benchmark as supplied and to examine wall-clock time. Measuring wall-clock time obviously makes sense; in a parallel processor, measuring CPU time can be misleading because the processors may be idle but unavailable for other uses.

Users and designers are often interested in knowing not just how well a multiprocessor performs with a certain fixed number of processors, but also how the performance scales as more processors are added. In many cases, it makes sense to scale the application or benchmark, since if the benchmark is unscaled, effects arising from limited parallelism and increases in communication can lead to results that are pessimistic when the expectation is that more processors will be used to solve larger problems. Thus, it is often useful to measure the speedup as processors are added both for a fixed-size problem and for a scaled version of the problem, providing an unscaled and a scaled version of the speedup curves. The choice of how to measure the uniprocessor algorithm is also important to avoid anomalous results, since using the parallel version of the benchmark may understate the uniprocessor performance and thus overstate the speedup.

Once we have decided to measure scaled speedup, the question is *how* to scale the application. Let's assume that we have determined that running a benchmark of size n on p processors makes sense. The question is how to scale the benchmark to run on $m \times p$ processors. There are two obvious ways to scale the problem: (1) keeping the amount of memory used per processor constant, and (2) keeping the total execution time, assuming perfect speedup, constant. The first method, called *memory-constrained scaling*, specifies running a problem of size $m \times n$ on $m \times p$ processors. The second method, called *time-constrained scaling*, requires that we know the relationship between the running time and the problem size, since the former is kept constant. For example, suppose the running time of the application with data size n on p processors is proportional to n^2/p. Then, with time-constrained scaling, the problem to run is the problem whose ideal running time on $m \times p$ processors is still n^2/p. The problem with this ideal running time has size $\sqrt{m} \times n$.

Example Suppose we have a problem whose execution time for a problem of size n is proportional to n^3. Suppose the actual running time on a 10-processor multiprocessor is 1 hour. Under the time-constrained and memory-constrained scaling models, find the size of the problem to run and the effective running time for a 100-processor multiprocessor.

Answer For the time-constrained problem, the ideal running time is the same, 1 hour, so the problem size is $\sqrt[3]{10} \times$n or 2.15 times larger than the original. For

memory-constrained scaling, the size of the problem is $10n$ and the ideal execution time is $10^3/10$, or 100 hours! Since most users will be reluctant to run a problem on an order of magnitude more processors for 100 times longer, this size problem is probably unrealistic.

In addition to the scaling methodology, there are questions as to how the program should be scaled when increasing the problem size affects the quality of the result. Often, we must change other application parameters to deal with this effect. As a simple example, consider the effect of time to convergence for solving a differential equation. This time typically increases as the problem size increases, since, for example, we often require more iterations for the larger problem. Thus, when we increase the problem size, the total running time may scale faster than the basic algorithmic scaling would indicate.

For example, suppose that the number of iterations grows as the log of the problem size. Then, for a problem whose algorithmic running time is linear in the size of the problem, the effective running time actually grows proportional to $n \log n$. If we scaled from a problem of size m on 10 processors, purely algorithmic scaling would allow us to run a problem of size $10\,m$ on 100 processors. Accounting for the increase in iterations means that a problem of size $k \times m$, where $k \log k = 10$, will have the same running time on 100 processors. This problem size yields a scaling of $5.72\,m$, rather than $10\,m$.

In practice, scaling to deal with error requires a good understanding of the application and may involve other factors, such as error tolerances (for example, it affects the cell-opening criteria in Barnes-Hut). In turn, such effects often significantly affect the communication or parallelism properties of the application as well as the choice of problem size.

Scaled speedup is not the same as unscaled (or true) speedup; confusing the two has led to erroneous claims (e.g., see the discussion in Section I.6). Scaled speedup has an important role, but only when the scaling methodology is sound and the results are clearly reported as using a scaled version of the application. Singh, Hennessy, and Gupta [1993] described these issues in detail.

I.7 Implementing Cache Coherence

In this section, we explore the challenge of implementing cache coherence, starting first by dealing with the challenges in a snooping coherence protocol, which we simply alluded to in Chapter 5. Implementing a directory protocol adds some additional complexity to a snooping protocol, primarily arising from the absence of broadcast, which forces the use of a different mechanism to resolve races. Furthermore, the larger processor count of a directory-based multiprocessor means that we cannot retain assumptions of unlimited buffering and must find new ways to avoid deadlock, Let's start with the snooping protocols.

As we mentioned in Chapter 5, the challenge of implementing misses in a snooping coherence protocol without a bus lies in finding a way to make the multistep miss process appear atomic. Both an upgrade miss and a write miss require the same basic processing and generate the same implementation challenges; for simplicity, we focus on upgrade misses. Here are the steps in handling an upgrade miss:

1. Detect the miss and compose an invalidate message for transmission to other caches.
2. When access to the broadcast communication link is available, transmit the message.
3. When the invalidates have been processed, the processor updates the state of the cache block and then proceeds with the write that caused the upgrade miss.

There are two related difficulties that can arise. First, how will two processors, P1 and P2, that attempt to upgrade the same cache block at the same time resolve the race? Second, when at step 3, how does a processor know when all invalidates have been processed so that it can complete the step?

The solution to finding a winner in the race lies in the ordering imposed by the broadcast communication medium. The communication medium must broadcast any cache miss to all the nodes. If P1 and P2 attempt to broadcast at the same time, we must ensure that either P1's message will reach P2 first or P2's will reach P1 first. This property will be true if there is a single channel through which all ingoing and outgoing requests from a node must pass through and if the communication network does not accept a message unless it can guarantee delivery (i.e., it is effectively circuit switched, see Appendix F). If both P1 and P2 initiate their attempts to broadcast an invalidate simultaneously, then the network can accept only one of these operations and delay the other. This ordering ensures that either P1 or P2 will complete its communication in step 2 first. The network can explicitly signal when it accepts a message and can guarantee it will be the next transmission; alternatively, a processor can simply watch the network for its own request, knowing that once the request is seen, it will be fully transmitted to all processors before any subsequent messages.

Now, suppose P1 wins the race to transmit its invalidate; once it knows it has won the race, it can continue with step 3 and complete the miss handling. There is a potential problem, however, for P2. When P2 undertook step 1, it believed that the block was in the shared state, but for P1 to advance at step 3, it must know that P2 has processed the invalidate, which must change the state of the block at P2 to invalid! One simple solution is for P2 to notice that it has lost the race, by observing that P1's invalidate is broadcast before its own invalidate. P2 can then invalidate the block and generate a write miss to get the data. P1 will see its invalidate before P2's, so it will change the block to modified and update the data, which guarantees forward progress and avoids deadlock. When P1 sees the subsequent invalidate to a block in the Modified state (a possibility that cannot arise in our basic protocol discussed in Chapter 5), it knows that it was the winner of a race. It can simply

ignore the invalidate, knowing that it will be followed by a write miss, or it can write the block back to memory and make its state invalid.

Another solution is to give precedence to incoming requests over outgoing requests, so that before P2 can transmit its invalidate it must handle any pending invalidates or write misses. If any of those misses are for blocks with the same address as a pending outgoing message, the processor must be prepared to restart the write operation, since the incoming request may cause the state of the block to change. Notice that P1 knows that the invalidates will be processed once it has successfully completed the broadcast, since precedence is given to invalidate messages over outgoing requests. (Because it does not employ broadcast, a processor using a directory protocol cannot know when an invalidate is received; instead, explicit acknowledgments are required, as we discuss in the next section. Indeed, as we will see, it cannot even know it has won the race to become the owner until its request is acknowledged.)

Reads will also require a multiple-step process, since we need to get the data back from memory or a remote cache (in a write-back cache system), but reads do not introduce fundamentally new problems beyond what exists for writes.

There are, however, a few additional tricky edge cases that must be handled correctly. For example, in a write-back cache, a processor can generate a read miss that requires a write-back, which it could delay, while giving the read miss priority. If a snoop request appears for the cache block that is to be written back, the processor must discover this and send the data back. Failure to do so can create a deadlock situation. A similar tricky situation exists when a processor generates a write miss, which will make a block exclusive, but, before the processor receives the data and can update the block, other processors generate read misses for that block. The read misses cannot be processed until the writing processor receives the data and updates the block.

One of the more difficult problems occurs in a write-back cache where the data for a read or write miss can come either from memory or from one of the processor caches, but the requesting processor will not know *a priori* where the data will come from. In most bus-based systems, a single global signal is used to indicate whether any processor has the exclusive (and hence the most up-to-date) copy; otherwise, the memory responds. These schemes can work with a pipelined interconnection by requiring that processors signal whether they have the exclusive copy within a fixed number of cycles after the miss is broadcast.

In a modern multiprocessor, however, it is essentially impossible to bound the amount of time required for a snoop request to be processed. Instead, a mechanism is required to determine whether the memory has an up-to-date copy. One solution is to add coherence bits to the memory, indicating whether the data are exclusive in a remote cache. This mechanism begins to move toward the directory approach, whose implementation challenges we consider next.

Implementing Cache Coherence in a DSM Multiprocessor

Implementing a directory-based cache coherence protocol requires overcoming all the problems related to nonatomic actions for a snooping protocol without

the use of broadcast (see Chapter 5), which forced a serialization on competing writes and also ensured the serialization required for the memory consistency model. Avoiding the need to broadcast is a central goal for a directory-based system, so another method for ensuring serialization is necessary.

The serialization of requests for exclusive access to a memory block is easily enforced since those requests will be serialized when they reach the unique directory for the specified block. If the directory controller simply ensures that one request is completely serviced before the next is begun, writes will be serialized. Because the requesters cannot know ahead of time who will win the race and because the communication is not a broadcast, the directory must signal to the winner when it completes the processing of the winner's request. This is done by a message that supplies the data on a write miss or by an explicit acknowledgment message that grants ownership in response to an invalidation request.

What about the loser in this race? The simplest solution is for the system to send a *negative acknowledge*, or *NAK*, which requires that the requesting node regenerate its request. (This is the equivalent of a collision in the broadcast network in a snooping scheme, which requires that one of the transmitting nodes retry its communication.) We will see in the next section why the NAK approach, as opposed to buffering the request, is attractive.

Although the acknowledgment that a requesting node has ownership is completed when the write miss or ownership acknowledgment message is transmitted, we still do not know that the invalidates have been received and processed by the nodes that were in the sharing set. All memory consistency models eventually require (either before the next cache miss or at a synchronization point, for example) that a processor knows that all the invalidates for a write have been processed. In a snooping scheme, the nature of the broadcast network provides this assurance.

How can we know when the invalidates are complete in a directory scheme? The only way to know that the invalidates have been completed is to have the destination nodes of the invalidate messages (the members of the sharing set) explicitly acknowledge the invalidation messages sent from the directory. Who should they be acknowledged to? There are two possibilities. In the first the acknowledgments can be sent to the directory, which can count them, and when all acknowledgments have been received, confirm this with a single message to the original requester. Alternatively, when granting ownership, the directory can tell the register how many acknowledgments to expect. The destinations of the invalidate messages can then send an acknowledgment directly to the requester, whose identity is provided by the directory. Most existing implementations use the latter scheme, since it reduces the possibility of creating a bottleneck at a directory. Although the requirement for acknowledgments is an additional complexity in directory protocols, this requirement arises from the avoidance of a serialization mechanism, such as the snooping broadcast operation, which in itself is the limit to scalability.

Avoiding Deadlock from Limited Buffering

A new complication in the implementation is introduced by the potential scale of a directory-based multiprocessor. In Chapter 5, we assumed that the network could always accept a coherence message and that the request would be acted upon at some point. In a much larger multiprocessor, this assumption of unlimited buffering may be unreasonable. What happens when the network does not have unlimited buffering? The major implication of this limit is that a cache or directory controller may be unable to complete a message send. This could lead to deadlock.

The potential deadlock arises from three properties, which characterize many deadlock situations:

1. More than one resource is needed to complete a transaction: Message buffers are needed to generate requests, create replies and acknowledgments, and accept replies.
2. Resources are held until a nonatomic transaction completes: The buffer used to create the reply cannot be freed until the reply is accepted, for reasons we will see shortly.
3. There is no global partial order on the acquisition of resources: Nodes can generate requests and replies at will.

These characteristics lead to deadlock, and avoiding deadlock requires breaking one of these properties. Freeing up resources without completing a transaction is difficult, since the transaction must be completely backed out and cannot be left half-finished. Hence, our approach will be to try to resolve the need for multiple resources. We cannot simply eliminate this need, but we can try to ensure that the resources will always be available.

One way to ensure that a transaction can always complete is to guarantee that there are always buffers to accept messages. Although this is possible for a small multiprocessor with processors that block on a cache miss or have a small number of outstanding misses, it may not be very practical in a directory protocol, since a single write could generate many invalidate messages. In addition, features such as prefetch and multiple outstanding misses increase the amount of buffering required. There is an alternative strategy, which most systems use and which ensures that a transaction will not actually be initiated until we can guarantee that it has the resources to complete. The strategy has four parts:

1. A separate network (physical or virtual) is used for requests and replies, where a reply is any message that a controller waits for in transitioning between states. This ensures that new requests cannot block replies that will free up buffers.
2. Every request that expects a reply allocates space to accept the reply when the request is generated. If no space is available, the request waits. This ensures that a node can always accept a reply message, which will allow the replying node to free its buffer.

3. Any controller can reject with a NAK any request, but it can never NAK a reply. This prevents a transaction from starting if the controller cannot guarantee that it has buffer space for the reply.
4. Any request that receives a NAK in response is simply retried.

To see that there are no deadlocks with the four properties above, we must ensure that all replies can be accepted and that every request is eventually serviced. Since a cache controller or directory controller always allocates a buffer to handle the reply before issuing a request, it can always accept the reply when it returns. To see that every request is eventually serviced, we need only show that any request could be completed. Since every request starts with a read or write miss at a cache, it is sufficient to show that any read or write miss is eventually serviced. Since the write miss case includes the actions for a read miss as a subset, we focus on showing the write misses are serviced. The simplest situation is when the block is uncached; since that case is subsumed by the case when the block is shared, we focus on the shared and exclusive cases. Let's consider the case where the block is shared:

- The CPU attempts to do a write and generates a write miss that is sent to the directory. For simplicity, we can assume that the processor is stalled. Although it may issue further requests, it should not issue a request for the same cache block until the first one is completed. Requests for independent blocks can be handled separately.
- The write miss is sent to the directory controller for this memory block. Note that although one cache controller handles all the requests for a given cache block, regardless of its memory contents, the directory controller handles requests for different blocks as independent events (assuming sufficient buffering, which is allocated before the directory issues any further messages on behalf of the request). The only conflict at the directory controller is when two requests arrive for the same block. The controller must wait for the first operation to be completed. It can simply NAK the second request or buffer it, but it should not service the second request for a given memory block until the first is completed.
- Now consider what happens at the directory controller: Suppose the write miss is the next thing to arrive at the directory controller. The controller sends out the invalidates, which can always be accepted after a limited delay by the cache controller. Note that one possibility is that the cache controller has an outstanding miss for the same block. This is the dual case to the snooping scheme, and we must once again break the tie by forcing the cache controller to accept and act on the directory request. Depending on the exact timing, this cache controller will either get the cache line later from the directory or will receive a NAK and have to restart the process.

The case where the block is exclusive is somewhat trickier. Our analysis begins when the write miss arrives at the directory controller for processing. There are two cases to consider:

- The directory controller sends a fetch/invalidate message to the processor where it arrives to find the block in the exclusive state. The cache controller sends a data write-back to the home directory and makes its state invalid. This reply arrives at the home directory controller, which can always accept the reply, since it preallocated the buffer. The directory controller sends back the data to the requesting processor, which can always accept the reply; after the cache is updated, the requesting cache controller notifies the processor.
- The directory controller sends a fetch/invalidate message to the node indicated as owner. When the message arrives at the owner node, it finds that this cache controller has taken a read or write miss that caused the block to be replaced. In this case, the cache controller has already sent the block to the home directory with a data write-back and made the data unavailable. Since this is exactly the effect of the fetch/invalidate message, the protocol operates correctly in this case as well.

We have shown that our coherence mechanism operates correctly when the cache and directory controller can accept requests for operation on cache blocks for which they have no outstanding operations in progress, when replies are always accepted, and when requests can be NAKed and forced to retry. Like the case of the snooping protocol, the cache controller must be able to break ties, and it always does so by favoring the instructions from the directory. The ability to NAK requests is what allows an implementation with finite buffering to avoid deadlock.

Implementing the Directory Controller

To implement a cache coherence scheme, the cache controller must have the same abilities it needed in the snooping case, namely, the capability of handling requests for independent blocks while awaiting a response to a request from the local processor. The incoming requests are still processed in order, and each one is completed before beginning the next. Should a cache controller receive too many requests in a short period of time, it can NAK them, knowing that the directory will subsequently regenerate the request.

The directory must also be multithreaded and able to handle requests for multiple blocks independently. This situation is somewhat different than having the cache controller handle incoming requests for independent blocks, since the directory controller will need to begin processing one request while an earlier one is still underway. The directory controller cannot wait for one to complete before servicing the next request, since this could lead to deadlock. Instead, the directory controller must be *reentrant*; that is, it must be capable of suspending its execution

while waiting for a reply and accepting another transaction. The only place this must occur is in response to read or write misses, while waiting for a response from the owner. This leads to three important observations:

1. The state of the controller need only be saved and restored while either a fetch operation from a remote location or a fetch/invalidate is outstanding.
2. The implementation can bound the number of outstanding transactions being handled in the directory by simply NAKing read or write miss requests that could cause the number of outstanding requests to be exceeded.
3. If instead of returning the data through the directory, the owner node forwards the data directly to the requester (as well as returning it to the directory), we can eliminate the need for the directory to handle more than one outstanding request. This motivation, in addition to the reduction of latency, is the reason for using the forwarding style of protocol. There are other complexities from forwarding protocols that arise when requests arrive closely spaced in time.

The major remaining implementation difficulty is to handle NAKs. One alternative is for each processor to keep track of its outstanding transactions so it knows, when the NAK is received, what the requested transaction was. The alternative is to bundle the original request into the NAK, so that the controller receiving the NAK can determine what the original request was. Because every request allocates a slot to receive a reply and a NAK is a reply, NAKs can always be received. In fact, the buffer holding the return slot for the request can also hold information about the request, allowing the processor to reissue the request if it is NAKed.

In practice, great care is required to implement these protocols correctly and to avoid deadlock. The key ideas we have seen in this section—dealing with nonatomicity and finite buffering—are critical to ensuring a correct implementation. Designers have found that both formal and informal verification techniques are helpful for ensuring that implementations are correct.

I.8 The Custom Cluster Approach: Blue Gene/L

Blue Gene/L (BG/L) is a scalable message-passing supercomputer whose design offers unprecedented computing density as measured by compute power per watt. By focusing on power efficiency, BG/L also achieves unmatched throughput per cubic foot. High computing density, combined with cost-effective nodes and extensive support for RAS, allows BG/L to efficiently scale to very large processor counts.

BG/L is a distributed-memory, message-passing computer but one that is quite different from the cluster-based, often throughput-oriented computers that rely on commodity technology in the processors, interconnect, and, sometimes, the packaging and system-level organization. BG/L uses a special customized processing node that contains two processors (derived from low-power, lower-clock-rate PowerPC 440 chips used in the embedded market), caches, and interconnect logic.

A complete computing node is formed by adding SDRAM chips, which are the only commodity semiconductor parts in the BG/L design.

BG/L consists of up to 64 K nodes organized into 32 racks each containing 1 K nodes in about 50 cubic feet. Each rack contains two double-sided boards with 512 nodes each. Due to the high density within a board and rack, 85% of the interconnect is within a single rack, greatly reducing the complexity and latency associated with connections between racks. Furthermore, the compact size of a rack, which is enabled by the low power and high density of each node, greatly improves efficiency, since the interconnection network for connections within a single rack are integrated into the single compute chip that comprises each node.

Appendix F discusses the main BL/G interconnect network, which is a three-dimensional torus. There are four other networks: Gigabit Ethernet, connected at designated I/O nodes; a JTAG network used for test; a barrier network; and a global collective network. The barrier network contains four independent channels and can be used for performing a global `or` or a global `and` across all the processors with latency of less than 1.5 microseconds. The global collective network connects all the processors in a tree and is used for global operations. It supports a variety of integer reductions directly, avoiding the need to involve the processor, and leading to times for large-scale reductions that are 10 to 100 times faster than in typical supercomputers. The collective network can also be used to broadcast a single value efficiently. Support for the collective network as well as the torus is included in the chip that forms of the heart of each processing node.

The Blue Gene/L Computing Node

Each BG/L node consists of a single processing chip and several SDRAM chips. The BG/L processing chip, shown in Figure I.18, contains the following:

1. Two PowerPC 440 CPUs, each a two-issue superscalar with a seven-stage pipeline and speculative out-order issue capability, clocked at a modest (and power-saving) 700 MHz. Each CPU has separate 32 KB I and D caches that are nonbblocking with up to four outstanding misses. Cache coherence must be enforced in software. Each CPU also contains a pair of floating-point coprocessors, each with its own FP register set and each capable of issuing a multiply-add each clock cycle, supporting a special SIMD instruction set capability that includes complex arithmetic using a pair of registers and 128-bit operands.
2. Separate fully associative L2 caches, each with 2 KB of data and a 128-byte block size, that act essentially like prefetch buffers. The L2 cache controllers recognize streamed data access and also handle prefetch from L3 or main memory. They have low latency (11 cycles) and provide high bandwidth (5 bytes per clock). The L2 prefetch buffer can supply 5.5 GB/sec to the L1 caches.
3. A 4 MB L3 cache implemented with embedded DRAM. Each L2 buffer is connected by a bus supplying 11 GB/sec of bandwidth from the L3 cache.

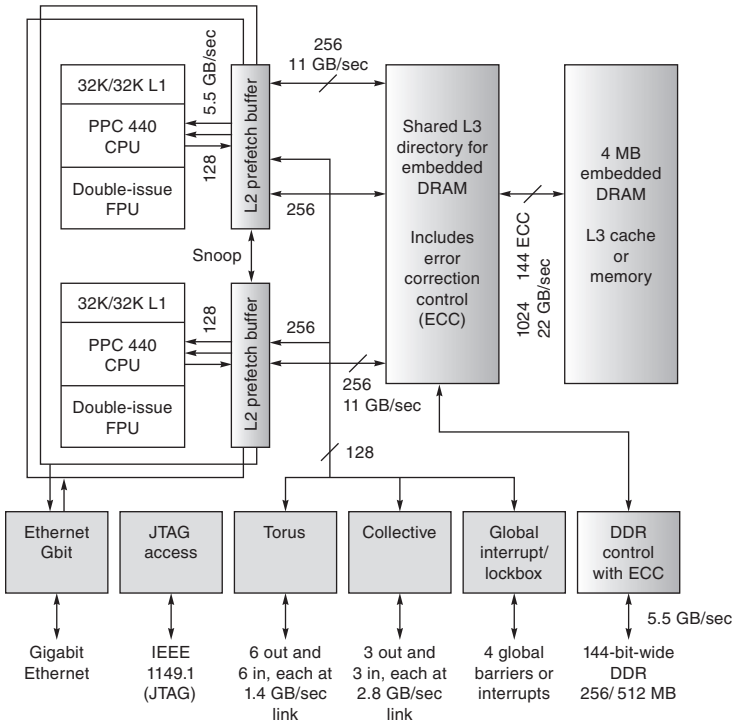


Figure I.18 The BG/L processing node. The unfilled boxes are the PowerPC processors with added floating-point units. The solid gray boxes are network interfaces, and the shaded lighter gray boxes are part of the memory system, which is supplemented by DDR RAMS.

4. A memory bus supporting 256 to 512 MB of DDR DRAMS and providing 5.5 GB/sec of memory bandwidth to the L3 cache. This amount of memory might seem rather modest for each node, given that the node contains two processors, each with two FP units. Indeed Amdahl's rule of thumb (1 MB per 1 MIPS) and an assumption of 25% of peak performance would favor about 2.7 times the memory per node. For floating-point-intensive applications where the computational need usually grows faster than linear in the memory size, the upper limit of 512 MB/node is probably reasonable.
5. Support logic for the five interconnection networks.

By placing all the logic other than DRAMs into a single chip, BG/L achieves higher density, lower power, and lower cost, making it possible to pack the processing nodes extremely densely. The density in terms allows the interconnection networks to be low latency, high bandwidth, and quite cost effective. The combination yields a supercomputer that scales very cost-effectively, yielding an order-of-magnitude improvement



Figure I.19 The 64 K-processor Blue Gene/L system.

in GFLOPs/watt over other approaches as well as significant improvements in GFLOPS/$ for very large-scale multiprocessors.

For example, BG/L with 64 K nodes has a peak performance of 360 TF and uses about 1.4 megawatts. To achieve 360 TF peak using the Power5+, which is the most power-efficient, high-end FP processor, would require about 23,500 processors (the dual processor can execute up to 8 FLOPs/clock at 1.9 GHz). The power requirement for just the processors, without external cache, DRAM, or interconnect, would be about 2.9 megawatts, or about double the power of the entire BG/L system. Likewise, the smaller die size of the BG/L node and its need for DRAMs as the only external chip produce significant cost savings versus a node built using a high-end multiprocessor. Figure I.19 shows a photo of the 64K node BG/L. The total size occupied by this 128K-processor multiprocessor is comparable to that occupied by earlier multiprocessors with 16K processors.

I.9 Concluding Remarks

The landscape of large-scale multiprocessors has changed dramatically over the past five to ten years. While some form of clustering is now used for all the largest-scale multiprocessors, calling them all "clusters" ignores significant differences in architecture, implementation style, cost, and performance. Bell and Gray

Terminology	Characteristics	Examples
MPP	Originally referred to a class of architectures characterized by large numbers of small, typically custom processors and usually using an SIMD style architecture.	Connection Machines CM-2
SMP (symmetric multiprocessor)	Shared-memory multiprocessors with a symmetric relationship to memory; also called UMA (uniform memory access). Scalable versions of these architectures used multistage interconnection networks, typically configured with at most 64 to 128 processors.	SUN Sunfire, NEC Earth Simulator
DSM (distributed shared memory)	A class of architectures that support scalable shared memory in a distributed fashion. These architectures are available both with and without cache coherence and typically can support hundreds to thousands of processors.	SGI Origin and Altix, Cray T3E, Cray X1, IBM p5 590/5
Cluster	A class of multiprocessors using message passing. The individual nodes are either commodities or customized, likewise the interconnect.	See commodity and custom clusters
Commodity cluster	A class of clusters where the nodes are truly commodities, typically headless workstations, motherboards, or blade servers, connected with a SAN or LAN usually accessible via an I/O bus.	"Beowulf" and other "homemade" clusters
Custom cluster	A cluster architecture where the nodes and the interconnect are customized and more tightly integrated than in a commodity cluster. Also called distributed memory or message passing multiprocessors.	IBM Blue Gene, Cray XT3
Constellation	Large-scale multiprocessors that use clustering of smaller-scale multiprocessors, typically with a DSM or SMP architecture and 32 or more processors.	Larger SGI Origin/Altix, ASC Purple

Figure I.20 A classification of large-scale multiprocessors. The term *MPP,* which had the original meaning described above, has been used more recently, and less precisely, to refer to all large-scale multiprocessors. None of the commercial shipping multiprocessors is a true MPP in the original sense of the word, but such an approach may make sense in the future. Both the SMP and DSM class includes multiprocessors with vector support. The term *constellation* has been used in different ways; the above usage seems both intuitive and precise [Dongarra et al. 2005].

[2002] discussed this trend, arguing that clusters will dominate. While Dongarra et al. [2005] agreed that some form of clustering is almost inevitable in the largest multiprocessors, they developed a more nuanced classification that attempts to distinguish among a variety of different approaches.

In Figure I.20 we summarize the range of terminology that has been used for large-scale multiprocessors and focus on defining the terms from an architectural and implementation perspective. Figure I.21 shows the hierarchical relationship of these different architecture approaches. Although there has been some convergence in architectural approaches over the past 15 years, the TOP500 list, which reports the 500 fastest computers in the world as measured by the Linpack benchmark, includes commodity clusters, customized clusters, Symmetric Multiprocessors (SMPs), DSMs, and constellations, as well as processors that are both scalar and vector.

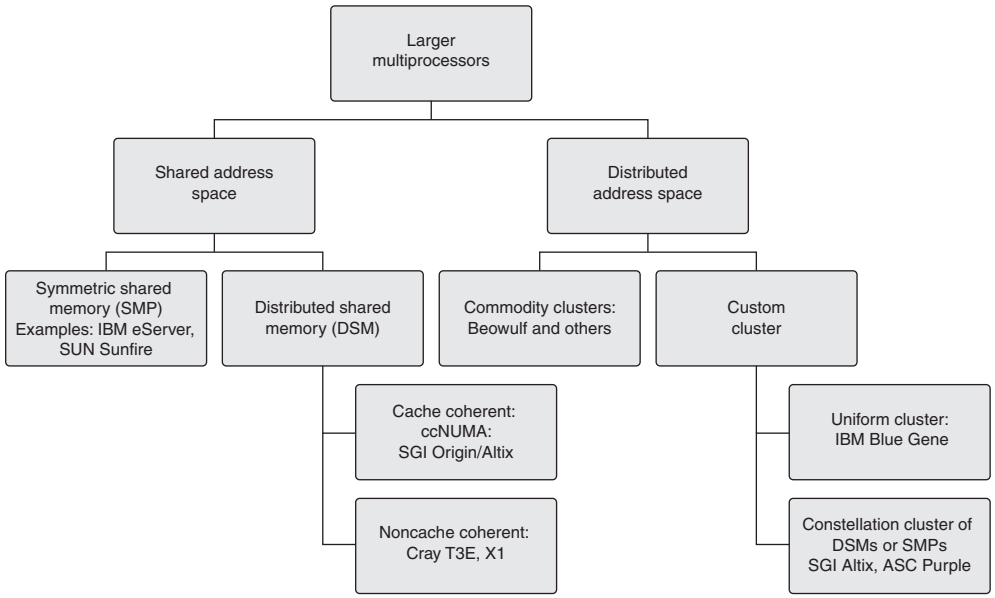


Figure I.21 The space of large-scale multiprocessors and the relation of different classes.

Nonetheless, there are some clearly emerging trends, which we can see by looking at the distribution of types of multiprocessors in the TOP500 list:

1. Clusters represent a majority of the systems. The lower development effort for clusters has clearly been a driving force in making them more popular. The high-end multiprocessor market has not grown sufficiently large to support full-scale, highly customized designs as the dominant choice.
2. The majority of the clusters are commodity clusters, often put together by users, rather than a system vendor designing a standard product.
3. Although commodity clusters dominate in their representation, the top 25 entries on the list are much more varied and include 9 custom clusters (primarily instances of Blue Gene or Cray XT3 systems), 2 constellations, 8 commodity clusters, 2 SMPs (one of which is the NEC Earth Simulator, which has nodes with vector processors), and 4 DSM multiprocessors.
4. Vector processors, which once dominated the list, have almost disappeared.
5. The IBM Blue Gene dominates the top 10 systems, showing the advantage of an approach the uses some commodity processor cores, but customizes many other functions and balances performance, power, and packaging density.
6. Architectural convergence has been driven more by market effects (lack of growth, limited suppliers, etc.) than by a clear-cut consensus on the best architectural approaches.

Software, both applications and programming languages and environments, remains the big challenge for parallel computing, just as it was 30 years ago, when multiprocessors such as the Illiac IV were being designed. The combination of ease of programming with high parallel performance remains elusive. Until better progress is made on this front, convergence toward a single programming model and underlying architectural approach (remembering that for uniprocessors we essentially have one programming model and one architectural approach!) will be slow or will be driven by factors other than proven architectural superiority.

J.1	Introduction	J-2
J.2	Basic Techniques of Integer Arithmetic	J-2
J.3	Floating Point	J-13
J.4	Floating-Point Multiplication	J-17
J.5	Floating-Point Addition	J-21
J.6	Division and Remainder	J-27
J.7	More on Floating-Point Arithmetic	J-32
J.8	Speeding Up Integer Addition	J-37
J.9	Speeding Up Integer Multiplication and Division	J-44
J.10	Putting It All Together	J-57
J.11	Fallacies and Pitfalls	J-62
J.12	Historical Perspective and References	J-63
	Exercises	J-67

J

Computer Arithmetic

by David Goldberg
Xerox Palo Alto Research Center

The Fast drives out the Slow even if the Fast is wrong.

W. Kahan

J.1 Introduction

Although computer arithmetic is sometimes viewed as a specialized part of CPU design, it is a very important part. This was brought home for Intel in 1994 when their Pentium chip was discovered to have a bug in the divide algorithm. This floating-point flaw resulted in a flurry of bad publicity for Intel and also cost them a lot of money. Intel took a $300 million write-off to cover the cost of replacing the buggy chips.

In this appendix, we will study some basic floating-point algorithms, including the division algorithm used on the Pentium. Although a tremendous variety of algorithms have been proposed for use in floating-point accelerators, actual implementations are usually based on refinements and variations of the few basic algorithms presented here. In addition to choosing algorithms for addition, subtraction, multiplication, and division, the computer architect must make other choices. What precisions should be implemented? How should exceptions be handled? This appendix will give you the background for making these and other decisions.

Our discussion of floating point will focus almost exclusively on the IEEE floating-point standard (IEEE 754) because of its rapidly increasing acceptance. Although floating-point arithmetic involves manipulating exponents and shifting fractions, the bulk of the time in floating-point operations is spent operating on fractions using integer algorithms (but not necessarily sharing the hardware that implements integer instructions). Thus, after our discussion of floating point, we will take a more detailed look at integer algorithms.

Some good references on computer arithmetic, in order from least to most detailed, are Chapter 3 of Patterson and Hennessy [2009]; Chapter 7 of Hamacher, Vranesic, and Zaky [1984]; Gosling [1980]; and Scott [1985].

J.2 Basic Techniques of Integer Arithmetic

Readers who have studied computer arithmetic before will find most of this section to be review.

Ripple-Carry Addition

Adders are usually implemented by combining multiple copies of simple components. The natural components for addition are *half adders* and *full adders*. The half adder takes two bits a and b as input and produces a sum bit s and a carry bit c_{out} as output. Mathematically, $s = (a+b) \bmod 2$, and $c_{\text{out}} = \lfloor (a+b)/2 \rfloor$, where $\lfloor\ \rfloor$ is the floor function. As logic equations, $s = a\overline{b} + \overline{a}b$ and $c_{\text{out}} = ab$, where ab means $a \wedge b$ and $a+b$ means $a \vee b$. The half adder is also called a (2,2) adder, since it takes two inputs and produces two outputs. The full adder

is a (3,2) adder and is defined by $s = (a+b+c) \bmod 2$, $c_{\text{out}} = \lfloor (a+b+c)/2 \rfloor$, or the logic equations

J.2.1 $$s = a\bar{b}\bar{c} + \bar{a}b\bar{c} + \bar{a}\bar{b}c + abc$$

J.2.2 $$c_{\text{out}} = ab + ac + bc$$

The principal problem in constructing an adder for n-bit numbers out of smaller pieces is propagating the carries from one piece to the next. The most obvious way to solve this is with a *ripple-carry adder*, consisting of n full adders, as illustrated in Figure J.1. (In the figures in this appendix, the least-significant bit is always on the right.) The inputs to the adder are $a_{n-1}a_{n-2}\cdots a_0$ and $b_{n-1}b_{n-2}\cdots b_0$, where $a_{n-1}a_{n-2}\cdots a_0$ represents the number $a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \cdots + a_0$. The c_{i+1} output of the ith adder is fed into the c_{i+1} input of the next adder (the $(i+1)$-th adder) with the lower-order carry-in c_0 set to 0. Since the low-order carry-in is wired to 0, the low-order adder could be a half adder. Later, however, we will see that setting the low-order carry-in bit to 1 is useful for performing subtraction.

In general, the time a circuit takes to produce an output is proportional to the maximum number of logic levels through which a signal travels. However, determining the exact relationship between logic levels and timings is highly technology dependent. Therefore, when comparing adders we will simply compare the number of logic levels in each one. How many levels are there for a ripple-carry adder? It takes two levels to compute c_1 from a_0 and b_0. Then it takes two more levels to compute c_2 from c_1, a_1, b_1, and so on, up to c_n. So, there are a total of $2n$ levels. Typical values of n are 32 for integer arithmetic and 53 for double-precision floating point. The ripple-carry adder is the slowest adder, but also the cheapest. It can be built with only n simple cells, connected in a simple, regular way.

Because the ripple-carry adder is relatively slow compared with the designs discussed in Section J.8, you might wonder why it is used at all. In technologies like CMOS, even though ripple adders take time $O(n)$, the constant factor is very small. In such cases short ripple adders are often used as building blocks in larger adders.

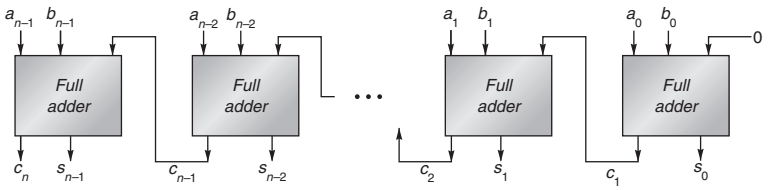


Figure J.1 Ripple-carry adder, consisting of *n* full adders. The carry-out of one full adder is connected to the carry-in of the adder for the next most-significant bit. The carries ripple from the least-significant bit (on the right) to the most-significant bit (on the left).

Radix-2 Multiplication and Division

The simplest multiplier computes the product of two unsigned numbers, one bit at a time, as illustrated in Figure J.2(a). The numbers to be multiplied are $a_{n-1}a_{n-2}\cdots a_0$ and $b_{n-1}b_{n-2}\cdots b_0$, and they are placed in registers A and B, respectively. Register P is initially 0. Each multiply step has two parts:

Multiply Step

(*i*) If the least-significant bit of A is 1, then register B, containing $b_{n-1}b_{n-2}\cdots b_0$, is added to P; otherwise, $00\cdots00$ is added to P. The sum is placed back into P.

(*ii*) Registers P and A are shifted right, with the carry-out of the sum being moved into the high-order bit of P, the low-order bit of P being moved into register A, and the rightmost bit of A, which is not used in the rest of the algorithm, being shifted out.

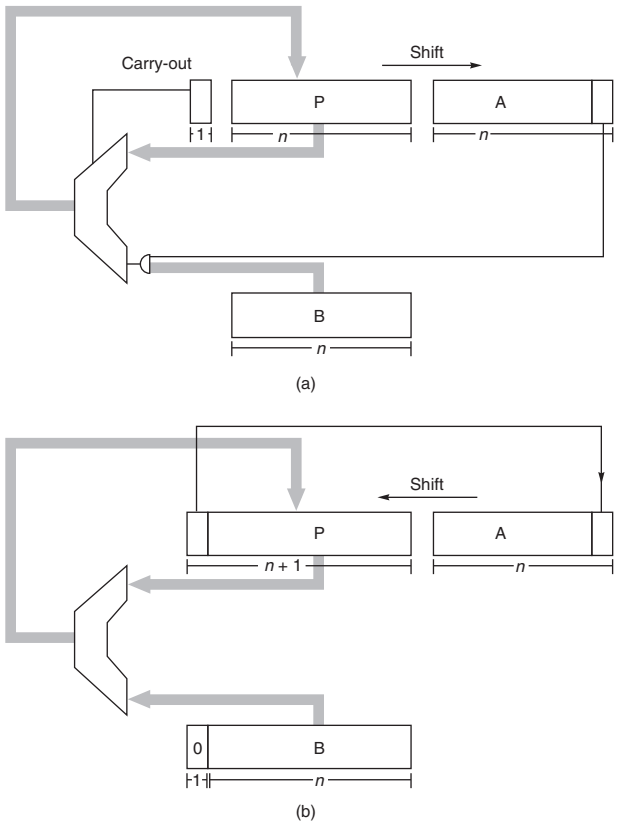


Figure J.2 Block diagram of (a) multiplier and (b) divider for *n*-bit unsigned integers. Each multiplication step consists of adding the contents of P to either B or 0 (depending on the low-order bit of A), replacing P with the sum, and then shifting both P and A one bit right. Each division step involves first shifting P and A one bit left, subtracting B from P, and, if the difference is nonnegative, putting it into P. If the difference is nonnegative, the low-order bit of A is set to 1.

After n steps, the product appears in registers P and A, with A holding the lower-order bits.

The simplest divider also operates on unsigned numbers and produces the quotient bits one at a time. A hardware divider is shown in Figure J.2(b). To compute a/b, put a in the A register, b in the B register, and 0 in the P register and then perform n divide steps. Each divide step consists of four parts:

Divide Step

(*i*) Shift the register pair (P,A) one bit left.

(*ii*) Subtract the content of register B (which is $b_{n-1}b_{n-2}\cdots b_0$) from register P, putting the result back into P.

(*iii*) If the result of step 2 is negative, set the low-order bit of A to 0, otherwise to 1.

(*iv*) If the result of step 2 is negative, restore the old value of P by adding the contents of register B back into P.

After repeating this process n times, the A register will contain the quotient, and the P register will contain the remainder. This algorithm is the binary version of the paper-and-pencil method; a numerical example is illustrated in Figure J.3(a).

Notice that the two block diagrams in Figure J.2 are very similar. The main difference is that the register pair (P,A) shifts right when multiplying and left when dividing. By allowing these registers to shift bidirectionally, the same hardware can be shared between multiplication and division.

The division algorithm illustrated in Figure J.3(a) is called *restoring,* because if subtraction by b yields a negative result, the P register is restored by adding b back in. The restoring algorithm has a variant that skips the restoring step and instead works with the resulting negative numbers. Each step of this *nonrestoring* algorithm has three parts:

Nonrestoring Divide Step

If P is negative,

(*i-a*) Shift the register pair (P,A) one bit left.

(*ii-a*) Add the contents of register B to P.

Else,

(*i-b*) Shift the register pair (P,A) one bit left.

(*ii-b*) Subtract the contents of register B from P.

(*iii*) If P is negative, set the low-order bit of A to 0, otherwise set it to 1.

After repeating this n times, the quotient is in A. If P is nonnegative, it is the remainder. Otherwise, it needs to be restored (i.e., add b), and then it will be the remainder. A numerical example is given in Figure J.3(b). Since steps (*i-a*) and (*i-b*) are the same, you might be tempted to perform this common step first, and then test the sign of P. That doesn't work, since the sign bit can be lost when shifting.

P	A	
00000	1110	Divide $14 = 1110_2$ by $3 = 11_2$. B always contains 0011_2.
00001	110	step 1(i): shift.
−00011		step 1(ii): subtract.
−00010	110**0**	step 1(iii): result is negative, set quotient bit to 0.
00001	110**0**	step 1(iv): restore.
00011	10**0**	step 2(i): shift.
−00011		step 2(ii): subtract.
00000	10**01**	step 2(iii): result is nonnegative, set quotient bit to 1.
00001	0**01**	step 3(i): shift.
−00011		step 3(ii): subtract.
−00010	0**010**	step 3(iii): result is negative, set quotient bit to 0.
00001	0**010**	step 3(iv): restore.
00010	**010**	step 4(i): shift.
−00011		step 4(ii): subtract.
−00001	**0100**	step 4(iii): result is negative, set quotient bit to 0.
00010	**0100**	step 4(iv): restore. The quotient is 0100_2 and the remainder is 00010_2.

(a)

00000	1110	Divide $14 = 1110_2$ by $3 = 11_2$. B always contains 0011_2.
00001	110	step 1(i-b): shift.
+11101		step 1(ii-b): subtract b (add two's complement).
11110	110**0**	step 1(iii): P is negative, so set quotient bit to 0.
11101	10**0**	step 2(i-a): shift.
+00011		step 2(ii-a): add b.
00000	10**01**	step 2(iii): P is nonnegative, so set quotient bit to 1.
00001	0**01**	step 3(i-b): shift.
+11101		step 3(ii-b): subtract b.
11110	0**010**	step 3(iii): P is negative, so set quotient bit to 0.
11100	**010**	step 4(i-a): shift.
+00011		step 4(ii-a): add b.
11111	**0100**	step 4(iii): P is negative, so set quotient bit to 0.
+00011		Remainder is negative, so do final restore step.
00010		The quotient is 0100_2 and the remainder is 00010_2.

(b)

Figure J.3 Numerical example of (a) restoring division and (b) nonrestoring division.

The explanation for why the nonrestoring algorithm works is this. Let r_k be the contents of the (P,A) register pair at step k, ignoring the quotient bits (which are simply sharing the unused bits of register A). In Figure J.3(a), initially A contains 14, so $r_0 = 14$. At the end of the first step, $r_1 = 28$, and so on. In the restoring algorithm, part (i) computes $2r_k$ and then part (ii) $2r_k - 2^n b$ ($2^n b$ since b is subtracted from the left half). If $2r_k - 2^n b \geq 0$, both algorithms end the step with identical values in (P,A). If $2r_k - 2^n b < 0$, then the restoring algorithm restores this to $2r_k$, and the next step begins by computing $r_{\text{res}} = 2(2r_k) - 2^n b$. In the non-restoring algorithm, $2r_k - 2^n b$ is kept as a negative number, and in the next step $r_{\text{nonres}} = 2(2r_k - 2^n b) + 2^n b = 4r_k - 2^n b = r_{\text{res}}$. Thus (P,A) has the same bits in both algorithms.

If a and b are unsigned n-bit numbers, hence in the range $0 \leq a,b \leq 2^n - 1$, then the multiplier in Figure J.2 will work if register P is n bits long. However, for division, P must be extended to $n+1$ bits in order to detect the sign of P. Thus the adder must also have $n+1$ bits.

Why would anyone implement restoring division, which uses the same hardware as nonrestoring division (the control is slightly different) but involves an extra addition? In fact, the usual implementation for restoring division doesn't actually perform an add in step (*iv*). Rather, the sign resulting from the subtraction is tested at the output of the adder, and only if the sum is nonnegative is it loaded back into the P register.

As a final point, before beginning to divide, the hardware must check to see whether the divisor is 0.

Signed Numbers

There are four methods commonly used to represent signed n-bit numbers: *sign magnitude*, *two's complement*, *one's complement*, and *biased*. In the sign magnitude system, the high-order bit is the sign bit, and the low-order $n-1$ bits are the magnitude of the number. In the two's complement system, a number and its negative add up to 2^n. In one's complement, the negative of a number is obtained by complementing each bit (or, alternatively, the number and its negative add up to $2^n - 1$). In each of these three systems, nonnegative numbers are represented in the usual way. In a biased system, nonnegative numbers do not have their usual representation. Instead, all numbers are represented by first adding them to the bias and then encoding this sum as an ordinary unsigned number. Thus, a negative number k can be encoded as long as $k + \text{bias} \geq 0$. A typical value for the bias is 2^{n-1}.

Example Using 4-bit numbers ($n = 4$), if $k = 3$ (or in binary, $k = 0011_2$), how is $-k$ expressed in each of these formats?

Answer In signed magnitude, the leftmost bit in $k = \mathbf{0}011_2$ is the sign bit, so flip it to 1: $-k$ is represented by $\mathbf{1}011_2$. In two's complement, $k + 1101_2 = 2^n = 16$. So $-k$ is represented by 1101_2. In one's complement, the bits of $k = 0011_2$ are flipped, so $-k$ is represented by 1100_2. For a biased system, assuming a bias of $2^{n-1} = 8$, k is represented by $k + \text{bias} = 1011_2$, and $-k$ by $-k + \text{bias} = 0101_2$.

The most widely used system for representing integers, two's complement, is the system we will use here. One reason for the popularity of two's complement is that it makes signed addition easy: Simply discard the carry-out from the highorder bit. To add $5 + -2$, for example, add 0101_2 and 1110_2 to obtain 0011_2, resulting in the correct value of 3. A useful formula for the value of a two's complement number $a_{n-1}a_{n-2}\cdots a_1a_0$ is

J.2.3
$$-a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \cdots + a_1 2^1 + a_0$$

As an illustration of this formula, the value of 1101_2 as a 4-bit two's complement number is $-1\cdot 2^3 + 1\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = -8+4+1 = -3$, confirming the result of the example above.

Overflow occurs when the result of the operation does not fit in the representation being used. For example, if unsigned numbers are being represented using 4 bits, then $6 = 0110_2$ and $11 = 1011_2$. Their sum (17) overflows because its binary equivalent (10001_2) doesn't fit into 4 bits. For unsigned numbers, detecting overflow is easy; it occurs exactly when there is a carry-out of the most-significant bit. For two's complement, things are trickier: Overflow occurs exactly when the carry into the high-order bit is different from the (to be discarded) carry-out of the high-order bit. In the example of $5 + -2$ above, a 1 is carried both into and out of the leftmost bit, avoiding overflow.

Negating a two's complement number involves complementing each bit and then adding 1. For instance, to negate 0011_2, complement it to get 1100_2 and then add 1 to get 1101_2. Thus, to implement $a - b$ using an adder, simply feed a and $\overline{b}$ (where $\overline{b}$ is the number obtained by complementing each bit of b) into the adder and set the low-order, carry-in bit to 1. This explains why the rightmost adder in Figure J.1 is a full adder.

Multiplying two's complement numbers is not quite as simple as adding them. The obvious approach is to convert both operands to be nonnegative, do an unsigned multiplication, and then (if the original operands were of opposite signs) negate the result. Although this is conceptually simple, it requires extra time and hardware. Here is a better approach: Suppose that we are multiplying a times b using the hardware shown in Figure J.2(a). Register A is loaded with the number a; B is loaded with b. Since the content of register B is always b, we will use B and b interchangeably. If B is potentially negative but A is nonnegative, the only change needed to convert the unsigned multiplication algorithm into a two's complement one is to ensure that when P is shifted, it is shifted arithmetically; that is, the bit shifted into the high-order bit of P should be the sign bit of P (rather than the carry-out from the addition). Note that our n-bit-wide adder will now be adding n-bit two's complement numbers between -2^{n-1} and $2^{n-1} - 1$.

Next, suppose a is negative. The method for handling this case is called *Booth recoding*. Booth recoding is a very basic technique in computer arithmetic and will play a key role in Section J.9. The algorithm on page J-4 computes $a \times b$ by examining the bits of a from least significant to most significant. For example, if $a = 7 = 0111_2$, then step (i) will successively add B, add B, add B, and add 0. Booth recoding "recodes" the number 7 as $8 - 1 = 1000_2 - 0001_2 = 100\overline{1}$, where $\overline{1}$

represents -1. This gives an alternative way to compute $a \times b$, namely, successively subtract B, add 0, add 0, and add B. This is more complicated than the unsigned algorithm on page J-4, since it uses both addition and subtraction. The advantage shows up for negative values of a. With the proper recoding, we can treat a as though it were unsigned. For example, take $a = -4 = 1100_2$. Think of 1100_2 as the unsigned number 12, and recode it as $12 = 16 - 4 = 10000_2 - 0100_2 = 10\overline{1}00$. If the multiplication algorithm is only iterated n times ($n = 4$ in this case), the high-order digit is ignored, and we end up subtracting $0100_2 = 4$ times the multiplier—exactly the right answer. This suggests that multiplying using a recoded form of a will work equally well for both positive and negative numbers. And, indeed, to deal with negative values of a, all that is required is to sometimes subtract b from P, instead of adding either b or 0 to P. Here are the precise rules: If the initial content of A is $a_{n-1} \cdots a_0$, then at the ith multiply step the low-order bit of register A is a_i, and step (i) in the multiplication algorithm becomes:

I. If $a_i = 0$ and $a_{i-1} = 0$, then add 0 to P.

II. If $a_i = 0$ and $a_{i-1} = 1$, then add B to P.

III. If $a_i = 1$ and $a_{i-1} = 0$, then subtract B from P.

IV. If $a_i = 1$ and $a_{i-1} = 1$, then add 0 to P.

For the first step, when $i = 0$, take a_{i-1} to be 0.

Example When multiplying -6 times -5, what is the sequence of values in the (P,A) register pair?

Answer See Figure J.4.

P	A	
0000	1010	Put $-6 = 1010_2$ into A, $-5 = 1011_2$ into B.
0000	1010	step 1(i): $a_0 = a_{-1} = 0$, so from rule I add 0.
0000	0101	step 1(ii): shift.
+0101		step 2(i): $a_1 = 1$, $a_0 = 0$.Rule III says subtract b (or add $-b = -1011_2 = 0101_2$).
0101	0101	
0010	1010	step 2(ii): shift.
+ 1011		step 3(i): $a_2 = 0$, $a_1 = 1$. Rule II says add b (1011).
1101	1010	
1110	1101	step 3(ii): shift. (Arithmetic shift—load 1 into leftmost bit.)
+ 0101		step 4(i): $a_3 = 1$, $a_2 = 0$. Rule III says subtract b.
0011	1101	
0001	1110	step 4(ii): shift. Final result is $00011110_2 = 30$.

Figure J.4 Numerical example of Booth recoding. Multiplication of $a = -6$ by $b = -5$ to get 30.

The four prior cases can be restated as saying that in the *i*th step you should add $(a_{i-1} - a_i)$B to P. With this observation, it is easy to verify that these rules work, because the result of all the additions is

$$\sum_{i=0}^{n-1} b(a_{i-1} - a_i)2^i = b\left(-a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \ldots + a_1 2 + a_0\right) + ba_{-1}$$

Using Equation J.2.3 (page J-8) together with $a_{-1} = 0$, the right-hand side is seen to be the value of $b \times a$ as a two's complement number.

The simplest way to implement the rules for Booth recoding is to extend the A register one bit to the right so that this new bit will contain a_{i-1}. Unlike the naive method of inverting any negative operands, this technique doesn't require extra steps or any special casing for negative operands. It has only slightly more control logic. If the multiplier is being shared with a divider, there will already be the capability for subtracting b, rather than adding it. To summarize, a simple method for handling two's complement multiplication is to pay attention to the sign of P when shifting it right, and to save the most recently shifted-out bit of A to use in deciding whether to add or subtract b from P.

Booth recoding is usually the best method for designing multiplication hardware that operates on signed numbers. For hardware that doesn't directly implement it, however, performing Booth recoding in software or microcode is usually too slow because of the conditional tests and branches. If the hardware supports arithmetic shifts (so that negative b is handled correctly), then the following method can be used. Treat the multiplier a as if it were an unsigned number, and perform the first $n-1$ multiply steps using the algorithm on page J-4. If $a < 0$ (in which case there will be a 1 in the low-order bit of the A register at this point), then subtract b from P; otherwise ($a \geq 0$), neither add nor subtract. In either case, do a final shift (for a total of n shifts). This works because it amounts to multiplying b by $-a_{n-1}2^{n-1} + \cdots + a_1 2 + a_0$, which is the value of $a_{n-1} \cdots a_0$ as a two's complement number by Equation J.2.3. If the hardware doesn't support arithmetic shift, then converting the operands to be nonnegative is probably the best approach.

Two final remarks: A good way to test a signed-multiply routine is to try $-2^{n-1} \times -2^{n-1}$, since this is the only case that produces a $2n-1$ bit result. Unlike multiplication, division is usually performed in hardware by converting the operands to be nonnegative and then doing an unsigned divide. Because division is substantially slower (and less frequent) than multiplication, the extra time used to manipulate the signs has less impact than it does on multiplication.

Systems Issues

When designing an instruction set, a number of issues related to integer arithmetic need to be resolved. Several of them are discussed here.

First, what should be done about integer overflow? This situation is complicated by the fact that detecting overflow differs depending on whether the operands are signed or unsigned integers. Consider signed arithmetic first. There are three

approaches: Set a bit on overflow, trap on overflow, or do nothing on overflow. In the last case, software has to check whether or not an overflow occurred. The most convenient solution for the programmer is to have an enable bit. If this bit is turned on, then overflow causes a trap. If it is turned off, then overflow sets a bit (or, alternatively, have two different add instructions). The advantage of this approach is that both trapping and nontrapping operations require only one instruction. Furthermore, as we will see in Section J.7, this is analogous to how the IEEE floating-point standard handles floating-point overflow. Figure J.5 shows how some common machines treat overflow.

What about unsigned addition? Notice that none of the architectures in Figure J.5 traps on unsigned overflow. The reason for this is that the primary use of unsigned arithmetic is in manipulating addresses. It is convenient to be able to subtract from an unsigned address by adding. For example, when $n = 4$, we can subtract 2 from the unsigned address $10 = 1010_2$ by adding $14 = 1110_2$. This generates an overflow, but we would not want a trap to be generated.

A second issue concerns multiplication. Should the result of multiplying two n-bit numbers be a $2n$-bit result, or should multiplication just return the low-order n bits, signaling overflow if the result doesn't fit in n bits? An argument in favor of an n-bit result is that in virtually all high-level languages, multiplication is an operation in which arguments are integer variables and the result is an integer variable of the same type. Therefore, compilers won't generate code that utilizes a double-precision result. An argument in favor of a $2n$-bit result is that it can be used by an assembly language routine to substantially speed up multiplication of multiple-precision integers (by about a factor of 3).

A third issue concerns machines that want to execute one instruction every cycle. It is rarely practical to perform a multiplication or division in the same amount of time that an addition or register-register move takes. There are three possible approaches to this problem. The first is to have a single-cycle *multiply-step* instruction. This might do one step of the Booth algorithm. The second approach is to do integer multiplication in the floating-point unit and have it be part of the floating-point instruction set.

Machine	Trap on signed overflow?	Trap on unsigned overflow?	Set bit on signed overflow?	Set bit on unsigned overflow?
VAX	If enable is on	No	Yes. `Add` sets V bit.	Yes. `Add` sets C bit.
IBM 370	If enable is on	No	Yes. `Add` sets cond code.	Yes. Logical `add` sets cond code.
Intel 8086	No	No	Yes. `Add` sets V bit.	Yes. `Add` sets C bit.
MIPS R3000	Two `add` instructions; one always traps, the other never does.	No	No. Software must deduce it from sign of operands and result.	
SPARC	No	No	`Addcc` sets V bit. `Add` does not.	`Addcc` sets C bit. `Add` does not.

Figure J.5 Summary of how various machines handle integer overflow. Both the 8086 and SPARC have an instruction that traps if the V bit is set, so the cost of trapping on overflow is one extra instruction.

(This is what DLX does.) The third approach is to have an autonomous unit in the CPU do the multiplication. In this case, the result either can be guaranteed to be delivered in a fixed number of cycles—and the compiler charged with waiting the proper amount of time—or there can be an interlock. The same comments apply to division as well. As examples, the original SPARC had a multiply-step instruction but no divide-step instruction, while the MIPS R3000 has an autonomous unit that does multiplication and division (newer versions of the SPARC architecture added an integer multiply instruction). The designers of the HP Precision Architecture did an especially thorough job of analyzing the frequency of the operands for multiplication and division, and they based their multiply and divide steps accordingly. (See Magenheimer et al. [1988] for details.)

The final issue involves the computation of integer division and remainder for negative numbers. For example, what is -5 DIV 3 and -5 MOD 3? When computing x DIV y and x MOD y, negative values of x occur frequently enough to be worth some careful consideration. (On the other hand, negative values of y are quite rare.) If there are built-in hardware instructions for these operations, they should correspond to what high-level languages specify. Unfortunately, there is no agreement among existing programming languages. See Figure J.6.

One definition for these expressions stands out as clearly superior, namely, x DIV $y = \lfloor x/y \rfloor$, so that 5 DIV $3 = 1$ and -5 DIV $3 = -2$. And MOD should satisfy $x = (x \text{ DIV } y) \times y + x \text{ MOD } y$, so that x MOD $y \geq 0$. Thus, 5 MOD $3 = 2$, and -5 MOD $3 = 1$. Some of the many advantages of this definition are as follows:

1. A calculation to compute an index into a hash table of size N can use MOD N and be guaranteed to produce a valid index in the range from 0 to $N-1$.
2. In graphics, when converting from one coordinate system to another, there is no "glitch" near 0. For example, to convert from a value x expressed in a system that uses 100 dots per inch to a value y on a bitmapped display with 70 dots per inch, the formula $y = (70 \times x)$ DIV 100 maps one or two x coordinates into each y coordinate. But if DIV were defined as in Pascal to be x/y rounded to 0, then 0 would have three different points $(-1, 0, 1)$ mapped into it.
3. x MOD 2^k is the same as performing a bitwise AND with a mask of k bits, and x DIV 2^k is the same as doing a k-bit arithmetic right shift.

Language	Division	Remainder
FORTRAN	$-5/3 = -1$	MOD$(-5, 3) = -2$
Pascal	-5 DIV $3 = -1$	-5 MOD $3 = 1$
Ada	$-5/3 = -1$	-5 MOD $3 = 1$ -5 REM $3 = -2$
C	$-5/3$ undefined	-5% 3 undefined
Modula-3	-5 DIV $3 = -2$	-5 MOD $3 = 1$

Figure J.6 Examples of integer division and integer remainder in various programming languages.

Finally, a potential pitfall worth mentioning concerns multiple-precision addition. Many instruction sets offer a variant of the `add` instruction that adds three operands: two n-bit numbers together with a third single-bit number. This third number is the carry from the previous addition. Since the multiple-precision number will typically be stored in an array, it is important to be able to increment the array pointer without destroying the carry bit.

J.3 Floating Point

Many applications require numbers that aren't integers. There are a number of ways that nonintegers can be represented. One is to use *fixed point*; that is, use integer arithmetic and simply imagine the binary point somewhere other than just to the right of the least-significant digit. Adding two such numbers can be done with an integer `add`, whereas multiplication requires some extra shifting. Other representations that have been proposed involve storing the logarithm of a number and doing multiplication by adding the logarithms, or using a pair of integers (a,b) to represent the fraction a/b. However, only one noninteger representation has gained widespread use, and that is *floating point*. In this system, a computer word is divided into two parts, an exponent and a significand. As an example, an exponent of -3 and a significand of 1.5 might represent the number $1.5 \times 2^{-3} = 0.1875$. The advantages of standardizing a particular representation are obvious. Numerical analysts can build up high-quality software libraries, computer designers can develop techniques for implementing high-performance hardware, and hardware vendors can build standard accelerators. Given the predominance of the floating-point representation, it appears unlikely that any other representation will come into widespread use.

The semantics of floating-point instructions are not as clear-cut as the semantics of the rest of the instruction set, and in the past the behavior of floating-point operations varied considerably from one computer family to the next. The variations involved such things as the number of bits allocated to the exponent and significand, the range of exponents, how rounding was carried out, and the actions taken on exceptional conditions like underflow and overflow. Computer architecture books used to dispense advice on how to deal with all these details, but fortunately this is no longer necessary. That's because the computer industry is rapidly converging on the format specified by IEEE standard 754-1985 (also an international standard, IEC 559). The advantages of using a standard variant of floating point are similar to those for using floating point over other noninteger representations.

IEEE arithmetic differs from many previous arithmetics in the following major ways:

1. When rounding a "halfway" result to the nearest floating-point number, it picks the one that is even.
2. It includes the *special values* NaN, ∞, and $-\infty$.

3. It uses *denormal* numbers to represent the result of computations whose value is less than $1.0 \times 2^{E_{\min}}$.
4. It rounds to nearest by default, but it also has three other rounding modes.
5. It has sophisticated facilities for handling exceptions.

To elaborate on (1), note that when operating on two floating-point numbers, the result is usually a number that cannot be exactly represented as another floating-point number. For example, in a floating-point system using base 10 and two significant digits, $6.1 \times 0.5 = 3.05$. This needs to be rounded to two digits. Should it be rounded to 3.0 or 3.1? In the IEEE standard, such halfway cases are rounded to the number whose low-order digit is even. That is, 3.05 rounds to 3.0, not 3.1. The standard actually has four *rounding modes*. The default is *round to nearest*, which rounds ties to an even number as just explained. The other modes are round toward 0, round toward $+\infty$, and round toward $-\infty$.

We will elaborate on the other differences in following sections. For further reading, see IEEE [1985], Cody et al. [1984], and Goldberg [1991].

Special Values and Denormals

Probably the most notable feature of the standard is that by default a computation continues in the face of exceptional conditions, such as dividing by 0 or taking the square root of a negative number. For example, the result of taking the square root of a negative number is a *NaN* (*N*ot *a N*umber), a bit pattern that does not represent an ordinary number. As an example of how NaNs might be useful, consider the code for a zero finder that takes a function F as an argument and evaluates F at various points to determine a zero for it. If the zero finder accidentally probes outside the valid values for F, then F may well cause an exception. Writing a zero finder that deals with this case is highly language and operating-system dependent, because it relies on how the operating system reacts to exceptions and how this reaction is mapped back into the programming language. In IEEE arithmetic it is easy to write a zero finder that handles this situation and runs on many different systems. After each evaluation of F, it simply checks to see whether F has returned a NaN; if so, it knows it has probed outside the domain of F.

In IEEE arithmetic, if the input to an operation is a NaN, the output is NaN (e.g., $3 + \text{NaN} = \text{NaN}$). Because of this rule, writing floating-point subroutines that can accept NaN as an argument rarely requires any special case checks. For example, suppose that arccos is computed in terms of arctan, using the formula $\arccos x = 2 \arctan\left(\sqrt{(1-x)/(1+x)}\right)$. If arctan handles an argument of NaN properly, arccos will automatically do so, too. That's because if x is a NaN, $1 + x$, $1 - x$, $(1 + x)/(1 - x)$, and $\sqrt{(1-x)/(1+x)}$ will also be NaNs. No checking for NaNs is required.

While the result of $\sqrt{-1}$ is a NaN, the result of 1/0 is not a NaN, but $+\infty$, which is another special value. The standard defines arithmetic on infinities (there are

both $+\infty$ and $-\infty$) using rules such as $1/\infty = 0$. The formula $\arccos x = 2\arctan\left(\sqrt{(1-x)/(1+x)}\right)$ illustrates how infinity arithmetic can be used. Since arctan x asymptotically approaches $\pi/2$ as x approaches ∞, it is natural to define $\arctan(\infty) = \pi/2$, in which case $\arccos(-1)$ will automatically be computed correctly as $2\arctan(\infty) = \pi$.

The final kind of special values in the standard are *denormal* numbers. In many floating-point systems, if $E_{\min}$ is the smallest exponent, a number less than $1.0 \times 2^{E_{\min}}$ cannot be represented, and a floating-point operation that results in a number less than this is simply flushed to 0. In the IEEE standard, on the other hand, numbers less than $1.0 \times 2^{E_{\min}}$ are represented using significands less than 1. This is called *gradual underflow*. Thus, as numbers decrease in magnitude below $2^{E_{\min}}$, they gradually lose their significance and are only represented by 0 when all their significance has been shifted out. For example, in base 10 with four significant figures, let $x = 1.234 \times 10^{E_{\min}}$. Then, $x/10$ will be rounded to $0.123 \times 10^{E_{\min}}$, having lost a digit of precision. Similarly $x/100$ rounds to $0.012 \times 10^{E_{\min}}$, and $x/1000$ to $0.001 \times 10^{E_{\min}}$, while $x/10000$ is finally small enough to be rounded to 0. Denormals make dealing with small numbers more predictable by maintaining familiar properties such as $x = y \Leftrightarrow x - y = 0$. For example, in a flush-to-zero system (again in base 10 with four significant digits), if $x = 1.256 \times 10^{E_{\min}}$ and $y = 1.234 \times 10^{E_{\min}}$, then $x - y = 0.022 \times 10^{E_{\min}}$, which flushes to zero. So even though $x \neq y$, the computed value of $x - y = 0$. This never happens with gradual underflow. In this example, $x - y = 0.022 \times 10^{E_{\min}}$ is a denormal number, and so the computation of $x - y$ is exact.

Representation of Floating-Point Numbers

Let us consider how to represent single-precision numbers in IEEE arithmetic. Single-precision numbers are stored in 32 bits: 1 for the sign, 8 for the exponent, and 23 for the fraction. The exponent is a signed number represented using the bias method (see the subsection "Signed Numbers," page J-7) with a bias of 127. The term *biased exponent* refers to the unsigned number contained in bits 1 through 8, and *unbiased exponent* (or just exponent) means the actual power to which 2 is to be raised. The fraction represents a number less than 1, but the *significand* of the floating-point number is 1 plus the fraction part. In other words, if e is the biased exponent (value of the exponent field) and f is the value of the fraction field, the number being represented is $1.f \times 2^{e-127}$.

Example What single-precision number does the following 32-bit word represent?

1 10000001 01000000000000000000000

Answer Considered as an unsigned number, the exponent field is 129, making the value of the exponent $129 - 127 = 2$. The fraction part is $.01_2 = .25$, making the significand 1.25. Thus, this bit pattern represents the number $-1.25 \times 2^2 = -5$.

The fractional part of a floating-point number (.25 in the example above) must not be confused with the significand, which is 1 plus the fractional part. The leading 1 in the significand $1.f$ does not appear in the representation; that is, the leading bit is implicit. When performing arithmetic on IEEE format numbers, the fraction part is usually *unpacked*, which is to say the implicit 1 is made explicit.

Figure J.7 summarizes the parameters for single (and other) precisions. It shows the exponents for single precision to range from −126 to 127; accordingly, the biased exponents range from 1 to 254. The biased exponents of 0 and 255 are used to represent special values. This is summarized in Figure J.8. When the biased exponent is 255, a zero fraction field represents infinity, and a nonzero fraction field represents a NaN. Thus, there is an entire family of NaNs. When the biased exponent and the fraction field are 0, then the number represented is 0. Because of the implicit leading 1, ordinary numbers always have a significand greater than or equal to 1. Thus, a special convention such as this is required to represent 0. Denormalized numbers are implemented by having a word with a zero exponent field represent the number $0.f \times 2^{E_{\min}}$.

The primary reason why the IEEE standard, like most other floating-point formats, uses biased exponents is that it means nonnegative numbers are ordered in the same way as integers. That is, the magnitude of floating-point numbers can be compared using an integer comparator. Another (related) advantage is that 0 is represented by a word of all 0s. The downside of biased exponents is that adding them is slightly awkward, because it requires that the bias be subtracted from their sum.

	Single	Single extended	Double	Double extended
p (bits of precision)	24	≥32	53	≥64
$E_{\max}$	127	≥1023	1023	≥16383
$E_{\min}$	−126	≤ −1022	−1022	≤ −16382
Exponent bias	127		1023	

Figure J.7 Format parameters for the IEEE 754 floating-point standard. The first row gives the number of bits in the significand. The blanks are unspecified parameters.

Exponent	Fraction	Represents
$e = E_{\min} - 1$	$f = 0$	±0
$e = E_{\min} - 1$	$f \neq 0$	$0.f \times 2^{E_{\min}}$
$E_{\min} \leq e \leq E_{\max}$	—	$1.f \times 2^{e}$
$e = E_{\max} + 1$	$f = 0$	±∞
$e = E_{\max} + 1$	$f \neq 0$	NaN

Figure J.8 Representation of special values. When the exponent of a number falls outside the range $E_{\min} \leq e \leq E_{\max}$, then that number has a special interpretation as indicated in the table.

J.4 Floating-Point Multiplication

The simplest floating-point operation is multiplication, so we discuss it first. A binary floating-point number x is represented as a significand and an exponent, $x = s \times 2^e$. The formula

$$(s_1 \times 2^{e1}) \bullet (s_2 \times 2^{e2}) = (s_1 \bullet s_2) \times 2^{e1+e2}$$

shows that a floating-point multiply algorithm has several parts. The first part multiplies the significands using ordinary integer multiplication. Because floating-point numbers are stored in sign magnitude form, the multiplier need only deal with unsigned numbers (although we have seen that Booth recoding handles signed two's complement numbers painlessly). The second part rounds the result. If the significands are unsigned p-bit numbers (e.g., $p = 24$ for single precision), then the product can have as many as $2p$ bits and must be rounded to a p-bit number. The third part computes the new exponent. Because exponents are stored with a bias, this involves subtracting the bias from the sum of the biased exponents.

Example How does the multiplication of the single-precision numbers

$$1 \quad 1000001 \quad 0000\ldots \quad = \quad -1 \times 2^3$$
$$0 \quad 1000001 \quad 1000\ldots \quad = \quad 1 \times 2^4$$

proceed in binary?

Answer When unpacked, the significands are both 1.0, their product is 1.0, and so the result is of the form:

$$1 \quad ???????? \quad 000\ldots$$

To compute the exponent, use the formula:

$$\text{biased exp}(e_1 + e_2) = \text{biased exp}(e_1) + \text{biased exp}(e_2) - \text{bias}$$

From Figure J.7, the bias is $127 = 01111111_2$, so in two's complement -127 is 10000001_2. Thus, the biased exponent of the product is

$$\begin{array}{r} 10000010 \\ 10000011 \\ +\ 10000001 \\ \hline 10000110 \end{array}$$

Since this is 134 decimal, it represents an exponent of $134 - \text{bias} = 134 - 127$, as expected.

The interesting part of floating-point multiplication is rounding. Some of the different cases that can occur are illustrated in Figure J.9. Since the cases are similar in all bases, the figure uses human-friendly base 10, rather than base 2.

(a)	1.23	
	× 6.78	$r = 9 > 5$ so round up
	8.3394	rounds to 8.34
	↑	
(b)	2.83	
	× 4.47	$r = 5$ and a following digit $\neq 0$ so round up
	12.6501	rounds to 1.27×10^1
	↑	
(c)	1.28	
	× 7.81	$r = 6 > 5$ so round up
	0**9.99**68	rounds to 1.00×10^1
	↑	

Figure J.9 Examples of rounding a multiplication. Using base 10 and $p = 3$, parts (a) and (b) illustrate that the result of a multiplication can have either $2p - 1$ or $2p$ digits; hence, the position where a 1 is added when rounding up (just left of the arrow) can vary. Part (c) shows that rounding up can cause a carry-out.

In the figure, $p = 3$, so the final result must be rounded to three significant digits. The three most-significant digits are in boldface. The fourth most-significant digit (marked with an arrow) is the *round* digit, denoted by r.

If the round digit is less than 5, then the bold digits represent the rounded result. If the round digit is greater than 5 (as in part (a)), then 1 must be added to the least-significant bold digit. If the round digit is exactly 5 (as in part (b)), then additional digits must be examined to decide between truncation or incrementing by 1. It is only necessary to know if any digits past 5 are nonzero. In the algorithm below, this will be recorded in a *sticky bit*. Comparing parts (a) and (b) in the figure shows that there are two possible positions for the round digit (relative to the least-significant digit of the product). Case (c) illustrates that, when adding 1 to the least-significant bold digit, there may be a carry-out. When this happens, the final significand must be 10.0.

There is a straightforward method of handling rounding using the multiplier of Figure J.2 (page J-4) together with an extra sticky bit. If p is the number of bits in the significand, then the A, B, and P registers should be p bits wide. Multiply the two significands to obtain a $2p$-bit product in the (P,A) registers (see Figure J.10). During the multiplication, the first $p - 2$ times a bit is shifted into the A register, OR it into the sticky bit. This will be used in halfway cases. Let s represent the sticky bit, g (for guard) the most-significant bit of A, and r (for round) the second most-significant bit of A. There are two cases:

1. The high-order bit of P is 0. Shift P left 1 bit, shifting in the g bit from A. Shifting the rest of A is not necessary.
2. The high-order bit of P is 1. Set $s := s \vee r$ and $r := g$, and add 1 to the exponent.

Now if $r = 0$, P is the correctly rounded product. If $r = 1$ and $s = 1$, then P + 1 is the product (where by P + 1 we mean adding 1 to the least-significant bit of P).

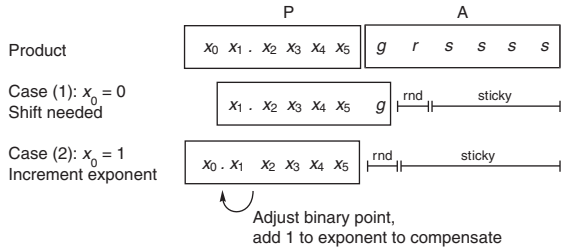


Figure J.10 The two cases of the floating-point multiply algorithm. The top line shows the contents of the P and A registers after multiplying the significands, with $p=6$. In case (1), the leading bit is 0, and so the P register must be shifted. In case (2), the leading bit is 1, no shift is required, but both the exponent and the round and sticky bits must be adjusted. The sticky bit is the logical OR of the bits marked *s*.

If $r=1$ and $s=0$, we are in a halfway case and round up according to the least-significant bit of P. As an example, apply the decimal version of these rules to Figure J.9(b). After the multiplication, P = 126 and A = 501, with $g=5$, $r=0$ and $s=1$. Since the high-order digit of P is nonzero, case (2) applies and $r := g$, so that $r=5$, as the arrow indicates in Figure J.9. Since $r=5$, we could be in a halfway case, but $s=1$ indicates that the result is in fact slightly over 1/2, so add 1 to P to obtain the correctly rounded product.

The precise rules for rounding depend on the rounding mode and are given in Figure J.11. Note that P is nonnegative, that is, it contains the magnitude of the result. A good discussion of more efficient ways to implement rounding is in Santoro, Bewick, and Horowitz [1989].

Example In binary with $p=4$, show how the multiplication algorithm computes the product -5×10 in each of the four rounding modes.

Answer In binary, -5 is $-1.010_2 \times 2^2$ and $10 = 1.010_2 \times 2^3$. Applying the integer multiplication algorithm to the significands gives 01100100_2, so P = 0110_2, A = 0100_2, $g=0$, $r=1$, and $s=0$. The high-order bit of P is 0, so case (1) applies. Thus, P becomes 1100_2, and since the result is negative, Figure J.11 gives:

round to $-\infty$	1101_2	add 1 since $r \vee s = 1$ / 0 = TRUE	
round to $+\infty$	1100_2		
round to 0	1100_2		
round to nearest	1100_2	no add since	$r \wedge p_0 = 1 \wedge 0$ = FALSE and $r \wedge s = 1 \wedge 0$ = FALSE

The exponent is $2+3=5$, so the result is $-1.100_2 \times 2^5 = -48$, except when rounding to $-\infty$, in which case it is $-1.101_2 \times 2^5 = -52$.

Rounding mode	Sign of result ≥ 0	Sign of result < 0
$-\infty$		+1 if $r \vee s$
$+\infty$	+1 if $r \vee s$	
0		
Nearest	+1 if $r \wedge p_0$ or $r \wedge s$	+1 if $r \wedge p_0$ or $r \wedge s$

Figure J.11 Rules for implementing the IEEE rounding modes. Let S be the magnitude of the preliminary result. Blanks mean that the p most-significant bits of S are the actual result bits. If the condition listed is true, add 1 to the pth most-significant bit of S. The symbols r and s represent the round and sticky bits, while p_0 is the pth most-significant bit of S.

Overflow occurs when the rounded result is too large to be represented. In single precision, this occurs when the result has an exponent of 128 or higher. If e_1 and e_2 are the two biased exponents, then $1 \leq e_i \leq 254$, and the exponent calculation $e_1 + e_2 - 127$ gives numbers between $1 + 1 - 127$ and $254 + 254 - 127$, or between -125 and 381. This range of numbers can be represented using 9 bits. So one way to detect overflow is to perform the exponent calculations in a 9-bit adder (see Exercise J.12). Remember that you must check for overflow *after* rounding—the example in Figure J.9(c) shows that this can make a difference.

Denormals

Checking for underflow is somewhat more complex because of denormals. In single precision, if the result has an exponent less than -126, that does not necessarily indicate underflow, because the result might be a denormal number. For example, the product of (1×2^{-64}) with (1×2^{-65}) is 1×2^{-129}, and -129 is below the legal exponent limit. But this result is a valid denormal number, namely, 0.125×2^{-126}. In general, when the unbiased exponent of a product dips below -126, the resulting product must be shifted right and the exponent incremented until the exponent reaches -126. If this process causes the entire significand to be shifted out, then underflow has occurred. The precise definition of underflow is somewhat subtle—see Section J.7 for details.

When one of the operands of a multiplication is denormal, its significand will have leading zeros, and so the product of the significands will also have leading zeros. If the exponent of the product is less than -126, then the result is denormal, so right-shift and increment the exponent as before. If the exponent is greater than -126, the result may be a normalized number. In this case, *left*-shift the product (while decrementing the exponent) until either it becomes normalized or the exponent drops to -126.

Denormal numbers present a major stumbling block to implementing floating-point multiplication, because they require performing a variable shift in the multiplier, which wouldn't otherwise be needed. Thus, high-performance, floating-point multipliers often do not handle denormalized

numbers, but instead trap, letting software handle them. A few practical codes frequently underflow, even when working properly, and these programs will run quite a bit slower on systems that require denormals to be processed by a trap handler.

So far we haven't mentioned how to deal with operands of zero. This can be handled by either testing both operands before beginning the multiplication or testing the product afterward. If you test afterward, be sure to handle the case $0 \times \infty$ properly: This results in NaN, not 0. Once you detect that the result is 0, set the biased exponent to 0. Don't forget about the sign. The sign of a product is the XOR of the signs of the operands, even when the result is 0.

Precision of Multiplication

In the discussion of integer multiplication, we mentioned that designers must decide whether to deliver the low-order word of the product or the entire product. A similar issue arises in floating-point multiplication, where the exact product can be rounded to the precision of the operands or to the next higher precision. In the case of integer multiplication, none of the standard high-level languages contains a construct that would generate a "single times single gets double" instruction. The situation is different for floating point. Many languages allow assigning the product of two single-precision variables to a double-precision one, and the construction can also be exploited by numerical algorithms. The best-known case is using iterative refinement to solve linear systems of equations.

J.5 Floating-Point Addition

Typically, a floating-point operation takes two inputs with p bits of precision and returns a p-bit result. The ideal algorithm would compute this by first performing the operation exactly, and then rounding the result to p bits (using the current rounding mode). The multiplication algorithm presented in the previous section follows this strategy. Even though hardware implementing IEEE arithmetic must return the same result as the ideal algorithm, it doesn't need to actually perform the ideal algorithm. For addition, in fact, there are better ways to proceed. To see this, consider some examples.

First, the sum of the binary 6-bit numbers 1.10011_2 and $1.10001_2 \times 2^{-5}$: When the summands are shifted so they have the same exponent, this is

$$\begin{array}{r} 1.10011 \\ + \ .0000110001 \\ \hline \end{array}$$

Using a 6-bit adder (and discarding the low-order bits of the second addend) gives

$$\begin{array}{r} 1.10011 \\ + \quad .00001 \\ \hline + \ 1.10100 \end{array}$$

The first discarded bit is 1. This isn't enough to decide whether to round up. The rest of the discarded bits, 0001, need to be examined. Or, actually, we just need to record whether any of these bits are nonzero, storing this fact in a sticky bit just as in the multiplication algorithm. So, for adding two p-bit numbers, a p-bit adder is sufficient, as long as the first discarded bit (round) and the OR of the rest of the bits (sticky) are kept. Then Figure J.11 can be used to determine if a roundup is necessary, just as with multiplication. In the example above, sticky is 1, so a roundup is necessary. The final sum is 1.10101_2.

Here's another example:

$$\begin{array}{rr} & 1.11011 \\ + & .0101001 \\ \hline \end{array}$$

A 6-bit adder gives:

$$\begin{array}{rr} & 1.11011 \\ + & .01010 \\ \hline + & 10.00101 \end{array}$$

Because of the carry-out on the left, the round bit isn't the first discarded bit; rather, it is the low-order bit of the sum (1). The discarded bits, 01, are OR'ed together to make sticky. Because round and sticky are both 1, the high-order 6 bits of the sum, 10.0010_2, must be rounded up for the final answer of 10.0011_2.

Next, consider subtraction and the following example:

$$\begin{array}{rr} & 1.00000 \\ - & .00000101111 \\ \hline \end{array}$$

The simplest way of computing this is to convert $-.00000101111_2$ to its two's complement form, so the difference becomes a sum:

$$\begin{array}{rr} & 1.00000 \\ + & 1.11111010001 \end{array}$$

Computing this sum in a 6-bit adder gives:

$$\begin{array}{rr} & 1.00000 \\ + & 1.11111 \\ \hline & 0.11111 \end{array}$$

Because the top bits canceled, the first discarded bit (the guard bit) is needed to fill in the least-significant bit of the sum, which becomes 0.111110_2, and the second discarded bit becomes the round bit. This is analogous to case (1) in the multiplication algorithm (see page J-19). The round bit of 1 isn't enough to decide whether to round up. Instead, we need to OR all the remaining bits (0001) into a sticky bit. In this case, sticky is 1, so the final result must be rounded up to 0.111111. This example shows that if subtraction causes the most-significant bit to cancel, then one guard bit is needed. It is natural to ask whether two guard bits are needed for the case when the *two* most-significant bits cancel. The answer is no, because if x and y are so close that the top two bits of $x - y$ cancel, then $x - y$ will be exact, so guard bits aren't needed at all.

To summarize, addition is more complex than multiplication because, depending on the signs of the operands, it may actually be a subtraction. If it is an addition, there can be carry-out on the left, as in the second example. If it is subtraction, there can be cancellation, as in the third example. In each case, the position of the round bit is different. However, we don't need to compute the exact sum and then round. We can infer it from the sum of the high-order p bits together with the round and sticky bits.

The rest of this section is devoted to a detailed discussion of the floatingpoint addition algorithm. Let a_1 and a_2 be the two numbers to be added. The notations e_i and s_i are used for the exponent and significand of the addends a_i. This means that the floating-point inputs have been unpacked and that s_i has an explicit leading bit. To add a_1 and a_2, perform these eight steps:

1. If $e_1 < e_2$, swap the operands. This ensures that the difference of the exponents satisfies $d = e_1 - e_2 \geq 0$. Tentatively set the exponent of the result to e_1.
2. If the signs of a_1 and a_2 differ, replace s_2 by its two's complement.
3. Place s_2 in a p-bit register and shift it $d = e_1 - e_2$ places to the right (shifting in 1's if s_2 was complemented in the previous step). From the bits shifted out, set g to the most-significant bit, set r to the next most-significant bit, and set sticky to the OR of the rest.
4. Compute a preliminary significand $S = s_1 + s_2$ by adding s_1 to the p-bit register containing s_2. If the signs of a_1 and a_2 are different, the most-significant bit of S is 1, and there was no carry-out, then S is negative. Replace S with its two's complement. This can only happen when $d = 0$.
5. Shift S as follows. If the signs of a_1 and a_2 are the same and there was a carryout in step 4, shift S right by one, filling in the high-order position with 1 (the carry-out). Otherwise, shift it left until it is normalized. When left-shifting, on the first shift fill in the low-order position with the g bit. After that, shift in zeros. Adjust the exponent of the result accordingly.
6. Adjust r and s. If S was shifted right in step 5, set $r :=$ low-order bit of S before shifting and $s := g$ OR r OR s. If there was no shift, set $r := g$, $s := r$ OR s. If there was a single left shift, don't change r and s. If there were two or more left shifts, $r := 0$, $s := 0$. (In the last case, two or more shifts can only happen when a_1 and a_2 have opposite signs and the same exponent, in which case the computation $s_1 + s_2$ in step 4 will be exact.)
7. Round S using Figure J.11; namely, if a table entry is nonempty, add 1 to the low-order bit of S. If rounding causes carry-out, shift S right and adjust the exponent. This is the significand of the result.
8. Compute the sign of the result. If a_1 and a_2 have the same sign, this is the sign of the result. If a_1 and a_2 have different signs, then the sign of the result depends on which of a_1 or a_2 is negative, whether there was a swap in step 1, and whether S was replaced by its two's complement in step 4. See Figure J.12.

swap	compl	sign(a_1)	sign(a_2)	sign(result)
Yes		+	−	−
Yes		−	+	+
No	No	+	−	+
No	No	−	+	−
No	Yes	+	−	−
No	Yes	−	+	+

Figure J.12 Rules for computing the sign of a sum when the addends have different signs. The *swap* column refers to swapping the operands in step 1, while the *compl* column refers to performing a two's complement in step 4. Blanks are "don't care."

Example Use the algorithm to compute the sum $(-1.001_2 \times 2^{-2}) + (-1.111_2 \times 2^0)$.

Answer $s_1 = 1.001$, $e_1 = -2$, $s_2 = 1.111$, $e_2 = 0$

1. $e_1 < e_2$, so swap. $d = 2$. Tentative exp $= 0$.
2. Signs of both operands negative, don't negate s_2.
3. Shift s_2 (1.001 after swap) right by 2, giving $s_2 = .010$, $g = 0$, $r = 1$, $s = 0$.
4. $$\begin{array}{rl} & 1.111 \\ + & .010 \\ \hline & (1)0.001 \end{array} \qquad S = 0.001, \text{ with a carry} - \text{out}.$$
5. Carry-out, so shift S right, $S = 1.000$, exp $=$ exp $+ 1$, so exp $= 1$.
6. $r =$ low-order bit of sum $= 1$, $s = g \vee r \vee s = 0 \vee 1 \vee 0 = 1$.
7. r AND $s =$ TRUE, so Figure J.11 says round up, $S = S + 1$ or $S = 1.001$.
8. Both signs negative, so sign of result is negative. Final answer: $-S \times 2^{\text{exp}} = 1.001_2 \times 2^1$.

Example Use the algorithm to compute the sum $(-1.010_2) + 1.100_2$.

Answer $s_1 = 1.010$, $e_1 = 0$, $s_2 = 1.100$, $e_2 = 0$

1. No swap, $d = 0$, tentative exp $= 0$.
2. Signs differ, replace s_2 with 0.100.
3. $d = 0$, so no shift. $r = g = s = 0$.
4. $$\begin{array}{rl} & 1.010 \\ + & 0.100 \\ \hline & 1.110 \end{array} \qquad \begin{array}{l} \text{Signs are different, most-significant bit is 1, no carry-out, so} \\ \text{must two's complement sum, giving } S = 0.010. \end{array}$$

5. Shift left twice, so $S = 1.000$, $\exp = \exp - 2$, or $\exp = -2$.
6. Two left shifts, so $r = g = s = 0$.
7. No addition required for rounding.
8. Answer is $\text{sign} \times S \times 2^{\exp}$ or $\text{sign} \times 1.000 \times 2^{-2}$. Get sign from Figure J.12. Since complement but no swap and $\text{sign}(a_1)$ is $-$, the sign of the sum is +. Thus, the answer $= 1.000_2 \times 2^{-2}$.

Speeding Up Addition

Let's estimate how long it takes to perform the algorithm above. Step 2 may require an addition, step 4 requires one or two additions, and step 7 may require an addition. If it takes T time units to perform a p-bit add (where $p = 24$ for single precision, 53 for double), then it appears the algorithm will take at least 4 T time units. But that is too pessimistic. If step 4 requires two adds, then a_1 and a_2 have the same exponent and different signs, but in that case the difference is exact, so no roundup is required in step 7. Thus, only three additions will ever occur. Similarly, it appears that a variable shift may be required both in step 3 and step 5. But if $|e_1 - e_2| \leq 1$, then step 3 requires a right shift of at most one place, so only step 5 needs a variable shift. And, if $|e_1 - e_2| > 1$, then step 3 needs a variable shift, but step 5 will require a left shift of at most one place. So only a single variable shift will be performed. Still, the algorithm requires three sequential adds, which, in the case of a 53-bit double-precision significand, can be rather time consuming.

A number of techniques can speed up addition. One is to use pipelining. The "Putting It All Together" section gives examples of how some commercial chips pipeline addition. Another method (used on the Intel 860 [Kohn and Fu 1989]) is to perform two additions in parallel. We now explain how this reduces the latency from $3T$ to T.

There are three cases to consider. First, suppose that both operands have the same sign. We want to combine the addition operations from steps 4 and 7. The position of the high-order bit of the sum is not known ahead of time, because the addition in step 4 may or may not cause a carry-out. Both possibilities are accounted for by having two adders. The first adder assumes the add in step 4 will not result in a carry-out. Thus, the values of r and s can be computed before the add is actually done. If r and s indicate that a roundup is necessary, the first adder will compute $S = s_1 + s_2 + 1$, where the notation +1 means adding 1 at the position of the least-significant bit of s_1. This can be done with a regular adder by setting the low-order carry-in bit to 1. If r and s indicate no roundup, the adder computes $S = s_1 + s_2$ as usual. One extra detail: When $r = 1$, $s = 0$, you will also need to know the low-order bit of the sum, which can also be computed in advance very quickly. The second adder covers the possibility that there will be carry-out. The values of r and s and the position where the roundup 1 is added are different from above, but again they can be quickly computed in advance. It is not known whether there will be a carry-out until after the add is actually done, but that doesn't matter. By doing both adds in parallel, one adder is guaranteed to reduce the correct answer.

The next case is when a_1 and a_2 have opposite signs but the same exponent. The sum $a_1 + a_2$ is exact in this case (no roundup is necessary) but the sign isn't known until the add is completed. So don't compute the two's complement (which requires an add) in step 2, but instead compute $\overline{s}_1 + s_2 + 1$ and $s_1 + \overline{s}_2 + 1$ in parallel. The first sum has the result of simultaneously complementing s_1 and computing the sum, resulting in $s_2 - s_1$. The second sum computes $s_1 - s_2$. One of these will be nonnegative and hence the correct final answer. Once again, all the additions are done in one step using two adders operating in parallel.

The last case, when a_1 and a_2 have opposite signs and different exponents, is more complex. If $|e_1 - e_2| > 1$, the location of the leading bit of the difference is in one of two locations, so there are two cases just as in addition. When $|e_1 - e_2| = 1$, cancellation is possible and the leading bit could be almost anywhere. However, only if the leading bit of the difference is in the same position as the leading bit of s_1 could a roundup be necessary. So one adder assumes a roundup, and the other assumes no roundup. Thus, the addition of step 4 and the rounding of step 7 can be combined. However, there is still the problem of the addition in step 2!

To eliminate this addition, consider the following diagram of step 4:

$$
\begin{array}{ll}
| - - & p - - | \\
s_1 & 1.xxxxxxx \\
s_2 - & \underline{\qquad\qquad 1yyzzzzz}
\end{array}
$$

If the bits marked z are all 0, then the high-order p bits of $S = s_1 - s_2$ can be computed as $s_1 + \overline{s}_2 + 1$. If at least one of the z bits is 1, use $s_1 + \overline{s}_2$. So $s_1 - s_2$ can be computed with one addition. However, we still don't know g and r for the two's complement of s_2, which are needed for rounding in step 7.

To compute $s_1 - s_2$ and get the proper g and r bits, combine steps 2 and 4 as follows. Don't complement s_2 in step 2. Extend the adder used for computing S two bits to the right (call the extended sum S'). If the preliminary sticky bit (computed in step 3) is 1, compute $S' = s_1' + \overline{s}_2'$, where s_1' has two 0 bits tacked onto the right, and s_2' has preliminary g and r appended. If the sticky bit is 0, compute $s_1' + \overline{s}_2' + 1$. Now the two low-order bits of S' have the correct values of g and r (the sticky bit was already computed properly in step 3). Finally, this modification can be combined with the modification that combines the addition from steps 4 and 7 to provide the final result in time T, the time for one addition.

A few more details need to be considered, as discussed in Santoro, Bewick, and Horowitz [1989] and Exercise J.17. Although the Santoro paper is aimed at multiplication, much of the discussion applies to addition as well. Also relevant is Exercise J.19, which contains an alternative method for adding signed magnitude numbers.

Denormalized Numbers

Unlike multiplication, for addition very little changes in the preceding description if one of the inputs is a denormal number. There must be a test to see if the exponent field is 0. If it is, then when unpacking the significand there will not be a leading 1.

By setting the biased exponent to 1 when unpacking a denormal, the algorithm works unchanged.

To deal with denormalized outputs, step 5 must be modified slightly. Shift S until it is normalized, or until the exponent becomes $E_{\min}$ (that is, the biased exponent becomes 1). If the exponent is $E_{\min}$ and, after rounding, the high-order bit of S is 1, then the result is a normalized number and should be packed in the usual way, by omitting the 1. If, on the other hand, the high-order bit is 0, the result is denormal. When the result is unpacked, the exponent field must be set to 0. Section J.7 discusses the exact rules for detecting underflow.

Incidentally, detecting overflow is very easy. It can only happen if step 5 involves a shift right and the biased exponent at that point is bumped up to 255 in single precision (or 2047 for double precision), or if this occurs after rounding.

J.6 Division and Remainder

In this section, we'll discuss floating-point division and remainder.

Iterative Division

We earlier discussed an algorithm for integer division. Converting it into a floating-point division algorithm is similar to converting the integer multiplication algorithm into floating point. The formula

$$(s_1 \times 2^{e_1})/(s_2 \times 2^{e_2}) = (s_1/s_2) \times 2^{e_1-e_2}$$

shows that if the divider computes s_1/s_2, then the final answer will be this quotient multiplied by $2^{e_1-e_2}$. Referring to Figure J.2(b) (page J-4), the alignment of operands is slightly different from integer division. Load s_2 into B and s_1 into P. The A register is not needed to hold the operands. Then the integer algorithm for division (with the one small change of skipping the very first left shift) can be used, and the result will be of the form $q_0.q_1\cdots$. To round, simply compute two additional quotient bits (guard and round) and use the remainder as the sticky bit. The guard digit is necessary because the first quotient bit might be 0. However, since the numerator and denominator are both normalized, it is not possible for the two most-significant quotient bits to be 0. This algorithm produces one quotient bit in each step.

A different approach to division converges to the quotient at a quadratic rather than a linear rate. An actual machine that uses this algorithm will be discussed in Section J.10. First, we will describe the two main iterative algorithms, and then we will discuss the pros and cons of iteration when compared with the direct algorithms. A general technique for constructing iterative algorithms, called *Newton's iteration*, is shown in Figure J.13. First, cast the problem in the form of finding the zero of a function. Then, starting from a guess for the zero, approximate the function by its tangent at that guess and form a new guess based

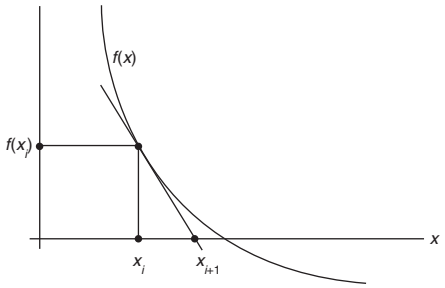


Figure J.13 Newton's iteration for zero finding. If x_i is an estimate for a zero of f, then x_{i+1} is a better estimate. To compute x_{i+1}, find the intersection of the x-axis with the tangent line to f at $f(x_i)$.

on where the tangent has a zero. If x_i is a guess at a zero, then the tangent line has the equation:

$$y - f(x_i) = f'(x_i)(x - x_i)$$

This equation has a zero at

J.6.1
$$x = x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

To recast division as finding the zero of a function, consider $f(x) = x^{-1} - b$. Since the zero of this function is at $1/b$, applying Newton's iteration to it will give an iterative method of computing $1/b$ from b. Using $f'(x) = -1/x^2$, Equation J.6.1 becomes:

J.6.2
$$x_{i+1} = x_i - \frac{1/x_i - b}{-1/x_i^2} = x_i + x_i - x_i^2 b = x_i(2 - x_i b)$$

Thus, we could implement computation of a/b using the following method:

1. Scale b to lie in the range $1 \le b < 2$ and get an approximate value of $1/b$ (call it x_0) using a table lookup.
2. Iterate $x_{i+1} = x_i(2 - x_i b)$ until reaching an x_n that is accurate enough.
3. Compute ax_n and reverse the scaling done in step 1.

Here are some more details. How many times will step 2 have to be iterated? To say that x_i is accurate to p bits means that $|(x_i - 1/b)/(1/b)| = 2^{-p}$, and a simple algebraic manipulation shows that when this is so, then $(x_{i+1} - 1/b)/(1/b) = 2^{-2p}$. Thus, the number of correct bits doubles at each step. Newton's iteration is *self-correcting* in the sense that making an error in x_i doesn't really matter. That is, it treats x_i as a guess at $1/b$ and returns x_{i+1} as an improvement on it (roughly doubling the digits). One thing that would cause x_i to be in error is rounding error. More

importantly, however, in the early iterations we can take advantage of the fact that we don't expect many correct bits by performing the multiplication in reduced precision, thus gaining speed without sacrificing accuracy. Another application of Newton's iteration is discussed in Exercise J.20.

The second iterative division method is sometimes called *Goldschmidt's algorithm*. It is based on the idea that to compute a/b, you should multiply the numerator and denominator by a number r with $rb \approx 1$. In more detail, let $x_0 = a$ and $y_0 \text{þ} = b$. At each step compute $x_{i+1} = r_i x_i$ and $y_{i+1} = r_i y_i$. Then the quotient $x_{i+1}/y_{i+1} = x_i/y_i = a/b$ is constant. If we pick r_i so that $y_i \to 1$, then $x_i \to a/b$, so the x_i converge to the answer we want. This same idea can be used to compute other functions. For example, to compute the square root of a, let $x_0 = a$ and $y_0 = a$, and at each step compute $x_{i+1} = r_i^2 x_i$, $y_{i+1} = r_i y_i$. Then $x_{i+1}/y_{i+1}^2 = x_i/y_i^2 = 1/a$, so if the r_i are chosen to drive $x_i \to 1$, then $y_i \to \sqrt{a}$. This technique is used to compute square roots on the TI 8847.

Returning to Goldschmidt's division algorithm, set $x_0 = a$ and $y_0 = b$, and write $b = 1 - \delta$, where $|\delta| < 1$. If we pick $r_0 = 1 + \delta$, then $y_1 = r_0 y_0 = 1 - \delta^2$. We next pick $r_1 = 1 + \delta^2$, so that $y_2 = r_1 y_1 = 1 - \delta^4$, and so on. Since $|\delta| < 1$, $y_i \to 1$. With this choice of r_i, the x_i will be computed as $x_{i+1} = r_i x_i = \left(1 + \delta^{2^i}\right) x_i = \left(1 + (1-b)^{2^i}\right) x_i$, or

J.6.3
$$x_{i+1} = a[1 + (1-b)]\left[1 + (1-b)^2\right]\left[1 + (1-b)^4\right] \cdots \left[1 + (1-b)^{2^i}\right]$$

There appear to be two problems with this algorithm. First, convergence is slow when b is not near 1 (that is, δ is not near 0), and, second, the formula isn't self-correcting—since the quotient is being computed as a product of independent terms, an error in one of them won't get corrected. To deal with slow convergence, if you want to compute a/b, look up an approximate inverse to b (call it b'), and run the algorithm on ab'/bb'. This will converge rapidly since $bb' \approx 1$.

To deal with the self-correction problem, the computation should be run with a few bits of extra precision to compensate for rounding errors. However, Goldschmidt's algorithm does have a weak form of self-correction, in that the precise value of the r_i does not matter. Thus, in the first few iterations, when the full precision of $1 - \delta^{2^i}$ is not needed you can choose r_i to be a truncation of $1 + \delta^{2^i}$, which may make these iterations run faster without affecting the speed of convergence. If r_i is truncated, then y_i is no longer exactly $1 - \delta^{2^i}$. Thus, Equation J.6.3 can no longer be used, but it is easy to organize the computation so that it does not depend on the precise value of r_i. With these changes, Goldschmidt's algorithm is as follows (the notes in brackets show the connection with our earlier formulas).

1. Scale a and b so that $1 \le b < 2$.
2. Look up an approximation to $1/b$ (call it b') in a table.
3. Set $x_0 = ab'$ and $y_0 = bb'$.

4. Iterate until x_i is close enough to a/b:

   Loop

$$r \approx 2 - y \qquad [\text{if } y_i = 1 + \delta_i, \text{then } r \approx 1 - \delta_i]$$
$$y = y \times r \qquad [y_{i+1} = y_i \times r \approx 1 - \delta_i{}^2]$$
$$x_{i+1} = x_i \times r \quad [x_{i+1} = x_i \times r]$$

   End loop

The two iteration methods are related. Suppose in Newton's method that we unroll the iteration and compute each term x_{i+1} directly in terms of b, instead of recursively in terms of x_i. By carrying out this calculation (see Exercise J.22), we discover that

$$x_{i+1} = x_0(2 - x_0 b)\left[\left(1 + (x_0 b - 1)^2\right]\left[1 + (x_0 b - 1)^4\right]\cdots\left[1 + (x_0 b - 1)^{2^i}\right]\right.$$

This formula is very similar to Equation J.6.3. In fact, they are identical if a and b in J.6.3 are replaced with ax_0, bx_0, and $a = 1$. Thus, if the iterations were done to infinite precision, the two methods would yield exactly the same sequence x_i.

The advantage of iteration is that it doesn't require special divide hardware. Instead, it can use the multiplier (which, however, requires extra control). Further, on each step, it delivers twice as many digits as in the previous step—unlike ordinary division, which produces a fixed number of digits at every step.

There are two disadvantages with inverting by iteration. The first is that the IEEE standard requires division to be correctly rounded, but iteration only delivers a result that is close to the correctly rounded answer. In the case of Newton's iteration, which computes $1/b$ instead of a/b directly, there is an additional problem. Even if $1/b$ were correctly rounded, there is no guarantee that a/b will be. An example in decimal with $p = 2$ is $a = 13$, $b = 51$. Then $a/b = .2549\ldots$, which rounds to .25. But $1/b = .0196\ldots$, which rounds to .020, and then $a \times .020 = .26$, which is off by 1. The second disadvantage is that iteration does not give a remainder. This is especially troublesome if the floating-point divide hardware is being used to perform integer division, since a remainder operation is present in almost every high-level language.

Traditional folklore has held that the way to get a correctly rounded result from iteration is to compute $1/b$ to slightly more than $2p$ bits, compute a/b to slightly more than $2p$ bits, and then round to p bits. However, there is a faster way, which apparently was first implemented on the TI 8847. In this method, a/b is computed to about 6 extra bits of precision, giving a preliminary quotient q. By comparing qb with a (again with only 6 extra bits), it is possible to quickly decide whether q is correctly rounded or whether it needs to be bumped up or down by 1 in the least-significant place. This algorithm is explored further in Exercise J.21.

One factor to take into account when deciding on division algorithms is the relative speed of division and multiplication. Since division is more complex than multiplication, it will run more slowly. A common rule of thumb is that division algorithms should try to achieve a speed that is about one-third that of multiplication.

One argument in favor of this rule is that there are real programs (such as some versions of spice) where the ratio of division to multiplication is 1:3. Another place where a factor of 3 arises is in the standard iterative method for computing square root. This method involves one division per iteration, but it can be replaced by one using three multiplications. This is discussed in Exercise J.20.

Floating-Point Remainder

For nonnegative integers, integer division and remainder satisfy:

$$a = (a \text{ DIV } b)b + a \text{ REM } b, \qquad 0 \le a \text{ REM } b < b$$

A floating-point remainder x REM y can be similarly defined as $x = \text{INT}(x/y)y + x$ REM y. How should x/y be converted to an integer? The IEEE remainder function uses the round-to-even rule. That is, pick $n = \text{INT}\,(x/y)$ so that $|x/y - n| \le 1/2$. If two different n satisfy this relation, pick the even one. Then REM is defined to be $x - yn$. Unlike integers where $0 \le a$ REM $b < b$, for floating-point numbers $|x \text{ REM } y| \le y/2$. Although this defines REM precisely, it is not a practical operational definition, because n can be huge. In single precision, n could be as large as $2^{127}/2^{-126} = 2^{253} \approx 10^{76}$.

There is a natural way to compute REM if a direct division algorithm is used. Proceed as if you were computing x/y. If $x = s_1 2^{e_1}$ and $y = s_2 2^{e_2}$ and the divider is as in Figure J.2(b) (page J-4), then load s_1 into P and s_2 into B. After $e_1 - e_2$ division steps, the P register will hold a number r of the form $x - yn$ satisfying $0 \le r < y$. Since the IEEE remainder satisfies $|\text{REM}| \le y/2$, REM is equal to either r or $r - y$. It is only necessary to keep track of the last quotient bit produced, which is needed to resolve halfway cases. Unfortunately, $e_1 - e_2$ can be a lot of steps, and floating-point units typically have a maximum amount of time they are allowed to spend on one instruction. Thus, it is usually not possible to implement REM directly. None of the chips discussed in Section J.10 implements REM, but they could by providing a remainder-step instruction—this is what is done on the Intel 8087 family. A remainder step takes as arguments two numbers x and y, and performs divide steps until either the remainder is in P or n steps have been performed, where n is a small number, such as the number of steps required for division in the highest-supported precision. Then REM can be implemented as a software routine that calls the REM step instruction $\lfloor (e_1 - e_2)/n \rfloor$ times, initially using x as the numerator but then replacing it with the remainder from the previous REM step.

REM can be used for computing trigonometric functions. To simplify things, imagine that we are working in base 10 with five significant figures, and consider computing $\sin x$. Suppose that $x = 7$. Then we can reduce by $\pi = 3.1416$ and compute $\sin(7) = \sin(7 - 2 \times 3.1416) = \sin(0.7168)$ instead. But, suppose we want to compute $\sin(2.0 \times 10^5)$. Then $2 \times 10^5/3.1416 = 63661.8$, which in our five-place system comes out to be 63662. Since multiplying 3.1416 times 63662 gives 200000.5392, which rounds to 2.0000×10^5, argument reduction reduces 2×10^5 to 0, which is not even close to being correct. The problem is that our

five-place system does not have the precision to do correct argument reduction. Suppose we had the REM operator. Then we could compute 2×10^5 REM 3.1416 and get $-.53920$. However, this is still not correct because we used 3.1416, which is an approximation for π. The value of 2×10^5 REM π is $-.071513$.

Traditionally, there have been two approaches to computing periodic functions with large arguments. The first is to return an error for their value when x is large. The second is to store π to a very large number of places and do exact argument reduction. The REM operator is not much help in either of these situations. There is a third approach that has been used in some math libraries, such as the Berkeley UNIX 4.3bsd release. In these libraries, π is computed to the nearest floating-point number. Let's call this machine π, and denote it by π'. Then, when computing sin x, reduce x using x REM π'. As we saw in the above example, x REM π' is quite different from x REM π when x is large, so that computing sin x as sin(x REM π') will not give the exact value of sin x. However, computing trigonometric functions in this fashion has the property that all familiar identities (such as $\sin^2 x + \cos^2 x = 1$) are true to within a few rounding errors. Thus, using REM together with machine π provides a simple method of computing trigonometric functions that is accurate for small arguments and still may be useful for large arguments.

When REM is used for argument reduction, it is very handy if it also returns the low-order bits of n (where x REM $y = x - ny$). This is because a practical implementation of trigonometric functions will reduce by something smaller than 2π. For example, it might use $\pi/2$, exploiting identities such as $\sin(x - \pi/2) = -\cos x$, $\sin(x - \pi) = -\sin x$. Then the low bits of n are needed to choose the correct identity.

J.7 More on Floating-Point Arithmetic

Before leaving the subject of floating-point arithmetic, we present a few additional topics.

Fused Multiply-Add

Probably the most common use of floating-point units is performing matrix operations, and the most frequent matrix operation is multiplying a matrix times a matrix (or vector), which boils down to computing an inner product, $x_1 \cdot y_1 + x_2 \cdot y_2 + \ldots + x_n \cdot y_n$. Computing this requires a series of multiply-add combinations.

Motivated by this, the IBM RS/6000 introduced a single instruction that computes $ab + c$, the *fused multiply-add*. Although this requires being able to read three operands in a single instruction, it has the potential for improving the performance of computing inner products.

The fused multiply-add computes $ab + c$ exactly and then rounds. Although rounding only once increases the accuracy of inner products somewhat, that is not its primary motivation. There are two main advantages of rounding once. First,

as we saw in the previous sections, rounding is expensive to implement because it may require an addition. By rounding only once, an addition operation has been eliminated. Second, the extra accuracy of fused multiply-add can be used to compute correctly rounded division and square root when these are not available directly in hardware. Fused multiply-add can also be used to implement efficient floating-point multiple-precision packages.

The implementation of correctly rounded division using fused multiply-add has many details, but the main idea is simple. Consider again the example from Section J.6 (page J-30), which was computing a/b with $a = 13$, $b = 51$. Then $1/b$ rounds to $b' = .020$, and ab' rounds to $q' = .26$, which is not the correctly rounded quotient. Applying fused multiply-add twice will correctly adjust the result, via the formulas

$$r = a - bq'$$
$$q'' = q' + rb'$$

Computing to two-digit accuracy, $bq' = 51 \times .26$ rounds to 13, and so $r = a - bq'$ would be 0, giving no adjustment. But using fused multiply-add gives $r = a - bq' = 13 - (51 \times .26) = -.26$, and then $q'' = q' + rb' = .26 - .0052 = .2548$, which rounds to the correct quotient, .25. More details can be found in the papers by Montoye, Hokenek, and Runyon [1990] and Markstein [1990].

Precisions

The standard specifies four precisions: *single*, *single extended*, *double*, and *double extended*. The properties of these precisions are summarized in Figure J.7 (page J-16). Implementations are not required to have all four precisions, but are encouraged to support either the combination of single and single extended or all of single, double, and double extended. Because of the widespread use of double precision in scientific computing, double precision is almost always implemented. Thus, the computer designer usually only has to decide whether to support double extended and, if so, how many bits it should have.

The Motorola 68882 and Intel 387 coprocessors implement extended precision using the smallest allowable size of 80 bits (64 bits of significand). However, many of the more recently designed, high-performance floating-point chips do not implement 80-bit extended precision. One reason is that the 80-bit width of extended precision is awkward for 64-bit buses and registers. Some new architectures, such as SPARC V8 and PA-RISC, specify a 128-bit extended (or *quad*) precision. They have established a *de facto* convention for quad that has 15 bits of exponent and 113 bits of significand.

Although most high-level languages do not provide access to extended precision, it is very useful to writers of mathematical software. As an example, consider writing a library routine to compute the length of a vector (x,y) in the plane, namely, $\sqrt{x^2 + y^2}$. If x is larger than $2^{E_{\max}/2}$, then computing this in the obvious way will overflow. This means that either the allowable exponent range for this subroutine

will be cut in half or a more complex algorithm using scaling will have to be employed. But, if extended precision is available, then the simple algorithm will work. Computing the length of a vector is a simple task, and it is not difficult to come up with an algorithm that doesn't overflow. However, there are more complex problems for which extended precision means the difference between a simple, fast algorithm and a much more complex one. One of the best examples of this is binary-to-decimal conversion. An efficient algorithm for binary-to-decimal conversion that makes essential use of extended precision is very readably presented in Coonen [1984]. This algorithm is also briefly sketched in Goldberg [1991]. Computing accurate values for transcendental functions is another example of a problem that is made much easier if extended precision is present.

One very important fact about precision concerns *double rounding*. To illustrate in decimals, suppose that we want to compute 1.9×0.66 and that single precision is two digits, while extended precision is three digits. The exact result of the product is 1.254. Rounded to extended precision, the result is 1.25. When further rounded to single precision, we get 1.2. However, the result of 1.9×0.66 correctly rounded to single precision is 1.3. Thus, rounding twice may not produce the same result as rounding once. Suppose you want to build hardware that only does double-precision arithmetic. Can you simulate single precision by computing first in double precision and then rounding to single? The above example suggests that you can't. However, double rounding is not always dangerous. In fact, the following rule is true (this is not easy to prove, but see Exercise J.25).

If x *and* y *have* p*-bit significands, and* x + y *is computed exactly and then rounded to* q *places, a second rounding to* p *places will not change the answer if* $q \geq 2p+2$. *This is true not only for addition, but also for multiplication, division, and square root.*

In our example above, $q=3$ and $p=2$, so $q \not\geq 2p+2$ is not true. On the other hand, for IEEE arithmetic, double precision has $q=53$ and $p=24$, so $q=53 \not\geq 2p+2=50$. Thus, single precision can be implemented by computing in double precision—that is, computing the answer exactly and then rounding to double—and then rounding to single precision.

Exceptions

The IEEE standard defines five exceptions: underflow, overflow, divide by zero, inexact, and invalid. By default, when these exceptions occur, they merely set a flag and the computation continues. The flags are *sticky*, meaning that once set they remain set until explicitly cleared. The standard strongly encourages implementations to provide a trap-enable bit for each exception. When an exception with an enabled trap handler occurs, a user trap handler is called, and the value of the associated exception flag is undefined. In Section J.3 we mentioned that $\sqrt{-3}$ has the value NaN and 1/0 is ∞. These are examples of operations that raise an exception.

By default, computing $\sqrt{-3}$ sets the invalid flag and returns the value NaN. Similarly 1/0 sets the divide-by-zero flag and returns ∞.

The underflow, overflow, and divide-by-zero exceptions are found in most other systems. The *invalid exception* is for the result of operations such as $\sqrt{-1}$, 0/0, or $\infty - \infty$, which don't have any natural value as a floating-point number or as $\pm\infty$. The *inexact exception* is peculiar to IEEE arithmetic and occurs either when the result of an operation must be rounded or when it overflows. In fact, since 1/0 and an operation that overflows both deliver ∞, the exception flags must be consulted to distinguish between them. The inexact exception is an unusual "exception," in that it is not really an exceptional condition because it occurs so frequently. Thus, enabling a trap handler for the inexact exception will most likely have a severe impact on performance. Enabling a trap handler doesn't affect whether an operation is exceptional except in the case of underflow. This is discussed below.

The IEEE standard assumes that when a trap occurs, it is possible to identify the operation that trapped and its operands. On machines with pipelining or multiple arithmetic units, when an exception occurs, it may not be enough to simply have the trap handler examine the program counter. Hardware support may be necessary to identify exactly which operation trapped.

Another problem is illustrated by the following program fragment.

```
r1 = r2/r3
r2 = r4 + r5
```

These two instructions might well be executed in parallel. If the divide traps, its argument `r2` could already have been overwritten by the addition, especially since addition is almost always faster than division. Computer systems that support trapping in the IEEE standard must provide some way to save the value of `r2`, either in hardware or by having the compiler avoid such a situation in the first place. This kind of problem is not peculiar to floating point. In the sequence

```
r1 = 0(r2)
r2 = r3
```

it would be efficient to execute `r2 = r3` while waiting for memory. But, if accessing `0(r2)` causes a page fault, `r2` might no longer be available for restarting the instruction `r1 = 0(r2)`.

One approach to this problem, used in the MIPS R3010, is to identify instructions that may cause an exception early in the instruction cycle. For example, an addition can overflow only if one of the operands has an exponent of $E_{\max}$, and so on. This early check is conservative: It might flag an operation that doesn't actually cause an exception. However, if such false positives are rare, then this technique will have excellent performance. When an instruction is tagged as being possibly exceptional, special code in a trap handler can compute it without destroying any state. Remember that all these problems occur only when trap handlers are enabled. Otherwise, setting the exception flags during normal processing is straightforward.

Underflow

We have alluded several times to the fact that detection of underflow is more complex than for the other exceptions. The IEEE standard specifies that if an underflow trap handler is enabled, the system must trap if the result is denormal. On the other hand, if trap handlers are disabled, then the underflow flag is set only if there is a loss of accuracy—that is, if the result must be rounded. The rationale is, if no accuracy is lost on an underflow, there is no point in setting a warning flag. But if a trap handler is enabled, the user might be trying to simulate flush-to-zero and should therefore be notified whenever a result dips below $1.0 \times 2^{E_{\min}}$.

So if there is no trap handler, the underflow exception is signaled only when the result is denormal and inexact, but the definitions of *denormal* and *inexact* are both subject to multiple interpretations. Normally, inexact means there was a result that couldn't be represented exactly and had to be rounded. Consider the example (in a base 2 floating-point system with 3-bit significands) of $\left(1.11_2 \times 2^{-2}\right) \times \left(1.11_2 \times 2^{E_{\min}}\right) = 0.110001_2 \times 2^{E_{\min}}$, with round to nearest in effect. The delivered result is $0.11_2 \times 2^{E_{\min}}$, which had to be rounded, causing inexact to be signaled. But is it correct to also signal underflow? Gradual underflow loses significance because the exponent range is bounded. If the exponent range were unbounded, the delivered result would be $1.10_2 \times 2^{E_{\min}-1}$, exactly the same answer obtained with gradual underflow. The fact that denormalized numbers have fewer bits in their significand than normalized numbers therefore doesn't make any difference in this case. The commentary to the standard [Cody et al. 1984] encourages this as the criterion for setting the underflow flag. That is, it should be set whenever the delivered result is different from what would be delivered in a system with the same fraction size, but with a very large exponent range. However, owing to the difficulty of implementing this scheme, the standard allows setting the underflow flag whenever the result is denormal and different from the infinitely precise result.

There are two possible definitions of what it means for a result to be denormal. Consider the example of $1.10_2 \times 2^{-1}$ multiplied by $1.10_2 \times 2^{E_{\min}}$. The exact product is $0.1111 \times 2^{E_{\min}}$. The rounded result is the normal number $1.00_2 \times 2^{E_{\min}}$. Should underflow be signaled? Signaling underflow means that you are using the *before rounding* rule, because the result was denormal before rounding. Not signaling underflow means that you are using the *after rounding* rule, because the result is normalized after rounding. The IEEE standard provides for choosing either rule; however, the one chosen must be used consistently for all operations.

To illustrate these rules, consider floating-point addition. When the result of an addition (or subtraction) is denormal, it is always exact. Thus, the underflow flag never needs to be set for addition. That's because if traps are not enabled then no exception is raised. And if traps are enabled, the value of the underflow flag is undefined, so again it doesn't need to be set.

One final subtlety should be mentioned concerning underflow. When there is no underflow trap handler, the result of an operation on p-bit numbers that causes

an underflow is a denormal number with $p-1$ or fewer bits of precision. When traps are enabled, the trap handler is provided with the result of the operation rounded to p bits and with the exponent wrapped around. Now there is a potential double-rounding problem. If the trap handler wants to return the denormal result, it can't just round its argument, because that might lead to a double-rounding error. Thus, the trap handler must be passed at least one extra bit of information if it is to be able to deliver the correctly rounded result.

J.8 Speeding Up Integer Addition

The previous section showed that many steps go into implementing floating-point operations; however, each floating-point operation eventually reduces to an integer operation. Thus, increasing the speed of integer operations will also lead to faster floating point.

Integer addition is the simplest operation and the most important. Even for programs that don't do explicit arithmetic, addition must be performed to increment the program counter and to calculate addresses. Despite the simplicity of addition, there isn't a single best way to perform high-speed addition. We will discuss three techniques that are in current use: carry-lookahead, carry-skip, and carry-select.

Carry-Lookahead

An n-bit adder is just a combinational circuit. It can therefore be written by a logic formula whose form is a sum of products and can be computed by a circuit with two levels of logic. How do you figure out what this circuit looks like? From Equation J.2.1 (page J-3) the formula for the ith sum can be written as:

J.8.1
$$s_i = a_i\overline{b}_i\overline{c}_i + \overline{a}_i b_i\overline{c}_i + \overline{a}_i\overline{b}_i c_i + a_i b_i c_i$$

where c_i is both the carry-in to the ith adder and the carry-out from the $(i-1)$-st adder.

The problem with this formula is that, although we know the values of a_i and b_i—they are inputs to the circuit—we don't know c_i. So our goal is to write c_i in terms of a_i and b_i. To accomplish this, we first rewrite Equation J.2.2 (page J-3) as:

J.8.2
$$c_i = g_{i-1} + p_{i-1}c_{i-1},\quad g_{i-1} = a_{i-1}b_{i-1},\quad p_{i-1} = a_{i-1} + b_{i-1}$$

Here is the reason for the symbols p and g: If g_{i-1} is true, then c_i is certainly true, so a carry is *generated*. Thus, g is for generate. If p_{i-1} is true, then if c_{i-1} is true, it is *propagated* to c_i. Start with Equation J.8.1 and use Equation J.8.2 to replace c_i with $g_{i-1}+p_{i-1}c_{i-1}$. Then, use Equation J.8.2 with $i-1$ in place of i to replace c_{i-1} with c_{i-2}, and so on. This gives the result:

J.8.3
$$c_i = g_{i-1} + p_{i-1}g_{i-2} + p_{i-1}p_{i-2}g_{i-3} + \cdots + p_{i-1}p_{i-2}\cdots p_1 g_0 + p_{i-1}p_{i-2}\cdots p_1 p_0 c_0$$

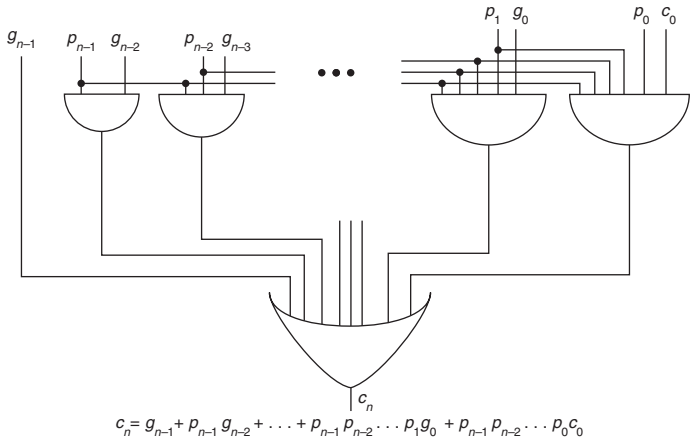


Figure J.14 Pure carry-lookahead circuit for computing the carry-out c_n of an *n*-bit adder.

An adder that computes carries using Equation J.8.3 is called a *carry-lookahead adder*, or CLA. A CLA requires one logic level to form p and g, two levels to form the carries, and two for the sum, for a grand total of five logic levels. This is a vast improvement over the $2n$ levels required for the ripple-carry adder.

Unfortunately, as is evident from Equation J.8.3 or from Figure J.14, a carry-lookahead adder on n bits requires a fan-in of $n+1$ at the OR gate as well as at the rightmost AND gate. Also, the p_{n-1} signal must drive n AND gates. In addition, the rather irregular structure and many long wires of Figure J.14 make it impractical to build a full carry-lookahead adder when n is large.

However, we can use the carry-lookahead idea to build an adder that has about $\log_2 n$ logic levels (substantially fewer than the $2n$ required by a ripplecarry adder) and yet has a simple, regular structure. The idea is to build up the p's and g's in steps. We have already seen that

$$c_1 = g_0 + c_0 p_0$$

This says there is a carry-out of the 0th position (c_1) either if there is a carry generated in the 0th position or if there is a carry into the 0th position and the carry propagates. Similarly,

$$c_2 = G_{01} + P_{01} c_0$$

G_{01} means there is a carry generated out of the block consisting of the first two bits. P_{01} means that a carry propagates through this block. P and G have the following logic equations:

$$G_{01} = g_1 + p_1 g_0$$
$$P_{01} = p_1 p_0$$

More generally, for any j with $i<j$, $j+1<k$, we have the recursive relations:

$$c_{k+1} = G_{ik} + P_{ik}c_i \quad \text{J.8.4}$$

$$G_{ik} = G_{j+1,k} + P_{j+1,k}G_{ij} \quad \text{J.8.5}$$

$$P_{ik} = P_{ij}P_{j+1,k} \quad \text{J.8.6}$$

Equation J.8.5 says that a carry is generated out of the block consisting of bits i through k inclusive if it is generated in the high-order part of the block $(j+1, k)$ or if it is generated in the low-order part of the block (i,j) and then propagated through the high part. These equations will also hold for $i \le j < k$ if we set $G_{ii} = g_i$ and $P_{ii} = p_i$.

Example Express P_{03} and G_{03} in terms of p's and g's.

Answer Using Equation J.8.6, $P_{03} = P_{01}P_{23} = P_{00}P_{11}P_{22}P_{33}$. Since $P_{ii} = p_i$, $P_{03} = p_0p_1p_2p_3$. For G_{03}, Equation J.8.5 says $G_{03} = G_{23} + P_{23}G_{01} = (G_{33} + P_{33}G_{22}) + (P_{22}P_{33})(G_{11} + P_{11}G_{00}) = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0$.

With these preliminaries out of the way, we can now show the design of a practical CLA. The adder consists of two parts. The first part computes various values of P and G from p_i and g_i, using Equations J.8.5 and J.8.6; the second part uses these P and G values to compute all the carries via Equation J.8.4. The first part of the design is shown in Figure J.15. At the top of the diagram, input numbers $a_7 \ldots a_0$ and $b_7 \ldots b_0$ are converted to p's and g's using cells of type 1. Then various P's and G's are generated by combining cells of type 2 in a binary tree structure. The second part of the design is shown in Figure J.16. By feeding c_0 in at the bottom of this tree, all the carry bits come out at the top. Each cell must know a pair of (P,G) values in order to do the conversion, and the value it needs is written inside the cells. Now compare Figures J.15 and J.16. There is a one-to-one correspondence between cells, and the value of (P,G) needed by the carry-generating cells is exactly the value known by the corresponding (P,G)-generating cells. The combined cell is shown in Figure J.17. The numbers to be added flow into the top and downward through the tree, combining with c_0 at the bottom and flowing back up the tree to form the carries. Note that one thing is missing from Figure J.17: a small piece of extra logic to compute c_8 for the carry-out of the adder.

The bits in a CLA must pass through about $\log_2 n$ logic levels, compared with $2n$ for a ripple-carry adder. This is a substantial speed improvement, especially for a large n. Whereas the ripple-carry adder had n cells, however, the CLA has $2n$ cells, although in our layout they will take $n \log n$ space. The point is that a small investment in size pays off in a dramatic improvement in speed.

A number of technology-dependent modifications can improve CLAs. For example, if each node of the tree has three inputs instead of two, then the height

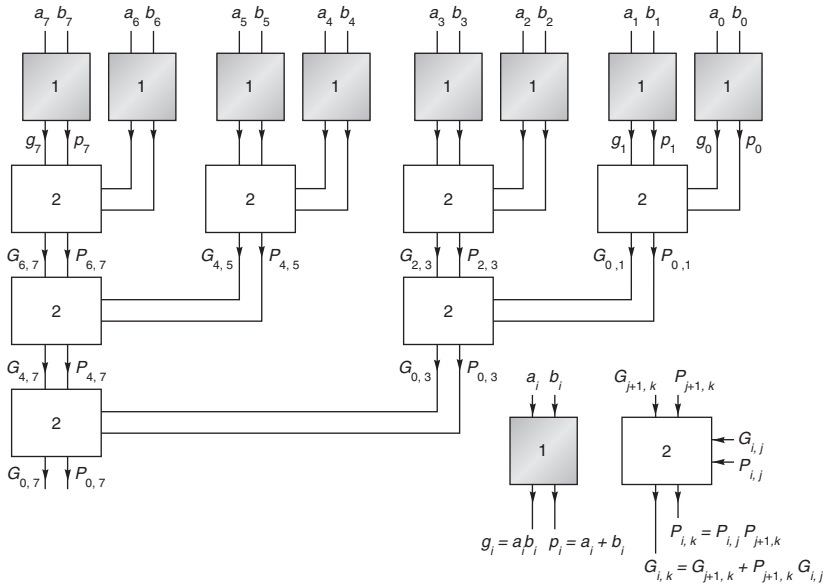


Figure J.15 First part of carry-lookahead tree. As signals flow from the top to the bottom, various values of *P* and *G* are computed.

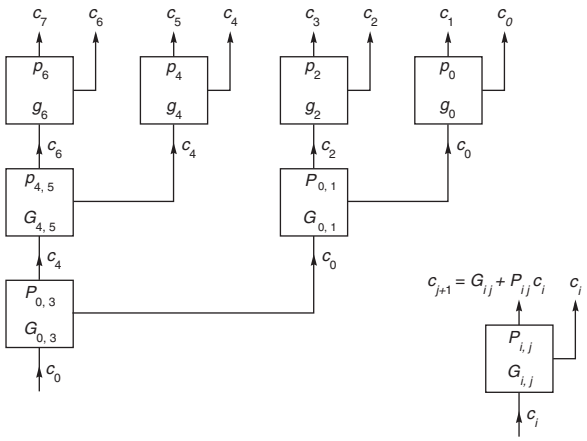


Figure J.16 Second part of carry-lookahead tree. Signals flow from the bottom to the top, combining with *P* and *G* to form the carries.

of the tree will decrease from $\log_2 n$ to $\log_3 n$. Of course, the cells will be more complex and thus might operate more slowly, negating the advantage of the decreased height. For technologies where rippling works well, a hybrid design might be better. This is illustrated in Figure J.19. Carries ripple between adders

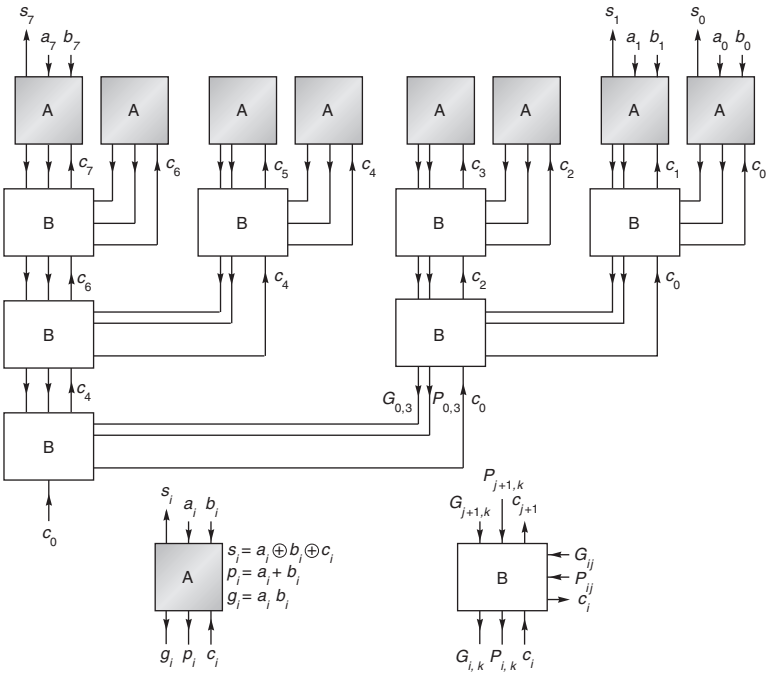


Figure J.17 Complete carry-lookahead tree adder. This is the combination of Figures J.15 and J.16. The numbers to be added enter at the top, flow to the bottom to combine with c_0, and then flow back up to compute the sum bits.

at the top level, while the "B" boxes are the same as those in Figure J.17. This design will be faster if the time to ripple between four adders is faster than the time it takes to traverse a level of "B" boxes. (To make the pattern more clear, Figure J.19 shows a 16-bit adder, so the 8-bit adder of Figure J.17 corresponds to the right half of Figure J.19.)

Carry-Skip Adders

A *carry-skip adder* sits midway between a ripple-carry adder and a carry-lookahead adder, both in terms of speed and cost. (A carry-skip adder is not called a CSA, as that name is reserved for carry-save adders.) The motivation for this adder comes from examining the equations for P and G. For example,

$$P_{03} = p_0 p_1 p_2 p_3$$
$$G_{03} = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$$

Computing P is much simpler than computing G, and a carry-skip adder only computes the P's. Such an adder is illustrated in Figure J.18. Carries begin rippling

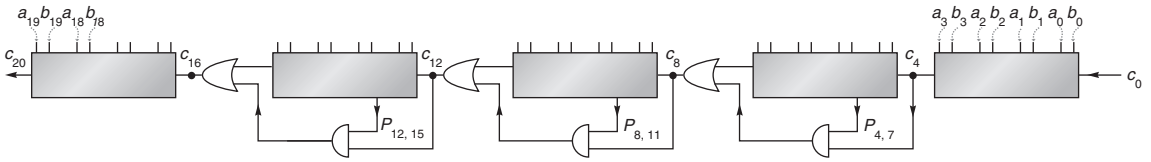


Figure J.18 Carry-skip adder. This is a 20-bit carry-skip adder ($n = 20$) with each block 4 bits wide ($k = 4$).

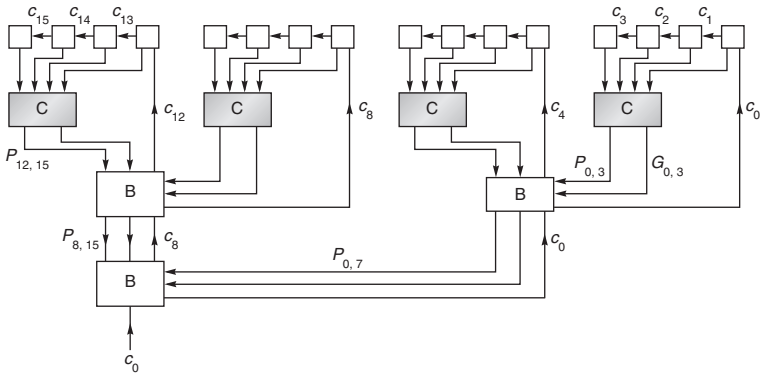


Figure J.19 Combination of CLA and ripple-carry adder. In the top row, carries ripple within each group of four boxes.

simultaneously through each block. If any block generates a carry, then the carry-out of a block will be true, even though the carry-in to the block may not be correct yet. If at the start of each add operation the carry-in to each block is 0, then no spurious carry-outs will be generated. Thus, the carry-out of each block can be thought of as if it were the *G* signal. Once the carry-out from the least-significant block is generated, it not only feeds into the next block but is also fed through the AND gate with the *P* signal from that next block. If the carry-out and *P* signals are both true, then the carry *skips* the second block and is ready to feed into the third block, and so on. The carry-skip adder is only practical if the carry-in signals can be easily cleared at the start of each operation—for example, by precharging in CMOS.

To analyze the speed of a carry-skip adder, let's assume that it takes 1 time unit for a signal to pass through two logic levels. Then it will take k time units for a carry to ripple across a block of size k, and it will take 1 time unit for a carry to skip a block. The longest signal path in the carry-skip adder starts with a carry being generated at the 0th position. If the adder is n bits wide, then it takes k time units to ripple through the first block, $n/k - 2$ time units to skip blocks, and k more to ripple through the last block. To be specific: if we have a 20-bit adder broken into groups of 4 bits, it will take $4 + (20/4 - 2) + 4 = 11$ time units to perform an

add. Some experimentation reveals that there are more efficient ways to divide 20 bits into blocks. For example, consider five blocks with the least-significant 2 bits in the first block, the next 5 bits in the second block, followed by blocks of size 6, 5, and 2. Then the add time is reduced to 9 time units. This illustrates an important general principle. For a carry-skip adder, making the interior blocks larger will speed up the adder. In fact, the same idea of varying the block sizes can sometimes speed up other adder designs as well. Because of the large amount of rippling, a carry-skip adder is most appropriate for technologies where rippling is fast.

Carry-Select Adder

A *carry-select adder* works on the following principle: Two additions are performed in parallel, one assuming the carry-in is 0 and the other assuming the carry-in is 1. When the carry-in is finally known, the correct sum (which has been precomputed) is simply selected. An example of such a design is shown in Figure J.20. An 8-bit adder is divided into two halves, and the carry-out from the lower half is used to select the sum bits from the upper half. If each block is computing its sum using rippling (a linear time algorithm), then the design in Figure J.20 is twice as fast at 50% more cost. However, note that the c_4 signal must drive many muxes, which may be very slow in some technologies. Instead of dividing the adder into halves, it could be divided into quarters for a still further speedup. This is illustrated in Figure J.21. If it takes k time units for a block to add k-bit numbers, and if it takes 1 time unit to compute the mux input from the two carry-out signals, then for optimal operation each block should be 1 bit wider than the next, as shown in Figure J.21. Therefore, as in the carry-skip adder, the best design involves variable-size blocks.

As a summary of this section, the asymptotic time and space requirements for the different adders are given in Figure J.22. (The times for carry-skip and

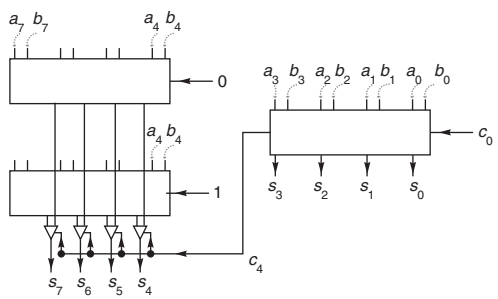


Figure J.20 Simple carry-select adder. At the same time that the sum of the low-order 4 bits is being computed, the high-order bits are being computed twice in parallel: once assuming that $c_4 = 0$ and once assuming $c_4 = 1$.

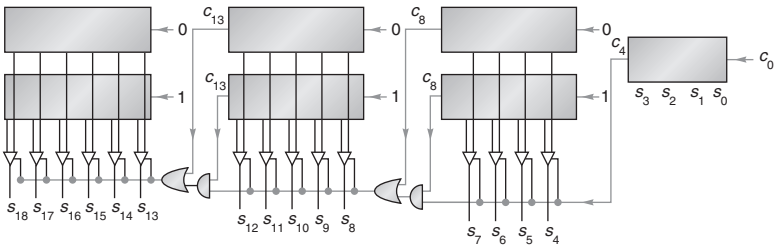


Figure J.21 Carry-select adder. As soon as the carry-out of the rightmost block is known, it is used to select the other sum bits.

Adder	Time	Space
Ripple	$0(n)$	$0(n)$
CLA	$0(\log n)$	$0(n \log n)$
Carry-skip	$0(\sqrt{n})$	$0(n)$
Carry-select	$0(\sqrt{n})$	$0(n)$

Figure J.22 Asymptotic time and space requirements for four different types of adders.

carry-select come from a careful choice of block size. See Exercise J.26 for the carry-skip adder.) These different adders shouldn't be thought of as disjoint choices, but rather as building blocks to be used in constructing an adder. The utility of these different building blocks is highly dependent on the technology used. For example, the carry-select adder works well when a signal can drive many muxes, and the carry-skip adder is attractive in technologies where signals can be cleared at the start of each operation. Knowing the asymptotic behavior of adders is useful in understanding them, but relying too much on that behavior is a pitfall. The reason is that asymptotic behavior is only important as n grows very large. But n for an adder is the bits of precision, and double precision today is the same as it was 20 years ago—about 53 bits. Although it is true that as computers get faster, computations get longer—and thus have more rounding error, which in turn requires more precision—this effect grows very slowly with time.

J.9 Speeding Up Integer Multiplication and Division

The multiplication and division algorithms presented in Section J.2 are fairly slow, producing 1 bit per cycle (although that cycle might be a fraction of the CPU instruction cycle time). In this section, we discuss various techniques for higher-performance multiplication and division, including the division algorithm used in the Pentium chip.

Shifting over Zeros

Although the technique of shifting over zeros is not currently used much, it is instructive to consider. It is distinguished by the fact that its execution time is operand dependent. Its lack of use is primarily attributable to its failure to offer enough speedup over bit-at-a-time algorithms. In addition, pipelining, synchronization with the CPU, and good compiler optimization are difficult with algorithms that run in variable time. In multiplication, the idea behind shifting over zeros is to add logic that detects when the low-order bit of the A register is 0 (see Figure J.2(a) on page J-4) and, if so, skips the addition step and proceeds directly to the shift step—hence the term *shifting over zeros*.

What about shifting for division? In nonrestoring division, an ALU operation (either an addition or subtraction) is performed at every step. There appears to be no opportunity for skipping an operation. But think about division this way: To compute a/b, subtract multiples of b from a, and then report how many subtractions were done. At each stage of the subtraction process the remainder must fit into the P register of Figure J.2(b) (page J-4). In the case when the remainder is a small positive number, you normally subtract b; but suppose instead you only shifted the remainder and subtracted b the next time. As long as the remainder was sufficiently small (its high-order bit 0), after shifting it still would fit into the P register, and no information would be lost. However, this method does require changing the way we keep track of the number of times b has been subtracted from a. This idea usually goes under the name of *SRT division*, for Sweeney, Robertson, and Tocher, who independently proposed algorithms of this nature. The main extra complication of SRT division is that the quotient bits cannot be determined immediately from the sign of P at each step, as they can be in ordinary nonrestoring division.

More precisely, to divide a by b where a and b are n-bit numbers, load a and b into the A and B registers, respectively, of Figure J.2 (page J-4).

SRT Division

1. If B has k leading zeros when expressed using n bits, shift all the registers left k bits.
2. For $i = 0, n-1$,
   a) If the top three bits of P are equal, set $q_i = 0$ and shift (P,A) one bit left.
   b) If the top three bits of P are not all equal and P is negative, set $q_i = -1$ (also written as $\overline{1}$), shift (P,A) one bit left, and add B.
   c) Otherwise set $q_i = 1$, shift (P,A) one bit left, and subtract B.

   End loop
3. If the final remainder is negative, correct the remainder by adding B, and correct the quotient by subtracting 1 from q_0. Finally, the remainder must be shifted k bits right, where k is the initial shift.

P	A	
00000	1000	Divide 8 = 1000 by 3 = 0011. B contains 0011.
00010	0000	Step 1: B had two leading 0 s, so shift left by 2. B now contains 1100.
		Step 2.1: Top three bits are equal. This is case (a), so
00100	0000	set $q_0 = 0$ and shift.
		Step 2.2: Top three bits not equal and P ≥ 0 is case (c), so
01000	0001	set $q_1 = 1$ and shift.
+ 10100		Subtract B.
11100	0001	Step 2.3: Top bits equal is case (a), so
11000	0010	set $q_2 = 0$ and shift.
		Step 2.4: Top three bits unequal is case (b), so
10000	0101	set $q_3 = -1$ and shift.
+ 01100		Add B.
11100		Step 3. remainder is negative so restore it and subtract 1 from q.
+ 01100		
01000		Must undo the shift in step 1, so right-shift by 2 to get true remainder. Remainder = 10, quotient = $0101 - 1 = 0010$.

Figure J.23 SRT division of $1000_2/0011_2$. The quotient bits are shown in bold, using the notation 1 for −1.

A numerical example is given in Figure J.23. Although we are discussing integer division, it helps in explaining the algorithm to imagine the binary point just left of the most-significant bit. This changes Figure J.23 from $01000_2/0011_2$ to $0.1000_2/.0011_2$. Since the binary point is changed in both the numerator and denominator, the quotient is not affected. The (P,A) register pair holds the remainder and is a two's complement number. For example, if P contains 11110_2 and A = 0, then the remainder is $1.1110_2 = -1/8$. If r is the value of the remainder, then $-1 \le r < 1$.

Given these preliminaries, we can now analyze the SRT division algorithm. The first step of the algorithm shifts b so that $b \ge 1/2$. The rule for which ALU operation to perform is this: If $-1/4 \le r < 1/4$ (true whenever the top three bits of P are equal), then compute $2r$ by shifting (P,A) left one bit; if $r < 0$ (and hence $r < -1/4$, since otherwise it would have been eliminated by the first condition), then compute $2r + b$ by shifting and then adding; if $r \ge 1/4$ and subtract b from $2r$. Using $b \ge 1/2$, it is easy to check that these rules keep $-1/2 \le r < 1/2$. For nonrestoring division, we only have $|r| \le b$, and we need P to be $n+1$ bits wide. But, for SRT division, the bound on r is tighter, namely, $-1/2 \le r < 1/2$. Thus, we can save a bit by eliminating the high-order bit of P (and b and the adder). In particular, the test for equality of the top three bits of P becomes a test on just two bits.

The algorithm might change slightly in an implementation of SRT division. After each ALU operation, the P register can be shifted as many places as necessary to make either $r \ge 1/4$ or $r < -1/4$. By shifting k places, k quotient bits are set equal to zero all at once. For this reason SRT division is sometimes described as one that keeps the remainder normalized to $|r| \ge 1/4$.

Notice that the value of the quotient bit computed in a given step is based on which operation is performed in that step (which in turn depends on the result of the operation from the previous step). This is in contrast to nonrestoring division, where the quotient bit computed in the *i*th step depends on the result of the operation in the same step. This difference is reflected in the fact that when the final remainder is negative, the last quotient bit must be adjusted in SRT division, but not in nonrestoring division. However, the key fact about the quotient bits in SRT division is that they can include $\overline{1}$. Although Figure J.23 shows the quotient bits being stored in the low-order bits of A, an actual implementation can't do this because you can't fit the three values -1, 0, 1 into one bit. Furthermore, the quotient must be converted to ordinary two's complement in a full adder. A common way to do this is to accumulate the positive quotient bits in one register and the negative quotient bits in another, and then subtract the two registers after all the bits are known. Because there is more than one way to write a number in terms of the digits -1, 0, 1, SRT division is said to use a *redundant* quotient representation.

The differences between SRT division and ordinary nonrestoring division can be summarized as follows:

1. ALU decision rule—In nonrestoring division, it is determined by the sign of P; in SRT, it is determined by the two most-significant bits of P.
2. Final quotient—In nonrestoring division, it is immediate from the successive signs of P; in SRT, there are three quotient digits (1, 0, $\overline{1}$), and the final quotient must be computed in a full *n*-bit adder.
3. Speed—SRT division will be faster on operands that produce zero quotient bits.

The simple version of the SRT division algorithm given above does not offer enough of a speedup to be practical in most cases. However, later on in this section we will study variants of SRT division that are quite practical.

Speeding Up Multiplication with a Single Adder

As mentioned before, shifting-over-zero techniques are not used much in current hardware. We now discuss some methods that are in widespread use. Methods that increase the speed of multiplication can be divided into two classes: those that use a single adder and those that use multiple adders. Let's first discuss techniques that use a single adder.

In the discussion of addition we noted that, because of carry propagation, it is not practical to perform addition with two levels of logic. Using the cells of Figure J.17, adding two 64-bit numbers will require a trip through seven cells to compute the *P*'s and *G*'s and seven more to compute the carry bits, which will require at least 28 logic levels. In the simple multiplier of Figure J.2 on page J-4, each multiplication step passes through this adder. The amount of computation in each step can be dramatically reduced by using *carry-save adders* (CSAs). A carry-save adder is simply a collection of *n* independent full adders. A multiplier using

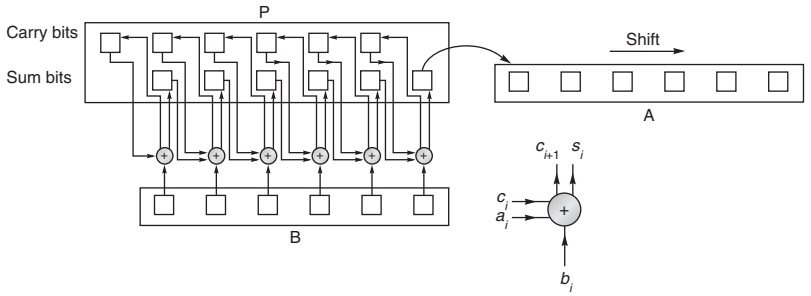


Figure J.24 Carry-save multiplier. Each circle represents a (3,2) adder working independently. At each step, the only bit of P that needs to be shifted is the low-order sum bit.

such an adder is illustrated in Figure J.24. Each circle marked "+" is a single-bit full adder, and each box represents one bit of a register. Each addition operation results in a pair of bits, stored in the sum and carry parts of P. Since each add is independent, only two logic levels are involved in the add—a vast improvement over 28.

To operate the multiplier in Figure J.24, load the sum and carry bits of P with zero and perform the first ALU operation. (If Booth recoding is used, it might be a subtraction rather than an addition.) Then shift the low-order sum bit of P into A, as well as shifting A itself. The $n-1$ high-order bits of P don't need to be shifted because on the next cycle the sum bits are fed into the next lower-order adder. Each addition step is substantially increased in speed, since each add cell is working independently of the others, and no carry is propagated.

There are two drawbacks to carry-save adders. First, they require more hardware because there must be a copy of register P to hold the carry outputs of the adder. Second, after the last step, the high-order word of the result must be fed into an ordinary adder to combine the sum and carry parts. One way to accomplish this is by feeding the output of P into the adder used to perform the addition operation. Multiplying with a carry-save adder is sometimes called *redundant multiplication* because P is represented using two registers. Since there are many ways to represent P as the sum of two registers, this representation is redundant. The term *carry-propagate adder* (CPA) is used to denote an adder that is not a CSA. A propagate adder may propagate its carries using ripples, carry-lookahead, or some other method.

Another way to speed up multiplication without using extra adders is to examine k low-order bits of A at each step, rather than just one bit. This is often called *higher-radix multiplication*. As an example, suppose that $k=2$. If the pair of bits is 00, add 0 to P; if it is 01, add B. If it is 10, simply shift b one bit left before adding it to P. Unfortunately, if the pair is 11, it appears we would have to compute $b+2b$. But this can be avoided by using a higher-radix version of Booth recoding. Imagine A as a base 4 number: When the digit 3 appears, change it to $\overline{1}$ and add 1 to the next higher digit to compensate. An extra benefit of using this scheme is that just like ordinary Booth recoding, it works for negative as well as positive integers (Section J.2).

The precise rules for radix-4 Booth recoding are given in Figure J.25. At the *i*th multiply step, the two low-order bits of the A register contain a_{2i} and a_{2i+1}. These two bits, together with the bit just shifted out (a_{2i-1}), are used to select the multiple of b that must be added to the P register. A numerical example is given in Figure J.26. Another name for this multiplication technique is *overlapping triplets,* since it looks at 3 bits to determine what multiple of b to use, whereas ordinary Booth recoding looks at 2 bits.

Besides having more complex control logic, overlapping triplets also requires that the P register be 1 bit wider to accommodate the possibility of $2b$ or $-2b$ being added to it. It is possible to use a radix-8 (or even higher) version of Booth recoding. In that case, however, it would be necessary to use the multiple 3B as a potential summand. Radix-8 multipliers normally compute 3B once and for all at the beginning of a multiplication operation.

Low-order bits of A		Last bit shifted out	
$2i+1$	$2i$	$2i-1$	Multiple
0	0	0	0
0	0	1	$+b$
0	1	0	$+b$
0	1	1	$+2b$
1	0	0	$-2b$
1	0	1	$-b$
1	1	0	$-b$
1	1	1	0

Figure J.25 Multiples of *b* to use for radix-4 Booth recoding. For example, if the two low-order bits of the A register are both 1, and the last bit to be shifted out of the A register is 0, then the correct multiple is $-b$, obtained from the second-to-last row of the table.

P	A	L	
00000	1001		Multiply $-7=1001$ times $-5=1011$. B contains 1011.
+ 11011			Low-order bits of A are 0, 1; L=0, so add B.
11011	1001		
11110	1110	0	Shift right by two bits, shifting in 1 s on the left.
+ 01010			Low-order bits of A are 1, 0; L=0, so add −2b.
01000	1110	0	
00010	0011	1	Shift right by two bits.
			Product is $35=0100011$.

Figure J.26 Multiplication of −7 times −5 using radix-4 Booth recoding. The column labeled L contains the last bit shifted out the right end of A.

Faster Multiplication with Many Adders

If the space for many adders is available, then multiplication speed can be improved. Figure J.27 shows a simple array multiplier for multiplying two 5-bit numbers, using three CSAs and one propagate adder. Part (a) is a block diagram of the kind we will use throughout this section. Parts (b) and (c) show the adder in more detail. All the inputs to the adder are shown in (b); the actual adders with their interconnections are shown in (c). Each row of adders in (c) corresponds to a box in

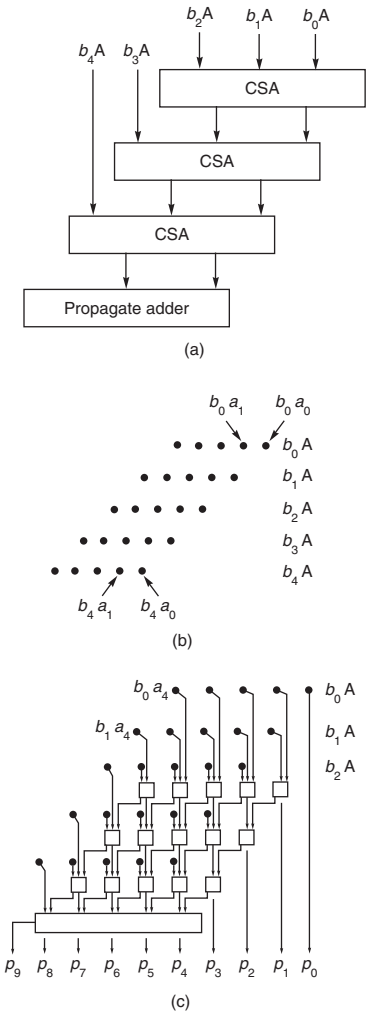


Figure J.27 An array multiplier. The 5-bit number in A is multiplied by $b_4b_3b_2b_1b_0$. Part (a) shows the block diagram, (b) shows the inputs to the array, and (c) expands the array to show all the adders.

(a). The picture is "twisted" so that bits of the same significance are in the same column. In an actual implementation, the array would most likely be laid out as a square instead.

The array multiplier in Figure J.27 performs the same number of additions as the design in Figure J.24, so its latency is not dramatically different from that of a single carry-save adder. However, with the hardware in Figure J.27, multiplication can be pipelined, increasing the total throughput. On the other hand, although this level of pipelining is sometimes used in array processors, it is not used in any of the single-chip, floating-point accelerators discussed in Section J.10. Pipelining is discussed in general in Appendix C and by Kogge [1981] in the context of multipliers.

Sometimes the space budgeted on a chip for arithmetic may not hold an array large enough to multiply two double-precision numbers. In this case, a popular design is to use a two-pass arrangement such as the one shown in Figure J.28. The first pass through the array "retires" 5 bits of B. Then the result of this first pass is fed back into the top to be combined with the next three summands. The result of this second pass is then fed into a CPA. This design, however, loses the ability to be pipelined.

If arrays require as many addition steps as the much cheaper arrangements in Figures J.2 and J.24, why are they so popular? First of all, using an array has a smaller latency than using a single adder—because the array is a combinational circuit, the signals flow through it directly without being clocked. Although the two-pass adder of Figure J.28 would normally still use a clock, the cycle time for passing through k arrays can be less than k times the clock that would be needed for designs like the ones in Figures J.2 or J.24. Second, the array is amenable to various schemes for further speedup. One of them is shown in Figure J.29. The idea of this design is that two adds proceed in parallel or, to put it another way, each stream passes through only half the adders. Thus, it runs at almost twice

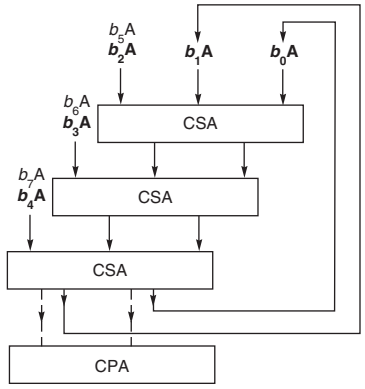


Figure J.28 Multipass array multiplier. Multiplies two 8-bit numbers with about half the hardware that would be used in a one-pass design like that of Figure J.27. At the end of the second pass, the bits flow into the CPA. The inputs used in the first pass are marked in bold.

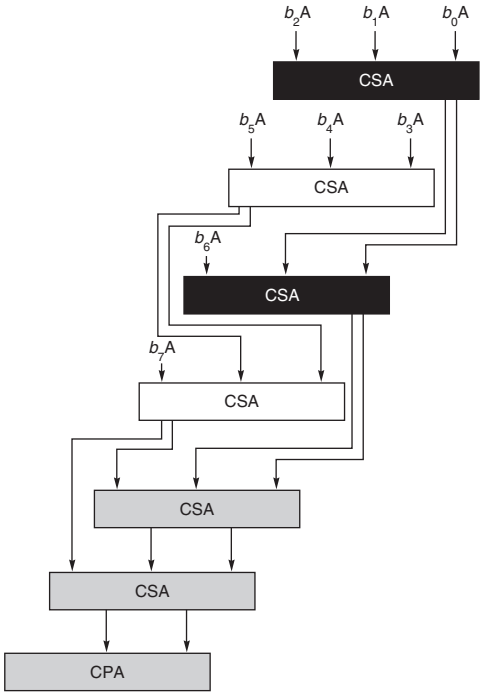


Figure J.29 Even/odd array. The first two adders work in parallel. Their results are fed into the third and fourth adders, which also work in parallel, and so on.

the speed of the multiplier in Figure J.27. This *even/odd* multiplier is popular in VLSI because of its regular structure. Arrays can also be speeded up using asynchronous logic. One of the reasons why the multiplier of Figure J.2 (page J-4) needs a clock is to keep the output of the adder from feeding back into the input of the adder before the output has fully stabilized. Thus, if the array in Figure J.28 is long enough so that no signal can propagate from the top through the bottom in the time it takes for the first adder to stabilize, it may be possible to avoid clocks altogether. Williams et al. [1987] discussed a design using this idea, although it is for dividers instead of multipliers.

The techniques of the previous paragraph still have a multiply time of $0(n)$, but the time can be reduced to log n using a tree. The simplest tree would combine pairs of summands $b_0\text{A}\cdots b_{n-1}\text{A}$, cutting the number of summands from n to $n/2$. Then these $n/2$ numbers would be added in pairs again, reducing to $n/4$, and so on, and resulting in a single sum after log n steps. However, this simple binary tree idea doesn't map into full (3,2) adders, which reduce three inputs to two rather than reducing two inputs to one. A tree that does use full adders, known as a *Wallace tree*, is shown in Figure J.30. When computer arithmetic units were built out of

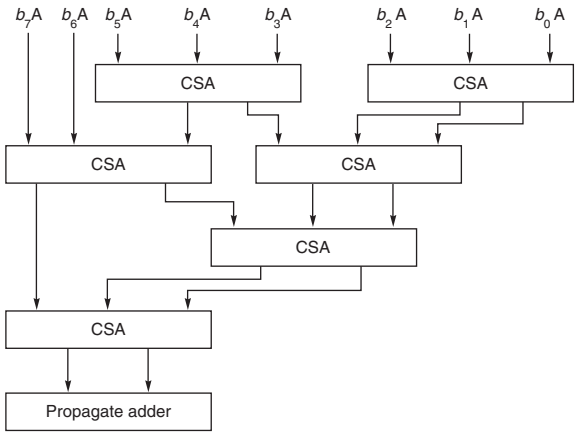


Figure J.30 Wallace tree multiplier. An example of a multiply tree that computes a product in 0(log *n*) steps.

MSI parts, a Wallace tree was the design of choice for high-speed multipliers. There is, however, a problem with implementing it in VLSI. If you try to fill in all the adders and paths for the Wallace tree of Figure J.30, you will discover that it does not have the nice, regular structure of Figure J.27. This is why VLSI designers have often chosen to use other log *n* designs such as the *binary tree multiplier*, which is discussed next.

The problem with adding summands in a binary tree is coming up with a (2,1) adder that combines two digits and produces a single-sum digit. Because of carries, this isn't possible using binary notation, but it can be done with some other representation. We will use the *signed-digit representation* 1, $\overline{1}$, and 0, which we used previously to understand Booth's algorithm. This representation has two costs. First, it takes 2 bits to represent each signed digit. Second, the algorithm for adding two signed-digit numbers a_i and b_i is complex and requires examining $a_i a_{i-1} a_{i-2}$ and $b_i b_{i-1} b_{i-2}$. Although this means you must look 2 bits back, in binary addition you might have to look an arbitrary number of bits back because of carries.

We can describe the algorithm for adding two signed-digit numbers as follows. First, compute sum and carry bits s_i and c_{i+1} using Figure J.31. Then compute the final sum as $s_i + c_i$. The tables are set up so that this final sum does not generate a carry.

Example What is the sum of the signed-digit numbers $1\overline{1}0_2$ and 001_2?

Answer The two low-order bits sum to $0 + 1 = 1\overline{1}$, the next pair sums to $\overline{1} + 0 = 0\overline{1}$, and the high-order pair sums to $1 + 0 = 01$, so the sum is $1\overline{1} + 0\overline{1}0 + 0100 = 10\overline{1}_2$.

$\begin{array}{r} 1 \\ +\,1 \\ \hline 1\;0 \end{array}$	$\begin{array}{r} 1 \\ +\,\overline{1} \\ \hline 0\;0 \end{array}$	$\begin{array}{r} \overline{1} \\ +\,\overline{1} \\ \hline \overline{1}\;0 \end{array}$	$\begin{array}{r} 0 \\ +\,0 \\ \hline 0\;0 \end{array}$	$\begin{array}{r} \overline{1}\;x \\ +\;0\;y \\ \hline 1\;\overline{1} \\ 0\;1 \end{array}$	if $x \geq 0$ and $y \geq 0$ otherwise	$\begin{array}{r} \overline{1}\;x \\ +\;0\;y \\ \hline 1\;\overline{1} \\ \overline{1}\;1 \end{array}$	if $x \geq 0$ and $y \geq 0$ otherwise

Figure J.31 Signed-digit addition table. The leftmost sum shows that when computing 1 + 1, the sum bit is 0 and the carry bit is 1.

This, then, defines a (2,1) adder. With this in hand, we can use a straightforward binary tree to perform multiplication. In the first step it adds $b_0\text{A} + b_1\text{A}$ in parallel with $b_2\text{A} + b_3\text{A}$, …, $b_{n-2}\text{A} + b_{n-1}\text{A}$. The next step adds the results of these sums in pairs, and so on. Although the final sum must be run through a carry-propagate adder to convert it from signed-digit form to two's complement, this final add step is necessary in any multiplier using CSAs.

To summarize, both Wallace trees and signed-digit trees are log n multipliers. The Wallace tree uses fewer gates but is harder to lay out. The signed-digit tree has a more regular structure, but requires 2 bits to represent each digit and has more complicated add logic. As with adders, it is possible to combine different multiply techniques. For example, Booth recoding and arrays can be combined. In Figure J.27 instead of having each input be $b_i\text{A}$, we could have it be $b_i b_{i-1}\text{A}$. To avoid having to compute the multiple $3b$, we can use Booth recoding.

Faster Division with One Adder

The two techniques we discussed for speeding up multiplication with a single adder were carry-save adders and higher-radix multiplication. However, there is a difficulty when trying to utilize these approaches to speed up nonrestoring division. If the adder in Figure J.2(b) on page J-4 is replaced with a carry-save adder, then P will be replaced with two registers, one for the sum bits and one for the carry bits (compare with the multiplier in Figure J.24). At the end of each cycle, the sign of P is uncertain (since P is the unevaluated sum of the two registers), yet it is the sign of P that is used to compute the quotient digit and decide the next ALU operation. When a higher radix is used, the problem is deciding what value to subtract from P. In the paper-and-pencil method, you have to guess the quotient digit. In binary division, there are only two possibilities. We were able to finesse the problem by initially guessing one and then adjusting the guess based on the sign of P. This doesn't work in higher radices because there are more than two possible quotient digits, rendering quotient selection potentially quite complicated: You would have to compute all the multiples of b and compare them to P.

Both the carry-save technique and higher-radix division can be made to work if we use a redundant quotient representation. Recall from our discussion of SRT division (page J-45) that by allowing the quotient digits to be −1, 0, or 1, there is often a choice of which one to pick. The idea in the previous algorithm was to choose 0 whenever possible, because that meant an ALU operation could be

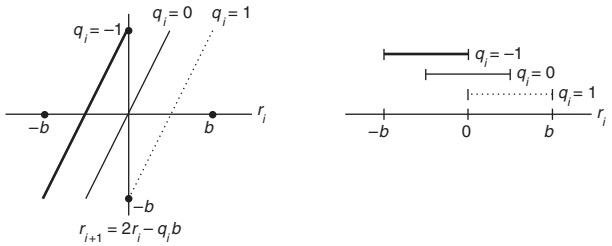


Figure J.32 Quotient selection for radix-2 division. The *x*-axis represents the *i*th remainder, which is the quantity in the (P,A) register pair. The *y*-axis shows the value of the remainder after one additional divide step. Each bar on the right-hand graph gives the range of r_i values for which it is permissible to select the associated value of q_i.

skipped. In carry-save division, the idea is that, because the remainder (which is the value of the (P,A) register pair) is not known exactly (being stored in carry-save form), the exact quotient digit is also not known. But, thanks to the redundant representation, the remainder doesn't have to be known precisely in order to pick a quotient digit. This is illustrated in Figure J.32, where the *x*-axis represents r_i, the remainder after i steps. The line labeled $q_i = 1$ shows the value that r_{i+1} would be if we chose $q_i = 1$, and similarly for the lines $q_i = 0$ and $q_i = -1$. We can choose any value for q_i, as long as $r_{i+1} = 2r_i - q_i b$ satisfies $|r_{i+1}| \leq b$. The allowable ranges are shown in the right half of Figure J.32. This shows that you don't need to know the precise value of r_i in order to choose a quotient digit q_i. You only need to know that r lies in an interval small enough to fit entirely within one of the overlapping bars shown in the right half of Figure J.32.

This is the basis for using carry-save adders. Look at the high-order bits of the carry-save adder and sum them in a propagate adder. Then use this approximation of r (together with the divisor, b) to compute q_i, usually by means of a lookup table. The same technique works for higher-radix division (whether or not a carry-save adder is used). The high-order bits P can be used to index a table that gives one of the allowable quotient digits.

The design challenge when building a high-speed SRT divider is figuring out how many bits of P and B need to be examined. For example, suppose that we take a radix of 4, use quotient digits of 2, 1, 0, $\overline{1}$, $\overline{2}$, but have a propagate adder. How many bits of P and B need to be examined? Deciding this involves two steps. For ordinary radix-2 nonrestoring division, because at each stage $|r| \leq b$, the P buffer won't overflow. But, for radix 4, $r_{i+1} = 4r_i - q_i b$ is computed at each stage, and if r_i is near b, then $4r_i$ will be near $4b$, and even the largest quotient digit will not bring r back to the range $|r_{i+1}| \leq b$. In other words, the remainder might grow without bound. However, restricting $|r_i| \leq 2b/3$ makes it easy to check that r_i will stay bounded.

After figuring out the bound that r_i must satisfy, we can draw the diagram in Figure J.33, which is analogous to Figure J.32. For example, the diagram shows

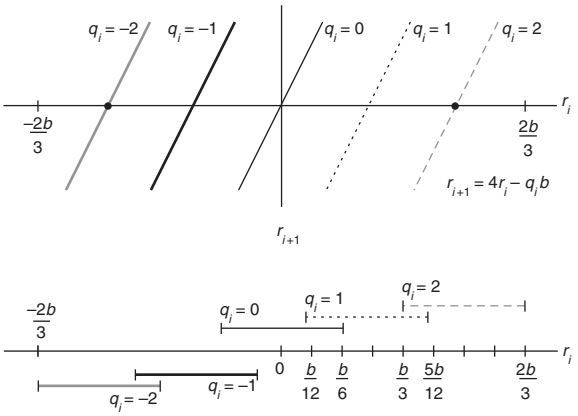


Figure J.33 Quotient selection for radix-4 division with quotient digits −2, −1, 0, 1, 2.

that if r_i is between (1/12)b and (5/12)b, we can pick $q = 1$, and so on. Or, to put it another way, if r/b is between 1/12 and 5/12, we can pick $q = 1$. Suppose the divider examines 5 bits of P (including the sign bit) and 4 bits of b (ignoring the sign, since it is always nonnegative). The interesting case is when the high bits of P are 00011$xxx\cdots$, while the high bits of b are 1001$xxx\cdots$. Imagine the binary point at the left end of each register. Since we truncated, r (the value of P concatenated with A) could have a value from 0.0011_2 to 0.0100_2, and b could have a value from $.1001_2$ to $.1010_2$. Thus, r/b could be as small as $0.0011_2/.1010_2$ or as large as $0.0100_2/.1001_2$, but $0.0011_2/.1010_2 = 3/10 < 1/3$ would require a quotient bit of 1, while $0.0100_2/.1001_2 = 4/9 > 5/12$ would require a quotient bit of 2. In other words, 5 bits of P and 4 bits of b aren't enough to pick a quotient bit. It turns out that 6 bits of P and 4 bits of b are enough. This can be verified by writing a simple program that checks all the cases. The output of such a program is shown in Figure J.34.

Example Using 8-bit registers, compute 149/5 using radix-4 SRT division.

Answer Follow the SRT algorithm on page J-45, but replace the quotient selection rule in step 2 with one that uses Figure J.34. See Figure J.35.

The Pentium uses a radix-4 SRT division algorithm like the one just presented, except that it uses a carry-save adder. Exercises J.34(c) and J.35 explore this in detail. Although these are simple cases, all SRT analyses proceed in the same way. First compute the range of r_i, then plot r_i against r_{i+1} to find the quotient ranges, and finally write a program to compute how many bits are necessary. (It is sometimes also possible to compute the required number of bits analytically.) Various details need to be considered in building a practical SRT divider.

b	Range of P		q	b	Range of P		q
8	−12	−7	−2	12	−18	−10	−2
8	−6	−3	−1	12	−10	−4	−1
8	−2	1	0	12	−4	3	0
8	2	5	1	12	3	9	1
8	6	11	2	12	9	17	2
9	−14	−8	−2	13	−19	−11	−2
9	−7	−3	−1	13	−10	−4	−1
9	−3	2	0	13	−4	3	0
9	2	6	1	13	3	9	1
9	7	13	2	13	10	18	2
10	−15	−9	−2	14	−20	−11	−2
10	−8	−3	−1	14	−11	−4	−1
10	−3	2	0	14	−4	3	0
10	2	7	1	14	3	10	1
10	8	14	2	14	10	19	2
11	−16	−9	−2	15	−22	−12	−2
11	−9	−3	−1	15	−12	−4	−1
11	−3	2	0	15	−5	4	0
11	2	8	1	15	3	11	1
11	8	15	2	15	11	21	2

Figure J.34 Quotient digits for radix-4 SRT division with a propagate adder. The top row says that if the high-order 4 bits of b are $1000_2 = 8$, and if the top 6 bits of P are between $110100_2 = -12$ and $111001_2 = -7$, then -2 is a valid quotient digit.

For example, the quotient lookup table has a fairly regular structure, which means it is usually cheaper to encode it as a PLA rather than in ROM. For more details about SRT division, see Burgess and Williams [1995].

J.10 Putting It All Together

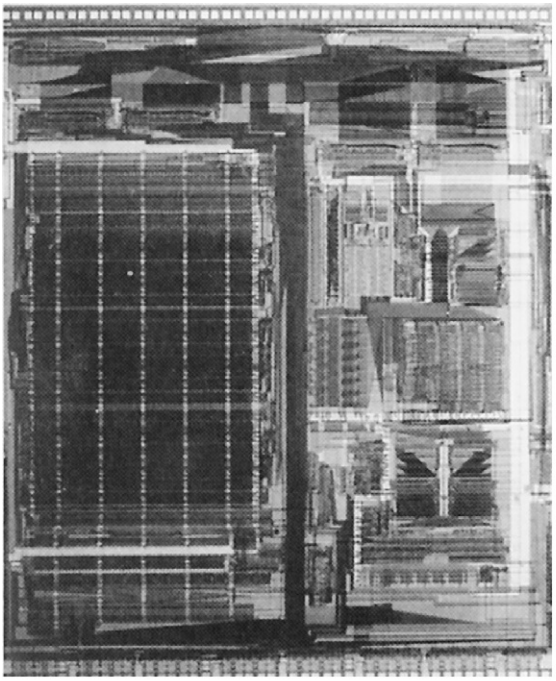
In this section, we will compare the Weitek 3364, the MIPS R3010, and the Texas Instruments 8847 (see Figures J.36 and J.37). In many ways, these are ideal chips to compare. They each implement the IEEE standard for addition, subtraction,

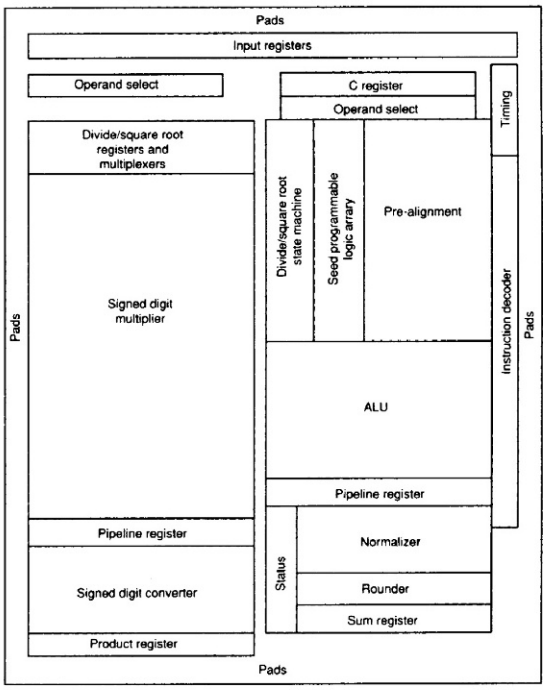
P	A		
000000000	10010101	Divide 149 by 5. B contains 00000101.	
000010010	10100000	Step 1:	B had 5 leading 0s, so shift left by 5. B now contains 10100000, so use $b=10$ section of table.
001001010	1000000	Step 2.1:	Top 6 bits of P are 2, so shift left by 2. From table, can pick q to be 0 or 1. Choose $q_0=0$.
100101010	000002	Step 2.2:	Top 6 bits of P are 9, so shift left 2. $q_1=2$.
+ 011000000			Subtract $2b$.
111101010	000002	Step 2.3:	Top bits $=-3$, so shift left 2. Can pick 0 or -1 for q, pick $q_2=0$.
110101000	00020		
010100000	0202	Step 2.4:	Top bits $=-11$, so shift left 2. $q_3=-2$.
+ 101000000			Add $2b$.
111100000		Step 3:	Remainder is negative, so restore by adding b and subtract 1 from q.
+ 010100000			
010000000		Answer:	$q=020\overline{2}-1=29$
			To get remainder, undo shift in step 1 so remainder $=010000000>>5=4$.

Figure J.35 Example of radix-4 SRT division. Division of 149 by 5.

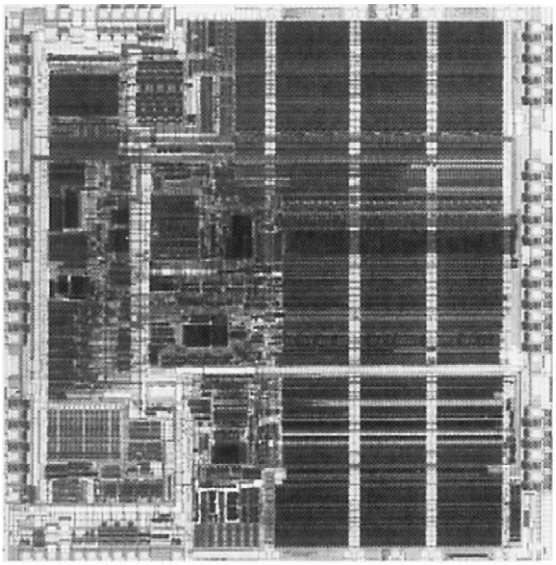
Features	MIPS R3010	Weitek 3364	TI 8847
Clock cycle time (ns)	40	50	30
Size (mil^2)	114,857	147,600	156,180
Transistors	75,000	165,000	180,000
Pins	84	168	207
Power (watts)	3.5	1.5	1.5
Cycles/add	2	2	2
Cycles/mult	5	2	3
Cycles/divide	19	17	11
Cycles/square root	–	30	14

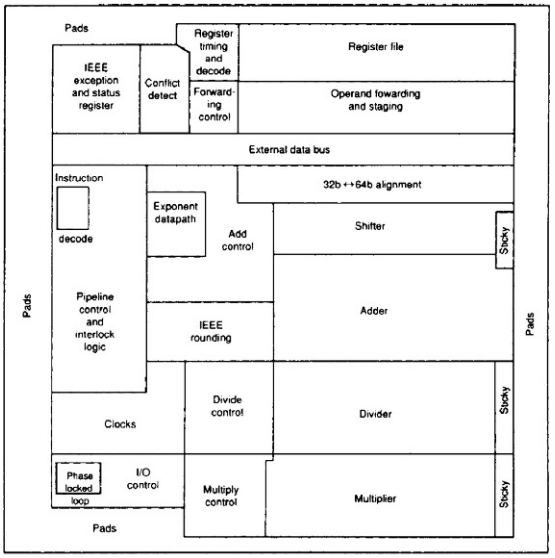
Figure J.36 Summary of the three floating-point chips discussed in this section. The cycle times are for production parts available in June 1989. The cycle counts are for double-precision operations.





TI 8847





MIPS R3010

Figure J.37 Chip layout for the TI 8847, MIPS R3010, and Weitek 3364. In the left-hand columns are the photomicrographs; the right-hand columns show the corresponding floor plans.

(Continued)

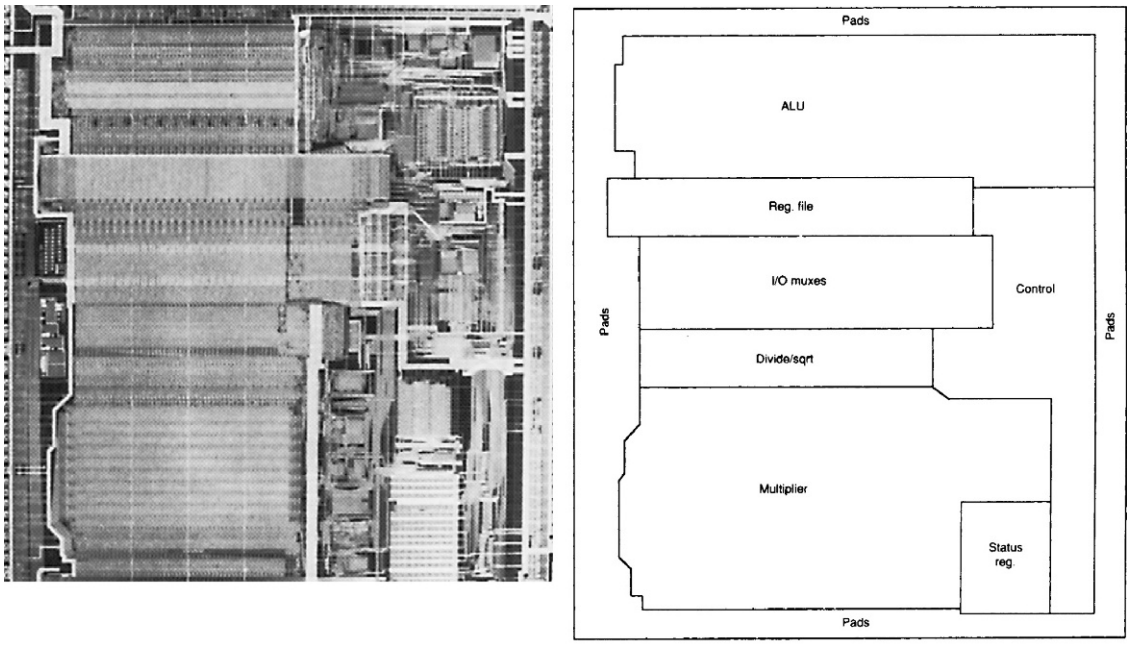


Figure J.37 *(Continued)*

multiplication, and division on a single chip. All were introduced in 1988 and run with a cycle time of about 40 nanoseconds. However, as we will see, they use quite different algorithms. The Weitek chip is well described in Birman et al. [1990], the MIPS chip is described in less detail in Rowen, Johnson, and Ries [1988], and details of the TI chip can be found in Darley et al. [1989].

These three chips have a number of things in common. They perform addition and multiplication in parallel, and they implement neither extended precision nor a remainder step operation. (Recall from Section J.6 that it is easy to implement the IEEE remainder function in software if a remainder step instruction is available.) The designers of these chips probably decided not to provide extended precision because the most influential users are those who run portable codes, which can't rely on extended precision. However, as we have seen, extended precision can make for faster and simpler math libraries.

In the summary of the three chips given in Figure J.36, note that a higher transistor count generally leads to smaller cycle counts. Comparing the cycles/op numbers needs to be done carefully, because the figures for the MIPS chip are those for a complete system (R3000/3010 pair), while the Weitek and TI numbers are for stand-alone chips and are usually larger when used in a complete system.

The MIPS chip has the fewest transistors of the three. This is reflected in the fact that it is the only chip of the three that does not have any pipelining or hardware

square root. Further, the multiplication and addition operations are not completely independent because they share the carry-propagate adder that performs the final rounding (as well as the rounding logic).

Addition on the R3010 uses a mixture of ripple, CLA, and carry-select. A carry-select adder is used in the fashion of Figure J.20 (page J-43). Within each half, carries are propagated using a hybrid ripple-CLA scheme of the type indicated in Figure J.19 (page J-42). However, this is further tuned by varying the size of each block, rather than having each fixed at 4 bits (as they are in Figure J.19). The multiplier is midway between the designs of Figures J.2 (page J-4) and J.27 (page J-50). It has an array just large enough so that output can be fed back into the input without having to be clocked. Also, it uses radix-4 Booth recoding and the even/odd technique of Figure J.29 (page J-52). The R3010 can do a divide and multiply in parallel (like the Weitek chip but unlike the TI chip). The divider is a radix-4 SRT method with quotient digits -2, -1, 0, 1, and 2, and is similar to that described in Taylor [1985]. Double-precision division is about four times slower than multiplication. The R3010 shows that for chips using an $0(n)$ multiplier, an SRT divider can operate fast enough to keep a reasonable ratio between multiply and divide.

The Weitek 3364 has independent add, multiply, and divide units. It also uses radix-4 SRT division. However, the add and multiply operations on the Weitek chip are pipelined. The three addition stages are (1) exponent compare, (2) add followed by shift (or *vice versa*), and (3) final rounding. Stages (1) and (3) take only a half-cycle, allowing the whole operation to be done in two cycles, even though there are three pipeline stages. The multiplier uses an array of the style of Figure J.28 but uses radix-8 Booth recoding, which means it must compute 3 times the multiplier. The three multiplier pipeline stages are (1) compute $3b$, (2) pass through array, and (3) final carry-propagation add and round. Single precision passes through the array once, double precision twice. Like addition, the latency is two cycles.

The Weitek chip uses an interesting addition algorithm. It is a variant on the carry-skip adder pictured in Figure J.18 (page J-42). However, P_{ij}, which is the logical AND of many terms, is computed by rippling, performing one AND per ripple. Thus, while the carries propagate left within a block, the value of P_{ij} is propagating right within the next block, and the block sizes are chosen so that both waves complete at the same time. Unlike the MIPS chip, the 3364 has hardware square root, which shares the divide hardware. The ratio of double-precision multiply to divide is 2:17. The large disparity between multiply and divide is due to the fact that multiplication uses radix-8 Booth recoding, while division uses a radix-4 method. In the MIPS R3010, multiplication and division use the same radix.

The notable feature of the TI 8847 is that it does division by iteration (using the Goldschmidt algorithm discussed in Section J.6). This improves the speed of division (the ratio of multiply to divide is 3:11), but means that multiplication and division cannot be done in parallel as on the other two chips. Addition has a two-stage pipeline. Exponent compare, fraction shift, and fraction addition are done in the first stage, normalization and rounding in the second stage. Multiplication uses

a binary tree of signed-digit adders and has a three-stage pipeline. The first stage passes through the array, retiring half the bits; the second stage passes through the array a second time; and the third stage converts from signed-digit form to two's complement. Since there is only one array, a new multiply operation can only be initiated in every other cycle. However, by slowing down the clock, two passes through the array can be made in a single cycle. In this case, a new multiplication can be initiated in each cycle. The 8847 adder uses a carry-select algorithm rather than carry-lookahead. As mentioned in Section J.6, the TI carries 60 bits of precision in order to do correctly rounded division.

These three chips illustrate the different trade-offs made by designers with similar constraints. One of the most interesting things about these chips is the diversity of their algorithms. Each uses a different add algorithm, as well as a different multiply algorithm. In fact, Booth recoding is the only technique that is universally used by all the chips.

J.11 Fallacies and Pitfalls

Fallacy *Underflows rarely occur in actual floating-point application code*

Although most codes rarely underflow, there are actual codes that underflow frequently. SDRWAVE [Kahaner 1988], which solves a one-dimensional wave equation, is one such example. This program underflows quite frequently, even when functioning properly. Measurements on one machine show that adding hardware support for gradual underflow would cause SDRWAVE to run about 50% faster.

Fallacy *Conversions between integer and floating point are rare*

In fact, in spice they are as frequent as divides. The assumption that conversions are rare leads to a mistake in the SPARC version 8 instruction set, which does not provide an instruction to move from integer registers to floating-point registers.

Pitfall *Don't increase the speed of a floating-point unit without increasing its memory bandwidth*

A typical use of a floating-point unit is to add two vectors to produce a third vector. If these vectors consist of double-precision numbers, then each floating-point add will use three operands of 64 bits each, or 24 bytes of memory. The memory bandwidth requirements are even greater if the floating-point unit can perform addition and multiplication in parallel (as most do).

Pitfall $-x$ *is not the same as* $0-x$

This is a fine point in the IEEE standard that has tripped up some designers. Because floating-point numbers use the sign magnitude system, there are two zeros, +0 and −0. The standard says that $0-0=+0$, whereas $-(0)=-0$. Thus, $-x$ is not the same as $0-x$ when $x=0$.

J.12 Historical Perspective and References

The earliest computers used fixed point rather than floating point. In "Preliminary Discussion of the Logical Design of an Electronic Computing Instrument," Burks, Goldstine, and von Neumann [1946] put it like this:

There appear to be two major purposes in a "floating" decimal point system both of which arise from the fact that the number of digits in a word is a constant fixed by design considerations for each particular machine. The first of these purposes is to retain in a sum or product as many significant digits as possible and the second of these is to free the human operator from the burden of estimating and inserting into a problem "scale factors"—multiplicative constants which serve to keep numbers within the limits of the machine.

There is, of course, no denying the fact that human time is consumed in arranging for the introduction of suitable scale factors. We only argue that the time so consumed is a very small percentage of the total time we will spend in preparing an interesting problem for our machine. The first advantage of the floating point is, we feel, somewhat illusory. In order to have such a floating point, one must waste memory capacity that could otherwise be used for carrying more digits per word. It would therefore seem to us not at all clear whether the modest advantages of a floating binary point offset the loss of memory capacity and the increased complexity of the arithmetic and control circuits.

This enables us to see things from the perspective of early computer designers, who believed that saving computer time and memory were more important than saving programmer time.

The original papers introducing the Wallace tree, Booth recoding, SRT division, overlapped triplets, and so on are reprinted in Swartzlander [1990]. A good explanation of an early machine (the IBM 360/91) that used a pipelined Wallace tree, Booth recoding, and iterative division is in Anderson et al. [1967]. A discussion of the average time for single-bit SRT division is in Freiman [1961]; this is one of the few interesting historical papers that does not appear in Swartzlander.

The standard book of Mead and Conway [1980] discouraged the use of CLAs as not being cost effective in VLSI. The important paper by Brent and Kung [1982] helped combat that view. An example of a detailed layout for CLAs can be found in Ngai and Irwin [1985] or in Weste and Eshraghian [1993], and a more theoretical treatment is given by Leighton [1992]. Takagi, Yasuura, and Yajima [1985] provide a detailed description of a signed-digit tree multiplier.

Before the ascendancy of IEEE arithmetic, many different floating-point formats were in use. Three important ones were used by the IBM 370, the DEC VAX, and the Cray. Here is a brief summary of these older formats. The VAX format is closest to the IEEE standard. Its single-precision format (F format) is like IEEE single precision in that it has a hidden bit, 8 bits of exponent, and 23 bits of fraction. However, it does not have a sticky bit, which causes it to round halfway cases up instead of to even. The VAX has a slightly different exponent range from IEEE

single: $E_{\min}$ is -128 rather than -126 as in IEEE, and $E_{\max}$ is 126 instead of 127. The main differences between VAX and IEEE are the lack of special values and gradual underflow. The VAX has a reserved operand, but it works like a signaling NaN: It traps whenever it is referenced. Originally, the VAX's double precision (D format) also had 8 bits of exponent. However, as this is too small for many applications, a G format was added; like the IEEE standard, this format has 11 bits of exponent. The VAX also has an H format, which is 128 bits long.

The IBM 370 floating-point format uses base 16 rather than base 2. This means it cannot use a hidden bit. In single precision, it has 7 bits of exponent and 24 bits (6 hex digits) of fraction. Thus, the largest representable number is $16^{27} = 2^4 \times 2^7 = 2^{29}$, compared with 2^{28} for IEEE. However, a number that is normalized in the hexadecimal sense only needs to have a nonzero leading digit. When interpreted in binary, the three most-significant bits could be zero. Thus, there are potentially fewer than 24 bits of significance. The reason for using the higher base was to minimize the amount of shifting required when adding floating-point numbers. However, this is less significant in current machines, where the floating-point add time is usually fixed independently of the operands. Another difference between 370 arithmetic and IEEE arithmetic is that the 370 has neither a round digit nor a sticky digit, which effectively means that it truncates rather than rounds. Thus, in many computations, the result will systematically be too small. Unlike the VAX and IEEE arithmetic, every bit pattern is a valid number. Thus, library routines must establish conventions for what to return in case of errors. In the IBM FORTRAN library, for example, $\sqrt{-4}$ returns 2!

Arithmetic on Cray computers is interesting because it is driven by a motivation for the highest possible floating-point performance. It has a 15-bit exponent field and a 48-bit fraction field. Addition on Cray computers does not have a guard digit, and multiplication is even less accurate than addition. Thinking of multiplication as a sum of p numbers, each $2p$ bits long, Cray computers drop the low-order bits of each summand. Thus, analyzing the exact error characteristics of the multiply operation is not easy. Reciprocals are computed using iteration, and division of a by b is done by multiplying a times $1/b$. The errors in multiplication and reciprocation combine to make the last three bits of a divide operation unreliable. At least Cray computers serve to keep numerical analysts on their toes!

The IEEE standardization process began in 1977, inspired mainly by W. Kahan and based partly on Kahan's work with the IBM 7094 at the University of Toronto [Kahan 1968]. The standardization process was a lengthy affair, with gradual underflow causing the most controversy. (According to Cleve Moler, visitors to the United States were advised that the sights not to be missed were Las Vegas, the Grand Canyon, and the IEEE standards committee meeting.) The standard was finally approved in 1985. The Intel 8087 was the first major commercial IEEE implementation and appeared in 1981, before the standard was finalized. It contains features that were eliminated in the final standard, such as projective bits. According to Kahan, the length of double-extended precision was based on what could be implemented in the 8087. Although the IEEE standard was not based on any existing floating-point system, most of its features were present in some other system. For example, the CDC 6600 reserved special bit patterns for INDEFINITE

and INFINITY, while the idea of denormal numbers appears in Goldberg [1967] as well as in Kahan [1968]. Kahan was awarded the 1989 Turing prize in recognition of his work on floating point.

Although floating point rarely attracts the interest of the general press, newspapers were filled with stories about floating-point division in November 1994. A bug in the division algorithm used on all of Intel's Pentium chips had just come to light. It was discovered by Thomas Nicely, a math professor at Lynchburg College in Virginia. Nicely found the bug when doing calculations involving reciprocals of prime numbers. News of Nicely's discovery first appeared in the press on the front page of the November 7 issue of *Electronic Engineering Times.* Intel's immediate response was to stonewall, asserting that the bug would only affect theoretical mathematicians. Intel told the press, "This doesn't even qualify as an errata … even if you're an engineer, you're not going to see this."

Under more pressure, Intel issued a white paper, dated November 30, explaining why they didn't think the bug was significant. One of their arguments was based on the fact that if you pick two floating-point numbers at random and divide one into the other, the chance that the resulting quotient will be in error is about 1 in 9 billion. However, Intel neglected to explain why they thought that the typical customer accessed floating-point numbers randomly.

Pressure continued to mount on Intel. One sore point was that Intel had known about the bug before Nicely discovered it, but had decided not to make it public. Finally, on December 20, Intel announced that they would unconditionally replace any Pentium chip that used the faulty algorithm and that they would take an unspecified charge against earnings, which turned out to be $300 million.

The Pentium uses a simple version of SRT division as discussed in Section J.9. The bug was introduced when they converted the quotient lookup table to a PLA. Evidently there were a few elements of the table containing the quotient digit 2 that Intel thought would never be accessed, and they optimized the PLA design using this assumption. The resulting PLA returned 0 rather than 2 in these situations. However, those entries were really accessed, and this caused the division bug. Even though the effect of the faulty PLA was to cause 5 out of 2048 table entries to be wrong, the Pentium only computes an incorrect quotient 1 out of 9 billion times on random inputs. This is explored in Exercise J.34.

References

Anderson, S.F., Earle, J.G., Goldschmidt, R.E., Powers, D.M., 1967. The IBM System/360 Model 91: Floating-point execution unit. IBM J. Research and Development 11, 34–53. Reprinted in Swartzlander [1990]. *Good description of an early high-performance floating-point unit that used a pipelined Wallace tree multiplier and iterative division.*

Bell, C.G., Newell, A., 1971. Computer Structures: Readings and Examples. McGraw-Hill, New York.

Birman, M., Samuels, A., Chu, G., Chuk, T., Hu, L., McLeod, J., Barnes, J., 1990. Developing the WRL3170/3171 SPARC floating-point coprocessors. IEEE Micro 10 (1), 55–64. *These chips have the same floating-point core as the Weitek 3364, and this paper has a fairly detailed description of that floating-point design.*

Brent, R.P., Kung, H.T., 1982. A regular layout for parallel adders. IEEE Trans. on Computers C-31, 260–264. *This is the paper that popularized CLAs in VLSI.*

Burgess, N., Williams, T., 1995. Choices of operand truncation in the SRT division algorithm. IEEE Trans. on Computers 44, 7. *Analyzes how many bits of divisor and remainder need to be examined in SRT division.*

Burks, A.W., Goldstine, H.H., von Neumann, J., 1946. Preliminary discussion of the logical design of an electronic computing instrument. In: Aspray, W., Burks, A. (Eds.), Papers of John von Neumann. Report to the U.S. Army Ordnance Department, p. 1; also appears. MIT Press, Cambridge, Mass, pp. 97–146. Tomash Publishers, Los Angeles, 1987.

Cody, W.J., Coonen, J.T., Gay, D.M., Hanson, K., Hough, D., Kahan, W., Karpinski, R., Palmer, J., Ris, F.N., Stevenson, D., 1984. A proposed radix- and word-length-independent standard for floating-point arithmetic. IEEE Micro 4 (4), 86–100. *Contains a draft of the 854 standard, which is more general than 754. The significance of this article is that it contains commentary on the standard, most of which is equally relevant to 754. However, be aware that there are some differences between this draft and the final standard.*

Coonen, J., 1984. Contributions to a proposed standard for binary floating point arithmetic. Ph.D. thesis. University of California–Berkeley. *The only detailed discussion of how rounding modes can be used to implement efficient binary decimal conversion.*

Darley, H.M., et al., 1989. Floating point/integer processor with divide and square root functions. U.S. Patent. 4 (878,190) October 31, 1989. *Pretty readable as patents go. Gives a high-level view of the TI 8847 chip, but doesn't have all the details of the division algorithm.*

Demmel, J.W., Li, X., 1994. Faster numerical algorithms via exception handling. IEEE Trans. on Computers 43 (8), 983–992. *A good discussion of how the features unique to IEEE floating point can improve the performance of an important software library.*

Freiman, C.V., 1961. Statistical analysis of certain binary division algorithms. Proc. IRE 49 (1), 91–103. *Contains an analysis of the performance of shifting-over-zeros SRT division algorithm.*

Goldberg, D., 1991. What every computer scientist should know about floating-point arithmetic. Computing Surveys 23 (1), 5–48. *Contains an in-depth tutorial on the IEEE standard from the software point of view.*

Goldberg, I.B., 1967. 27 bits are not enough for 8-digit accuracy. Comm. ACM 10 (2), 105–106. *This paper proposes using hidden bits and gradual underflow.*

Gosling, J.B., 1980. Design of Arithmetic Units for Digital Computers. Springer-Verlag, New York. *A concise, well-written book, although it focuses on MSI designs.*

Hamacher, V.C., Vranesic, Z.G., Zaky, S.G., 1984. Computer Organization, 2nd ed. McGraw-Hill, New York. *Introductory computer architecture book with a good chapter on computer arithmetic.*

Hwang, K., 1979. Computer Arithmetic: Principles, Architecture, and Design. Wiley, New York. *This book contains the widest range of topics of the computer arithmetic books.*

IEEE, 1985. IEEE standard for binary floating-point arithmetic. SIGPLAN Notices 22 (2), 9–25. *IEEE 754 is reprinted here.*

Kahan, W., 1968. 7094-II system support for numerical analysis. SHARE Secretarial Distribution. SSD-159. *This system had many features that were incorporated into the IEEE floating-point standard.*

Kahaner, D.K., 1988. Benchmarks for 'real' programs. SIAM News.(November).. *The benchmark presented in this article turns out to cause many underflows.*

Knuth, D., 1981. 2nd ed. The Art of Computer Programming.Vol. II. Addison-Wesley, Reading, Mass. *Has a section on the distribution of floating-point numbers.*

Kogge, P., 1981. The Architecture of Pipelined Computers. McGraw-Hill, New York. *Has a brief discussion of pipelined multipliers.*

Kohn, L., Fu, S.-W., 1989. A 1,000,000 transistor microprocessor. In: IEEE Int'l. Solid-State Circuits Conf. Digest of Technical Papers, pp. 54–55. *There are several articles about the i860, but this one contains the most details about its floating-point algorithms.*

Koren, I., 1989. Computer Arithmetic Algorithms. Prentice Hall, Englewood Cliffs, N.J..

Leighton, F.T., 1992. Introduction to Parallel Algorithms and Architectures: Arrays. Trees, Hypercubes, Morgan Kaufmann, San Francisco. *This is an excellent book, with emphasis on the complexity analysis of algorithms. Section 1.2.1 has a nice discussion of carry-lookahead addition on a tree.*

Magenheimer, D.J., Peters, L., Pettis, K.W., Zuras, D., 1988. Integer multiplication and division on the HP Precision architecture. IEEE Trans. on Computers 37 (8), 980–990. *Gives rationale for the integer- and divide-step instructions in the Precision architecture.*

Markstein, P.W., 1990. Computation of elementary functions on the IBM RISC System/6000 processor. IBM J. of Research and Development 34 (1), 111–119. *Explains how to use fused muliply-add to compute correctly rounded division and square root.*

Mead, C., Conway, L., 1980. Introduction to VLSI Systems. Addison-Wesley, Reading, Mass.

Montoye, R.K., Hokenek, E., Runyon, S.L., 1990. Design of the IBM RISC System/6000 floating-point execution. IBM J. of Research and Development 34 (1), 59–70. *Describes one implementation of fused multiply-add.*

Ngai, T.-F., Irwin, M.J., 1985. Regular, area-time efficient carry-lookahead adders. In: Proc. Seventh IEEE Symposium on Computer Arithmetic, pp. 9–15. *Describes a CLA like that of Figure J.17, where the bits flow up and then come back down.*

Patterson, D.A., Hennessy, J.L., 2009. Computer Organization and Design: The Hardware/Software Interface, 4th Edition Morgan Kaufmann, San Francisco. *Chapter 3 is a gentler introduction to the first third of this appendix.*

Peng, V., Samudrala, S., Gavrielov, M., 1987. On the implementation of shifters, multipliers, and dividers in VLSI floating point units. In: Proc. Eighth IEEE Symposium on Computer Arithmetic, pp. 95–102. *Highly recommended survey of different techniques actually used in VLSI designs.*

Rowen, C., Johnson, M., Ries, P., 1988. The MIPS R3010 floating-point coprocessor. IEEE Micro 53–62 (June).

Santoro, M.R., Bewick, G., Horowitz, M.A., 1989. Rounding algorithms for IEEE multipliers. In: Proc. Ninth IEEE Symposium on Computer Arithmetic, pp. 176–183. *A very readable discussion of how to efficiently implement rounding for floating-point multiplication.*

Scott, N.R., 1985. Computer Number Systems and Arithmetic. Prentice Hall, Englewood Cliffs, N.J.

Swartzlander, E. (Ed.), 1990. Computer Arithmetic. IEEE Computer Society Press, Los Alamitos, Calif. *A collection of historical papers in two volumes.*

Takagi, N., Yasuura, H., Yajima, S., 1985. High-speed VLSI multiplication algorithm with a redundant binary addition tree. IEEE Trans. on Computers C-34 (9), 789–796. *A discussion of the binary tree signed multiplier that was the basis for the design used in the TI 8847.*

Taylor, G.S., 1981. Compatible hardware for division and square root. In: Proc. Fifth IEEE Symposium on Computer Arithmetic, May 18–19, 1981. Ann Arbor, Mich, pp. 127–134. *Good discussion of a radix-4 SRT division algorithm.*

Taylor, G.S., 1985. Radix 16 SRT dividers with overlapped quotient selection stages. In: Proc. Seventh IEEE Symposium on Computer Arithmetic, June 4–6, 1985, pp. 64–71 Urbana, Ill.. *Describes a very sophisticated high-radix division algorithm.*

Weste, N., Eshraghian, K., 1993. Principles of CMOS VLSI Design: A Systems Perspective, 2nd ed. Addison-Wesley, Reading, Mass. *This textbook has a section on the layouts of various kinds of adders.*

Williams, T.E., Horowitz, M., Alverson, R.L., Yang, T.S., 1987. A self-timed chip for division. In: Advanced Research in VLSI, Proc. 1987 Stanford Conf. MIT Press, Cambridge, Mass. *Describes a divider that tries to get the speed of a combinational design without using the area that would be required by one.*

Exercises

J.1 [12] <J.2> Using n bits, what is the largest and smallest integer that can be represented in the two's complement system?

J.2 [20/25] <J.2> In the subsection "Signed Numbers" (page J-7), it was stated that two's complement overflows when the carry into the high-order bit position is different from the carry-out from that position.

a. [20] <J.2> Give examples of pairs of integers for all four combinations of carry-in and carry-out. Verify the rule stated above.

b. [25] <J.2> Explain why the rule is always true.

J.3 [12] <J.2> Using 4-bit binary numbers, multiply -8×-8 using Booth recoding.

J.4 [15] <J.2> Equations J.2.1 and J.2.2 are for adding two n-bit numbers. Derive similar equations for subtraction, where there will be a borrow instead of a carry.

J.5 [25] <J.2> On a machine that doesn't detect integer overflow in hardware, show how you would detect overflow on a signed addition operation in software.

J.6 [15/15/20] <J.3> Represent the following numbers as single-precision and double-precision IEEE floating-point numbers:

a. [15] <J.3> 10.

b. [15] <J.3> 10.5.

c. [20] <J.3> 0.1.

J.7 [12/12/12/12/12] <J.3> Below is a list of floating-point numbers. In single precision, write down each number in binary, in decimal, and give its representation in IEEE arithmetic.

a. [12] <J.3> The largest number less than 1.

b. [12] <J.3> The largest number.

c. [12] <J.3> The smallest positive normalized number.

d. [12] <J.3> The largest denormal number.

e. [12] <J.3> The smallest positive number.

J.8 [15] <J.3> Is the ordering of nonnegative floating-point numbers the same as integers when denormalized numbers are also considered?

J.9 [20] <J.3> Write a program that prints out the bit patterns used to represent floating-point numbers on your favorite computer. What bit pattern is used for NaN?

J.10 [15] <J.4> Using $p = 4$, show how the binary floating-point multiply algorithm computes the product of 1.875×1.875.

J.11 [12/10] <J.4> Concerning the addition of exponents in floating-point multiply:

a. [12] <J.4> What would the hardware that implements the addition of exponents look like?

b. [10] <J.4> If the bias in single precision were 129 instead of 127, would addition be harder or easier to implement?

J.12 [15/12] <J.4> In the discussion of overflow detection for floating-point multiplication, it was stated that (for single precision) you can detect an overflowed exponent by performing exponent addition in a 9-bit adder.

a. [15] <J.4> Give the exact rule for detecting overflow.

b. [12] <J.4> Would overflow detection be any easier if you used a 10-bit adder instead?

J.13 [15/10] <J.4> Floating-point multiplication:

a. [15] <J.4> Construct two single-precision floating-point numbers whose product doesn't overflow until the final rounding step.

b. [10] <J.4> Is there any rounding mode where this phenomenon cannot occur?

J.14 [15] <J.4> Give an example of a product with a denormal operand but a normalized output. How large was the final shifting step? What is the maximum possible shift that can occur when the inputs are double-precision numbers?

J.15 [15] <J.5> Use the floating-point addition algorithm on page J-23 to compute $1.010_2 - .1001_2$ (in 4-bit precision).

J.16 [10/15/20/20/20] <J.5> In certain situations, you can be sure that $a+b$ is exactly representable as a floating-point number, that is, no rounding is necessary.

a. [10] <J.5> If a, b have the same exponent and different signs, explain why $a+b$ is exact. This was used in the subsection "Speeding Up Addition" on page J-25.

b. [15] <J.5> Give an example where the exponents differ by 1, a and b have different signs, and $a+b$ is not exact.

c. [20] <J.5> If $a \geq b \geq 0$, and the top two bits of a cancel when computing $a-b$, explain why the result is exact (this fact is mentioned on page J-22).

d. [20] <J.5> If $a \geq b \geq 0$, and the exponents differ by 1, show that $a-b$ is exact unless the high order bit of $a-b$ is in the same position as that of a (mentioned in "Speeding Up Addition," page J-25).

e. [20] <J.5> If the result of $a-b$ or $a+b$ is denormal, show that the result is exact (mentioned in the subsection "Underflow," on page J-36).

J.17 [15/20] <J.5> Fast floating-point addition (using parallel adders) for $p=5$.

a. [15] <J.5> Step through the fast addition algorithm for $a+b$, where $a=1.0111_2$ and $b=.11011_2$.

b. [20] <J.5> Suppose the rounding mode is toward $+\infty$. What complication arises in the above example for the adder that assumes a carry-out? Suggest a solution.

J.18 [12] <J.4, J.5> How would you use two parallel adders to avoid the final round-up addition in floating-point multiplication?

J.19 [30/10] <J.5> This problem presents a way to reduce the number of addition steps in floating-point addition from three to two using only a single adder.

a. [30] <J.5> Let A and B be integers of opposite signs, with a and b their magnitudes. Show that the following rules for manipulating the unsigned numbers a and b gives $A+B$.

1. Complement one of the operands.
2. Use end-around carry to add the complemented operand and the other (uncomplemented) one.
3. If there was a carry-out, the sign of the result is the sign associated with the uncomplemented operand.
4. Otherwise, if there was no carry-out, complement the result, and give it the sign of the complemented operand.

b. [10] < J.5 > Use the above to show how steps 2 and 4 in the floating-point addition algorithm on page J-23 can be performed using only a single addition.

J.20 [20/15/20/15/20/15] < J.6 > Iterative square root.

a. [20] < J.6 > Use Newton's method to derive an iterative algorithm for square root. The formula will involve a division.

b. [15] < J.6 > What is the fastest way you can think of to divide a floating-point number by 2?

c. [20] < J.6 > If division is slow, then the iterative square root routine will also be slow. Use Newton's method on $f(x) = 1/x^2 - a$ to derive a method that doesn't use any divisions.

d. [15] < J.6 > Assume that the ratio division by 2 : floating-point add : floating-point multiply is 1:2:4. What ratios of multiplication time to divide time makes each iteration step in the method of part (c) faster than each iteration in the method of part (a)?

e. [20] < J.6 > When using the method of part (a), how many bits need to be in the initial guess in order to get double-precision accuracy after three iterations? (You may ignore rounding error.)

f. [15] < J.6 > Suppose that when spice runs on the TI 8847, it spends 16.7% of its time in the square root routine (this percentage has been measured on other machines). Using the values in Figure J.36 and assuming three iterations, how much slower would spice run if square root were implemented in software using the method of part(a)?

J.21 [10/20/15/15/15] < J.6 > Correctly rounded iterative division. Let a and b be floating-point numbers with p-bit significands ($p = 53$ in double precision). Let q be the exact quotient $q = a/b$, $1 \le q < 2$. Suppose that $\overline{q}$ is the result of an iteration process, that $\overline{q}$ has a few extra bits of precision, and that $0 < q - \overline{q} < 2^{-p}$. For the following, it is important that $\overline{q} < q$, even when q can be exactly represented as a floating-point number.

a. [10] < J.6 > If x is a floating-point number, and $1 \le x < 2$, what is the next representable number after x?

b. [20] < J.6 > Show how to compute q' from $\overline{q}$, where q' has $p + 1$ bits of precision and $|q - q'| < 2^{-p}$.

c. [15] < J.6 > Assuming round to nearest, show that the correctly rounded quotient is either q', $q' - 2^{-p}$, or $q' + 2^{-p}$.

d. [15] <J.6> Give rules for computing the correctly rounded quotient from q' based on the low-order bit of q' and the sign of $a - bq'$.

e. [15] <J.6> Solve part (c) for the other three rounding modes.

J.22 [15] <J.6> Verify the formula on page J-30. (*Hint*: If $x_n = x_0(2 - x_0 b) \times \Pi_{i=1,n}\left[1 + (1 - x_0 b)^{2^i}\right]$, then $2 - x_n b = 2 - x_0 b(2 - x_0 b)\Pi\left[1 + (1 - x_0 b)^{2^i}\right] = 2 - \left[1 - (1 - x_0 b)^2\right]\Pi\left[1 + (1 - x_0 b)^{2^i}\right]$.)

J.23 [15] <J.7> Our example that showed that double rounding can give a different answer from rounding once used the round-to-even rule. If halfway cases are always rounded up, is double rounding still dangerous?

J.24 [10/10/20/20] <J.7> Some of the cases of the italicized statement in the "Precisions" subsection (page J-33) aren't hard to demonstrate.

a. [10] <J.7> What form must a binary number have if rounding to q bits followed by rounding to p bits gives a different answer than rounding directly to p bits?

b. [10] <J.7> Show that for multiplication of p-bit numbers, rounding to q bits followed by rounding to p bits is the same as rounding immediately to p bits if $q \geq 2p$.

c. [20] <J.7> If a and b are p-bit numbers with the same sign, show that rounding $a + b$ to q bits followed by rounding to p bits is the same as rounding immediately to p bits if $q \geq 2p + 1$.

d. [20] <J.7> Do part (c) when a and b have opposite signs.

J.25 [Discussion] <J.7> In the MIPS approach to exception handling, you need a test for determining whether two floating-point operands could cause an exception. This should be fast and also not have too many false positives. Can you come up with a practical test? The performance cost of your design will depend on the distribution of floating-point numbers. This is discussed in Knuth [1981] and the Hamming paper in Swartzlander [1990].

J.26 [12/12/10] <J.8> Carry-skip adders.

a. [12] <J.8> Assuming that time is proportional to logic levels, how long does it take an n-bit adder divided into (fixed) blocks of length k bits to perform an addition?

b. [12] <J.8> What value of k gives the fastest adder?

c. [10] <J.8> Explain why the carry-skip adder takes time $0(\sqrt{n})$.

J.27 [10/15/20] <J.8> Complete the details of the block diagrams for the following adders.

a. [10] <J.8> In Figure J.15, show how to implement the "1" and "2" boxes in terms of AND and OR gates.

b. [15] <J.8> In Figure J.19, what signals need to flow from the adder cells in the top row into the "C" cells? Write the logic equations for the "C" box.

c. [20] <J.8> Show how to extend the block diagram in J.17 so it will produce the carry-out bit c_8.

J.28 [15] <J.9> For ordinary Booth recoding, the multiple of b used in the ith step is simply $a_{i-1} - a_i$. Can you find a similar formula for radix-4 Booth recoding (overlapped triplets)?

J.29 [20] <J.9> Expand Figure J.29 in the fashion of J.27, showing the individual adders.

J.30 [25] <J.9> Write out the analog of Figure J.25 for radix-8 Booth recoding.

J.31 [18] <J.9> Suppose that $a_{n-1} \ldots a_1 a_0$ and $b_{n-1} \ldots b_1 b_0$ are being added in a signed-digit adder as illustrated in the example on page J-53. Write a formula for the ith bit of the sum, s_i, in terms of a_i, a_{i-1}, a_{i-2}, b_i, b_{i-1}, and b_{i-2}.

J.32 [15] <J.9> The text discussed radix-4 SRT division with quotient digits of -2, -1, 0, 1, 2. Suppose that 3 and -3 are also allowed as quotient digits. What relation replaces $|r_i| \leq 2b/3$?

J.33 [25/20/30] <J.9> Concerning the SRT division table, Figure J.34:

a. [25] <J.9> Write a program to generate the results of Figure J.34.

b. [20] <J.9> Note that Figure J.34 has a certain symmetry with respect to positive and negative values of P. Can you find a way to exploit the symmetry and only store the values for positive P?

c. [30] <J.9> Suppose a carry-save adder is used instead of a propagate adder. The input to the quotient lookup table will be k bits of divisor and l bits of remainder, where the remainder bits are computed by summing the top l bits of the sum and carry registers. What are k and l? Write a program to generate the analog of Figure J.34.

J.34 [12/12/12] <J.9, J.12> The first several million Pentium chips produced had a flaw that caused division to sometimes return the wrong result. The Pentium uses a radix-4 SRT algorithm similar to the one illustrated in the example on page J-56 (but with the remainder stored in carry-save format; see Exercise J.33(c)). According to Intel, the bug was due to five incorrect entries in the quotient lookup table.

a. [12] <J.9, J.12> The bad entries should have had a quotient of plus or minus 2, but instead had a quotient of 0. Because of redundancy, it's conceivable that the algorithm could "recover" from a bad quotient digit on later iterations. Show that this is not possible for the Pentium flaw.

b. [12] <J.9, J.12> Since the operation is a floating-point divide rather than an integer divide, the SRT division algorithm on page J-45 must be modified in two ways. First, step 1 is no longer needed, since the divisor is already normalized. Second, the very first remainder may not satisfy the proper bound ($|r| \leq 2b/3$ for Pentium; see page J-55). Show that skipping the very first left shift in step 2(a) of the SRT algorithm will solve this problem.

c. [12] <J.9, J.12> If the faulty table entries were indexed by a remainder that could occur at the very first divide step (when the remainder is the divisor), random testing would quickly reveal the bug. This didn't happen. What does that tell you about the remainder values that index the faulty entries?

J.35 [12] <J.6, J.9> The discussion of the remainder-step instruction assumed that division was done using a bit-at-a-time algorithm. What would have to change if division were implemented using a higher-radix method?

J.36 [25] <J.9> In the array of Figure J.28, the fact that an array can be pipelined is not exploited. Can you come up with a design that feeds the output of the bottom CSA into the bottom CSAs instead of the top one, and that will run faster than the arrangement of Figure J.28?

K.1	Introduction	K-2
K.2	A Survey of RISC Architectures for Desktop, Server, and Embedded Computers	K-3
K.3	The Intel 80x86	K-30
K.4	The VAX Architecture	K-50
K.5	The IBM 360/370 Architecture for Mainframe Computers	K-69
K.6	Historical Perspective and References	K-75

K

Survey of Instruction Set Architectures

RISC: any computer announced after 1985.

Steven Przybylski
A Designer of the Stanford MIPS

K.1 Introduction

This appendix covers 10 instruction set architectures, some of which remain a vital part of the IT industry and some of which have retired to greener pastures. We keep them all in part to show the changes in fashion of instruction set architecture over time.

We start with eight RISC architectures, using RISC V as our basis for comparison. There are billions of dollars of computers shipped each year for ARM (including Thumb-2), MIPS (including microMIPS), Power, and SPARC. ARM dominates in both the PMD (including both smart phones and tablets) and the embedded markets.

The 80x86 remains the highest dollar-volume ISA, dominating the desktop and the much of the server market. The 80x86 did not get traction in either the embedded or PMD markets, and has started to lose ground in the server market. It has been extended more than any other ISA in this book, and there are no plans to stop it soon. Now that it has made the transition to 64-bit addressing, we expect this architecture to be around, although it may play a smaller role in the future then it did in the past 30 years.

The VAX typifies an ISA where the emphasis was on code size and offering a higher level machine language in the hopes of being a better match to programming languages. The architects clearly expected it to be implemented with large amounts of microcode, which made single chip and pipelined implementations more challenging. Its successor was the Alpha, a RISC architecture similar to MIPS and RISC V, but which had a short life.

The vulnerable IBM 360/370 remains a classic that set the standard for many instruction sets to follow. Among the decisions the architects made in the early 1960s were:

- 8-bit byte
- Byte addressing
- 32-bit words
- 32-bit single precision floating-point format + 64-bit double precision floating-point format
- 32-bit general-purpose registers, separate 64-bit floating-point registers
- Binary compatibility across a family of computers with different cost-performance
- Separation of architecture from implementation

As mentioned in Chapter 2, the IBM 370 was extended to be virtualizable, so it had the lowest overhead for a virtual machine of any ISA. The IBM 360/370 remains the foundation of the IBM mainframe business in a version that has extended to 64 bits.

K.2 A Survey of RISC Architectures for Desktop, Server, and Embedded Computers

Introduction

We cover two groups of Reduced Instruction Set Computer (RISC) architectures in this section. The first group is the desktop, server RISCs, and PMD processors:

- Advanced RISC Machines ARMv8, AArch64, the 64-bit ISA,
- MIPS64, version 6, the most recent the 64-bit ISA,
- Power version 3.0, which merges the earlier IBM Power architecture and the PowerPC architecture.
- RISC-V, specifically RV64G, the 64-bit extension of RISC-V.
- SPARCv9, the 64-bit ISA.

As Figure K.1 shows these architectures are remarkably similar.

There are two other important historical RISC processors that are almost identical to those in the list above: the DEC Alpha processor, which was made by Digital Equipment Corporation from 1992 to 2004 and is almost identical to MIPS64. Hewlett-Packard's PA-RISC was produced by HP from about 1986 to 2005, when it was replaced by Itanium. PA-RISC is most closely related to the Power ISA, which emerged from the IBM Power design, itself a descendant of IBM 801.

The second group is the embedded RISCs designed for lower-end applications:

- Advanced RISC Machines, Thumb-2: an 32-bit instruction set with 16-bit and 32-bit instructions. The architecture includes features from both ARMv7 and ARMv8.
- microMIPS64: a version of the MIPS64 instruction set with 16-it instructions, and
- RISC-V Compressed extension (RV64GC), a set of 16-bit instructions added to RV64G

Both RV64GC and microMIPS64 have corresponding 32-bit versions: RV32GC and microMIPS32.

Since the comparison of the base 32-bit or 64-bit desktop and server architecture will examine the differences among those ISAs, our discussion of the embedded architectures focuses on the 16-bit instructions. Figure K.2 shows that these embedded architectures are also similar. In all three, the 16-bit instructions are versions of 32-bit instructions, typically with a restricted set of registers.The idea is to reduce the code size by replacing common 32-bit instructions with 16-bit versions. For RV32GC or Thumb-2, including the 16-bit instructions yields a reduction in code size to about 0.73 of the code size using only the 32-bit ISA (either RV32G or ARMv7).

	ARMv8	MIPS64 R6	Power v3.0	RV64G	SPARCv9
Original date (base ISA)	1986	1986	1990	2016	1987
Date of this ISA	2011	2014	2013	2016	2008
Instruction size (bits)	32	32	32	32	32
Address space (size, model)	64 bits (flat)	64 bits, flat	64 bits, flat	64 bits, flat	64 bits, flat
Data alignment	Aligned preferred	Aligned preferred	Unaligned	Aligned preferred	Aligned
Data addressing modes	8 (including scaled, pre/post increment)	1 (+1 for FP only)	4	1	2
Integer registers (number, model, size)	31 GPR x 64, plus stack pointer	31 GPR × 64 bits	31 GPR × 64 bits	31 GPR × 64 bits	31 GPR × 64 bits
Separate floating-point registers	32x32 or 32x64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits
Floating-point format	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double

Figure K.1 Summary of the most recent version of five architectures for desktop, server, and PMD use (all had earlier versions). Except for the number of data address modes and some instruction set details, the integer instruction sets of these architectures are very similar. Contrast this with Figure K.29. In ARMv8, register 31 is a 0 (like register 0 in the other architectures), but when it is used in a load or store, it is the current stack pointer, a special purpose register. We can either think of SP-based addressing as a different mode (which is how the assembly mnemonics operate) or as simply a register + offset addressing mode (which is how the instruction is encoded).

	microMIPS64	RV64GC	Thumb-2
Date announced	2009	2016	2003
Instruction size (bits)	16/32	16/32	16/32
Address space (size, model)	32/64 bits, flat	32/64 bits flat	32 bits, flat
Data alignment	Aligned	Aligned, preferred	Aligned
Data addressing modes	2	1	6
Integer registers (number, model, size)	31 GPR × 64 bits	31 GPR × 64 bits	15 GPR × 32 bits
Integer registers accessible by most 16-bit instructions (which use should specifiers)	8 GPR + SP + GP +RA GPRs: 0, 2-7, 17, or 2-7, 16, 17	8 GPRs + SP GPRs: 8-15	8 GPR + SP × 32 bits

Figure K.2 Summary of three recent architectures for embedded applications. All three use 16-bit extensions of a base instruction set. Except for number of data address modes and a number of instruction set details, the integer instruction sets of these architectures are similar. Contrast this with Figure K.29. An earlier 16-bit version of the MIPS instruction set, called MIPS16, was created in 1995 and was replaced by microMIPS32 and microMIPS64. The first Thumb architecture had only 16-bit instructions and was created in 1996. Thumb-2 is built primarily on ARMv7, the 32-bit ARM instruction set; it offers 16 registers. RISC-V also defines RV32E, which has only 16 registers, includes the 16-bit instructions, and cannot have floating point. It appears that most implementations for embedded applications opt for RV32C or RV64GC.

A key difference among these three architectures is the structure of the base 32-bit ISA. In the case of RV64GC, the 32-bit instructions are exactly those of RV64G. This is possible because RISC V planned for the 16-it option from the beginning, and branch addresses and jump addresses are specified to 16-it boundaries. In the case of microMIPS64, the base ISA is MIPS64, with one change: branch and jump offsets are interpreted as 16-bit rather than 32-bit aligned. (microMIPS also uses the encoding space that was reserved in MIPS64 for user-defined instruction set extensions; such extensions are not part of the base ISA.)

Thumb-2 uses a slightly different approach. The 32-bit instructions in Thumb-2 are mostly a subset of those in ARMv7; certain features that were dropped in ARMv8 are not included (e.g., conditional execution of most instructions and the ability to write the PC as a GPR). Thumb-2 also includes a few dozen instructions introduced in ARMv8, specifically bit field manipulation, additional system instructions, and synchronization support. Thus, the 32-bit instructions in Thumb-2 constitute a unique ISA.

Earlier versions of the 16-bit instruction sets for MIPS (MIPS16) and ARM (Thumb), took the approach of creating a separate mode, invoked by a procedure call, to transfer control to a code segment that employed only 16-bit instructions.

The 16-bit instruction set was not complete and was only intended for user programs that were code-size critical.

One complication of this description is that some of the older RISCs have been extended over the years. We decided to describe the most recent versions of the architectures: ARMv8 (the 64-bit architecture AArch64), MIPS64 R6, Power v3.0, RV64G, and SPARC v9 for the desktop/server/PMD, and the 16-bit subset of the ISAs for microMIPS64, RV64GC, and Thumb-2.

The remaining sections proceed as follows. After discussing the addressing modes and instruction formats of our RISC architectures, we present the survey of the instructions in five steps:

- Instructions found in the RV64G core, described in Appendix A.
- Instructions not found in the RV64G or RV64GC but found in two or more of the other architectures. We describe and organize these by functionality, e.g. instructions that support extended integer arithmetic.
- Instruction groups unique to ARM, MIPS, Power, or SPARC, organized by function.
- Multimedia extensions of the desktop/server/PMD RISCs
- Digital signal-processing extensions of the embedded RISCs

Although the majority of the instructions in these architectures are included, we have not included every single instruction; this is especially true for the Power and ARM ISAs, which have *many* instructions.

Addressing Modes and Instruction Formats

Figure K.3 shows the data addressing modes supported by the desktop/server/PMD architectures. Since all, but ARM, have one register that always has the value 0 when used in address modes, the absolute address mode with limited range can be synthesized using register 0 as the base in displacement addressing. (This register can be changed by arithmetic-logical unit (ALU) operations in PowerPC, but is always zero when it is used in an address calculation.) Similarly, register indirect addressing is synthesized by using displacement addressing with an offset of 0. Simplified addressing modes is one distinguishing feature of RISC architectures.

As Figure K.4 shows, the embedded architectures restrict the registers that can be accessed with the 16-bit instructions, typically to only 8 registers, for most instructions, and a few special instructions that refer to other registers. Figure K.5 shows the data addressing modes supported by the embedded architectures in their 16-bit instruction mode. These versions of load/store instructions restrict the registers that can be used in address calculations, as well as significantly shorten the immediate fields, used for displacements.

References to code are normally PC-relative, although jump register indirect is supported for returning from procedures, for case statements, and for pointer function calls. One variation is that PC-relative branch addresses are often shifted left 2 bits before being added to the PC for the desktop RISCs, thereby increasing the branch distance. This works because the length of all instructions for the desktop

	ARMv8	MIPS64 R6	Power v3.0	RV64G	SPARCv9
Register + offset (displacement or based)	B, H, W, D	B, H, W, D	B, H, W, D	B, H, W, D	B, H, W, D
Register + register (indexed)	B, H, W, D		B, H, W, D		B, H, W, D
Register + scaled register (scaled)	B, H, W, D	W,D			
Register + register + offset	B, H, W, D				
Register + offset & update register to effective address (based with update)	B, H, W, D		B, H, W, D		
Register & update register to register + offset (register with update)	B, H, W, D				
Register + Register & update register to effective address (indexed with update)	B, H, W, D		B, H, W, D		
PC-relative (PC + displacement)	W, D	W, D			

Figure K.3 Summary of data addressing modes supported by the desktop architectures, where B, H, W, D indicate what datatypes can use the addressing mode. Note that ARM includes two different types of address modes with updates, one of which is included in Power.

Register specifier	microMIPS64	RV64GC	Thumb-2
3-bit	2-7,16, 17	8-15	0-7
stack pointer register	29	2	0 (when used in load/store)
global pointer register	28		
return address register	31	1	14
Using special register	stack pointer or global pointer; 5-bit offset	stack pointer; 5-bit offset	stack pointer; 8-bit offset

Figure K.4 Register encodings for the 16-bit subsets of microMIPS64, RV64GC, and Thumb-2, including the core general purpose registers, and special-purpose registers accessible by some instructions.

Addressing mode	microMIPS64	RV64GC	Thumb-2
Register + offset (displacement or based)	4-bit offset, one of 8 registers	5-bit offset, one of 8 registers	5-bit offset, one of 8 registers
PC-relative data			word only; 8-bit offset
Using special register	stack pointer or global pointer; 5-bit offset	stack pointer; 5-bit offset	stack pointer; 8-bit offset

Figure K.5 Summary of data addressing modes supported by the embedded architectures. microMIPS64, RV64c, and Thumb-2 show only the modes supported in 16-bit instruction formats. The stack pointer in RV64GC and microMIPS64 is a designed GPR; it is another version of r31 is Thumb-2. In microMIPS64, the global pointer is register 30 and is used by the linkage convention to point to the global variable data pool. Notice that typically only 8 registers are accessible as base registers (and as we will see as ALU sources and destinations).

RISCs is 32 bits and instructions must be aligned on 32-bit words in memory. Embedded architectures and RISC V (when extended) have 16-bit-long instructions and usually shift the PC-relative address by 1 for similar reasons.

Figure K.6 shows the most important instruction formats of the desktop/server/PMD RISC instructions. Each instruction set architecture uses four primary instruction formats, which typically include 90–98% of the instructions. The register-register format is used for register-register ALU instructions, while the ALU immediate format is used for ALU instructions with an immediate operand and also for loads and stores. The branch format is used for conditional branches, and the jump/call format for unconditional branches (jumps) and procedures calls.

There are a number of less frequently used instruction formats that Figure K.6 leaves out. Figure K.7 summarizes these for the desktop/server/PMD architectures.

Unlike, their 32-bit base architectures, the 16-bit extensions (microMIPS64, RV64GC, and Thumb-2) are focused on minimizing code. As a result, there are a larger number of instruction formats, even though there are far fewer instructions.

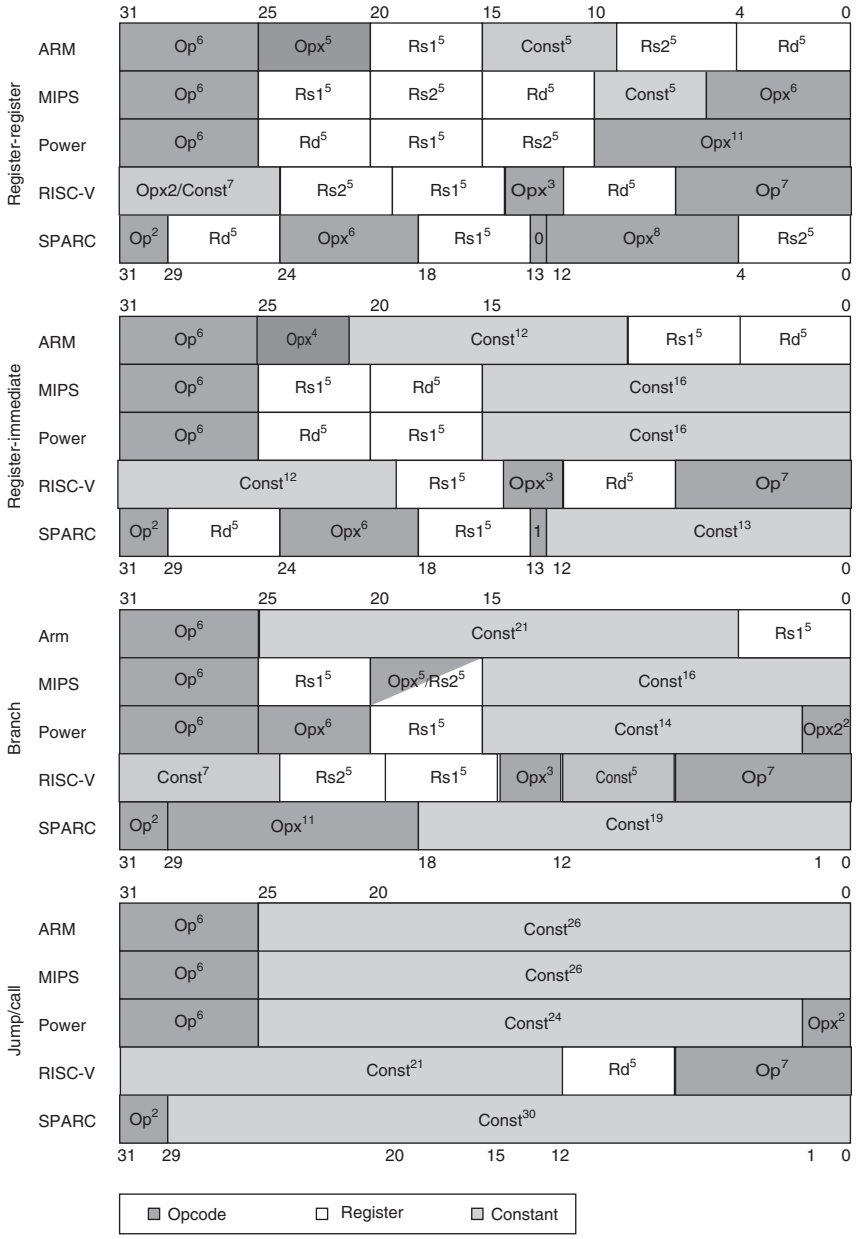


Figure K.6 Instruction formats for desktop/server RISC architectures. These four formats are found in all five architectures. (The superscript notation in this figure means the width of a field in bits.) Although the register fields are located in similar pieces of the instruction, be aware that the destination and two source fields are sometimes scrambled. Op = the main opcode, Opx = an opcode extension, Rd = the destination register, Rs1 = source register 1, Rs2 = source register 2, and Const = a constant (used as an immediate, address, mask, or sift amount). Although the labels on the instruction formats tell where various instructions are encoded, there are variations. For example, loads and stores, both use the ALU immediate form in MIPS. In RISC-V, loads use the ALU immediate format, while stores use the branch format.

Architecture	Additional instruction formats	Format function and use
ARMv8	At least 10 (many small variations); major forms are shown.	Logical immediates with 13-bit immediate field.
		Shifts with constant amount.(16-bit opcode)
		16-bit immediate form
		Exclusive operations: three register fields
		Branch register: long opcode
		Load/store with address mode bits.
MIPS64	1	A PC-relative set of load/stores using register-immediate format but with 18-bit immediates (since the other source is the PC).
Power	9 (not including a number of small variations or the vector extensions)	DQ-mode: uses the ALU immediate form but takes four bits of the displacement for other functions.
		DS-mode: uses the ALU immediate form but takes two bits of the displacement for other functions.
		DX-form: Like register-immediate, but with a register-source replaced by PC.
		MD, MDS formats: like register-register but used for shifts and rotates.
		X, XS, and several minor variations: used for indexed addressing modes, shifts, and a variety of extended purposes.
		Z22, Z23 formats: used for manipulating floating point numbers
RV64	2	SB format: a variant of the branch format with different immediate treatment
		UJ format: a variant of the jump/call format with different immediate treatment
SPARC	3	Another format for conditional branches containing 3 more bits of displacement (22 total versus 19) but no prediction hints.
		A format with 22-bit immediate used to load the upper half of a register,
		A format for conditional branches based on a register compare with zero.

Figure K.7 Other instruction formats beyond the four major formats of the previous figure. In some cases, there are formats very similar to one of the four core formats, but where a register field is used for other purposes. The Power architecture also includes a number of formats for vector operations.

microMIPs64 and RV64GC have eight and seven major formats, respectively, and Thumb-2 has 15. As Figure K.8 shows, these involve varying number of register operands (0 to 3), different immediate sizes, and even different size register specifiers, with a small number of registers accessible my most instructions, and fewer instructions able to access all 32 registers.

Instructions

The similarities of each architecture allow simultaneous descriptions, starting with the operations equivalent to the RISC-V 64-bit ISA.

Architecture	Opcode main: extended	Register specifiers x length	Immediate field length	Typical instructions
microMIPS64	6	none	10	Jumps
	6	1x5	5	Register-register operation (32 registers) and Load using SP as base register; any destination
	6	1x3	7	Branches equal/not equal zero. Loads using GP. as base.
	6:4	2x3		Register-register operation, rd/rs1, and rs2; 8 registers
	6:1	2x3	3	Register-register immediate, rd/rs1, and rs2; 8 registers
	6	2x3	4	Loads and stores; 8 registers
	6:4	2x3		Register-register operation, rd, and rs1; 8 registers
	6	2x5		Register-register operation; 32 registers.
RV64GC	2:3		11	Jumps
	2:3	1x3	7	Branch
	2:3	1x3	8	Immediate one source register.
	2:3	1x5	6	Store using SP as base.
	2:3	1x5	6	ALU immediate and load using SP as base.
	2:4	2x5		Register-register operation
	2:3	2x3	5	Loads and stores using 8 registers.
Thumb-2	3:2	2x3	5	Shift, move, load/store word/byte
	3:2	1x3	8	immediates: add, subtract, move, and compare
	4:1	1x3	8	Load/store with stack pointer as base, Add to SP or PC, Load/store multiple
	4:3	3x3		Load register indexed
	4:4		8	Conditional branch, system instruction
	4:12			Miscellaneous: 22 different instructions with 12 formats (includes compare and branch on zero, pop/push registers, adjust stack pointer, reverse bytes, IF-THEN instruction).
	5	1x3	8	Load relative to PC
	5		11	Unconditional branch
	6:1	3x3		Add/subtract
	6:3	1x4, 1x3		Special data processing
	6:4	2x3		Logical data processing
	6:6	1x4		Branch and change instruction set (ARM vs. Thumb)

Figure K.8 Instruction formats for the 16-bit instructions of microMIPS64, RV64GC, and Thumb-2. For instructions with a destination and two sources, but only two register fields, the instruction uses one of the registers as both source and destination. Note that the extended opcode field (or function field) and immediate field sometimes overlap or are identical. For RV64GC and microMIPS64, all the formats are shown; for Thumb-2, the Miscellaneous format includes 22 instructions with 12 slightly different formats; we use the extended opcode field, but a few of these instructions have immediate or register fields.

RV64G Core Instructions

Almost every instruction found in the RV64G is found in the other architectures, as Figures K.9 through K.19 show. (For reference, definitions of the RISC-V instructions are found in Section A.9.) Instructions are listed under four categories: data transfer (Figure K.9); arithmetic, logical (Figure K.10); control (Figure K.11 and Figure K.12); and floating point (Figure K.13).

If a RV64G core instruction requires a short sequence of instructions in other architectures, these instructions are separated by semicolons in Figure K.9 through Figure K.13. (To avoid confusion, the destination register will always be the leftmost operand in this appendix, independent of the notation normally used with each architecture.).

Compare and Conditional Branch

Every architecture must have a scheme for compare and conditional branch, but despite all the similarities, each of these architectures has found a different way to perform the operation! Figure K.11 summarizes the control instructions, while Figure K.12 shows details of how conditional branches are handled. SPARC uses the traditional four condition code bits stored in the program status word: *negative, zero, carry*, and *overflow*. They can be set on any arithmetic or logical instruction; unlike earlier architectures, this setting is optional on each instruction. An explicit option leads to fewer problems in pipelined implementation. Although condition codes can be set as a side effect of an operation, explicit compares are synthesized with a subtract using `r0` as the destination. SPARC conditional branches test condition codes to determine all possible unsigned and signed relations. Floating point uses separate condition codes to encode the EEE 754 conditions, requiring a floating-point compare instruction. Version 9 expanded SPARC branches in four ways: a separate set of condition codes for 64-bit operations; a branch that tests the contents of a register and branches if the value is `=`, `not=`, `<`, `<=`, `>=`, or `<= 0`; three more sets of floating-point condition codes; and branch instructions that encode static branch prediction.

Power also uses four condition codes: *less than, greater than, equal*, and *summary overflow*, but it has eight copies of them. This redundancy allows the Power instructions to use different condition codes without conflict, essentially giving Power eight extra 4-bit registers. Any of these eight condition codes can be the target of a compare instruction, and any can be the source of a conditional branch. The integer instructions have an option bit that behaves as if the integer is followed by a compare to zero that sets the first condition "register." Power also lets the second "register" be optionally set by floating-point instructions. PowerPC provides logical operations among these eight 4-bit condition code registers (`CRAND`, `CROR`, `CRXOR`, `CRNAND`, `CRNOR`, `CREQV`), allowing more complex conditions to be tested by a single branch. Finally, Power includes a set of branch count registers, that are automatically decremented when tested, and can be used in a branch condition. There are also special instructions for moving from/to the condition register.

Data transfer (instruction formats)	R-I	R-I	R-I, R-R	R-I	R-I, R-R
Instruction name	**ARMv8**	**MIPS64**	**Power**	**RV64G**	**SPARC**
Load byte signed/unsigned.	LDR_B	LB_	LBZ; EXTSB	LB_	LD_B
Load halfword signed, unsigned	LDR_H	LH_	LHA/LHZ	LH_	LD_H
Load word	LDRSW/LDR	LW_	LW_	LW_	LD_W
Load double	LDRX	LD	LD	LD	LDD
Load float register SP/DP	LD_	L_C1	LF_	FL_	LD_F
Store byte	STB	SB	STB	SB	STB
Store half word	STW	SH	STH	STH	STH
Store word	STL	SW	STW	SW	ST
Store double word	STX	SD	SD	SD	STD
Store float SP/DP	ST_	S_C1	STF_	FS_	ST_F
Load reserved	LDEXB,LDEXH LDEXW,LDEXD	LL, LLD	lwarx, ldarx,	LR	
Store conditional	STEXB, STEXH, STEXW, STEXD	SC, SCD	stwcx, stdcx	SC	
Read/write spec. register	MF_, MT_	MF, MT_	M_SPR,	csrr_, csrr_i,	RD__,WR__
Move integer to FP register	ITOFS	MFC1/ DMFC1	STW; LDFS	STW; FLDWX	ST; LDF
Move FP to integer register	FTTOIS	MTC1/ DMTC1	STFS; LW	FSTWX; LDW	STF; LD
Synchronize data, instruction stream	DSB ISB	SYNC, SYNCI	SYNC, ISYNC	Fence Fence.i	MEMBAR FLUSH
Atomic operations	LDWAT, LDDAT STWAT, STDAT	LLWP, LLDP, SCWP, SCDP		AMOSWAP.W/D, AMOADD,W.D, AMOAND.W/D, AMOXOR.W/D, AMOOR.W/D, AMOMIN_.W/D, AMOMAX_.W/D	CASA, SWAP, LDSTUB

Figure K.9 Desktop RISC data transfer instructions equivalent to RV64G core. A sequence of instructions to synthesize a RV64G instruction is shown separated by semicolons. The MIPS and Power instructions for atomic operations load and conditionally store a pair of registers and can be used to implement the RV64G atomic operations with at most one intervening ALU instruction. The SPARC instructions: compare-and-swap, swap, LDSTUB provide atomic updates to a memory location and can be used to build the RV64G instructions. The Power3 instructions provide all the functionality, as the RV64G instructions, depending on a function field.

Arithmetic/logical (instruction formats)	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I
Instruction name	**ARM v8**	**MIPS64**	**Power v3**	**RISC-V**	**SPARC v.9**
Add word, immediate	ADD, ADDI	ADDU, ADDUI,	ADD, ADDI	ADDW, ADDWI	ADD
Add double word	ADDX	DADDU, DADDUI	ADD, ADDI	ADD, ADDI	ADD
Subtract	SUB, SUBI	SUBU, SUBI	SUBF	SUBW, SUBWI	SUB
Subtract double word	SUBX	DSUBU, DSUBUI	SUBF	SUB,SUBI	SUB
Multiply	MUL, SMUL	MUL, MULU, DMUL, DMULU	MULLW, MULLI	MUL, MULU, MULW, MULWU	MULX
Divide	MULX, SMULX	DIV, DIVU,DDIV, DDIVU	DIVW	DIV, DIVU, DIVW, DIVWU	DIVX
Remainder		MOD, MODU, DMOD, DMODU	MODSW, MODUW	REM, REMU, REMW, REMWU	
And	AND, ANDI	AND, ANDI	AND, ANDI	AND, ANDI	AND
Or	OR, ORI	OR, ORI	OR, ORI	OR, ORI	OR
Xor	XOR, XORI	XOR, XORI	XOR, XORI	XOR, XORI	XOR
Load bits 31..16	MOV	LUI	ADDIS	ADDIS	SETHI (Bfmt.)
Load upper bits of PC	ADR	ADDIUPC	ADDPCIS	AUIPC	
Shift left logical, double word and word versions, immediate and variable	LSL	SLLV, SLL	RLWINM	SLL, SLLI, SLLW, SLLWI	SLL
Shift right logical, double word and word version, immediate and variables	RSL	SRLV, SRL	RLWINM 32-i	SRL, SRLI, SRLW, SRLWI	SRL
Shift right arithmetic, double word and word versions, immediate and variable	RSA	SRAV, SRA	SRAW	SRA, SRAI, SRAW, SRAWI	SRA
Compare	CMP	SLT/U, SLTI/U	CMP(I)CLR	SLT/U, SLTI/U	SUBcc r0,...

Figure K.10 Desktop RISC arithmetic/logical instructions equivalent to RISC-V integer ISA. MIPS also provides instructions that trap on arithmetic overflow, which are synthesized in other architectures with multiple instructions. Note that in the "Arithmetic/logical" category all machines but SPARC use separate instruction mnemonics to indicate an immediate operand; SPARC offers immediate versions of these instructions but uses a single mnemonic. (Of course, these are separate opcodes!)

Instruction name	ARMv8	MIPS64	PowerPC	RISC-V	SPARC v.9
Branch on integer compare	B.cond, CBZ, CBNZ	BEQ, BNE, B_Z (<, >, <=, >=) OR S***; BEZ	BC	BEQ, BNE, BLT, BGE, BLTU, BGEU	BR_Z, BPcc (<, >, <=, >=, =, not=)
Branch on floating-point compare	B.cond	BC1T, BC1F	BC	BEZ, BNZ	FBPfcc (<, >, <=, >=, =,...)
Jump, jump register	B, BR	J, JR	B, BCLR, BCCTR	JAL, JALR (with x0)	BA, JMPL r0,...
Call, call register	BL, BLR	JAL, JALR	BL, BLA, BCLRL, BCCTRL	JAL, JALR	CALL, JMPL
Trap	SVC, HVC, SMC	BREAK	TW, TWI	ECALL	Ticc, SIR
Return from interrupt	ERET	JR; ERET	RFI	EBREAK	DONE, RETRY, RETURN

Figure K.11 Desktop RISC control instructions equivalent to RV64G.

	ARMv8	MIPS64	PowerPC	RISC-V	SPARC v.9
Number of condition code bits (integer and FP)	16 (8 + the inverse)	none	8 × 4 both	none	2 × 4 integer, 4 × 2 FP
Basic compare instructions (integer and FP)	1 integer; 1 FP	1 integer, 1 FP	4 integer, 2 FP	2 integer; 3 FP	1 FP
Basic branch instructions (integer and FP)	1	2 integer, 1 FP	1 both	4 integer (used for FP as well)	3 integer, 1 FP
Compare register with register/ constant and branch	—	=, not=	—	=, not =, >=, <	—
Compare register to zero and branch	—	=, not=, <, <=, >, >=	—	=, not=, <, <=, >, >=	=, not=, <, <=, >, >=

Figure K.12 Summary of five desktop RISC approaches to conditional branches. Integer compare on SPARC is synthesized with an arithmetic instruction that sets the condition codes using `r0` as the destination.

RISC-V and MIPS are most similar. RISC-V uses a compare and branch with a full set of arithmetic comparisons. MIPS also uses compare and branch, but the comparisons are limited to equality and tests against zero. This limited set of conditions simplifies the branch determination (since an ALU operation is not required to test the condition), at the cost of sometimes requiring the use of a set-on-less-than instruction (`SLT`, `SLTI`, `SLTU`, `SLTIU`), which compares two operands and then set the destination register to 1 if less and to 0 otherwise. Figure K.12 provides

Floating point (instruction formats)	R-R	R-R	R-R	R-R	R-R
Instruction name	**ARMv8**	**MIPS64**	**PowerPC**	**RISC-V**	**SPARC v.9**
Add single, double	FADD	ADD.*	FADD*	FADD.*	FADD*
Subtract single, double	FSUB	SUB.*	FSUB*	FSUB.*	FSUB*
Multiply single, double	FMUL	MUL.*	FMUL*	FMUL.*	FMUL*
Divide single, double	FDIV	DIV.*	FDIV*	FDIV.*	FDIV*
Square root single, double	FSQRT	SQRT.*	FSQRT*	FSQRT.*	FSQRT*
Multiply add; Negative multiply add: single, double	FMADD, FNMADD	MADD.* NMAD.*	FMADD*, FNMADD*	FMADD.* FNMADD.*	
Multiply subtract single, double, Negative multiply subtract: single, double	FMSUB, FNMSUB	MSUB.*, NMSUB.*	FMSUB*, FNMSUB*	FMSUB.*, FNMSUB.*	
Copy sign or negative sign double or single to another FP register	FMOV, FNEG	FMOV.*, FNEG.*	FMOV*, FNEG*	FSGNJ.*, FSGNJN.*	FMOV*, FNEG*
Replace sign bit with XOR of sign bits single double	FABS	FABS.*	FABS*	FSGNJX.*	FABS*
Maximum or minimum single, double	FMAX, FMIN	MAX.*, MIN.*		FMAX.*, FMIN.*	
Classify floating point value single double		CLASS.*		FCLASS.*	
Compare	FCMP	CMP.*	FCMP*	FCMP.*	FCMP*
Convert between FP single or double and FP single or double, OR integer single or double, signed and unsigned with rounding	FCVT	CVT, CEIL, FLOOR		FCVT	F*TO*

Figure K.13 Desktop RISC floating-point instructions equivalent to RV64G ISA with an empty entry meaning that the instruction is unavailable. ARMv8 uses the same assembly mnemonic for single and double precision; the register designator indicates the precision. "*" is used as an abbreviation for S or D. For floating point compares all conditions: equal, not equal, less than, and less-then or equal are provided. Moves operate in both directions from/to integer registers. Classify sets a register based on whether the floating point quantity is plus or minus infinity, denorm, +/ − 0, etc.). The sign-injection instructions take two operands, but are primarily used to form floating point move, negate, and absolute value, which are separate instructions in the other ISAs.

additional details on conditional branch. RISC-V floating point comparisons sets an integer register to 0 or 1, and then use conditional branches on that content.MIPS also uses separate floating-point compare, which sets a floating point register to 0 or 1, which is then tested by a floating-point conditional branch.

ARM is similar to SPARC, in that it provides four traditional condition codes that are optionally set. `CMP` subtracts one operand from the other and the difference sets the condition codes. Compare negative (`CMN`) *adds* one operand to the other, and the sum sets the condition codes. `TST` performs logical `AND` on the two operands to set all condition codes but overflow, while `TEQ` uses exclusive OR to set the first three condition codes. Like SPARC, the conditional version of the ARM branch instruction tests condition codes to determine all possible unsigned and signed relations. ARMv8 added both bit-test instructions and also compare and branch against zero. Floating point compares on ARM, set the integer condition codes, which are used by the `B.cond` instruction.

As Figure K.13 shows the floating point support is similar on all five architectures.

RV64GC Core 16-bit Instructions

Figures K.14 through K.17 summarize the data transfer, ALU, and control instructions for our three embedded processors: microMIPS64, RV64GC, and Thumb-2. Since these architectures are all based on 32-bit or 64-bit versions of the full architecture, we focus our attention on the functionality implemented by the 16-bit instructions. Since floating point is optional, we do not include it. I

Instruction name	microMIPS64 rs1;rs2/dst; offset	RV64GC rs1;rs2/dst; offset	Thumb-2 rs1;rs2/dst; offset
Load word	8;8;4	8;8;5	8;8;5
Load double word		8;8;5	
Load word with stack pointer as base register	1;32;5	1;32;6	1;3;8
Load double word with stack pointer as base register		1;32;6	
Store word	8;8;4	8;8;5	8;8;5
Store double word		8;8;5	
Store word with stack pointer as base register	1;32;5	1;32;6	1;3;8
Store double with stack pointer as base register		1;32;6	

Figure K.14 Embedded RISC data transfer instructions equivalent to RV64GC 16-bit ISA; a blank indicates that the instruction is not a 16-bit instruction. Rather than show the instruction name, where appropriate, we show the number of registers that can the base register for the address calculation, followed by the number of registers that can be the destination for a load or the source for a store, and finally, the size of the immediate used for address calculation. For example: 8; 8; 5 for a load means that there are 8 possible base registers, 8 possible destination registers for the load, and a 5-bit offset for the address calculation. For a store, 8; 8; 5, specifies that the source of the value to store comes from one of 8 registers. Remember that Thumb-2 also has 32-bit instructions (although not the full ARMv8 set) and that RV64GC and microMIPS64 have the full set of 32-bit instructions in RV64I or MIPS64.

Instruction Name/Function	microMIPS64	RV64GC	Thumb-2
Load immediate	8;7	32;6	8;8
Load upper immediate		32;6	
add immediate	32;4	32;6	8;8;3
add immediate word (32 bits) & sign extend		32;6	
add immediate to stack pointer	1;9	1;6 (adds 16x imm.)	1;7
add immediate to stack pointer store in reg.	1;8;6	1;8;6 (adds 4x imm.)	
shift left/right logical	8;8;3 (shift amt.)	8;6(shift amt.)	8;8;5 (shift amt.)
shift right arithmetic		8;6(shift amt.)	8;8;5 (shift amt.)
AND immediate	8;8;4	8;6	8;8
move	32;32	32;32	16;16
add	8;8;8	32;32	8;8;8 16;16
AND, OR, XOR	8;8	8;8	8;8
subtract	8;8;8	8;8	8;8;8
add word, subtract word (32 bits) & sign extend		8;8	

Figure K.15 ALU instructions provided in RV64GC and the equivalents, if any, in the 16-bit instructions of microMIPS64 or Thumb-2. An entry shows the number of register sources/destinations, followed by the size of the immediate field, if it exists for that instruction. The add to stack pointer with scaled immediate instructions are used for adjusting the stack pointer and creating a pointer to a location on the stack. In Thumb, the add has two forms one with three operands from the 8-register subset (Lo) and one with two operands but any of 16-registers.

	microMIPS64	RV64GC	Thumb-2
Unconditional branch	10-bit offset	11-bit offset	11-bit offset
Unconditional branch and link		11-bit offset	11-bit offset
Unconditional branch to register w/wo link	any of 32 registers	any of 32 registers	
Compare register to zero (=/!=) and branch	8 registers; 7-bit offset	8 registers; 8-bit offset	no: but see caption

Figure K.16 Summary of three embedded RISC approaches to conditional branches. A blank indicates that the instruction does not exist. Thumb-2 uses 4 condition code bits; it provides a conditional branch that tests the 4-bit condition code and has a branch offset of 8 bits.

Function	Definition	ARMv8	MIPS64	PowerPC	SPARC v.9
Load/store multiple registers	Loads or stores 2 or more registers	`Load pair, store pair`		`Load store multiple (<=31 registers),`	
Cache manipulation and prefetch	Modifies status of a cache line or does a prefetch	`Prefetch`	`CACHE, PREFETCH`	`Prefetch`	`Prefetch`

Figure K.17 Data transfer instructions not found in RISC-V core but found in two or more of the five desktop architectures. SPARC requires memory accesses to be aligned, while the other architectures support unaligned access, albeit, often with major performance penalties. The other architectures do not require alignment, but may use slow mechanisms to handle unaligned accesses.MIPS provides a set of instructions to handle misaligned accesses: LDL and LDR (load double left and load double right instructions) work as a pair to load a misaligned word; the corresponding store instructions perform the inverse. The Prefetch instruction causes a cache prefetch, while CACHE provides limited user control over the cache state.

Instructions: Common Extensions beyond RV64G

Figures K.15 through K.18 list instructions not found in Figures K.9 through K.13 in the same four categories (data transfer, ALU, and control. The only significant floating point extension is the reciprocal instruction, which both MIPS64 and Power support. Instructions are put in these lists if they appear in more than one of the standard architectures. Recall that Figure K.3 on page 6 showed the address modes supported by the various instruction sets. All three processors provide more address modes than provided by RV64G. The loads and stores using these additional address modes are not shown in Figure K.17, but are effectively additional data transfer instructions. This means that ARM has 64 additional load and store instructions, while Power3 has 12, and MIPS64 and SPARVv9 each have 4.

To accelerate branches, modern processors use dynamic branch prediction (see Section 3.3). Many of these architectures in earlier versions supported delayed branches, although they have been dropped or largely eliminated in later versions

Name	Definition	ARMv8	MIPS64	PowerPC	SPARC v.9
Delayed branches	Delayed branches with/without cancellation		`BEQ, BNE, BGTZ, BLEZ, BCxEQZ, BCxNEZ`		`BPcc, A, FPBcc, A`
Conditional trap	Traps if a condition is true		`TEQ, TNE, TGE, TLT, TGEU, TLTU`	`TW, TD, TWI, TDI`	`Tcc`

Figure K.18 Control instructions not found in RV64G core but found in two or more of the other architectures. MIPS64 Release 6 has nondelayed and normal delayed branches, while SPARC v.9 has delayed branches with cancellation based on the static prediction.

of the architecture, typically by offering a nondelayed version, as the preferred conditional branch. The SPARC "annulling" branch is an optimized form of delayed branch that executes the instruction in the delay slot only if the branch is taken; otherwise, the instruction is annulled. This means the instruction at the target of the branch can safely be copied into the delay slot since it will only be executed if the branch is taken. The restrictions are that the target is not another branch and that the target is known at compile time. (SPARC also offers a nondelayed jump because an unconditional branch with the annul bit set does *not* execute the following instruction.).

In contrast to the differences among the full ISAs, the 16-bit subsets of the three embedded ISAs have essentially no significant differences other than those described in the earlier figures (e.g. size of immediate fields, uses of SP or other registers, etc.).

Now that we have covered the similarities, we will focus on the unique features of each architecture. We first cover the desktop/server RISCs, ordering them by length of description of the unique features from shortest to longest, and then the embedded RISCs.

Instructions Unique to MIPS64 R6

MIPS has gone through six generations of instruction sets. Generations 1–4 mostly added instructions. Release 6 eliminated many older instructions but also provided support for nondelayed branches and misaligned data access. Figure K.19 summarizes the unique instructions in MIPS64 R6.

Instruction class	Instruction name(s)	Function
ALU	Byte align	Take a pair of registers and extract a word or double word of bytes. Used to implement unaligned byte copies.
	Align Immediate to PC	Adds the upper 16 bits of the PC to an immediate shifted left 16 bits and puts the result in a register; Used to get a PC-relative address.
	Bit swap	Reverses the bits in each byte of a register.
	No-op and link	Puts the value of PC+8 into a register
	Logical NOR	Computes the NOR of 2 registers
Control transfer	Branch and Link conditional	Compares a register to 0 and does a branch if condition is true; places the return address in the link register.
	Jump indexed, Jump and link indexed	Adds an offset and register to get new PC, w/wo link address

Figure K.19 Additional instructions provided MIPS64 R6. In addition, there are several instructions for supporting virtual machines, most are privileged.

Instructions Unique to SPARC v.9

Several features are unique to SPARC. We review the major figures and then summarize those and small differences in a figure.

Register Windows

The primary unique feature of SPARC is register windows, an optimization for reducing register traffic on procedure calls. Several banks of registers are used, with a new one allocated on each procedure call. Although this could limit the depth of procedure calls, the limitation is avoided by operating the banks as a circular buffer. The knee of the cost-performance curve seems to be six to eight banks; programs with deeper call stacks, would need to save and restore the registers to memory.

SPARC can have between 2 and 32 windows, typically using 8 registers each for the globals, locals, incoming parameters, and outgoing parameters. (Given that each window has 16 unique registers, an implementation of SPARC can have as few as 40 physical registers and as many as 520, although most have 128 to 136, so far.) Rather than tie window changes with call and return instructions, SPARC has the separate instructions `SAVE` and `RESTORE`. `SAVE` is used to "save" the caller's window by pointing to the next window of registers in addition to performing an add instruction. The trick is that the source registers are from the caller's window of the addition operation, while the destination register is in the callee's window. SPARC compilers typically use this instruction for changing the stack pointer to allocate local variables in a new stack frame. `RESTORE` is the inverse of `SAVE`, bringing back the caller's window while acting as an add instruction, with the source registers from the callee's window and the destination register in the caller's window. This automatically deallocates the stack frame. Compilers can also make use of it for generating the callee's final return value.

The danger of register windows is that the larger number of registers could slow down the clock rate. This was not the case for early implementations. The SPARC architecture (with register windows) and the MIPS R2000 architecture (without) have been built in several technologies since 1987. For several generations the SPARC clock rate has not been slower than the MIPS clock rate for implementations in similar technologies, probably because cache access times dominate register access times in these implementations. With the advent of multiple issue, which requires many more register ports, as will as register renaming or reorder buffers, register windows posed a larger penalty.Register windows were a feature of the original Berkeley RISC designs, and their inclusion in SPARC was inspired by those designs. Tensilica is the only other major architecture in use today employs them, and they were not included in the RISC-V ISA.

Fast Traps

SPARCv9 includes support to make traps fast. It expands the single level of traps to at least four levels, allowing the window overflow and underflow trap handlers to be interrupted. The extra levels mean the handler does not need to check for page faults or

misaligned stack pointers explicitly in the code, thereby making the handler faster. Two new instructions were added to return from this multilevel handler: RETRY (which retries the interrupted instruction) and DONE (which does not). To support user-level traps, the instruction RETURN will return from the trap in nonprivileged mode.

Support for LISP and Smalltalk

The primary remaining arithmetic feature is tagged addition and subtraction. The designers of SPARC spent some time thinking about languages like LISP and Smalltalk, and this influenced some of the features of SPARC already discussed: register windows, conditional trap instructions, calls with 32-bit instruction addresses, and multi-word arithmetic (see Taylor et al. [1986] and Ungar et al. [1984]). A small amount of support is offered for tagged data types with operations for addition, subtraction, and hence comparison. The two least-significant bits indicate whether the operand is an integer (coded as 00), so TADDcc and TSUBcc set the overflow bit if either operand is not tagged as an integer or if the result is too large. A subsequent conditional branch or trap instruction can decide what to do. (If the operands are not integers, software recovers the operands, checks the types of the operands, and invokes the correct operation based on those types.) It turns out that the misaligned memory access trap can also be put to use for tagged data, since loading from a pointer with the wrong tag can be an invalid access. Figure K.20 shows both types of tag support.

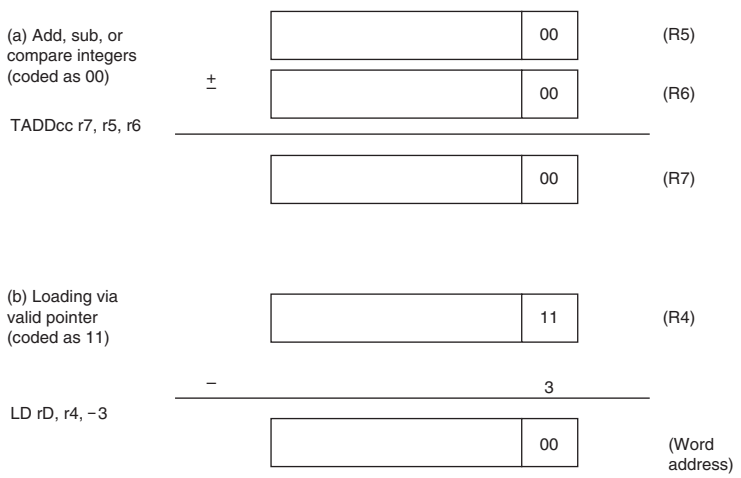


Figure K.20 SPARC uses the two least-significant bits to encode different data types for the tagged arithmetic instructions. (a) Integer arithmetic, which takes a single cycle as long as the operands and the result are integers. (b) The misaligned trap can be used to catch invalid memory accesses, such as trying to use an integer as a pointer. For languages with paired data like LISP, an offset of –3 can be used to access the even word of a pair (CAR) and +1 can be used for the odd word of a pair (CDR).

Instruction class	Instruction name(s)	Function
Data transfer	SAVE, RESTORE	Save or restore a register window
	Nonfaulting load	Version of load instructions that do not generate faults on address exceptions; allows speculation for loads.
ALU	Tagged add, Tagged subtract, with and without trap	Perform a tagged add/subtract, set condition codes, optionally trap.
Control transfer	Retry, Return, and Done	To provide handling for traps.
Floating Point Instructions	FMOVcc	Conditional move between FP registers based on integer or FP condition codes.

Figure K.21 Additional instructions provided in SPARCv9. Although register windows are by far the most significant distinction, they do not require many instructions!

Figure K.21 summarizes the additional instructions mentioned above as well as several others.

Instructions Unique to ARM

Earlier versions of the ARM architecture (ARM v6 and v7) had a number of unusual features including conditional execution of all instructions, and making the PC a general purpose register. These features were eliminated with the arrival of ARMv8 (in both the 32-bit and 64-bit ISA). What remains, however, is much of the complexity, at least in terms of the size of the instruction set. As Figure K.3 on page 6 shows, ARM has the most addressing modes, including all those listed in the table; remember that these addressing modes add dozens of load/store instructions compared to RVG, even though they are not listed in the table that follows. As Figure K.6 on page 8 shows, ARMv8 also has by far the largest number of different instruction formats, which reflects a variety of instructions, as well as the different addressing modes, some of which are applicable to some loads and stores but not others.

Most ARMv8 ALU instructions allow the second operand to be shifted before the operation is completed. This extends the range of immediates, but operand shifting is not limited to immediates. The shift options are shift left logical, shift right logical, shift right arithmetic, and rotate right. In addition, as in Power3, most ALU instructions can optionally set the condition flags. Figure K.22 includes the additional instructions, but does not enumerate all the varieties (such as optional setting of the condition flags); see the caption for more detail. While conditional execution of all instructions was eliminated, ARMv8 provides a number of conditional instructions beyond the conditional move and conditional set, mentioned earlier.

Instruction class	Instruction name(s)	Function
Data transfer	Load/Store Non-temporal pair	Loads/stores a pair of registers with an indication not to cache the data. Base + scaled offset addressing mode only.
ALU	Add Extended word/double word	Add 2 registers after left shifting the second register operand and extending it.
	Add with shift; add immediate with shift	Adds with shift of the second operand.
	Address of page	Computes the address of a page based on PC (similar to ADDUIPC, which is the same as ADR in ARMv8)
	AND, OR, XOR, XOR NOT shifted register	Logical operation on a register and a shifted register.
	Bit field clear shifted	Shift operand, invert and AND with another operand
	Conditional compare, immediate, negative, negative immediate	If condition true, then set condition flags to compare result, otherwise leave condition flags untouched.
	Conditional increment, invert, negate	If condition then set destination to increment/invert/negate of source register
	CRC	Computes a CRC checksum: byte, word, halfword, double
	Multiply add, subtract	Integer multiply-add or multiply-subtract
	Multiply negate	Negate the product of two integers; word & double word
	Move immediate or inverse	Replace 16-bits in a register with immediate, possibly shifted
	Reverse bit order	Reverses the order of bits in a register
	Signed bit field move	Move a signed bit field; sign extend to left; zero extend to right
	Unsigned divide, multiple, multiply negate, multiply-add, multiply-sub	Unsigned versions of the basic instructions
Control transfer	CBNZ, CBZ	Compare branch =/!= 0, indicating this is not a call or return.
	TBNZ, TBZ	Tests bit in a register =/!= 0, and branch.

Figure K.22 Additional instructions provided in ARMv8, the AArch64 instruction set. Unless noted the instruction is available in a word and double word format, if there is a difference. Most of the ALU instructions can optionally set the condition codes; these are not included as separate instructions here or in earlier tables.

Instructions Unique to Power3

Power3 is the result of several generations of IBM commercial RISC machines—IBM RT/PC, IBM Power1, and IBM Power2, and the PowerPC development, undertaken primarily by IBM and Motorola. First, we describe branch registers and the support for loop branches. Figure K.23 then lists the other instructions provided only in Power3.

Instruction class	Instruction name(s)	Function
Data transfer	LHBRX, LWBRX, LDBRX	Loads a halfword/word/double word but reverses the byte order.
	SHBRX, SWBRX, SDBRX	Stores a halfword/word/double word but reverses the byte order
	LDQ, STQ	Load/store quadword to a register pair.
ALU	DRAN	Generate a random number in a register
	CMPB	Compares the individual bytes in a register and sets another register byte by byte.
	CMPRB	Compares a byte (x) against two other bytes (y and z) and sets a condition to indicate if the value of y<=x<=z.
	CRAND, CRNAND, CROR, CRNOR, CRXOR, CREQV, CORC, CRANDC	Logical operations on the condition register.
	ZCMPEQB	Compares a byte (x) against the eight bytes in another register and sets a condition to indicate if x = any of the 8 bytes
	EXTSWSL	Sign extend word and shift left
	POPCNTB, POPCNTW POPCNTD	Count number of 1s in each byte and place total in another byte. Count number of 1s in each word and place total in another word. Count number of 1s in a double word.
	PRTYD, PRTYW	Compute byte parity of the bytes in a word or double word.
	BPERMD	Permutes the bits in a double word, producing a permuted byte.
	CDTBCD, CDCBCD, ADDGCS	Instructions to convert from/to binary coded decimal (BCD) or operate on two BCD values
Control transfer	BA, BCA	Branches to an absolute address, conditionally & unconditionally
	BCCTR, BCCTRL	Conditional branch to address in the count register, w/wo linking
	BCTSAR, BCTARL	Conditional branch to address in the Branch Target Address register, w/wo linking
	CLRBHRB, MFBHRBE	Manipulate the branch history rolling buffer.
Floating Point Instructions	FRSQRTE	Computes an estimate of reciprocal of the square root,
	FTDIV, FTSQRT	Tests for divide by zero or square of negative number
	FSEL	Test register against zero and select one of two operands to move
	Decimal floating point operations	A series of 48 instructions to support decimal floating point.

Figure K.23 Additional instructions provided in Power3. Rotate instructions have two forms: one that sets a condition register and one that does not. There are a set of string instructions that load up to 32 bytes from an arbitrary address to a set of registers. These instructions will be phased out in future implementations, and hence we just mention them here.

Branch Registers: Link and Counter

Rather than dedicate one of the 32 general-purpose registers to save the return address on procedure call, Power3 puts the address into a special register called the *link register*. Since many procedures will return without calling another procedure, link doesn't always have to be saved away. Making the return address a special register makes the return jump faster since the hardware need not go through the register read pipeline stage for return jumps.

In a similar vein, Power3 has a *count register* to be used in for loops where the program iterates for a fixed number of times. By using a special register the branch hardware can determine quickly whether a branch based on the count register is likely to branch, since the value of the register is known early in the execution cycle. Tests of the value of the count register in a branch instruction will automatically decrement the count register.

Given that the count register and link register are already located with the hardware that controls branches, and that one of the problems in branch prediction is getting the target address early in the pipeline (see Appendix C), the Power architects decided to make a second use of these registers. Either register can hold a target address of a conditional branch. Thus, PowerPC supplements its basic conditional branch with two instructions that get the target address from these registers (`BCLR`, `BCCTR`). Figure K.23 shows the several dozen instructions that have been added; note that there is an extensive facility for decimal floating point, as well.

Instructions: Multimedia Extensions of the Desktop/Server RISCs

Support for multimedia and graphics operations developed in several phases, beginning in 1996 with Intel MMX, MIPS MDMX, and SPARC VIS. As described in Section 4.3, which we assume the reader has read, these extensions allowed a register to be treated as multiple independent small integers (8 or 16 bits long) with arithmetic and logical operations done in parallel on all the items in a register. These initial SIMD extensions, sometimes called packed SIMD, were further developed after 2000 by widening the registers, partially or totally separating them from the general purpose or floating pointer registers, and by adding support for parallel floating point operations. RISC-V has reserved an extension for such packed SIMD instructions, but the designers have opted to focus on a true vector extension for the present. The vector extension RV64V is a vector architecture, and, as Section 4.3 points out, a true vector instruction set is considerably more general, and can typically perform the operations handled by the SIMD extensions using vector operations.

Figure K.24 shows the basic structure of the SIMD extensions in ARM, MIPS, Power, and SPARC. Note the difference in how the SIMD "vector registers" are structured: repurposing the floating point, extending the floating point, or adding additional registers. Other key differences include support for FP as well as integers,

	ARMv8	MIPS64 R6	Power v3.0	SPARCv9
Name of ISA extension	Advanced SIMD	MIPS64 SIMD Architecture	Vector Facility	VIS
Date of Current Version	2011	2012	2015	1995
Vector registers: # x size	32 x 128 bits	32 x 128 bits	32 x 128 bits	32 x 64 bits
Use GP/FP registers or independent set	extend FP registers doubling width	extend FP registers doubling width	Independent	Same as FP registers
Integer data sizes	8, 16, 32, 64	8, 16, 32, 64	8, 16, 32, 64, 128	8,16, 32
FP data sizes	32, 64	32, 64	32	
Immediates for integer and logical operations		5 bits arithmetic 8 bits logical		

Figure K.24 Structure of the SIMD extensions intended for multimedia support. In addition to the vector facility, The last row states whether the SIMD instruction set supports immediates (e.g, add vector immediate or AND vector immediate); the entry states the size of immediates for those ISAs that support them. Note that the fact that an immediate is present is encoded in the opcode space, and could alternatively be added to the next table as additional instructions. Power 3 has an optional Vector-Scalar Extension. The Vector-Scalar Extension defines a set of vector registers that overlap the FP and normal vector registers, eliminating the need to move data back and forth to the vector registers. It also supports double precision floating point operations.

support for 128-bit integers, and provisions for immediate fields as operands in integer and logical operations. Standard load and store instructions are used for moving data from the SIMD registers to memory with special extensions to handle moving less than a full SIMD register. SPARC VIS, which was one of the earliest ISA extensions for graphics, is much more limited: only add, subtract, and multiply are included, there is no FP support, and only limited instructions for bit element operations; we include it in Figure K.24 but will not be going into more detail.

Figure K.25 shows the arithmetic instructions included in these SIMD extensions; only those appearing in at least two extensions are included. MIPS SIMD includes many other instructions, as does the Power 3 Vector-Scalar extension, which we do not cover. One frequent feature not generally found in general-purpose microprocessors is saturating operations. Saturation means that when a calculation overflows the result is set to the largest positive number or most negative number, rather than a modulo calculation as in two's complement arithmetic. Commonly found in digital signal processors (see the next subsection), these saturating operations are helpful in routines for filtering. Another common extension are instructions for accumulating values within a single register; the dot product instruction an the maximum/minimum instructions are typical examples.

In addition to the arithmetic instructions, the most common additions are logical and bitwise operations and instructions for doing version of permutations and packing elements into the SIMD registers. These additions are summarized in Figure K.26, Lastly, all three extensions support SIMD FP operations, as summarized in Figure K.27.

Instruction category	ARM Advanced SIMD	MIPS SIMD	Power Vector Facility
Add/subtract	16B, 8H, 4W; 2 D	16B, 8H; 4W; 2 D	16B, 8H, 4W, 2 D, Q
Saturating add/sub	16B, 8H, 4W; 2 D	16B, 8H; 4W; 2 D	16B, 8H, 4W, 2 D, Q
Absolute value of difference	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Adjacent add & subtract (pairwise)	16B, 8H, 4W	16B, 8H, 4W	16B, 8H, 4W; 2 D
Average		16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Dot product add, dot product subtract	16B, 8H, 4W	16B, 8H, 4W	16B, 8H, 4W; 2 D
Divide: signed, unsigned	16B, 8H, 4W	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Multiply: signed, unsigned	16B, 8H, 4W	16B, 8H, 4W	16B, 8H, 4W; 2 D
Multiply add, multiply subtract	16B, 8H, 4W	16B, 8H, 4W	16B, 8H, 4W; 2 D
Maximum, signed & unsigned	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Minimum, signed & unsigned	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Modulo, signed & unsigned		16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Compare equal	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Compare <, <=, signed, unsigned	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q

Figure K.25 Summary of arithmetic SIMD instructions. B stands for byte (8 bits), H for half word (16 bits), and W for word (32 bits), D for double word (64 bits), and Q for quad word (128 bits). Thus, 8B means an operation on 8 bytes in a single instruction. Note that some instructions–such as adjacent add/subtract, or multiply–produce results that are twice the width of the inputs (e.g. multiply on 16 bytes produces 8 halfword results). Dot product is a multiply and accumulate. The SPARC VIS instructions are aimed primarily at graphics and are structured accordingly.

Instruction category	ARM Advanced SIMD	MIPS SIMD	Power Vector Facility
Shift right/left, logical, arithmetic	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q	16B, 8H, 4W; 2 D; Q
Count leading or trailing zeros	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
and/or/xor	Q	Q	Q
Bit insert & extract	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Population count		16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D; Q
Interleave even/odd, left/right		16B, 8H, 4W; 2 D	6B, 8H, 4W; 2 D
Pack even/odd		16B, 8H, 4W; 2 D	6B, 8H, 4W; 2 D
Shuffle		16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D
SPLAT		16B, 8H, 4W; 2 D	16B, 8H, 4W; 2 D

Figure K.26 Summary of logical, bitwise, permute, and pack/unpack instructions, using the same format as the previous figure. When there is a single operand the instruction applies to the entire register; for logical operations there is no difference.Interleave puts together the elements (all even, odd, leftmost or rightmost) from two different registers to create one value; it can be used for unpacking. Pack moves the even or odd elements from two different registers to the leftmost and rightmost halves of the result. Shuffle creates a from two registers based on a mask that selects which source for each item. SPLAT copies a value into each item in a register.

Instruction category	ARM Advanced SIMD	MIPS SIMD	Power Vector Facility
FP add, subtract, multiply, divide	4W, 2D	4W, 2D	4W, 2D
FP multiply add/subtract	4W, 2D	4W, 2D	4W, 2D
FP maximum/minimum	4W, 2D	4W, 2D	4W, 2D
FP SQRT and 1/SQRT	4W, 2D	4W, 2D	4W, 2D
FP Compare	4W, 2D	4W, 2D	4W, 2D
FP Convert to/from integer	4W, 2D	4W, 2D	4W, 2D

Figure K.27 Summary of floating point, using the same format as the previous figure.

Instructions: Digital Signal-Processing Extensions of the Embedded RISCs

Both Thumb2 and microMIPS32 provide instructions for DSP (Digital Signal Processing) and multimedia operations. In Thumb2, these are part of the core instruction set; in microMIPS32, they are part of the DSP extension. These extensions, which are encoded as 32-bit instructions, are less extensive than the multimedia and graphics support provided in the SIMD/Vector extensions of MIPS64 or ARMv8 (AArch64). Like those more comprehensive extensions, the ones in Thumb2 and microMIPS32 also rely on packed SIMD, but they use the existing integer registers, with a small extension to allow a wide accumulator, and only operate on integer data. RISC-V has specified that the "P" extension will support packed integer SIMD using the floating point registers, but at the time of publication, the specification was not completed.

DSP operations often include linear algebra functions and operations such as convolutions; these operations produce intermediate results that will be larger than the inputs. In Thumb2, this is handled by a set of operations that produce 64-bit results using a pair of integer registers. In microMIPS32 DSP, there are 4 64-bit accumulator registers, including the Hi-Lo register, which is already exists for doing integer multiply and divide. Both architectures provide parallel arithmetic using bytes, halfwords, and words, as in the multimedia extensions in ARMv8 and MIPS64. In addition, the MIPS DSP extension handles fractional data, such data is heavily used in DSP operations. Fractional data items have a sign bit and the remaining bits are used to represent the fraction, providing a range of values from -1.0 to 0.9999 (in decimal). MIPS DSP supports two fractional data sizes Q15 and Q31 each with one sign bit and 15 or 31 bits of fraction.

Figure K.28 shows the common operations using the same notation as was used in Figure K.25. Remember that the basic 32-bit instruction set provides additional functionality, including basic arithmetic, logical, and bit manipulation.

Function	Thumb-2	microMIPS32 DSP
Add/Subtract	4B, 2H	4B, 2Q15
Add /Subtract with saturation	4B, 2H	4B, 2Q15, Q31
Add/Subtract with Exchange (exchanges halfwords in rt, then adds first halfword and subtracts second) with optional saturation	2H	
Reduce by add (sum the values)		4B
Absolute value		2Q15, Q31
Precision reduce/increase (reduces or increases the precision of a value)		2B, Q15, 2Q15, Q31
Shifts: left, right, logical & arithmetic, with optional saturation		4B, 2H
Multiply	2H	2B, 2H, 2Q15
Multiply add/subtract (to GPR or accumulator register in MIPS)	2H	2Q15
Complex multiplication step (2 multiplies and addition/subtraction)	2H	2Q15
Multiply and accumulate (by addition or subtraction)	2H	Q15, Q31
Replicate bits		B, H
Compare: =, <, <=, sets condition field		4B, 2H
Pick (use condition bits to choose bytes or halfwords from two operands)		4B, 2H
Pack choosing a halfword from each operand		H
Extract		Q63
Move from/to accumulator		DW

Figure K.28 Summary of two embedded RISC DSP operations, showing the data types for each operation. A blank indicates that the operation is not supported as a single instruction. Byte quantities are usually unsigned. Complex multiplication step implements multiplication of complex numbers where each component is a Q15 value. ARM uses its standard condition register, while MIPS adds a set of condition bits as part of the state in the DSP extension.

Concluding Remarks

This survey covers the addressing modes, instruction formats, and almost all the instructions found in 8 RISC architectures. Although the later sections concentrate on the differences, it would not be possible to cover 8 architectures in these few pages if there were not so many similarities. In fact, we would guess that more than 90% of the instructions executed for any of these architectures would be found in Figures K.9 through K.13. To contrast this homogeneity, Figure K.29 gives a summary for four architectures from the 1970s in a format similar to that shown in Figure K.1. (Since it would be impossible to write a single section in this style for those architectures, the next three sections cover the 80x86, VAX, and IBM 360/370.) In the history of computing, there has never been such widespread agreement on computer architecture as there has been since the RISC ideas emerged in the 1980s.

	IBM 360/370	Intel 8086	Motorola 68000	DEC VAX
Date announced	1964/1970	1978	1980	1977
Instruction size(s) (bits)	16, 32, 48	8, 16, 24, 32, 40, 48	16, 32, 48, 64, 80	8, 16, 24, 32, ... , 432
Addressing (size, model)	24 bits, flat/ 31 bits, flat	4 + 16 bits, segmented	24 bits, flat	32 bits, flat
Data aligned?	Yes 360/No 370	No	16-bit aligned	No
Data addressing modes	2/3	5	9	=14
Protection	Page	None	Optional	Page
Page size	2 KB & 4 KB	—	0.25 to 32 KB	0.5 KB
I/O	Opcode	Opcode	Memory mapped	Memory mapped
Integer registers (size, model, number)	16 GPR × 32 bits	8 dedicated data × 16 bits	8 data and 8 address × 32 bits	15 GPR × 32 bits
Separate floating-point registers	4 × 64 bits	Optional: 8 × 80 bits	Optional: 8 × 80 bits	0
Floating-point format	IBM (floating hexadecimal)	IEEE 754 single, double, extended	IEEE 754 single, double, extended	DEC

Figure K.29 Summary of four 1970s architectures. Unlike the architectures in Figure K.1, there is little agreement between these architectures in any category. (See Section K.3 for more details on the 80x86 and Section K.4 for a description of the VAX.)

K.3 The Intel 80x86

Introduction

MIPS was the vision of a single architect. The pieces of this architecture fit nicely together and the whole architecture can be described succinctly. Such is not the case of the 80x86: It is the product of several independent groups who evolved the architecture over 20 years, adding new features to the original instruction set as you might add clothing to a packed bag. Here are important 80x86 milestones:

- 1978—The Intel 8086 architecture was announced as an assembly language–compatible extension of the then-successful Intel 8080, an 8-bit microprocessor. The 8086 is a 16-bit architecture, with all internal registers 16 bits wide. Whereas the 8080 was a straightforward accumulator machine, the 8086 extended the architecture with additional registers. Because nearly every register has a dedicated use, the 8086 falls somewhere between an accumulator machine and a general-purpose register machine, and can fairly be called an *extended accumulator* machine.
- 1980—The Intel 8087 floating-point coprocessor is announced. This architecture extends the 8086 with about 60 floating-point instructions. Its architects rejected extended accumulators to go with a hybrid of stacks and registers,

essentially an *extended stack* architecture: A complete stack instruction set is supplemented by a limited set of register-memory instructions.

- 1982—The 80286 extended the 8086 architecture by increasing the address space to 24 bits, by creating an elaborate memory mapping and protection model, and by adding a few instructions to round out the instruction set and to manipulate the protection model. Because it was important to run 8086 programs without change, the 80286 offered a *real addressing mode* to make the machine look just like an 8086.
- 1985—The 80386 extended the 80286 architecture to 32 bits. In addition to a 32-bit architecture with 32-bit registers and a 32-bit address space, the 80386 added new addressing modes and additional operations. The added instructions make the 80386 nearly a general-purpose register machine. The 80386 also added paging support in addition to segmented addressing (see Chapter 2). Like the 80286, the 80386 has a mode to execute 8086 programs without change.

This history illustrates the impact of the "golden handcuffs" of compatibility on the 80x86, as the existing software base at each step was too important to jeopardize with significant architectural changes. Fortunately, the subsequent 80486 in 1989, Pentium in 1992, and P6 in 1995 were aimed at higher performance, with only four instructions added to the user-visible instruction set: three to help with multiprocessing plus a conditional move instruction.

Since 1997 Intel has added hundreds of instructions to support multimedia by operating on many narrower data types within a single clock (see Appendix A). These SIMD or vector instructions are primarily used in hand-coded libraries or drivers and rarely generated by compilers. The first extension, called MMX, appeared in 1997. It consists of 57 instructions that pack and unpack multiple bytes, 16-bit words, or 32-bit double words into 64-bit registers and performs shift, logical, and integer arithmetic on the narrow data items in parallel. It supports both saturating and nonsaturating arithmetic. MMX uses the registers comprising the floating-point stack and hence there is no new state for operating systems to save.

In 1999 Intel added another 70 instructions, labeled SSE, as part of Pentium III. The primary changes were to add eight separate registers, double their width to 128 bits, and add a single-precision floating-point data type. Hence, four 32-bit floating-point operations can be performed in parallel. To improve memory performance, SSE included cache prefetch instructions plus streaming store instructions that bypass the caches and write directly to memory.

In 2001, Intel added yet another 144 instructions, this time labeled SSE2. The new data type is double-precision arithmetic, which allows pairs of 64-bit floating-point operations in parallel. Almost all of these 144 instructions are versions of existing MMX and SSE instructions that operate on 64 bits of data in parallel. Not only does this change enable multimedia operations, but it also gives the compiler a different target for floating-point operations than the unique stack architecture. Compilers can choose to use the eight SSE registers as floating-point registers as found in the RISC machines. This change has boosted performance on the Pentium 4, the first microprocessor to include SSE2 instructions. At the time of

announcement, a 1.5 GHz Pentium 4 was 1.24 times faster than a 1 GHz Pentium III for SPECint2000(base), but it was 1.88 times faster for SPECfp2000(base).

In 2003 a company other than Intel enhanced the IA-32 architecture this time. AMD announced a set of architectural extensions to increase the address space for 32 to 64 bits. Similar to the transition from 16- to 32-bit address space in 1985 with the 80386, AMD64 widens all registers to 64 bits. It also increases the number of registers to sixteen and has 16 128-bit registers to support XMM, AMD's answer to SSE2. Rather than expand the instruction set, the primary change is adding a new mode called *long mode* that redefines the execution of all IA-32 instructions with 64-bit addresses. To address the larger number of registers, it adds a new prefix to instructions. AMD64 still has a 32-bit mode that is backwards compatible to the standard Intel instruction set, allowing a more graceful transition to 64-bit addressing than the HP/Intel Itanium. Intel later followed AMD's lead, making almost identical changes so that most software can run on either 64-bit address version of the 80x86 without change.

Whatever the artistic failures of the 80x86, keep in mind that there are more instances of this architectural family than of any other server or desktop processor in the world. Nevertheless, its checkered ancestry has led to an architecture that is difficult to explain and impossible to love.

We start our explanation with the registers and addressing modes, move on to the integer operations, then cover the floating-point operations, and conclude with an examination of instruction encoding.

80x86 Registers and Data Addressing Modes

The evolution of the instruction set can be seen in the registers of the 80x86 (Figure K.30). Original registers are shown in black type, with the extensions of the 80386 shown in a lighter shade, a coloring scheme followed in subsequent figures. The 80386 basically extended all 16-bit registers (except the segment registers) to 32 bits, prefixing an "E" to their name to indicate the 32-bit version. The arithmetic, logical, and data transfer instructions are two-operand instructions that allow the combinations shown in Figure K.31.

To explain the addressing modes, we need to keep in mind whether we are talking about the 16-bit mode used by both the 8086 and 80286 or the 32-bit mode available on the 80386 and its successors. The seven data memory addressing modes supported are

- Absolute
- Register indirect
- Based
- Indexed
- Based indexed with displacement
- Based with scaled indexed
- Based with scaled indexed and displacement

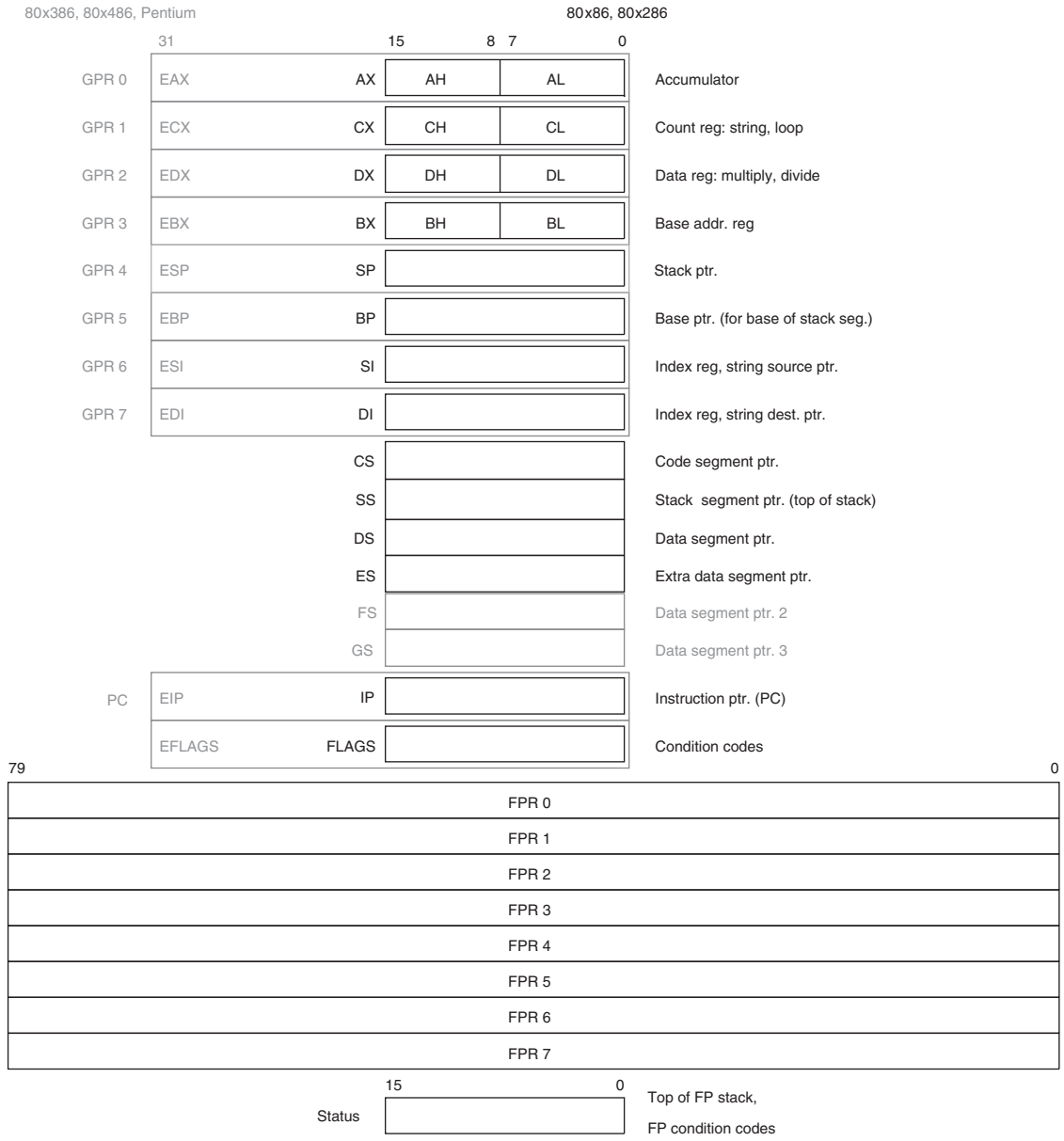


Figure K.30 The 80x86 has evolved over time, and so has its register set. The original set is shown in black and the extended set in gray. The 8086 divided the first four registers in half so that they could be used either as one 16-bit register or as two 8-bit registers. Starting with the 80386, the top eight registers were extended to 32 bits and could also be used as general-purpose registers. The floating-point registers on the bottom are 80 bits wide, and although they look like regular registers they are not. They implement a stack, with the top of stack pointed to by the status register. One operand must be the top of stack, and the other can be any of the other seven registers below the top of stack.

Source/destination operand type	Second source operand
Register	Register
Register	Immediate
Register	Memory
Memory	Register
Memory	Immediate

Figure K.31 Instruction types for the arithmetic, logical, and data transfer instructions. The 80x86 allows the combinations shown. The only restriction is the absence of a memory-memory mode. Immediates may be 8, 16, or 32 bits in length; a register is any one of the 14 major registers in Figure K.30 (not IP or FLAGS).

Displacements can be 8 or 32 bits in 32-bit mode, and 8 or 16 bits in 16-bit mode. If we count the size of the address as a separate addressing mode, the total is 11 addressing modes.

Although a memory operand can use any addressing mode, there are restrictions on what registers can be used in a mode. The section "80x86 Instruction Encoding" on page K-11 gives the full set of restrictions on registers, but the following description of addressing modes gives the basic register options:

- *Absolute*—With 16-bit or 32-bit displacement, depending on the mode.
- *Register indirect*—`BX`, `SI`, `DI` in 16-bit mode and `EAX`, `ECX`, `EDX`, `EBX`, `ESI`, and `EDI` in 32-bit mode.
- *Based mode with 8-bit or 16-bit/32-bit displacement*—`BP`, `BX`, `SI`, and `DI` in 16-bit mode and `EAX`, `ECX`, `EDX`, `EBX`, `ESI`, and `EDI` in 32-bit mode. The displacement is either 8 bits or the size of the address mode: 16 or 32 bits. (Intel gives two different names to this single addressing mode, *based* and *indexed*, but they are essentially identical and we combine them. This book uses indexed addressing to mean something different, explained next.)
- *Indexed*—The address is the sum of two registers. The allowable combinations are `BX+SI`, `BX+DI`, `BP+SI`, and `BP+DI`. This mode is called *based indexed* on the 8086. (The 32-bit mode uses a different addressing mode to get the same effect.)
- *Based indexed with 8- or 16-bit displacement*—The address is the sum of displacement and contents of two registers. The same restrictions on registers apply as in indexed mode.
- *Base plus scaled indexed*—This addressing mode and the next were added in the 80386 and are only available in 32-bit mode. The address calculation is

$$\text{Base register} + 2^{\text{Scale}} \times \text{Index} \times \text{register}$$

where *Scale* has the value 0, 1, 2, or 3; *Index register* can be any of the eight 32-bit general registers except ESP; and *Base register* can be any of the eight 32-bit general registers.

- *Base plus scaled index with 8- or 32-bit displacement*—The address is the sum of the displacement and the address calculated by the scaled mode immediately above. The same restrictions on registers apply.

The 80x86 uses Little Endian addressing.

Ideally, we would refer discussion of 80x86 logical and physical addresses to Chapter 2, but the segmented address space prevents us from hiding that information. Figure K.32 shows the memory mapping options on the generations of 80x86 machines; Chapter 2 describes the segmented protection scheme in greater detail.

The assembly language programmer clearly must specify which segment register should be used with an address, no matter which address mode is used. To save space in the instructions, segment registers are selected automatically depending on which address register is used. The rules are simple: References to instructions (IP) use the code segment register (CS), references to the stack (BP or SP) use the stack segment register (SS), and the default segment register for the other registers is the data segment register (DS). The next section explains how they can be overridden.

80x86 Integer Operations

The 8086 provides support for both 8-bit (*byte*) and 16-bit (called *word*) data types. The data type distinctions apply to register operations as well as memory accesses. The 80386 adds 32-bit addresses and data, called *double words.* Almost every operation works on both 8-bit data and one longer data size. That size is determined by the mode and is either 16 or 32 bits.

Clearly some programs want to operate on data of all three sizes, so the 80x86 architects provide a convenient way to specify each version without expanding code size significantly. They decided that most programs would be dominated by either 16- or 32-bit data, and so it made sense to be able to set a default large size. This default size is set by a bit in the code segment register. To override the default size, an 8-bit *prefix* is attached to the instruction to tell the machine to use the other large size for this instruction.

The prefix solution was borrowed from the 8086, which allows multiple prefixes to modify instruction behavior. The three original prefixes override the default segment register, lock the bus so as to perform a semaphore (see Chapter 5), or repeat the following instruction until CX counts down to zero. This last prefix was intended to be paired with a byte move instruction to move a variable number of bytes. The 80386 also added a prefix to override the default address size.

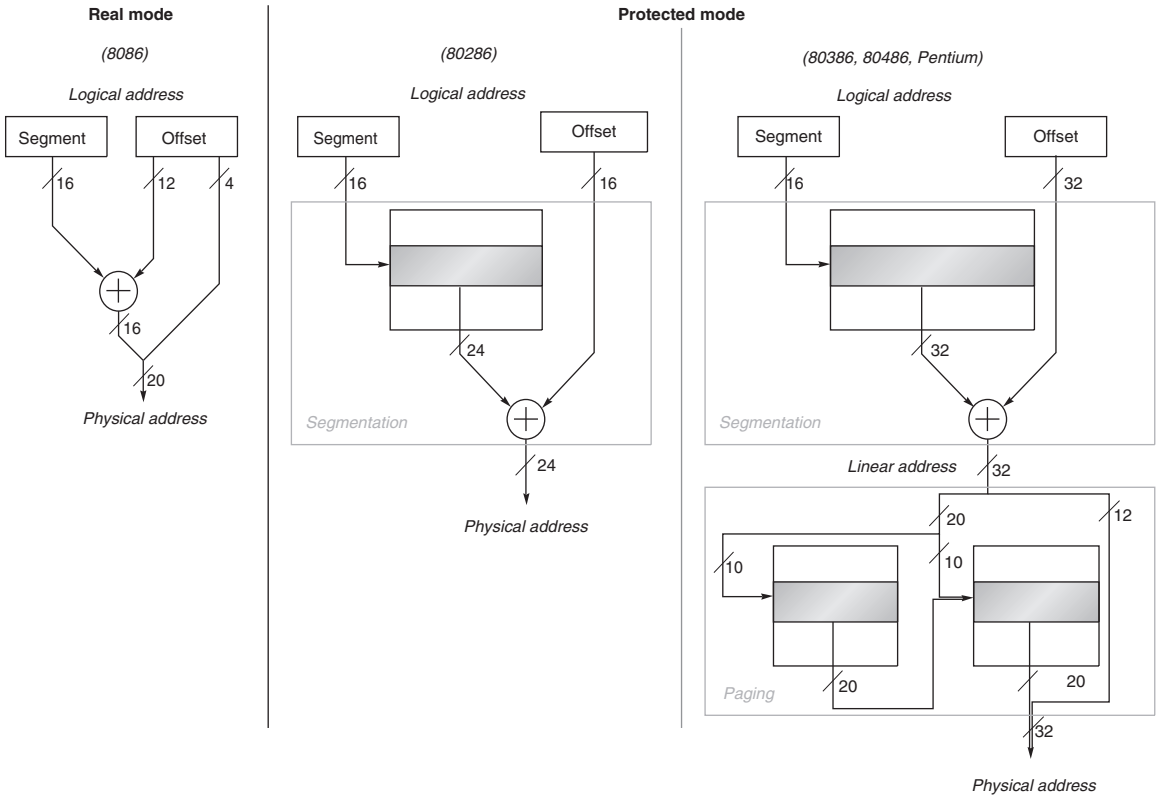


Figure K.32 The original segmented scheme of the 8086 is shown on the left. All 80x86 processors support this style of addressing, called *real mode*. It simply takes the contents of a segment register, shifts it left 4 bits, and adds it to the 16-bit offset, forming a 20-bit physical address. The 80286 (center) used the contents of the segment register to select a segment descriptor, which includes a 24-bit base address among other items. It is added to the 16-bit offset to form the 24-bit physical address. The 80386 and successors (right) expand this base address in the segment descriptor to 32 bits and also add an optional paging layer below segmentation. A 32-bit linear address is first formed from the segment and offset, and then this address is divided into two 10-bit fields and a 12-bit page offset. The first 10-bit field selects the entry in the first-level page table, and then this entry is used in combination with the second 10-bit field to access the second-level page table to select the upper 20 bits of the physical address. Prepending this 20-bit address to the final 12-bit field gives the 32-bit physical address. Paging can be turned off, redefining the 32-bit linear address as the physical address. Note that a "flat" 80x86 address space comes simply by loading the same value in all the segment registers; that is, it doesn't matter which segment register is selected.

The 80x86 integer operations can be divided into four major classes:

1. Data movement instructions, including move, push, and pop
2. Arithmetic and logic instructions, including logical operations, test, shifts, and integer and decimal arithmetic operations
3. Control flow, including conditional branches and unconditional jumps, calls, and returns
4. String instructions, including string move and string compare

Instruction	Function
JE name	if equal(CC) {IP←name};IP-128≤name≤IP+128
JMP name	IP← name
CALLF name, seg	SP←SP-2;M[SS:SP]←IP+5;SP←SP-2; M[SS:SP]←CS;IP← name;CS ←seg;
	MOVW BX,[DI+45] BX←$_{16}$M[DS:DI+45]
PUSH SI	SP←SP-2;M[SS:SP]←SI
POP DI	DI ←M[SS:SP];SP←SP+2
ADD AX,#6765	AX ←AX+6765
SHL BX,1	BX ←BX$_{1..15}$## 0
TEST DX,#42	Set CC flags with DX & 42
MOVSB	M[ES:DI]←$_{8}$M[DS:SI];DI←DI+1;SI←SI+1

Figure K.33 Some typical 80x86 instructions and their functions. A list of frequent operations appears in Figure K.34. We use the abbreviation SR:X to indicate the formation of an address with segment register SR and offset X. This effective address corresponding to SR:X is (SR<<4)+X. The CALLF saves the IP of the next instruction and the current CS on the stack.

Figure K.33 shows some typical 80x86 instructions and their functions.

The data transfer, arithmetic, and logic instructions are unremarkable, except that the arithmetic and logic instruction operations allow the destination to be either a register or a memory location.

Control flow instructions must be able to address destinations in another segment. This is handled by having two types of control flow instructions: "near" for intrasegment (within a segment) and "far" for intersegment (between segments) transfers. In far jumps, which must be unconditional, two 16-bit quantities follow the opcode in 16-bit mode. One of these is used as the instruction pointer, while the other is loaded into CS and becomes the new code segment. In 32-bit mode the first field is expanded to 32 bits to match the 32-bit program counter (EIP).

Calls and returns work similarly—a far call pushes the return instruction pointer and return segment on the stack and loads both the instruction pointer and the code segment. A far return pops both the instruction pointer and the code segment from the stack. Programmers or compiler writers must be sure to always use the same type of call *and* return for a procedure—a near return does not work with a far call, and *vice versa*.

String instructions are part of the 8080 ancestry of the 80x86 and are not commonly executed in most programs.

Figure K.34 lists some of the integer 80x86 instructions. Many of the instructions are available in both byte and word formats.

Instruction	Meaning
Control	**Conditional and unconditional branches**
JNZ, JZ	Jump if condition to IP + 8-bit offset; JNE (for JNZ) and JE (for JZ) are alternative names
JMP, JMPF	Unconditional jump—8- or 16-bit offset intrasegment (near) and intersegment (far) versions
CALL, CALLF	Subroutine call—16-bit offset; return address pushed; near and far versions
RET, RETF	Pops return address from stack and jumps to it; near and far versions
LOOP	Loop branch—decrement CX; jump to IP + 8-bit displacement if CX ¦ 0
Data transfer	**Move data between registers or between register and memory**
MOV	Move between two registers or between register and memory
PUSH	Push source operand on stack
POP	Pop operand from stack top to a register
LES	Load ES and one of the GPRs from memory
Arithmetic/logical	**Arithmetic and logical operations using the data registers and memory**
ADD	Add source to destination; register-memory format
SUB	Subtract source from destination; register-memory format
CMP	Compare source and destination; register-memory format
SHL	Shift left
SHR	Shift logical right
RCR	Rotate right with carry as fill
CBW	Convert byte in AL to word in AX
TEST	Logical AND of source and destination sets flags
INC	Increment destination; register-memory format
DEC	Decrement destination; register-memory format
OR	Logical OR; register-memory format
XOR	Exclusive OR; register-memory format
String instructions	**Move between string operands; length given by a repeat prefix**
MOVS	Copies from string source to destination; may be repeated
LODS	Loads a byte or word of a string into the A register

Figure K.34 Some typical operations on the 80x86. Many operations use register-memory format, where either the source or the destination may be memory and the other may be a register or immediate operand.

80x86 Floating-Point Operations

Intel provided a stack architecture with its floating-point instructions: loads push numbers onto the stack, operations find operands in the top two elements of the stacks, and stores can pop elements off the stack, just as the stack example in Figure A.31 on page A-4 suggests.

Intel supplemented this stack architecture with instructions and addressing modes that allow the architecture to have some of the benefits of a register-memory model. In addition to finding operands in the top two elements of the stack, one operand can be in memory or in one of the seven registers below the top of the stack.

This hybrid is still a restricted register-memory model, however, in that loads always move data to the top of the stack while incrementing the top of stack pointer and stores can only move the top of stack to memory. Intel uses the notation `ST` to indicate the top of stack, and `ST(i)` to represent the *i*th register below the top of stack.

One novel feature of this architecture is that the operands are wider in the register stack than they are stored in memory, and all operations are performed at this wide internal precision. Numbers are automatically converted to the internal 80-bit format on a load and converted back to the appropriate size on a store. Memory data can be 32-bit (single-precision) or 64-bit (double-precision) floating-point numbers, called *real* by Intel. The register-memory version of these instructions will then convert the memory operand to this Intel 80-bit format before performing the operation. The data transfer instructions also will automatically convert 16- and 32-bit integers to reals, and *vice versa*, for integer loads and stores.

The 80x86 floating-point operations can be divided into four major classes:

1. Data movement instructions, including load, load constant, and store
2. Arithmetic instructions, including add, subtract, multiply, divide, square root, and absolute value
3. Comparison, including instructions to send the result to the integer CPU so that it can branch
4. Transcendental instructions, including sine, cosine, log, and exponentiation

Figure K.35 shows some of the 60 floating-point operations. We use the curly brackets `{}` to show optional variations of the basic operations: `{I}` means there is an integer version of the instruction, `{P}` means this variation will pop one operand off the stack after the operation, and `{R}` means reverse the sense of the operands in this operation.

Not all combinations are provided. Hence,

```
F{I}SUB{R}{P}
```

represents these instructions found in the 80x86:

```
FSUB
FISUB
FSUBR
FISUBR
FSUBP
FSUBRP
```

Data transfer	Arithmetic	Compare	Transcendental
F{I}LD mem/ST(i)	F{I}ADD{P}mem/ST(i)	F{I}COM{P}{P}	FPATAN
F{I}ST{P} mem/ST(i)	F{I}SUB{R}{P}mem/ST(i)	F{I}UCOM{P}{P}	F2XM1
FLDPI	F{I}MUL{P}mem/ST(i)	FSTSW AX/mem	FCOS
FLD1	F{I}DIV{R}{P}mem/ST(i)		FPTAN
FLDZ	FSQRT		FPREM
	FABS		FSIN
	FRNDINT		FYL2X

Figure K.35 The floating-point instructions of the 80x86. The first column shows the data transfer instructions, which move data to memory or to one of the registers below the top of the stack. The last three operations push constants on the stack: pi, 1.0, and 0.0. The second column contains the arithmetic operations described above. Note that the last three operate only on the top of stack. The third column is the compare instructions. Since there are no special floating-point branch instructions, the result of the compare must be transferred to the integer CPU via the FSTSW instruction, either into the AX register or into memory, followed by an SAHF instruction to set the condition codes. The floating-point comparison can then be tested using integer branch instructions. The final column gives the higher-level floating-point operations.

There are no pop or reverse pop versions of the integer subtract instructions.

Note that we get even more combinations when including the operand modes for these operations. The floating-point add has these options, ignoring the integer and pop versions of the instruction:

FADD		Both operands are in the in stack, and the result replaces the top of stack.
FADD	ST(i)	One source operand is *i*th register below the top of stack, and the result replaces the top of stack.
FADD	ST(i),ST	One source operand is the top of stack, and the result replaces *i*th register below the top of stack.
FADD	mem32	One source operand is a 32-bit location in memory, and the result replaces the top of stack.
FADD	mem64	One source operand is a 64-bit location in memory, and the result replaces the top of stack.

As mentioned earlier SSE2 presents a model of IEEE floating-point registers.

80x86 Instruction Encoding

Saving the worst for last, the encoding of instructions in the 8086 is complex, with many different instruction formats. Instructions may vary from 1 byte, when there are no operands, to up to 6 bytes, when the instruction contains a 16-bit immediate

and uses 16-bit displacement addressing. Prefix instructions increase 8086 instruction length beyond the obvious sizes.

The 80386 additions expand the instruction size even further, as Figure K.36 shows. Both the displacement and immediate fields can be 32 bits long, two more prefixes are possible, the opcode can be 16 bits long, and the scaled index mode specifier adds another 8 bits. The maximum possible 80386 instruction is 17 bytes long.

Figure K.37 shows the instruction format for several of the example instructions in Figure K.33. The opcode byte usually contains a bit saying whether the operand is a byte wide or the larger size, 16 bits or 32 bits depending on the mode. For some instructions, the opcode may include the addressing mode and the register; this is true in many instructions that have the form `register ← register op immediate`. Other instructions use a "postbyte" or extra opcode byte, labeled "mod, reg, r/m" in Figure K.36, which contains the addressing mode information. This postbyte is used for many of the instructions that address memory. The based with scaled index uses a second postbyte, labeled "sc, index, base" in Figure K.36.

The floating-point instructions are encoded in the escape opcode of the 8086 and the postbyte address specifier. The memory operations reserve 2 bits to decide

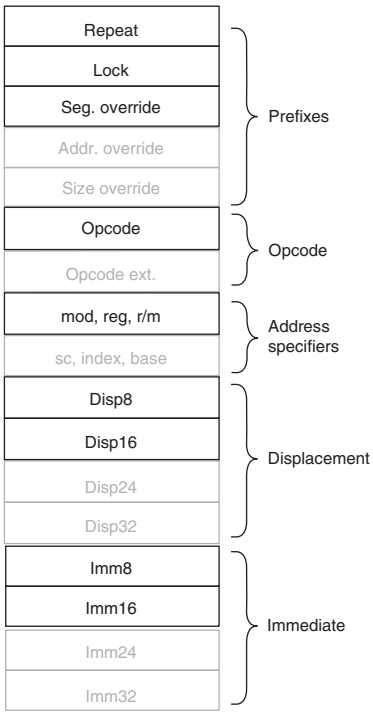


Figure K.36 The instruction format of the 8086 (black type) and the extensions for the 80386 (shaded type). Every field is optional except the opcode.

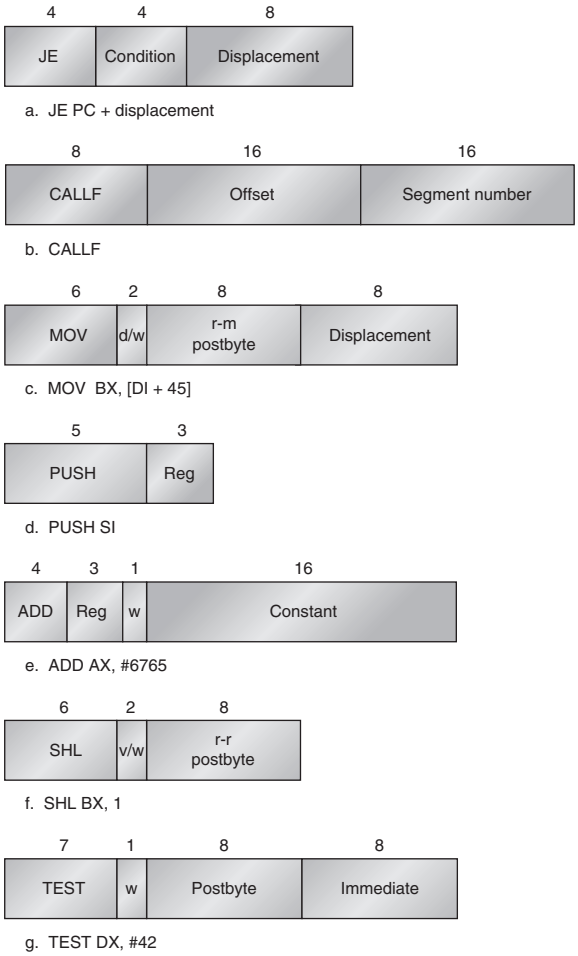


Figure K.37 Typical 8086 instruction formats. The encoding of the postbyte is shown in Figure K.38. Many instructions contain the 1-bit field w, which says whether the operation is a byte or a word. Fields of the form v/w or d/w are a d-field or v-field followed by the w-field. The d-field in MOV is used in instructions that may move to or from memory and shows the direction of the move. The field v in the SHL instruction indicates a variable-length shift; variable-length shifts use a register to hold the shift count. The ADD instruction shows a typical optimized short encoding usable only when the first operand is AX. Overall instructions may vary from 1 to 6 bytes in length.

whether the operand is a 32- or 64-bit real or a 16- or 32-bit integer. Those same 2 bits are used in versions that do not access memory to decide whether the stack should be popped after the operation and whether the top of stack or a lower register should get the result.

Alas, you cannot separate the restrictions on registers from the encoding of the addressing modes in the 80x86. Hence, Figures K.38 and K.39 show the encoding of the two postbyte address specifiers for both 16- and 32-bit mode.

		w = 1			mod = 0		mod = 1		mod = 2		
reg	w = 0	16b	32b	r/m	16b	32b	16b	32b	16b	32b	mod = 3
0	AL	AX	EAX	0	addr=BX+SI	=EAX	*same*	*same*	*same*	*same*	*same*
1	CL	CX	ECX	1	addr=BX+DI	=ECX	*addr as*	*addr as*	*addr as*	*addr as*	*as*
2	DL	DX	EDX	2	addr=BP+SI	=EDX	*mod= 0*	*mod= 0*	*mod= 0*	*mod= 0*	*reg*
3	BL	BX	EBX	3	addr=BP+SI	=EBX	*+ disp 8*	*+ disp 8*	*+ disp1 6*	*+ disp3 2*	*field*
4	AH	SP	ESP	4	addr=SI	=(si)b	SI+disp16	(sib)+disp8	SI+disp8	(sib)+disp32	"
5	CH	BP	EBP	5	addr=DI	=disp32	DI+disp8	EBP+disp8	DI+disp16	EBP+disp32	"
6	DH	SI	ESI	6	addr=disp16	=ESI	BP+disp8	ESI+disp8	BP+disp16	ESI+disp32	"
7	BH	DI	EDI	7	addr=BX	=EDI	BX+disp8	EDI+disp8	BX+disp16	EDI+disp32	"

Figure K.38 The encoding of the first address specifier of the 80x86, *mod, reg, r/m*. The first four columns show the encoding of the 3-bit reg field, which depends on the w bit from the opcode and whether the machine is in 16- or 32-bit mode. The remaining columns explain the mod and r/m fields. The meaning of the 3-bit r/m field depends on the value in the 2-bit mod field and the address size. Basically, the registers used in the address calculation are listed in the sixth and seventh columns, under mod = 0, with mod = 1 adding an 8-bit displacement and mod = 2 adding a 16- or 32-bit displacement, depending on the address mode. The exceptions are r/m = 6 when mod = 1 or mod = 2 in 16-bit mode selects BP plus the displacement; r/m = 5 when mod = 1 or mod = 2 in 32-bit mode selects EBP plus displacement; and r/m = 4 in 32-bit mode when mod ¦3 (sib) means use the scaled index mode shown in Figure K.39. When mod = 3, the r/m field indicates a register, using the same encoding as the reg field combined with the w bit.

Index		Base
0	EAX	EAX
1	ECX	ECX
2	EDX	EDX
3	EBX	EBX
4	No index	ESP
5	EBP	If mod = 0, disp32 If mod ¦ 0, EBP
6	ESI	ESI
7	EDI	EDI

Figure K.39 Based plus scaled index mode address specifier found in the 80386. This mode is indicated by the (sib) notation in Figure K.38. Note that this mode expands the list of registers to be used in other modes: Register indirect using ESP comes from Scale = 0, Index = 4, and Base = 4, and base displacement with EBP comes from Scale = 0, Index = 5, and mod = 0. The two-bit scale field is used in this formula of the effective address: Base register + 2^{Scale} × Index register.

Putting It All Together: Measurements of Instruction Set Usage

In this section, we present detailed measurements for the 80x86 and then compare the measurements to MIPS for the same programs. To facilitate comparisons among dynamic instruction set measurements, we use a subset of the SPEC92 programs. The 80x86 results were taken in 1994 using the Sun Solaris FORTRAN and C compilers V2.0 and executed in 32-bit mode. These compilers were comparable in quality to the compilers used for MIPS.

Remember that these measurements depend on the benchmarks chosen and the compiler technology used. Although we feel that the measurements in this section are reasonably indicative of the usage of these architectures, other programs may behave differently from any of the benchmarks here, and different compilers may yield different results. In doing a real instruction set study, the architect would want to have a much larger set of benchmarks, spanning as wide an application range as possible, and consider the operating system and its usage of the instruction set. Single-user benchmarks like those measured here do not necessarily behave in the same fashion as the operating system.

We start with an evaluation of the features of the 80x86 in isolation, and later compare instruction counts with those of DLX.

Measurements of 80x86 Operand Addressing

We start with addressing modes. Figure K.40 shows the distribution of the operand types in the 80x86. These measurements cover the "second" operand of the operation; for example,

```
mov EAX, [45]
```

counts as a single memory operand. If the types of the first operand were counted, the percentage of register usage would increase by about a factor of 1.5.

The 80x86 memory operands are divided into their respective addressing modes in Figure K.41. Probably the biggest surprise is the popularity of the

	Integer average	FP average
Register	45%	22%
Immediate	16%	6%
Memory	39%	72%

Figure K.40 Operand type distribution for the average of five SPECint92 programs (compress, eqntott, espresso, gcc, li) and the average of five SPECfp92 programs (doduc, ear, hydro2d, mdljdp2, su2cor).

Addressing mode	Integer average	FP average
Register indirect	13%	3%
Base + 8-bit disp.	31%	15%
Base + 32-bit disp.	9%	25%
Indexed	0%	0%
Based indexed + 8-bit disp.	0%	0%
Based indexed + 32-bit disp.	0%	1%
Base + scaled indexed	22%	7%
Base + scaled indexed + 8-bit disp.	0%	8%
Base + scaled indexed + 32-bit disp.	4%	4%
32-bit direct	20%	37%

Figure K.41 Operand addressing mode distribution by program. This chart does not include addressing modes used by branches or control instructions.

addressing modes added by the 80386, the last four rows of the figure. They account for about half of all the memory accesses. Another surprise is the popularity of direct addressing. On most other machines, the equivalent of the direct addressing mode is rare. Perhaps the segmented address space of the 80x86 makes direct addressing more useful, since the address is relative to a base address from the segment register.

These addressing modes largely determine the size of the Intel instructions. Figure K.42 shows the distribution of instruction sizes. The average number of bytes per instruction for integer programs is 2.8, with a standard deviation of 1.5, and 4.1 with a standard deviation of 1.9 for floating-point programs. The difference in length arises partly from the differences in the addressing modes: Integer programs rely more on the shorter register indirect and 8-bit displacement addressing modes, while floating-point programs more frequently use the 80386 addressing modes with the longer 32-bit displacements.

Given that the floating-point instructions have aspects of both stacks and registers, how are they used? Figure K.43 shows that, at least for the compilers used in this measurement, the stack model of execution is rarely followed. (See Section L.3 for a historical explanation of this observation.)

Finally, Figures K.44 and K.45 show the instruction mixes for 10 SPEC92 programs.

Comparative Operation Measurements

Figures K.46 and K.47 show the number of instructions executed for each of the 10 programs on the 80x86 and the ratio of instruction execution compared with that

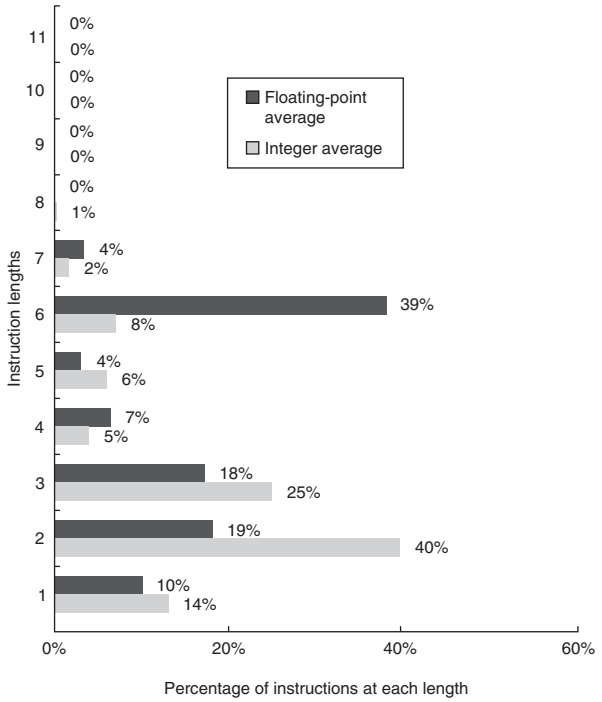


Figure K.42 Averages of the histograms of 80x86 instruction lengths for five SPECint92 programs and for five SPECfp92 programs, all running in 32-bit mode.

Option	doduc	ear	hydro2d	mdljdp2	su2cor	FP average
Stack (2nd operand ST (1))	1.1%	0.0%	0.0%	0.2%	0.6%	0.4%
Register (2nd operand ST(i), i > 1)	17.3%	63.4%	14.2%	7.1%	30.7%	26.5%
Memory	81.6%	36.6%	85.8%	92.7%	68.7%	73.1%

Figure K.43 The percentage of instructions for the floating-point operations (add, sub, mul, div) that use each of the three options for specifying a floating-point operand on the 80x86. The three options are (1) the strict stack model of implicit operands on the stack, (2) register version naming an explicit operand that is not one of the top two elements of the stack, and (3) memory operand.

for DLX: Numbers less than 1.0 mean that the 80x86 executes fewer instructions than DLX. The instruction count is surprisingly close to DLX for many integer programs, as you would expect a load-store instruction set architecture like DLX to execute more instructions than a register-memory architecture like the 80x86. The floating-point programs always have higher counts for the 80x86,

Instruction	doduc	ear	hydro2d	mdljdp2	su2cor	FP average
Load	8.9%	6.5%	18.0%	27.6%	27.6%	20%
Store	12.4%	3.1%	11.5%	7.8%	7.8%	8%
Add	5.4%	6.6%	14.6%	8.8%	8.8%	10%
Sub	1.0%	2.4%	3.3%	2.4%	2.4%	3%
Mul						0%
Div						0%
Compare	1.8%	5.1%	0.8%	1.0%	1.0%	2%
Mov reg-reg	3.2%	0.1%	1.8%	2.3%	2.3%	2%
Load imm	0.4%	1.5%				0%
Cond. branch	5.4%	8.2%	5.1%	2.7%	2.7%	5%
Uncond branch	0.8%	0.4%	1.3%	0.3%	0.3%	1%
Call	0.5%	1.6%		0.1%	0.1%	0%
Return, jmp indirect	0.5%	1.6%		0.1%	0.1%	0%
Shift	1.1%		4.5%	2.5%	2.5%	2%
AND	0.8%	0.8%	0.7%	1.3%	1.3%	1%
OR	0.1%			0.1%	0.1%	0%
Other (XOR, not, . . .)						0%
Load FP	14.1%	22.5%	9.1%	12.6%	12.6%	14%
Store FP	8.6%	11.4%	4.1%	6.6%	6.6%	7%
Add FP	5.8%	6.1%	1.4%	6.6%	6.6%	5%
Sub FP	2.2%	2.7%	3.1%	2.9%	2.9%	3%
Mul FP	8.9%	8.0%	4.1%	12.0%	12.0%	9%
Div FP	2.1%		0.8%	0.2%	0.2%	0%
Compare FP	9.4%	6.9%	10.8%	0.5%	0.5%	5%
Mov reg-reg FP	2.5%	0.8%	0.3%	0.8%	0.8%	1%
Other (abs, sqrt, . . .)	3.9%	3.8%	4.1%	0.8%	0.8%	2%

Figure K.44 80x86 instruction mix for five SPECfp92 programs.

presumably due to the lack of floating-point registers and the use of a stack architecture.

Another question is the total amount of data traffic for the 80x86 versus DLX, since the 80x86 can specify memory operands as part of operations while DLX can only access via loads and stores. Figures K.46 and K.47 also show the data reads, data writes, and data read-modify-writes for these 10 programs. The total

Instruction	compress	eqntott	espresso	gcc (cc1)	li	Int. average
Load	20.8%	18.5%	21.9%	24.9%	23.3%	22%
Store	13.8%	3.2%	8.3%	16.6%	18.7%	12%
Add	10.3%	8.8%	8.15%	7.6%	6.1%	8%
Sub	7.0%	10.6%	3.5%	2.9%	3.6%	5%
Mul				0.1%		0%
Div						0%
Compare	8.2%	27.7%	15.3%	13.5%	7.7%	16%
Mov reg-reg	7.9%	0.6%	5.0%	4.2%	7.8%	4%
Load imm	0.5%	0.2%	0.6%	0.4%		0%
Cond. branch	15.5%	28.6%	18.9%	17.4%	15.4%	20%
Uncond. branch	1.2%	0.2%	0.9%	2.2%	2.2%	1%
Call	0.5%	0.4%	0.7%	1.5%	3.2%	1%
Return, jmp indirect	0.5%	0.4%	0.7%	1.5%	3.2%	1%
Shift	3.8%		2.5%	1.7%		1%
AND	8.4%	1.0%	8.7%	4.5%	8.4%	6%
OR	0.6%		2.7%	0.4%	0.4%	1%
Other (XOR, not, . . .)	0.9%		2.2%	0.1%		1%
Load FP						0%
Store FP						0%
Add FP						0%
Sub FP						0%
Mul FP						0%
Div FP						0%
Compare FP						0%
Mov reg-reg FP						0%
Other (abs, sqrt, . . .)						0%

Figure K.45 80x86 instruction mix for five SPECint92 programs.

accesses ratio to DLX of each memory access type is shown in the bottom rows, with the read-modify-write counting as one read and one write. The 80x86 performs about two to four times as many data accesses as DLX for floating-point programs, and 1.25 times as many for integer programs. Finally, Figure K.48 shows the percentage of instructions in each category for 80x86 and DLX.

	compress	eqntott	espresso	gcc (cc1)	li	Int. avg.
Instructions executed on 80x86 (millions)	2226	1203	2216	3770	5020	
Instructions executed ratio to DLX	0.61	1.74	0.85	0.96	0.98	1.03
Data reads on 80x86 (millions)	589	229	622	1079	1459	
Data writes on 80x86 (millions)	311	39	191	661	981	
Data read-modify-writes on 80x86 (millions)	26	1	129	48	48	
Total data reads on 80x86 (millions)	615	230	751	1127	1507	
Data read ratio to DLX	0.85	1.09	1.38	1.25	0.94	1.10
Total data writes on 80x86 (millions)	338	40	319	709	1029	
Data write ratio to DLX	1.67	9.26	2.39	1.25	1.20	3.15
Total data accesses on 80x86 (millions)	953	269	1070	1836	2536	
Data access ratio to DLX	1.03	1.25	1.58	1.25	1.03	1.23

Figure K.46 Instructions executed and data accesses on 80x86 and ratios compared to DLX for five SPECint92 programs.

	doduc	ear	hydro2d	mdljdp2	su2cor	FP average
Instructions executed on 80x86 (millions)	1223	15,220	13,342	6197	6197	
Instructions executed ratio to DLX	1.19	1.19	2.53	2.09	1.62	1.73
Data reads on 80x86 (millions)	515	6007	5501	3696	3643	
Data writes on 80x86 (millions)	260	2205	2085	892	892	
Data read-modify-writes on 80x86 (millions)	1	0	189	124	124	
Total data reads on 80x86 (millions)	517	6007	5690	3820	3767	
Data read ratio to DLX	2.04	2.36	4.48	4.77	3.91	3.51
Total data writes on 80x86 (millions)	261	2205	2274	1015	1015	
Data write ratio to DLX	3.68	33.25	38.74	16.74	9.35	20.35
Total data accesses on 80x86 (millions)	778	8212	7965	4835	4782	
Data access ratio to DLX	2.40	3.14	5.99	5.73	4.47	4.35

Figure K.47 Instructions executed and data accesses for five SPECfp92 programs on 80x86 and ratio to DLX.

Concluding Remarks

Beauty is in the eye of the beholder.

Old Adage

As we have seen, "orthogonal" is not a term found in the Intel architectural dictionary. To fully understand which registers and which addressing modes are available, you need to see the encoding of all addressing modes and sometimes the encoding of the instructions.

	Integer average		FP average	
Category	**x86**	**DLX**	**x86**	**DLX**
Total data transfer	34%	36%	28%	2%
Total integer arithmetic	34%	31%	16%	12%
Total control	24%	20%	6%	10%
Total logical	8%	13%	3%	2%
Total FP data transfer	0%	0%	22%	33%
Total FP arithmetic	0%	0%	25%	41%

Figure K.48 Percentage of instructions executed by category for 80x86 and DLX for the averages of five SPECint92 and SPECfp92 programs of Figures K.46 and K.47.

Some argue that the inelegance of the 80x86 instruction set is unavoidable, the price that must be paid for rampant success by any architecture. We reject that notion. Obviously, no successful architecture can jettison features that were added in previous implementations, and over time some features may be seen as undesirable. The awkwardness of the 80x86 began at its core with the 8086 instruction set and was exacerbated by the architecturally inconsistent expansions of the 8087, 80286, and 80386.

A counterexample is the IBM 360/370 architecture, which is much older than the 80x86. It dominates the mainframe market just as the 80x86 dominates the PC market. Due undoubtedly to a better base and more compatible enhancements, this instruction set makes much more sense than the 80x86 more than 30 years after its first implementation.

For better or worse, Intel had a 16-bit microprocessor years before its competitors' more elegant architectures, and this head start led to the selection of the 8086 as the CPU for the IBM PC. What it lacks in style is made up in quantity, making the 80x86 beautiful from the right perspective.

The saving grace of the 80x86 is that its architectural components are not too difficult to implement, as Intel has demonstrated by rapidly improving performance of integer programs since 1978. High floating-point performance is a larger challenge in this architecture.

K.4 The VAX Architecture

VAX: the most successful minicomputer design in industry history . . . the VAX was probably the hacker's favorite machine . . . . Especially noted for its large, assembler-programmer-friendly instruction set—an asset that became a liability after the RISC revolution.

Eric Raymond
The New Hacker's Dictionary (1991)

Introduction

To enhance your understanding of instruction set architectures, we chose the VAX as the representative *Complex Instruction Set Computer* (CISC) because it is so different from MIPS and yet still easy to understand. By seeing two such divergent styles, we are confident that you will be able to learn other instruction sets on your own.

At the time the VAX was designed, the prevailing philosophy was to create instruction sets that were close to programming languages in order to simplify compilers. For example, because programming languages had loops, instruction sets should have loop instructions. As VAX architect William Strecker said ("VAX-11/780—A Virtual Address Extension to the PDP-11 Family," *AFIPS Proc.,* National Computer Conference, 1978):

> A major goal of the VAX-11 instruction set was to provide for effective compiler generated code. Four decisions helped to realize this goal: 1) A very regular and consistent treatment of operators . . . . 2) An avoidance of instructions unlikely to be generated by a compiler . . . . 3) Inclusions of several forms of common operators . . . . 4) Replacement of common instruction sequences with single instructions . . . . Examples include procedure calling, multiway branching, loop control, and array subscript calculation.

Recall that DRAMs of the mid-1970s contained less than 1/1000th the capacity of today's DRAMs, so code space was also critical. Hence, another prevailing philosophy was to minimize code size, which is de-emphasized in fixed-length instruction sets like MIPS. For example, MIPS address fields always use 16 bits, even when the address is very small. In contrast, the VAX allows instructions to be a variable number of bytes, so there is little wasted space in address fields.

Whole books have been written just about the VAX, so this VAX extension cannot be exhaustive. Hence, the following sections describe only a few of its addressing modes and instructions. To show the VAX instructions in action, later sections show VAX assembly code for two C procedures. The general style will be to contrast these instructions with the MIPS code that you are already familiar with.

The differing goals for VAX and MIPS have led to very different architectures. The VAX goals, simple compilers and code density, led to the powerful addressing modes, powerful instructions, and efficient instruction encoding. The MIPS goals were high performance via pipelining, ease of hardware implementation, and compatibility with highly optimizing compilers. The MIPS goals led to simple instructions, simple addressing modes, fixed-length instruction formats, and a large number of registers.

VAX Operands and Addressing Modes

The VAX is a 32-bit architecture, with 32-bit-wide addresses and 32-bit-wide registers. Yet, the VAX supports many other data sizes and types, as Figure K.49 shows. Unfortunately, VAX uses the name "word" to refer to 16-bit quantities; in this text, a word means 32 bits. Figure K.49 shows the conversion between

Bits	Data type	MIPS name	VAX name
8	Integer	Byte	Byte
16	Integer	Half word	Word
32	Integer	Word	Long word
32	Floating point	Single precision	F_floating
64	Integer	Double word	Quad word
64	Floating point	Double precision	D_floating or G_floating
8n	Character string	Character	Character

Figure K.49 VAX data types, their lengths, and names. The first letter of the VAX type (b, w, l, f, q, d, g, c) is often used to complete an instruction name. Examples of move instructions include `movb`, `movw`, `movl`, `movf`, `movq`, `movd`, `movg`, and `movc3`. Each move instruction transfers an operand of the data type indicated by the letter following `mov`.

the MIPS data type names and the VAX names. Be careful when reading about VAX instructions, as they refer to the names of the VAX data types.

The VAX provides sixteen 32-bit registers. The VAX assembler uses the notation `r0, r1, . . . , r15` to refer to these registers, and we will stick to that notation. Alas, 4 of these 16 registers are effectively claimed by the instruction set architecture. For example, `r14` is the stack pointer (`sp`) and `r15` is the program counter (`pc`). Hence, `r15` cannot be used as a general-purpose register, and using `r14` is very difficult because it interferes with instructions that manipulate the stack. The other dedicated registers are `r12`, used as the argument pointer (`ap`), and `r13`, used as the frame pointer (`fp`); their purpose will become clear later. (Like MIPS, the VAX assembler accepts either the register number or the register name.)

VAX addressing modes include those discussed in Appendix A, which has all the MIPS addressing modes: *register, displacement, immediate*, and *PC-relative*. Moreover, all these modes can be used for jump addresses or for data addresses.

But that's not all the addressing modes. To reduce code size, the VAX has three lengths of addresses for displacement addressing: 8-bit, 16-bit, and 32-bit addresses called, respectively, *byte displacement, word displacement*, and *long displacement* addressing. Thus, an address can be not only as small as possible but also as large as necessary; large addresses need not be split, so there is no equivalent to the MIPS lui instruction (see Figure A.24 on page A-37).

Those are still not all the VAX addressing modes. Several have a *deferred* option, meaning that the object addressed is only the *address* of the real object, requiring another memory access to get the operand. This addressing mode is called *indirect addressing* in other machines. Thus, *register deferred, autoincrement deferred*, and *byte/word/long displacement deferred* are other addressing modes to choose from. For example, using the notation of the VAX assembler,

r1 means the operand is register 1 and (r1) means the operand is the location in memory pointed to by r1.

There is yet another addressing mode. *Indexed addressing* automatically converts the value in an index operand to the proper byte address to add to the rest of the address. For a 32-bit word, we needed to multiply the index of a 4-byte quantity by 4 before adding it to a base address. Indexed addressing, called *scaled addressing* on some computers, automatically multiplies the index of a 4-byte quantity by 4 as part of the address calculation.

To cope with such a plethora of addressing options, the VAX architecture separates the specification of the addressing mode from the specification of the operation. Hence, the opcode supplies the operation and the number of operands, and each operand has its own addressing mode specifier. Figure K.50 shows the name, assembler notation, example, meaning, and length of the address specifier.

The VAX style of addressing means that an operation doesn't know where its operands come from; a VAX add instruction can have three operands in registers, three operands in memory, or any combination of registers and memory operands.

Addressing mode name	Syntax	Example	Meaning	Length of address specifier in bytes
Literal	#value	#–1	–1	1 (6-bit signed value)
Immediate	#value	#100	100	1 + length of the immediate
Register	rn	r3	r3	1
Register deferred	(rn)	(r3)	Memory[r3]	1
Byte/word/long displacement	Displacement (rn)	100(r3)	Memory[r3 + 100]	1 + length of the displacement
Byte/word/long displacement deferred	@displacement (rn)	@100(r3)	Memory[Memory [r3 + 100]]	1 + length of the displacement
Indexed (scaled)	Base mode [rx]	(r3)[r4]	Memory[r3 + r4 × *d*] (where *d* is data size in bytes)	1 + length of base addressing mode
Autoincrement	(rn)+	(r3)+	Memory[r3]; r3 = r3 + *d*	1
Autodecrement	– (rn)	–(r3)	r3 = r3 – *d;* Memory[r3]	1
Autoincrement deferred	@(rn)+	@(r3)+	Memory[Memory[r3]]; r3 = r3 + *d*	1

Figure K.50 Definition and length of the VAX operand specifiers. The length of each addressing mode is 1 byte plus the length of any displacement or immediate field needed by the mode. Literal mode uses a special 2-bit tag and the remaining 6 bits encode the constant value. If the constant is too big, it must use the immediate addressing mode. Note that the length of an immediate operand is dictated by the length of the data type indicated in the opcode, not the value of the immediate. The symbol *d* in the last four modes represents the length of the data in bytes; *d* is 4 for 32-bit add.

Example How long is the following instruction?

```
addl3 r1,737(r2),(r3)[r4]
```

The name addl3 means a 32-bit add instruction with three operands. Assume the length of the VAX opcode is 1 byte.

Answer The first operand specifier—`r1`—indicates register addressing and is 1 byte long. The second operand specifier—`737(r2)`—indicates displacement addressing and has two parts: The first part is a byte that specifies the word displacement addressing mode and base register `(r2)`; the second part is the 2-byte-long displacement `(737)`. The third operand specifier—`(r3)[r4]`—also has two parts: The first byte specifies register deferred addressing mode `((r3))`, and the second byte specifies the Index register and the use of indexed addressing ([r4]). Thus, the total length of the instruction is 1 + (1) + (1 + 2) + (1 + 1) = 7 bytes.

In this example instruction, we show the VAX destination operand on the left and the source operands on the right, just as we show MIPS code. The VAX assembler actually expects operands in the opposite order, but we felt it would be less confusing to keep the destination on the left for both machines. Obviously, left or right orientation is arbitrary; the only requirement is consistency.

Elaboration Because the PC is 1 of the 16 registers that can be selected in a VAX addressing mode, 4 of the 22 VAX addressing modes are synthesized from other addressing modes. Using the PC as the chosen register in each case, immediate addressing is really autoincrement, PC-relative is displacement, absolute is autoincrement deferred, and relative deferred is displacement deferred.

Encoding VAX Instructions

Given the independence of the operations and addressing modes, the encoding of instructions is quite different from MIPS.

VAX instructions begin with a single byte opcode containing the operation and the number of operands. The operands follow the opcode. Each operand begins with a single byte, called the *address specifier*, that describes the addressing mode for that operand. For a simple addressing mode, such as register addressing, this byte specifies the register number as well as the mode (see the rightmost column in Figure K.50). In other cases, this initial byte can be followed by many more bytes to specify the rest of the address information.

As a specific example, let's show the encoding of the add instruction from the example on page K-24:

```
addl3 r1,737(r2),(r3)[r4]
```

Assume that this instruction starts at location 201.

Figure K.51 shows the encoding. Note that the operands are stored in memory in opposite order to the assembly code above. The execution of VAX instructions

Byte address	Contents at each byte	Machine code
201	Opcode containing addl3	$c1_{hex}$
202	Index mode specifier for [r4]	44_{hex}
203	Register indirect mode specifier for (r3)	63_{hex}
204	Word displacement mode specifier using r2 as base	$c2_{hex}$
205	The 16-bit constant 737	$e1_{hex}$
206		02_{hex}
207	Register mode specifier for r1	51_{hex}

Figure K.51 The encoding of the VAX instruction addl3 r1,737(r2),(r3)[r4], assuming it starts at address 201. To satisfy your curiosity, the right column shows the actual VAX encoding in hexadecimal notation. Note that the 16-bit constant 737_{ten} takes 2 bytes.

begins with fetching the source operands, so it makes sense for them to come first. Order is not important in fixed-length instructions like MIPS, since the source and destination operands are easily found within a 32-bit word.

The first byte, at location 201, is the opcode. The next byte, at location 202, is a specifier for the index mode using register `r4`. Like many of the other specifiers, the left 4 bits of the specifier give the mode and the right 4 bits give the register used in that mode. Since `addl3` is a 4-byte operation, `r4` will be multiplied by 4 and added to whatever address is specified next. In this case it is register deferred addressing using register `r3`. Thus, bytes 202 and 203 combined define the third operand in the assembly code.

The following byte, at address 204, is a specifier for word displacement addressing using register `r2` as the base register. This specifier tells the VAX that the following two bytes, locations 205 and 206, contain a 16-bit address to be added to `r2`.

The final byte of the instruction gives the destination operand, and this specifier selects register addressing using register `r1`.

Such variability in addressing means that a single VAX operation can have many different lengths; for example, an integer add varies from 3 bytes to 19 bytes. VAX implementations must decode the first operand before they can find the second, and so implementors are strongly tempted to take 1 clock cycle to decode each operand; thus, this sophisticated instruction set architecture can result in higher clock cycles per instruction, even when using simple addresses.

VAX Operations

In keeping with its philosophy, the VAX has a large number of operations as well as a large number of addressing modes. We review a few here to give the flavor of the machine.

Given the power of the addressing modes, the VAX *move* instruction performs several operations found in other machines. It transfers data between any two addressable locations and subsumes load, store, register-register moves, and

memory-memory moves as special cases. The first letter of the VAX data type (b, w, l, f, q, d, g, c in Figure K.49) is appended to the acronym mov to determine the size of the data. One special move, called *move address*, moves the 32-bit *address* of the operand rather than the data. It uses the acronym `mova`.

The arithmetic operations of MIPS are also found in the VAX, with two major differences. First, the type of the data is attached to the name. Thus, `addb`, `addw`, and `addl` operate on 8-bit, 16-bit, and 32-bit data in memory or registers, respectively; MIPS has a single add instruction that operates only on the full 32-bit register. The second difference is that to reduce code size the add instruction specifies the number of unique operands; MIPS always specifies three even if one operand is redundant. For example, the MIPS instruction

```
add $1, $1, $2
```

takes 32 bits like all MIPS instructions, but the VAX instruction

```
addl2 r1, r2
```

uses `r1` for both the destination and a source, taking just 24 bits: 8 bits for the opcode and 8 bits each for the two register specifiers.

Number of Operations

Now we can show how VAX instruction names are formed:

$$(\text{operation})(\text{datatype})\left(\frac{2}{3}\right)$$

The operation add works with data types byte, word, long, float, and double and comes in versions for either 2 or 3 unique operands, so the following instructions are all found in the VAX:

```
addb2    addw2    addl2    addf2    addd2
addb3    addw3    addl3    addf3    addd3
```

Accounting for all addressing modes (but ignoring register numbers and immediate values) and limiting to just byte, word, and long, there are more than 30,000 versions of integer add in the VAX; MIPS has just 4!

Another reason for the large number of VAX instructions is the instructions that either replace sequences of instructions or take fewer bytes to represent a single instruction. Here are four such examples (* means the data type):

VAX operation	Example	Meaning
`clr*`	`clrl r3`	`r3 = 0`
`inc*`	`incl r3`	`r3 = r3+1`
`dec*`	`decl r3`	`r3 = r3-1`
`push*`	`pushl r3`	`sp = sp-4;Memory[sp]=r3;`

The *push* instruction in the last row is exactly the same as using the move instruction with autodecrement addressing on the stack pointer:

```
movl -(sp), r3
```

Brevity is the advantage of pushl: It is 1 byte shorter since sp is implied.

Branches, Jumps, and Procedure Calls

The VAX branch instructions are related to the arithmetic instructions because the branch instructions rely on *condition codes*. Condition codes are set as a side effect of an operation, and they indicate whether the result is positive, negative, or zero or if an overflow occurred. Most instructions set the VAX condition codes according to their result; instructions without results, such as branches, do not. The VAX condition codes are N (Negative), Z (Zero), V (oVerflow), and C (Carry). There is also a *compare* instruction `cmp*` just to set the condition codes for a subsequent branch.

The VAX branch instructions include all conditions. Popular branch instructions include `beql`(=), `bneq`($\neq$), `blss`(<), `bleq`($\leq$), `bgtr`(>), and `bgeq`($\geq$), which do just what you would expect. There are also unconditional branches whose name is determined by the size of the PC-relative offset. Thus, `brb` (*branch byte*) has an 8-bit displacement, and `brw` (*branch word*) has a 16-bit displacement.

The final major category we cover here is the procedure *call and return* instructions. Unlike the MIPS architecture, these elaborate instructions can take dozens of clock cycles to execute. The next two sections show how they work, but we need to explain the purpose of the pointers associated with the stack manipulated by `calls` and `ret`. The *stack pointer*, `sp`, is just like the stack pointer in MIPS; it points to the top of the stack. The *argument pointer*, `ap`, points to the base of the list of arguments or parameters in memory that are passed to the procedure. The *frame pointer*, `fp`, points to the base of the local variables of the procedure that are kept in memory (the *stack frame*). The VAX call and return instructions manipulate these pointers to maintain the stack in proper condition across procedure calls and to provide convenient base registers to use when accessing memory operands. As we shall see, call and return also save and restore the general-purpose registers as well as the program counter. Figure K.52 gives a further sampling of the VAX instruction set.

An Example to Put It All Together: swap

To see programming in VAX assembly language, we translate two C procedures, `swap` and `sort`. The C code for `swap` is reproduced in Figure K.53. The next section covers `sort`.

We describe the `swap` procedure in three general steps of assembly language programming:

1. Allocate registers to program variables.
2. Produce code for the body of the procedure.
3. Preserve registers across the procedure invocation.

Instruction type	Example	Instruction meaning
Data transfers	Move data between byte, half-word, word, or double-word operands; * is data type	
	`mov*`	Move between two operands
	`movzb*`	Move a byte to a half word or word, extending it with zeros
	`mova*`	Move the 32-bit address of an operand; data type is last
	`push*`	Push operand onto stack
Arithmetic/logical	Operations on integer or logical bytes, half words (16 bits), words (32 bits); * is data type	
	`add*_`	Add with 2 or 3 operands
	`cmp*`	Compare and set condition codes
	`tst*`	Compare to zero and set condition codes
	`ash*`	Arithmetic shift
	`clr*`	Clear
	`cvtb*`	Sign-extend byte to size of data type
Control	Conditional and unconditional branches	
	`beql, bneq`	Branch equal, branch not equal
	`bleq, bgeq`	Branch less than or equal, branch greater than or equal
	`brb, brw`	Unconditional branch with an 8-bit or 16-bit address
	`jmp`	Jump using any addressing mode to specify target
	`aobleq`	Add one to operand; branch if result ≤ second operand
	`case_`	Jump based on case selector
Procedure	Call/return from procedure	
	`calls`	Call procedure with arguments on stack (see "A Longer Example: `sort`" on page K-33)
	`callg`	Call procedure with FORTRAN-style parameter list
	`jsb`	Jump to subroutine, saving return address (like MIPS jal)
	`ret`	Return from procedure call
Floating point	Floating-point operations on D, F, G, and H formats	
	`addd_`	Add double-precision D-format floating numbers
	`subd_`	Subtract double-precision D-format floating numbers
	`mulf_`	Multiply single-precision F-format floating point
	`polyf`	Evaluate a polynomial using table of coefficients in F format
Other	Special operations	
	`crc`	Calculate cyclic redundancy check
	`insque`	Insert a queue entry into a queue

Figure K.52 Classes of VAX instructions with examples. The asterisk stands for multiple data types: b, w, l, d, f, g, h, and q. The underline, as in `addd_`, means there are 2-operand (`addd2`) and 3-operand (`addd3`) forms of this instruction.

```
swap(int v[], int k)
{
   int temp;
   temp = v[k];
   v[k] = v[k + 1];
   v[k + 1] = temp;
}
```

Figure K.53 A C procedure that swaps two locations in memory. This procedure will be used in the sorting example in the next section.

The VAX code for these procedures is based on code produced by the VMS C compiler using optimization.

Register Allocation for `swap`

In contrast to MIPS, VAX parameters are normally allocated to memory, so this step of assembly language programming is more properly called "variable allocation." The standard VAX convention on parameter passing is to use the stack. The two parameters, `v[]` and `k`, can be accessed using register ap, the argument pointer: The address `4(ap)` corresponds to `v[]` and `8(ap)` corresponds to k. Remember that with byte addressing the address of sequential 4-byte words differs by 4. The only other variable is `temp`, which we associate with register `r3`.

Code for the Body of the Procedure `swap`

The remaining lines of C code in `swap` are

```
temp = v[k];
v[k] = v[k + 1];
v[k + 1] = temp;
```

Since this program uses `v[]` and `k` several times, to make the programs run faster the VAX compiler first moves both parameters into registers:

```
movl r2, 4(ap) ;r2 = v[]
movl r1, 8(ap) ;r1 = k
```

Note that we follow the VAX convention of using a semicolon to start a comment; the MIPS comment symbol # represents a constant operand in VAX assembly language.

The VAX has indexed addressing, so we can use index k without converting it to a byte address. The VAX code is then straightforward:

```
movl  r3,(r2)[r1]       ;r3 (temp) = v[k]
addl3 r0,#1,8(ap)       ;r0 = k + 1
movl  (r2)[r1],(r2)[r0] ;v[k] = v[r0] (v[k + 1])
movl  (r2)[r0],r3       ;v[k + 1] = r3 (temp)
```

Unlike the MIPS code, which is basically two loads and two stores, the key VAX code is one memory-to-register move, one memory-to-memory move, and one register-to-memory move. Note that the `addl3` instruction shows the flexibility of the VAX addressing modes: It adds the constant 1 to a memory operand and places the result in a register.

Now we have allocated storage and written the code to perform the operations of the procedure. The only missing item is the code that preserves registers across the routine that calls `swap`.

Preserving Registers across Procedure Invocation of `swap`

The VAX has a pair of instructions that preserve registers, `calls` and `ret`. This example shows how they work.

The VAX C compiler uses a form of callee convention. Examining the code above, we see that the values in registers `r0`, `r1`, `r2`, and `r3` must be saved so that they can later be restored. The `calls` instruction expects a 16-bit mask at the beginning of the procedure to determine which registers are saved: if bit *i* is set in the mask, then register *i* is saved on the stack by the `calls` instruction. In addition, `calls` saves this mask on the stack to allow the return instruction (`ret`) to restore the proper registers. Thus, the calls executed by the caller does the saving, but the callee sets the call mask to indicate what should be saved.

One of the operands for `calls` gives the number of parameters being passed, so that `calls` can adjust the pointers associated with the stack: the argument pointer (`ap`), frame pointer (`fp`), and stack pointer (`sp`). Of course, `calls` also saves the program counter so that the procedure can return!

Thus, to preserve these four registers for swap, we just add the mask at the beginning of the procedure, letting the calls instruction in the caller do all the work:

```
.word ^m<r0,r1,r2,r3> ;set bits in mask for 0,1,2,3
```

This directive tells the assembler to place a 16-bit constant with the proper bits set to save registers `r0` through `r3`.

The return instruction undoes the work of `calls`. When finished, `ret` sets the stack pointer from the current frame pointer to pop everything `calls` placed on the stack. Along the way, it restores the register values saved by `calls`, including those marked by the mask and old values of the `fp`, `ap`, and `pc`.

To complete the procedure `swap`, we just add one instruction:

```
ret   ;restore registers and return
```

The Full Procedure `swap`

We are now ready for the whole routine. Figure K.54 identifies each block of code with its purpose in the procedure, with the MIPS code on the left and the VAX code on the right. This example shows the advantage of the scaled indexed addressing and the sophisticated call and return instructions of the VAX in reducing the number of lines of code. The 17 lines of MIPS assembly code became 8 lines of VAX assembly code. It also shows that passing parameters in memory results in extra memory accesses.

Keep in mind that the number of instructions executed is not the same as performance; the fallacy on page K-38 makes this point.

Note that VAX software follows a convention of treating registers `r0` and `r1` as temporaries that are not saved across a procedure call, so the VMS C compiler does include registers `r0` and `r1` in the register saving mask. Also, the C compiler should have used `r1` instead of `8(ap)` in the `addl3` instruction; such examples inspire computer architects to try to write compilers!

MIPS versus VAX

		Saving register			
swap:	addi	$29,$29, -12	swap:	.word	^m<r0,r1,r2,r3>
	sw	$2, 0($29)			
	sw	$15, 4($29)			
	sw	$16, 8($29)			
		Procedure body			
	muli	$2, $5,4		movl	r2, 4(a)
	add	$2, $4,$2		movl	r1, 8(a)
	lw	$15, 0($2)		movl	r3, (r2)[r1]
	lw	$16, 4($2)		addl3	r0, #1,8(ap)
	sw	$16, 0($2)		movl	(r2)[r1],(r2)[r0]
	sw	$15, 4($2)		movl	(r2)[r0],r3
		Restoring registers			
	lw	$2, 0($29)			
	lw	$15, 4($29)			
	lw	$16, 8($29)			
	addi	$29,$29, 12			
		Procedure return			
	jr	$31		ret	

Figure K.54 MIPS versus VAX assembly code of the procedure swap in Figure K.53 on page K-30.

A Longer Example: sort

We show the longer example of the sort procedure. Figure K.55 shows the C version of the program. Once again we present this procedure in several steps, concluding with a side-by-side comparison to MIPS code.

Register Allocation for `sort`

The two parameters of the procedure sort, v and n, are found in the stack in locations 4(ap) and 8(ap), respectively. The two local variables are assigned to registers: i to r6 and j to r4. Because the two parameters are referenced frequently in the code, the VMS C compiler copies the *address* of these parameters into registers upon entering the procedure:

```
moval  r7,8(ap)  ;move  address  of  n  into  r7
moval  r5,4(ap)  ;move  address  of  v  into  r5
```

It would seem that moving the *value* of the operand to a register would be more useful than its address, but once again we bow to the decision of the VMS C compiler. Apparently the compiler cannot be sure that v and n don't overlap in memory.

Code for the Body of the sort Procedure

The procedure body consists of two nested *for* loops and a call to swap, which includes parameters. Let's unwrap the code from the outside to the middle.

The Outer Loop

The first translation step is the first for loop:

```
for (i = 0; i < n; i = i + 1) {
```

Recall that the C for statement has three parts: initialization, loop test, and iteration increment. It takes just one instruction to initialize `i` to `0`, the first part of the for statement:

```
clrl    r6    ;i = 0
```

```
sort (int v[], int n)
{
  int i, j;
  for (i = 0; i < n; i = i + 1) {
  for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j = j - 1)
    { swap(v,j);
   }
  }
}
```

Figure K.55 A C procedure that performs a bubble sort on the array v.

It also takes just one instruction to increment i, the last part of the for:

```
incl    r6    ;i = i + 1
```

The loop should be exited if i < n is *false,* or said another way, exit the loop if i ≥ n. This test takes two instructions:

```
for1tst: cmpl r6,(r7) ;compare r6 and memory[r7] (i:n)
         bgeq exit1   ;go to exit1 if r6 ≥ mem[r7] (i ≥ n)
```

Note that cmpl sets the condition codes for use by the conditional branch instruction bgeq.

The bottom of the loop just jumps back to the loop test:

```
         brb  for1tst  ;branch to test of outer loop
exit1:
```

The skeleton code of the first for loop is then

```
         clrl  r6      ;i = 0
for1tst: cmpl  r6,(r7) ;compare r6 and memory[r7] (i:n)
         bgeq exit1    ;go to exit1 if r6 ≥ mem[r7] (i ≥ n)
               ...
               (body of first for loop)
               ...
         incl  r6      ;i = i + 1
         brb   for1tst ;branch to test of outer loop
exit1:
```

The Inner Loop

The second for loop is

```
for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j = j - 1) {
```

The initialization portion of this loop is again one instruction:

```
subl3   r4,r6,#1    ;j = i - 1
```

The decrement of j is also one instruction:

```
decl    r4          ;j = j - 1
```

The loop test has two parts. We exit the loop if either condition fails, so the first test must exit the loop if it fails (j < 0):

```
for2tst:blss      exit2      ;go to exit2 if r4 < 0 (j < 0)
```

Notice that there is no explicit comparison. The lack of comparison is a benefit of condition codes, with the conditions being set as a side effect of the prior instruction. This branch skips over the second condition test.

The second test exits if v[j] > v[j + 1] is false, or exits if v[j] ≤ v[j + 1]. First we load v and put j + 1 into registers:

```
movl    r3,(r5)    ;r3 = Memory[r5] (r3 = v)
addl3   r2,r4,#1   ;r2 = r4 + 1 (r2 = j + 1)
```

Register indirect addressing is used to get the operand pointed to by r5.

Once again the index addressing mode means we can use indices without converting to the byte address, so the two instructions for v[j] ≤ v[j + 1] are

```
cmpl (r3)[r4],(r3)[r2]    ;v[r4] : v[r2] (v[j]:v[j + 1])
bleq exit2                ;go to exit2 if v[j] ≤ v[j + 1]
```

The bottom of the loop jumps back to the full loop test:

```
brb     for2tst #     jump to test of inner loop
```

Combining the pieces, the second for loop looks like this:

```
          subl3 r4,r6, #1     ;j = i - 1
for2tst:  blss  exit2         ;go to exit2 if r4 < 0 (j < 0)
          movl  r3,(r5)       ;r3 = Memory[r5] (r3 = v)
          addl3 r2,r4,#1      ;r2 = r4 + 1 (r2 = j + 1)
          cmpl  (r3)[r4],(r3)[r2];v[r4] : v[r2]
          bleq  exit2         ;go to exit2 if v[j] ð [j+1]
                ...
                (body of second for loop) ...
          decl  r4            ;j = j - 1
          brb   for2tst       ;jump to test of inner loop
exit2:
```

Notice that the instruction `blss` (at the top of the loop) is testing the condition codes based on the new value of `r4 (j)`, set either by the `subl3` before entering the loop or by the `decl` at the bottom of the loop.

The Procedure Call

The next step is the body of the second for loop:

```
swap(v,j);
```

Calling swap is easy enough:

```
calls     #2,swap
```

The constant 2 indicates the number of parameters pushed on the stack.

Passing Parameters

The C compiler passes variables on the stack, so we pass the parameters to `swap` with these two instructions:

```
pushl    (r5)       ;first swap parameter is v
pushl     r4        ;second swap parameter is j
```

Register indirect addressing is used to get the operand of the first instruction.

Preserving Registers across Procedure Invocation of sort

The only remaining code is the saving and restoring of registers using the callee save convention. This procedure uses registers `r2` through `r7`, so we add a mask with those bits set:

```
.word ^m<r2,r3,r4,r5,r6,r7>; set mask for registers 2-7
```

Since ret will undo all the operations, we just tack it on the end of the procedure.

The Full Procedure sort

Now we put all the pieces together in Figure K.56. To make the code easier to follow, once again we identify each block of code with its purpose in the procedure and list the MIPS and VAX code side by side. In this example, 11 lines of the sort procedure in C become the 44 lines in the MIPS assembly language and 20 lines in VAX assembly language. The biggest VAX advantages are in register saving and restoring and indexed addressing.

Fallacies and Pitfalls

The ability to simplify means to eliminate the unnecessary so that the necessary may speak.

Hans Hoffman
Search for the Real (1967)

Fallacy *It is possible to design a flawless architecture.*

All architecture design involves trade-offs made in the context of a set of hardware and software technologies. Over time those technologies are likely to change, and decisions that may have been correct at one time later look like mistakes. For example, in 1975 the VAX designers overemphasized the importance of code size efficiency and underestimated how important ease of decoding and pipelining would be 10 years later. And, almost all architectures eventually succumb to the lack of sufficient address space. Avoiding these problems in the long run, however, would probably mean compromising the efficiency of the architecture in the short run.

Fallacy *An architecture with flaws cannot be successful.*

The IBM 360 is often criticized in the literature—the branches are not PC-relative, and the address is too small in displacement addressing. Yet, the machine has been an enormous success because it correctly handled several new problems. First, the architecture has a large amount of address space. Second, it is byte addressed and handles bytes well. Third, it is a general-purpose register machine. Finally, it is simple enough to be efficiently implemented across a wide performance and cost range.

The Intel 8086 provides an even more dramatic example. The 8086 architecture is the only widespread architecture in existence today that is not truly a general-purpose register machine. Furthermore, the segmented address space of the 8086 causes major problems for both programmers and compiler writers. Nevertheless, the 8086 architecture—because of its selection as the microprocessor in the IBM PC—has been enormously successful.

MIPS versus VAX					
Saving registers					
	sort:	addi $29,$29, -36	sort:	.word ^m<r2,r3,r4,r5,r6,r7>	
		sw $15, 0($29)			
		sw $16, 4($29)			
		sw $17, 8($29)			
		sw $18,12($29)			
		sw $19,16($29)			
		sw $20,20($29)			
		sw $24,24($29)			
		sw $25,28($29)			
		sw $31,32($29)			
Procedure body					
Move parameters		move $18, $4		moval r7,8(ap)	
		move $20, $5		moval r5,4(ap)	
Outer loop		add $19, $0, $0		clrl r6	
	for1tst:	slt $8, $19, $20	for1tst:	cmpl r6,(r7)	
		beq $8, $0, exit1		bgeq exit1	
Inner loop		addi $17, $19, -1		subl3 r4,r6,#1	
	for2tst:	slti $8, $17, 0	for2tst:		
		bne $8, $0, exit2		blss exit2	
		muli $15, $17, 4		movl r3,(r5)	
		add $16, $18, $15			
		lw $24, 0($16)			
		lw $25, 4($16)		addl3 r2,r4,#1	
		slt $8, $25, $24		cmpl (r3)[r4],(r3)[r2]	
		beq $8, $0, exit2		bleq exit2	
Pass parameters and call		move $4, $18		pushl (r5)	
		move $5, $17		pushl r4	
		jal swap		calls #2,swap	
Inner loop		addi $17, $17, -1		decl r4	
		j for2tst		brb for2tst	
Outer loop	exit2:	addi $19, $19, 1	exit2:	incl r6	
		j for1tst		brb for1tst	
Restoring registers					
	exit1:	lw $15,0($29)			
		lw $16, 4($29)			
		lw $17, 8($29)			
		lw $18,12($29)			
		lw $19,16($29)			
		lw $20,20($29)			
		lw $24,24($29)			
		lw $25,28($29)			
		lw $31,32($29)			
		addi $29,$29, 36			
Procedure return					
		jr $31	exit1:	ret	

Figure K.56 MIPS32 versus VAX assembly version of procedure sort **in Figure K.55 on page K-33.**

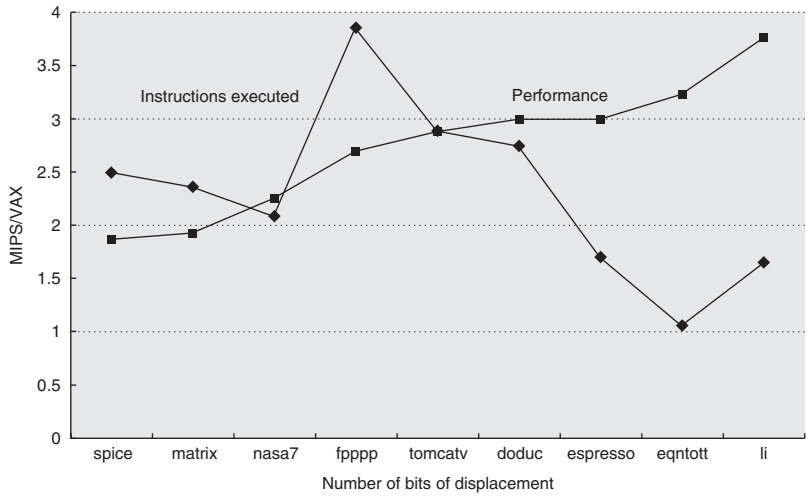


Figure K.57 Ratio of MIPS M2000 to VAX 8700 in instructions executed and performance in clock cycles using SPEC89 programs. On average, MIPS executes a little over twice as many instructions as the VAX, but the CPI for the VAX is almost six times the MIPS CPI, yielding almost a threefold performance advantage. (Based on data from "Performance from Architecture: Comparing a RISC and CISC with Similar Hardware Organization," by D. Bhandarkar and D. Clark, in *Proc. Symp. Architectural Support for Programming Languages and Operating Systems IV*, 1991.)

Fallacy *The architecture that executes fewer instructions is faster.*

Designers of VAX machines performed a quantitative comparison of VAX and MIPS for implementations with comparable organizations, the VAX 8700 and the MIPS M2000. Figure K.57 shows the ratio of the number of instructions executed and the ratio of performance measured in clock cycles. MIPS executes about twice as many instructions as the VAX while the MIPS M2000 has almost three times the performance of the VAX 8700.

Concluding Remarks

The Virtual Address eXtension of the PDP-11 architecture … provides a virtual address of about 4.3 gigabytes which, even given the rapid improvement of memory technology, should be adequate far into the future.

William Strecker

"VAX-11/780—A Virtual Address Extension to the PDP-11 Family,"
AFIPS Proc., *National Computer Conference* (1978)

We have seen that instruction sets can vary quite dramatically, both in how they access operands and in the operations that can be performed by a single instruction. Figure K.58 compares instruction usage for both architectures for two programs; even very different architectures behave similarly in their use of instruction classes.

Program	Machine	Branch	Arithmetic/ logical	Data transfer	Floating point	Totals
gcc	VAX	30%	40%	19%		89%
	MIPS	24%	35%	27%		86%
spice	VAX	18%	23%	15%	23%	79%
	MIPS	4%	29%	35%	15%	83%

Figure K.58 The frequency of instruction distribution for two programs on VAX and MIPS.

A product of its time, the VAX emphasis on code density and complex operations and addressing modes conflicts with the current emphasis on easy decoding, simple operations and addressing modes, and pipelined performance.

With more than 600,000 sold, the VAX architecture has had a very successful run. In 1991, DEC made the transition from VAX to Alpha.

Orthogonality is key to the VAX architecture; the opcode is independent of the addressing modes, which are independent of the data types and even the number of unique operands. Thus, a few hundred operations expand to hundreds of thousands of instructions when accounting for the data types, operand counts, and addressing modes.

Exercises

K.1 [3] <K.4> The following VAX instruction decrements the location pointed to be register r5:

```
decl (r5)
```

What is the single MIPS instruction, or if it cannot be represented in a single instruction, the shortest sequence of MIPS instructions, that performs the same operation? What are the lengths of the instructions on each machine?

K.2 [5] <K.4> This exercise is the same as Exercise K.1, except this VAX instruction clears a location using autoincrement deferred addressing:

```
clrl @(r5)+
```

K.3 [5] <K.4> This exercise is the same as Exercise K.1, except this VAX instruction adds 1 to register r5, placing the sum back in register r5, compares the sum to register r6, and then branches to L1 if r5 < r6:

```
aoblss r6, r5, L1 # r5 = r5 + 1; if (r5 < r6) goto L1.
```

K.4 [5] <K.4> Show the single VAX instruction, or minimal sequence of instructions, for this C statement:

```
a = b + 100;
```

Assume a corresponds to register r3 and b corresponds to register r4.

K.5 [10] <K.4> Show the single VAX instruction, or minimal sequence of instructions, for this C statement:

```
x[i + 1] = x[i] + c;
```

Assume c corresponds to register r3, i to register r4, and x is an array of 32-bit words beginning at memory location $4{,}000{,}000_{ten}$.

K.5 The IBM 360/370 Architecture for Mainframe Computers

Introduction

The term "computer architecture" was coined by IBM in 1964 for use with the IBM 360. Amdahl, Blaauw, and Brooks [1964] used the term to refer to the programmer-visible portion of the instruction set. They believed that a family of machines of the same architecture should be able to run the same software. Although this idea may seem obvious to us today, it was quite novel at the time. IBM, even though it was the leading company in the industry, had five different architectures before the 360. Thus, the notion of a company standardizing on a single architecture was a radical one. The 360 designers hoped that six different divisions of IBM could be brought together by defining a common architecture. Their definition of *architecture* was

> … the structure of a computer that a machine language programmer must understand to write a correct (timing independent) program for that machine.

The term "machine language programmer" meant that compatibility would hold, even in assembly language, while "timing independent" allowed different implementations.

The IBM 360 was introduced in 1964 with six models and a 25:1 performance ratio. Amdahl, Blaauw, and Brooks [1964] discussed the architecture of the IBM 360 and the concept of permitting multiple object-code-compatible implementations. The notion of an instruction set architecture as we understand it today was the most important aspect of the 360. The architecture also introduced several important innovations, now in wide use:

1. 32-bit architecture
2. Byte-addressable memory with 8-bit bytes
3. 8-, 16-, 32-, and 64-bit data sizes
4. 32-bit single-precision and 64-bit double-precision floating-point data

In 1971, IBM shipped the first System/370 (models 155 and 165), which included a number of significant extensions of the 360, as discussed by Case and Padegs [1978], who also discussed the early history of System/360. The most important addition was virtual memory, though virtual memory 370 s did not ship until 1972, when a virtual memory operating system was ready. By 1978, the high-end 370 was several hundred times faster than the low-end 360 s shipped 10 years earlier. In 1984, the 24-bit addressing model built into the IBM 360 needed to be abandoned, and the 370-XA (eXtended Architecture) was introduced. While old 24-bit programs could be supported without change, several instructions could not function in the same manner when extended to a 32-bit addressing model (31-bit addresses supported) because they would not produce 31-bit addresses. Converting the operating system, which was written mostly in assembly language, was no doubt the biggest task.

Several studies of the IBM 360 and instruction measurement have been made. Shustek's thesis [1978] is the best known and most complete study of the 360/370 architecture. He made several observations about instruction set complexity that

were not fully appreciated until some years later. Another important study of the 360 is the Toronto study by Alexander and Wortman [1975] done on an IBM 360 using 19 XPL programs.

System/360 Instruction Set

The 360 instruction set is shown in the following tables, organized by instruction type and format. System/370 contains 15 additional user instructions.

Integer/Logical and Floating-Point R-R Instructions

The * indicates the instruction is floating point, and may be either D (double precision) or E (single precision).

Instruction	Description
ALR	Add logical register
AR	Add register
A*R	FP addition
CLR	Compare logical register
CR	Compare register
C*R	FP compare
DR	Divide register
D*R	FP divide
H*R	FP halve
LCR	Load complement register
LC*R	Load complement
LNR	Load negative register
LN*R	Load negative
LPR	Load positive register
LP*R	Load positive
LR	Load register
L*R	Load FP register
LTR	Load and test register
LT*R	Load and test FP register
MR	Multiply register
M*R	FP multiply
NR	And register
OR	Or register
SLR	Subtract logical register
SR	Subtract register
S*R	FP subtraction
XR	Exclusive or register

Branches and Status Setting R-R Instructions

These are R-R format instructions that either branch or set some system status; several of them are privileged and legal only in supervisor mode.

Instruction	Description
BALR	Branch and link
BCTR	Branch on count
BCR	Branch/condition
ISK	Insert key
SPM	Set program mask
SSK	Set storage key
SVC	Supervisor call

Branches/Logical and Floating-Point Instructions—RX Format

These are all RX format instructions. The symbol "+" means either a word operation (and then stands for nothing) or H (meaning half word); for example, A+ stands for the two opcodes A and AH. The "*" represents D or E, standing for double- or single-precision floating point.

Instruction	Description
A+	Add
A*	FP add
AL	Add logical
C+	Compare
C*	FP compare
CL	Compare logical
D	Divide
D*	FP divide
L+	Load
L*	Load FP register
M+	Multiply
M*	FP multiply
N	And
O	Or
S+	Subtract
S*	FP subtract
SL	Subtract logical
ST+	Store
ST*	Store FP register
X	Exclusive or

Branches and Special Loads and Stores—RX Format

Instruction	Description
BAL	Branch and link
BC	Branch condition
BCT	Branch on count
CVB	Convert-binary
CVD	Convert-decimal
EX	Execute
IC	Insert character
LA	Load address
STC	Store character

RS and SI Format Instructions

These are the RS and SI format instructions. The symbol "*" may be A (arithmetic) or L (logical).

Instruction	Description
BXH	Branch/high
BXLE	Branch/low-equal
CLI	Compare logical immediate
HIO	Halt I/O
LPSW	Load PSW
LM	Load multiple
MVI	Move immediate
NI	And immediate
OI	Or immediate
RDD	Read direct
SIO	Start I/O
SL*	Shift left A/L
SLD*	Shift left double A/L
SR*	Shift right A/L
SRD*	Shift right double A/L
SSM	Set system mask
STM	Store multiple
TCH	Test channel
TIO	Test I/O
TM	Test under mask
TS	Test-and-set
WRD	Write direct
XI	Exclusive or immediate

SS Format Instructions

These are add decimal or string instructions.

Instruction	Description
AP	Add packed
CLC	Compare logical chars
CP	Compare packed
DP	Divide packed
ED	Edit
EDMK	Edit and mark
MP	Multiply packed
MVC	Move character
MVN	Move numeric
MVO	Move with offset
MVZ	Move zone
NC	And characters
OC	Or characters
PACK	Pack (Character → decimal)
SP	Subtract packed
TR	Translate
TRT	Translate and test
UNPK	Unpack
XC	Exclusive or characters
ZAP	Zero and add packed

360 Detailed Measurements

Figure K.59 shows the frequency of instruction usage for four IBM 360 programs.

Instruction	PLIC	FORTGO	PLIGO	COBOLGO	Average
Control	**32%**	**13%**	**5%**	**16%**	**16%**
BC, BCR	28%	13%	5%	14%	15%
BAL, BALR	3%			2%	1%
Arithmetic/logical	**29%**	**35%**	**29%**	**9%**	**26%**
A, AR	3%	17%	21%		10%
SR	3%	7%			3%
SLL		6%	3%		2%
LA	8%	1%	1%		2%
CLI	7%				2%
NI				7%	2%
C	5%	4%	4%	0%	3%
TM	3%	1%		3%	2%
MH			2%		1%
Data transfer	**17%**	**40%**	**56%**	**20%**	**33%**
L, LR	7%	23%	28%	19%	19%
MVI	2%		16%	1%	5%
ST	3%		7%		3%
LD		7%	2%		2%
STD		7%	2%		2%
LPDR		3%			1%
LH	3%				1%
IC	2%				1%
LTR		1%			0%
Floating point		**7%**			**2%**
AD		3%			1%
MDR		3%			1%
Decimal, string	**4%**			**40%**	**11%**
MVC	4%			7%	3%
AP				11%	3%
ZAP				9%	2%
CVD				5%	1%
MP				3%	1%
CLC				3%	1%
CP				2%	1%
ED				1%	0%
Total	**82%**	**95%**	**90%**	**85%**	**88%**

Figure K.59 Distribution of instruction execution frequencies for the four 360 programs. All instructions with a frequency of execution greater than 1.5% are included. Immediate instructions, which operate on only a single byte, are included in the section that characterized their operation, rather than with the long character-string versions of the same operation. By comparison, the average frequencies for the major instruction classes of the VAX are 23% (control), 28% (arithmetic), 29% (data transfer), 7% (floating point), and 9% (decimal). Once again, a 1% entry in the average column can occur because of entries in the constituent columns. These programs are a compiler for the programming language PL-I and runtime systems for the programming languages FORTRAN, PL/I, and Cobol.

K.6 Historical Perspective and References

Section L.4 (available online) features a discussion on the evolution of instruction sets and includes references for further reading and exploration of related topics.

Acknowledgments

We would like to thank the following people for comments on drafts of this survey: Professor Steven B. Furber, University of Manchester; Dr. Dileep Bhandarkar, Intel Corporation; Dr. Earl Killian, Silicon Graphics/MIPS; and Dr. Hiokazu Takata, Mitsubishi Electric Corporation.

L

Advanced Concepts on Address Translation

by Abhishek Bhattacharjee

Appendix L is available online at https://www.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1

M.1	Introduction	M-2
M.2	The Early Development of Computers (Chapter 1)	M-2
M.3	The Development of Memory Hierarchy and Protection (Chapter 2 and Appendix B)	M-9
M.4	The Evolution of Instruction Sets (Appendices A, J, and K)	M-17
M.5	The Development of Pipelining and Instruction-Level Parallelism (Chapter 3 and Appendices C and H)	M-27
M.6	The Development of SIMD Supercomputers, Vector Computers, Multimedia SIMD Instruction Extensions, and Graphical Processor Units (Chapter 4)	M-45
M.7	The History of Multiprocessors and Parallel Processing (Chapter 5 and Appendices F, G, and I)	M-55
M.8	The Development of Clusters (Chapter 6)	M-74
M.9	Historical Perspectives and References	M-79
M.10	The History of Magnetic Storage, RAID, and I/O Buses (Appendix D)	M-84

M

Historical Perspectives and References

If … history … teaches us anything, it is that man in his quest for knowledge and progress is determined and cannot be deterred.

John F. Kennedy

Address at Rice University (1962)

Those who cannot remember the past are condemned to repeat it.

George Santayana

The Life of Reason (1905), Vol. 2, Chapter 3

M.1 Introduction

This appendix provides historical background on some of the key ideas presented in the chapters. We may trace the development of an idea through a series of machines or describe significant projects. If you are interested in examining the initial development of an idea or machine or are interested in further reading, references are provided at the end of each section.

Section M.2 starts us off with the invention of the digital computer and corresponds to Chapter 1. Section M.3, on memory hierarchy, corresponds to Chapter 2 and Appendix B. Section M.4, on instruction set architecture, covers Appendices A, J, and K. Section M.5, on pipelining and instruction-level parallelism, corresponds to Chapter 3 and Appendices C and H. Section M.6, on data-level parallelism in vector, SIMD, and GPU architectures, corresponds to Chapter 4. Section M.7, on multiprocessors and parallel programming, covers Chapter 5 and Appendices F, G, and I. Section M.8, on the development of clusters, covers Chapter 6. Finally, Section M.9, on I/O, corresponds to Appendix D.

M.2 The Early Development of Computers (Chapter 1)

In this historical section, we discuss the early development of digital computers and the development of performance measurement methodologies.

The First General-Purpose Electronic Computers

J. Presper Eckert and John Mauchly at the Moore School of the University of Pennsylvania built the world's first fully operational electronic general-purpose computer. This machine, called ENIAC (Electronic Numerical Integrator and Calculator), was funded by the U.S. Army and became operational during World War II, but it was not publicly disclosed until 1946. ENIAC was used for computing artillery firing tables. The machine was enormous—100 feet long, 8½ feet high, and several feet wide. Each of the 20 ten-digit registers was 2 feet long. In total, there were 18,000 vacuum tubes.

Although the size was three orders of magnitude bigger than the size of the average machines built today, it was more than five orders of magnitude slower, with an add taking 200 microseconds. The ENIAC provided conditional jumps and was programmable, which clearly distinguished it from earlier calculators. Programming was done manually by plugging up cables and setting switches and required from a half hour to a whole day. Data were provided on punched cards. The ENIAC was limited primarily by a small amount of storage and tedious programming.

In 1944, John von Neumann was attracted to the ENIAC project. The group wanted to improve the way programs were entered and discussed storing programs as numbers; von Neumann helped crystallize the ideas and wrote a memo proposing a stored-program computer called EDVAC (Electronic Discrete Variable

Automatic Computer). Herman Goldstine distributed the memo and put von Neumann's name on it, much to the dismay of Eckert and Mauchly, whose names were omitted. This memo has served as the basis for the commonly used term *von Neumann computer*. Several early inventors in the computer field believe that this term gives too much credit to von Neumann, who conceptualized and wrote up the ideas, and too little to the engineers, Eckert and Mauchly, who worked on the machines. Like most historians, your authors (winners of the 2000 IEEE von Neumann Medal) believe that all three individuals played a key role in developing the stored-program computer. Von Neumann's role in writing up the ideas, in generalizing them, and in thinking about the programming aspects was critical in transferring the ideas to a wider audience.

In 1946, Maurice Wilkes of Cambridge University visited the Moore School to attend the latter part of a series of lectures on developments in electronic computers. When he returned to Cambridge, Wilkes decided to embark on a project to build a stored-program computer named EDSAC (Electronic Delay Storage Automatic Calculator). (The EDSAC used mercury delay lines for its memory; hence, the phrase "delay storage" in its name.) The EDSAC became operational in 1949 and was the world's first full-scale, operational, stored-program computer [Wilkes, Wheeler, and Gill 1951; Wilkes 1985, 1995]. (A small prototype called the Mark I, which was built at the University of Manchester and ran in 1948, might be called the first operational stored-program machine.) The EDSAC was an accumulator-based architecture. This style of instruction set architecture remained popular until the early 1970s. (Appendix A starts with a brief summary of the EDSAC instruction set.)

In 1947, Mauchly took the time to help found the Association for Computing Machinery. He served as the ACM's first vice-president and second president. That same year, Eckert and Mauchly applied for a patent on electronic computers. The dean of the Moore School, by demanding that the patent be turned over to the university, may have helped Eckert and Mauchly conclude that they should leave. Their departure crippled the EDVAC project, which did not become operational until 1952.

Goldstine left to join von Neumann at the Institute for Advanced Study at Princeton in 1946. Together with Arthur Burks, they issued a report based on the 1944 memo [Burks, Goldstine, and von Neumann 1946]. The paper led to the IAS machine built by Julian Bigelow at Princeton's Institute for Advanced Study. It had a total of 1024 40-bit words and was roughly 10 times faster than ENIAC. The group thought about uses for the machine, published a set of reports, and encouraged visitors. These reports and visitors inspired the development of a number of new computers, including the first IBM computer, the 701, which was based on the IAS machine. The paper by Burks, Goldstine, and von Neumann was incredible for the period. Reading it today, you would never guess this landmark paper was written more than 50 years ago, as most of the architectural concepts seen in modern computers are discussed there (e.g., see the quote at the beginning of Chapter 2).

In the same time period as ENIAC, Howard Aiken was designing an electromechanical computer called the Mark-I at Harvard. The Mark-I was built by a team

of engineers from IBM. He followed the Mark-I with a relay machine, the Mark-II, and a pair of vacuum tube machines, the Mark-III and Mark-IV. The Mark-III and Mark-IV were built after the first stored-program machines. Because they had separate memories for instructions and data, the machines were regarded as reactionary by the advocates of stored-program computers. The term *Harvard architecture* was coined to describe this type of machine. Though clearly different from the original sense, this term is used today to apply to machines with a single main memory but with separate instruction and data caches.

The Whirlwind project [Redmond and Smith 1980] began at MIT in 1947 and was aimed at applications in real-time radar signal processing. Although it led to several inventions, its overwhelming innovation was the creation of magnetic core memory, the first reliable and inexpensive memory technology. Whirlwind had 2048 16-bit words of magnetic core. Magnetic cores served as the main memory technology for nearly 30 years.

Important Special-Purpose Machines

During World War II, major computing efforts in both Great Britain and the United States focused on special-purpose code-breaking computers. The work in Great Britain was aimed at decrypting messages encoded with the German Enigma coding machine. This work, which occurred at a location called Bletchley Park, led to two important machines. The first, an electromechanical machine, conceived of by Alan Turing, was called BOMB [see Good in Metropolis, Howlett, and Rota 1980]. The second, much larger and electronic machine, conceived and designed by Newman and Flowers, was called COLOSSUS [see Randall in Metropolis, Howlett, and Rota 1980]. These were highly specialized cryptanalysis machines, which played a vital role in the war by providing the ability to read coded messages, especially those sent to U-boats. The work at Bletchley Park was highly classified (indeed, some of it is still classified), so its direct impact on the development of ENIAC, EDSAC, and other computers is difficult to trace, but it certainly had an indirect effect in advancing the technology and gaining understanding of the issues.

Similar work on special-purpose computers for cryptanalysis went on in the United States. The most direct descendent of this effort was the company Engineering Research Associates (ERA) [see Thomash in Metropolis, Howlett, and Rota 1980], which was founded after the war to attempt to commercialize on the key ideas. ERA built several machines that were sold to secret government agencies, and it was eventually purchased by Sperry-Rand, which had earlier purchased the Eckert Mauchly Computer Corporation.

Another early set of machines that deserves credit was a group of special-purpose machines built by Konrad Zuse in Germany in the late 1930s and early 1940s [see Bauer and Zuse in Metropolis, Howlett, and Rota 1980]. In addition to producing an operating machine, Zuse was the first to implement floating point, which von Neumann claimed was unnecessary! His early machines used a mechanical store that was smaller than other electromechanical solutions of the

time. His last machine was electromechanical but, because of the war, was never completed.

An important early contributor to the development of electronic computers was John Atanasoff, who built a small-scale electronic computer in the early 1940s [Atanasoff 1940]. His machine, designed at Iowa State University, was a special-purpose computer (called the ABC, for Atanasoff Berry Computer) that was never completely operational. Mauchly briefly visited Atanasoff before he built ENIAC, and several of Atanasoff's ideas (e.g., using binary representation) likely influenced Mauchly. The presence of the Atanasoff machine, delays in filing the ENIAC patents (the work was classified, and patents could not be filed until after the war), and the distribution of von Neumann's EDVAC paper were used to break the Eckert–Mauchly patent [Larson 1973]. Though controversy still rages over Atanasoff's role, Eckert and Mauchly are usually given credit for building the first working, general-purpose, electronic computer [Stern 1980]. Atanasoff, however, demonstrated several important innovations included in later computers. Atanasoff deserves much credit for his work, and he might fairly be given credit for the world's first special-purpose electronic computer and for possibly influencing Eckert and Mauchly.

Commercial Developments

In December 1947, Eckert and Mauchly formed Eckert-Mauchly Computer Corporation. Their first machine, the BINAC, was built for Northrop and was shown in August 1949. After some financial difficulties, the Eckert-Mauchly Computer Corporation was acquired by Remington-Rand, later called Sperry-Rand. Sperry-Rand merged the Eckert-Mauchly acquisition, ERA, and its tabulating business to form a dedicated computer division, called UNIVAC. UNIVAC delivered its first computer, the UNIVAC I, in June 1951. The UNIVAC I sold for $250,000 and was the first successful commercial computer—48 systems were built! Today, this early machine, along with many other fascinating pieces of computer lore, can be seen at the Computer History Museum in Mountain View, California. Other places where early computing systems can be visited include the Deutsches Museum in Munich and the Smithsonian Institution in Washington, D.C., as well as numerous online virtual museums.

IBM, which earlier had been in the punched card and office automation business, didn't start building computers until 1950. The first IBM computer, the IBM 701 based on von Neumann's IAS machine, shipped in 1952 and eventually sold 19 units [see Hurd in Metropolis, Howlett, and Rota 1980]. In the early 1950s, many people were pessimistic about the future of computers, believing that the market and opportunities for these "highly specialized" machines were quite limited. Nonetheless, IBM quickly became the most successful computer company. Their focus on reliability and customer- and market-driven strategies were key. Although the 701 and 702 were modest successes, IBM's follow-up machines, the 650, 704, and 705 (delivered in 1954 and 1955) were significant successes, each selling from 132 to 1800 computers.

Several books describing the early days of computing have been written by the pioneers [Goldstine 1972; Wilkes 1985, 1995], as well as Metropolis, Howlett, and Rota [1980], which is a collection of recollections by early pioneers. There are numerous independent histories, often built around the people involved [Slater 1987], as well as a journal, *Annals of the History of Computing*, devoted to the history of computing.

Development of Quantitative Performance Measures: Successes and Failures

In the earliest days of computing, designers set performance goals—ENIAC was to be 1000 times faster than the Harvard Mark-I, and the IBM Stretch (7030) was to be 100 times faster than the fastest machine in existence. What wasn't clear, though, was how this performance was to be measured. In looking back over the years, it is a consistent theme that each generation of computers obsoletes the performance evaluation techniques of the prior generation.

The original measure of performance was time to perform an individual operation, such as addition. Since most instructions took the same execution time, the timing of one gave insight into the others. As the execution times of instructions in a machine became more diverse, however, the time for one operation was no longer useful for comparisons. To take these differences into account, an *instruction mix* was calculated by measuring the relative frequency of instructions in a computer across many programs. The Gibson mix [Gibson 1970] was an early popular instruction mix. Multiplying the time for each instruction times its weight in the mix gave the user the *average instruction execution time*. (If measured in clock cycles, average instruction execution time is the same as average cycles per instruction.) Since instruction sets were similar, this was a more accurate comparison than add times. From average instruction execution time, then, it was only a small step to MIPS (as we have seen, the one is the inverse of the other). MIPS had the virtue of being easy for the layperson to understand.

As CPUs became more sophisticated and relied on memory hierarchies and pipelining, there was no longer a single execution time per instruction; MIPS could not be calculated from the mix and the manual. The next step was benchmarking using kernels and synthetic programs. Curnow and Wichmann [1976] created the Whetstone synthetic program by measuring scientific programs written in Algol 60. This program was converted to FORTRAN and was widely used to characterize scientific program performance. An effort with similar goals to Whetstone, the Livermore FORTRAN Kernels, was made by McMahon [1986] and researchers at Lawrence Livermore Laboratory in an attempt to establish a benchmark for supercomputers. These kernels, however, consisted of loops from real programs.

As it became clear that using MIPS to compare architectures with different instruction sets would not work, a notion of relative MIPS was created. When the VAX-11/780 was ready for announcement in 1977, DEC ran small benchmarks that were also run on an IBM 370/158. IBM marketing referred to the 370/158 as a

1 MIPS computer, and, because the programs ran at the same speed, DEC marketing called the VAX-11/780 a 1 MIPS computer. Relative MIPS for a machine M was defined based on some reference machine as:

$$\text{MIPS}_{\text{M}} = \frac{\text{Performance}_{\text{M}}}{\text{Performance}_{\text{reference}}} \times \text{MIPS}_{\text{reference}}$$

The popularity of the VAX-11/780 made it a popular reference machine for relative MIPS, especially since relative MIPS for a 1 MIPS computer is easy to calculate: If a machine was five times faster than the VAX-11/780, for that benchmark its rating would be 5 relative MIPS. The 1 MIPS rating was unquestioned for 4 years, until Joel Emer of DEC measured the VAX-11/780 under a time-sharing load. He found that the VAX-11/780 native MIPS rating was 0.5. Subsequent VAXes that ran 3 native MIPS for some benchmarks were therefore called 6 MIPS machines because they ran six times faster than the VAX-11/780. By the early 1980s, the term *MIPS* was almost universally used to mean relative MIPS.

The 1970s and 1980s marked the growth of the supercomputer industry, which was defined by high performance on floating-point-intensive programs. Average instruction time and MIPS were clearly inappropriate metrics for this industry, hence the invention of MFLOPS (millions of floating-point operations per second), which effectively measured the inverse of execution time for a benchmark. Unfortunately, customers quickly forget the program used for the rating, and marketing groups decided to start quoting peak MFLOPS in the supercomputer performance wars.

SPEC (System Performance and Evaluation Cooperative) was founded in the late 1980s to try to improve the state of benchmarking and make a more valid basis for comparison. The group initially focused on workstations and servers in the UNIX marketplace, and these remain the primary focus of these benchmarks today. The first release of SPEC benchmarks, now called SPEC89, was a substantial improvement in the use of more realistic benchmarks. SPEC2006 still dominates processor benchmarks almost two decades later.

References

Amdahl, G. M. [1967]. "Validity of the single processor approach to achieving large scale computing capabilities," *Proc. AFIPS Spring Joint Computer Conf.*, April 18–20, 1967, Atlantic City, N.J., 483–485.

Atanasoff, J. V. [1940]. "Computing machine for the solution of large systems of linear equations," Internal Report, Iowa State University, Ames.

Azizi, O., Mahesri, A., Lee, B. C., Patel, S. J., & Horowitz, M. [2010]. Energy-performance tradeoffs in processor architecture and circuit design: a marginal cost analysis. Proc. International Symposium on Computer Architecture, 26-36.

Bell, C. G. [1984]. "The mini and micro industries," *IEEE Computer* 17:10 (October), 14–30.

Bell, C. G., J. C. Mudge, and J. E. McNamara [1978]. *A DEC View of Computer Engineering*, Digital Press, Bedford, Mass.

Burks, A. W., H. H. Goldstine, and J. von Neumann [1946]. "Preliminary discussion of the logical design of an electronic computing instrument," Report to the U.S. Army Ordnance Department, p. 1; also appears in *Papers of John von Neumann*, W. Aspray and A. Burks, eds., MIT Press, Cambridge, Mass., and Tomash Publishers, Los Angeles, Calif., 1987, 97–146.

Curnow, H. J., and B. A. Wichmann [1976]. "A synthetic benchmark," *The Computer J.* 19:1, 43–49.

Dally, William J., "High Performance Hardware for Machine Learning," Cadence Embedded Neural Network Summit, February 9, 2016. http://ip.cadence.com/uploads/presentations/1000AM_Dally_Cadence_ENN.pdf

Flemming, P. J., and J. J. Wallace [1986]. "How not to lie with statistics: The correct way to summarize benchmarks results," *Communications of the ACM* 29:3 (March), 218–221.

Fuller, S. H., and W. E. Burr [1977]. "Measurement and evaluation of alternative computer architectures," *Computer* 10:10 (October), 24–35.

Gibson, J. C. [1970]. "The Gibson mix," Rep. TR. 00.2043, IBM Systems Development Division, Poughkeepsie, N.Y. (research done in 1959).

Goldstine, H. H. [1972]. *The Computer: From Pascal to von Neumann*, Princeton University Press, Princeton, N.J.

Gray, J., and C. van Ingen [2005]. *Empirical Measurements of Disk Failure Rates and Error Rates*, MSR-TR-2005-166, Microsoft Research, Redmond, Wash.

Jain, R. [1991]. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, Wiley, New York.

Kembel, R. [2000]. "Fibre Channel: A comprehensive introduction," *Internet Week* (April).

Larson, E. R. [1973]. "Findings of fact, conclusions of law, and order for judgment," File No. 4-67, Civ. 138, *Honeywell v. Sperry-Rand and Illinois Scientific Development*, U.S. District Court for the State of Minnesota, Fourth Division (October 19).

Lubeck, O., J. Moore, and R. Mendez [1985]. "A benchmark comparison of three supercomputers: Fujitsu VP-200, Hitachi S810/20, and Cray X-MP/2," *Computer* 18:12 (December), 10–24.

Landstrom, B. [2014]. "The Cost Of Downtime," http://www.interxion.com/blogs/2014/07/the-cost-of-downtime/

McMahon, F. M. [1986]. *The Livermore FORTRAN Kernels: A Computer Test of Numerical Performance Range*, Tech. Rep. UCRL-55745, Lawrence Livermore National Laboratory, University of California, Livermore.

Metropolis, N., J. Howlett, and G. C. Rota, eds. [1980]. *A History of Computing in the Twentieth Century*, Academic Press, New York.

Mukherjee S. S., C. Weaver, J. S. Emer, S. K. Reinhardt, and T. M. Austin [2003]. "Measuring architectural vulnerability factors," *IEEE Micro* 23:6, 70–75.

Oliker, L., A. Canning, J. Carter, J. Shalf, and S. Ethier [2004]. "Scientific computations on modern parallel vector systems," *Proc. ACM/IEEE Conf. on Supercomputing*, November 6–12, 2004, Pittsburgh, Penn., 10.

Patterson, D. [2004]. "Latency lags bandwidth," *Communications of the ACM* 47:10 (October), 71–75.
Redmond, K. C., and T. M. Smith [1980]. *Project Whirlwind—The History of a Pioneer Computer*, Digital Press, Boston.
Shurkin, J. [1984]. *Engines of the Mind: A History of the Computer*, W. W. Norton, New York.
Slater, R. [1987]. *Portraits in Silicon*, MIT Press, Cambridge, Mass.
Smith, J. E. [1988]. "Characterizing computer performance with a single number," *Communications of the ACM* 31:10 (October), 1202–1206.
SPEC. [1989]. *SPEC Benchmark Suite Release 1.0* (October 2).
SPEC. [1994]. *SPEC Newsletter* (June).
Stern, N. [1980]. "Who invented the first electronic digital computer?" *Annals of the History of Computing* 2:4 (October), 375–376.
Touma, W. R. [1993]. *The Dynamics of the Computer Industry: Modeling the Supply of Workstations and Their Components*, Kluwer Academic, Boston.
Weicker, R. P. [1984]. "Dhrystone: A synthetic systems programming benchmark," *Communications of the ACM* 27:10 (October), 1013–1030.
Wilkes, M. V. [1985]. *Memoirs of a Computer Pioneer*, MIT Press, Cambridge, Mass.
Wilkes, M. V. [1995]. *Computing Perspectives*, Morgan Kaufmann, San Francisco.
Wilkes, M. V., D. J. Wheeler, and S. Gill [1951]. *The Preparation of Programs for an Electronic Digital Computer*, Addison-Wesley, Cambridge, Mass.

M.3 The Development of Memory Hierarchy and Protection (Chapter 2 and Appendix B)

Although the pioneers of computing knew of the need for a memory hierarchy and coined the term, the automatic management of two levels was first proposed by Kilburn et al. [1962]. It was demonstrated with the Atlas computer at the University of Manchester. This computer appeared the year before the IBM 360 was announced. Although IBM planned for its introduction with the next generation (System/370), the operating system TSS was not up to the challenge in 1970. Virtual memory was announced for the 370 family in 1972, and it was for this computer that the term *translation lookaside buffer* was coined [Case and Padegs 1978]. The only computers today without virtual memory are a few supercomputers, embedded processors, and older personal computers.

Both the Atlas and the IBM 360 provided protection on pages, and the GE 645 was the first system to provide paged segmentation. The earlier Burroughs computers provided virtual memory using segmentation, similar to the segmented address scheme of the Intel 8086. The 80286, the first 80x86 to have the protection mechanisms described in Appendix C, was inspired by the Multics protection software that ran on the GE 645. Over time, computers

evolved more elaborate mechanisms. The most elaborate mechanism was *capabilities*, which attracted the greatest interest in the late 1970s and early 1980s [Fabry 1974; Wulf, Levin, and Harbison 1981]. Wilkes [1982], one of the early workers on capabilities, had this to say:

Anyone who has been concerned with an implementation of the type just described [capability system], or has tried to explain one to others, is likely to feel that complexity has got out of hand. It is particularly disappointing that the attractive idea of capabilities being tickets that can be freely handed around has become lost ….

Compared with a conventional computer system, there will inevitably be a cost to be met in providing a system in which the domains of protection are small and frequently changed. This cost will manifest itself in terms of additional hardware, decreased runtime speed, and increased memory occupancy. It is at present an open question whether, by adoption of the capability approach, the cost can be reduced to reasonable proportions. [p. 112]

Today there is little interest in capabilities either from the operating systems or the computer architecture communities, despite growing interest in protection and security.

Bell and Strecker [1976] reflected on the PDP-11 and identified a small address space as the only architectural mistake that is difficult to recover from. At the time of the creation of PDP-11, core memories were increasing at a very slow rate. In addition, competition from 100 other minicomputer companies meant that DEC might not have a cost-competitive product if every address had to go through the 16-bit data path twice, hence the architect's decision to add only 4 more address bits than found in the predecessor of the PDP-11.

The architects of the IBM 360 were aware of the importance of address size and planned for the architecture to extend to 32 bits of address. Only 24 bits were used in the IBM 360, however, because the low-end 360 models would have been even slower with the larger addresses in 1964. Unfortunately, the architects didn't reveal their plans to the software people, and programmers who stored extra information in the upper 8 "unused" address bits foiled the expansion effort. (Apple made a similar mistake 20 years later with the 24-bit address in the Motorola 68000, which required a procedure to later determine "32-bit clean" programs for the Macintosh when later 68000s used the full 32-bit virtual address.) Virtually every computer since then will check to make sure the unused bits stay unused and trap if the bits have the wrong value.

As mentioned in the text, system virtual machines were pioneered at IBM as part of its investigation into virtual memory. IBM's first computer with virtual memory was the IBM 360/67, introduced in 1967. IBM researchers wrote the program CP-67 that created the illusion of several independent 360 computers. They then wrote an interactive, single-user operating system called CMS that ran on these virtual machines. CP-67 led to the product VM/370, and today IBM sells z/VM for its mainframe computers [Meyer and Seawright 1970; Van Vleck 2005].

A few years after the Atlas paper, Wilkes published the first paper describing the concept of a cache [1965]:

The use is discussed of a fast core memory of, say, 32,000 words as slave to a slower core memory of, say, one million words in such a way that in practical cases the effective access time is nearer that of the fast memory than that of the slow memory. [p. 270]

This two-page paper describes a direct-mapped cache. Although this is the first publication on caches, the first implementation was probably a direct-mapped instruction cache built at the University of Cambridge. It was based on tunnel diode memory, the fastest form of memory available at the time. Wilkes stated that G. Scarott suggested the idea of a cache memory.

Subsequent to that publication, IBM started a project that led to the first commercial computer with a cache, the IBM 360/85 [Liptay 1968]. Gibson [1967] described how to measure program behavior as memory traffic as well as miss rate and showed how the miss rate varies between programs. Using a sample of 20 programs (each with 3 million references!), Gibson also relied on average memory access time to compare systems with and without caches. This precedent is more than 40 years old, and yet many used miss rates until the early 1990s.

Conti, Gibson, and Pitkowsky [1968] described the resulting performance of the 360/85. The 360/91 outperforms the 360/85 on only 3 of the 11 programs in the paper, even though the 360/85 has a slower clock cycle time (80 ns versus 60 ns), less memory interleaving (4 versus 16), and a slower main memory (1.04 microsecond versus 0.75 microsecond). This paper was also the first to use the term *cache*.

Others soon expanded the cache literature. Strecker [1976] published the first comparative cache design paper examining caches for the PDP-11. Smith [1982] later published a thorough survey paper that used the terms *spatial locality* and *temporal locality*; this paper has served as a reference for many computer designers.

Although most studies relied on simulations, Clark [1983] used a hardware monitor to record cache misses of the VAX-11/780 over several days. Clark and Emer [1985] later compared simulations and hardware measurements for translations.

Hill [1987] proposed the three C's used in Appendix B to explain cache misses. Jouppi [1998] retrospectively said that Hill's three C's model led directly to his invention of the victim cache to take advantage of faster direct-mapped caches and yet avoid most of the cost of conflict misses. Sugumar and Abraham [1993] argued that the baseline cache for the three C's model should use optimal replacement; this would eliminate the anomalies of least recently used (LRU)-based miss classification and allow conflict misses to be broken down into those caused by mapping and those caused by a nonoptimal replacement algorithm.

One of the first papers on nonblocking caches was by Kroft [1981]. Kroft [1998] later explained that he was the first to design a computer with a cache at

Control Data Corporation, and when using old concepts for new mechanisms he hit upon the idea of allowing his two-ported cache to continue to service other accesses on a miss.

Baer and Wang [1988] did one of the first examinations of the multilevel inclusion property. Wang, Baer, and Levy [1989] then produced an early paper on performance evaluation of multilevel caches. Later, Jouppi and Wilton [1994] proposed multilevel exclusion for multilevel caches on chip.

In addition to victim caches, Jouppi [1990] also examined prefetching via streaming buffers. His work was extended by Farkas, Jouppi, and Chow [1995] to streaming buffers that work well with nonblocking loads and speculative execution for in-order processors, and later Farkas et al. [1997] showed that, while out-of-order processors can tolerate unpredictable latency better, they still benefit. They also refined memory bandwidth demands of stream buffers.

Proceedings of the Symposium on Architectural Support for Compilers and Operating Systems (ASPLOS) and the International Computer Architecture Symposium (ISCA) from the 1990s are filled with papers on caches. (In fact, some wags claimed ISCA really stood for the International *Cache* Architecture Symposium.)

Chapter 2 relies on the measurements of SPEC2000 benchmarks collected by Cantin and Hill [2001]. There are several other papers used in Chapter 2 that are cited in the captions of the figures that use the data: Agarwal and Pudar [1993]; Barroso, Gharachorloo, and Bugnion [1998]; Farkas and Jouppi [1994]; Jouppi [1990]; Lam, Rothberg, and Wolf [1991]; Lebeck and Wood [1994]; McCalpin [2005]; Mowry, Lam, and Gupta [1992]; and Torrellas, Gupta, and Hennessy [1992].

References

Agarwal, A. [1987]. "Analysis of Cache Performance for Operating Systems and Multiprogramming," Ph.D. thesis, Tech. Rep. No. CSL-TR-87-332, Stanford University, Palo Alto, Calif.

Agarwal, A., and S. D. Pudar [1993]. "Column-associative caches: A technique for reducing the miss rate of direct-mapped caches," *20th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 16–19, 1993, San Diego, Calif. (*Computer Architecture News* 21:2 (May), 179–190).

Baer, J.-L., and W.-H. Wang [1988]. "On the inclusion property for multi-level cache hierarchies," *Proc. 15th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 30–June 2, 1988, Honolulu, Hawaii, 73–80.

Barham, P., B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, and R. Neugebauer [2003]. "Xen and the art of virtualization," *Proc. of the 19th ACM Symposium on Operating Systems Principles*, October 19–22, 2003, Bolton Landing, N.Y.

Barroso, L. A., K. Gharachorloo, and E. Bugnion [1998]. "Memory system characterization of commercial workloads," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 3–14.

Bell, C. G., and W. D. Strecker [1976]. "Computer structures: What have we learned from the PDP-11?" *Proc. Third Annual Int'l. Symposium on Computer Architecture (ISCA)*, January 19–21, 1976, Tampa, Fla., 1–14.

Bhandarkar, D. P. [1995]. *Alpha Architecture Implementations*, Digital Press, Newton, Mass.

Borg, A., R. E. Kessler, and D. W. Wall [1990]. "Generation and analysis of very long address traces," *Proc. 17th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–31, 1990, Seattle, Wash., 270–279.

Cantin, J. F., and M. D. Hill [2001]. "Cache performance for selected SPEC CPU2000 benchmarks," *http://www.cs.wisc.edu/multifacet/misc/spec2000cache-data/*.

Cantin, J., and M. Hill [2003]. "Cache performance for SPEC CPU2000 benchmarks, version 3.0," *http://www.cs.wisc.edu/multifacet/misc/spec2000cache-data/index.html*.

Case, R. P., and A. Padegs [1978]. "The architecture of the IBM System/370," *Communications of the ACM* 21:1, 73–96. Also appears in D. P. Siewiorek, C. G. Bell, and A. Newell, *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 1982, 830–855.

Clark, B., T. Deshane, E. Dow, S. Evanchik, M. Finlayson, J. Herne, and J. Neefe Matthews [2004]. "Xen and the art of repeated research," *Proc. USENIX Annual Technical Conf.*, June 27–July 2, 2004, Boston, 1135–1144.

Clark, D. W. [1983]. "Cache performance of the VAX-11/780," *ACM Trans. on Computer Systems* 1:1, 24–37.

Clark, D. W., and J. S. Emer [1985]. "Performance of the VAX-11/780 translation buffer: Simulation and measurement," *ACM Trans. on Computer Systems* 3:1 (February), 31–62.

Compaq Computer Corporation. [1999]. *Compiler Writer's Guide for the Alpha 21264*, Order Number EC-RJ66A-TE, June.

Conti, C., D. H. Gibson, and S. H. Pitkowsky [1968]. "Structural aspects of the System/360 Model 85. Part I. General organization," *IBM Systems J.* 7:1, 2–14.

Crawford, J., and P. Gelsinger [1988]. *Programming the 80386*, Sybex, Alameda, Calif.

Cvetanovic, Z., and R. E. Kessler [2000]. "Performance analysis of the Alpha 21264-based Compaq ES40 system," *Proc. 27th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 10–14, 2000, Vancouver, Canada, 192–202.

Fabry, R. S. [1974]. "Capability based addressing," *Communications of the ACM* 17:7 (July), 403–412.

Farkas, K. I., P. Chow, N. P. Jouppi, and Z. Vranesic [1997]. "Memory-system design considerations for dynamically-scheduled processors," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo., 133–143.

Farkas, K. I., and N. P. Jouppi [1994]. "Complexity/performance trade-offs with non-blocking loads," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago.

Farkas, K. I., N. P. Jouppi, and P. Chow [1995]. "How useful are non-blocking loads, stream buffers and speculative execution in multiple issue processors?" *Proc. First IEEE Symposium on High-Performance Computer Architecture*, January 22–25, 1995, Raleigh, N.C., 78–89.

Gao, Q. S. [1993]. "The Chinese remainder theorem and the prime memory system," *20th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 16–19, 1993, San Diego, Calif. (*Computer Architecture News* 21:2 (May), 337–340).

Gee, J. D., M. D. Hill, D. N. Pnevmatikatos, and A. J. Smith [1993]. "Cache performance of the SPEC92 benchmark suite," *IEEE Micro* 13:4 (August), 17–27.

Gibson, D. H. [1967]. "Considerations in block-oriented systems design," *AFIPS Conf. Proc.* 30, 75–80.

Handy, J. [1993]. *The Cache Memory Book*, Academic Press, Boston.

Heald, R., K. Aingaran, C. Amir, M. Ang, M. Boland, A. Das, P. Dixit, G. Goulsberry, J. Hart, T. Horel, W.-J. Hsu, J. Kaku, C. Kim, S. Kim, F. Klass, H. Kwan, R. Lo, H. McIntyre, A. Mehta, D. Murata, S. Nguyen, Y.-P. Pai, S. Patel, K. Shin, K. Tam, S. Vishwanthaiah, J. Wu, G. Yee, and H. You [2000]. "Implementation of third-generation SPARC V9 64-b microprocessor," *ISSCC Digest of Technical Papers*, 412–413 and slide supplement.

Hill, M. D. [1987]. "Aspects of Cache Memory and Instruction Buffer Performance," Ph.D. thesis, Tech. Rep. UCB/CSD 87/381, Computer Science Division, University of California, Berkeley.

Hill, M. D. [1988]. "A case for direct mapped caches," *Computer* 21:12 (December), 25–40.

Horel, T., and G. Lauterbach [1999]. "UltraSPARC-III: Designing third-generation 64-bit performance," *IEEE Micro* 19:3 (May–June), 73–85.

Hughes, C. J., P. Kaul, S. V. Adve, R. Jain, C. Park, and J. Srinivasan [2001]. "Variability in the execution of multimedia applications and implications for architecture," *Proc. 28th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 30–July 4, 2001, Goteborg, Sweden, 254–265.

IEEE. [2005]. "Intel virtualization technology, computer," *IEEE Computer Society* 38:5 (May), 48–56.

Jouppi, N. P. [1990]. "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," *Proc. 17th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–31, 1990, Seattle, Wash., 364–373.

Jouppi, N. P. [1998]. "Retrospective: Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in G. S. Sohi, ed., *25 Years of the International Symposia on Computer Architecture (Selected Papers)*, ACM, New York, 71–73.

Jouppi, N. P., and S. J. E. Wilton [1994]. "Trade-offs in two-level on-chip caching," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago, 34–45.

Kessler, R. E. [1999]. "The Alpha 21264 microprocessor," *IEEE Micro* 19:2 (March/April), 24–36.

Kilburn, T., D. B. G. Edwards, M. J. Lanigan, and F. H. Sumner [1962]. "One-level storage system," *IRE Trans. on Electronic Computers* EC-11 (April) 223–235. Also appears in D. P. Siewiorek, C. G. Bell, and A. Newell, *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 1982, 135–148.

Kroft, D. [1981]. "Lockup-free instruction fetch/prefetch cache organization," *Proc. Eighth Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 12–14, 1981, Minneapolis, Minn., 81–87.

Kroft, D. [1998]. "Retrospective: Lockup-free instruction fetch/prefetch cache organization," in G. S. Sohi, ed., *25 Years of the International Symposia on Computer Architecture (Selected Papers)*, ACM, New York, 20–21.

Kunimatsu, A., N. Ide, T. Sato, Y. Endo, H. Murakami, T. Kamei, M. Hirano, F. Ishihara, H. Tago, M. Oka, A. Ohba, T. Yutaka, T. Okada, and M. Suzuoki [2000]. "Vector unit architecture for emotion synthesis," *IEEE Micro* 20:2 (March–April), 40–47.

Lam, M. S., E. E. Rothberg, and M. E. Wolf [1991]. "The cache performance and optimizations of blocked algorithms," *Proc. Fourth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 8–11, 1991, Santa Clara, Calif. (*SIGPLAN Notices* 26:4 (April), 63–74).

Lebeck, A. R., and D. A. Wood [1994]. "Cache profiling and the SPEC benchmarks: A case study," *Computer* 27:10 (October), 15–26.

Liptay, J. S. [1968]. "Structural aspects of the System/360 Model 85. Part II. The cache," *IBM Systems J.* 7:1, 15–21.

Luk, C.-K., and T. C Mowry [1999]. "Automatic compiler-inserted prefetching for pointer-based applications," *IEEE Trans. on Computers*, 48:2 (February), 134–141.

McCalpin, J. D. [2005]. "STREAM: Sustainable Memory Bandwidth in High Performance Computers," *www.cs.virginia.edu/stream/*.

McFarling, S. [1989]. "Program optimization for instruction caches," *Proc. Third Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 3–6, 1989, Boston, 183–191.

Menon, A., J. Renato Santos, Y. Turner, G. Janakiraman, and W. Zwaenepoel [2005]. "Diagnosing performance overheads in the xen virtual machine environment," *Proc. First ACM/USENIX Int'l. Conf. on Virtual Execution Environments*, June 11–12, 2005, Chicago, 13–23.

Meyer, R. A., and L. H. Seawright [1970]. "A virtual machine time sharing system," *IBM Systems J.* 9:3, 199–218.

Mowry, T. C., S. Lam, and A. Gupta [1992]. "Design and evaluation of a compiler algorithm for prefetching," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston (*SIGPLAN Notices* 27:9 (September), 62–73).

Oka, M., and M. Suzuoki [1999]. "Designing and programming the emotion engine," *IEEE Micro* 19:6 (November–December), 20–28.

Pabst, T. [2000]. "Performance Showdown at 133 MHz FSB—The Best Platform for Coppermine," *www6.tomshardware.com/mainboard/00q1/000302/*.

Palacharla, S., and R. E. Kessler [1994]. "Evaluating stream buffers as a secondary cache replacement," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago, 24–33.

Przybylski, S. A. [1990]. *Cache Design: A Performance-Directed Approach*, Morgan Kaufmann, San Francisco.

Przybylski, S. A., M. Horowitz, and J. L. Hennessy [1988]. "Performance tradeoffs in cache design," *Proc. 15th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 30–June 2, 1988, Honolulu, Hawaii, 290–298.

Reinman, G., and N. P. Jouppi. [1999]. "Extensions to CACTI."

Robin, J., and C. Irvine [2000]. "Analysis of the Intel Pentium's ability to support a secure virtual machine monitor," *Proc. USENIX Security Symposium*, August 14–17, 2000, Denver, Colo.

Saavedra-Barrera, R. H. [1992]. "CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking," Ph.D. dissertation, University of California, Berkeley.

Samples, A. D., and P. N. Hilfinger [1988]. *Code Reorganization for Instruction Caches*, Tech. Rep. UCB/CSD 88/447, University of California, Berkeley.

Sites, R. L. (ed.) [1992]. *Alpha Architecture Reference Manual*, Digital Press, Burlington, Mass.

Skadron, K., and D. W. Clark [1997]. "Design issues and tradeoffs for write buffers," *Proc. Third Int'l. Symposium on High-Performance Computer Architecture*, February 1–5, 1997, San Antonio, Tex., 144–155.

Smith, A. J. [1982]. "Cache memories," *Computing Surveys* 14:3 (September), 473–530.

Smith, J. E., and J. R. Goodman [1983]. "A study of instruction cache organizations and replacement policies," *Proc. 10th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1982, Stockholm, Sweden, 132–137.

Stokes, J. [2000]. "Sound and Vision: A Technical Overview of the Emotion Engine," *http://arstechnica.com/hardware/reviews/2000/02/ee.ars*.

Strecker, W. D. [1976]. "Cache memories for the PDP-11?" *Proc. Third Annual Int'l. Symposium on Computer Architecture (ISCA)*, January 19–21, 1976, Tampa, Fla., 155–158.

Sugumar, R. A., and S. G. Abraham [1993]. "Efficient simulation of caches under optimal replacement with applications to miss characterization," *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, May 17–21, 1993, Santa Clara, Calif., 24–35.

Tarjan, D., S. Thoziyoor, and N. Jouppi [2006]. CACTI 4.0. Technical Report HPL-2006-86, HP Laboratories.

Torrellas, J., A. Gupta, and J. Hennessy [1992]. "Characterizing the caching and synchronization performance of a multiprocessor operating system," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston (*SIGPLAN Notices* 27:9 (September), 162–174).

Van Vleck, T. [2005]. "The IBM 360/67 and CP/CMS," *http://www.multicians.org/thvv/360-67.html*.

Wang, W.-H., J.-L. Baer, and H. M. Levy [1989]. "Organization and performance of a two-level virtual-real cache hierarchy," *Proc. 16th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–June 1, 1989, Jerusalem, 140–148.

Wilkes, M. [1965]. "Slave memories and dynamic storage allocation," *IEEE Trans. Electronic Computers* EC-14:2 (April), 270–271.

Wilkes, M. V. [1982]. "Hardware support for memory protection: Capability implementations," *Proc. Symposium on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, March 1–3, 1982, Palo Alto, Calif., 107–116.

Wulf, W. A., R. Levin, and S. P. Harbison [1981]. *Hydra/C.mmp: An Experimental Computer System*, McGraw-Hill, New York.

M.4 The Evolution of Instruction Sets (Appendices A, J, and K)

One's eyebrows should rise whenever a future architecture is developed with a stack- or register-oriented instruction set.

Meyers [1978, p. 20]

The earliest computers, including the UNIVAC I, the EDSAC, and the IAS computers, were accumulator-based computers. The simplicity of this type of computer made it the natural choice when hardware resources were very constrained. The first general-purpose register computer was the Pegasus, built by Ferranti, Ltd., in 1956. The Pegasus had eight general-purpose registers, with R0 always being zero. Block transfers loaded the eight registers from the drum memory.

Stack Architectures

In 1963, Burroughs delivered the B5000. The B5000 was perhaps the first computer to seriously consider software and hardware-software trade-offs. Barton and the designers at Burroughs made the B5000 a stack architecture (as described in Barton [1961]). Designed to support high-level languages such as ALGOL, this stack architecture used an operating system (MCP) written in a high-level language. The B5000 was also the first computer from a U.S. manufacturer to support virtual memory. The B6500, introduced in 1968 (and discussed in Hauck and Dent [1968]), added hardware-managed activation records. In both the B5000 and B6500, the top two elements of the stack were kept in the processor and the rest of the stack was kept in memory. The stack architecture yielded good code density, but only provided two high-speed storage locations. The authors of both the original IBM 360 paper [Amdahl, Blaauw, and Brooks 1964] and the original PDP-11 paper [Bell et al. 1970] argued against the stack organization. They cited three major points in their arguments against stacks:

- Performance is derived from fast registers, not the way they are used.
- The stack organization is too limiting and requires many swap and copy operations.
- The stack has a bottom, and when placed in slower memory there is a performance loss.

Stack-based hardware fell out of favor in the late 1970s and, except for the Intel 80x86 floating-point architecture, essentially disappeared; for example, except for the 80x86, none of the computers listed in the SPEC report uses a stack.

In the 1990s, however, stack architectures received a shot in the arm with the success of the Java Virtual Machine (JVM). The JVM is a software interpreter for an intermediate language produced by Java compilers, called *Java bytecodes* [Lindholm and Yellin 1999]. The purpose of the interpreter is to provide software compatibility across many platforms, with the hope of "write once, run everywhere." Although the slowdown is about a factor of 10 due to interpretation, there are times when compatibility is more important than performance, such as when downloading a Java "applet" into an Internet browser.

Although a few have proposed hardware to directly execute the JVM instructions (see McGhan and O'Connor [1998]), thus far none of these proposals has been significant commercially. The hope instead is that *just-in-time* (JIT) Java compilers—which compile during runtime to the native instruction set of the computer running the Java program—will overcome the performance penalty of interpretation. The popularity of Java has also led to compilers that compile directly into the native hardware instruction sets, bypassing the illusion of the Java bytecodes.

Computer Architecture Defined

IBM coined the term *computer architecture* in the early 1960s. Amdahl, Blaauw, and Brooks [1964] used the term to refer to the programmer-visible portion of the IBM 360 instruction set. They believed that a *family* of computers of the same architecture should be able to run the same software. Although this idea may seem obvious to us today, it was quite novel at that time. IBM, although it was the leading company in the industry, had five different architectures before the 360; thus, the notion of a company standardizing on a single architecture was a radical one. The 360 designers hoped that defining a common architecture would bring six different divisions of IBM together. Their definition of architecture was

… the structure of a computer that a machine language programmer must understand to write a correct (timing independent) program for that machine.

The term *machine language programmer* meant that compatibility would hold, even in machine language, while *timing independent* allowed different implementations. This architecture blazed the path for binary compatibility, which others have followed.

The IBM 360 was the first computer to sell in large quantities with both byte addressing using 8-bit bytes and general-purpose registers. The 360 also had register-memory and limited memory-memory instructions. Appendix K summarizes this instruction set.

In 1964, Control Data delivered the first supercomputer, the CDC 6600. As Thornton [1964] discussed, he, Cray, and the other 6600 designers were among the first to explore pipelining in depth. The 6600 was the first general-purpose, load-store computer. In the 1960s, the designers of the 6600 realized the need to simplify architecture for the sake of efficient pipelining. Microprocessor and minicomputer designers largely neglected this interaction between architectural simplicity and implementation during the 1970s, but it returned in the 1980s.

High-Level Language Computer Architecture

In the late 1960s and early 1970s, people realized that software costs were growing faster than hardware costs. McKeeman [1967] argued that compilers and operating systems were getting too big and too complex and taking too long to develop. Because of inferior compilers and the memory limitations of computers, most systems programs at the time were still written in assembly language. Many researchers proposed alleviating the software crisis by creating more powerful, software-oriented architectures. Tanenbaum [1978] studied the properties of high-level languages. Like other researchers, he found that most programs are simple. He argued that architectures should be designed with this in mind and that they should optimize for program size and ease of compilation. Tanenbaum proposed a stack computer with frequency-encoded instruction formats to accomplish these goals; however, as we have observed, program size does not translate directly to cost-performance, and stack computers faded out shortly after this work.

Strecker's article [1978] discusses how he and the other architects at DEC responded to this by designing the VAX architecture. The VAX was designed to simplify compilation of high-level languages. Compiler writers had complained about the lack of complete orthogonality in the PDP-11. The VAX architecture was designed to be highly orthogonal and to allow the mapping of a high-level language statement into a single VAX instruction. Additionally, the VAX designers tried to optimize code size because compiled programs were often too large for available memories. Appendix K summarizes this instruction set.

The VAX-11/780 was the first computer announced in the VAX series. It is one of the most successful—and most heavily studied—computers ever built. The cornerstone of DEC's strategy was a single architecture, VAX, running a single operating system, VMS. This strategy worked well for over 10 years. The large number of papers reporting instruction mixes, implementation measurements, and analysis of the VAX makes it an ideal case study [Clark and Levy 1982; Wiecek 1982]. Bhandarkar and Clark [1991] gave a quantitative analysis of the disadvantages of the VAX versus a RISC computer, essentially a technical explanation for the demise of the VAX.

While the VAX was being designed, a more radical approach, called *high-level language computer architecture* (HLLCA), was being advocated in the research community. This movement aimed to eliminate the gap between high-level languages and computer hardware—what Gagliardi [1973] called the "semantic gap"—by bringing the hardware "up to" the level of the programming language. Meyers [1982] provided a good summary of the arguments and a history of high-level language computer architecture projects. HLLCA never had a significant commercial impact. The increase in memory size on computers eliminated the code size problems arising from high-level languages and enabled operating systems to be written in high-level languages. The combination of simpler architectures together with software offered greater performance and more flexibility at lower cost and lower complexity.

Reduced Instruction Set Computers

In the early 1980s, the direction of computer architecture began to swing away from providing high-level hardware support for languages. Ditzel and Patterson [1980] analyzed the difficulties encountered by the high-level language architectures and argued that the answer lay in simpler architectures. In another paper [Patterson and Ditzel 1980], these authors first discussed the idea of Reduced Instruction Set Computers (RISCs) and presented the argument for simpler architectures. Clark and Strecker [1980], who were VAX architects, rebutted their proposal.

The simple load-store computers such as MIPS are commonly called RISC architectures. The roots of RISC architectures go back to computers like the 6600, where Thornton, Cray, and others recognized the importance of instruction set simplicity in building a fast computer. Cray continued his tradition of keeping computers simple in the CRAY-1. Commercial RISCs are built primarily on the work of three research projects: the Berkeley RISC processor, the IBM 801, and the Stanford MIPS processor. These architectures have attracted enormous industrial interest because of claims of a performance advantage of anywhere from two to five times over other computers using the same technology.

Begun in 1975, the IBM project was the first to start but was the last to become public. The IBM computer was designed as a 24-bit ECL minicomputer, while the university projects were both MOS-based, 32-bit microprocessors. John Cocke is considered the father of the 801 design. He received both the Eckert–Mauchly and Turing awards in recognition of his contribution. Radin [1982] described the highlights of the 801 architecture. The 801 was an experimental project that was never designed to be a product. In fact, to keep down costs and complexity, the computer was built with only 24-bit registers.

In 1980, Patterson and his colleagues at Berkeley began the project that was to give this architectural approach its name (see Patterson and Ditzel [1980]). They built two computers called RISC-I and RISC-II. Because the IBM project was not widely known or discussed, the role played by the Berkeley group in promoting the RISC approach was critical to acceptance of the technology. They also built one of

the first instruction caches to support hybrid-format RISCs (see Patterson et al. [1983]). It supported 16-bit and 32-bit instructions in memory but 32 bits in the cache. The Berkeley group went on to build RISC computers targeted toward Smalltalk, described by Ungar et al. [1984], and LISP, described by Taylor et al. [1986].

In 1981, Hennessy and his colleagues at Stanford published a description of the Stanford MIPS computer. Efficient pipelining and compiler-assisted scheduling of the pipeline were both important aspects of the original MIPS design. MIPS stood for Microprocessor without Interlocked Pipeline Stages, reflecting the lack of hardware to stall the pipeline, as the compiler would handle dependencies.

These early RISC computers—the 801, RISC-II, and MIPS—had much in common. Both university projects were interested in designing a simple computer that could be built in VLSI within the university environment. All three computers used a simple load-store architecture and fixed-format 32-bit instructions, and emphasized efficient pipelining. Patterson [1985] described the three computers and the basic design principles that have come to characterize what a RISC computer is, and Hennessy [1984] provided another view of the same ideas, as well as other issues in VLSI processor design.

In 1985, Hennessy published an explanation of the RISC performance advantage and traced its roots to a substantially lower CPI—under 2 for a RISC processor and over 10 for a VAX-11/780 (though not with identical workloads). A paper by Emer and Clark [1984] characterizing VAX-11/780 performance was instrumental in helping the RISC researchers understand the source of the performance advantage seen by their computers.

Since the university projects finished up, in the 1983–1984 time frame, the technology has been widely embraced by industry. Many manufacturers of the early computers (those made before 1986) claimed that their products were RISC computers. These claims, however, were often born more of marketing ambition than of engineering reality.

In 1986, the computer industry began to announce processors based on the technology explored by the three RISC research projects. Moussouris et al. [1986] described the MIPS R2000 integer processor, while Kane's book [1986] provides a complete description of the architecture. Hewlett-Packard converted their existing minicomputer line to RISC architectures; Lee [1989] described the HP Precision Architecture. IBM never directly turned the 801 into a product. Instead, the ideas were adopted for a new, low-end architecture that was incorporated in the IBM RT-PC and described in a collection of papers [Waters 1986]. In 1990, IBM announced a new RISC architecture (the RS 6000), which is the first superscalar RISC processor. In 1987, Sun Microsystems began delivering computers based on the SPARC architecture, a derivative of the Berkeley RISC-II processor; SPARC is described in Garner et al. [1988]. The PowerPC joined the forces of Apple, IBM, and Motorola. Appendix K summarizes several RISC architectures.

To help resolve the RISC versus traditional design debate, designers of VAX processors later performed a quantitative comparison of VAX and a RISC processor for implementations with comparable organizations. Their choices were the

VAX 8700 and the MIPS M2000. The differing goals for VAX and MIPS have led to very different architectures. The VAX goals, simple compilers and code density, led to powerful addressing modes, powerful instructions, efficient instruction encoding, and few registers. The MIPS goals were high performance via pipelining, ease of hardware implementation, and compatibility with highly optimizing compilers. These goals led to simple instructions, simple addressing modes, fixed-length instruction formats, and a large number of registers.

Figure M.1 shows the ratio of the number of instructions executed, the ratio of CPIs, and the ratio of performance measured in clock cycles. Since the organizations were similar, clock cycle times were assumed to be the same. MIPS executes about twice as many instructions as the VAX, while the CPI for the VAX is about six times larger than that for the MIPS. Hence, the MIPS M2000 has almost three times the performance of the VAX 8700. Furthermore, much less hardware is needed to build the MIPS processor than the VAX processor. This cost-performance gap is the reason why the company that used to make the VAX introduced a MIPS-based product and then has dropped the VAX completely and switched to Alpha, which is quite similar to MIPS. Bell and Strecker [1998] summarized the debate inside the company. Today, DEC, once the second largest computer company and the major success of the minicomputer industry, exists only as remnants within HP and Intel.

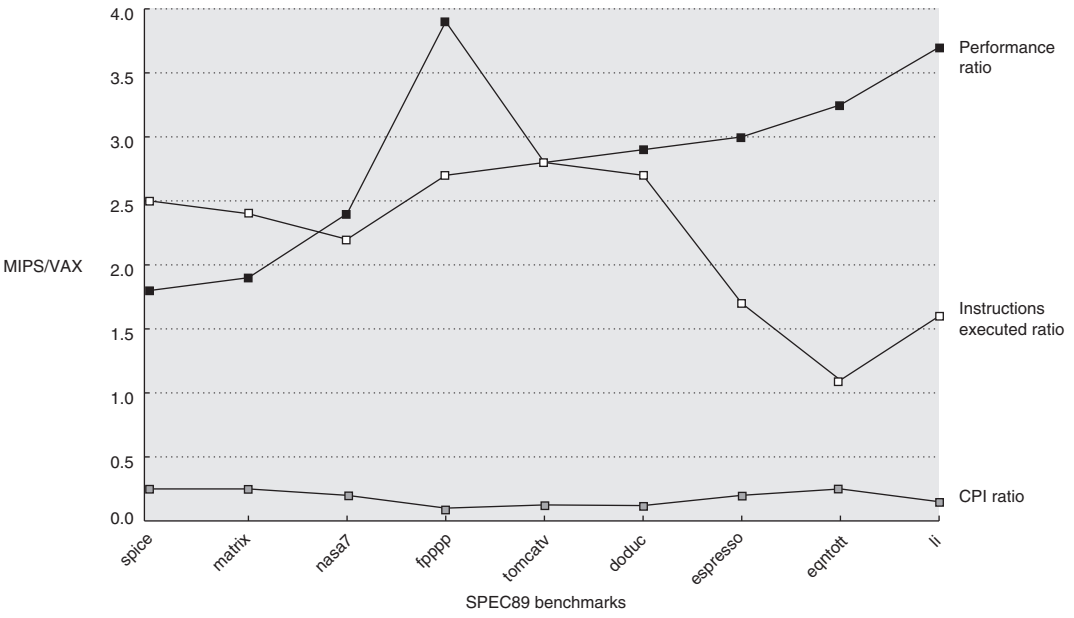


Figure M.1 Ratio of MIPS M2000 to VAX 8700 in instructions executed and performance in clock cycles using SPEC89 programs. On average, MIPS executes a little over twice as many instructions as the VAX, but the CPI for the VAX is almost six times the MIPS CPI, yielding almost a threefold performance advantage. (Based on data from Bhandarkar and Clark [1991].)

Looking back, only one Complex Instruction Set Computer (CISC) instruction set survived the RISC/CISC debate, and that one had binary compatibility with PC software. The volume of chips is so high in the PC industry that there is a sufficient revenue stream to pay the extra design costs—and sufficient resources due to Moore's law—to build microprocessors that translate from CISC to RISC internally. Whatever loss in efficiency occurred (due to longer pipeline stages and bigger die size to accommodate translation on the chip) was overcome by the enormous volume and the ability to dedicate IC processing lines specifically to this product.

Interestingly, Intel also concluded that the future of the 80x86 line was doubtful. They created the IA-64 architecture to support 64-bit addressing and to move to a RISC-style instruction set. The embodiment of the IA-64 (see Huck et al. [2000]) architecture in the Itanium-1 and Itanium-2 has been a mixed success. Although high performance has been achieved for floating-point applications, the integer performance was never impressive. In addition, the Itanium implementations have been large in transistor count and die size and power hungry. The complexity of the IA-64 instruction set, standing at least in partial conflict with the RISC philosophy, no doubt contributed to this area and power inefficiency.

AMD decided instead to just stretch the architecture from a 32-bit address to a 64-bit address, much as Intel had done when the 80386 stretched it from a 16-bit address to a 32-bit address. Intel later followed AMD's example. In the end, the tremendous marketplace advantage of the 80x86 presence was too much even for Intel, the owner of this legacy, to overcome!

References

Alexander, W. G., and D. B. Wortman [1975]. "Static and dynamic characteristics of XPL programs," *IEEE Computer* 8:11 (November), 41–46.

Amdahl, G. M., G. A. Blaauw, and F. P. Brooks, Jr. [1964]. "Architecture of the IBM System 360," *IBM J. Research and Development* 8:2 (April), 87–101.

Barton, R. S. [1961]. "A new approach to the functional design of a computer," *Proc. Western Joint Computer Conf.*, May 9–11, 1961, Los Angeles, Calif., 393–396.

Bell, G., R. Cady, H. McFarland, B. DeLagi, J. O'Laughlin, R. Noonan, and W. Wulf [1970]. "A new architecture for mini-computers: The DEC PDP-11," *Proc. AFIPS SJCC*, May 5–7, 1970, Atlantic City, N.J., 657–675.

Bell, G., and W. D. Strecker [1998]. "Computer structures: What have we learned from the PDP-11?" in G. S. Sohi, ed., *25 Years of the International Symposia on Computer Architecture (Selected Papers)*, ACM, New York, 138–151.

Bhandarkar, D. P. [1995]. *Alpha Architecture and Implementations*, Digital Press, Newton, Mass.

Bhandarkar, D., and D. W. Clark [1991]. "Performance from architecture: Comparing a RISC and a CISC with similar hardware organizations," *Proc. Fourth*

Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 8–11, 1991, Palo Alto, Calif., 310–319.

Bier, J. [1997]. "The evolution of DSP processors," paper presented at University of California, Berkeley, November 14.

Boddie, J. R. [2000]. "History of DSPs," *www.lucent.com/micro/dsp/dsphist.html*.

Case, R. P., and A. Padegs [1978]. "The architecture of the IBM System/370," *Communications of the ACM* 21:1, 73–96.

Chow, F. C. [1983]. "A Portable Machine-Independent Global Optimizer—Design and Measurements," Ph.D. thesis, Stanford University, Palo Alto, Calif.

Clark, D., and H. Levy [1982]. "Measurement and analysis of instruction set use in the VAX-11/780," *Proc. Ninth Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 26–29, 1982, Austin, Tex., 9–17.

Clark, D., and W. D. Strecker [1980]. "Comments on 'the case for the reduced instruction set computer,'" *Computer Architecture News* 8:6 (October), 34–38.

Crawford, J., and P. Gelsinger [1988]. *Programming the 80386*, Sybex Books, Alameda, Calif.

Darcy, J. D., and D. Gay [1996]. "FLECKmarks: Measuring floating point performance using a full IEEE compliant arithmetic benchmark," CS 252 class project, University of California, Berkeley (see *http://www.sonic.net/~jddarcy/Research/fleckmrk.pdf*).

Digital Semiconductor. [1996]. *Alpha Architecture Handbook, Version 3*, Digital Press, Maynard, Mass.

Ditzel, D. R., and D. A. Patterson [1980]. "Retrospective on high-level language computer architecture," *Proc. Seventh Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 6–8, 1980, La Baule, France, 97–104.

Emer, J. S., and D. W. Clark [1984]. "A characterization of processor performance in the VAX-11/780," *Proc. 11th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1984, Ann Arbor, Mich., 301–310.

Furber, S. B. [2000]. *ARM system-on-chip architecture*. Addison-Wesley, Boston, Mass.

Gagliardi, U. O. [1973]. "Report of workshop 4—software-related advances in computer hardware," *Proc. Symposium on the High Cost of Software*, September 17–19, 1973, Monterey, Calif., 99–120.

Game, M., and A. Booker [1999]. "CodePack code compression for PowerPC processors," *MicroNews*, 5:1.

Garner, R., A. Agarwal, F. Briggs, E. Brown, D. Hough, B. Joy, S. Kleiman, S. Muchnick, M. Namjoo, D. Patterson, J. Pendleton, and R. Tuck [1988]. "Scalable processor architecture (SPARC)," *Proc. IEEE COMPCON*, February 29–March 4, 1988, San Francisco, 278–283.

Hauck, E. A., and B. A. Dent [1968]. "Burroughs' B6500/B7500 stack mechanism," *Proc. AFIPS SJCC*, April 30–May 2, 1968, Atlantic City, N.J., 245–251.

Hennessy, J. [1984]. "VLSI processor architecture," *IEEE Trans. on Computers* C-33:11 (December), 1221–1246.

Hennessy, J. [1985]. "VLSI RISC processors," *VLSI Systems Design* 6:10 (October), 22–32.

Hennessy, J., N. Jouppi, F. Baskett, and J. Gill [1981]. "MIPS: A VLSI processor architecture," in *CMU Conference on VLSI Systems and Computations*, Computer Science Press, Rockville, Md.

Hewlett-Packard. [1994]. *PA-RISC 2.0 Architecture Reference Manual*, 3rd ed., Hewlett-Packard, Palo Alto, Calif.

Hitachi. [1997]. *SuperH RISC Engine SH7700 Series Programming Manual*, Hitachi, Santa Clara, Calif.

Huck, J. et al. [2000]. "Introducing the IA-64 Architecture" *IEEE Micro*, 20:5, (September–October), 12–23.

IBM. [1994]. *The PowerPC Architecture*, Morgan Kaufmann, San Francisco.

Intel. [2001]. "Using MMX instructions to convert RGB to YUV color conversion," *cedar.intel.com/cgi-bin/ids.dll/content/content.jsp?cntKey=Legacy::irtm_AP548_9996&cntType=IDS_EDITORIAL*.

Kahan, J. [1990]. "On the advantage of the 8087's stack," unpublished course notes, Computer Science Division, University of California, Berkeley.

Kane, G. [1986]. *MIPS R2000 RISC Architecture*, Prentice Hall, Englewood Cliffs, N.J.

Kane, G. [1996]. *PA-RISC 2.0 Architecture*, Prentice Hall, Upper Saddle River, N.J.

Kane, G., and J. Heinrich [1992]. *MIPS RISC Architecture*, Prentice Hall, Englewood Cliffs, N.J.

Kissell, K. D. [1997]. "MIPS16: High-density for the embedded market," *Proc. Real Time Systems '97*, June 15, 1997, Las Vegas, Nev.

Kozyrakis, C. [2000]. "Vector IRAM: A media-oriented vector processor with embedded DRAM," paper presented at Hot Chips 12, August 13–15, 2000, Palo Alto, Calif, 13–15.

Lee, R. [1989]. "Precision architecture," *Computer* 22:1 (January), 78–91.

Levy, H., and R. Eckhouse [1989]. *Computer Programming and Architecture: The VAX*, Digital Press, Boston.

Lindholm, T., and F. Yellin [1999]. *The Java Virtual Machine Specification*, 2nd ed., Addison-Wesley, Reading, Mass.

Lunde, A. [1977]. "Empirical evaluation of some features of instruction set processor architecture," *Communications of the ACM* 20:3 (March), 143–152.

Magenheimer, D. J., L. Peters, K. W. Pettis, and D. Zuras [1988]. "Integer multiplication and division on the HP precision architecture," *IEEE Trans. on Computers* 37:8, 980–990.

McGhan, H., and M. O'Connor [1998]. "PicoJava: A direct execution engine for Java bytecode," *Computer* 31:10 (October), 22–30.

McKeeman, W. M. [1967]. "Language directed computer design," *Proc. AFIPS Fall Joint Computer Conf.*, November 14–16, 1967, Washington, D.C., 413–417.

Meyers, G. J. [1978]. "The evaluation of expressions in a storage-to-storage architecture," *Computer Architecture News* 7:3 (October), 20–23.

Meyers, G. J. [1982]. *Advances in Computer Architecture*, 2nd ed., Wiley, New York.

MIPS. [1997]. *MIPS16 Application Specific Extension Product Description*.

Mitsubishi. [1996]. *Mitsubishi 32-Bit Single Chip Microcomputer M32R Family Software Manual*, Mitsubishi, Cypress, Calif.

Morse, S., B. Ravenal, S. Mazor, and W. Pohlman [1980]. "Intel microprocessors—8080 to 8086," *Computer* 13:10 (October).

Moussouris, J., L. Crudele, D. Freitas, C. Hansen, E. Hudson, S. Przybylski, T. Riordan, and C. Rowen [1986]. "A CMOS RISC processor with integrated system functions," *Proc. IEEE COMPCON*, March 3–6, 1986, San Francisco, 191.

Muchnick, S. S. [1988]. "Optimizing compilers for SPARC," *Sun Technology* 1:3 (Summer), 64–77.

Palmer, J., and S. Morse [1984]. *The 8087 Primer*, John Wiley & Sons, New York, 93.

Patterson, D. [1985]. "Reduced instruction set computers," *Communications of the ACM* 28:1 (January), 8–21.

Patterson, D. A., and D. R. Ditzel [1980]. "The case for the reduced instruction set computer," *Computer Architecture News* 8:6 (October), 25–33.

Patterson, D. A., P. Garrison, M. Hill, D. Lioupis, C. Nyberg, T. Sippel, and K. Van Dyke [1983]. "Architecture of a VLSI instruction cache for a RISC," *10th Annual Int'l. Conf. on Computer Architecture Conf. Proc.*, June 13–16, 1983, Stockholm, Sweden, 108–116.

Radin, G. [1982]. "The 801 minicomputer," *Proc. Symposium Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, March 1–3, 1982, Palo Alto, Calif., 39–47.

Riemens, A., K. A. Vissers, R. J. Schutten, F. W. Sijstermans, G. J. Hekstra, and G. D. La Hei [1999]. "Trimedia CPU64 application domain and benchmark suite," *Proc. IEEE Int'l. Conf. on Computer Design: VLSI in Computers and Processors (ICCD'99)*, October 10–13, 1999, Austin, Tex., 580–585.

Ropers, A., H. W. Lollman, and J. Wellhausen [1999]. *DSPstone: Texas Instruments TMS320C54x*, Tech. Rep. Nr. IB 315 1999/9-ISS-Version 0.9, Aachen University of Technology, Aaachen, Germany (*www.ert.rwth-aachen.de/Projekte/Tools/coal/dspstone_c54x/index.html*).

Shustek, L. J. [1978]. "Analysis and Performance of Computer Instruction Sets," Ph.D. dissertation, Stanford University, Palo Alto, Calif.

Silicon Graphics. [1996]. *MIPS V Instruction Set* (see *http://www.sgi.com/MIPS/arch/ISA5/#MIPSV_indx*).

Sites, R. L., and R. Witek, eds. [1995]. *Alpha Architecture Reference Manual*, 2nd ed., Digital Press, Newton, Mass.

Strauss, W. [1998]. "DSP Strategies 2002," *www.usadata.com/market_research/spr_05/spr_r127-005.htm*.

Strecker, W. D. [1978]. "VAX-11/780: A virtual address extension of the PDP-11 family," *Proc. AFIPS National Computer Conf.*, June 5–8, 1978, Anaheim, Calif., 47, 967–980.

Sun Microsystems. [1989]. *The SPARC Architectural Manual*, Version 8, Part No. 800-1399-09, Sun Microsystems, Santa Clara, Calif.

Tanenbaum, A. S. [1978]. "Implications of structured programming for machine architecture," *Communications of the ACM* 21:3 (March), 237–246.

Taylor, G., P. Hilfinger, J. Larus, D. Patterson, and B. Zorn [1986]. "Evaluation of the SPUR LISP architecture," *Proc. 13th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1986, Tokyo.

Texas Instruments [2000]. "History of innovation: 1980s," *www.ti.com/corp/docs/company/history/1980s.shtml*.

Thornton, J. E. [1964]. "Parallel operation in Control Data 6600," *Proc. AFIPS Fall Joint Computer Conf., Part II*, October 27–29, 1964, San Francisco, 26, 33–40.

Ungar, D., R. Blau, P. Foley, D. Samples, and D. Patterson [1984]. "Architecture of SOAR: Smalltalk on a RISC," *Proc. 11th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1984, Ann Arbor, Mich., 188–197.

van Eijndhoven, J. T. J., F. W. Sijstermans, K. A. Vissers, E. J. D. Pol, M. I. A. Tromp, P. Struik, R. H. J. Bloks, P. van der Wolf, A. D. Pimentel, and H. P. E. Vranken [1999]. "Trimedia CPU64 architecture," *Proc. IEEE Int'l. Conf. on Computer Design: VLSI in Computers and Processors (ICCD'99)*, October 10–13, 1999, Austin, Tex., 586–592.

Wakerly, J. [1989]. *Microcomputer Architecture and Programming*, Wiley, New York.

Waters, F. (ed.) [1986]. *IBM RT Personal Computer Technology*, SA 23-1057, IBM, Austin, Tex.

Weaver, D. L., and T. Germond [1994]. *The SPARC Architectural Manual*, Version 9,
Prentice Hall, Englewood Cliffs, N.J.

Weiss, S., and J. E. Smith [1994]. *Power and PowerPC*, Morgan Kaufmann, San Francisco.

Wiecek, C. [1982]. "A case study of the VAX 11 instruction set usage for compiler execution," *Proc. Symposium on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, March 1–3, 1982, Palo Alto, Calif., 177–184.

Wulf, W. [1981]. "Compilers and computer architecture," *Computer* 14:7 (July), 41–47.

M.5 The Development of Pipelining and Instruction-Level Parallelism (Chapter 3 and Appendices C and H)

Early Pipelined CPUs

The first general-purpose pipelined processor is considered to be Stretch, the IBM 7030. Stretch followed the IBM 704 and had a goal of being 100 times faster than the 704. The goal was a stretch from the state of the art at that time, hence the

nickname. The plan was to obtain a factor of 1.6 from overlapping fetch, decode, and execute, using a four-stage pipeline. Bloch [1959] and Bucholtz [1962] described the design and engineering trade-offs, including the use of ALU bypasses.

A series of general pipelining descriptions that appeared in the late 1970s and early 1980s provided most of the terminology and described most of the basic techniques used in simple pipelines. These surveys include Keller [1975], Ramamoorthy and Li [1977], and Chen [1980], as well as Kogge [1981], whose book is devoted entirely to pipelining. Davidson and his colleagues [1971, 1975] developed the concept of pipeline reservation tables as a design methodology for multicycle pipelines with feedback (also described in Kogge [1981]). Many designers use a variation of these concepts, in either designing pipelines or in creating software to schedule them.

The RISC processors were originally designed with ease of implementation and pipelining in mind. Several of the early RISC papers, published in the early 1980s, attempt to quantify the performance advantages of the simplification in instruction set. The best analysis, however, is a comparison of a VAX and a MIPS implementation published by Bhandarkar and Clark in 1991, 10 years after the first published RISC papers (see Figure M.1). After 10 years of arguments about the implementation benefits of RISC, this paper convinced even the most skeptical designers of the advantages of a RISC instruction set architecture.

J. E. Smith and his colleagues have written a number of papers examining instruction issue, exception handling, and pipeline depth for high-speed scalar CPUs. Kunkel and Smith [1986] evaluated the impact of pipeline overhead and dependences on the choice of optimal pipeline depth; they also provided an excellent discussion of latch design and its impact on pipelining. Smith and Pleszkun [1988] evaluated a variety of techniques for preserving precise exceptions. Weiss and Smith [1984] evaluated a variety of hardware pipeline scheduling and instruction issue techniques.

The MIPS R4000 was one of the first deeply pipelined microprocessors and is described by Killian [1991] and by Heinrich [1993]. The initial Alpha implementation (the 21064) has a similar instruction set and similar integer pipeline structure, with more pipelining in the floating-point unit.

The Introduction of Dynamic Scheduling

In 1964, CDC delivered the first CDC 6600. The CDC 6600 was unique in many ways. In addition to introducing scoreboarding, the CDC 6600 was the first processor to make extensive use of multiple functional units. It also had peripheral processors that used multithreading. The interaction between pipelining and instruction set design was understood, and a simple, load-store instruction set was used to promote pipelining. The CDC 6600 also used an advanced packaging technology. Thornton [1964] described the pipeline and I/O processor architecture, including the concept of out-of-order instruction execution. Thornton's book [1970] provides an excellent description of the entire processor, from technology

to architecture, and includes a foreword by Cray. (Unfortunately, this book is currently out of print.) The CDC 6600 also has an instruction scheduler for the FORTRAN compilers, described by Thorlin [1967].

The IBM 360 Model 91: A Landmark Computer

The IBM 360/91 introduced many new concepts, including tagging of data, register renaming, dynamic detection of memory hazards, and generalized forwarding. Tomasulo's algorithm is described in his 1967 paper. Anderson, Sparacio, and Tomasulo [1967] described other aspects of the processor, including the use of branch prediction. Many of the ideas in the 360/91 faded from use for nearly 25 years before being broadly resurrected in the 1990s. Unfortunately, the 360/91 was not successful, and only a handful were sold. The complexity of the design made it late to the market and allowed the Model 85, which was the first IBM processor with a cache, to outperform the 91.

Branch-Prediction Schemes

The 2-bit dynamic hardware branch-prediction scheme was described by J. E. Smith [1981]. Ditzel and McLellan [1987] described a novel branch-target buffer for CRISP, which implements branch folding. The correlating predictor we examine was described by Pan, So, and Rameh [1992]. Yeh and Patt [1992, 1993] generalized the correlation idea and described multilevel predictors that use branch histories for each branch, similar to the local history predictor used in the 21264. McFarling's tournament prediction scheme, which he refers to as a combined predictor, is described in his 1993 technical report. There are a variety of more recent papers on branch prediction based on variations in the multilevel and correlating predictor ideas. Kaeli and Emma [1991] described return address prediction, and Evers et al. [1998] provided an in-depth analysis of multilevel predictors. The data shown in Chapter 3 are from Skadron et al. [1999]. There are several schemes for prediction that may offer some additional benefit beyond tournament predictors. Eden and Mudge [1998] and Jimenez and Lin [2002] have described such approaches.

The Development of Multiple-Issue Processors

IBM did pioneering work on multiple issue. In the 1960s, a project called ACS was under way in California. It included multiple-issue concepts, a proposal for dynamic scheduling (although with a simpler mechanism than Tomasulo's scheme, which used backup registers), and fetching down both branch paths. The project originally started as a new architecture to follow Stretch and surpass the CDC 6600/6800. ACS started in New York but was moved to California, later changed to be S/360 compatible, and eventually canceled. John Cocke was one of the intellectual forces behind the team that included a number of IBM veterans and younger contributors,

many of whom went on to other important roles in IBM and elsewhere: Jack Bertram, Ed Sussenguth, Gene Amdahl, Herb Schorr, Fran Allen, Lynn Conway, and Phil Dauber, among others. While the compiler team published many of their ideas and had great influence outside IBM, the architecture ideas were not widely disseminated at that time. The most complete accessible documentation of this important project is at *www.cs.clemson.edu/~mark/acs.html*, which includes interviews with the ACS veterans and pointers to other sources. Sussenguth [1999] is a good overview of ACS.

Most of the early multiple-issue processors that actually reached the market followed an LIW or VLIW design approach. Charlesworth [1981] reported on the Floating Point Systems AP-120B, one of the first wide-instruction processors containing multiple operations per instruction. Floating Point Systems applied the concept of software pipelining both in a compiler and by handwriting assembly language libraries to use the processor efficiently. Because the processor was an attached processor, many of the difficulties of implementing multiple issue in general-purpose processors (for example, virtual memory and exception handling) could be ignored.

One of the interesting approaches used in early VLIW processors, such as the AP-120B and i860, was the idea of a pipeline organization that requires operations to be "pushed through" a functional unit and the results to be caught at the end of the pipeline. In such processors, operations advance only when another operation pushes them from behind (in sequence). Furthermore, an instruction specifies the destination for an instruction issued earlier that will be pushed out of the pipeline when this new operation is pushed in. Such an approach has the advantage that it does not specify a result destination when an operation first issues but only when the result register is actually written. This separation eliminates the need to detect write after write (WAW) and write after read (WAR) hazards in the hardware. The disadvantage is that it increases code size since no-ops may be needed to push results out when there is a dependence on an operation that is still in the pipeline and no other operations of that type are immediately needed. Instead of the "push-and-catch" approach used in these two processors, almost all designers have chosen to use *self-draining pipelines* that specify the destination in the issuing instruction and in which an issued instruction will complete without further action. The advantages in code density and simplifications in code generation seem to outweigh the advantages of the more unusual structure.

Several research projects introduced some form of multiple issue in the mid-1980s. For example, the Stanford MIPS processor had the ability to place two operations in a single instruction, although this capability was dropped in commercial variants of the architecture, primarily for performance reasons. Along with his colleagues at Yale, Fisher [1983] proposed creating a processor with a very wide instruction (512 bits) and named this type of processor a VLIW. Code was generated for the processor using trace scheduling, which Fisher [1981] had developed originally for generating horizontal microcode. The implementation of trace scheduling for the Yale processor is described by Fisher et al. [1984] and by Ellis [1986].

Although IBM canceled ACS, active research in the area continued in the 1980s. More than 10 years after ACS was canceled, John Cocke made a new proposal for a superscalar processor that dynamically made issue decisions; he and Tilak Agerwala described the key ideas in several talks in the mid-1980s and coined the term *superscalar*. He called the design America; it is described by Agerwala and Cocke [1987]. The IBM Power1 architecture (the RS/6000 line) is based on these ideas (see Bakoglu et al. [1989]).

J. E. Smith [1984] and his colleagues at Wisconsin proposed the decoupled approach that included multiple issue with limited dynamic pipeline scheduling. A key feature of this processor is the use of queues to maintain order among a class of instructions (such as memory references) while allowing it to slip behind or ahead of another class of instructions. The Astronautics ZS-1 described by Smith et al. [1987] embodies this approach with queues to connect the load-store unit and the operation units. The Power2 design uses queues in a similar fashion. J. E. Smith [1989] also described the advantages of dynamic scheduling and compared that approach to static scheduling.

The concept of speculation has its roots in the original 360/91, which performed a very limited form of speculation. The approach used in recent processors combines the dynamic scheduling techniques of the 360/91 with a buffer to allow in-order commit. Smith and Pleszkun [1988] explored the use of buffering to maintain precise interrupts and described the concept of a reorder buffer. Sohi [1990] described adding renaming and dynamic scheduling, making it possible to use the mechanism for speculation. Patt and his colleagues were early proponents of aggressive reordering and speculation. They focused on checkpoint and restart mechanisms and pioneered an approach called HPSm, which is also an extension of Tomasulo's algorithm [Hwu and Patt 1986].

The use of speculation as a technique in multiple-issue processors was evaluated by Smith, Johnson, and Horowitz [1989] using the reorder buffer technique; their goal was to study available ILP in nonscientific code using speculation and multiple issue. In a subsequent book, Johnson [1990] described the design of a speculative superscalar processor. Johnson later led the AMD K-5 design, one of the first speculative superscalars.

In parallel with the superscalar developments, commercial interest in VLIW approaches also increased. The Multiflow processor (see Colwell et al. [1987]) was based on the concepts developed at Yale, although many important refinements were made to increase the practicality of the approach. Among these was a control-lable store buffer that provided support for a form of speculation. Although more than 100 Multiflow processors were sold, a variety of problems, including the difficulties of introducing a new instruction set from a small company and competition from commercial RISC microprocessors that changed the economics in the mini-computer market, led to the failure of Multiflow as a company.

Around the same time as Multiflow, Cydrome was founded to build a VLIW-style processor (see Rau et al. [1989]), which was also unsuccessful commercially. Dehnert, Hsu, and Bratt [1989] explained the architecture and performance of the

Cydrome Cydra 5, a processor with a wide-instruction word that provides dynamic register renaming and additional support for software pipelining. The Cydra 5 is a unique blend of hardware and software, including conditional instructions and register rotation, aimed at extracting ILP. Cydrome relied on more hardware than the Multiflow processor and achieved competitive performance primarily on vector-style codes. In the end, Cydrome suffered from problems similar to those of Multiflow and was not a commercial success. Both Multiflow and Cydrome, although unsuccessful as commercial entities, produced a number of people with extensive experience in exploiting ILP as well as advanced compiler technology; many of those people have gone on to incorporate their experience and the pieces of the technology in newer processors. Fisher and Rau [1993] edited a comprehensive collection of papers covering the hardware and software of these two important processors.

Rau had also developed a scheduling technique called *polycyclic scheduling*, which is a basis for most software-pipelining schemes (see Rau, Glaeser, and Picard [1982]). Rau's work built on earlier work by Davidson and his colleagues on the design of optimal hardware schedulers for pipelined processors. Other historical LIW processors have included the Apollo DN 10000 and the Intel i860, both of which could dual-issue FP and integer operations.

Compiler Technology and Hardware Support for Scheduling

Loop-level parallelism and dependence analysis were developed primarily by D. Kuck and his colleagues at the University of Illinois in the 1970s. They also coined the commonly used terminology of *antidependence* and *output dependence* and developed several standard dependence tests, including the GCD and Banerjee tests. The latter test was named after Uptal Banerjee and comes in a variety of flavors. Recent work on dependence analysis has focused on using a variety of exact tests ending with a linear programming algorithm called Fourier–Motzkin. D. Maydan and W. Pugh both showed that the sequences of exact tests were a practical solution.

In the area of uncovering and scheduling ILP, much of the early work was connected to the development of VLIW processors, described earlier. Lam [1988] developed algorithms for software pipelining and evaluated their use on Warp, a wide-instruction-word processor designed for special-purpose applications. Weiss and Smith [1987] compared software pipelining versus loop unrolling as techniques for scheduling code on a pipelined processor. Rau [1994] developed modulo scheduling to deal with the issues of software-pipelining loops and simultaneously handling register allocation.

Support for speculative code scheduling was explored in a variety of contexts, including several processors that provided a mode in which exceptions were ignored, allowing more aggressive scheduling of loads (e.g., the MIPS TFP processor [Hsu 1994]). Several groups explored ideas for more aggressive hardware support for speculative code scheduling. For example, Smith, Horowitz, and Lam

[1992] created a concept called boosting that contains a hardware facility for supporting speculation but provides a checking and recovery mechanism, similar to those in IA-64 and Crusoe. The sentinel scheduling idea, which is also similar to the speculate-and-check approach used in both Crusoe and the IA-64 architectures, was developed jointly by researchers at the University of Illinois and HP Laboratories (see Mahlke et al. [1992]).

In the early 1990s, Wen-Mei Hwu and his colleagues at the University of Illinois developed a compiler framework, called IMPACT (see Chang et al. [1991]), for exploring the interaction between multiple-issue architectures and compiler technology. This project led to several important ideas, including superblock scheduling (see Hwu et al. [1993]), extensive use of profiling for guiding a variety of optimizations (e.g., procedure inlining), and the use of a special buffer (similar to the ALAT or program-controlled store buffer) for compile-aided memory conflict detection (see Gallagher et al. [1994]). They also explored the performance trade-offs between partial and full support for predication in Mahlke et al. [1995].

The early RISC processors all had delayed branches, a scheme inspired from microprogramming, and several studies on compile time branch prediction were inspired by delayed branch mechanisms. McFarling and Hennessy [1986] did a quantitative comparison of a variety of compile time and runtime branch-prediction schemes. Fisher and Freudenberger [1992] evaluated a range of compile time branch-prediction schemes using the metric of distance between mispredictions. Ball and Larus [1993] and Calder et al. [1997] described static prediction schemes using collected program behavior.

EPIC and the IA-64 Development

The roots of the EPIC approach lie in earlier attempts to build LIW and VLIW machines—especially those at Cydrome and Multiflow—and in a long history of compiler work that continued after these companies failed at HP, the University of Illinois, and elsewhere. Insights gained from that work led designers at HP to propose a VLIW-style, 64-bit architecture to follow the HP PA RISC architecture. Intel was looking for a new architecture to replace the x86 (now called IA-32) architecture and to provide 64-bit capability. In 1995, they formed a partnership to design a new architecture, IA-64 (see Huck et al. [2000]), and build processors based on it. Itanium (see Sharangpani and Arora [2000]) is the first such processor. In 2002, Intel introduced the second-generation IA-64 design, the Itanium 2 (see McNairy and Soltis [2003] and McCormick and Knies [2002]).

Studies of ILP and Ideas to Increase ILP

A series of early papers, including Tjaden and Flynn [1970] and Riseman and Foster [1972], concluded that only small amounts of parallelism could be available at the instruction level without investing an enormous amount of hardware. These papers dampened the appeal of multiple instruction issue for more than 10 years.

Nicolau and Fisher [1984] published a paper based on their work with trace scheduling and asserted the presence of large amounts of potential ILP in scientific programs.

Since then there have been many studies of the available ILP. Such studies have been criticized because they presume some level of both hardware support and compiler technology. Nonetheless, the studies are useful to set expectations as well as to understand the sources of the limitations. Wall has participated in several such studies, including Jouppi and Wall [1989] and Wall [1991, 1993]. Although the early studies were criticized as being conservative (e.g., they didn't include speculation), the last study is by far the most ambitious study of ILP to date and the basis for the data in Section 3.10. Sohi and Vajapeyam [1989] provided measurements of available parallelism for wide-instruction-word processors. Smith, Johnson, and Horowitz [1989] also used a speculative superscalar processor to study ILP limits. At the time of their study, they anticipated that the processor they specified was an upper bound on reasonable designs. Recent and upcoming processors, however, are likely to be at least as ambitious as their processor. Skadron et al. [1999] examined the performance trade-offs and limitations in a processor comparable to the most aggressive processors in 2005, concluding that the larger window sizes will not make sense without significant improvements on branch prediction for integer programs.

Lam and Wilson [1992] looked at the limitations imposed by speculation and showed that additional gains are possible by allowing processors to speculate in multiple directions, which requires more than one PC. (Such schemes cannot exceed what perfect speculation accomplishes, but they help close the gap between realistic prediction schemes and perfect prediction.) Wall's 1993 study includes a limited evaluation of this approach (up to eight branches are explored).

Going Beyond the Data Flow Limit

One other approach that has been explored in the literature is the use of value prediction. Value prediction can allow speculation based on data values. There have been a number of studies of the use of value prediction. Lipasti and Shen published two papers in 1996 evaluating the concept of value prediction and its potential impact on ILP exploitation. Calder, Reinman, and Tullsen [1999] explored the idea of selective value prediction. Sodani and Sohi [1997] approached the same problem from the viewpoint of reusing the values produced by instructions. Moshovos et al. [1997] showed that deciding when to speculate on values, by tracking whether such speculation has been accurate in the past, is important to achieving performance gains with value speculation. Moshovos and Sohi [1997] and Chrysos and Emer [1998] focused on predicting memory dependences and using this information to eliminate the dependence through memory. González and González [1998], Babbay and Mendelson [1998], and Calder, Reinman, and Tullsen [1999] are more recent studies of the use of value prediction. This area is currently highly active, with new results being published in every conference.

Recent Advanced Microprocessors

The years 1994 and 1995 saw the announcement of wide superscalar processors (three or more issues per clock) by every major processor vendor: Intel Pentium Pro and Pentium II (these processors share the same core pipeline architecture, described by Colwell and Steck [1995]); AMD K-5, K-6, and Athlon; Sun UltraSPARC (see Lauterbach and Horel [1999]); Alpha 21164 (see Edmondson et al. [1995]) and 21264 (see Kessler [1999]); MIPS R10000 and R12000 (see Yeager [1996]); PowerPC 603, 604, and 620 (see Diep, Nelson, and Shen [1995]); and HP 8000 (Kumar [1997]). The latter part of the decade (1996–2000) saw second generations of many of these processors (Pentium III, AMD Athlon, and Alpha 21264, among others). The second generation, although similar in issue rate, could sustain a lower CPI and provided much higher clock rates. All included dynamic scheduling, and they almost universally supported speculation. In practice, many factors, including the implementation technology, the memory hierarchy, the skill of the designers, and the type of applications benchmarked, all play a role in determining which approach is best.

The period from 2000 to 2005 was dominated by three trends among superscalar processors: the introduction of higher clock rates achieved through deeper pipelining (e.g., in the Pentium 4; see Hinton et al. [2001]), the introduction of multithreading by IBM in the Power 4 and by Intel in the Pentium 4 Extreme, and the beginning of the movement to multicore by IBM in the Power 4, AMD in Opteron (see Keltcher et al. [2003]), and most recently by Intel (see Douglas [2005]).

Multithreading and Simultaneous Multithreading

The concept of multithreading dates back to one of the earliest transistorized computers, the TX-2. TX-2 is also famous for being the computer on which Ivan Sutherland created Sketchpad, the first computer graphics system. TX-2 was built at MIT's Lincoln Laboratory and became operational in 1959. It used multiple threads to support fast context switching to handle I/O functions. Clark [1957] described the basic architecture, and Forgie [1957] described the I/O architecture. Multithreading was also used in the CDC 6600, where a fine-grained multithreading scheme with interleaved scheduling among threads was used as the architecture of the I/O processors. The HEP processor, a pipelined multiprocessor designed by Denelcor and shipped in 1982, used fine-grained multithreading to hide the pipeline latency as well as to hide the latency to a large memory shared among all the processors. Because the HEP had no cache, this hiding of memory latency was critical. Burton Smith, one of the primary architects, described the HEP architecture in a 1978 paper, and Jordan [1983] published a performance evaluation. The TERA processor extends the multithreading ideas and is described by Alverson et al. in a 1992 paper. The Niagara multithreading approach is similar to those of the HEP and TERA systems, although Niagara employs caches reducing the need for thread-based latency hiding.

In the late 1980s and early 1990s, researchers explored the concept of coarse-grained multithreading (also called *block multithreading*) as a way to tolerate

latency, especially in multiprocessor environments. The SPARCLE processor in the Alewife system used such a scheme, switching threads whenever a highlatency exceptional event, such as a long cache miss, occurred. Agarwal et al. described SPARCLE in a 1993 paper. The IBM Pulsar processor uses similar ideas.

By the early 1990s, several research groups had arrived at two key insights. First, they realized that fine-grained multithreading was needed to get the maximum performance benefit, since in a coarse-grained approach, the overhead of thread switching and thread start-up (e.g., filling the pipeline from the new thread) negated much of the performance advantage (see Laudon, Gupta, and Horowitz [1994]). Second, several groups realized that to effectively use large numbers of functional units would require both ILP and thread-level parallelism (TLP). These insights led to several architectures that used combinations of multithreading and multiple issue. Wolfe and Shen [1991] described an architecture called XIMD that statically interleaves threads scheduled for a VLIW processor. Hirata et al. [1992] described a proposed processor for media use that combines a static superscalar pipeline with support for multithreading; they reported speedups from combining both forms of parallelism. Keckler and Dally [1992] combined static scheduling of ILP and dynamic scheduling of threads for a processor with multiple functional units. The question of how to balance the allocation of functional units between ILP and TLP and how to schedule the two forms of parallelism remained open.

When it became clear in the mid-1990s that dynamically scheduled superscalars would be delivered shortly, several research groups proposed using the dynamic scheduling capability to mix instructions from several threads on the fly. Yamamoto et al. [1994] appear to have published the first such proposal, though the simulation results for their multithreaded superscalar architecture use simplistic assumptions. This work was quickly followed by Tullsen, Eggers, and Levy [1995], who provided the first realistic simulation assessment and coined the term *simultaneous multithreading*. Subsequent work by the same group together with industrial coauthors addressed many of the open questions about SMT. For example, Tullsen et al. [1996] addressed questions about the challenges of scheduling ILP versus TLP. Lo et al. [1997] provided an extensive discussion of the SMT concept and an evaluation of its performance potential, and Lo et. al. [1998] evaluated database performance on an SMT processor. Tuck and Tullsen [2003] reviewed the performance of SMT on the Pentium 4.

The IBM Power4 introduced multithreading (see Tendler et al. [2002]), while the Power5 used simultaneous multithreading. Mathis et al. [2005] explored the performance of SMT in the Power5, while Sinharoy et al. [2005] described the system architecture.

References

Agarwal, A., J. Kubiatowicz, D. Kranz, B.-H. Lim, D. Yeung, G. D'Souza, and M. Parkin [1993]. "Sparcle: An evolutionary processor design for large-scale multiprocessors," *IEEE Micro* 13 (June), 48–61.

Agerwala, T., and J. Cocke [1987]. *High Performance Reduced Instruction Set Processors*, Tech. Rep. RC12434, IBM Thomas Watson Research Center, Yorktown Heights, N.Y.

Alverson, G., R. Alverson, D. Callahan, B. Koblenz, A. Porterfield, and B. Smith [1992]. "Exploiting heterogeneous parallelism on a multithreaded multiprocessor," *Proc. ACM/IEEE Conf. on Supercomputing*, November 16–20, 1992, Minneapolis, Minn., 188–197.

Anderson, D. W., F. J. Sparacio, and R. M. Tomasulo [1967]. "The IBM 360 Model 91: Processor philosophy and instruction handling," *IBM J. Research and Development* 11:1 (January), 8–24.

Austin, T. M., and G. Sohi [1992]. "Dynamic dependency analysis of ordinary programs," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 342–351.

Babbay, F., and A. Mendelson [1998]. "Using value prediction to increase the power of speculative execution hardware," *ACM Trans. on Computer Systems* 16:3 (August), 234–270.

Bakoglu, H. B., G. F. Grohoski, L. E. Thatcher, J. A. Kaeli, C. R. Moore, D. P. Tattle, W. E. Male, W. R. Hardell, D. A. Hicks, M. Nguyen Phu, R. K. Montoye, W. T. Glover, and S. Dhawan [1989]. "IBM second-generation RISC processor organization," *Proc. IEEE Int'l. Conf. on Computer Design*, October, Rye Brook, N.Y., 138–142.

Ball, T., and J. Larus [1993]. "Branch prediction for free," *Proc. ACM SIGPLAN'93 Conference on Programming Language Design and Implementation (PLDI)*, June 23–25, 1993, Albuquerque, N.M., 300–313.

Bhandarkar, D., and D. W. Clark [1991]. "Performance from architecture: Comparing a RISC and a CISC with similar hardware organizations," *Proc. Fourth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 8–11, 1991, Palo Alto, Calif., 310–319.

Bhandarkar, D., and J. Ding [1997]. "Performance characterization of the Pentium Pro processor," *Proc. Third Int'l. Symposium on High Performance Computer Architecture*, February 1–5, 1997, San Antonio, Tex., 288–297.

Bloch, E. [1959]. "The engineering design of the Stretch computer," *Proc. Eastern Joint Computer Conf.*, December 1–3, 1959, Boston, Mass., 48–59.

Bucholtz, W. [1962]. *Planning a Computer System: Project Stretch*, McGraw-Hill, New York.

Calder, B., D. Grunwald, M. Jones, D. Lindsay, J. Martin, M. Mozer, and B. Zorn [1997]. "Evidence-based static branch prediction using machine learning," *ACM Trans. Program. Lang. Syst.* 19:1, 188–222.

Calder, B., G. Reinman, and D. M. Tullsen [1999]. "Selective value prediction," *Proc. 26th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 2–4, 1999, Atlanta, Ga.

Chang, P. P., S. A. Mahlke, W. Y. Chen, N. J. Warter, and W. W. Hwu [1991]. "IMPACT: An architectural framework for multiple-instruction-issue processors," *Proc. 18th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 27–30, 1991, Toronto, Canada, 266–275.

Charlesworth, A. E. [1981]. "An approach to scientific array processing: The architecture design of the AP-120B/FPS-164 family," *Computer* 14:9 (September), 18–27.

Chen, T. C. [1980]. "Overlap and parallel processing," in *Introduction to Computer Architecture*, H. Stone, ed., Science Research Associates, Chicago, 427–486.

Chrysos, G. Z., and J. S. Emer [1998]. "Memory dependence prediction using store sets," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 142–153.

Clark, D. W. [1987]. "Pipelining and performance in the VAX 8800 processor," *Proc. Second Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 5–8, 1987, Palo Alto, Calif., 173–177.

Clark, W. A. [1957]. "The Lincoln TX-2 computer development," *Proc. Western Joint Computer Conference*, February 26–28, 1957, Los Angeles, 143–145.

Colwell, R. P., and R. Steck [1995]. "A 0.6 μm BiCMOS processor with dynamic execution." *Proc. of IEEE Int'l. Symposium on Solid State Circuits (ISSCC)*, February 15–17, 1995, San Francisco, 176–177.

Colwell, R. P., R. P. Nix, J. J. O'Donnell, D. B. Papworth, and P. K. Rodman [1987]. "A VLIW architecture for a trace scheduling compiler," *Proc. Second Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 5–8, 1987, Palo Alto, Calif., 180–192.

Cvetanovic, Z., and R. E. Kessler [2000]. "Performance analysis of the Alpha 21264-based Compaq ES40 system," *27th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 10–14, 2000, Vancouver, Canada, 192–202.

Davidson, E. S. [1971]. "The design and control of pipelined function generators," *Proc. IEEE Conf. on Systems, Networks, and Computers*, January 19–21, 1971, Oaxtepec, Mexico, 19–21.

Davidson, E. S., A. T. Thomas, L. E. Shar, and J. H. Patel [1975]. "Effective control for pipelined processors," *Proc. IEEE COMPCON*, February 25–27, 1975, San Francisco, 181–184.

Dehnert, J. C., P. Y.-T. Hsu, and J. P. Bratt [1989]. "Overlapped loop support on the Cydra 5," *Proc. Third Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 3–6, 1989, Boston, Mass., 26–39.

Diep, T. A., C. Nelson, and J. P. Shen [1995]. "Performance evaluation of the PowerPC 620 microarchitecture," *Proc. 22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy.

Ditzel, D. R., and H. R. McLellan [1987]. "Branch folding in the CRISP microprocessor: Reducing the branch delay to zero," *Proc. 14th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1987, Pittsburgh, Penn., 2–7.

Douglas, J. [2005]. "Intel 8xx series and Paxville Xeon-MP Microprocessors," paper presented at Hot Chips 17, August 14–16, 2005, Stanford University, Palo Alto, Calif.

Eden, A., and T. Mudge [1998]. "The YAGS branch prediction scheme," *Proc. of the 31st Annual ACM/IEEE Int'l. Symposium on Microarchitecture*, November 30–December 2, 1998, Dallas, Tex., 69–80.

Edmondson, J. H., P. I. Rubinfield, R. Preston, and V. Rajagopalan [1995]. "Superscalar instruction execution in the 21164 Alpha microprocessor," *IEEE Micro* 15:2, 33–43.

Ellis, J. R. [1986]. *Bulldog: A Compiler for VLIW Architectures*, MIT Press, Cambridge, Mass.

Emer, J. S., and D. W. Clark [1984]. "A characterization of processor performance in the VAX-11/780," *Proc. 11th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1984, Ann Arbor, Mich., 301–310.

Evers, M., S. J. Patel, R. S. Chappell, and Y. N. Patt [1998]. "An analysis of correlation and predictability: What makes two-level branch predictors work," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 52–61.

Fisher, J. A. [1981]. "Trace scheduling: A technique for global microcode compaction," *IEEE Trans. on Computers* 30:7 (July), 478–490.

Fisher, J. A. [1983]. "Very long instruction word architectures and ELI-512," *10th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1982, Stockholm, Sweden, 140–150.

Fisher, J. A., and S. M. Freudenberger [1992]. "Predicting conditional branches from previous runs of a program," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston, 85–95.

Fisher, J. A., and B. R. Rau [1993]. *Journal of Supercomputing*, January (special issue).

Fisher, J. A., J. R. Ellis, J. C. Ruttenberg, and A. Nicolau [1984]. "Parallel processing: A smart compiler and a dumb processor," *Proc. SIGPLAN Conf. on Compiler Construction*, June 17–22, 1984, Montreal, Canada, 11–16.

Forgie, J. W. [1957]. "The Lincoln TX-2 input-output system," *Proc. Western Joint Computer Conference*, February 26–28, 1957, Los Angeles, 156–160.

Foster, C. C., and E. M. Riseman [1972]. "Percolation of code to enhance parallel dispatching and execution," *IEEE Trans. on Computers* C-21:12 (December), 1411–1415.

Gallagher, D. M., W. Y. Chen, S. A. Mahlke, J. C. Gyllenhaal, and W.W. Hwu [1994]. "Dynamic memory disambiguation using the memory conflict buffer," *Proc. Sixth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 4–7, Santa Jose, Calif., 183–193.

González, J., and A. González [1998]. "Limits of instruction level parallelism with data speculation," *Proc. Vector and Parallel Processing (VECPAR) Conf.*, June 21–23, 1998, Porto, Portugal, 585–598.

Heinrich, J. [1993]. *MIPS R4000 User's Manual*, Prentice Hall, Englewood Cliffs, N.J.

Hinton, G., D. Sager, M. Upton, D. Boggs, D. Carmean, A. Kyker, and P. Roussel [2001]. "The microarchitecture of the Pentium 4 processor," *Intel Technology Journal*, February.

Hirata, H., K. Kimura, S. Nagamine, Y. Mochizuki, A. Nishimura, Y. Nakase, and T. Nishizawa [1992]. "An elementary processor architecture with simultaneous instruction issuing from multiple threads," *Proc. 19th Annual Int'l. Symposium*

on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, 136–145.

Hopkins, M. [2000]. "A critical look at IA-64: Massive resources, massive ILP, but can it deliver?" *Microprocessor Report*, February.

Hsu, P. [1994]. "Designing the TFP microprocessor," *IEEE Micro* 18:2 (April), 2333.

Huck, J. et al. [2000]. "Introducing the IA-64 Architecture" *IEEE Micro*, 20:5 (September–October), 12–23.

Hwu, W.-M., and Y. Patt [1986]. "HPSm, a high performance restricted data flow architecture having minimum functionality," *13th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1986, Tokyo, 297–307.

Hwu, W. W., S. A. Mahlke, W. Y. Chen, P. P. Chang, N. J. Warter, R. A. Bringmann, R. O. Ouellette, R. E. Hank, T. Kiyohara, G. E. Haab, J. G. Holm, and D. M. Lavery [1993]. "The superblock: An effective technique for VLIW and superscalar compilation," *J. Supercomputing* 7:1, 2 (March), 229–248.

IBM. [1990]. "The IBM RISC System/6000 processor" (collection of papers), *IBM J. Research and Development* 34:1 (January).

Jimenez, D. A., and C. Lin [2002]. "Neural methods for dynamic branch prediction," *ACM Trans. Computer Sys* 20:4 (November), 369–397.

Johnson, M. [1990]. *Superscalar Microprocessor Design*, Prentice Hall, Englewood Cliffs, N.J.

Jordan, H. F. [1983]. "Performance measurements on HEP—a pipelined MIMD computer," *Proc. 10th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1982, Stockholm, Sweden, 207–212.

Jouppi, N. P., and D. W. Wall [1989]. "Available instruction-level parallelism for superscalar and superpipelined processors," *Proc. Third Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 3–6, 1989, Boston, 272–282.

Kaeli, D. R., and P. G. Emma [1991]. "Branch history table prediction of moving target branches due to subroutine returns," *Proc. 18th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 27–30, 1991, Toronto, Canada, 34–42.

Keckler, S. W., and W. J. Dally [1992]. "Processor coupling: Integrating compile time and runtime scheduling for parallelism," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 202–213.

Keller, R. M. [1975]. "Look-ahead processors," *ACM Computing Surveys* 7:4 (December), 177–195.

Keltcher, C. N., K. J. McGrath, A. Ahmed, and P. Conway [2003]. "The AMD Opteron processor for multiprocessor servers," *IEEE Micro* 23:2 (March–April), 66–76.

Kessler, R. [1999]. "The Alpha 21264 microprocessor," *IEEE Micro* 19:2 (March/April) 24–36.

Killian, E. [1991]. "MIPS R4000 technical overview–64 bits/100 MHz or bust," *Hot Chips III Symposium Record*, August 26–27, 1991, Stanford University, Palo Alto, Calif., 1.6–1.19.

Kogge, P. M. [1981]. *The Architecture of Pipelined Computers*, McGraw-Hill, New York.

Kumar, A. [1997]. "The HP PA-8000 RISC CPU," *IEEE Micro* 17:2 (March/April).

Kunkel, S. R., and J. E. Smith [1986]. "Optimal pipelining in supercomputers," *Proc. 13th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1986, Tokyo, 404–414.

Lam, M. [1988]. "Software pipelining: An effective scheduling technique for VLIW processors," *SIGPLAN Conf. on Programming Language Design and Implementation*, June 22–24, 1988, Atlanta, Ga., 318–328.

Lam, M. S., and R. P. Wilson [1992]. "Limits of control flow on parallelism," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 46–57.

Laudon, J., A. Gupta, and M. Horowitz [1994]. "Interleaving: A multithreading technique targeting multiprocessors and workstations," *Proc. Sixth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 4–7, San Jose, Calif., 308–318.

Lauterbach, G., and T. Horel [1999]. "UltraSPARC-III: Designing third generation 64-bit performance," *IEEE Micro* 19:3 (May/June).

Lipasti, M. H., and J. P. Shen [1996]. "Exceeding the dataflow limit via value prediction," *Proc. 29th Int'l. Symposium on Microarchitecture*, December 2–4, 1996, Paris, France.

Lipasti, M. H., C. B. Wilkerson, and J. P. Shen [1996]. "Value locality and load value prediction," *Proc. Seventh Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 1–5, 1996, Cambridge, Mass., 138–147.

Lo, J., L. Barroso, S. Eggers, K. Gharachorloo, H. Levy, and S. Parekh [1998]. "An analysis of database workload performance on simultaneous multithreaded processors," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 39–50.

Lo, J., S. Eggers, J. Emer, H. Levy, R. Stamm, and D. Tullsen [1997]. "Converting thread-level parallelism into instruction-level parallelism via simultaneous multithreading," *ACM Trans. on Computer Systems* 15:2 (August), 322–354.

Mahlke, S. A., W. Y. Chen, W.-M. Hwu, B. R. Rau, and M. S. Schlansker [1992]. "Sentinel scheduling for VLIW and superscalar processors," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston, 238–247.

Mahlke, S. A., R. E. Hank, J. E. McCormick, D. I. August, and W. W. Hwu [1995]. "A comparison of full and partial predicated execution support for ILP processors," *Proc. 22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy, 138–149.

Mathis, H. M., A. E. Mercias, J. D. McCalpin, R. J. Eickemeyer, and S. R. Kunkel [2005]. "Characterization of the multithreading (SMT) efficiency in Power5," *IBM J. of Research and Development*, 49:4/5 (July/September), 555–564.

McCormick, J., and A. Knies [2002]. "A brief analysis of the SPEC CPU2000 benchmarks on the Intel Itanium 2 processor," paper presented at Hot Chips 14, August 18–20, 2002, Stanford University, Palo Alto, Calif.

McFarling, S. [1993]. *Combining Branch Predictors*, WRL Technical Note TN-36, Digital Western Research Laboratory, Palo Alto, Calif.

McFarling, S., and J. Hennessy [1986]. "Reducing the cost of branches," *Proc. 13th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1986, Tokyo, 396–403.

McNairy, C., and D. Soltis [2003]. "Itanium 2 processor microarchitecture," *IEEE Micro* 23:2 (March–April), 44–55.

Moshovos, A., and G. S. Sohi [1997]. "Streamlining inter-operation memory communication via data dependence prediction," *Proc. 30th Annual Int'l. Symposium on Microarchitecture*, December 1–3, Research Triangle Park, N.C., 235–245.

Moshovos, A., S. Breach, T. N. Vijaykumar, and G. S. Sohi [1997]. "Dynamic speculation and synchronization of data dependences," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo.

Nicolau, A., and J. A. Fisher [1984]. "Measuring the parallelism available for very long instruction word architectures," *IEEE Trans. on Computers* C-33:11 (November), 968–976.

Pan, S.-T., K. So, and J. T. Rameh [1992]. "Improving the accuracy of dynamic branch prediction using branch correlation," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston, 76–84.

Postiff, M.A., D. A. Greene, G. S. Tyson, and T. N. Mudge [1999]. "The limits of instruction level parallelism in SPEC95 applications," *Computer Architecture News* 27:1 (March), 31–40.

Ramamoorthy, C. V., and H. F. Li [1977]. "Pipeline architecture," *ACM Computing Surveys* 9:1 (March), 61–102.

Rau, B. R. [1994]. "Iterative modulo scheduling: An algorithm for software pipelining loops," *Proc. 27th Annual Int'l. Symposium on Microarchitecture*, November 30–December 2, 1994, San Jose, Calif., 63–74.

Rau, B. R., C. D. Glaeser, and R. L. Picard [1982]. "Efficient code generation for horizontal architectures: Compiler techniques and architectural support," *Proc. Ninth Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 26–29, 1982, Austin, Tex., 131–139.

Rau, B. R., D. W. L. Yen, W. Yen, and R. A. Towle [1989]. "The Cydra 5 departmental supercomputer: Design philosophies, decisions, and trade-offs," *IEEE Computers* 22:1 (January), 12–34.

Riseman, E. M., and C. C. Foster [1972]. "Percolation of code to enhance paralleled dispatching and execution," *IEEE Trans. on Computers* C-21:12 (December), 1411–1415.

Rymarczyk, J. [1982]. "Coding guidelines for pipelined processors," *Proc. Symposium Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, March 1–3, 1982, Palo Alto, Calif., 12–19.

Sharangpani, H., and K. Arora [2000]. "Itanium Processor Microarchitecture," *IEEE Micro*, 20:5 (September–October), 24–43.

Sinharoy, B., R. N. Koala, J. M. Tendler, R. J. Eickemeyer, and J. B. Joyner [2005]. "POWER5 system microarchitecture," *IBM J. of Research and Development*, 49:4–5, 505–521.

Sites, R. [1979]. *Instruction Ordering for the CRAY-1 Computer*, Tech. Rep. 78-CS-023, Dept. of Computer Science, University of California, San Diego.

Skadron, K., P. S. Ahuja, M. Martonosi, and D. W. Clark [1999]. "Branch prediction, instruction-window size, and cache size: Performance tradeoffs and simulation techniques," *IEEE Trans. on Computers*, 48:11 (November).

Smith, A., and J. Lee [1984]. "Branch prediction strategies and branch-target buffer design," *Computer* 17:1 (January), 6–22.

Smith, B. J. [1978]. "A pipelined, shared resource MIMD computer," *Proc. Int'l. Conf. on Parallel Processing (ICPP)*, August, Bellaire, Mich., 6–8.

Smith, J. E. [1981]. "A study of branch prediction strategies," *Proc. Eighth Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 12–14, 1981, Minneapolis, Minn., 135–148.

Smith, J. E. [1984]. "Decoupled access/execute computer architectures," *ACM Trans. on Computer Systems* 2:4 (November), 289–308.

Smith, J. E. [1989]. "Dynamic instruction scheduling and the Astronautics ZS-1," *Computer* 22:7 (July), 21–35.

Smith, J. E., and A. R. Pleszkun [1988]. "Implementing precise interrupts in pipelined processors," *IEEE Trans. on Computers* 37:5 (May), 562–573. (This paper is based on an earlier paper that appeared in *Proc. 12th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 17–19, 1985, Boston, Mass.)

Smith, J. E., G. E. Dermer, B. D. Vanderwarn, S. D. Klinger, C. M. Rozewski, D. L. Fowler, K. R. Scidmore, and J. P. Laudon [1987]. "The ZS-1 central processor," *Proc. Second Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 5–8, 1987, Palo Alto, Calif., 199–204.

Smith, M. D., M. Horowitz, and M. S. Lam [1992]. "Efficient superscalar performance through boosting," *Proc. Fifth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 12–15, 1992, Boston, 248–259.

Smith, M. D., M. Johnson, and M. A. Horowitz [1989]. "Limits on multiple instruction issue," *Proc. Third Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 3–6, 1989, Boston, 290–302.

Sodani, A., and G. Sohi [1997]. "Dynamic instruction reuse," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo.

Sohi, G. S. [1990]. "Instruction issue logic for high-performance, interruptible, multiple functional unit, pipelined computers," *IEEE Trans. on Computers* 39:3 (March), 349–359.

Sohi, G. S., and S. Vajapeyam [1989]. "Tradeoffs in instruction format design for horizontal architectures," *Proc. Third Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 3–6, 1989, Boston, 15–25.

Sussenguth, E. [1999]. "IBM's ACS-1 Machine," *IEEE Computer* 22:11 (November).

Tendler, J. M., J. S. Dodson, J. S. Fields, Jr., H. Le, and B. Sinharoy [2002]. "Power4 system microarchitecture," *IBM J. of Research and Development*, 46:1, 5–26.

Thorlin, J. F. [1967]. "Code generation for PIE (parallel instruction execution) computers," *Proc. Spring Joint Computer Conf.*, April 18–20, 1967, Atlantic City, N.J., 27.

Thornton, J. E. [1964]. "Parallel operation in the Control Data 6600," *Proc. AFIPS Fall Joint Computer Conf., Part II*, October 27–29, 1964, San Francisco, 26, 33–40.

Thornton, J. E. [1970]. *Design of a Computer, the Control Data 6600*, Scott, Foresman, Glenview, Ill.

Tjaden, G. S., and M. J. Flynn [1970]. "Detection and parallel execution of independent instructions," *IEEE Trans. on Computers* C-19:10 (October), 889–895.

Tomasulo, R. M. [1967]. "An efficient algorithm for exploiting multiple arithmetic units," *IBM J. Research and Development* 11:1 (January), 25–33.

Tuck, N., and D. Tullsen [2003]. "Initial observations of the simultaneous multithreading Pentium 4 processor," *Proc. 12th Int. Conf. on Parallel Architectures and Compilation Techniques (PACT'03)*, September 27–October 1, New Orleans, La., 26–34.

Tullsen, D. M., S. J. Eggers, and H. M. Levy [1995]. "Simultaneous multithreading: Maximizing on-chip parallelism," *Proc. 22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy, 392–403.

Tullsen, D. M., S. J. Eggers, J. S. Emer, H. M. Levy, J. L. Lo, and R. L. Stamm [1996]. "Exploiting choice: Instruction fetch and issue on an implementable simultaneous multithreading processor," *Proc. 23rd Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 22–24, 1996, Philadelphia, Penn., 191–202.

Wall, D. W. [1991]. "Limits of instruction-level parallelism," *Proc. Fourth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 8–11, 1991, Palo Alto, Calif., 248–259.

Wall, D. W. [1993]. *Limits of Instruction-Level Parallelism*, Research Rep. 93/6, Western Research Laboratory, Digital Equipment Corp., Palo Alto, Calif.

Weiss, S., and J. E. Smith [1984]. "Instruction issue logic for pipelined supercomputers," *Proc. 11th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1984, Ann Arbor, Mich., 110–118.

Weiss, S., and J. E. Smith [1987]. "A study of scalar compilation techniques for pipelined supercomputers," *Proc. Second Int'l. Conf. on Architectural Support*

for Programming Languages and Operating Systems (ASPLOS), October 5–8, 1987, Palo Alto, Calif., 105–109.

Wilson, R. P., and M. S. Lam [1995]. "Efficient context-sensitive pointer analysis for C programs," *Proc. ACM SIGPLAN'95 Conf. on Programming Language Design and Implementation*, June 18–21, 1995, La Jolla, Calif., 1–12.

Wolfe, A., and J. P. Shen [1991]. "A variable instruction stream extension to the VLIW architecture," *Proc. Fourth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 8–11, 1991, Palo Alto, Calif., 2–14.

Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky [1994]. "Performance estimation of multistreamed, superscalar processors," *Proc. 27th Annual Hawaii Int'l. Conf. on System Sciences*, January 4–7, 1994, Maui, 195–204.

Yeager, K. [1996]. "The MIPS R10000 superscalar microprocessor," *IEEE Micro* 16:2 (April), 28–40.

Yeh, T., and Y. N. Patt [1992]. "Alternative implementations of two-level adaptive branch prediction," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 124–134.

Yeh, T., and Y. N. Patt [1993]. "A comparison of dynamic branch predictors that use two levels of branch history," *Proc. 20th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 16–19, 1993, San Diego, Calif., 257–266.

M.6 The Development of SIMD Supercomputers, Vector Computers, Multimedia SIMD Instruction Extensions, and Graphical Processor Units (Chapter 4)

In this historical section, we start with perhaps the most infamous supercomputer, the Illiac IV, as a representative of the early SIMD (Single Instruction, Multiple Data) architectures and then move to perhaps the most famous supercomputer, the Cray-1, as a representative of vector architectures. The next step is Multimedia SIMD Extensions, which got its name in part due to an advertising campaign involving the "Bunny People," a disco-dancing set of workers in cleansuits on a semiconductor fabrication line. We conclude with the history of GPUs, which is not quite as colorful.

SIMD Supercomputers

The cost of a general multiprocessor is, however, very high and further design options were considered which would decrease the cost without seriously degrading the power or efficiency of the system. The options consist of recentralizing one of the three major components. … Centralizing the [control unit] gives rise to the basic organization of [an] … array processor such as the Illiac IV.

Bouknight et al. [1972]

… with Iliac IV, programming the machine was very difficult and the architecture probably was not very well suited to some of the applications we were trying to run. The key idea was that I did not think we had a very good match in Iliac IV between applications and architecture.

David Kuck
Software designer for the Illiac IV and early pioneer in parallel software
David Kuck
An oral history conducted in 1991 by Andrew Goldstein, IEEE History Center, New Brunswick, N.J.

The SIMD model was one of the earliest models of parallel computing, dating back to the first large-scale multiprocessor, the Illiac IV. Rather than pipelining the data computation as in vector architectures, these machines had an array of functional units; hence, they might be considered array processors.

The earliest ideas on SIMD-style computers are from Unger [1958] and Slotnick, Borck, and McReynolds [1962]. Slotnick's Solomon design formed the basis of the Illiac IV, perhaps the most infamous of the supercomputer projects. Although successful in pushing several technologies that proved useful in later projects, it failed as a computer. Costs escalated from the $8 million estimate in 1966 to $31 million by 1972, despite construction of only a quarter of the planned multiprocessor. (In 2011 dollars, that was an increase from $54M to $152M.) Actual performance was at best 15 MFLOPS versus initial predictions of 1000 MFLOPS for the full system [Hord 1982]. Delivered to NASA Ames Research in 1972, the computer required three more years of engineering before it was usable. These events slowed investigation of SIMD, but Danny Hillis [1985] resuscitated this style in the Connection Machine, which had 65,536 1-bit processors.

The basic trade-off in SIMD multiprocessors is performance of a processor versus number of processors. SIMD supercomputers of the 1980s emphasized a large degree of parallelism over performance of the individual processors. The Connection Multiprocessor 2, for example, offered 65,536 single-bit-wide processors, while the Illiac IV planned for 64 64-bit processors. Massively parallel SIMD multiprocessors relied on interconnection or communication networks to exchange data between processing elements.

After being resurrected in the 1980s, first by Thinking Machines and then by MasPar, the SIMD model faded away as supercomputers for two main reasons. First, it is too inflexible. A number of important problems were not data parallel, and the architecture did not scale down in a competitive fashion; that is, small-scale SIMD multiprocessors often have worse cost-performance compared with that of the alternatives. Second, SIMD could not take advantage of the tremendous performance and cost advantages of SISD (Single Instruction, Single Data) microprocessor technology of the 1980s, which was doubling in performance every 18 months. Instead of leveraging this low-cost technology, designers of SIMD multiprocessors had to build custom processors for their multiprocessors.

Vector Computers

I'm certainly not inventing vector processors. There are three kinds that I know of existing today. They are represented by the Illiac-IV, the (CDC) Star processor, and the TI (ASC) processor. Those three were all pioneering processors. … One of the problems of being a pioneer is you always make mistakes and I never, never want to be a pioneer. It's always best to come second when you can look at the mistakes the pioneers made.

Seymour Cray
Public lecture at Lawrence Livermore Laboratories on the introduction of the Cray-1 (1976)

The first vector processors were the Control Data Corporation (CDC) STAR-100 (see Hintz and Tate [1972]) and the Texas Instruments ASC (see Watson [1972]), both announced in 1972. Both were memory-memory vector processors. They had relatively slow scalar units—the STAR used the same units for scalars and vectors—making the scalar pipeline extremely deep. Both processors had high start-up overhead and worked on vectors of several hundred to several thousand elements. The crossover between scalar and vector could be over 50 elements. It appears that not enough attention was paid to the role of Amdahl's law on these two processors.

Seymour Cray, who worked on the 6600 and the 7600 at CDC, founded Cray Research and introduced the Cray-1 in 1976 (see Russell [1978]). The Cray-1 used a vector-register architecture to lower start-up overhead significantly and to reduce memory bandwidth requirements. He also had efficient support for non-unit stride and invented chaining. Most importantly, the Cray-1 was the fastest scalar processor in the world at that time. This matching of good scalar and vector performance was probably the most significant factor in making the Cray-1 a success. Some customers bought the processor primarily for its outstanding scalar performance. Many subsequent vector processors are based on the architecture of this first commercially successful vector processor. Baskett and Keller [1977] provided a good evaluation of the Cray-1.

In 1981, CDC started shipping the CYBER 205 (see Lincoln [1982]). The 205 had the same basic architecture as the STAR but offered improved performance all around as well as expandability of the vector unit with up to four lanes, each with multiple functional units and a wide load-store pipe that provided multiple words per clock. The peak performance of the CYBER 205 greatly exceeded the performance of the Cray-1; however, on real programs, the performance difference was much smaller.

In 1983, Cray Research shipped the first Cray X-MP (see Chen [1983]). With an improved clock rate (9.5 ns versus 12.5 ns on the Cray-1), better chaining support (allowing vector operations with RAW dependencies to operate in parallel), and multiple memory pipelines, this processor maintained the Cray Research lead in supercomputers. The Cray-2, a completely new design configurable with up to four processors, was introduced later. A major feature of the Cray-2 was the use

of DRAM, which made it possible to have very large memories at the time. The first Cray-2, with its 256M word (64-bit words) memory, contained more memory than the total of all the Cray machines shipped to that point! The Cray-2 had a much faster clock than the X-MP, but also much deeper pipelines; however, it lacked chaining, had enormous memory latency, and had only one memory pipe per processor. In general, the Cray-2 was only faster than the Cray X-MP on problems that required its very large main memory.

That same year, processor vendors from Japan entered the supercomputer marketplace. First were the Fujitsu VP100 and VP200 (see Miura and Uchida [1983]), and later came the Hitachi S810 and the NEC SX/2 (see Watanabe [1987]). These processors proved to be close to the Cray X-MP in performance. In general, these three processors had much higher peak performance than the Cray X-MP. However, because of large start-up overhead, their typical performance was often lower than that of the Cray X-MP. The Cray X-MP favored a multiple-processor approach, first offering a two-processor version and later a four-processor version. In contrast, the three Japanese processors had expandable vector capabilities.

In 1988, Cray Research introduced the Cray Y-MP—a bigger and faster version of the X-MP. The Y-MP allowed up to eight processors and lowered the cycle time to 6 ns. With a full complement of eight processors, the Y-MP was generally the fastest supercomputer, though the single-processor Japanese supercomputers could be faster than a one-processor Y-MP. In late 1989, Cray Research was split into two companies, both aimed at building high-end processors available in the early 1990s. Seymour Cray headed the spin-off, Cray Computer Corporation, until its demise in 1995. Their initial processor, the Cray-3, was to be implemented in gallium arsenide, but they were unable to develop a reliable and cost-effective implementation technology. Shortly before his tragic death in a car accident in 1996, Seymour Cray started yet another company to develop high-performance systems but this time using commodity components.

Cray Research focused on the C90, a new high-end processor with up to 16 processors and a clock rate of 240 MHz. This processor was delivered in 1991. In 1993, Cray Research introduced their first highly parallel processor, the T3D, employing up to 2048 Digital Alpha21064 microprocessors. In 1995, they announced the availability of both a new low-end vector machine, the J90, and a high-end machine, the T90. The T90 was much like the C90, but with a clock that was twice as fast (460 MHz), using three-dimensional packaging and optical clock distribution.

In 1995, Cray Research was acquired by Silicon Graphics. In 1998, it released the SV1 system, which grafted considerably faster CMOS processors onto the J90 memory system. It also added a data cache for vectors to each CPU to help meet the increased memory bandwidth demands. Silicon Graphics sold Cray Research to Tera Computer in 2000, and the joint company was renamed Cray Inc.

The Japanese supercomputer makers continued to evolve their designs. In 2001, the NEC SX/5 was generally held to be the fastest available vector supercomputer, with 16 lanes clocking at 312 MHz and with up to 16 processors sharing the same memory. The NEC SX/6, released in 2001, was the first commercial single-chip vector microprocessor, integrating an out-of-order quad-issue

superscalar processor, scalar instruction and data caches, and an eight-lane vector unit on a single die [Kitagawa et al. 2003]. The Earth Simulator is constructed from 640 nodes connected with a full crossbar, where each node comprises eight SX-6 vector microprocessors sharing a local memory. The SX-8, released in 2004, reduces the number of lanes to four but increases the vector clock rate to 2 GHz. The scalar unit runs at a slower 1 GHz clock rate, a common pattern in vector machines where the lack of hazards simplifies the use of deeper pipelines in the vector unit.

In 2002, Cray Inc. released the X1 based on a completely new vector ISA. The X1 SSP processor chip integrates an out-of-order superscalar with scalar caches running at 400 MHz and a two-lane vector unit running at 800 MHz. When four SSP chips are ganged together to form an MSP, the resulting peak vector performance of 12.8 GFLOPS is competitive with the contemporary NEC SX machines. The X1E enhancement, delivered in 2004, raises the clock rates to 565 and 1130 MHz, respectively. Many of the ideas were borrowed from the Cray T3E design, which is a MIMD (Multiple Instruction, Multiple Data) computer that uses off-the-shelf microprocessors. X1 has a new instruction set with a larger number of registers and with memory distributed locally with the processor in shared address space. The out-of-order scalar unit and vector units are decoupled, so that the scalar unit can get ahead of the vector unit. Vectors become shorter when the data are blocked to utilize the MSP caches, which is not a good match to an eight-lane vector unit. To handle these shorter vectors, each processor with just two vector lanes can work on a different loop.

The Cray X2 was announced in 2007, and it may prove to be the last Cray vector architecture to be built, as it's difficult to justify the investment in new silicon given the size of the market. The processor has a 1.3 GHz clock rate and 8 vector lanes for a processor peak performance of 42 GFLOP/sec for single precision. It includes both L1 and L2 caches. Each node is a 4-way SMP with up to 128 GBytes of DRAM, and the maximum size is 8K nodes.

The NEC SX-9 has up to 16 processors per node, with each processor having 8 lanes and running at 3.2 GHz. It was announced in 2008. The peak double precision vector performance is 102 GFLOP/sec. The 16 processor SMP can have 1024 GBytes of DRAM. The maximum size is 512 nodes.

The basis for modern vectorizing compiler technology and the notion of data dependence was developed by Kuck and his colleagues [1974] at the University of Illinois. Padua and Wolfe [1986] gave a good overview of vectorizing compiler technology.

Multimedia SIMD Instruction Extensions

What could a computer hardware company … possibly have in common with disco dancing. A lot, if one goes by an advertisement campaign released by the world's largest microprocessor company … Intel, in 1997.

IBS Center for Management Research
"Dancing Its Way Towards Leadership," 2002

Going through the history books, the 1957 TX-2 had partitioned ALUs to support media of the time, but these ideas faded away to be rediscovered 30 years later in the personal computer era. Since every desktop microprocessor by definition has its own graphical displays, as transistor budgets increased it was inevitable that support would be added for graphics operations. Many graphics systems use 8 bits to represent each of the 3 primary colors plus 8 bits for a transparency of a pixel. The addition of speakers and microphones for teleconferencing and video games suggested support of sound as well. Audio samples need more than 8 bits of precision, but 16 bits are sufficient.

Every microprocessor has special support so that bytes and half words take up less space when stored in memory, but due to the infrequency of arithmetic operations on these data sizes in typical integer programs, there is little support beyond data transfers. The Intel i860 was justified as a graphical accelerator within the company. Its architects recognized that many graphics and audio applications would perform the same operation on vectors of these data [Atkins 1991; Kohn 1989]. Although a vector unit was beyond the transistor budget of the i860 in 1989, by partitioning the carry chains within a 64-bit ALU, it could perform simultaneous operations on short vectors of eight 8-bit operands, four 16-bit operands, or two 32-bit operands. The cost of such partitioned ALUs was small. Applications that lend themselves to such support include MPEG (video), video games (3D graphics), digital photography, and teleconferencing (audio and image processing).

Like a virus, over time such multimedia support has spread to nearly every desktop microprocessor. HP was the first successful desktop RISC to include such support, but soon every other manufacturer had their own take on the idea in the 1990s.

These extensions were originally called *subword parallelism* or *vector*. Since Intel marketing used SIMD to describe the MMX extension of the 80x86 announced in 1996, that became the popular name, due in part to a successful television advertising campaign involving disco dancers wearing clothing modeled after the cleansuits worn in semiconductor fabrication lines.

Graphical Processor Units

It's been almost three years since GPU computing broke into the mainstream of HPC with the introduction of NVIDIA's CUDA API in September 2007. Adoption of the technology since then has proceeded at a surprisingly strong and steady pace. Many organizations that began with small pilot projects a year or two ago have moved on to enterprise deployment, and GPU accelerated machines are now represented on the TOP500 list starting at position two. The relatively rapid adoption of CUDA by a community not known for the rapid adoption of much of anything is a noteworthy signal. Contrary to the accepted wisdom that GPU computing is more difficult, I believe its success thus far signals that it is no more complicated than good CPU programming. Further, it more clearly and succinctly expresses the parallelism of a large class of problems leading to code

that is easier to maintain, more scalable and better positioned to map to future many-core architectures.

Vincent Natol
"Kudos for CUDA," HPCwire (2010)

3D graphics pipeline hardware evolved from the large expensive systems of the early 1980s to small workstations and then to PC accelerators in the mid- to late 1990s. During this period, three major transitions occurred:

- Performance-leading graphics subsystems declined in price from $50,000 to $200.
- Performance increased from 50 million pixels per second to 1 billion pixels per second and from 100,000 vertices per second to 10 million vertices per second.
- Native hardware capabilities evolved from wireframe (polygon outlines) to flat-shaded (constant color) filled polygons, to smooth-shaded (interpolated color) filled polygons, to full-scene anti-aliasing with texture mapping and rudimentary multitexturing.

Scalable GPUs

Scalability has been an attractive feature of graphics systems from the beginning. Workstation graphics systems gave customers a choice in pixel horse-power by varying the number of pixel processor circuit boards installed. Prior to the mid-1990s PC graphics scaling was almost nonexistent. There was one option—the VGA controller. As 3D-capable accelerators appeared, the market had room for a range of offerings. 3dfx introduced multiboard scaling with the original SLI (Scan Line Interleave) on their Voodoo2, which held the performance crown for its time (1998). Also in 1998, NVIDIA introduced distinct products as variants on a single architecture with Riva TNT Ultra (high-performance) and Vanta (low-cost), first by speed binning and packaging, then with separate chip designs (GeForce 2 GTS and GeForce 2 MX). At present, for a given architecture generation, four or five separate GPU chip designs are needed to cover the range of desktop PC performance and price points. In addition, there are separate segments in notebook and workstation systems. After acquiring 3dfx, NVIDIA continued the multi-GPU SLI concept in 2004, starting with GeForce 6800—providing multi-GPU scalability transparently to the programmer and to the user. Functional behavior is identical across the scaling range; one application will run unchanged on any implementation of an architectural family.

Graphics Pipelines

Early graphics hardware was configurable, but not programmable by the application developer. With each generation, incremental improvements were offered; however, developers were growing more sophisticated and asking for more new features than could be reasonably offered as built-in fixed functions. The NVIDIA

GeForce 3, described by Lindholm et al. [2001], took the first step toward true general shader programmability. It exposed to the application developer what had been the private internal instruction set of the floating-point vertex engine. This coincided with the release of Microsoft's DirectX 8 and OpenGL's vertex shader extensions. Later GPUs, at the time of DirectX 9, extended general programmability and floating-point capability to the pixel fragment stage and made texture available at the vertex stage. The ATI Radeon 9700, introduced in 2002, featured a programmable 24-bit floating-point pixel fragment processor programmed with DirectX 9 and OpenGL. The GeForce FX added 32-bit floating-point pixel processors. This was part of a general trend toward unifying the functionality of the different stages, at least as far as the application programmer was concerned. NVIDIA's GeForce 6800 and 7800 series were built with separate processor designs and separate hardware dedicated to the vertex and to the fragment processing. The XBox 360 introduced an early unified processor GPU in 2005, allowing vertex and pixel shaders to execute on the same processor.

GPGPU: An Intermediate Step

As DirectX 9-capable GPUs became available, some researchers took notice of the raw performance growth path of GPUs and began to explore the use of GPUs to solve complex parallel problems. DirectX 9 GPUs had been designed only to match the features required by the graphics API. To access the computational resources, a programmer had to cast their problem into native graphics operations. For example, to run many simultaneous instances of a pixel shader, a triangle had to be issued to the GPU (with clipping to a rectangle shape if that was what was desired). Shaders did not have the means to perform arbitrary scatter operations to memory. The only way to write a result to memory was to emit it as a pixel color value and configure the framebuffer operation stage to write (or blend, if desired) the result to a two-dimensional framebuffer. Furthermore, the only way to get a result from one pass of computation to the next was to write all parallel results to a pixel framebuffer, then use that framebuffer as a texture map as input to the pixel fragment shader of the next stage of the computation. Mapping general computations to a GPU in this era was quite awkward. Nevertheless, intrepid researchers demonstrated a handful of useful applications with painstaking efforts. This field was called "GPGPU" for general-purpose computing on GPUs.

GPU Computing

While developing the Tesla architecture for the GeForce 8800, NVIDIA realized its potential usefulness would be much greater if programmers could think of the GPU as a processor. NVIDIA selected a programming approach in which programmers would explicitly declare the data-parallel aspects of their workload.

For the DirectX 10 generation, NVIDIA had already begun work on a higheff-ficiency floating-point and integer processor that could run a variety of

simultaneous workloads to support the logical graphics pipeline. This processor was designed to take advantage of the common case of groups of threads executing the same code path. NVIDIA added memory load and store instructions with integer byte addressing to support the requirements of compiled C programs. It introduced the thread block (cooperative thread array), grid of thread blocks, and barrier synchronization to dispatch and manage highly parallel computing work. Atomic memory operations were added. NVIDIA developed the CUDA C/C++ compiler, libraries, and runtime software to enable programmers to readily access the new data-parallel computation model and develop applications.

To create a vendor-neutral GPU programming language, a large number of companies are creating compilers for the OpenCL language, which has many of the features of CUDA but which runs on many more platforms. In 2011, the performance is much higher if you write CUDA code for GPUs than if you write OpenCL code.

AMD's acquisition of ATI, the second leading GPU vendor, suggests a spread of GPU computing. The AMD Fusion architecture, announced just as this edition was being finished, is an initial merger between traditional GPUs and traditional CPUs. NVIDIA also announced Project Denver, which combines an ARM scalar processor with NVIDIA GPUs in a single address space. When these systems are shipped, it will be interesting to learn just how tightly integrated they are and the impact of integration on performance and energy of both data parallel and graphics applications.

References

SIMD Supercomputers

Bouknight, W. J., S. A. Deneberg, D. E. McIntyre, J. M. Randall, A. H. Sameh, and D. L. Slotnick [1972]. "The Illiac IV system," *Proc. IEEE* 60:4, 369–379. Also appears in D. P. Siewiorek, C. G. Bell, and A. Newell, *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 1982, 306–316.

Hillis, W. D. [1985]. *The Connection Multiprocessor*, MIT Press, Cambridge, Mass.

Hord, R. M. [1982]. *The Illiac-IV, The First Supercomputer*, Computer Science Press, Rockville, Md.

Slotnick, D. L., W. C. Borck, and R. C. McReynolds [1962]. "The Solomon computer," *Proc. AFIPS Fall Joint Computer Conf.*, December 4–6, 1962, Philadelphia, Penn., 97–107.

Unger, S. H. [1958]. "A computer oriented towards spatial problems," *Proc. Institute of Radio Engineers* 46:10 (October), 1744–1750.

Vector Architecture

Asanovic, K. [1998]. "Vector Microprocessors," Ph.D. thesis, Computer Science Division, University of California, Berkeley.

Baskett, F., and T. W. Keller [1977]. "An Evaluation of the Cray-1 Processor," in *High Speed Computer and Algorithm Organization*, D. J. Kuck, D. H. Lawrie, and A. H. Sameh, eds., Academic Press, San Diego, Calif., 71–84.

Chen, S. [1983]. "Large-scale and high-speed multiprocessor system for scientific applications," *Proc. NATO Advanced Research Workshop on High Speed Computing*, June 20–22, Julich, West Germany. Also in K. Hwang, ed., "Superprocessors: Design and applications," *IEEE*, August, 59–73, 1984.

Flynn, M. J. [1966]. "Very high-speed computing systems," *Proc. IEEE* 54:12 (December), 1901–1909.

Gebis, J. and Patterson, D. [2007]. "Embracing and extending 20th-century instruction set architectures," *IEEE Computer*, 40:4 (April), 68–75.

Hintz, R. G., and D. P. Tate [1972]. "Control data STAR-100 processor design," *Proc. IEEE COMPCON*, September 12–14, 1972, San Francisco, 1–4.

Kitagawa, K., S. Tagaya, Y. Hagihara, and Y. Kanoh [2003]. "A hardware overview of SX-6 and SX-7 supercomputer," *NEC Research and Development Journal* 44:1 (January), 2–7.

Kozyrakis, C., and D. Patterson [2002]. "Vector vs. superscalar and VLIW architectures for embedded multimedia benchmarks," *Proc. 35th Annual Intl. Symposium on Microarchitecture (MICRO)*, November 18–22, 2002, Istanbul, Turkey.

Kuck, D., P. P. Budnik, S.-C. Chen, D. H. Lawrie, R. A. Towle, R. E. Strebendt, E. W. Davis, Jr., J. Han, P. W. Kraska, and Y. Muraoka [1974]. "Measurements of parallelism in ordinary Fortran programs," *Computer* 7:1 (January), 37–46.

Lincoln, N. R. [1982]. "Technology and design trade offs in the creation of a modern supercomputer," *IEEE Trans. on Computers* C-31:5 (May), 363–376.

Miura, K., and K. Uchida [1983]. "FACOM vector processing system: VP100/200," *Proc. NATO Advanced Research Workshop on High Speed Computing*, June 20–22, Julich, West Germany. Also in K. Hwang, ed., "Superprocessors: Design and applications," *IEEE*, August, 59–73, 1984.

Padua, D., and M. Wolfe [1986]. "Advanced compiler optimizations for supercomputers," *Communications of the ACM* 29:12 (December), 1184–1201.

Russell, R. M. [1978]. "The Cray-1 processor system," *Communications of the ACM* 21:1 (January), 63–72.

Vajapeyam, S. [1991]. "Instruction-Level Characterization of the Cray Y-MP Processor," Ph.D. thesis, Computer Sciences Department, University of Wisconsin–Madison.

Watanabe, T. [1987]. "Architecture and performance of the NEC supercomputer SX system," *Parallel Computing* 5, 247–255.

Watson, W. J. [1972]. "The TI ASC—a highly modular and flexible super processor architecture," *Proc. AFIPS Fall Joint Computer Conf.*, December 5–7, 1972, Anaheim, Calif., 221–228.

Multimedia SIMD

Atkins, M. [1991]. "Performance and the i860 Microprocessor," *IEEE Micro*, 11:5 (September), 24–27, 72–78.

Kohn, L., and N. Margulis [1989]. "Introducing the Intel i860 64-Bit Microprocessor," *IEEE Micro*, 9:4 (July), 15–30.

GPU

Akeley, K., and T. Jermoluk [1988]. "High-performance polygon rendering," *Proc. SIGGRAPH 88*, August 1–5, 1988, Atlanta, Ga., 239–46.

Hillis, W. D., and G. L. Steele [1986]. "Data parallel algorithms," *Communications of the ACM* 29:12 (December), 1170–1183 (*http://doi.acm.org/10.1145/7902.7903*).

IEEE 754-2008 Working Group. [2006]. *DRAFT Standard for Floating-Point Arithmetic*, 754-2008 (https://doi.org/10.1109/IEEESTD.2008.4610935).

Lee, W. V., et al. [2010]. "Debunking the 100X GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU," *Proc. ISCA '10*, June 19–23, 2010, Saint-Malo, France.

Lindholm, E., M. J. Kligard, and H. Moreton [2001]. A user-programmable vertex engine. In *SIGGRAPH '01*: *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, 149–158.

Moore, G. E. [1965]. "Cramming more components onto integrated circuits," *Electronics* 38:8 (April 19), 114–117.

Williams, S., A. Waterman, and D. Patterson [2009]. "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, 52:4 (April), 65–76.

M.7 The History of Multiprocessors and Parallel Processing (Chapter 5 and Appendices F, G, and I)

There is a tremendous amount of history in multiprocessors; in this section, we divide our discussion by both time period and architecture. We start with the SIMD approach and the Illiac IV. We then turn to a short discussion of some other early experimental multiprocessors and progress to a discussion of some of the great debates in parallel processing. Next we discuss the historical roots of the present multiprocessors and conclude by discussing recent advances.

SIMD Computers: Attractive Idea, Many Attempts, No Lasting Successes

The cost of a general multiprocessor is, however, very high and further design options were considered which would decrease the cost without seriously degrading the power or efficiency of the system. The options consist of recentralizing one of the three major components. … Centralizing the [control unit] gives rise to the basic organization of [an] … array processor such as the Illiac IV.

Bouknight et al. [1972]

The SIMD model was one of the earliest models of parallel computing, dating back to the first large-scale multiprocessor, the Illiac IV. The key idea in that multiprocessor, as in more recent SIMD multiprocessors, is to have a single instruction that operates on many data items at once, using many functional units.

The earliest ideas on SIMD-style computers are from Unger [1958] and Slotnick, Borck, and McReynolds [1962]. Slotnick's Solomon design formed the basis of the Illiac IV, perhaps the most infamous of the supercomputer projects. Although successful in pushing several technologies that proved useful in later projects, it failed as a computer. Costs escalated from the $8 million estimate in 1966 to $31 million by 1972, despite construction of only a quarter of the planned multiprocessor. Actual performance was at best 15 MFLOPS versus initial predictions of 1000 MFLOPS for the full system [Hord 1982]. Delivered to NASA Ames Research in 1972, the computer took three more years of engineering before it was usable. These events slowed investigation of SIMD, but Danny Hillis [1985] resuscitated this style in the Connection Machine, which had 65,636 1-bit processors.

Real SIMD computers need to have a mixture of SISD and SIMD instructions. There is an SISD host computer to perform operations such as branches and address calculations that do not need parallel operation. The SIMD instructions are broadcast to all the execution units, each of which has its own set of registers. For flexibility, individual execution units can be disabled during an SIMD instruction. In addition, massively parallel SIMD multiprocessors rely on interconnection or communication networks to exchange data between processing elements.

SIMD works best in dealing with arrays in for loops; hence, to have the opportunity for massive parallelism in SIMD there must be massive amounts of data, or *data parallelism*. SIMD is at its weakest in case statements, where each execution unit must perform a different operation on its data, depending on what data it has. The execution units with the wrong data are disabled so that the proper units can continue. Such situations essentially run at 1/*n*th performance, where *n* is the number of cases.

The basic trade-off in SIMD multiprocessors is performance of a processor versus number of processors. Recent multiprocessors emphasize a large degree of parallelism over performance of the individual processors. The Connection Multiprocessor 2, for example, offered 65,536 single-bit-wide processors, while the Illiac IV had 64 64-bit processors.

After being resurrected in the 1980s, first by Thinking Machines and then by MasPar, the SIMD model has once again been put to bed as a general-purpose multiprocessor architecture, for two main reasons. First, it is too inflexible. A number of important problems cannot use such a style of multiprocessor, and the architecture does not scale down in a competitive fashion; that is, small-scale SIMD multiprocessors often have worse cost-performance compared with that of the alternatives. Second, SIMD cannot take advantage of the tremendous performance and cost advantages of microprocessor technology. Instead of leveraging this low-cost technology, designers of SIMD multiprocessors must build custom processors for their multiprocessors.

Although SIMD computers have departed from the scene as general-purpose alternatives, this style of architecture will continue to have a role in special-purpose

designs. Many special-purpose tasks are highly data parallel and require a limited set of functional units. Thus, designers can build in support for certain operations, as well as hardwired interconnection paths among functional units. Such organizations are often called *array processors*, and they are useful for such tasks as image and signal processing.

Other Early Experiments

It is difficult to distinguish the first MIMD multiprocessor. Surprisingly, the first computer from the Eckert-Mauchly Corporation, for example, had duplicate units to improve availability. Holland [1959] gave early arguments for multiple processors. Two of the best-documented multiprocessor projects were undertaken in the 1970s at Carnegie Mellon University. The first of these was C.mmp [Wulf and Bell 1972; Wulf and Harbison 1978], which consisted of 16 PDP-11s connected by a crossbar switch to 16 memory units. It was among the first multiprocessors with more than a few processors, and it had a shared-memory programming model. Much of the focus of the research in the C.mmp project was on software, especially in the OS area. A later multiprocessor, Cm* [Swan et al. 1977], was a cluster-based multiprocessor with a distributed memory and a nonuniform access time. The absence of caches and a long remote access latency made data placement critical. This multiprocessor and a number of application experiments are well described by Gehringer, Siewiorek, and Segall [1987]. Many of the ideas in these multiprocessors would be reused in the 1980s when the microprocessor made it much cheaper to build multiprocessors.

Great Debates in Parallel Processing

The turning away from the conventional organization came in the middle 1960s, when the law of diminishing returns began to take effect in the effort to increase the operational speed of a computer. … Electronic circuits are ultimately limited in their speed of operation by the speed of light … and many of the circuits were already operating in the nanosecond range.

Bouknight et al. [1972]

… sequential computers are approaching a fundamental physical limit on their potential computational power. Such a limit is the speed of light …

Angel L. DeCegama
The Technology of Parallel Processing, Vol. I (1989)

… today's multiprocessors … are nearing an impasse as technologies approach the speed of light. Even if the components of a sequential processor could be made to work this fast, the best that could be expected is no more than a few million instructions per second.

David Mitchell
The Transputer: The Time Is Now (1989)

The quotes above give the classic arguments for abandoning the current form of computing, and Amdahl [1967] gave the classic reply in support of continued focus on the IBM 360 architecture. Arguments for the advantages of parallel execution can be traced back to the 19th century [Menabrea 1842]! Yet, the effectiveness of the multiprocessor for reducing latency of individual important programs is still being explored. Aside from these debates about the advantages and limitations of parallelism, several hot debates have focused on how to build multiprocessors.

It's hard to predict the future, yet in 1989 Gordon Bell made two predictions for 1995. We included these predictions in the first edition of the book, when the outcome was completely unclear. We discuss them in this section, together with an assessment of the accuracy of the prediction.

The first was that a computer capable of sustaining a teraFLOPS—one million MFLOPS—would be constructed by 1995, using either a multicomputer with 4K to 32K nodes or a Connection Multiprocessor with several million processing elements [Bell 1989]. To put this prediction in perspective, each year the Gordon Bell Prize acknowledges advances in parallelism, including the fastest real program (highest MFLOPS). In 1989, the winner used an eight-processor Cray Y-MP to run at 1680 MFLOPS. On the basis of these numbers, multiprocessors and programs would have to have improved by a factor of 3.6 each year for the fastest program to achieve 1 TFLOPS in 1995. In 1999, the first Gordon Bell prize winner crossed the 1 TFLOPS bar. Using a 5832-processor IBM RS/6000 SST system designed specially for Livermore Laboratories, they achieved 1.18 TFLOPS on a shock-wave simulation. This ratio represents a year-to-year improvement of 1.93, which is still quite impressive.

What has become recognized since the 1990s is that, although we may have the technology to build a TFLOPS multiprocessor, it is not clear that the machine is cost effective, except perhaps for a few very specialized and critically important applications related to national security. We estimated in 1990 that to achieve 1 TFLOPS would require a machine with about 5000 processors and would cost about $100 million. The 5832-processor IBM system at Livermore cost $110 million. As might be expected, improvements in the performance of individual microprocessors both in cost and performance directly affect the cost and performance of large-scale multiprocessors, but a 5000-processor system will cost more than 5000 times the price of a desktop system using the same processor. Since that time, much faster multiprocessors have been built, but the major improvements have increasingly come from the processors in the past five years, rather than fundamental breakthroughs in parallel architecture.

The second Bell prediction concerned the number of data streams in supercomputers shipped in 1995. Danny Hillis believed that, although supercomputers with a small number of data streams may be the best sellers, the biggest multiprocessors would be multiprocessors with many data streams, and these would perform the bulk of the computations. Bell bet Hillis that in the last quarter of calendar year 1995 more sustained MFLOPS would be shipped in multiprocessors using few data streams ($\leq$100) rather than many data streams ($\geq$1000). This bet concerned

only supercomputers, defined as multiprocessors costing more than $1 million and used for scientific applications. Sustained MFLOPS was defined for this bet as the number of floating-point operations per *month*, so availability of multiprocessors affects their rating.

In 1989, when this bet was made, it was totally unclear who would win. In 1995, a survey of the current publicly known supercomputers showed only six multiprocessors in existence in the world with more than 1000 data streams, so Bell's prediction was a clear winner. In fact, in 1995, much smaller microprocessor-based multiprocessors ($\leq$20 processors) were becoming dominant. In 1995, a survey of the 500 highest-performance multiprocessors in use (based on Linpack ratings), called the TOP500, showed that the largest number of multiprocessors were bus-based shared-memory multiprocessors! By 2005, various clusters or multicomputers played a large role. For example, in the top 25 systems, 11 were custom clusters, such as the IBM Blue Gene system or the Cray XT3; 10 were clusters of shared-memory multiprocessors (both using distributed and centralized memory); and the remaining 4 were clusters built using PCs with an off-the-shelf interconnect.

More Recent Advances and Developments

With the primary exception of the parallel vector multiprocessors (see Appendix G) and more recently of the IBM Blue Gene design, all other recent MIMD computers have been built from off-the-shelf microprocessors using a bus and logically central memory or an interconnection network and a distributed memory. A number of experimental multiprocessors built in the 1980s further refined and enhanced the concepts that form the basis for many of today's multiprocessors.

The Development of Bus-Based Coherent Multiprocessors

Although very large mainframes were built with multiple processors in the 1960s and 1970s, multiprocessors did not become highly successful until the 1980s. Bell [1985] suggested that the key was that the smaller size of the microprocessor allowed the memory bus to replace the interconnection network hardware and that portable operating systems meant that multiprocessor projects no longer required the invention of a new operating system. In his paper, Bell defined the terms *multiprocessor* and *multicomputer* and set the stage for two different approaches to building larger scale multiprocessors.

The first bus-based multiprocessor with snooping caches was the Synapse N + 1 described by Frank [1984]. Goodman [1983] wrote one of the first papers to describe snooping caches. The late 1980s saw the introduction of many commercial bus-based, snooping cache architectures, including the Silicon Graphics 4D/240 [Baskett, Jermoluk, and Solomon 1988], the Encore Multimax [Wilson 1987], and the Sequent Symmetry [Lovett and Thakkar 1988]. The mid-1980s

Name	Protocol type	Memory write policy	Unique feature	Multiprocessors using
Write Once	Write invalidate	Write-back after first write	First snooping protocol described in literature	
Synapse N + 1	Write invalidate	Write-back	Explicit state where memory is the owner	Synapse multiprocessors; first cache-coherent multiprocessors available
Berkeley (MOESI)	Write invalidate	Write-back	Owned shared state	Berkeley SPUR multiprocessor; Sun Enterprise servers
Illinois (MESI)	Write invalidate	Write-back	Clean private state; can supply data from any cache with a clean copy	SGI Power and Challenge series
"Firefly"	Write broadcast	Write-back when private, write through when shared	Memory updated on broadcast	No current multiprocessors; SPARCCenter 2000 closest

Figure M.2 Five snooping protocols summarized. Archibald and Baer [1986] use these names to describe the five protocols, and Eggers [1989] summarizes the similarities and differences as shown in this figure. The Firefly protocol was named for the experimental DEC Firefly multiprocessor, in which it appeared. The alternative names for protocols are based on the states they support: M = Modified, E = Exclusive (private clean), S = Shared, I = Invalid, O = Owner (shared dirty).

saw an explosion in the development of alternative coherence protocols, and Archibald and Baer [1986] provided a good survey and analysis, as well as references to the original papers. Figure M.2 summarizes several snooping cache coherence protocols and shows some multiprocessors that have used or are using that protocol.

The early 1990s saw the beginning of an expansion of such systems with the use of very wide, high-speed buses (the SGI Challenge system used a 256-bit, packet-oriented bus supporting up to 8 processor boards and 32 processors) and later the use of multiple buses and crossbar interconnects—for example, in the Sun SPARCCenter and Enterprise systems (Charlesworth [1998] discussed the interconnect architecture of these multiprocessors). In 2001, the Sun Enterprise servers represented the primary example of large-scale (>16 processors), symmetric multiprocessors in active use. Today, most bus-based machines offer only four or so processors and switches, or alternative designs are used for eight or more.

Toward Large-Scale Multiprocessors

In the effort to build large-scale multiprocessors, two different directions were explored: message-passing multicomputers and scalable shared-memory multiprocessors. Although there had been many attempts to build mesh and hypercube-connected multiprocessors, one of the first multiprocessors to successfully bring together all the pieces was the Cosmic Cube built at Caltech [Seitz 1985]. It introduced important advances in routing and interconnect technology and substantially

reduced the cost of the interconnect, which helped make the multicomputer viable. The Intel iPSC 860, a hypercube-connected collection of i860s, was based on these ideas. More recent multiprocessors, such as the Intel Paragon, have used networks with lower dimensionality and higher individual links. The Paragon also employed a separate i860 as a communications controller in each node, although a number of users have found it better to use both i860 processors for computation as well as communication. The Thinking Multiprocessors CM-5 made use of off-the-shelf microprocessors and a fat tree interconnect (see Appendix F). It provided user-level access to the communication channel, thus significantly improving communication latency. In 1995, these two multiprocessors represented the state of the art in message-passing multicomputers.

Early attempts at building a scalable shared-memory multiprocessor include the IBM RP3 [Pfister et al. 1985], the NYU Ultracomputer [Elder et al. 1985; Schwartz 1980], the University of Illinois Cedar project [Gajksi et al. 1983], and the BBN Butterfly and Monarch [BBN Laboratories 1986; Rettberg et al. 1990]. These multiprocessors all provided variations on a nonuniform distributed-memory model and, hence, are distributed shared-memory (DSM) multiprocessors, but they did not support cache coherence, which substantially complicated programming. The RP3 and Ultracomputer projects both explored new ideas in synchronization (fetch-and-operate) as well as the idea of combining references in the network. In all four multiprocessors, the interconnect networks turned out to be more costly than the processing nodes, raising problems for smaller versions of the multiprocessor. The Cray T3D/E (see Arpaci et al. [1995] for an evaluation of the T3D and Scott [1996] for a description of the T3E enhancements) builds on these ideas, using a noncoherent shared address space but building on the advances in interconnect technology developed in the multicomputer domain (see Scott and Thorson [1996]).

Extending the shared-memory model with scalable cache coherence was done by combining a number of ideas. Directory-based techniques for cache coherence were actually known before snooping cache techniques. In fact, the first cache coherence protocols actually used directories, as described by Tang [1976] and implemented in the IBM 3081. Censier and Feautrier [1978] described a directory coherence scheme with tags in memory. The idea of distributing directories with the memories to obtain a scalable implementation of cache coherence was first described by Agarwal et al. [1988] and served as the basis for the Stanford DASH multiprocessor (see Lenoski et al. [1990, 1992]), which was the first operational cache-coherent DSM multiprocessor. DASH was a "plump" node cc-NUMA machine that used four-processor SMPs as its nodes, interconnecting them in a style similar to that of Wildfire but using a more scalable two-dimensional grid rather than a crossbar for the interconnect.

The Kendall Square Research KSR-1 [Burkhardt et al. 1992] was the first commercial implementation of scalable coherent shared memory. It extended the basic DSM approach to implement a concept called *cache-only memory architecture* (COMA), which makes the main memory a cache. In the KSR-1, memory blocks could be replicated in the main memories of each node with hardware support to

handle the additional coherence requirements for these replicated blocks. (The KSR-1 was not strictly a pure COMA because it did not migrate the home location of a data item but always kept a copy at home. Essentially, it implemented only replication.) Many other research proposals [Falsafi and Wood 1997; Hagersten, Landin, and Haridi 1992; Saulsbury et al. 1995; Stenström, Joe, and Gupta 1992] for COMA-style architectures and similar approaches that reduce the burden of nonuniform memory access through migration [Chandra et al. 1994; Soundararajan et al. 1998] were developed, but there have been no further commercial implementations.

The Convex Exemplar implemented scalable coherent shared memory using a two-level architecture: At the lowest level, eight-processor modules are built using a crossbar. A ring can then connect up to 32 of these modules, for a total of 256 processors (see Thekkath et al. [1997] for an evaluation). Laudon and Lenoski [1997] described the SGI Origin, which was first delivered in 1996 and is closely based on the original Stanford DASH machine, although including a number of innovations for scalability and ease of programming. Origin uses a bit vector for the directory structure, which is either 16 or 32 bits long. Each bit represents a node, which consists of two processors; a coarse bit vector representation allows each bit to represent up to 8 nodes for a total of 1024 processors. As Galles [1996] described, a high-performance fat hypercube is used for the global interconnect. Hristea, Lenoski, and Keen [1997] have provided a thorough evaluation of the performance of the Origin memory system.

Several research prototypes were undertaken to explore scalable coherence with and without multithreading. These include the MIT Alewife machine [Agarwal et al. 1995] and the Stanford FLASH multiprocessor [Gibson et al. 2000; Kuskin et al. 1994].

Clusters

Clusters were probably "invented" in the 1960s by customers who could not fit all their work on one computer or who needed a backup machine in case of failure of the primary machine [Pfister 1998]. Tandem introduced a 16-node cluster in 1975. Digital followed with VAX clusters, introduced in 1984. They were originally independent computers that shared I/O devices, requiring a distributed operating system to coordinate activity. Soon they had communication links between computers, in part so that the computers could be geographically distributed to increase availability in case of a disaster at a single site. Users log onto the cluster and are unaware of which machine they are running on. DEC (now HP) sold more than 25,000 clusters by 1993. Other early companies were Tandem (now HP) and IBM (still IBM). Today, virtually every company has cluster products. Most of these products are aimed at availability, with performance scaling as a secondary benefit.

Scientific computing on clusters emerged as a competitor to MPPs. In 1993, the Beowulf project started with the goal of fulfilling NASA's desire for a 1 GFLOPS computer for under $50,000. In 1994, a 16-node cluster built from off-the-shelf

PCs using 80486s achieved that goal [Bell and Gray 2001]. This emphasis led to a variety of software interfaces to make it easier to submit, coordinate, and debug large programs or a large number of independent programs.

Efforts were made to reduce latency of communication in clusters as well as to increase bandwidth, and several research projects worked on that problem. (One commercial result of the low-latency research was the VI interface standard, which has been embraced by Infiniband, discussed below.) Low latency then proved useful in other applications. For example, in 1997 a cluster of 100 UltraSPARC desktop computers at the University of California–Berkeley, connected by 160 MB/sec per link Myrinet switches, was used to set world records in database sort—sorting 8.6 GB of data originally on disk in 1 minute—and in cracking an encrypted message—taking just 3.5 hours to decipher a 40-bit DES key.

This research project, called Network of Workstations [Anderson, Culler, and Patterson 1995], also developed the Inktomi search engine, which led to a startup company with the same name. Google followed the example of Inktomi to build search engines from clusters of desktop computers rather large-scale SMPs, which was the strategy of the leading search engine Alta Vista that Google overtook [Brin and Page 1998]. In 2011, nearly all Internet services rely on clusters to serve their millions of customers.

Clusters are also very popular with scientists. One reason is their low cost, so individual scientists or small groups can own a cluster dedicated to their programs. Such clusters can get results faster than waiting in the long job queues of the shared MPPs at supercomputer centers, which can stretch to weeks. For those interested in learning more, Pfister [1998] wrote an entertaining book on clusters.

Recent Trends in Large-Scale Multiprocessors

In the mid- to late 1990s, it became clear that the hoped for growth in the market for ultralarge-scale parallel computing was unlikely to occur. Without this market growth, it became increasingly clear that the high-end parallel computing market could not support the costs of highly customized hardware and software designed for a small market. Perhaps the most important trend to come out of this observation was that clustering would be used to reach the highest levels of performance. There are now four general classes of large-scale multiprocessors:

- Clusters that integrate standard desktop motherboards using interconnection technology such as Myrinet or Infiniband.
- Multicomputers built from standard microprocessors configured into processing elements and connected with a custom interconnect. These include the Cray XT3 (which used an earlier version of Cray interconnect with a simple cluster architecture) and IBM Blue Gene (more on this unique machine momentarily).
- Clusters of small-scale shared-memory computers, possibly with vector support, which includes the Earth Simulator (which has its own journal available online).

- Large-scale shared-memory multiprocessors, such as the Cray X1 [Dunigan et al. 2005] and SGI Origin and Altix systems. The SGI systems have also been configured into clusters to provide more than 512 processors, although only message passing is supported across the clusters.

The IBM Blue Gene is the most interesting of these designs since its rationale parallels the underlying causes of the recent trend toward multicore in uniprocessor architectures. Blue Gene started as a research project within IBM aimed at the protein sequencing and folding problem. The Blue Gene designers observed that power was becoming an increasing concern in large-scale multiprocessors and that the performance/watt of processors from the embedded space was much better that those in the high-end uniprocessor space. If parallelism was the route to high performance, why not start with the most efficient building block and simply have more of them?

Thus, Blue Gene is constructed using a custom chip that includes an embedded PowerPC microprocessor offering half the performance of a high-end PowerPC, but at a much smaller fraction of the area of power. This allows more system functions, including the global interconnect, to be integrated onto the same die. The result is a highly replicable and efficient building block, allowing Blue Gene to reach much larger processor counts more efficiently. Instead of using stand-alone microprocessors or standard desktop boards as building blocks, Blue Gene uses processor cores. There is no doubt that such an approach provides much greater efficiency. Whether the market can support the cost of a customized design and special software remains an open question.

In 2006, a Blue Gene processor at Lawrence Livermore with 32K processors (and scheduled to go to 65K in late 2005) holds a factor of 2.6 lead in Linpack performance over the third-place system consisting of 20 SGI Altix 512-processor systems interconnected with Infiniband as a cluster.

Blue Gene's predecessor was an experimental machine, QCDOD, which pioneered the concept of a machine using a lower-power embedded microprocessor and tightly integrated interconnect to drive down the cost and power consumption of a node.

Developments in Synchronization and Consistency Models

A wide variety of synchronization primitives have been proposed for shared-memory multiprocessors. Mellor-Crummey and Scott [1991] provided an overview of the issues as well as efficient implementations of important primitives, such as locks and barriers. An extensive bibliography supplies references to other important contributions, including developments in spin locks, queuing locks, and barriers. Lamport [1979] introduced the concept of sequential consistency and what correct execution of parallel programs means. Dubois, Scheurich, and Briggs [1988] introduced the idea of weak ordering (originally in 1986). In 1990, Adve and Hill provided a better definition of weak ordering and also defined the concept of data-race-free; at the same conference, Gharachorloo and his colleagues [1990] introduced release consistency and provided the first data on the performance of

relaxed consistency models. More relaxed consistency models have been widely adopted in microprocessor architectures, including the Sun SPARC, Alpha, and IA-64. Adve and Gharachorloo [1996] have provided an excellent tutorial on memory consistency and the differences among these models.

Other References

The concept of using virtual memory to implement a shared address space among distinct machines was pioneered in Kai Li's Ivy system in 1988. There have been subsequent papers exploring hardware support issues, software mechanisms, and programming issues. Amza et al. [1996] described a system built on workstations using a new consistency model, Kontothanassis et al. [1997] described a software shared-memory scheme using remote writes, and Erlichson et al. [1996] described the use of shared virtual memory to build large-scale multiprocessors using SMPs as nodes.

There is an almost unbounded amount of information on multiprocessors and multicomputers: Conferences, journal papers, and even books seem to appear faster than any single person can absorb the ideas. No doubt many of these papers will go unnoticed—not unlike the past. Most of the major architecture conferences contain papers on multiprocessors. An annual conference, Supercomputing *XY* (where *X* and *Y* are the last two digits of the year), brings together users, architects, software developers, and vendors, and the proceedings are published in book, CD-ROM, and online (see *www.scXY.org*) form. Two major journals, *Journal of Parallel and Distributed Computing* and the *IEEE Transactions on Parallel and Distributed Systems*, contain papers on all aspects of parallel processing. Several books focusing on parallel processing are included in the following references, with Culler, Singh, and Gupta [1999] being the most recent, large-scale effort. For years, Eugene Miya of NASA Ames Research Center has collected an online bibliography of parallel-processing papers. The bibliography, which now contains more than 35,000 entries, is available online at *liinwww.ira.uka.de/bibliography/Parallel/Eugene/index.html*.

In addition to documenting the discovery of concepts now used in practice, these references also provide descriptions of many ideas that have been explored and found wanting, as well as ideas whose time has just not yet come. Given the move toward multicore and multiprocessors as the future of high-performance computer architecture, we expect that many new approaches will be explored in the years ahead. A few of them will manage to solve the hardware and software problems that have been the key to using multiprocessing for the past 40 years!

References

Adve, S. V., and K. Gharachorloo [1996]. "Shared memory consistency models: A tutorial," *IEEE Computer* 29:12 (December), 66–76.

Adve, S. V., and M. D. Hill [1990]. "Weak ordering—a new definition," *Proc. 17th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–31, 1990, Seattle, Wash., 2–14.

Agarwal, A., R. Bianchini, D. Chaiken, K. Johnson, and D. Kranz [1995]. "The MIT Alewife machine: Architecture and performance," *22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy, 2–13.

Agarwal, A., J. L. Hennessy, R. Simoni, and M. A. Horowitz [1988]. "An evaluation of directory schemes for cache coherence," *Proc. 15th Annual Int'l. Symposium on Computer Architecture*, May 30–June 2, 1988, Honolulu, Hawaii, 280–289.

Agarwal, A., J. Kubiatowicz, D. Kranz, B.-H. Lim, D. Yeung, G. D'Souza, and M. Parkin [1993]. "Sparcle: An evolutionary processor design for large-scale multiprocessors," *IEEE Micro* 13 (June), 48–61.

Alles, A. [1995]. "ATM Internetworking," White Paper (May), Cisco Systems, Inc., San Jose, Calif. (*www.cisco.com/warp/public/614/12.html*).

Almasi, G. S., and A. Gottlieb [1989]. *Highly Parallel Computing*, Benjamin/Cummings, Redwood City, Calif.

Alverson, G., R. Alverson, D. Callahan, B. Koblenz, A. Porterfield, and B. Smith [1992]. "Exploiting heterogeneous parallelism on a multithreaded multiprocessor," *Proc. ACM/IEEE Conf. on Supercomputing*, November 16–20, 1992, Minneapolis, Minn., 188–197.

Amdahl, G. M. [1967]. "Validity of the single processor approach to achieving large scale computing capabilities," *Proc. AFIPS Spring Joint Computer Conf.*, April 18–20, 1967, Atlantic City, N.J., 483–485.

Amza C., A. L. Cox, S. Dwarkadas, P. Keleher, H. Lu, R. Rajamony, W. Yu, and W. Zwaenepoel [1996]. "Treadmarks: Shared memory computing on networks of workstations," *IEEE Computer* 29:2 (February), 18–28.

Anderson, T. E., D. E. Culler, and D. Patterson [1995]. "A case for NOW (networks of workstations)," *IEEE Micro* 15:1 (February), 54–64.

Ang, B., D. Chiou, D. Rosenband, M. Ehrlich, L. Rudolph, and Arvind [1998]. "StarT-Voyager: A flexible platform for exploring scalable SMP issues," *Proc. ACM/IEEE Conf. on Supercomputing*, November 7–13, 1998, Orlando, FL.

Archibald, J., and J.-L. Baer [1986]. "Cache coherence protocols: Evaluation using a multiprocessor simulation model," *ACM Trans. on Computer Systems* 4:4 (November), 273–298.

Arpaci, R. H., D. E. Culler, A. Krishnamurthy, S. G. Steinberg, and K. Yelick [1995]. "Empirical evaluation of the CRAY-T3D: A compiler perspective," *Proc. 22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy.

Baer, J.-L., and W.-H. Wang [1988]. "On the inclusion properties for multi-level cache hierarchies," *Proc. 15th Annual Int'l. Symposium on Computer Architecture*, May 30–June 2, 1988, Honolulu, Hawaii, 73–80.

Balakrishnan, H. V., N. Padmanabhan, S. Seshan, and R. H. Katz [1997]. "A comparison of mechanisms for improving TCP performance over wireless links," *IEEE/ACM Trans. on Networking* 5:6 (December), 756–769.

Barroso, L. A., K. Gharachorloo, and E. Bugnion [1998]. "Memory system characterization of commercial workloads," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 3–14.

Baskett, F., T. Jermoluk, and D. Solomon [1988]. "The 4D-MP graphics superworkstation: Computing + graphics = 40 MIPS + 40 MFLOPS and 10,000 lighted polygons per second," *Proc. IEEE COMPCON*, February 29–March 4, 1988, San Francisco, 468–471.

BBN Laboratories. [1986]. *Butterfly Parallel Processor Overview*, Tech. Rep. 6148, BBN Laboratories, Cambridge, Mass.

Bell, C. G. [1985]. "Multis: A new class of multiprocessor computers," *Science* 228 (April 26), 462–467.

Bell, C. G. [1989]. "The future of high performance computers in science and engineering," *Communications of the ACM* 32:9 (September), 1091–1101.

Bell, C. G., and J. Gray [2001]. *Crays, Clusters and Centers*, Tech. Rep. MSR-TR-2001-76, Microsoft Research, Redmond, Wash.

Bell, C. G., and J. Gray [2002]. "What's next in high performance computing," *CACM*, 45:2 (February), 91–95.

Bouknight, W. J., S. A. Deneberg, D. E. McIntyre, J. M. Randall, A. H. Sameh, and D. L. Slotnick [1972]. "The Illiac IV system," *Proc. IEEE* 60:4, 369–379. Also appears in D. P. Siewiorek, C. G. Bell, and A. Newell, *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 1982, 306–316.

Brain, M. [2000]. *Inside a Digital Cell Phone, www.howstuffworks.com/inside-cell-phone.htm.*

Brewer, E. A., and B. C. Kuszmaul [1994]. "How to get good performance from the CM-5 data network," *Proc. Eighth Int'l. Parallel Processing Symposium (IPPS)*, April 26–29, 1994, Cancun, Mexico.

Brin, S., and L. Page [1998]. "The anatomy of a large-scale hypertextual Web search engine," *Proc. 7th Int'l. World Wide Web Conf.*, April 14–18, 1998, Brisbane, Queensland, Australia, 107–117.

Burkhardt III, H., S. Frank, B. Knobe, and J. Rothnie [1992]. *Overview of the KSR1 Computer System*, Tech. Rep. KSR-TR-9202001, Kendall Square Research, Boston.

Censier, L., and P. Feautrier [1978]. "A new solution to coherence problems in multicache systems," *IEEE Trans. on Computers* C-27:12 (December), 1112–1118.

Chandra, R., S. Devine, B. Verghese, A. Gupta, and M. Rosenblum [1994]. "Scheduling and page migration for multiprocessor compute servers," *Proc. Sixth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 4–7, 1994, San Jose, Calif., 12–24.

Charlesworth, A. [1998]. "Starfire: Extending the SMP envelope," *IEEE Micro* 18:1 (January/February), 39–49.

Clark, W. A. [1957]. "The Lincoln TX-2 computer development," *Proc. Western Joint Computer Conference*, February 26–28, 1957, Los Angeles, 143–145.

Comer, D. [1993]. *Internetworking with TCP/IP*, 2nd ed., Prentice Hall, Englewood Cliffs, N.J.

Culler, D. E., J. P. Singh, and A. Gupta [1999]. *Parallel Computer Architecture: A Hardware/Software Approach*, Morgan Kaufmann, San Francisco.

Dally, W. J., and C. I. Seitz [1986]. "The torus routing chip," *Distributed Computing* 1:4, 187–196.

Davie, B. S., L. L. Peterson, and D. Clark [1999]. *Computer Networks: A Systems Approach*, 2nd ed., Morgan Kaufmann, San Francisco.

Desurvire, E. [1992]. "Lightwave communications: The fifth generation," *Scientific American* (International Edition) 266:1 (January), 96–103.

Dongarra, J., T. Sterling, H. Simon, and E. Strohmaier [2005]. "High-performance computing: Clusters, constellations, MPPs, and future directions," *Computing in Science & Engineering*, 7:2 (March/April), 51–59.

Dubois, M., C. Scheurich, and F. Briggs [1988]. "Synchronization, coherence, and event ordering," *IEEE Computer* 21:2 (February), 9–21.

Dunigan, W., K. Vetter, K. White, and P. Worley [2005]. "Performance evaluation of the Cray X1 distributed shared memory architecture," *IEEE Micro*, January/February, 30–40.

Eggers, S. [1989]. "Simulation Analysis of Data Sharing in Shared Memory Multiprocessors," Ph.D. thesis, Computer Science Division, University of California, Berkeley.

Elder, J., A. Gottlieb, C. K. Kruskal, K. P. McAuliffe, L. Randolph, M. Snir, P. Teller, and J. Wilson [1985]. "Issues related to MIMD shared-memory computers: The NYU Ultracomputer approach," *Proc. 12th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 17–19, 1985, Boston, Mass., 126–135.

Erlichson, A., N. Nuckolls, G. Chesson, and J. L. Hennessy [1996]. "SoftFLASH: Analyzing the performance of clustered distributed virtual shared memory," *Proc. Seventh Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 1–5, 1996, Cambridge, Mass., 210–220.

Falsafi, B., and D. A. Wood [1997]. "Reactive NUMA: A design for unifying S-COMA and CC-NUMA," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo., 229–240.

Flynn, M. J. [1966]. "Very high-speed computing systems," *Proc. IEEE* 54:12 (December), 1901–1909.

Forgie, J. W. [1957]. "The Lincoln TX-2 input-output system," *Proc. Western Joint Computer Conference*, February 26–28, 1957, Los Angeles, 156–160.

Frank, S. J. [1984]. "Tightly coupled multiprocessor systems speed memory access time," *Electronics* 57:1 (January), 164–169.

Gajski, D., D. Kuck, D. Lawrie, and A. Sameh [1983]. "CEDAR—a large scale multiprocessor," *Proc. Int'l. Conf. on Parallel Processing (ICPP)*, August, Columbus, Ohio, 524–529.

Galles, M. [1996]. "Scalable pipelined interconnect for distributed endpoint routing: The SGI SPIDER chip," *Proc. IEEE HOT Interconnects '96*, August 15–17, 1996, Stanford University, Palo Alto, Calif.

Gehringer, E. F., D. P. Siewiorek, and Z. Segall [1987]. *Parallel Processing: The Cm* Experience*, Digital Press, Bedford, Mass.

Gharachorloo, K., A. Gupta, and J. L. Hennessy [1992]. "Hiding memory latency using dynamic scheduling in shared-memory multiprocessors," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia.

Gharachorloo, K., D. Lenoski, J. Laudon, P. Gibbons, A. Gupta, and J. L. Hennessy [1990]. "Memory consistency and event ordering in scalable shared-memory multiprocessors," *Proc. 17th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–31, 1990, Seattle, Wash., 15–26.

Gibson, J., R. Kunz, D. Ofelt, M. Horowitz, J. Hennessy, and M. Heinrich [2000]. "FLASH vs. (simulated) FLASH: Closing the simulation loop," *Proc. Ninth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, November 12–15, Cambridge, Mass., 49–58.

Goodman, J. R. [1983]. "Using cache memory to reduce processor memory traffic," *Proc. 10th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1982, Stockholm, Sweden, 124–131.

Goralski, W. [1997]. *SONET: A Guide to Synchronous Optical Network*, McGraw-Hill, New York.

Grice, C., and M. Kanellos [2000]. "Cell phone industry at crossroads: Go high or low?" *CNET News (*August 31), *technews.netscape.com/news/0-1004-201-2518386-0.html?tag=st.ne.1002.tgif.sf*.

Groe, J. B., and L. E. Larson [2000]. *CDMA Mobile Radio Design*, Artech House, Boston.

Hagersten E., and M. Koster [1998]. "WildFire: A scalable path for SMPs," *Proc. Fifth Int'l. Symposium on High-Performance Computer Architecture*, January 9–12, 1999, Orlando, Fla.

Hagersten, E., A. Landin, and S. Haridi [1992]. "DDM—a cache-only memory architecture," *IEEE Computer* 25:9 (September), 44–54.

Hill, M. D. [1998]. "Multiprocessors should support simple memory consistency models," *IEEE Computer* 31:8 (August), 28–34.

Hillis, W. D. [1985]. *The Connection Multiprocessor*, MIT Press, Cambridge, Mass.

Hirata, H., K. Kimura, S. Nagamine, Y. Mochizuki, A. Nishimura, Y. Nakase, and T. Nishizawa [1992]. "An elementary processor architecture with simultaneous instruction issuing from multiple threads," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 136–145.

Hockney, R. W., and C. R. Jesshope [1988]. *Parallel Computers 2: Architectures, Programming and Algorithms*, Adam Hilger, Ltd., Bristol, England.

Holland, J. H. [1959]. "A universal computer capable of executing an arbitrary number of subprograms simultaneously," *Proc. East Joint Computer Conf.* 16, 108–113.

Hord, R. M. [1982]. *The Illiac-IV, The First Supercomputer*, Computer Science Press, Rockville, Md.

Hristea, C., D. Lenoski, and J. Keen [1997]. "Measuring memory hierarchy performance of cache-coherent multiprocessors using micro benchmarks," *Proc. ACM/IEEE Conf. on Supercomputing*, November 15–21, 1997, San Jose, Calif.

Hwang, K. [1993]. *Advanced Computer Architecture and Parallel Programming*, McGraw-Hill, New York.

IBM. [2005]. "Blue Gene," *IBM J. of Research and Development*, 49:2/3 (special issue).

Infiniband Trade Association. [2001]. *InfiniBand Architecture Specifications Release 1.0.a, www.infinibandta.org*.

Jordan, H. F. [1983]. "Performance measurements on HEP—a pipelined MIMD computer," *Proc. 10th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 5–7, 1982, Stockholm, Sweden, 207–212.

Kahn, R. E. [1972]. "Resource-sharing computer communication networks," *Proc. IEEE* 60:11 (November), 1397–1407.

Keckler, S. W., and W. J. Dally [1992]. "Processor coupling: Integrating compile time and runtime scheduling for parallelism," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 202–213.

Kontothanassis, L., G. Hunt, R. Stets, N. Hardavellas, M. Cierniak, S. Parthasarathy, W. Meira, S. Dwarkadas, and M. Scott [1997]. "VM-based shared memory on low-latency, remotememory-access networks," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo.

Kurose, J. F., and K. W. Ross [2001]. *Computer Networking: A Top-Down Approach Featuring the Internet*, Addison-Wesley, Boston.

Kuskin, J., D. Ofelt, M. Heinrich, J. Heinlein, R. Simoni, K. Gharachorloo, J. Chapin, D. Nakahira, J. Baxter, M. Horowitz, A. Gupta, M. Rosenblum, and J. L. Hennessy [1994]. "The Stanford FLASH multiprocessor," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago.

Lamport, L. [1979]. "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Trans. on Computers* C-28:9 (September), 241–248.

Laudon, J., A. Gupta, and M. Horowitz [1994]. "Interleaving: A multithreading technique targeting multiprocessors and workstations," *Proc. Sixth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 4–7, 1994, San Jose, Calif., 308–318.

Laudon, J., and D. Lenoski [1997]. "The SGI Origin: A ccNUMA highly scalable server," *Proc. 24th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–4, 1997, Denver, Colo., 241–251.

Lenoski, D., J. Laudon, K. Gharachorloo, A. Gupta, and J. L. Hennessy [1990]. "The Stanford DASH multiprocessor," *Proc. 17th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 28–31, 1990, Seattle, Wash., 148–159.

Lenoski, D., J. Laudon, K. Gharachorloo, W.-D. Weber, A. Gupta, J. L. Hennessy, M. A. Horowitz, and M. Lam [1992]. "The Stanford DASH multiprocessor," *IEEE Computer* 25:3 (March), 63–79.

Li, K. [1988]. "IVY: A shared virtual memory system for parallel computing," *Proc. Int'l. Conf. on Parallel Processing (ICCP)*, August, The Pennsylvania State University, University Park, Penn.

Lo, J., L. Barroso, S. Eggers, K. Gharachorloo, H. Levy, and S. Parekh [1998]. "An analysis of database workload performance on simultaneous multithreaded

processors," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 39–50.

Lo, J., S. Eggers, J. Emer, H. Levy, R. Stamm, and D. Tullsen [1997]. "Converting thread-level parallelism into instruction-level parallelism via simultaneous multithreading," *ACM Trans. on Computer Systems* 15:2 (August), 322–354.

Lovett, T., and S. Thakkar [1988]. "The Symmetry multiprocessor system," *Proc. Int'l. Conf. on Parallel Processing (ICCP)*, August, The Pennsylvania State University, University Park, Penn., 303–310.

Mellor-Crummey, J. M., and M. L. Scott [1991]. "Algorithms for scalable synchronization on shared-memory multiprocessors," *ACM Trans. on Computer Systems* 9:1 (February), 21–65.

Menabrea, L. F. [1842]. "Sketch of the analytical engine invented by Charles Babbage," *Bibliothèque Universelle de Genève*, 82 (October).

Metcalfe, R. M. [1993]. "Computer/network interface design: Lessons from Arpanet and Ethernet." *IEEE J. on Selected Areas in Communications* 11:2 (February), 173–180.

Metcalfe, R. M., and D. R. Boggs [1976]. "Ethernet: Distributed packet switching for local computer networks," *Communications of the ACM* 19:7 (July), 395–404.

Mitchell, D. [1989]. "The Transputer: The time is now," *Computer Design* (RISC suppl.), 40–41.

Miya, E. N. [1985]. "Multiprocessor/distributed processing bibliography," *Computer Architecture News* 13:1, 27–29.

National Research Council. [1997]. *The Evolution of Untethered Communications*, Computer Science and Telecommunications Board, National Academy Press, Washington, D.C.

Nikhil, R. S., G. M. Papadopoulos, and Arvind [1992]. "*T: A multithreaded massively parallel architecture," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 156–167.

Noordergraaf, L., and R. van der Pas [1999]. "Performance experiences on Sun's WildFire prototype," *Proc. ACM/IEEE Conf. on Supercomputing*, November 13–19, 1999, Portland, Ore.

Partridge, C. [1994]. *Gigabit Networking*, Addison-Wesley, Reading, Mass.

Pfister, G. F. [1998]. *In Search of Clusters*, 2nd ed., Prentice Hall, Upper Saddle River, N.J.

Pfister, G. F., W. C. Brantley, D. A. George, S. L. Harvey, W. J. Kleinfekder, K. P. McAuliffe, E. A. Melton, V. A. Norton, and J. Weiss [1985]. "The IBM research parallel processor prototype (RP3): Introduction and architecture," *Proc. 12th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 17–19, 1985, Boston, Mass., 764–771.

Reinhardt, S. K., J. R. Larus, and D. A. Wood [1994]. "Tempest and Typhoon: User-level shared memory," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago, 325–336.

Rettberg, R. D., W. R. Crowther, P. P. Carvey, and R. S. Towlinson [1990]. "The Monarch parallel processor hardware design," *IEEE Computer* 23:4 (April), 18–30.

Rosenblum, M., S. A. Herrod, E. Witchel, and A. Gupta [1995]. "Complete computer simulation: The SimOS approach," in *IEEE Parallel and Distributed Technology* (now called *Concurrency*) 4:3, 34–43.

Saltzer, J. H., D. P. Reed, and D. D. Clark [1984]. "End-to-end arguments in system design," *ACM Trans. on Computer Systems* 2:4 (November), 277–288.

Satran, J., D. Smith, K. Meth, C. Sapuntzakis, M. Wakeley, P. Von Stamwitz, R. Haagens, E. Zeidner, L. Dalle Ore, and Y. Klein [2001]. "iSCSI," IPS Working Group of IETF, Internet draft *www.ietf.org/internet-drafts/draft-ietf-ips-iscsi-07.txt*.

Saulsbury, A., T. Wilkinson, J. Carter, and A. Landin [1995]. "An argument for Simple COMA," *Proc. First IEEE Symposium on High-Performance Computer Architectures*, January 22–25, 1995, Raleigh, N.C., 276–285.

Schwartz, J. T. [1980]. "Ultracomputers," *ACM Trans. on Programming Languages and Systems* 4:2, 484–521.

Scott, S. L. [1996]. "Synchronization and communication in the T3E multiprocessor," *Seventh Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 1–5, 1996, Cambridge, Mass., 26–36.

Scott, S. L., and G. M. Thorson [1996]. "The Cray T3E network: Adaptive routing in a high-performance 3D torus," *Proc. IEEE HOT Interconnects '96*, August 15–17, 1996, Stanford University, Palo Alto, Calif., 14–156.

Seitz, C. L. [1985]. "The Cosmic Cube (concurrent computing)," *Communications of the ACM* 28:1 (January), 22–33.

Singh, J. P., J. L. Hennessy, and A. Gupta [1993]. "Scaling parallel programs for multiprocessors: Methodology and examples," *Computer* 26:7 (July), 22–33.

Slotnick, D. L., W. C. Borck, and R. C. McReynolds [1962]. "The Solomon computer," *Proc. AFIPS Fall Joint Computer Conf.*, December 4–6, 1962, Philadelphia, Penn., 97–107.

Smith, B. J. [1978]. "A pipelined, shared resource MIMD computer," *Proc. Int'l. Conf. on Parallel Processing (ICPP)*, August, Bellaire, Mich., 6–8.

Soundararajan, V., M. Heinrich, B. Verghese, K. Gharachorloo, A. Gupta, and J. L. Hennessy [1998]. "Flexible use of memory for replication/migration in cache-coherent DSM multiprocessors," *Proc. 25th Annual Int'l. Symposium on Computer Architecture (ISCA)*, July 3–14, 1998, Barcelona, Spain, 342–355.

Spurgeon, C. [2001]. "Charles Spurgeon's Ethernet Web site," *www.host.ots.utexas.edu/ethernet/ethernet-home.html*.

Stenström, P., T. Joe, and A. Gupta [1992]. "Comparative performance evaluation of cachecoherent NUMA and COMA architectures," *Proc. 19th Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 19–21, 1992, Gold Coast, Australia, 80–91.

Sterling, T. [2001]. *Beowulf PC Cluster Computing with Windows and Beowulf PC Cluster Computing with Linux*, MIT Press, Cambridge, Mass.

Stevens, W. R. [1994–1996]. *TCP/IP Illustrated* (three volumes), Addison-Wesley, Reading, Mass.

Stone, H. [1991]. *High Performance Computers*, Addison-Wesley, New York.

Swan, R. J., A. Bechtolsheim, K. W. Lai, and J. K. Ousterhout [1977]. "The implementation of the Cm* multi-microprocessor," *Proc. AFIPS National Computing Conf.*, June 13–16, 1977, Dallas, Tex., 645–654.

Swan, R. J., S. H. Fuller, and D. P. Siewiorek [1977]. "Cm*—a modular, multi-microprocessor," *Proc. AFIPS National Computing Conf.*, June 13–16, 1977, Dallas, Tex., 637–644.

Tanenbaum, A. S. [1988]. *Computer Networks*, 2nd ed., Prentice Hall, Englewood Cliffs, N.J.

Tang, C. K. [1976]. "Cache design in the tightly coupled multiprocessor system," *Proc. AFIPS National Computer Conf.*, June 7–10, 1976, New York, 749–753.

Thacker, C. P., E. M. McCreight, B. W. Lampson, R. F. Sproull, and D. R. Boggs [1982]. "Alto: A personal computer," in D. P. Siewiorek, C. G. Bell, and A. Newell, eds., *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 549–572.

Thekkath, R., A. P. Singh, J. P. Singh, S. John, and J. L. Hennessy [1997]. "An evaluation of a commercial CC-NUMA architecture—the CONVEX Exemplar SPP1200," *Proc. 11th Int'l. Parallel Processing Symposium (IPPS)*, April 1–7, 1997, Geneva, Switzerland.

Tullsen, D. M., S. J. Eggers, J. S. Emer, H. M. Levy, J. L. Lo, and R. L. Stamm [1996]. "Exploiting choice: Instruction fetch and issue on an implementable simultaneous multithreading processor," *Proc. 23rd Annual Int'l. Symposium on Computer Architecture (ISCA)*, May 22–24, 1996, Philadelphia, Penn., 191–202.

Tullsen, D. M., S. J. Eggers, and H. M. Levy [1995]. "Simultaneous multithreading: Maximizing on-chip parallelism," *Proc. 22nd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 22–24, 1995, Santa Margherita, Italy, 392–403.

Unger, S. H. [1958]. "A computer oriented towards spatial problems," *Proc. Institute of Radio Engineers* 46:10 (October), 1744–1750.

Walrand, J. [1991]. *Communication Networks: A First Course*, Aksen Associates: Irwin, Homewood, Ill.

Wilson, A. W., Jr. [1987]. "Hierarchical cache/bus architecture for shared-memory multiprocessors," *Proc. 14th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 2–5, 1987, Pittsburgh, Penn., 244–252.

Wolfe, A., and J. P. Shen [1991]. "A variable instruction stream extension to the VLIW architecture." *Proc. Fourth Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, April 8–11, 1991, Palo Alto, Calif., 2–14.

Wood, D. A., and M. D. Hill [1995]. "Cost-effective parallel computing," *IEEE Computer* 28:2 (February), 69–72.

Wulf, W., and C. G. Bell [1972]. "C.mmp—A multi-mini-processor," *Proc. AFIPS Fall Joint Computer Conf.*, December 5–7, 1972, Anaheim, Calif., 765–777.

Wulf, W., and S. P. Harbison [1978]. "Reflections in a pool of processors—an experience report on C.mmp/Hydra," *Proc. AFIPS National Computing Conf.* June 5–8, 1978, Anaheim, Calif., 939–951.

Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky [1994]. "Performance estimation of multistreamed, superscalar processors," *Proc. 27th Hawaii Int'l. Conf. on System Sciences*, January 4–7, 1994, Wailea, 195–204.

M.8 The Development of Clusters (Chapter 6)

In this section, we cover the development of clusters that were the foundation of warehouse-scale computers (WSCs) and of utility computing. (Readers interested in learning more should start with Barroso and Hölzle [2009] and the blog postings and talks of James Hamilton at *http://perspectives.mvdirona.com*.)

Clusters, the Forerunner of WSCs

Clusters were probably "invented" in the 1960s by customers who could not fit all their work on one computer or who needed a backup machine in case of failure of the primary machine [Pfister 1998]. Tandem introduced a 16-node cluster in 1975. Digital followed with VAX clusters, introduced in 1984. They were originally independent computers that shared I/O devices, requiring a distributed operating system to coordinate activity. Soon they had communication links between computers, in part so that the computers could be geographically distributed to increase availability in case of a disaster at a single site. Users log onto the cluster and are unaware of which machine they are running on. DEC (now HP) sold more than 25,000 clusters by 1993. Other early companies were Tandem (now HP) and IBM (still IBM). Today, virtually every company has cluster products. Most of these products are aimed at availability, with performance scaling as a secondary benefit.

Scientific computing on clusters emerged as a competitor to MPPs. In 1993, the Beowulf project started with the goal of fulfilling NASA's desire for a 1 GFLOPS computer for under $50,000. In 1994, a 16-node cluster built from off-the-shelf PCs using 80486s achieved that goal [Bell and Gray 2001]. This emphasis led to a variety of software interfaces to make it easier to submit, coordinate, and debug large programs or a large number of independent programs.

Efforts were made to reduce latency of communication in clusters as well as to increase bandwidth, and several research projects worked on that problem. (One commercial result of the low-latency research was the VI interface standard, which has been embraced by Infiniband, discussed below.) Low latency then proved useful in other applications. For example, in 1997 a cluster of 100 UltraSPARC desktop computers at the University of California–Berkeley, connected by 160 MB/sec per link Myrinet switches, was used to set world records in database sort—sorting 8.6 GB of data originally on disk in 1 minute—and in cracking an encrypted message—taking just 3.5 hours to decipher a 40-bit DES key.

This research project, called Network of Workstations [Anderson, Culler, and Patterson 1995], also developed the Inktomi search engine, which led to a start-up

company with the same name. Eric Brewer led the Inktomi effort at Berkeley and then at the company to demonstrate the use of commodity hardware to build computing infrastructure for Internet services. Using standardized networks within a rack of PC servers gave Inktomi better scalability. In contrast, the strategy of the prior leading search engine Alta Vista was to build from large-scale SMPs. Compared to the high-performance computing work in clusters, the emphasis was on a relatively large number of low-cost nodes and a clear programming model. Hence, the NOW project and Inktomi are considered the foundation of WSCs and Cloud Computing. Google followed the example of Inktomi technology when it took the leading search engine mantle from Inktomi just as Inktomi had taken it from Alta Vista [Brin and Page 1998]. (Google's initial innovation was search quality; the WSC innovations came much later.) For many years now, all Internet services have relied on cluster technology to serve their millions of customers.

Utility Computing, the Forerunner of Cloud Computing

As stated in the text, the earliest version of utility computing was timesharing. Although timesharing faded away over time with the creation of smaller and cheaper personal computers, in the last decade there have been many less than fully successful attempts to resuscitate utility computing. Sun began selling time on Sun Cloud at $1 per hour in 2000, HP offered a Utility Data Center in 2001, and Intel tried selling time on internal supercomputers in the early 2000s. Although they were commercially available, few customers used them.

A related topic is *grid computing*, which was originally invented so that scientific programs could be run across geographically distributed computing facilities. At the time, some questioned the wisdom of this goal, setting aside how difficult it would be to achieve. Grid computing tended to require very large systems running very large programs, using multiple datacenters for the tasks. Single applications did not really run well when geographically distributed, given the long latencies inherent with long distance. This first step eventually led to some conventions for data access, but the grid computing community did not develop APIs that were useful beyond the high-performance computing community, so the cloud computing effort shares little code or history with grid computing.

Armbrust et al [2009] argued that, once the Internet service companies solved the operational problems to work at large scale, the significant economies of scale that they uncovered brought their costs down below those of smaller datacenters. Amazon recognized that if this cost advantage was true then Amazon should be able to make a profit selling this service. In 2006, Amazon announced Elastic Cloud Computing (EC2) at $0.10 per hour per instance. The subsequent popularity of EC2 led other Internet companies to offer cloud computing services, such as Google App Engine and Microsoft Azure, albeit at higher abstraction levels than the x86 virtual machines of Amazon Web Services. Hence, the current popularity of pay-as-you go computing isn't because someone recently came up with the idea;

it's because the technology and business models have aligned so that companies can make money offering a service that many people want to use. Time will tell whether there will be many successful utility computing models or whether the industry will converge around a single standard. It will certainly be interesting to watch.

Containers

In the fall of 2003, many people were thinking about using containers to hold servers. Brewster Kahle, director and founder of the Internet Archive, gave talks about how he could fit the whole archive in a single 40-foot container. His interest was making copies of the Archive and distributing it around the world to ensure its survivability, thereby avoiding the fate of the Library of Alexandria that was destroyed by fire in 48 B.C.E. People working with Kahle wrote a white paper based on his talk in November 2003 to get more detail about what a container design would look like.

That same year, engineers at Google were also looking at building datacenters using containers and submitted a patent on aspects of it in December 2003. The first container for a datacenter was delivered in January 2005, and Google received the patent in October 2007. Google publicly revealed the use of containers in April 2009.

Greg Papadopolous of Sun Microsystems and Danny Hillis of Applied Minds heard Kahle's talk and designed a product called the Sun Modular Datacenter that debuted in October 2006. (The project code name was Black Box, a term many people still use.) This half-length (20-foot) container could hold 280 servers. This product release combined with Microsoft's announcement that they were building a datacenter designed to hold 220 40-foot containers inspired many other companies to offer containers and servers designed to be placed in them.

In a nice turn of events, in 2009 the Internet Archive migrated its data to a Sun Modular Datacenter. A copy of the Internet Archive is now at the New Library of Alexandria in Egypt, near the site of the original library.

References

Anderson, T. E., D. E. Culler, and D. Patterson [1995]. "A case for NOW (networks of workstations)," *IEEE Micro* 15:1 (February), 54–64.

Apache Software Foundation. [2011]. Apache Hadoop project, *http://hadoop.apache.org*.

Armbrust, M., A. Fox, R. Griffith, A.D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia [2009]. *Above the Clouds: A Berkeley View of Cloud Computing*, Tech. Rep. UCB/EECS-2009-28, University of California, Berkeley (http://www.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.html).

Barroso, L. A. [2010]. "Warehouse scale computing [keynote address]," *Proc. ACM SIG-MOD*, June 8–10, 2010, Indianapolis, Ind.

Barroso, L. A., and U. Hölzle [2007]. "The case for energy-proportional computing," *IEEE Computer* 40:12 (December), 33–37.

Barroso, L.A., and U. Hölzle [2009]. "The datacenter as a computer: An introduction to the design of warehouse-scale machines," in M. D. Hill, ed., *Synthesis Lectures on Computer Architecture*, Morgan & Claypool, San Rafael, Calif.

Barroso, L.A., Clidaras, J. and Hölzle, U., 2013. *The datacenter as a computer:An introduction to the design of warehouse-scale machines*. Synthesis lectures on computer architecture, 8(3), pp.1–154.

Barroso, L.A., Marty, M., Patterson, D., and Ranganathan, P. 2017. Attack of the Killer Microseconds. *Communications of the ACM*, 56(2).

Bell, C. G., and J. Gray [2002]. "What's next in high performance computing," *Communications of the ACM* 45:2 (February), 91–95.

Brady, J.T., 1986. A theory of productivity in the creative process. IEEE Computer Graphics and Applications, 6(5), pp.25–34.

Brin, S., and L. Page [1998]. "The anatomy of a large-scale hypertextual Web search engine," *Proc. 7th Int'l. World Wide Web Conf.*, April 14–18, 1998, Brisbane, Queensland, Australia, 107–117.

Carter, J., and K. Rajamani [2010]. "Designing energy-efficient servers and data centers," *IEEE Computer* 43:7 (July), 76–78.

Chang, F., J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber [2006]. "Bigtable: A distributed storage system for structured data," in *Proc. Operating Systems Design and Implementation (OSDI '06)*, November 6–8, 2006, Seattle, Wash.

Chang, J., J. Meza, P. Ranganathan, C. Bash, and A. Shah [2010]. "Green server design: Beyond operational energy to sustainability," *Workshop on Power Aware Computing and Systems (HotPower '10)*, October 4–6, 2010, Vancouver, British Columbia.

Clark, J., 2014 Five Numbers That Illustrate the Mind-Bending Size of Amazon's Cloud, Bloomberg, https://www.bloomberg.com/news/2014-11-14/5-numbersthat-illustrate-the-mind-bending-size-of-amazon-s-cloud.html.

Clidaras, J., C. Johnson, and B. Felderman [2010]. Private communication.

Climate Savers Computing. [2007]. Efficiency specs, *http://www.climatesaverscomputing.org/*.

Clos, C., 1953. A Study of Non-Blocking Switching Networks. *Bell Labs Technical Journal*, 32(2), pp.406-424.

Dean, J. [2009]. "Designs, lessons and advice from building large distributed systems [keynote address]," *Proc. 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware, Co-located with the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009)*, October 10–11, 2009, Big Sky, Mont.

Dean, J. and Barroso, L.A., 2013. The tail at scale. *Communications of the ACM*, 56(2), pp.74-80.

Dean, J., and S. Ghemawat [2004]. "MapReduce: Simplified data processing on large clusters." In *Proc. Operating Systems Design and Implementation (OSDI '04)*, December 6–8, 2004, San Francisco, 137–150.

Dean, J., and S. Ghemawat [2008]. "MapReduce: simplified data processing on large clusters," *Communications of the ACM* 51:1, 107–113.

DeCandia, G., D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels [2007]. "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symposium on Operating Systems Principles*, October 14–17, 2007, Stevenson, Wash.

Doherty, W.J. and Thadhani, A.J., 1982. The economic value of rapid response time. IBM Report.

Fan, X., W. Weber, and L. A. Barroso [2007]. "Power provisioning for a warehouse-sized computer," in *Proc. 34th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 9–13, 2007, San Diego, Calif.

A. Fikes, "Storage architecture and challenges," in Google Faculty Summit, 2010.

Ghemawat, S., H. Gobioff, and S.-T. Leung (2003). "The Google file system," in *Proc. 19th ACM Symposium on Operating Systems Principles*, October 19–22, 2003, Lake George, N.Y.

Greenberg, A., N. Jain, S. Kandula, C. Kim, P. Lahiri, D. Maltz, P. Patel, and S. Sengupta [2009]. "VL2: A scalable and flexible data center network," in *Proc. SIGCOMM*, August 17–21, Barcelona, Spain.

González, A.and Day, M. April 27, 2016, "Amazon, Microsoft invest billions as computing shifts to cloud," *The Seattle Times*. http://www.seattletimes.com/business/technology/amazon-microsoft-invest-billions-as-computing-shifts-to-cloud/

Hamilton, J. [2009]. "Data center networks are in my way," Stanford Clean Slate CTO Summit, October 23, 2009, *http://mvdirona.com/jrh/TalksAndPapers/JamesHamilton_CleanSlateCTO2009.pdf*.

Hamilton, J. [2010]. "Cloud computing economies of scale," *Proc. AWS Workshop on Genomics & Cloud Computing*, June 8, 2010, Seattle, Wash. (*http://mvdirona.com/jrh/TalksAndPapers/JamesHamilton_GenomicsCloud20100608.pdf*).

Hamilton, J., 2014. AWS Innovation at Scale, AWS Re-invent conference. https://www.youtube.com/watch?v=JIQETrFC_SQ

Hamilton, J., May 2015. The Return to the Cloud, http://perspectives.mvdirona.com/2015/05/the-return-to-the-cloud//

Hamilton, J., April 2017. How Many Data Centers Needed World-Wide, http://perspectives.mvdirona.com/2017/04/how-many-data-centers-needed-worldwide/

Hölzle, U. [2010]. "Brawny cores still beat wimpy cores, most of the time," *IEEE Micro*, July/August.

Kanev, S., Darago, J.P., Hazelwood, K., Ranganathan, P., Moseley, T., Wei, G.Y. and Brooks, D., 2015, June. Profiling a warehouse-scale computer. *ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*.

Lang, W., J. M. Patel, and S. Shankar [2010]. "Wimpy node clusters: What about non-wimpy workloads?" *Proc. Sixth Int'l. Workshop on Data Management on New Hardware*, June 7, 2010, Indianapolis, Ind.

Lim, K., P. Ranganathan, J. Chang, C. Patel, T. Mudge, and S. Reinhardt [2008]. "Understanding and designing new system architectures for emerging warehouse-computing environments," *Proc. 35th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 21–25, 2008, Beijing, China.

Narayanan, D., E. Thereska, A. Donnelly, S. Elnikety, and A. Rowstron [2009]. "Migrating server storage to SSDs: Analysis of trade-offs," *Proc. 4th ACM European Conf. on Computer Systems*, April 1–3, 2009, Nuremberg, Germany.

Pfister, G. F. [1998]. *In Search of Clusters*, 2nd ed., Prentice Hall, Upper Saddle River, N.J.

Pinheiro, E., W.-D. Weber, and L. A. Barroso [2007]. "Failure trends in a large disk drive population," *Proc. 5th USENIX Conference on File and Storage Technologies (FAST '07)*, February 13–16, 2007, San Jose, Calif.

Ranganathan, P., P. Leech, D. Irwin, and J. Chase [2006]. "Ensemble-level power management for dense blade servers," *Proc. 33rd Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 17–21, 2006, Boston, Mass., 66–77.

Reddi, V. J., B. C. Lee, T. Chilimbi, and K. Vaid [2010]. "Web search using mobile cores: Quantifying and mitigating the price of efficiency," *Proc. 37th Annual Int'l. Symposium on Computer Architecture (ISCA)*, June 19–23, 2010, Saint-Malo, France.

Schroeder, B., and G. A. Gibson [2007]. "Understanding failures in petascale computers," *Journal of Physics: Conference Series 78*, 188–198.

Schroeder, B., E. Pinheiro, and W.-D. Weber [2009]. "DRAM errors in the wild: A large-scale field study," *Proc. Eleventh Int'l. Joint Conf. on Measurement and Modeling of Computer Systems (SIGMETRICS)*, June 15–19, 2009, Seattle, Wash.

Schurman, E. and J. Brutlag [2009]. "The User and Business Impact of Server Delays," *Proc. Velocity: Web Performance and Operations Conf.*, June 22–24, 2009, San Jose, Calif.

Tezzaron Semiconductor. [2004]. "*Soft Errors in Electronic Memory—A White Paper*, Tezzaron Semiconductor, Naperville, Ill. (*http://www.tezzaron.com/about/papers/soft_errors_1_1_secure.pdf*).

Vahdat, A., M. Al-Fares, N. Farrington, R. N. Mysore, G. Porter, and S. Radhakrishnan [2010]. "Scale-out networking in the data center," *IEEE Micro* July/August 2010.

M.9 Historical Perspectives and References

As architects experiment with DSAs, knowing architecture history may help. There are likely older architecture ideas that were unsuccessful for general-purpose computing that could nevertheless make eminent sense for domain-specific architectures. After all, they probably did some things well, and either they might match

your domain, or, conversely, your domain might omit features that were challenges for these architectures. For example, both the Illiac IV (Barnes et al., 1968) from the 1960s and the FPS 120a (Charlesworth, 1981) from the 1970s had two-dimensional arrays of processing elements, so they are proper ancestors to the TPU and Paintbox. Similarly, while VLIW architectures of the Multiflow (Rau and Fisher, 1993) and Itanium (Sharangpani and Arora, 2000) were not commercial successes for general-purpose computing, Paintbox does not have the erratic data cache misses, unpredictable branches, or large code footprint that were difficult for VLIW architectures.

Two survey articles document that custom neural network ASICs go back at least 25 years (Ienne et al., 1996; Asanović, 2002). For example, CNAPS chips contained a 64 SIMD array of 16-bit by 8-bit multipliers, and several CNAPS chips could be connected together with a sequencer (Hammerstrom, 1990). The Synapse-1 system was based on a custom systolic multiply-accumulate chip called the MA-16, which performed sixteen 16-bit multiplications at a time (Ramacher et al., 1991). The system concatenated MA-16 chips and had custom hardware to do activation functions.

Twenty-five SPERT-II workstations, accelerated by the T0 custom ASIC, were deployed starting in 1995 to do both NN training and inference for speech recognition (Asanović et al., 1998). The 40-MHz T0 added vector instructions to the MIPS instruction set architecture. The eight-lane vector unit could produce up to sixteen 32-bit arithmetic results per clock cycle based on 8-bit and 16-bit inputs, making it 25 times faster at inference and 20 times faster at training than a SPARC-20 workstation. They found that 16 bits were insufficient for training, so they used two 16-bit words instead, which doubled training time. To overcome that drawback, they introduced "bunches" (batches) of 32–1000 data sets to reduce time spent updating weights, which made it faster than training with one word but no batches.

We use the phrase *Image Processing Unit* for Paintbox to identify this emerging class of processor, but this is not the first use of the term. The earliest use we can find is 1999, when the Sony Playstation put the name on a chip that was basically an MPEG2 decoder (Sony/Toshiba, 1999). In 2006, Freescale used IPU to name part of the i.MX31 Applications Processor, which is closer to the more generic way we interpret it (Freescale as part of i.MX31 Applications Processor, 2006).

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., 2016. Tensor-flow: large-scale machine learning on heterogeneous distributed systems. arXiv preprint arXiv:1603.04467.

Adolf, R., Rama, S., Reagen, B., Wei, G.Y., Brooks, D., 2016. Fathom: reference workloads for modern deep learning methods. In: IEEE International Symposium on Workload Characterization (IISWC).

Amodei, D., et al., 2015. Deep speech 2: end-to-end speech recognition in English and mandarin, arXiv:1512.02595.

Asanović, K., 2002. Programmable neurocomputing. In: Arbib, M.A. (Ed.), The Handbook of Brain Theory and Neural Networks, second ed. MIT Press, Cambridge, MA. ISBN: 0-262-01197-2. https://people.eecs.berkeley.edu/~krste/papers/neurocomputing.pdf.

Asanović, K., Beck, A., Johnson, J., Wawrzynek, J., Kingsbury, B., Morgan, N., 1998. Training neural networks with Spert-II. In: Sundararajan, N., Saratchandran, P. (Eds.), Parallel Architectures for Artificial Networks: Paradigms and Implementations. IEEE Computer Society Press. ISBN: 0-8186-8399-6. (Chapter 11) https://people.eecs.berkeley.edu/~krste/papers/annbook.pdf.

Bachrach, J., Vo, H., Richards, B., Lee, Y., Waterman, A., Avižienis, R., Wawrzynek, J., Asanović, K., 2012. Chisel: constructing hardware in a Scala embedded language. In: Proceedings of the 49th Annual Design Automation Conference, pp. 1216–1225.

Barnes, G.H., Brown, R.M., Kato, M., Kuck, D.J., Slotnick, D.L., Stokes, R., 1968. The ILLIAC IV computer. IEEE Trans. Comput. 100 (8), 746–757.

Bhattacharya, S., Lane, N.D., 2016. Sparsification and separation of deep learning layers for constrained resource inference on wearables. In: Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM, pp. 176–189.

Brunhaver, J., 2014. PhD thesis. Stanford.

Canis, A., Choi, J., Aldham, M., Zhang, V., Kammoona, A., Czajkowski, T., Brown, S.D., Anderson, J.H., 2013. LegUp: an open-source high-level synthesis tool for FPGA-based processor/accelerator systems. ACM Trans. Embed. Comput. Syst. 13 (2).

Canny, J., et al., 2015. Machine learning at the limit. In: IEEE International Conference on Big Data.

Caulfield, A.M., Chung, E.S., Putnam, A., Haselman, H.A.J.F.M., Humphrey, S.H.M., Daniel, P.K.J.Y.K., Ovtcharov, L.T.M.K., Lanka, M.P.L.W.S., Burger, D.C.D., 2016. A cloud-scale acceleration architecture. In: MICRO Conference.

Charlesworth, A.E., 1981. An approach to scientific array processing: the architectural design of the AP-120B/FPS-164 family. Computer 9, 18–27.

Clark, J., October 26, 2015. Google Turning Its Lucrative Web Search Over to AI Machines. Bloomberg Technology, www.bloomberg.com.

Dally, W.J., 2002. Computer architecture is all about interconnect. In: Proceedings of the 8th International Symposium High Performance Computer Architecture.

Freescale as part of i.MX31 Applications Processor, 2006. http://cache.freescale.com/files/32bit/doc/white_paper/IMX31MULTIWP.pdf.

Galal, S., Shacham, O., Brunhaver II, J.S., Pu, J., Vassiliev, A., Horowitz, M., 2013. FPU generator for design space exploration. In: 21st IEEE Symposium on Computer Arithmetic (ARITH).

Hameed, R., Qadeer, W., Wachs, M., Azizi, O., Solomatnikov, A., Lee, B.C., Richardson, S., Kozyrakis, C., Horowitz, M., 2010. Understanding sources of inefficiency in general-purpose chips. ACM SIGARCH Comput. Architect. News 38 (3), 37–47.

Hammerstrom, D., 1990. A VLSI architecture for high-performance, low-cost, on-chip learning. In: IJCNN International Joint Conference on Neural Networks.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Identity mappings in deep residual networks. Also in arXiv preprint arXiv:1603.05027.

Huang, M., Wu, D., Yu, C.H., Fang, Z., Interlandi, M., Condie, T., Cong, J., 2016. Programming and runtime support to blaze FPGA accelerator deployment at datacenter scale. In: Proceedings of the Seventh ACM Symposium on Cloud Computing. ACM, pp. 456–469.

Iandola, F., 2016. Exploring the Design Space of Deep Convolutional Neural Networks at Large Scale (Ph.D. dissertation). UC Berkeley.

Ienne, P., Cornu, T., Kuhn, G., 1996. Special-purpose digital hardware for neural networks: an architectural survey. J. VLSI Signal Process. Syst. Signal Image Video Technol. 13 (1).

Jouppi, N., 2016. Google supercharges machine learning tasks with TPU custom chip. https://cloudplatform.googleblog.com.

Jouppi, N., Young, C., Patil, N., Patterson, D., Agrawal, G., et al., 2017. Datacenter performance analysis of a matrix processing unit. In: 44th International Symposium on Computer Architecture.

Karpathy, A., et al., 2014. Large-scale video classification with convolutional neural networks. CVPR.

Krizhevsky, A., Sutskever, I., Hinton, G., 2012. Imagenet classification with deep convolutional neural networks. Adv. Neural Inf. Process. Syst.

Kung, H.T., Leiserson, C.E., 1980. Algorithms for VLSI processor arrays. Introduction to VLSI systems.

Lee, Y., Waterman, A., Cook, H., Zimmer, B., Keller, B., Puggelli, A., Kwak, J., Jevtic, R., Bailey, S., Blagojevic, M., Chiu, P.-F., Avizienis, R., Richards, B., Bachrach, J., Patterson, D., Alon, E., Nikolic, B., Asanovic, K., 2016. An agile approach to building RISC-V microprocessors. IEEE Micro 36 (2), 8–20.

Lewis-Kraus, G., 2016. The Great A.I. Awakening. New York Times Magazine.

Nielsen, M., 2016. Neural Networks and Deep Learning. http://neuralnetworksanddeeplearning.com/.

Nvidia, 2016. Tesla GPU Accelerators For Servers. http://www.nvidia.com/object/teslaservers.html.

Olofsson, A., 2011. Debunking the myth of the $100 M ASIC. EE Times. http://www.eetimes.com/author.asp?section_id=36&doc_id=1266014.

Ovtcharov, K., Ruwase, O., Kim, J.Y., Fowers, J., Strauss, K., Chung, E.S., 2015a. Accelerating deep convolutional neural networks using specialized hardware. Microsoft Research Whitepaper. https://www.microsoft.com/en-us/research/publication/accelerating-deepconvolutional-neural-networks-using-specialized-hardware/.

Ovtcharov, K., Ruwase, O., Kim, J.Y., Fowers, J., Strauss, K., Chung, E.S., 2015b. Toward accelerating deep learning at scale using specialized hardware in the datacenter. In: 2015 IEEE Hot Chips 27 Symposium.

Patterson, D., Nikolić, B., 7/25/2015, Agile Design for Hardware, Parts I, II, and III. EE Times, http://www.eetimes.com/author.asp?doc_id=1327239.

Patterson, D.A., Ditzel, D.R., 1980. The case for the reduced instruction set computer. ACM SIGARCH Comput. Architect. News 8 (6), 25–33.

Prabhakar, R., Koeplinger, D., Brown, K.J., Lee, H., De Sa, C., Kozyrakis, C., Olukotun, K., 2016. Generating configurable hardware from parallel patterns. In: Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, pp. 651–665.

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2014. A reconfigurable fabric for accelerating large-scale datacenter services. In: 41st International Symposium on Computer Architecture.

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2015. A reconfigurable fabric for accelerating large-scale datacenter services. IEEE Micro. 35 (3).

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2016. A reconfigurable fabric for accelerating large-scale datacenter services. Commun. ACM.

Qadeer, W., Hameed, R., Shacham, O., Venkatesan, P., Kozyrakis, C., Horowitz, M.A., 2015. Convolution engine: balancing efficiency & flexibility in specialized computing. Commun. ACM 58 (4).

Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., Amarasinghe, S., 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. ACM SIGPLAN Not. 48 (6), 519–530.

Ramacher, U., Beichter, J., Raab, W., Anlauf, J., Bruels, N., Hachmann, A., Wesseling, M., 1991. Design of a 1st generation neurocomputer. VLSI Design of Neural Networks. Springer, USA.

Rau, B.R., Fisher, J.A., 1993. Instruction-level parallelism. J. Supercomput. 235, Springer Science & Business Media.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., 2015. Imagenet large scale visual recognition challenge. Int. J. Comput. Vis. 115 (3).

Sergio Guadarrama, 2015. BVLC googlenet. https://github.com/BVLC/caffe/tree/master/models/bvlc_googlenet.

Shao, Y.S., Brooks, D., 2015. Research infrastructures for hardware accelerators. Synth. Lect. Comput. Architect. 10 (4), 1–99.

Sharangpani, H., Arora, K., 2000. Itanium processor microarchitecture. IEEE Micro 20 (5), 24–43.

Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., 2016. Mastering the game of Go with deep neural networks and tree search. Nature 529 (7587).

Smith, J.E., 1982. Decoupled access/execute computer architectures. In: Proceedings of the 11th International Symposium on Computer Architecture.

Sony/Toshiba, 1999. 'Emotion Engine' in PS2 ("IPU is basically an MPEG2 decoder…"). http://www.cpu-collection.de/?l0=co&l1=Sony&l2=Emotion+Engine, http://arstechnica.com/gadgets/2000/02/ee/3/.

Steinberg, D., 2015. Full-Chip Simulations, Keys to Success. In: Proceedings of the Synopsys Users Group (SNUG) Silicon Valley 2015.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A., 2015. Going deeper with convolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

TensorFlow Tutorials, 2016. https://www.tensorflow.org/versions/r0.12/tutorials/index.html.

Tung, L., 2016. Google Translate: 'This landmark update is our biggest single leap in 10 years', ZDNet. http://www.zdnet.com/article/google-translate-this-landmarkupdate-is-our-biggest-single-leap-in-10years/.

Vanhoucke, V., Senior, A., Mao, M.Z., 2011. Improving the speed of neural networks on CPUs. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/37631.pdf.

Wu, Y., Schuster, M., Chen, Z., Le, Q., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Kaiser, Ł., Gouws, S., Kato, Y., Kudo, T., Kazawa, H., Stevens, K., Kurian, G., Patil, N., Wang, W., Young, C., Smith, J., Riesa, J., Rudnick, A., Vinyals, O., Corrado, G., Hughes, M., Dean, J., 2016. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. http://arxiv.org/abs/1609.08144.

M.10 The History of Magnetic Storage, RAID, and I/O Buses (Appendix D)

Mass storage is a term used there to imply a unit capacity in excess of one million alphanumeric characters …

Hoagland [1963]

The variety of storage I/O and issues leads to a varied history for the rest of the story. (Smotherman [1989] explored the history of I/O in more depth.) This section discusses magnetic storage, RAID, and I/O buses and controllers. Jain [1991] and Lazowska et al. [1984] are books for those interested in learning more about queuing theory.

Magnetic Storage

Magnetic recording was invented to record sound, and by 1941 magnetic tape was able to compete with other storage devices. It was the success of the ENIAC in 1947 that led to the push to use tapes to record digital information. Reels of magnetic tapes dominated removable storage through the 1970s. In the 1980s, the IBM 3480 cartridge became the *de facto* standard, at least for mainframes. It can transfer at 3 MB/sec by reading 18 tracks in parallel. The capacity is just 200 MB for this 1/2-inch tape. The 9840 cartridge, used by StorageTek in the Powder-Horn, transfers at 10 MB/sec and stores 20,000 MB. This device records the tracks in a zigzag fashion rather than just longitudinally, so that the head reverses direction to follow the track. This technique is called *serpentine recording*. Another 1/2-inch tape is Digital Linear Tape; the DLT7000 stores 35,000 MB and transfers at 5 MB/sec. Its competitor is helical scan, which rotates the head to get the increased recording density. In 2001, the 8-mm helical-scan tapes contain 20,000 MB and transfer at about 3 MB/sec. Whatever their density and cost, the serial nature of tapes creates an appetite for storage devices with random access.

In 1953, Reynold B. Johnson of IBM picked a staff of 15 scientists with the goal of building a radically faster random access storage system than tape. The goal was to have the storage equivalent of 50,000 standard IBM punch cards and to fetch the data in a single second. Johnson's disk drive design was simple but untried: The magnetic read/write sensors would have to float a few thousandths of an inch above the continuously rotating disk. Twenty-four months later the team emerged with the functional prototype. It weighed 1 ton and occupied about 300 cubic feet of space. The RAMAC-350 (Random Access Method of Accounting Control) used 50 platters that were 24 inches in diameter, rotated at 1200 RPM, with a total capacity of 5 MB and an access time of 1 second.

Starting with the RAMAC, IBM maintained its leadership in the disk industry, with its storage headquarters in San Jose, California, where Johnson's team did its work. Many of the future leaders of competing disk manufacturers started their careers at IBM, and many disk companies are located near San Jose.

Although RAMAC contained the first disk, a major breakthrough in magnetic recording was found in later disks with air-bearing read/write heads, where the head would ride on a cushion of air created by the fast-moving disk surface. This cushion meant the head could both follow imperfections in the surface and yet be very close to the surface. Subsequent advances have come largely from improved quality of components and higher precision. In 2001, heads flew 2 to 3 microinches above the surface, whereas in the RAMAC drive they were 1000 microinches away.

Moving-head disks quickly became the dominant high-speed magnetic storage, although their high cost meant that magnetic tape continued to be used extensively until the 1970s. The next important development for hard disks was the removable hard disk drive developed by IBM in 1962; this made it possible to share the expensive drive electronics and helped disks overtake tapes as the preferred storage medium. The IBM 1311 disk in 1962 had an areal density of 50,000

bits per square inch and a cost of about $800 per megabyte. IBM also invented the floppy disk drive in 1970, originally to hold microcode for the IBM 370 series. Floppy disks became popular with the PC about 10 years later.

The second major disk breakthrough was the so-called Winchester disk design in about 1973. Winchester disks benefited from two related properties. First, integrated circuits lowered the costs of not only CPUs but also of disk controllers and the electronics to control disk arms. Reductions in the cost of the disk electronics made it unnecessary to share the electronics and thus made nonremovable disks economical. Since the disk was fixed and could be in a sealed enclosure, both the environmental and control problems were greatly reduced. Sealing the system allowed the heads to fly closer to the surface, which in turn enabled increases in areal density. The first sealed disk that IBM shipped had two spindles, each with a 30 MB disk; the moniker "30-30" for the disk led to the name Winchester. (America's most popular sporting rifle, the Winchester 94, was nicknamed the "30-30" after the caliber of its cartridge.) Winchester disks grew rapidly in popularity in the 1980s, completely replacing removable disks by the middle of that decade. Before this time, the cost of the electronics to control the disk meant that the media had to be removable.

As mentioned in Appendix D, as DRAMs started to close the areal density gap and appeared to be catching up with disk storage, internal meetings at IBM called into question the future of disk drives. Disk designers concluded that disks must improve at 60% per year to forestall the DRAM threat, in contrast to the historical 29% per year. The essential enabler was magnetoresistive heads, with giant magnetoresistive heads enabling the current densities. Because of this competition, the gap in time between when a density record is achieved in the lab and when a disk is shipped with that density has closed considerably.

The personal computer created a market for small form factor (SFF) disk drives, since the 14-inch disk drives used in mainframes were bigger than the PC. In 2006, the 3.5-inch drive was the market leader, although the smaller 2.5-inch drive required for laptop computers was significant in sales volume. It remains to be seen whether handheld devices such as iPODs or video cameras, which require even smaller disks, will remain significant in sales volume. For example, 1.8-inch drives were developed in the early 1990s for palmtop computers, but that market chose Flash instead and 1.8-inch drives disappeared.

RAID

The SFF hard disks for PCs in the 1980s led a group at Berkeley to propose redundant arrays of inexpensive disks (RAID). This group had worked on the reduced instruction set computer effort and so expected much faster CPUs to become available. They asked: What could be done with the small disks that accompanied their PCs? and What could be done in the area of I/O to keep up with much faster processors? They argued to replace one mainframe drive with 50 small drives to gain much greater performance from that many independent arms. The many small

drives even offered savings in power consumption and floor space. The downside of many disks was much lower mean time to failure (MTTF). Hence, on their own they reasoned out the advantages of redundant disks and rotating parity to address how to get greater performance with many small drives yet have reliability as high as that of a single mainframe disk.

The problem they experienced when explaining their ideas was that some researchers had heard of disk arrays with some form of redundancy, and they didn't understand the Berkeley proposal. Hence, the first RAID paper [Patterson, Gibson, and Katz 1987] is not only a case for arrays of SFF disk drives but also something of a tutorial and classification of existing work on disk arrays. Mirroring (RAID 1) had long been used in fault-tolerant computers such as those sold by Tandem. Thinking Machines had arrays with 32 data disks and 7 check disks using ECC for correction (RAID 2) in 1987, and Honeywell Bull had a RAID 2 product even earlier. Also, disk arrays with a single parity disk had been used in scientific computers in the same time frame (RAID 3). Their paper then described a single parity disk with support for sector accesses (RAID 4) and rotated parity (RAID 5). Chen et al. [1994] surveyed the original RAID ideas, commercial products, and more recent developments.

Unknown to the Berkeley group, engineers at IBM working on the AS/400 computer also came up with rotated parity to give greater reliability for a collection of large disks. IBM filed a patent on RAID 5 before the Berkeley group wrote their paper. Patents for RAID 1, RAID 2, and RAID 3 from several companies predate the IBM RAID 5 patent, which has led to plenty of courtroom action.

The Berkeley paper was written before the World Wide Web, but it captured the imagination of many engineers, as copies were faxed around the world. One engineer at what is now Seagate received seven copies of the paper from friends and customers. EMC had been a supplier of DRAM boards for IBM computers, but around 1988 new policies from IBM made it nearly impossible for EMC to continue to sell IBM memory boards. Apparently, the Berkeley paper also crossed the desks of EMC executives, and they decided to go after the market dominated by IBM disk storage products instead. As the paper advocated, their model was to use many small drives to compete with mainframe drives, and EMC announced a RAID product in 1990. It relied on mirroring (RAID 1) for reliability; RAID 5 products came much later for EMC. Over the next year, Micropolis offered a RAID 3 product, Compaq offered a RAID 4 product, and Data General, IBM, and NCR offered RAID 5 products.

The RAID ideas soon spread to the rest of the workstation and server industry. An article explaining RAID in *Byte* magazine (see Anderson [1990]) led to RAID products being offered on desktop PCs, which was something of a surprise to the Berkeley group. They had focused on performance with good availability, but higher availability was attractive to the PC market.

Another surprise was the cost of the disk arrays. With redundant power supplies and fans, the ability to "hot swap" a disk drive, the RAID hardware controller itself, the redundant disks, and so on, the first disk arrays cost many times the cost of the disks. Perhaps as a result, the "inexpensive" in RAID morphed into

"independent." Many marketing departments and technical writers today know of RAID only as "redundant arrays of independent disks."

The EMC transformation was successful; in 2006, EMC was the leading supplier of storage systems, and NetApp was the leading supplier of Network-Attached Storage systems. RAID was a $30 billion industry in 2006, and more than 80% of the non-PC drive sales were found in RAIDs. In recognition of their role, in 1999 Garth Gibson, Randy Katz, and David Patterson received the IEEE Reynold B. Johnson Information Storage Award "for the development of Redundant Arrays of Inexpensive Disks (RAID)."

I/O Buses and Controllers

The ubiquitous microprocessor inspired not only the personal computers of the 1970s but also the trend in the late 1980s and 1990s of moving controller functions into I/O devices. I/O devices have continued this trend by moving controllers into the devices themselves. These devices are called *intelligent devices*, and some bus standards (e.g., SCSI) have been created specifically for them. Intelligent devices can relax the timing constraints by handling many low-level tasks themselves and queuing the results. For example, many SCSI-compatible disk drives include a track buffer on the disk itself, supporting read ahead and connect/disconnect. Thus, on a SCSI string some disks can be seeking and others loading their track buffer while one is transferring data from its buffer over the SCSI bus. The controller in the original RAMAC, built from vacuum tubes, only needed to move the head over the desired track, wait for the data to pass under the head, and transfer data with calculated parity. SCSI, which stands for *small computer systems interface*, is an example of one company inventing a bus and generously encouraging other companies to build devices that would plug into it. Shugart created this bus, originally called SASI. It was later standardized by the IEEE.

There have been several candidates to be the successor to SCSI, with the current leading contender being Fibre Channel Arbitrated Loop (FC-AL). The SCSI committee continues to increase the clock rate of the bus, giving this standard a new life, and SCSI is lasting much longer than some of its proposed successors. With the creation of serial interfaces for SCSI ("Serial Attach SCSI") and ATA ("Serial ATA"), they may have very long lives.

Perhaps the first multivendor bus was the PDP-11 Unibus in 1970 from DEC. Alas, this open-door policy on buses is in contrast to companies with proprietary buses using patented interfaces, thereby preventing competition from plug-compatible vendors. Making a bus proprietary also raises costs and lowers the number of available I/O devices that plug into it, since such devices must have an interface designed just for that bus. The PCI bus pushed by Intel represented a return to open, standard I/O buses inside computers. Its immediate successor is PCI-X, with Infiniband under development in 2000. Both were standardized by multicompany trade associations.

The machines of the RAMAC era gave us I/O interrupts as well as storage devices. The first machine to extend interrupts from detecting arithmetic abnormalities to detecting asynchronous I/O events is credited as the NBS DYSEAC in 1954 [Leiner and Alexander 1954]. The following year, the first machine with DMA was operational, the IBM SAGE. Just as today's DMA has, the SAGE had address counters that performed block transfers in parallel with CPU operations.

The early IBM 360s pioneered many of the ideas that we use in I/O systems today. The 360 was the first commercial machine to make heavy use of DMA, and it introduced the notion of I/O programs that could be interpreted by the device. Chaining of I/O programs was an important feature. The concept of channels introduced in the 360 corresponds to the I/O bus of today.

Myer and Sutherland [1968] wrote a classic paper on the trade-off of complexity and performance in I/O controllers. Borrowing the religious concept of the "wheel of reincarnation," they eventually noticed they were caught in a loop of continuously increasing the power of an I/O processor until it needed its own simpler coprocessor. Their quote in Appendix D captures their cautionary tale.

The IBM mainframe I/O channels, with their I/O processors, can be thought of as an inspiration for Infiniband, with their processors on their Host Channel Adaptor cards.

References

Anderson, D. [2003]. "You don't know jack about disks," *Queue* 1:4 (June), 20–30.

Anderson, D., J. Dykes, and E. Riedel [2003]. "SCSI vs. ATA—more than an interface," *Proc. 2nd USENIX Conf. on File and Storage Technology (FAST '03)*, March 31–April 2, 2003, San Francisco.

Anderson, M. H. [1990]. "Strength (and safety) in numbers (RAID, disk storage technology)," *Byte* 15:13 (December), 337–339.

Anon. et al. [1985]. *A Measure of Transaction Processing Power*, Tandem Tech. Rep. TR 85.2. Also appeared in *Datamation*, 31:7 (April), 112–118.

Bashe, C. J., W. Buchholz, G. V. Hawkins, J. L. Ingram, and N. Rochester [1981]. "The architecture of IBM's early computers," *IBM J. Research and Development* 25:5 (September), 363–375.

Bashe, C. J., L. R. Johnson, J. H. Palmer, and E. W. Pugh [1986]. *IBM's Early Computers*, MIT Press, Cambridge, Mass.

Blaum, M., J. Brady, J. Bruck, and J. Menon [1994]. "EVENODD: An optimal scheme for tolerating double disk failures in RAID architectures," *Proc. 21st Annual Int'l. Symposium on Computer Architecture (ISCA)*, April 18–21, 1994, Chicago, 245–254.

Blaum, M., J. Brady, J. Bruck, and J. Menon [1995]. "EVENODD: An optimal scheme for tolerating double disk failures in RAID architectures," *IEEE Trans. on Computers* 44:2 (February), 192–202.

Blaum, M., J. Brady, J., Bruck, J. Menon, and A. Vardy [2001]. "The EVENODD code and its generalization," in H. Jin, T. Cortes, and R. Buyya, eds., *High Performance Mass Storage and Parallel I/O: Technologies and Applications*, IEEE & Wiley Press, New York, 187–208.

Blaum, M., J. Bruck, and A. Vardy [1996]. "MDS array codes with independent parity symbols," *IEEE Trans. on Information Theory*, IT-42 (March), 529–542.

Brady, J. T. [1986]. "A theory of productivity in the creative process," *IEEE CG&A* (May), 25–34.

Brown, A., and D. A. Patterson [2000]. "Towards maintainability, availability, and growth benchmarks: A case study of software RAID systems." *Proc. 2000 USENIX Annual Technical Conf.*, June 18–23, San Diego, Calif.

Bucher, I. V., and A. H. Hayes [1980]. "I/O performance measurement on Cray-1 and CDC 7000 computers," *Proc. Computer Performance Evaluation Users Group, 16th Meeting*, October 20–23, 1980, Orlando, Fl., 245–254.

Chen, P. M., G. A. Gibson, R. H. Katz, and D. A. Patterson [1990]. "An evaluation of redundant arrays of inexpensive disks using an Amdahl 5890," *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, May 22–25, 1990, Boulder, Colo.

Chen, P. M., and E. K. Lee [1995]. "Striping in a RAID level 5 disk array," *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, May 15–19, 1995, Ottawa, Canada, 136–145.

Chen, P. M., E. K. Lee, G. A. Gibson, R. H. Katz, and D. A. Patterson [1994]. "RAID: High-performance, reliable secondary storage," *ACM Computing Surveys* 26:2 (June), 145–188.

Corbett, P., B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, and S. Sankar [2004]. "Row-diagonal parity for double disk failure correction," *Proc. 3rd USENIX Conf. on File and Storage Technology (FAST '04)*, March 31–April 2, 2004, San Francisco.

Denehy, T. E., J. Bent, F. I. Popovici, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau [2004]. "Deconstructing storage arrays," *Proc. 11th Int'l. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, October 7–13, 2004, Boston, Mass., 59–71.

Doherty, W. J., and R. P. Kelisky [1979]. "Managing VM/CMS systems for user effectiveness," *IBM Systems J.* 18:1, 143–166.

Douceur, J. R., and W. J. Bolosky [1999]. "A large scale study of file-system contents," *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, May 1–9, 1999, Atlanta, Ga., 59–69.

Enriquez, P. [2001]. "What happened to my dial tone? A study of FCC service disruption reports," poster, *Richard Tapia Symposium on the Celebration of Diversity in Computing*, October 18–20, 2001, Houston, Tex.

Friesenborg, S. E., and R. J. Wicks [1985]. *DASD Expectations: The 3380, 3380-23, and MVS/XA*, Tech. Bulletin GG22-9363-02, IBM Washington Systems Center, Gaithersburg, Md.

Gibson, G. A. [1992]. *Redundant Disk Arrays: Reliable, Parallel Secondary Storage*, ACM Distinguished Dissertation Series, MIT Press, Cambridge, Mass.

Goldstein, S. [1987]. *Storage Performance—An Eight Year Outlook*, Tech. Rep. TR 03.308-1, IBM Santa Teresa Laboratory, San Jose, Calif.

Gray, J. [1990]. "A census of Tandem system availability between 1985 and 1990," *IEEE Trans. on Reliability*, 39:4 (October), 409–418.

Gray, J. (ed.) [1993]. *The Benchmark Handbook for Database and Transaction Processing Systems*, 2nd ed., Morgan Kaufmann, San Francisco.

Gray, J., and A. Reuter [1993]. *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, San Francisco.

Gray, J., and D. P. Siewiorek [1991]. "High-availability computer systems." *Computer* 24:9 (September), 39–48.

Gray, J., and C. van Ingen [2005]. *Empirical Measurements of Disk Failure Rates and Error Rates*," MSR-TR-2005-166, Microsoft Research, Redmond, Wash.

Gurumurthi, S., A. Sivasubramaniam, and V. Natarajan [2005]. Disk Drive Roadmap from the Thermal Perspective: A Case for Dynamic Thermal Management, *Proceedings of the International Symposium on Computer Architecture (ISCA)*, June, 38–49.

Henly, M., and B. McNutt [1989]. *DASD I/O Characteristics: A Comparison of MVS to VM*, Tech. Rep. TR 02.1550, IBM General Products Division, San Jose, Calif.

Hewlett-Packard. [1998]. "HP's '5NINES:5MINUTES' vision extends leadership and re-defines high availability in mission-critical environments," February 10, *www.future.enterprisecomputing.hp.com/ia64/news/5nines_vision_pr.html*.

Hoagland, A. S. [1963]. *Digital Magnetic Recording*, Wiley, New York.

Hospodor, A. D., and A. S. Hoagland [1993]. "The changing nature of disk controllers." *Proc. IEEE* 81:4 (April), 586–594.

IBM. [1982]. *The Economic Value of Rapid Response Time*, GE20-0752-0, IBM, White Plains, N.Y., 11–82.

Imprimis. [1989]. *Imprimis Product Specification, 97209 Sabre Disk Drive IPI-2 Interface 1.2 GB*, Document No. 64402302, Imprimis, Dallas, Tex.

Jain, R. [1991]. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, Wiley, New York.

Katz, R. H., D. A. Patterson, and G. A. Gibson [1989]. "Disk system architectures for high performance computing," *Proc. IEEE* 77:12 (December), 1842–1858.

Kim, M. Y. [1986]. "Synchronized disk interleaving," *IEEE Trans. on Computers* C-35:11 (November), 978–988.

Kuhn, D. R. [1997]. "Sources of failure in the public switched telephone network," *IEEE Computer* 30:4 (April), 31–36.

Lambright, D. [2000]. "Experiences in measuring the reliability of a cache-based storage system," *Proc. of First Workshop on Industrial Experiences with Systems Software (WIESS 2000), Co-Located with the 4th Symposium on Operating Systems Design and Implementation (OSDI)*, October 22, 2000, San Diego, Calif.

Laprie, J.-C. [1985]. "Dependable computing and fault tolerance: Concepts and terminology," *Proc. 15th Annual Int'l. Symposium on Fault-Tolerant Computing*, June 19–21, 1985, Ann Arbor, Mich., 2–11.

Lazowska, E. D., J. Zahorjan, G. S. Graham, and K. C. Sevcik [1984]. *Quantitative System Performance: Computer System Analysis Using Queueing Network Models*, Prentice Hall, Englewood Cliffs, N.J. (Although out of print, it is available online at *www.cs.washington.edu/homes/lazowska/qsp/.)*

Leiner, A. L. [1954]. "System specifications for the DYSEAC," *J. ACM* 1:2 (April), 57–81.

Leiner, A. L., and S. N. Alexander [1954]. "System organization of the DYSEAC," *IRE Trans. of Electronic Computers* EC-3:1 (March), 1–10.

Maberly, N. C. [1966]. *Mastering Speed Reading*, New American Library, New York.

Major, J. B. [1989]. "Are queuing models within the grasp of the unwashed?" *Proc. Int'l. Conf. on Management and Performance Evaluation of Computer Systems*, December 11–15, 1989, Reno, Nev., 831–839.

Mueller, M., L. C. Alves, W. Fischer, M. L. Fair, and I. Modi [1999]. "RAS strategy for IBM S/390 G5 and G6," *IBM J. Research and Development*, 43:5–6 (September–November), 875–888.

Murphy, B., and T. Gent [1995]. "Measuring system and software reliability using an automated data collection process," *Quality and Reliability Engineering International*, 11:5 (September–October), 341–353.

Myer, T. H., and I. E. Sutherland [1968]. "On the design of display processors," *Communications of the ACM*, 11:6 (June), 410–414.

National Storage Industry Consortium. [1998]. "Tape Roadmap," *www.nsic.org*.

Nelson, V. P. [1990]. "Fault-tolerant computing: Fundamental concepts," *Computer* 23:7 (July), 19–25.

Nyberg, C. R., T. Barclay, Z. Cvetanovic, J. Gray, and D. Lomet [1994]. "AlphaSort: A RISC machine sort," *Proc. ACM SIGMOD*, May 24–27, 1994, Minneapolis, Minn.

Okada, S., S. Okada, Y. Matsuda, T. Yamada, and A. Kobayashi [1999]. "System on a chip for digital still camera," *IEEE Trans. on Consumer Electronics* 45:3 (August), 584–590.

Patterson, D. A., G. A. Gibson, and R. H. Katz [1987]. *A Case for Redundant Arrays of Inexpensive Disks (RAID)*, Tech. Rep. UCB/CSD 87/391, University of California, Berkeley. Also appeared in *Proc. ACM SIGMOD*, June 1–3, 1988, Chicago, 109–116.

Pavan, P., R. Bez, P. Olivo, and E. Zanoni [1997]. "Flash memory cells—an overview," *Proc. IEEE* 85:8 (August), 1248–1271.

Robinson, B., and L. Blount [1986]. *The VM/HPO 3880-23 Performance Results*, IBM Tech. Bulletin GG66-0247-00, IBM Washington Systems Center, Gaithersburg, Md.

Salem, K., and H. Garcia-Molina [1986]. "Disk striping," *Proc. 2nd Int'l. IEEE Conf. on Data Engineering*, February 5–7, 1986, Washington, D.C., 249–259.

Scranton, R. A., D. A. Thompson, and D. W. Hunter [1983]. *The Access Time Myth*, Tech. Rep. RC 10197 (45223), IBM, Yorktown Heights, N.Y.

Seagate. [2000]. *Seagate Cheetah 73 Family: ST173404LW/LWV/LC/LCV Product Manual*, Vol. 1, Seagate, Scotts Valley, Calif. (*www.seagate.com/support/disc/manuals/scsi/29478b.pdf*).

Smotherman, M. [1989]. "A sequencing-based taxonomy of I/O systems and review of historical machines," *Computer Architecture News* 17:5 (September), 5–15. Reprinted in *Computer Architecture Readings*, M. D. Hill, N. P. Jouppi, and G. S. Sohi, eds., Morgan Kaufmann, San Francisco, 1999, 451–461.

Talagala, N. [2000]. "Characterizing Large Storage Systems: Error Behavior and Performance Benchmarks," Ph.D. dissertation, Computer Science Division, University of California, Berkeley.

Talagala, N., and D. Patterson [1999]. *An Analysis of Error Behavior in a Large Storage System*, Tech. Report UCB//CSD-99-1042, Computer Science Division, University of California, Berkeley.

Talagala, N., R. Arpaci-Dusseau, and D. Patterson [2000]. *Micro-Benchmark Based Extraction of Local and Global Disk Characteristics*, CSD-99-1063, Computer Science Division, University of California, Berkeley.

Talagala, N., S. Asami, D. Patterson, R. Futernick, and D. Hart [2000]. "The art of massive storage: A case study of a Web image archive," *IEEE Computer* (November), 22–28.

Thadhani, A. J. [1981]. "Interactive user productivity," *IBM Systems J.* 20:4, 407–423.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., 2016. TensorFlow: A System for Large-Scale Machine Learning. In: OSDI (November), vol. 16, pp. 265–283.

Adolf, R., Rama, S., Reagen, B., Wei, G.Y., Brooks, D., 2016. Fathom: reference workloads for modern deep learning methods. In: IEEE International Symposium on Workload Characterization (IISWC).

Adve, S.V., Gharachorloo, K., 1996. Shared memory consistency models: a tutorial. IEEE Comput. 29 (12), 66–76.

Adve, S.V., Hill, M.D., 1990. Weak ordering: a new definition. In: Proceedings of 17th Annual International Symposium on Computer Architecture (ISCA), May 28–31, 1990, Seattle, Washington, pp. 2–14.

Agarwal, A., 1987. Analysis of Cache Performance for Operating Systems and Multiprogramming (Ph.D. thesis). Tech. Rep. No. CSL-TR-87-332. Stanford University, Palo Alto, CA.

Agarwal, A., 1991. Limits on interconnection network performance. IEEE Trans. Parallel Distrib. Syst. 2 (4), 398–412.

Agarwal, A., Pudar, S.D., 1993. Column-associative caches: a technique for reducing the miss rate of direct-mapped caches. In: 20th Annual International Symposium on Computer Architecture (ISCA), May 16–19, 1993, San Diego, California. Also appears in Computer Architecture News 21:2 (May), 179–190, 1993.

Agarwal, A., Hennessy, J.L., Simoni, R., Horowitz, M.A., 1988. An evaluation of directory schemes for cache coherence. In: Proceedings of 15th International Symposium on Computer Architecture (June), pp. 280–289.

Agarwal, A., Kubiatowicz, J., Kranz, D., Lim, B.-H., Yeung, D., D'Souza, G., Parkin, M., 1993. Sparcle: an evolutionary processor design for large-scale multiprocessors. IEEE Micro 13, 48–61.

Agarwal, A., Bianchini, R., Chaiken, D., Johnson, K., Kranz, D., 1995. The MIT Alewife machine: architecture and performance. In: International Symposium on Computer Architecture (Denver, CO), June, 2–13.

Agerwala, T., Cocke, J., 1987. High Performance Reduced Instruction Set Processors. IBM Tech. Rep. RC12434, IBM, Armonk, NY.

Akeley, K., Jermoluk, T., 1988. High-performance polygon rendering. In: Proceedings of 15th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH 1988), August 1–5, 1988, Atlanta, GA, pp. 239–246.

Alexander, W.G., Wortman, D.B., 1975. Static and dynamic characteristics of XPL programs. IEEE Comput. 8 (11), 41–46.

Alles, A., 1995. ATM Internetworking. White Paper (May). Cisco Systems, Inc., San Jose, CA. www.cisco.com/warp/public/614/12.html.

Alliant, 1987. Alliant FX/Series: Product Summary. Alliant Computer Systems Corp, Acton, MA.

Almasi, G.S., Gottlieb, A., 1989. Highly Parallel Computing. Benjamin/Cummings, Redwood City, CA.

Alverson, G., Alverson, R., Callahan, D., Koblenz, B., Porterfield, A., Smith, B., 1992. Exploiting heterogeneous parallelism on a multithreaded multiprocessor. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 16–20, 1992, Minneapolis, MN, pp. 188–197.

Amdahl, G.M., 1967. Validity of the single processor approach to achieving large scale computing capabilities. In: Proceedings of AFIPS Spring Joint Computer Conference, April 18–20, 1967, Atlantic City, NJ, pp. 483–485.

Amdahl, G.M., Blaauw, G.A., Brooks Jr., F.P., 1964. Architecture of the IBM System 360. IBM J. Res. Dev. 8 (2), 87–101.

Amodei, D., Ananthanarayanan, S., Anubhai, R., Bai, J., Battenberg, E., Case, C., Casper, J., Catanzaro, B., Cheng, Q., Chen, G., Chen, J., 2016. Deep speech 2: End-to-end speech recognition in english and mandarin. In: International Conference on Machine Learning (June), pp. 173–182.

Amza, C., Cox, A.L., Dwarkadas, S., Keleher, P., Lu, H., Rajamony, R., Yu, W., Zwaenepoel, W., 1996. Treadmarks: shared memory computing on networks of workstations. IEEE Comput. 29 (2), 18–28.

Anderson, M.H., 1990. Strength (and safety) in numbers (RAID, disk storage technology). Byte 15 (13), 337–339.

Anderson, D., 2003. You don't know jack about disks. Queue 1 (4), 20–30.

Anderson, D.W., Sparacio, F.J., Tomasulo, R.M., 1967. The IBM 360 Model 91: processor philosophy and instruction handling. IBM J. Res. Dev. 11 (1), 8–24.

Anderson, T.E., Culler, D.E., Patterson, D., 1995. A case for NOW (networks of workstations). IEEE Micro 15 (1), 54–64.

Anderson, D., Dykes, J., Riedel, E., 2003. SCSI vs. ATA—more than an interface. In: Proceedings of 2nd USENIX Conference on File and Storage Technology (FAST'03), March 31–April 2.

Ang, B., Chiou, D., Rosenband, D., Ehrlich, M., Rudolph, L., Arvind, A., 1998. StarT-Voyager: a flexible platform for exploring scalable SMP issues. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 7–13, 1998, Orlando, FL.

Anjan, K.V., Pinkston, T.M., 1995. An efficient, fully-adaptive deadlock recovery scheme: Disha. In: Proceedings of 22nd Annual International Symposium on Computer Architecture (ISCA), June 22–24, 1995, Santa Margherita, Italy.

Anon. et al., 1985. A Measure of Transaction Processing Power. Tandem Tech. Rep. TR85.2. Also appears in *Datamation* 31:7 (April), 112–118, 1985.

Apache Hadoop, 2011. http://hadoop.apache.org.

Archibald, J., Baer, J.-L., 1986. Cache coherence protocols: evaluation using a multiprocessor simulation model. ACM Trans. Comput. Syst. 4 (4), 273–298.

Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., Zaharia, M., 2009. Above the Clouds: A Berkeley View of Cloud Computing, Tech. Rep. UCB/EECS-2009-28, University of California, Berkeley. http://www.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.html.

Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., Zaharia, M., 2010. A view of cloud computing. Commun. ACM. 53 (4), 50–58.

Arpaci, R.H., Culler, D.E., Krishnamurthy, A., Steinberg, S.G., Yelick, K., 1995. Empirical evaluation of the CRAY-T3D: a compiler perspective. In: 22nd Annual International Symposium on Computer Architecture (ISCA), June 22–24, 1995, Santa Margherita, Italy.

Asanovic, K., 1998. Vector Microprocessors (Ph.D. thesis). Computer Science Division, University of California, Berkeley.

Asanović, K., 2002. Programmable neurocomputing. In: Arbib, M.A. (Ed.), The Handbook of Brain Theory and Neural Networks, second ed. MIT Press, Cambridge, MA. ISBN: 0-262-01197-2. https://people.eecs.berkeley.edu/~krste/papers/neurocomputing.pdf.

Asanović, K., Beck, A., Johnson, J., Wawrzynek, J., Kingsbury, B., Morgan, N., 1998. Training neural networks with Spert-II. In: Sundararajan, N., Saratchandran, P. (Eds.), Parallel Architectures for Artificial Networks: Paradigms and Implementations. IEEE Computer Society Press, California, USA. ISBN: 0-8186-8399-6. (Chapter 11) https://people.eecs.berkeley.edu/∼krste/papers/annbook.pdf.

Associated Press, 2005. Gap Inc. shuts down two Internet stores for major overhaul. USA-TODAY.com, August 8, 2005.

Atanasoff, J.V., 1940. Computing Machine for the Solution of Large Systems of Linear Equations. Internal Report. Iowa State University, Ames.

Atkins, M., 1991. Performance and the i860 microprocessor. IEEE Micro 11 (5), 24–27. 72–78.

Austin, T.M., Sohi, G., 1992. Dynamic dependency analysis of ordinary programs. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 342–351.

Azizi, O., Mahesri, A., Lee, B.C., Patel, S.J., Horowitz, M., 2010. Energy-performance tradeoffs in processor architecture and circuit design: a marginal cost analysis. In: Proceedings of the International Symposium on Computer Architecture, pp. 26–36.

Babbay, F., Mendelson, A., 1998. Using value prediction to increase the power of speculative execution hardware. ACM Trans. Comput. Syst. 16 (3), 234–270.

Bachrach, J., Vo, H., Richards, B., Lee, Y., Waterman, A., Avižienis, R., Wawrzynek, J., Asanović, K., 2012. Chisel: constructing hardware in a Scala embedded language. In: Proceedings of the 49th Annual Design Automation Conference, pp. 1216–1225.

Baer, J.-L., Wang, W.-H., 1988. On the inclusion property for multi-level cache hierarchies. In: Proceedings of 15th Annual International Symposium on Computer Architecture, May 30–June 2, 1988, Honolulu, Hawaii, pp. 73–80.

Bailey, D.H., Barszcz, E., Barton, J.T., Browning, D.S., Carter, R.L., Dagum, L., Fatoohi, R.A., Frederickson, P.O., Lasinski, T.A., Schreiber, R.S., Simon, H.D., Venkatakrishnan, V., Weeratunga, S.K., 1991. The NAS parallel benchmarks. Int. J. Supercomput. Appl. 5, 63–73.

Bakoglu, H.B., Grohoski, G.F., Thatcher, L.E., Kaeli, J.A., Moore, C.R., Tattle, D.P., Male, W.E., Hardell, W.R., Hicks, D.A., Nguyen Phu, M., Montoye, R.K., Glover, W.T., Dhawan, S., 1989. IBM second-generation RISC processor organization. In: Proceedings of IEEE International Conference on Computer Design, September 30–October 4, 1989, Rye, NY, pp. 138–142.

Balakrishnan, H., Padmanabhan, V.N., Seshan, S., Katz, R.H., 1997. A comparison of mechanisms for improving TCP performance over wireless links. IEEE/ACM Trans. Netw. 5 (6), 756–769.

Ball, T., Larus, J., 1993. Branch prediction for free. In: Proceedings of ACM SIGPLAN'93 Conference on Programming Language Design and Implementation (PLDI), June 23–25, 1993, Albuquerque, NM, pp. 30 0–313.

Banerjee, U., 1979. Speedup of Ordinary Programs (Ph.D. thesis). Department of Computer Science, University of Illinois at Urbana-Champaign.

Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., 2003. Xen and the art of virtualization. In: Proceedings of the 19th ACM Symposium on Operating Systems Principles, October 19–22, 2003, Bolton Landing, NY.

Barnes, G.H., Brown, R.M., Kato, M., Kuck, D.J., Slotnick, D.L., Stokes, R., 1968. The ILLIAC IV computer. IEEE Trans. Comput. 100 (8), 746–757.

Barroso, L.A., 2010. Warehouse scale computing [keynote address]. In: Proceedings of ACM SIGMOD, June 8–10, 2010, Indianapolis, IN.

Barroso, L.A., Hölzle, U., 2007. The case for energy-proportional computing. IEEE Comput. 40 (12), 33–37.

Barroso, L.A., Hölzle, U., 2009. The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines. Morgan & Claypool, San Rafael, CA.

Barroso, L.A., Gharachorloo, K., Bugnion, E., 1998. Memory system characterization of commercial workloads. In: Proceedings of 25th Annual International Symposium on Computer Architecture (ISCA), July 3–14, 1998, Barcelona, Spain, pp. 3–14.

Barroso, L.A., Clidaras, J., Hölzle, U., 2013. The datacenter as a computer: An introduction to the design of warehouse-scale machines. Synth. Lect. Comput. Architect. 8 (3), 1–154.

Barroso, L.A., Marty, M., Patterson, D., Ranganathan, P., 2017. Attack of the killer microseconds. Commun. ACM 56(2).

Barton, R.S., 1961. A new approach to the functional design of a computer. In: Proceedings of Western Joint Computer Conference, May 9–11, 1961, Los Angeles, CA, pp. 393–396.

Bashe, C.J., Buchholz, W., Hawkins, G.V., Ingram, J.L., Rochester, N., 1981. The architecture of IBM's early computers. IBM J. Res. Dev. 25 (5), 363–375.

Bashe, C.J., Johnson, L.R., Palmer, J.H., Pugh, E.W., 1986. IBM's Early Computers. MIT Press, Cambridge, MA.

Baskett, F., Keller, T.W., 1977. An evaluation of the Cray-1 processor. In: Kuck, D.J., Lawrie, D.H., Sameh, A.H. (Eds.), High Speed Computer and Algorithm Organization. Academic Press, San Diego, pp. 71–84.

Baskett, F., Jermoluk, T., Solomon, D., 1988. The 4D-MP graphics superworkstation: Computing + graphics = 40 MIPS + 40 MFLOPS and 10,000 lighted polygons per second. In: Proceedings of IEEE COMPCON, February 29–March 4, 1988, San Francisco, pp. 468–471.

BBN Laboratories, 1986. Butterfly Parallel Processor Overview, Tech. Rep. 6148. BBN Laboratories, Cambridge, MA.

Bell, C.G., 1984. The mini and micro industries. IEEE Comput. 17 (10), 14–30.

Bell, C.G., 1985. Multis: a new class of multiprocessor computers. Science 228 (6), 462–467.

Bell, C.G., 1989. The future of high performance computers in science and engineering. Commun. ACM 32 (9), 1091–1101.

Bell, G., Gray, J., 2001. Crays, Clusters and Centers, Tech. Rep. MSR-TR-2001-76. Microsoft Research, Redmond, WA.

Bell, C.G., Gray, J., 2002. What's next in high performance computing? CACM 45 (2), 91–95.

Bell, C.G., Newell, A., 1971. Computer Structures: Readings and Examples. McGraw-Hill, New York.

Bell, C.G., Strecker, W.D., 1976. Computer structures: what have we learned from the PDP-11? In: Third Annual International Symposium on Computer Architecture (ISCA), January 19–21, 1976, Tampa, FL, pp. 1–14.

Bell, C.G., Strecker, W.D., 1998. Computer structures: what have we learned from the PDP-11? In: 25 Years of the International Symposia on Computer Architecture (Selected Papers), ACM, New York, pp. 138–151.

Bell, C.G., Cady, R., McFarland, H., DeLagi, B., O'Laughlin, J., Noonan, R., Wulf, W., 1970. A new architecture for mini-computers: The DEC PDP-11. In: Proceedings of AFIPS Spring Joint Computer Conference, May 5–May 7, 1970, Atlantic City, NJ, pp. 657–675.

Bell, C.G., Mudge, J.C., McNamara, J.E., 1978. A DEC View of Computer Engineering. Digital Press, Bedford, MA.

Benes, V.E., 1962. Rearrangeable three stage connecting networks. Bell Syst. Tech. J. 41, 1481–1492.

Bertozzi, D., Jalabert, A., Murali, S., Tamhankar, R., Stergiou, S., Benini, L., De Micheli, G., 2005. NoC synthesis flow for customized domain specific multiprocessor systems-on-chip. IEEE Trans. Parallel Distrib. Syst. 16 (2), 113–130.

Bhandarkar, D.P., 1995. Alpha Architecture and Implementations. Digital Press, Newton, MA.

Bhandarkar, D.P., Clark, D.W., 1991. Performance from architecture: comparing a RISC and a CISC with similar hardware organizations. In: Proceedings of Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 8–11, 1991, Palo Alto, CA, pp. 310–319.

Bhandarkar, D.P., Ding, J., 1997. Performance characterization of the Pentium Pro processor. In: Proceedings of Third International Symposium on High-Performance Computer Architecture, February 1–February 5, 1997, San Antonio, TX, pp. 288–297.

Bhattacharya, S., Lane, N.D., 2016. Sparsification and separation of deep learning layers for constrained resource inference on wearables. In: Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM, pp. 176–189.

Bhuyan, L.N., Agrawal, D.P., 1984. Generalized hypercube and hyperbus structures for a computer network. IEEE Trans. Comput. 32 (4), 322–333.

Bienia, C., Kumar, S., Jaswinder, P.S., Li, K., 2008. The Parsec Benchmark Suite: Characterization and Architectural Implications, Tech. Rep. TR-811-08. Princeton University, Princeton, NJ.

Bier, J., 1997. The evolution of DSP processors. In: Presentation at University of California, Berkeley, November 14.

Bird, S., Phansalkar, A., John, L.K., Mericas, A., Indukuru, R., 2007. Characterization of performance of SPEC CPU benchmarks on Intel's Core Microarchitecture based processor. In: Proceedings of 2007 SPEC Benchmark Workshop, January 21, 2007, Austin, TX.

Birman, M., Samuels, A., Chu, G., Chuk, T., Hu, L., McLeod, J., Barnes, J., 1990. Developing the WRL3170/3171 SPARC floating-point coprocessors. IEEE Micro 10 (1), 55–64.

Blackburn, M., Garner, R., Hoffman, C., Khan, A.M., McKinley, K.S., Bentzur, R., Diwan, A., Feinberg, D., Frampton, D., Guyer, S.Z., Hirzel, M., Hosking, A., Jump, M., Lee, H., Moss, J.E.B., Phansalkar, A., Stefanovic, D., VanDrunen, T., von Dincklage, D., Wiedermann, B., 2006. The DaCapo benchmarks: Java benchmarking development and analysis. In: ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), October 22–26, 2006, pp. 169–190.

Blaum, M., Brady, J., Bruck, J., Menon, J., 1994. EVENODD: an optimal scheme for tolerating double disk failures in RAID architectures. In: Proceedings of 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago, IL, pp. 245–254.

Blaum, M., Brady, J., Bruck, J., Menon, J., 1995. EVENODD: an optimal scheme for tolerating double disk failures in RAID architectures. IEEE Trans. Comput. 44 (2), 192–202.

Blaum, M., Bruck, J., Vardy, A., 1996. MDS array codes with independent parity symbols. IEEE Trans. Inf. Theory 42, 529–542.

Blaum, M., Brady, J., Bruck, J., Menon, J., Vardy, A., 2001. The EVENODD code and its generalization. In: Jin, H., Cortes, T., Buyya, R. (Eds.), High Performance Mass Storage and Parallel I/O: Technologies and Applications. Wiley-IEEE, New York, pp. 187–208.

Bloch, E., 1959. The engineering design of the Stretch computer. In: 1959 Proceedings of the Eastern Joint Computer Conference, December 1–3, 1959, Boston, MA, pp. 48–59.

Boddie, J.R., 2000. History of DSPs, www.lucent.com/micro/dsp/dsphist.html.

Boggs, D., Baktha, A., Hawkins, J., Marr, D.T., Miller, J.A., Roussel, P., et al., 2004. The Microarchitecture of the Intel Pentium 4 processor on 90 nm technology. Intel Technol. J. 8 (1), 7–23.

Bolt, K.M., 2005. Amazon sees sales rise, profit fall. Seattle Post-Intelligencer. http://seattlepi.nwsource.com/business/245943_techearns26.html.

Bordawekar, R., Bondhugula, U., Rao, R., 2010. Believe it or not!: multi-core CPUs can match GPU performance for a FLOP-intensive application! In: 19th International Conference on Parallel Architecture and Compilation Techniques (PACT 2010). Vienna, Austria, September 11–15, 2010, pp. 537–538.

Borg, A., Kessler, R.E., Wall, D.W., 1990. Generation and analysis of very long address traces. In: 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 270–279.

Bouknight, W.J., Deneberg, S.A., McIntyre, D.E., Randall, J.M., Sameh, A.H., Slotnick, D.L., 1972. The Illiac IV system. Proc. IEEE 60 (4), 369–379. Also appears in Siewiorek, D.P., Bell, C.G., Newell, A. 1982. Computer Structures: Principles and Examples. McGraw-Hill, New York, pp. 306–316.

Brady, J.T., 1986. A theory of productivity in the creative process. IEEE Comput. Graph. Appl. 6 (5), 25–34.

Brain, M., 2000. Inside a Digital Cell Phone. www.howstuffworks.com/-inside-cellphone.htm.

Brandt, M., Brooks, J., Cahir, M., Hewitt, T., Lopez-Pineda, E., Sandness, D., 2000. The Benchmarker's Guide for Cray SV1 Systems. Cray Inc., Seattle, WA.

Brent, R.P., Kung, H.T., 1982. A regular layout for parallel adders. IEEE Trans. Comput. C-31, 260–264.

Brewer, E.A., Kuszmaul, B.C., 1994. How to get good performance from the CM-5 data network. In: Proceedings of Eighth International Parallel Processing Symposium, April 26–27, 1994, Cancun, Mexico.

Brin, S., Page, L., 1998. The anatomy of a large-scale hypertextual Web search engine. In: Proceedings of 7th International World Wide Web Conference, April 14–18, 1998, Brisbane, Queensland, Australia, pp. 107–117.

Brown, A., Patterson, D.A., 2000. Towards maintainability, availability, and growth benchmarks: a case study of software RAID systems. In: Proceedings of 2000 USENIX Annual Technical Conference, June 18–23, 2000, San Diego, CA.

Brunhaver, J.S., 2015. Design and optimization of a stencil engine (Ph.D. dissertation). Stanford University.

Bucher, I.Y., 1983. The computational speed of supercomputers. In: Proceedings of International Conference on Measuring and Modeling of Computer Systems (SIGMETRICS 1983), August 29–31, 1983, Minneapolis, MN, pp. 151–165.

Bucher, I.V., Hayes, A.H., 1980. I/O performance measurement on Cray-1 and CDC 7000 computers. In: Proceedings of Computer Performance Evaluation Users Group, 16th Meeting, NBS 500-65, pp. 245–254.

Bucholtz, W., 1962. Planning a Computer System: Project Stretch. McGraw-Hill, New York.

Burgess, N., Williams, T., 1995. Choices of operand truncation in the SRT division algorithm. IEEE Trans. Comput. 44 (7), 933–938.

Burkhardt III, H., Frank, S., Knobe, B., Rothnie, J., 1992. Overview of the KSR1 Computer System, Tech. Rep. KSR-TR-9202001. Kendall Square Research, Boston, MA.

Burks, A.W., Goldstine, H.H., von Neumann, J., 1946. Preliminary discussion of the logical design of an electronic computing instrument. Report to the U.S. Army Ordnance Department, p. 1; also appears in Papers of John von Neumann, Aspray, W., Burks, A. (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, CA, 1987, pp. 97–146.

Calder, B., Grunwald, D., Jones, M., Lindsay, D., Martin, J., Mozer, M., Zorn, B., 1997. Evidence-based static branch prediction using machine learning. ACM Trans. Program. Lang. Syst. 19 (1), 188–222.

Calder, B., Reinman, G., Tullsen, D.M., 1999. Selective value prediction. In: Proceedings of 26th Annual International Symposium on Computer Architecture (ISCA), May 2–4, 1999, Atlanta, GA.

Callahan, D., Dongarra, J., Levine, D., 1988. Vectorizing compilers: a test suite and results. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 12–17, 1988, Orland, FL, pp. 98–105.

Canis, A., Choi, J., Aldham, M., Zhang, V., Kammoona, A., Czajkowski, T., Brown, S.D., Anderson, J.H., 2013. LegUp: an open-source high-level synthesis tool for FPGA-based processor/accelerator systems. ACM Trans. Embed. Comput. Syst. 13(2).

Canny, J., et al., 2015. Machine learning at the limit. In: IEEE International Conference on Big Data.

Cantin, J.F., Hill, M.D., 2001. Cache performance for selected SPEC CPU2000 benchmarks. www.jfred.org/cache-data.html.

Cantin, J.F., Hill, M.D., 2003. Cache performance for SPEC CPU2000 benchmarks, version 3.0. www.cs.wisc.edu/multifacet/misc/spec2000cache-data/index.html.

Carles, S., 2005. Amazon reports record Xmas season, top game picks. Gamasutra, December 27. http://www.gamasutra.com/php-bin/news_index.php?story=7630.

Carter, J., Rajamani, K., 2010. Designing energy-efficient servers and data centers. IEEE Comput. 43 (7), 76–78.

Case, R.P., Padegs, A., 1978. The architecture of the IBM System/370. Commun. ACM 21 (1), 73–96. Also appears in Siewiorek, D.P., Bell, C.G., Newell, A., 1982. Computer Structures: Principles and Examples. McGraw-Hill, New York, pp. 830–855.

Caulfield, A.M., Chung, E.S., Putnam, A., Haselman, H.A.J.F.M., Humphrey, S.H.M., Daniel, P.K.J.Y.K., Ovtcharov, L.T.M.K., Lanka, M.P.L.W.S., Burger, D.C.D., 2016. A cloud-scale acceleration architecture. In: MICRO Conference.

Censier, L., Feautrier, P., 1978. A new solution to coherence problems in multicache systems. IEEE Trans. Comput. C-27 (12), 1112–1118.

Chandra, R., Devine, S., Verghese, B., Gupta, A., Rosenblum, M., 1994. Scheduling and page migration for multiprocessor compute servers. In: Sixth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 4–7, 1994, San Jose, CA, pp. 12–24.

Chang, P.P., Mahlke, S.A., Chen, W.Y., Warter, N.J., Hwu, W.W., 1991. IMPACT: an architectural framework for multiple-instruction-issue processors. In: 18th Annual International Symposium on Computer Architecture (ISCA), May 27–30, 1991, Toronto, Canada, pp. 266–275.

Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A., Gruber, R.E., 2006. Bigtable: a distributed storage system for structured data. In: Proceedings of 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI'06), November 6–8, 2006, Seattle, WA.

Chang, J., Meza, J., Ranganathan, P., Bash, C., Shah, A., 2010. Green server design: beyond operational energy to sustainability. In: Proceedings of Workshop on Power Aware Computing and Systems (HotPower'10), October 3, 2010, Vancouver, British Columbia.

Charlesworth, A.E., 1981. An approach to scientific array processing: the architectural design of the AP-120B/FPS-164 family. Computer 9, 18–27.

Charlesworth, A., 1998. Starfire: extending the SMP envelope. IEEE Micro 18 (1), 39–49.

Chen, T.C., 1980. Overlap and parallel processing. In: Stone, H. (Ed.), Introduction to Computer Architecture. Science Research Associates, Chicago, pp. 427–486.

Chen, S., 1983. Large-scale and high-speed multiprocessor system for scientific applications. In: Proceedings of NATO Advanced Research Workshop on High-Speed Computing, June 20–22, 1983, Jülich, West Germany. Also appears in Hwang, K. (Ed.), 1984. Superprocessors: design and applications, *IEEE* (August), 602–609.

Chen, P.M., Lee, E.K., 1995. Striping in a RAID level 5 disk array. In: Proceedings of ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems, May 15–19, 1995, Ottawa, Canada, pp. 136–145.

Chen, P.M., Gibson, G.A., Katz, R.H., Patterson, D.A., 1990. An evaluation of redundant arrays of inexpensive disks using an Amdahl 5890. In: Proceedings of ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems, May 22–25, 1990, Boulder, CO.

Chen, P.M., Lee, E.K., Gibson, G.A., Katz, R.H., Patterson, D.A., 1994. RAID: high-performance, reliable secondary storage. ACM Comput. Surv. 26 (2), 145–188.

Chow, F.C., 1983. A Portable Machine-Independent Global Optimizer—Design and Measurements (Ph.D. thesis). Stanford University, Palo Alto, CA.

Chrysos, G.Z., Emer, J.S., 1998. Memory dependence prediction using store sets. In: Proceedings of 25th Annual International Symposium on Computer Architecture (ISCA), July 3–14, 1998, Barcelona, Spain, pp. 142–153.

Clark, W.A., 1957. The Lincoln TX-2 computer development. In: Proceedings of Western Joint Computer Conference, February 26–28, 1957, Los Angeles, pp. 143–145.

Clark, D.W., 1983. Cache performance of the VAX-11/780. ACM Trans. Comput. Syst. 1 (1), 24–37.

Clark, D.W., 1987. Pipelining and performance in the VAX 8800 processor. In: Proceedings of Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 5–8, 1987, Palo Alto, CA, pp. 173–177.

Clark, J., 2014. Five Numbers That Illustrate the Mind-Bending Size of Amazon's Cloud. Bloomberg. https://www.bloomberg.com/news/2014-11-14/5-numbersthat-illustrate-the-mind-bending-size-of-amazon-s-cloud.html.

Clark, J., October 26, 2015. Google Turning Its Lucrative Web Search Over to AI Machines. Bloomberg Technology, www.bloomberg.com.

Clark, D.W., Emer, J.S., 1985. Performance of the VAX-11/780 translation buffer: simulation and measurement. ACM Trans. Comput. Syst. 3 (1), 31–62.

Clark, D., Levy, H., 1982. Measurement and analysis of instruction set use in the VAX-11/780. In: Proceedings of Ninth Annual International Symposium on Computer Architecture (ISCA), April 26–29, 1982, Austin, TX, pp. 9–17.

Clark, D., Strecker, W.D., 1980. Comments on 'the case for the reduced instruction set computer'. Comput. Architect. News 8 (6), 34–38.

Clark, B., Deshane, T., Dow, E., Evanchik, S., Finlayson, M., Herne, J., Neefe Matthews, J., 2004. Xen and the art of repeated research. In: Proceedings of USENIX Annual Technical Conference, June 27–July 2, 2004, pp. 135–144.

Clidaras, J., Johnson, C., Felderman, B., 2010. Private communication.

Climate Savers Computing Initiative, 2007. Efficiency Specs. http://www.climatesaverscomputing.org/.

Clos, C., 1953. A study of non-blocking switching networks. Bell Syst. Tech. J. 32 (2), 406–424.

Cloud, Bloomberg, n.d. https://www.bloomberg.com/news/2014-11-14/5-numbersthat-illustrate-the-mind-bending-size-of-amazon-s-cloud.html.

Cody, W.J., Coonen, J.T., Gay, D.M., Hanson, K., Hough, D., Kahan, W., Karpinski, R., Palmer, J., Ris, F.N., Stevenson, D., 1984. A proposed radix- and word-length independent standard for floating-point arithmetic. IEEE Micro 4 (4), 86–100.

Colwell, R.P., Steck, R., 1995. A 0.6 μm BiCMOS processor with dynamic execution. In: Proceedings of IEEE International Symposium on Solid State Circuits (ISSCC), February 15–17, 1995, San Francisco, pp. 176–177.

Colwell, R.P., Nix, R.P., O'Donnel, J.J., Papworth, D.B., Rodman, P.K., 1987. A VLIW architecture for a trace scheduling compiler. In: Proceedings of Second International

Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 5–8, 1987, Palo Alto, CA, pp. 180–192.

Comer, D., 1993. Internetworking with TCP/IP, second ed. Prentice Hall, Englewood Cliffs, NJ.

Compaq Computer Corporation, 1999. Compiler Writer's Guide for the Alpha 21264, Order Number EC-RJ66A-TE, June, www1.support.compaq.com/alpha-tools/-documentation/current/21264_EV67/ec-rj66a-te_comp_writ_gde_for_alpha21264.pdf.

Conti, C., Gibson, D.H., Pitkowsky, S.H., 1968. Structural aspects of the System/360 Model 85. Part I. General organization. IBM Syst. J. 7 (1), 2–14.

Coonen, J., 1984. Contributions to a Proposed Standard for Binary Floating-Point Arithmetic (Ph.D. thesis). University of California, Berkeley.

Corbett, P., English, B., Goel, A., Grcanac, T., Kleiman, S., Leong, J., Sankar, S., 2004. Row-diagonal parity for double disk failure correction. In: Proceedings of 3rd USENIX Conference on File and Storage Technology (FAST'04), March 31–April 2, 2004, San Francisco.

Crawford, J., Gelsinger, P., 1988. Programming the 80386. Sybex Books, Alameda, CA.

Culler, D.E., Singh, J.P., Gupta, A., 1999. Parallel Computer Architecture: A Hardware/Software Approach. Morgan Kaufmann, San Francisco.

Curnow, H.J., Wichmann, B.A., 1976. A synthetic benchmark. Comput. J. 19 (1), 43–49.

Cvetanovic, Z., Kessler, R.E., 2000. Performance analysis of the Alpha 21264-based Compaq ES40 system. In: Proceedings of 27th Annual International Symposium on Computer Architecture (ISCA), June 10–14, 2000, Vancouver, Canada, pp. 192–202.

Dally, W.J., 1990. Performance analysis of *k*-ary *n*-cube interconnection networks. IEEE Trans. Comput. 39 (6), 775–785.

Dally, W.J., 1992. Virtual channel flow control. IEEE Trans. Parallel Distrib. Syst. 3 (2), 194–205.

Dally, W.J., 1999. Interconnect limited VLSI architecture. In: Proceedings of the International Interconnect Technology Conference, May 24–26, 1999, San Francisco.

Dally, W.J., 2002. Computer architecture is all about interconnect. In: Proceedings of the 8th International Symposium High Performance Computer Architecture.

Dally, W.J., 2016. High Performance Hardware for Machine Learning. Cadence Embedded Neural Network Summit, February 9, 2016. http://ip.cadence.com/uploads/presentations/1000AM_Dally_Cadence_ENN.pdf.

Dally, W.J., Seitz, C.I., 1986. The torus routing chip. Distrib. Comput. 1 (4), 187–196.

Dally, W.J., Towles, B., 2001. Route packets, not wires: on-chip interconnection networks. In: Proceedings of 38th Design Automation Conference, June 18–22, 2001, Las Vegas.

Dally, W.J., Towles, B., 2003. Principles and Practices of Interconnection Networks. Morgan Kaufmann, San Francisco.

Darcy, J.D., Gay, D., 1996. FLECKmarks: measuring floating point performance using a full IEEE compliant arithmetic benchmark. CS 252 class project, University of California, Berkeley. See http.CS.Berkeley.EDU/~darcy/Projects/cs252/.

Darley, H.M., et al., 1989. Floating Point/Integer Processor with Divide and Square Root Functions, U.S. Patent 4,878,190, October 31.

Davidson, E.S., 1971. The design and control of pipelined function generators. In: Proceedings of IEEE Conference on Systems, Networks, and Computers, January 19–21, 1971, Oaxtepec, Mexico, pp. 19–21.

Davidson, E.S., Thomas, A.T., Shar, L.E., Patel, J.H., 1975. Effective control for pipelined processors. In: Proceedings of IEEE COMPCON, February 25–27, 1975, San Francisco, pp. 181–184.

Davie, B.S., Peterson, L.L., Clark, D., 1999. Computer Networks: A Systems Approach, second ed. Morgan Kaufmann, San Francisco.

Dean, J., 2009. Designs, lessons and advice from building large distributed systems [keynote address]. In: Proceedings of 3rd ACM SIGOPS International Workshop on Large-Scale Distributed Systems and Middleware, Co-located with the 22nd ACM Symposium on Operating Systems Principles, October 11–14, 2009, Big Sky, Mont.

Dean, J., Barroso, L.A., 2013. The tail at scale. Commun. ACM 56 (2), 74–80.

Dean, J., Ghemawat, S., 2004. MapReduce: simplified data processing on large clusters. In: Proceedings of Operating Systems Design and Implementation (OSDI), December 6–8, 2004, San Francisco, CA, pp. 137–150.

Dean, J., Ghemawat, S., 2008. MapReduce: simplified data processing on large clusters. Commun. ACM 51 (1), 107–113.

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., Vogels, W., 2007. Dynamo: Amazon's highly available key-value store. In: Proceedings of 21st ACM Symposium on Operating Systems Principles, October 14–17, 2007, Stevenson, WA.

Dehnert, J.C., Hsu, P.Y.-T., Bratt, J.P., 1989. Overlapped loop support on the Cydra 5. In: Proceedings of Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 3–6, 1989, Boston, MA, pp. 26–39.

Demmel, J.W., Li, X., 1994. Faster numerical algorithms via exception handling. IEEE Trans. Comput. 43 (8), 983–992.

Denehy, T.E., Bent, J., Popovici, F.I., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., 2004. Deconstructing storage arrays. In: Proceedings of 11th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 7–13, 2004, Boston, MA, pp. 59–71.

Desurvire, E., 1992. Lightwave communications: the fifth generation. Sci. Am. (Int. Ed.) 266 (1), 96–103.

Diep, T.A., Nelson, C., Shen, J.P., 1995. Performance evaluation of the PowerPC 620 microarchitecture. In: Proceedings of 22nd Annual International Symposium on Computer Architecture (ISCA), June 22–24, 1995, Santa Margherita, Italy.

Digital Semiconductor, 1996. Alpha Architecture Handbook, Version 3. Digital Press, Maynard, MA.

Ditzel, D.R., McLellan, H.R., 1987. Branch folding in the CRISP microprocessor: reducing the branch delay to zero. In: Proceedings of 14th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1987, Pittsburgh, PA, pp. 2–7.

Ditzel, D.R., Patterson, D.A., 1980. Retrospective on high-level language computer architecture. In: Proceedings of Seventh Annual International Symposium on Computer Architecture (ISCA), May 6–8, 1980, La Baule, France, pp. 97–104.

Doherty, W.J., Kelisky, R.P., 1979. Managing VM/CMS systems for user effectiveness. IBM Syst. J. 18 (1), 143–166.

Doherty, W.J., Thadhani, A.J., 1982. The economic value of rapid response time. IBM Report.

Dongarra, J.J., 1986. A survey of high performance processors. In: Proceedings of IEEE COMPCON, March 3–6, 1986, San Francisco, pp. 8–11.

Dongarra, J.J., Luszczek, P., Petitet, A., 2003. The LINPACK benchmark: past, present and future. Concurr. Comput. Pract. Exp. 15 (9), 803–820.

Dongarra, J., Sterling, T., Simon, H., Strohmaier, E., 2005. High-performance computing: clusters, constellations, MPPs, and future directions. Comput. Sci. Eng. 7 (2), 51–59.

Douceur, J.R., Bolosky, W.J., 1999. A large scale study of file-system contents. In: Proceedings of ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems, May 1–9, 1999, Atlanta, GA, pp. 59–69.

Douglas, J., 2005. Intel 8xx series and Paxville Xeon-MP microprocessors. In: Paper Presented at Hot Chips 17, August 14–16, 2005, Stanford University, Palo Alto, CA.

Duato, J., 1993. A new theory of deadlock-free adaptive routing in wormhole networks. IEEE Trans. Parallel Distrib. Syst. 4 (12), 1320–1331.
Duato, J., Pinkston, T.M., 2001. A general theory for deadlock-free adaptive routing using a mixed set of resources. IEEE Trans. Parallel Distrib. Syst. 12 (12), 1219–1235.
Duato, J., Yalamanchili, S., Ni, L., 2003. Interconnection Networks: An Engineering Approach, 2nd printing Morgan Kaufmann, San Francisco.
Duato, J., Johnson, I., Flich, J., Naven, F., Garcia, P., Nachiondo, T., 2005a. A new scalable and cost-effective congestion management strategy for lossless multistage interconnection networks. In: Proceedings of 11th International Symposium on High-Performance Computer Architecture, February 12–16, 2005, San Francisco.
Duato, J., Lysne, O., Pang, R., Pinkston, T.M., 2005b. Part I: a theory for deadlock-free dynamic reconfiguration of interconnection networks. IEEE Trans. Parallel Distrib. Syst. 16 (5), 412–427.
Dubois, M., Scheurich, C., Briggs, F., 1988. Synchronization, coherence, and event ordering. IEEE Comput. 21 (2), 9–21.
Dunigan, W., Vetter, K., White, K., Worley, P., 2005. Performance evaluation of the Cray X1 distributed shared memory architecture. IEEE Micro, 30–40.
Eden, A., Mudge, T., 1998. The YAGS branch prediction scheme. In: Proceedings of the 31st Annual ACM/IEEE International Symposium on Microarchitecture, November 30–December 2, 1998, Dallas, TX, pp. 69–80.
Edmondson, J.H., Rubinfield, P.I., Preston, R., Rajagopalan, V., 1995. Superscalar instruction execution in the 21164 Alpha microprocessor. IEEE Micro 15 (2), 33–43.
Eggers, S., 1989. Simulation Analysis of Data Sharing in Shared Memory Multiprocessors (Ph.D. thesis). University of California, Berkeley.
Elder, J., Gottlieb, A., Kruskal, C.K., McAuliffe, K.P., Randolph, L., Snir, M., Teller, P., Wilson, J., 1985. Issues related to MIMD shared-memory computers: the NYU ultracomputer approach. In: Proceedings of 12th Annual International Symposium on Computer Architecture (ISCA), June 17–19, 1985, Boston, MA, pp. 126–135.
Ellis, J.R., 1986. Bulldog: A Compiler for VLIW Architectures. MIT Press, Cambridge, MA.
Emer, J.S., Clark, D.W., 1984. A characterization of processor performance in the VAX-11/780. In: Proceedings of 11th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1984, Ann Arbor, MI, pp. 301–310.
Enriquez, P., 2001. What happened to my dial tone? A study of FCC service disruption reports. In: Poster, Richard Tapia Symposium on the Celebration of Diversity in Computing, October 18–20, Houston, TX.
Erlichson, A., Nuckolls, N., Chesson, G., Hennessy, J.L., 1996. SoftFLASH: analyzing the performance of clustered distributed virtual shared memory. In: Proceedings of Seventh International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 1–5, 1996, Cambridge, MA, pp. 210–220.
Esmaeilzadeh, H., Cao, T., Xi, Y., Blackburn, S.M., McKinley, K.S., 2011. Looking back on the language and hardware revolution: measured power, performance, and scaling. In: Proceedings of 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), March 5–11, 2011, Newport Beach, CA.
Esmaeilzadeh, H., Blem, E., St Amant, R., Sankaralingam, K., Burger, D., 2012. Power limitations and dark silicon challenge the future of multicore. ACM Trans. Comput. Syst. 30 (3), 115–138.
Evers, M., Patel, S.J., Chappell, R.S., Patt, Y.N., 1998. An analysis of correlation and predictability: what makes two-level branch predictors work. In: Proceedings of 25th Annual International Symposium on Computer Architecture (ISCA), July 3–14, 1998, Barcelona, Spain, pp. 52–61.
Fabry, R.S., 1974. Capability based addressing. Commun. ACM 17 (7), 403–412.

Falsafi, B., Wood, D.A., 1997. Reactive NUMA: a design for unifying S-COMA and CC-NUMA. In: Proceedings of 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO, pp. 229–240.

Fan, X., Weber, W., Barroso, L.A., 2007. Power provisioning for a warehouse-sized computer. In: Proceedings of 34th Annual International Symposium on Computer Architecture (ISCA), June 9–13, 2007, San Diego, CA.

Farkas, K.I., Jouppi, N.P., 1994. Complexity/performance trade-offs with non-blocking loads. In: Proceedings of 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago.

Farkas, K.I., Jouppi, N.P., Chow, P., 1995. How useful are non-blocking loads, stream buffers and speculative execution in multiple issue processors? In: Proceedings of First IEEE Symposium on High-Performance Computer Architecture, January 22–25, 1995, Raleigh, NC, pp. 78–89.

Farkas, K.I., Chow, P., Jouppi, N.P., Vranesic, Z., 1997. Memory-system design considerations for dynamically-scheduled processors. In: Proceedings of 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO, pp. 133–143.

Fazio, D., 1987. It's really much more fun building a supercomputer than it is simply inventing one. In: Proceedings of IEEE COMPCON, February 23–27, 1987, San Francisco, pp. 102–105.

Fikes, A., 2010. Storage architecture and challenges. In: Google Faculty Summit.

Fisher, J.A., 1981. Trace scheduling: a technique for global microcode compaction. IEEE Trans. Comput. 30 (7), 478–490.

Fisher, J.A., 1983. Very long instruction word architectures and ELI-512. In: 10th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1982, Stockholm, Sweden, pp. 140–150.

Fisher, J.A., Freudenberger, S.M., 1992. Predicting conditional branches from previous runs of a program. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston, MA, pp. 85–95.

Fisher, J.A., Rau, B.R., 1993. J. Supercomput., January (special issue).

Fisher, J.A., Ellis, J.R., Ruttenberg, J.C., Nicolau, A., 1984. Parallel processing: a smart compiler and a dumb processor. In: Proceedings of SIGPLAN Conference on Compiler Construction, June 17–22, 1984, Montreal, Canada, pp. 11–16.

Flemming, P.J., Wallace, J.J., 1986. How not to lie with statistics: the correct way to summarize benchmarks results. Commun. ACM 29 (3), 218–221.

Flynn, M.J., 1966. Very high-speed computing systems. Proc. IEEE 54 (12), 1901–1909.

Forgie, J.W., 1957. The Lincoln TX-2 input-output system. In: Proceedings of Western Joint Computer Conference (February), Institute of Radio Engineers, Los Angeles, pp. 156–160.

Foster, C.C., Riseman, E.M., 1972. Percolation of code to enhance parallel dispatching and execution. IEEE Trans. Comput. C-21 (12), 1411–1415.

Frank, S.J., 1984. Tightly coupled multiprocessor systems speed memory access time. Electronics 57 (1), 164–169.

Freescale as part of i.MX31 Applications Processor, 2006. http://cache.freescale.com/files/32bit/doc/white_paper/IMX31MULTIWP.pdf.

Freiman, C.V., 1961. Statistical analysis of certain binary division algorithms. Proc. IRE 49 (1), 91–103.

Friesenborg, S.E., Wicks, R.J., 1985. DASD Expectations: The 3380, 3380-23, and MVS/XA, Tech. Bulletin GG22-9363-02. IBM Washington Systems Center, Gaithersburg, MD.

Fuller, S.H., Burr, W.E., 1977. Measurement and evaluation of alternative computer architectures. Computer 10 (10), 24–35.

Furber, S.B., 1996. ARM System Architecture. Addison-Wesley, Harlow, England. www.cs.man.ac.uk/amulet/publications/books/ARMsysArch.

Gagliardi, U.O., 1973. Report of workshop 4—software-related advances in computer hardware. In: Proceedings of Symposium on the High Cost of Software, September 17–19, 1973, Monterey, CA, pp. 99–120.

Gajski, D., Kuck, D., Lawrie, D., Sameh, A., 1983. CEDAR—a large scale multiprocessor. In: Proceedings of International Conference on Parallel Processing (ICPP), August, Columbus, Ohio, pp. 524–529.

Galal, S., Shacham, O., Brunhaver II, J.S., Pu, J., Vassiliev, A., Horowitz, M., 2013. FPU generator for design space exploration. In: 21st IEEE Symposium on Computer Arithmetic (ARITH).

Gallagher, D.M., Chen, W.Y., Mahlke, S.A., Gyllenhaal, J.C., Hwu, W.W., 1994. Dynamic memory disambiguation using the memory conflict buffer. In: Proceedings of Sixth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 4–7, Santa Jose, CA, pp. 183–193.

Galles, M., 1996. Scalable pipelined interconnect for distributed endpoint routing: the SGI SPIDER chip. In: Proceedings of IEEE HOT Interconnects'96, August 15–17, 1996, Stanford University, Palo Alto, CA.

Game, M., Booker, A., 1999. CodePack code compression for PowerPC processors. MicroNews. 5 (1). www.chips.ibm.com/micronews/vol5_no1/codepack.html.

Gao, Q.S., 1993. The Chinese remainder theorem and the prime memory system. In: 20th Annual International Symposium on Computer Architecture (ISCA), May 16–19, 1993, San Diego, CA (Computer Architecture News 21:2 (May), pp. 337–340.

Gap, 2005. Gap Inc. Reports Third Quarter Earnings. http://gapinc.com/public/documents/PR_Q405EarningsFeb2306.pdf.

Gap, 2006. Gap Inc. Reports Fourth Quarter and Full Year Earnings. http://-gapinc.com/public/documents/Q32005PressRelease_Final22.pdf.

Garner, R., Agarwal, A., Briggs, F., Brown, E., Hough, D., Joy, B., Kleiman, S., Muchnick, S., Namjoo, M., Patterson, D., Pendleton, J., Tuck, R., 1988. Scalable processor architecture (SPARC). In: Proceedings of IEEE COMPCON, February 29–March 4, 1988, San Francisco, pp. 278–283.

Gebis, J., Patterson, D., 2007. Embracing and extending 20th-century instruction set architectures. IEEE Comput. 40 (4), 68–75.

Gee, J.D., Hill, M.D., Pnevmatikatos, D.N., Smith, A.J., 1993. Cache performance of the SPEC92 benchmark suite. IEEE Micro 13 (4), 17–27.

Gehringer, E.F., Siewiorek, D.P., Segall, Z., 1987. Parallel Processing: The Cm* Experience. Digital Press, Bedford, MA.

Gharachorloo, K., Lenoski, D., Laudon, J., Gibbons, P., Gupta, A., Hennessy, J.L., 1990. Memory consistency and event ordering in scalable shared-memory multiprocessors. In: Proceedings of 17th Annual International Symposium on Computer Architecture (ISCA), May 28–31, 1990, Seattle, WA, pp. 15–26.

Gharachorloo, K., Gupta, A., Hennessy, J.L., 1992. Hiding memory latency using dynamic scheduling in shared-memory multiprocessors. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia.

Ghemawat, S., Gobioff, H., Leung, S.-T., 2003. The Google file system. In: Proceedings of 19th ACM Symposium on Operating Systems Principles, October 19–22, 2003, Bolton Landing, NY.

Gibson, D.H., 1967. Considerations in block-oriented systems design. AFIPS Conf. Proc. 30, 75–80.

Gibson, J.C., 1970. The Gibson mix, Rep. TR. 00.2043. IBM Systems Development Division, Poughkeepsie, NY (research done in 1959).

Gibson, G.A., 1992. In: Redundant Disk Arrays: Reliable, Parallel Secondary Storage. ACM Distinguished Dissertation Series, MIT Press, Cambridge, MA.

Gibson, J., Kunz, R., Ofelt, D., Horowitz, M., Hennessy, J., Heinrich, M., 2000. FLASH vs. (simulated) FLASH: Closing the simulation loop. In: Proceedings of Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), November 12–15, Cambridge, MA, pp. 49–58.

Glass, C.J., Ni, L.M., 1992. The Turn Model for adaptive routing. In: 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia.

Goldberg, I.B., 1967. 27 bits are not enough for 8-digit accuracy. Commun. ACM 10 (2), 105–106.

Goldberg, D., 1991. What every computer scientist should know about floating-point arithmetic. Comput. Surv. 23 (1), 5–48.

Goldstein, S., 1987. Storage Performance—An Eight Year Outlook, Tech. Rep. TR 03.308-1. Santa Teresa Laboratory, IBM Santa Teresa Laboratory, San Jose, CA.

Goldstine, H.H., 1972. The Computer: From Pascal to von Neumann. Princeton University Press, Princeton, NJ.

González, A., Day, M., 2016. Amazon, Microsoft invest billions as computing shifts to cloud. The Seattle Times. http://www.seattletimes.com/business/technology/amazon-microsoft-invest-billions-as-computing-shifts-to-cloud/.

González, J., González, A., 1998. Limits of instruction level parallelism with data speculation. In: Proceedings of Vector and Parallel Processing (VECPAR) Conference, June 21–23, 1998, Porto, Portugal, pp. 585–598.

Goodman, J.R., 1983. Using cache memory to reduce processor memory traffic. In: Proceedings of 10th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1982, Stockholm, Sweden, pp. 124–131.

Goralski, W., 1997. SONET: A Guide to Synchronous Optical Network. McGraw-Hill, New York.

Gosling, J.B., 1980. Design of Arithmetic Units for Digital Computers. Springer-Verlag, New York.

Gray, J., 1990. A census of Tandem system availability between 1985 and 1990. IEEE Trans. Reliab. 39 (4), 409–418.

Gray, J. (Ed.), 1993. The Benchmark Handbook for Database and Transaction Processing Systems, second ed. Morgan Kaufmann, San Francisco.

Gray, J., 2006. Sort benchmark home page. http://sortbenchmark.org/.

Gray, J., Reuter, A., 1993. Transaction Processing: Concepts and Techniques. Morgan Kaufmann, San Francisco.

Gray, J., Siewiorek, D.P., 1991. High-availability computer systems. Computer 24 (9), 39–48.

Gray, J., van Ingen, C., 2005. Empirical Measurements of Disk Failure Rates and Error Rates, MSR-TR-2005-166. Microsoft Research, Redmond, WA.

Greenberg, A., Jain, N., Kandula, S., Kim, C., Lahiri, P., Maltz, D., Patel, P., Sengupta, S., 2009. VL2: a scalable and flexible data center network. In: Proceedings of ACM SIGCOMM, August 17–21, 2009, Barcelona, Spain.

Grice, C., Kanellos, M., 2000. Cell phone industry at crossroads: go high or low? CNET News.technews.netscape.com/news/0-1004-201-2518386-0.html?tag=st.ne.1002.tgif.sf.

Groe, J.B., Larson, L.E., 2000. CDMA Mobile Radio Design. Artech House, Boston.

Gunther, K.D., 1981. Prevention of deadlocks in packet-switched data transport systems. IEEE Trans. Commun. 29 (4), 512–524.
Hagersten, E., Koster, M., 1998. WildFire: a scalable path for SMPs. In: Proceedings of Fifth International Symposium on High-Performance Computer Architecture, January 9–12, 1999, Orlando, FL.
Hagersten, E., Landin, A., Haridi, S., 1992. DDM—a cache-only memory architecture. IEEE Comput. 25 (9), 44–54.
Hamacher, V.C., Vranesic, Z.G., Zaky, S.G., 1984. Computer Organization, second ed. McGraw-Hill, New York.
Hameed, R., Qadeer, W., Wachs, M., Azizi, O., Solomatnikov, A., Lee, B.C., Richardson, S., Kozyrakis, C., Horowitz, M., 2010. Understanding sources of inefficiency in general-purpose chips. ACM SIGARCH Comput. Architect. News 38 (3), 37–47.
Hamilton, J., 2009. Data center networks are in my way. In: Paper Presented at the Stanford Clean Slate CTO Summit, October 23, 2009. http://mvdirona.com/jrh/-TalksAndPapers/JamesHamilton_CleanSlateCTO2009.pdf.
Hamilton, J., 2010. Cloud computing economies of scale. In: Paper Presented at the AWS Workshop on Genomics and Cloud Computing, June 8, 2010, Seattle, WA. http://mvdirona.com/jrh/TalksAndPapers/JamesHamilton_GenomicsCloud20100608.pdf.
Hamilton, J., 2014. AWS Innovation at Scale, AWS Re-invent conference. https://www.youtube.com/watch?v=JIQETrFC_SQ.
Hamilton, J., 2015. The Return to the Cloud. http://perspectives.mvdirona.com/2015/05/the-return-to-the-cloud//.
Hamilton, J., 2017. How Many Data Centers Needed World-Wide. http://perspectives.mvdirona.com/2017/04/how-many-data-centers-needed-worldwide/.
Hammerstrom, D., 1990. A VLSI architecture for high-performance, low-cost, on-chip learning. In: IJCNN International Joint Conference on Neural Networks.
Handy, J., 1993. The Cache Memory Book. Academic Press, Boston.
Hauck, E.A., Dent, B.A., 1968. Burroughs' B6500/B7500 stack mechanism. In: Proceedings of AFIPS Spring Joint Computer Conference, April 30–May 2, 1968, Atlantic City, NJ, pp. 245–251.
He, K., Zhang, X., Ren, S., Sun, J., 2016. Identity mappings in deep residual networks. Also in arXiv preprint arXiv:1603.05027.
Heald, R., Aingaran, K., Amir, C., Ang, M., Boland, M., Das, A., Dixit, P., Gouldsberry, G., Hart, J., Horel, T., Hsu, W.-J., Kaku, J., Kim, C., Kim, S., Klass, F., Kwan, H., Lo, R., McIntyre, H., Mehta, A., Murata, D., Nguyen, S., Pai, Y.-P., Patel, S., Shin, K., Tam, K., Vishwanthaiah, S., Wu, J., Yee, G., You, H., 2000. Implementation of third-generation SPARC V9 64-b microprocessor. In: ISSCC Digest of Technical Papers, pp. 412–413.
Heinrich, J., 1993. MIPS R4000 User's Manual. Prentice Hall, Englewood Cliffs, NJ.
Henly, M., McNutt, B., 1989. DASD I/O Characteristics: A Comparison of MVS to VM, Tech. Rep. TR 02.1550 (May). IBM General Products Division, San Jose, CA.
Hennessy, J., 1984. VLSI processor architecture. IEEE Trans. Comput. C-33 (11), 1221–1246.
Hennessy, J., 1985. VLSI RISC processors. VLSI Syst. Des. 6 (10), 22–32.
Hennessy, J., Jouppi, N., Baskett, F., Gill, J., 1981. MIPS: a VLSI processor architecture. In: CMU Conference on VLSI Systems and Computations. Computer Science Press, Rockville, MD.
Hewlett-Packard, 1994. PA-RISC 2.0 Architecture Reference Manual, third ed. Hewlett-Packard, Palo Alto, CA.
Hewlett-Packard, 1998. HP's '5NINES:5MINUTES' Vision Extends Leadership and Redefines High Availability in Mission-Critical Environments. www.future.enterprisecomputing.hp.com/ia64/news/5nines_vision_pr.html.

Hill, M.D., 1987. Aspects of Cache Memory and Instruction Buffer Performance (Ph.D. thesis). Tech. Rep. UCB/CSD 87/381. Computer Science Division, University of California, Berkeley.

Hill, M.D., 1988. A case for direct mapped caches. Computer 21 (12), 25–40.

Hill, M.D., 1998. Multiprocessors should support simple memory consistency models. IEEE Comput. 31 (8), 28–34.

Hillis, W.D., 1985. The Connection Multiprocessor. MIT Press, Cambridge, MA.

Hillis, W.D., Steele, G.L., 1986. Data parallel algorithms. Commun. ACM 29 (12), 1170–1183.

Hinton, G., Sager, D., Upton, M., Boggs, D., Carmean, D., Kyker, A., Roussel, P., 2001. The microarchitecture of the Pentium 4 processor. Intel Technol. J.

Hintz, R.G., Tate, D.P., 1972. Control data STAR-100 processor design. In: Proceedings of IEEE COMPCON, September 12–14, 1972, San Francisco, pp. 1–4.

Hirata, H., Kimura, K., Nagamine, S., Mochizuki, Y., Nishimura, A., Nakase, Y., Nishizawa, T., 1992. An elementary processor architecture with simultaneous instruction issuing from multiple threads. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 136–145.

Hitachi, 1997. SuperH RISC Engine SH7700 Series Programming Manual. Hitachi, Santa Clara, CA. www.halsp.hitachi.com/tech_prod/.

Ho, R., Mai, K.W., Horowitz, M.A., 2001. The future of wires. In: Proc. of the IEEE, 89. 4, pp. 490–504.

Hoagland, A.S., 1963. Digital Magnetic Recording. Wiley, New York.

Hockney, R.W., Jesshope, C.R., 1988. Parallel Computers 2: Architectures, Programming and Algorithms. Adam Hilger, Ltd., Bristol, England.

Holland, J.H., 1959. A universal computer capable of executing an arbitrary number of subprograms simultaneously. Proc. East Joint Comput. Conf. 16, 108–113.

Holt, R.C., 1972. Some deadlock properties of computer systems. ACM Comput. Surv. 4 (3), 179–196.

Hölzle, U., 2010. Brawny cores still beat wimpy cores, most of the time. IEEE Micro 30, 4 (July/August).

Hopkins, M., 2000. A critical look at IA-64: massive resources, massive ILP, but can it deliver? Microprocessor Rep. February.

Hord, R.M., 1982. The Illiac-IV, The First Supercomputer. Computer Science Press, Rockville, MD.

Horel, T., Lauterbach, G., 1999. UltraSPARC-III: designing third-generation 64-bit performance. IEEE Micro 19 (3), 73–85.

Hospodor, A.D., Hoagland, A.S., et al., 1993. The changing nature of disk controllers. Proc. IEEE 81 (4), 586–594.

Hristea, C., Lenoski, D., Keen, J., 1997. Measuring memory hierarchy performance of cache-coherent multiprocessors using micro benchmarks. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 16–21, 1997, San Jose, CA.

Hsu, P., 1994. Designing the TFP microprocessor. IEEE Micro 18(2).

Huang, M., Wu, D., Yu, C.H., Fang, Z., Interlandi, M., Condie, T., Cong, J., 2016. Programming and runtime support to blaze FPGA accelerator deployment at datacenter scale. In: Proceedings of the Seventh ACM Symposium on Cloud Computing. ACM, pp. 456–469.

Huck, J., et al., 2000. Introducing the IA-64 Architecture. IEEE Micro 20 (5), 12–23.

Hughes, C.J., Kaul, P., Adve, S.V., Jain, R., Park, C., Srinivasan, J., 2001. Variability in the execution of multimedia applications and implications for architecture. In: Proceedings

of 28th Annual International Symposium on Computer Architecture (ISCA), June 30–July 4, 2001, Goteborg, Sweden, pp. 254–265.
Hwang, K., 1979. Computer Arithmetic: Principles, Architecture, and Design. Wiley, New York.
Hwang, K., 1993. Advanced Computer Architecture and Parallel Programming. McGraw-Hill, New York.
Hwu, W.-M., Patt, Y., 1986. HPSm, a high performance restricted data flow architecture having minimum functionality. In: Proceedings of 13th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1986, Tokyo, pp. 297–307.
Hwu, W.W., Mahlke, S.A., Chen, W.Y., Chang, P.P., Warter, N.J., Bringmann, R.A., Ouellette, R.O., Hank, R.E., Kiyohara, T., Haab, G.E., Holm, J.G., Lavery, D.M., 1993. The superblock: an effective technique for VLIW and superscalar compilation. J. Supercomput. 7 (1), 229–248.
Iandola, F., 2016. Exploring the Design Space of Deep Convolutional Neural Networks at Large Scale (Ph.D. dissertation). UC Berkeley.
IBM, 1982. The Economic Value of Rapid Response Time, GE20-0752-0. IBM, White Plains, NY, pp. 11–82.
IBM, 1990. The IBM RISC System/6000 processor. IBM J. Res. Dev. 34(1).
IBM, 1994. The PowerPC Architecture. Morgan Kaufmann, San Francisco.
IBM, 2005. Blue Gene. IBM J. Res. Dev. 49 (2/3) (Special issue).
IEEE, 1985. IEEE standard for binary floating-point arithmetic. SIGPLAN Notices 22 (2), 9–25.
IEEE, 2005. Intel virtualization technology, computer. IEEE Comput. Soc. 38 (5), 48–56.
IEEE 754-2008 Working Group, 2006. DRAFT Standard for Floating-Point Arithmetic 754-2008, https://doi.org/10.1109/IEEESTD.2008.4610935.
Ienne, P., Cornu, T., Kuhn, G., 1996. Special-purpose digital hardware for neural networks: an architectural survey. J. VLSI Signal Process. Syst. Signal Image Video Technol. 13(1).
Imprimis Product Specification, 97209 Sabre Disk Drive IPI-2 Interface 1.2 GB, Document No. 64402302, Imprimis, Dallas, TX.
InfiniBand Trade Association, 2001. InfiniBand Architecture Specifications Release 1.0.a. www.infinibandta.org.
Inoue, K., Ishihara, T., Murakami, K., 1999. Way-predicting set-associative cache for high performance and low energy consumption. In: Proc. 1999 International Symposium on Low Power Electronics and Design, ACM, pp. 273–275.
Intel, 2001. Using MMX Instructions to Convert RGB to YUV Color Conversion. cedar.intel.com/cgi-bin/ids.dll/content/content.jsp?cntKey=Legacy::irtm_AP548_9996&cntType=IDS_EDITORIAL.
Internet Retailer, 2005. The Gap launches a new site—after two weeks of downtime. Internet Retailer. http://www.internetretailer.com/2005/09/28/the-gap-launches-a-new-site-after-two-weeks-of-downtime.
Jain, R., 1991. The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. Wiley, New York.
Jantsch, A., Tenhunen, H. (Eds.), 2003. Networks on Chips. Kluwer Academic Publishers, The Netherlands.
Jimenez, D.A., Lin, C., 2001. Dynamic branch prediction with perceptrons. In: Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA '01). IEEE, Washington, DC, pp. 197–206.
Jimenez, D.A., Lin, C., 2002. Neural methods for dynamic branch prediction. ACM Trans. Comput. Syst. 20 (4), 369–397.

Johnson, M., 1990. Superscalar Microprocessor Design. Prentice Hall, Englewood Cliffs, NJ.

Jordan, H.F., 1983. Performance measurements on HEP—a pipelined MIMD computer. In: Proceedings of 10th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1982, Stockholm, Sweden, pp. 207–212.

Jordan, K.E., 1987. Performance comparison of large-scale scientific processors: scalar mainframes, mainframes with vector facilities, and supercomputers. Computer 20 (3), 10–23.

Jouppi, N.P., 1990. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In: Proceedings of 17th Annual International Symposium on Computer Architecture (ISCA), May 28–31, 1990, Seattle, WA, pp. 364–373.

Jouppi, N.P., 1998. Retrospective: Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In: 25 Years of the International Symposia on Computer Architecture (Selected Papers). ACM, New York, pp. 71–73.

Jouppi, N., 2016. Google supercharges machine learning tasks with TPU custom chip. https://cloudplatform.googleblog.com.

Jouppi, N.P., Wall, D.W., 1989. Available instruction-level parallelism for super-scalar and superpipelined processors. In: Proceedings of Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 3–6, 1989, Boston, pp. 272–282.

Jouppi, N.P., Wilton, S.J.E., 1994. Trade-offs in two-level on-chip caching. In: Proceedings of 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago, pp. 34–45.

Jouppi, N., Young, C., Patil, N., Patterson, D., Agrawal, G., et al., 2017. Datacenter performance analysis of a matrix processing unit. In: 44th International Symposium on Computer Architecture.

Kaeli, D.R., Emma, P.G., 1991. Branch history table prediction of moving target branches due to subroutine returns. In: Proceedings of 18th Annual International Symposium on Computer Architecture (ISCA), May 27–30, 1991, Toronto, Canada, pp. 34–42.

Kahan, W., 1968. 7094-II system support for numerical analysis, SHARE Secretarial Distribution SSD-159. Department of Computer Science, University of Toronto.

Kahan, J., 1990. On the advantage of the 8087's stack, unpublished course notes. Computer Science Division, University of California, Berkeley.

Kahaner, D.K., 1988. Benchmarks for 'real' programs. SIAM News. November.

Kahn, R.E., 1972. Resource-sharing computer communication networks. Proc. IEEE 60 (11), 1397–1407.

Kane, G., 1986. MIPS R2000 RISC Architecture. Prentice Hall, Englewood Cliffs, NJ.

Kane, G., 1996. PA-RISC 2.0 Architecture. Prentice Hall, Upper Saddle River, NJ.

Kane, G., Heinrich, J., 1992. MIPS RISC Architecture. Prentice Hall, Englewood Cliffs, NJ.

Kanev, S., Darago, J.P., Hazelwood, K., Ranganathan, P., Moseley, T., Wei, G.Y., Brooks, D., 2015. Profiling a warehouse-scale computer. In: ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA).

Karpathy, A., et al., 2014. Large-scale video classification with convolutional neural networks. CVPR.

Katz, R.H., Patterson, D.A., Gibson, G.A., 1989. Disk system architectures for high performance computing. Proc. IEEE 77 (12), 1842–1858.

Keckler, S.W., Dally, W.J., 1992. Processor coupling: integrating compile time and runtime scheduling for parallelism. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 202–213.

Keller, R.M., 1975. Look-ahead processors. ACM Comput. Surv. 7 (4), 177–195.

Keltcher, C.N., McGrath, K.J., Ahmed, A., Conway, P., 2003. The AMD Opteron processor for multiprocessor servers. IEEE Micro 23 (2), 66–76.
Kembel, R., 2000. Fibre channel: a comprehensive introduction. Internet Week. April.
Kermani, P., Kleinrock, L., 1979. Virtual cut-through: a new computer communication switching technique. Comput. Netw. 3, 267–286.
Kessler, R., 1999. The Alpha 21264 microprocessor. IEEE Micro 19 (2), 24–36.
Kilburn, T., Edwards, D.B.G., Lanigan, M.J., Sumner, F.H., 1962. One-level storage system. IRE Trans. Electron. Comput. EC-11, 223–235. Also appears in Siewiorek, D.P., Bell, C.G., Newell, A. 1982. Computer Structures: Principles and Examples. McGraw-Hill, New York. pp. 135–148.
Killian, E., 1991. MIPS R4000 technical overview–64 bits/100 MHz or bust. In: Hot Chips III Symposium Record, August 26–27, 1991, Stanford University, Palo Alto, CA. pp. 1.6–1.19.
Kim, M.Y., 1986. Synchronized disk interleaving. IEEE Trans. Comput. 35 (11), 978–988.
Kim, K., 2005. Technology for sub-50nm DRAM and NAND flash manufacturing. In: Electron Devices Meeting Technical Digest (December), pp. 323–326.
Kissell, K.D., 1997. MIPS16: High-density for the embedded market. In: Proceedings of Real Time Systems'97, June 15, 1997, Las Vegas, Nev. www.sgi.com/MIPS/arch/MIPS16/MIPS16.whitepaper.pdf.
Kitagawa, K., Tagaya, S., Hagihara, Y., Kanoh, Y., 2003. A hardware overview of SX-6 and SX-7 supercomputer. NEC Res. Dev. J. 44 (1), 2–7.
Knuth, D., 1981. second ed. The Art of Computer Programming, vol. II. Addison-Wesley, Reading, MA.
Kogge, P.M., 1981. The Architecture of Pipelined Computers. McGraw-Hill, New York.
Kohn, L., Fu, S.-W., 1989. A 1,000,000 transistor microprocessor. In: Proceedings of IEEE International Symposium on Solid State Circuits (ISSCC), February 15–17, 1989, New York, pp. 54–55.
Kohn, L., Margulis, N., 1989. Introducing the Intel i860 64-Bit Microprocessor. IEEE Micro 9 (4), 15–30.
Kontothanassis, L., Hunt, G., Stets, R., Hardavellas, N., Cierniak, M., Parthasarathy, S., Meira, W., Dwarkadas, S., Scott, M., 1997. VM-based shared memory on low-latency, remote-memory-access networks. In: Proceedings of 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO.
Koren, I., 1989. Computer Arithmetic Algorithms. Prentice Hall, Englewood Cliffs, NJ.
Kozyrakis, C., 2000. Vector IRAM: a media-oriented vector processor with embedded DRAM. In: Paper Presented at Hot Chips 12, August 13–15, 2000, Palo Alto, CA, pp. 13–15.
Kozyrakis, C., Patterson, D., 2002. Vector vs. superscalar and VLIW architectures for embedded multimedia benchmarks. In: Proceedings of 35th Annual International Symposium on Microarchitecture (MICRO-35), November 18–22, 2002, Istanbul, Turkey.
Krizhevsky, A., Sutskever, I., Hinton, G., 2012. Imagenet classification with deep convolutional neural networks. Adv. Neural Inf. Process. Syst.
Kroft, D., 1981. Lockup-free instruction fetch/prefetch cache organization. In: Proceedings of Eighth Annual International Symposium on Computer Architecture (ISCA), May 12–14, 1981, Minneapolis, MN, pp. 81–87.
Kroft, D., 1998. Retrospective: lockup-free instruction fetch/prefetch cache organization. In: 25 Years of the International Symposia on Computer Architecture (Selected Papers), ACM, New York, pp. 20–21.

Kuck, D., Budnik, P.P., Chen, S.-C., Lawrie, D.H., Towle, R.A., Strebendt, R.E., Davis Jr., E.W., Han, J., Kraska, P.W., Muraoka, Y., 1974. Measurements of parallelism in ordinary FORTRAN programs. Computer 7 (1), 37–46.

Kuhn, D.R., 1997. Sources of failure in the public switched telephone network. IEEE Comput. 30 (4), 31–36.

Kumar, A., 1997. The HP PA-8000 RISC CPU. IEEE Micro 17 (2), 27–32.

Kung, H.T., Leiserson, C.E., 1980. Algorithms for VLSI processor arrays. Introduction to VLSI systems.

Kunimatsu, A., Ide, N., Sato, T., Endo, Y., Murakami, H., Kamei, T., Hirano, M., Ishihara, F., Tago, H., Oka, M., Ohba, A., Yutaka, T., Okada, T., Suzuoki, M., 2000. Vector unit architecture for emotion synthesis. IEEE Micro 20 (2), 40–47.

Kunkel, S.R., Smith, J.E., 1986. Optimal pipelining in supercomputers. In: Proceedings of 13th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1986, Tokyo, pp. 404–414.

Kurose, J.F., Ross, K.W., 2001. Computer Networking: A Top-Down Approach Featuring the Internet. Addison-Wesley, Boston.

Kuskin, J., Ofelt, D., Heinrich, M., Heinlein, J., Simoni, R., Gharachorloo, K., Chapin, J., Nakahira, D., Baxter, J., Horowitz, M., Gupta, A., Rosenblum, M., Hennessy, J.L., 1994. The Stanford FLASH multiprocessor. In: Proceedings of 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago.

Lam, M., 1988. Software pipelining: an effective scheduling technique for VLIW processors. In: SIGPLAN Conference on Programming Language Design and Implementation, June 22–24, 1988, Atlanta, GA, pp. 318–328.

Lam, M.S., Wilson, R.P., 1992. Limits of control flow on parallelism. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 46–57.

Lam, M.S., Rothberg, E.E., Wolf, M.E., 1991. The cache performance and optimizations of blocked algorithms. In: Proceedings of Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 8–11, 1991, Santa Clara, CA. (SIGPLAN Notices 26:4 (April), 63–74).

Lambright, D., 2000. Experiences in measuring the reliability of a cache-based storage system. In: Proceedings of First Workshop on Industrial Experiences with Systems Software (WIESS 2000), Co-Located with the 4th Symposium on Operating Systems Design and Implementation (OSDI), October 22, 2000, San Diego, CA.

Lamport, L., 1979. How to make a multiprocessor computer that correctly executes multiprocess programs. IEEE Trans. Comput. C-28 (9), 241–248.

Landstrom, B., 2014. The Cost of Downtime. http://www.interxion.com/blogs/2014/07/the-cost-of-downtime/.

Lang, W., Patel, J.M., Shankar, S., 2010. Wimpy node clusters: what about non-wimpy workloads? In: Proceedings of Sixth International Workshop on Data Management on New Hardware (DaMoN), June 7, Indianapolis, IN.

Laprie, J.-C., 1985. Dependable computing and fault tolerance: concepts and terminology. In: Proceedings of 15th Annual International Symposium on Fault-Tolerant Computing, June 19–21, 1985, Ann Arbor, Mich, pp. 2–11.

Larabel, M., 2016. Google Looks To Open Up StreamExecutor To Make GPGPU Programming Easier. Phoronix, March 10. https://www.phoronix.com/.

Larson, E.R., 1973. Findings of fact, conclusions of law, and order for judgment, File No. 4-67, Civ. 138, Honeywell v. Sperry-Rand and Illinois Scientific Development, U.S. District Court for the State of Minnesota, Fourth Division (October 19).

Laudon, J., Lenoski, D., 1997. The SGI Origin: a ccNUMA highly scalable server. In: Proceedings of 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO, pp. 241–251.

Laudon, J., Gupta, A., Horowitz, M., 1994. Interleaving: a multithreading technique targeting multiprocessors and workstations. In: Proceedings of Sixth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 4–7, San Jose, CA, pp. 308–318.

Lauterbach, G., Horel, T., 1999. UltraSPARC-III: designing third generation 64-bit performance. IEEE Micro 19, 3 (May/June).

Lazowska, E.D., Zahorjan, J., Graham, G.S., Sevcik, K.C., 1984. Quantitative System Performance: Computer System Analysis Using Queueing Network Models. Prentice Hall, Englewood Cliffs, NJ (Although out of print, it is available online at www.cs.washington.edu/homes/lazowska/qsp/).

Lebeck, A.R., Wood, D.A., 1994. Cache profiling and the SPEC benchmarks: a case study. Computer 27 (10), 15–26.

Lee, R., 1989. Precision architecture. Computer 22 (1), 78–91.

Lee, W.V., et al., 2010. Debunking the 100X GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU. In: Proceedings of 37th Annual International Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

Lee, Y., Waterman, A., Cook, H., Zimmer, B., Keller, B., Puggelli, A., Kwak, J., Jevtic, R., Bailey, S., Blagojevic, M., Chiu, P.-F., Avizienis, R., Richards, B., Bachrach, J., Patterson, D., Alon, E., Nikolic, B., Asanovic, K., 2016. An agile approach to building RISC-V microprocessors. IEEE Micro 36 (2), 8–20.

Leighton, F.T., 1992. Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes. Morgan Kaufmann, San Francisco.

Leiner, A.L., 1954. System specifications for the DYSEAC. J. ACM 1 (2), 57–81.

Leiner, A.L., Alexander, S.N., 1954. System organization of the DYSEAC. IRE Trans. Electron. Comput. 3 (1), 1–10.

Leiserson, C.E., 1985. Fat trees: universal networks for hardware-efficient supercomputing. IEEE Trans. Comput. C-34 (10), 892–901.

Lenoski, D., Laudon, J., Gharachorloo, K., Gupta, A., Hennessy, J.L., 1990. The Stanford DASH multiprocessor. In: Proceedings of 17th Annual International Symposium on Computer Architecture (ISCA), May 28–31, 1990, Seattle, WA, pp. 148–159.

Lenoski, D., Laudon, J., Gharachorloo, K., Weber, W.-D., Gupta, A., Hennessy, J.L., Horowitz, M.A., Lam, M., 1992. The Stanford DASH multiprocessor. IEEE Comput. 25 (3), 63–79.

Levy, H., Eckhouse, R., 1989. Computer Programming and Architecture: The VAX. Digital Press, Boston.

Lewis-Kraus, G., 2016. The Great A.I. Awakening. New York Times Magazine..

Li, K., 1988. IVY: a shared virtual memory system for parallel computing. In: Proceedings of 1988 International Conference on Parallel Processing. Pennsylvania State University Press, University Park, PA.

Li, S., Chen, K., Brockman, J.B., Jouppi, N., 2011. Performance Impacts of Non-blocking Caches in Out-of-order Processors. HP Labs Tech Report HPL-2011-65 (full text available at http://Library.hp.com/techpubs/2011/Hpl-2011-65.html).

Lim, K., Ranganathan, P., Chang, J., Patel, C., Mudge, T., Reinhardt, S., 2008. Understanding and designing new system architectures for emerging warehouse-computing environments. In: Proceedings of 35th Annual International Symposium on Computer Architecture (ISCA), June 21–25, 2008, Beijing, China.

Lincoln, N.R., 1982. Technology and design trade offs in the creation of a modern supercomputer. IEEE Trans. Comput. C-31 (5), 363–376.

Lindholm, T., Yellin, F., 1999. The Java Virtual Machine Specification, 2nd ed. Addison-Wesley, Reading, MA (Also available online at java.sun.com/docs/books/vmspec/).

Lipasti, M.H., Shen, J.P., 1996. Exceeding the dataflow limit via value prediction. In: Proceedings of 29th International Symposium on Microarchitecture, December 2–4, 1996, Paris, France.

Lipasti, M.H., Wilkerson, C.B., Shen, J.P., 1996. Value locality and load value prediction. In: Proceedings of Seventh Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 1–5, 1996, Cambridge, MA, pp. 138–147.

Liptay, J.S., 1968. Structural aspects of the System/360 Model 85, Part II: The cache. IBM Syst. J. 7 (1), 15–21.

Lo, J., Eggers, S., Emer, J., Levy, H., Stamm, R., Tullsen, D., 1997. Converting thread-level parallelism into instruction-level parallelism via simultaneous multithreading. ACM Trans. Comput. Syst. 15 (2), 322–354.

Lo, J., Barroso, L., Eggers, S., Gharachorloo, K., Levy, H., Parekh, S., 1998. An analysis of database workload performance on simultaneous multithreaded processors. In: Proceedings of 25th Annual International Symposium on Computer Architecture (ISCA), July 3–14, 1998, Barcelona, Spain, pp. 39–50.

Lo, D., Cheng, L., Govindaraju, R., Barroso, L.A., Kozyrakis, C., 2014. Towards energy proportionality for large-scale latency-critical workloads. In: ACM/IEEE 41st Annual International Symposium on Computer Architecture (ISCA).

Loh, G.H., Hill, M.D., 2011. Efficiently enabling conventional block sizes for very large die-stacked DRAM caches. In: Proc. 44th Annual IEEE/ACM International Symposium on Microarchitecture, ACM, pp. 454–464.

Lovett, T., Thakkar, S., 1988. The symmetry multiprocessor system. In: Proceedings of 1988 International Conference of Parallel Processing, University Park, PA, pp. 303–310.

Lubeck, O., Moore, J., Mendez, R., 1985. A benchmark comparison of three supercomputers: Fujitsu VP-200, Hitachi S810/20, and Cray X-MP/2. Computer 18 (12), 10–24.

Luk, C.-K., Mowry, T.C., 1999. Automatic compiler-inserted prefetching for pointer-based applications. IEEE Trans. Comput. 48 (2), 134–141.

Lunde, A., 1977. Empirical evaluation of some features of instruction set processor architecture. Commun. ACM 20 (3), 143–152.

Luszczek, P., Dongarra, J.J., Koester, D., Rabenseifner, R., Lucas, B., Kepner, J., McCalpin, J., Bailey, D., Takahashi, D., 2005. Introduction to the HPC challenge benchmark suite. Lawrence Berkeley National Laboratory, Paper LBNL-57493 (April 25), repositories. cdlib.org/lbnl/LBNL-57493.

Maberly, N.C., 1966. Mastering Speed Reading. New American Library, New York.

Magenheimer, D.J., Peters, L., Pettis, K.W., Zuras, D., 1988. Integer multiplication and division on the HP precision architecture. IEEE Trans. Comput. 37 (8), 980–990.

Mahlke, S.A., Chen, W.Y., Hwu, W.-M., Rau, B.R., Schlansker, M.S., 1992. Sentinel scheduling for VLIW and superscalar processors. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston, pp. 238–247.

Mahlke, S.A., Hank, R.E., McCormick, J.E., August, D.I., Hwu, W.W., 1995. A comparison of full and partial predicated execution support for ILP processors. In: Proceedings of 22nd Annual International Symposium on Computer Architecture (ISCA), June 22–24, 1995, Santa Margherita, Italy, pp. 138–149.

Major, J.B., 1989. Are queuing models within the grasp of the unwashed? In: Proceedings of International Conference on Management and Performance Evaluation of Computer Systems, December 11–15, 1989, Reno, Nev, pp. 831–839.

Markstein, P.W., 1990. Computation of elementary functions on the IBM RISC System/6000 processor. IBM J. Res. Dev. 34 (1), 111–119.

Mathis, H.M., Mercias, A.E., McCalpin, J.D., Eickemeyer, R.J., Kunkel, S.R., 2005. Characterization of the multithreading (SMT) efficiency in Power5. IBM J. Res. Dev. 49 (4/5), 555–564.

McCalpin, J., 2005. STREAM: Sustainable Memory Bandwidth in High Performance Computers. www.cs.virginia.edu/stream/.

McCalpin, J., Bailey, D., Takahashi, D., 2005. Introduction to the HPC Challenge Benchmark Suite, Paper LBNL-57493. Lawrence Berkeley National Laboratory, University of California, Berkeley, repositories.cdlib.org/lbnl/LBNL-57493.

McCormick, J., Knies, A., 2002. A brief analysis of the SPEC CPU2000 benchmarks on the Intel Itanium 2 processor. In: Paper Presented at Hot Chips 14, August 18–20, 2002, Stanford University, Palo Alto, CA.

McFarling, S., 1989. Program optimization for instruction caches. In: Proceedings of Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 3–6, 1989, Boston, pp. 183–191.

McFarling, S., 1993. Combining Branch Predictors, WRL Technical Note TN-36, Digital Western Research Laboratory, Palo Alto, CA.

McFarling, S., Hennessy, J., 1986. Reducing the cost of branches. In: Proceedings of 13th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1986, Tokyo, pp. 396–403.

McGhan, H., O'Connor, M., 1998. PicoJava: a direct execution engine for Java bytecode. Computer 31 (10), 22–30.

McKeeman, W.M., 1967. Language directed computer design. In: Proceedings of AFIPS Fall Joint Computer Conference, November 14–16, 1967, Washington, DC, pp. 413–417.

McMahon, F.M., 1986. The Livermore FORTRAN Kernels: A Computer Test of Numerical Performance Range, Tech. Rep. UCRL-55745. Lawrence Livermore National Laboratory, University of California, Livermore.

McNairy, C., Soltis, D., 2003. Itanium 2 processor microarchitecture. IEEE Micro 23 (2), 44–55.

Mead, C., Conway, L., 1980. Introduction to VLSI Systems. Addison-Wesley, Reading, MA.

Mellor-Crummey, J.M., Scott, M.L., 1991. Algorithms for scalable synchronization on shared-memory multiprocessors. ACM Trans. Comput. Syst. 9 (1), 21–65.

Menabrea, L.F., 1842. Sketch of the analytical engine invented by Charles Babbage. Bibliothèque Universelle de Genève. 82.

Menon, A., Renato Santos, J., Turner, Y., Janakiraman, G., Zwaenepoel, W., 2005. Diagnosing performance overheads in the xen virtual machine environment. In: Proceedings of First ACM/USENIX International Conference on Virtual Execution Environments, June 11–12, 2005, Chicago, pp. 13–23.

Merlin, P.M., Schweitzer, P.J., 1980. Deadlock avoidance in store-and-forward networks. Part I. Store-and-forward deadlock. IEEE Trans. Commun. 28 (3), 345–354.

Metcalfe, R.M., 1993. Computer/network interface design: lessons from Arpanet and Ethernet. IEEE J. Sel. Area. Commun. 11 (2), 173–180.

Metcalfe, R.M., Boggs, D.R., 1976. Ethernet: distributed packet switching for local computer networks. Commun. ACM 19 (7), 395–404.

Metropolis, N., Howlett, J., Rota, G.C. (Eds.), 1980. A History of Computing in the Twentieth Century. Academic Press, New York.

Meyer, R.A., Seawright, L.H., 1970. A virtual machine time sharing system. IBM Syst. J. 9 (3), 199–218.

Meyers, G.J., 1978. The evaluation of expressions in a storage-to-storage architecture. Comput. Architect. News 7 (3), 20–23.

Meyers, G.J., 1982. Advances in Computer Architecture, second ed. Wiley, New York.

Micron, 2004. Calculating Memory System Power for DDR2. http://download.micron.com/pdf/pubs/designline/dl1Q04.pdf.

Micron, 2006. The Micron System-Power Calculator. http://www.micron.com/-systemcalc.
MIPS, 1997. MIPS16 Application Specific Extension Product Description. www.sgi.com/MIPS/arch/MIPS16/mips16.pdf.
Miranker, G.S., Rubenstein, J., Sanguinetti, J., 1988. Squeezing a Cray-class supercomputer into a single-user package. In: Proceedings of IEEE COMPCON, February 29–March 4, 1988, San Francisco, pp. 452–456.
Mitchell, D., 1989. The transputer: the time is now. Comput. Des. (RISC suppl.) 40–41.
Mitsubishi, 1996. Mitsubishi 32-Bit Single Chip Microcomputer M32R Family Software Manual. Mitsubishi, Cypress, CA.
Miura, K., Uchida, K., 1983. FACOM vector processing system: VP100/200. In: Proceedings of NATO Advanced Research Workshop on High-Speed Computing, June 20–22, 1983, Jülich, West Germany. Also appears in Hwang, K. (Ed.), 1984. Superprocessors: Design and Applications. IEEE (August), pp. 59–73.
Miya, E.N., 1985. Multiprocessor/distributed processing bibliography. Comput. Architect. News 13 (1), 27–29.
Money, M.S.N., 2005. Amazon Shares Tumble after Rally Fizzles. http://moneycentral.msn.com/content/CNBCTV/Articles/Dispatches/P133695.asp.
Montoye, R.K., Hokenek, E., Runyon, S.L., 1990. Design of the IBM RISC System/6000 floating-point execution. IBM J. Res. Dev. 34 (1), 59–70.
Moore, G.E., 1965. Cramming more components onto integrated circuits. Electronics 38 (8), 114–117.
Moore, B., Padegs, A., Smith, R., Bucholz, W., 1987. Concepts of the System/370 vector architecture. In: 14th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1987, Pittsburgh, PA, pp. 282–292.
Morgan, T., 2014. A rare peek into the massive scale of AWS. Enterprise Tech. https://www.enterprisetech.com/2014/11/14/rare-peek-massive-scale-aws/.
Morgan, T., 2016. How long can AWS keep climbing its steep growth curve? https://www.nextplatform.com/2016/02/01/how-long-can-aws-keep-climbingits-steep-growth-curve/.
Morse, S., Ravenal, B., Mazor, S., Pohlman, W., 1980. Intel microprocessors—8080 to 8086. Computer 13, 10.
Moshovos, A., Sohi, G.S., 1997. Streamlining inter-operation memory communication via data dependence prediction. In: Proceedings of 30th Annual International Symposium on Microarchitecture, December 1–3, Research Triangle Park, NC, pp. 235–245.
Moshovos, A., Breach, S., Vijaykumar, T.N., Sohi, G.S., 1997. Dynamic speculation and synchronization of data dependences. In: 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO.
Moussouris, J., Crudele, L., Freitas, D., Hansen, C., Hudson, E., Przybylski, S., Riordan, T., Rowen, C., 1986. A CMOS RISC processor with integrated system functions. In: Proceedings of IEEE COMPCON, March 3–6, 1986, San Francisco, p. 191.
Mowry, T.C., Lam, S., Gupta, A., 1992. Design and evaluation of a compiler algorithm for prefetching. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston (SIGPLAN Notices 27:9 (September), pp. 62–73.
Muchnick, S.S., 1988. Optimizing compilers for SPARC. Sun Technol. 1 (3), 64–77.
Mueller, M., Alves, L.C., Fischer, W., Fair, M.L., Modi, I., 1999. RAS strategy for IBM S/390 G5 and G6. IBM J. Res. Dev. 43 (5-6), 875–888.
Mukherjee, S.S., Weaver, C., Emer, J.S., Reinhardt, S.K., Austin, T.M., 2003. Measuring architectural vulnerability factors. IEEE Micro 23 (6), 70–75.
Murphy, B., Gent, T., 1995. Measuring system and software reliability using an automated data collection process. Qual. Reliab. Eng. Int. 11 (5), 341–353.

Myer, T.H., Sutherland, I.E., 1968. On the design of display processors. Commun. ACM 11 (6), 410–414.
Narayanan, D., Thereska, E., Donnelly, A., Elnikety, S., Rowstron, A., 2009. Migrating server storage to SSDs: analysis of trade-offs. In: Proceedings of 4th ACM European Conference on Computer Systems, April 1–3, 2009, Nuremberg, Germany.
National Research Council, 1997. The Evolution of Untethered Communications. Computer Science and Telecommunications Board, National Academy Press, Washington, DC.
National Storage Industry Consortium, 1998. Tape Roadmap. www.nsic.org.
Nelson, V.P., 1990. Fault-tolerant computing: fundamental concepts. Computer 23 (7), 19–25.
Ngai, T.-F., Irwin, M.J., 1985. Regular, area-time efficient carry-lookahead adders. In: Proceedings of Seventh IEEE Symposium on Computer Arithmetic, June 4–6, 1985, University of Illinois, Urbana, pp. 9–15.
Nicolau, A., Fisher, J.A., 1984. Measuring the parallelism available for very long instruction word architectures. IEEE Trans. Comput. C33 (11), 968–976.
Nielsen, M., 2016. Neural Networks and Deep Learning. http://neuralnetworksanddeeplearning.com/.
Nikhil, R.S., Papadopoulos, G.M., Arvind, 1992. *T: a multithreaded massively parallel architecture. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 156–167.
Noordergraaf, L., van der Pas, R., 1999. Performance experiences on Sun's WildFire prototype. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 13–19, 1999, Portland, Ore.
Nvidia, 2016. Tesla GPU Accelerators For Servers. http:// www.nvidia.com/object/tesla-servers.html.
Nyberg, C.R., Barclay, T., Cvetanovic, Z., Gray, J., Lomet, D., 1994. AlphaSort: a RISC machine sort. In: Proceedings of ACM SIGMOD, May 24–27, 1994, Minneapolis, Minn.
Oka, M., Suzuoki, M., 1999. Designing and programming the emotion engine. IEEE Micro 19 (6), 20–28.
Okada, S., Okada, S., Matsuda, Y., Yamada, T., Kobayashi, A., 1999. System on a chip for digital still camera. IEEE Trans. Consum. Electron. 45 (3), 584–590.
Oliker, L., Canning, A., Carter, J., Shalf, J., Ethier, S., 2004. Scientific computations on modern parallel vector systems. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 6–12, 2004, Pittsburgh, Penn, p. 10.
Olofsson, A., 2011. Debunking the myth of the $100M ASIC. EE Times. http://www.eetimes.com/author.asp?section_id=36&doc_id=1266014.
Ovtcharov, K., Ruwase, O., Kim, J.Y., Fowers, J., Strauss, K., Chung, E.S., 2015a. Accelerating deep convolutional neural networks using specialized hardware. Microsoft Research Whitepaper. https://www.microsoft.com/en-us/research/publication/accelerating-deep-convolutional-neural-networks-using-specialized-hardware/.
Ovtcharov, K., Ruwase, O., Kim, J.Y., Fowers, J., Strauss, K., Chung, E.S., 2015b. Toward accelerating deep learning at scale using specialized hardware in the datacenter. In: 2015 IEEE Hot Chips 27 Symposium.
Pabst, T., 2000. Performance Showdown at 133 MHz FSB—The Best Platform for Coppermine. www6.tomshardware.com/mainboard/00q1/000302/.
Padua, D., Wolfe, M., 1986. Advanced compiler optimizations for supercomputers. Commun. ACM 29 (12), 1184–1201.
Palacharla, S., Kessler, R.E., 1994. Evaluating stream buffers as a secondary cache replacement. In: Proceedings of 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago, pp. 24–33.
Palmer, J., Morse, S., 1984. The 8087 Primer. John Wiley & Sons, New York, p. 93.

Pan, S.-T., So, K., Rameh, J.T., 1992. Improving the accuracy of dynamic branch prediction using branch correlation. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston, pp. 76–84.

Partridge, C., 1994. Gigabit Networking. Addison-Wesley, Reading, MA.

Patterson, D., 1985. Reduced instruction set computers. Commun. ACM 28 (1), 8–21.

Patterson, D., 2004. Latency lags bandwidth. Commun. ACM 47 (10), 71–75.

Patterson, D.A., Ditzel, D.R., 1980. The case for the reduced instruction set computer. ACM SIGARCH Comput. Architect. News 8 (6), 25–33.

Patterson, D.A., Hennessy, J.L., 2004. Computer Organization and Design: The Hardware/Software Interface, third ed. Morgan Kaufmann, San Francisco.

Patterson, D., Nikolić, B., 7/25/2015, Agile Design for Hardware, Parts I, II, and III. EE Times, http://www.eetimes.com/author.asp?doc_id=1327239.

Patterson, D.A., Garrison, P., Hill, M., Lioupis, D., Nyberg, C., Sippel, T., Van Dyke, K., 1983. Architecture of a VLSI instruction cache for a RISC. In: 10th Annual International Conference on Computer Architecture Conf. Proc., June 13–16, 1983, Stockholm, Sweden, pp. 108–116.

Patterson, D.A., Gibson, G.A., Katz, R.H., 1987. A Case for Redundant Arrays of Inexpensive Disks (RAID), Tech. Rep. UCB/CSD 87/391, University of California, Berkeley. Also appeared in Proc. ACM SIGMOD, June 1–3, 1988, Chicago, pp. 109–116.

Pavan, P., Bez, R., Olivo, P., Zanoni, E., 1997. Flash memory cells—an overview. Proc. IEEE 85 (8), 1248–1271.

Peh, L.S., Dally, W.J., 2001. A delay model and speculative architecture for pipe-lined routers. In: Proceedings of 7th International Symposium on High-Performance Computer Architecture, January 22–24, 2001, Monterrey, Mexico.

Peng, V., Samudrala, S., Gavrielov, M., 1987. On the implementation of shifters, multipliers, and dividers in VLSI floating point units. In: Proceedings of 8th IEEE Symposium on Computer Arithmetic, May 19–21, 1987, Como, Italy, pp. 95–102.

Pfister, G.F., 1998. In Search of Clusters, second ed. Prentice Hall, Upper Saddle River, NJ.

Pfister, G.F., Brantley, W.C., George, D.A., Harvey, S.L., Kleinfekder, W.J., McAuliffe, K.P., Melton, E.A., Norton, V.A., Weiss, J., 1985. The IBM research parallel processor prototype (RP3): introduction and architecture. In: Proceedings of 12th Annual International Symposium on Computer Architecture (ISCA), June 17–19, 1985, Boston, MA, pp. 764–771.

Pinheiro, E., Weber, W.D., Barroso, L.A., 2007. Failure trends in a large disk drive population. In: Proceedings of 5th USENIX Conference on File and Storage Technologies (FAST '07), February 13–16, 2007, San Jose, CA.

Pinkston, T.M., 2004. Deadlock characterization and resolution in interconnection networks. In: Zhu, M.C., Fanti, M.P. (Eds.), Deadlock Resolution in Computer-Integrated Systems. CRC Press, Boca Raton, FL, pp. 445–492.

Pinkston, T.M., Shin, J., 2005. Trends toward on-chip networked microsystems. Int. J. High Perform. Comput. Netw. 3 (1), 3–18.

Pinkston, T.M., Warnakulasuriya, S., 1997. On deadlocks in interconnection networks. In: 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO.

Pinkston, T.M., Benner, A., Krause, M., Robinson, I., Sterling, T., 2003. InfiniBand: the 'de facto' future standard for system and local area networks or just a scalable replacement for PCI buses?". Cluster Comput. 6 (2), 95–104 (Special issue on communication architecture for clusters).

Postiff, M.A., Greene, D.A., Tyson, G.S., Mudge, T.N., 1999. The limits of instruction level parallelism in SPEC95 applications. Comput. Architect. News 27 (1), 31–40.

Prabhakar, R., Koeplinger, D., Brown, K.J., Lee, H., De Sa, C., Kozyrakis, C., Olukotun, K., 2016. Generating configurable hardware from parallel patterns. In: Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, pp. 651–665.

Prakash, T.K., Peng, L., 2008. Performance characterization of spec cpu2006 benchmarks on intel core 2 duo processor. ISAST Trans. Comput. Softw. Eng. 2 (1), 36–41.

Przybylski, S.A., 1990. Cache Design: A Performance-Directed Approach. Morgan Kaufmann, San Francisco.

Przybylski, S.A., Horowitz, M., Hennessy, J.L., 1988. Performance trade-offs in cache design. In: 15th Annual International Symposium on Computer Architecture, May 30–June 2, 1988, Honolulu, Hawaii, pp. 290–298.

Puente, V., Beivide, R., Gregorio, J.A., Prellezo, J.M., Duato, J., Izu, C., 1999. Adaptive bubble router: a design to improve performance in torus networks. In: Proceedings of the 28th International Conference on Parallel Processing, September 21–24, 1999, Aizu-Wakamatsu, Fukushima, Japan.

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2014. A reconfigurable fabric for accelerating large-scale datacenter services. In: 41st International Symposium on Computer Architecture.

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2015. A reconfigurable fabric for accelerating large-scale datacenter services. IEEE Micro. 35(3).

Putnam, A., Caulfield, A.M., Chung, E.S., Chiou, D., Constantinides, K., Demme, J., Esmaeilzadeh, H., Fowers, J., Gopal, G.P., Gray, J., Haselman, M., Hauck, S., Heil, S., Hormati, A., Kim, J.-Y., Lanka, S., Larus, J., Peterson, E., Pope, S., Smith, A., Thong, J., Xiao, P.Y., Burger, D., 2016. A reconfigurable fabric for accelerating large-scale datacenter services. Commun. ACM. 59 (11), 114–122.

Qadeer, W., Hameed, R., Shacham, O., Venkatesan, P., Kozyrakis, C., Horowitz, M.A., 2015. Convolution engine: balancing efficiency & flexibility in specialized computing. Commun. ACM 58(4).

Qureshi, M.K., Loh, G.H., 2012. Fundamental latency trade-off in architecting dram caches: Outperforming impractical sram-tags with a simple and practical design. In: Proc. 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture, IEEE Computer Society, pp. 235–246.

Radin, G., 1982. The 801 minicomputer. In: Proceedings of Symposium Architectural Support for Programming Languages and Operating Systems (ASPLOS), March 1–3, 1982, Palo Alto, CA, pp. 39–47.

Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., Amarasinghe, S., 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. ACM SIGPLAN Not. 48 (6), 519–530.

Ramacher, U., Beichter, J., Raab, W., Anlauf, J., Bruels, N., Hachmann, A., Wesseling, M., 1991. Design of a 1st generation neurocomputer. VLSI Design of Neural Networks. Springer, USA.

Ramamoorthy, C.V., Li, H.F., 1977. Pipeline architecture. ACM Comput. Surv. 9 (1), 61–102.

Ranganathan, P., Leech, P., Irwin, D., Chase, J., 2006. Ensemble-level power management for dense blade servers. In: Proceedings of 33rd Annual International Symposium on Computer Architecture (ISCA), June 17–21, 2006, Boston, MA, pp. 66–77.

Rau, B.R., 1994. Iterative modulo scheduling: an algorithm for software pipelining loops. In: Proceedings of 27th Annual International Symposium on Microarchitecture, November 30–December 2, 1994, San Jose, CA, pp. 63–74.

Rau, B.R., Fisher, J.A., 1993. Instruction-level parallelism. J. Supercomput. 235, Springer Science & Business Media.

Rau, B.R., Glaeser, C.D., Picard, R.L., 1982. Efficient code generation for horizontal architectures: compiler techniques and architectural support. In: Proceedings of Ninth Annual International Symposium on Computer Architecture (ISCA), April 26–29, 1982, Austin, TX, pp. 131–139.

Rau, B.R., Yen, D.W.L., Yen, W., Towle, R.A., 1989. The Cydra 5 departmental supercomputer: design philosophies, decisions, and trade-offs. IEEE Comput. 22 (1), 12–34.

Reddi, V.J., Lee, B.C., Chilimbi, T., Vaid, K., 2010. Web search using mobile cores: quantifying and mitigating the price of efficiency. In: Proceedings of 37th Annual International Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

Redmond, K.C., Smith, T.M., 1980. Project Whirlwind—The History of a Pioneer Computer. Digital Press, Boston.

Reinhardt, S.K., Larus, J.R., Wood, D.A., 1994. Tempest and typhoon: user-level shared memory. In: 21st Annual International Symposium on Computer Architecture (ISCA), April 18–21, 1994, Chicago, pp. 325–336.

Reinman, G., Jouppi, N.P., 1999. Extensions to CACTI. research.compaq.com/wrl/people/jouppi/CACTI.html.

Rettberg, R.D., Crowther, W.R., Carvey, P.P., Towlinson, R.S., 1990. The Monarch parallel processor hardware design. IEEE Comput. 23 (4), 18–30.

Riemens, A., Vissers, K.A., Schutten, R.J., Sijstermans, F.W., Hekstra, G.J., La Hei, G.D., 1999. Trimedia CPU64 application domain and benchmark suite. In: Proceedings of IEEE International Conference on Computer Design: VLSI in Computers and Processors (ICCD'99), October 10–13, 1999, Austin, TX, pp. 580–585.

Riseman, E.M., Foster, C.C., 1972. Percolation of code to enhance parallel dispatching and execution. IEEE Trans. Comput. C-21 (12), 1411–1415.

Robin, J., Irvine, C., 2000. Analysis of the Intel Pentium's ability to support a secure virtual machine monitor. In: Proceedings of USENIX Security Symposium, August 14–17, 2000, Denver, CO.

Robinson, B., Blount, L., 1986. The VM/HPO 3880-23 Performance Results, IBM Tech. Bulletin GG66-0247-00. IBM Washington Systems Center, Gaithersburg, MD.

Ropers, A., Lollman, H.W., Wellhausen, J., 1999. DSPstone: Texas Instruments TMS320C54x, Tech. Rep. IB 315 1999/9-ISS-Version 0.9. Aachen University of Technology, Aachen, Germany (www.ert.rwth-aachen.de/Projekte/Tools/coal/dspstone_c54x/index.html).

Rosenblum, M., Herrod, S.A., Witchel, E., Gupta, A., 1995. Complete computer simulation: the SimOS approach. IEEE Parallel Distrib. Technol. 4 (3), 34–43.

Rowen, C., Johnson, M., Ries, P., 1988. The MIPS R3010 floating-point coprocessor. IEEE Micro 8 (3), 53–62.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., 2015. Imagenet large scale visual recognition challenge. Int. J. Comput. Vis. 115(3).

Russell, R.M., 1978. The Cray-1 processor system. Commun. ACM 21 (1), 63–72.

Rymarczyk, J., 1982. Coding guidelines for pipelined processors. In: Proceeding of Symposium Architectural Support for Programming Languages and Operating Systems (ASPLOS), March 1–3, 1982, Palo Alto, CA, pp. 12–19.

Saavedra-Barrera, R.H., 1992. CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking (Ph.D. dissertation). University of California, Berkeley.

Salem, K., Garcia-Molina, H., 1986. Disk striping. In: Proceedings of 2nd International IEEE Conference on Data Engineering, February 5–7, 1986, Washington, DC, pp. 249–259.

Saltzer, J.H., Reed, D.P., Clark, D.D., 1984. End-to-end arguments in system design. ACM Trans. Comput. Syst. 2 (4), 277–288.

Samples, A.D., Hilfinger, P.N., 1988. Code Reorganization for Instruction Caches, Tech. Rep. UCB/CSD 88/447, University of California, Berkeley.

Santoro, M.R., Bewick, G., Horowitz, M.A., 1989. Rounding algorithms for IEEE multipliers. In: Proceedings of Ninth IEEE Symposium on Computer Arithmetic, September 6–8, Santa Monica, CA, pp. 176–183.

Satran, J., Smith, D., Meth, K., Sapuntzakis, C., Wakeley, M., Von Stamwitz, P., Haagens, R., Zeidner, E., Dalle Ore, L., Klein, Y., 2001. "iSCSI," IPS Working Group of IETF, Internet draft. www.ietf.org/internet-drafts/draft-ietf-ips-iscsi-07.txt.

Saulsbury, A., Wilkinson, T., Carter, J., Landin, A., 1995. An argument for simple COMA. In: Proceedings of First IEEE Symposium on High-Performance Computer Architectures, January 22–25, 1995, Raleigh, NC, pp. 276–285.

Schneck, P.B., 1987. Superprocessor Architecture. Kluwer Academic Publishers, Norwell, MA.

Schroeder, B., Gibson, G.A., 2007. Understanding failures in petascale computers. J. Phys. Conf. Ser. 78 (1), 188–198.

Schroeder, B., Pinheiro, E., Weber, W.-D., 2009. DRAM errors in the wild: a large-scale field study. In: Proceedings of Eleventh International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS), June 15–19, 2009, Seattle, WA.

Schurman, E., Brutlag, J., 2009. The user and business impact of server delays. In: Proceedings of Velocity: Web Performance and Operations Conference, June 22–24, 2009, San Jose, CA.

Schwartz, J.T., 1980. Ultracomputers. ACM Trans. Program. Lang. Syst. 4 (2), 484–521.

Scott, N.R., 1985. Computer Number Systems and Arithmetic. Prentice Hall, Englewood Cliffs, NJ.

Scott, S.L., 1996. Synchronization and communication in the T3E multiprocessor. In: Seventh International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 1–5, 1996, Cambridge, MA.

Scott, S.L., Goodman, J., 1994. The impact of pipelined channels on *k*-ary *n*-cube networks. IEEE Trans. Parallel Distrib. Syst. 5 (1), 1–16.

Scott, S.L., Thorson, G.M., 1996. The Cray T3E network: adaptive routing in a high performance 3D torus. In: Proceedings of IEEE HOT Interconnects '96, August 15–17, 1996, Stanford University, Palo Alto, CA, pp. 14–156.

Scranton, R.A., Thompson, D.A., Hunter, D.W., 1983. The Access Time Myth. Tech. Rep. RC 10197 (45223). IBM, Yorktown Heights, NY.

Seagate, 2000. Seagate Cheetah 73 Family: ST173404LW/LWV/LC/LCV Product Manual, vol. 1. Seagate, Scotts Valley, CA. www.seagate.com/support/disc/manuals/scsi/29478b.pdf.

Seitz, C.L., 1985. The Cosmic Cube (concurrent computing). Commun. ACM 28 (1), 22–33.

Senior, J.M., 1993. Optical Fiber Communications: Principles and Practice, second ed. Prentice Hall, Hertfordshire, UK.

Sergio Guadarrama, 2015. BVLC googlenet. https://github.com/BVLC/caffe/tree/ master/models/bvlc_googlenet.

Seznec, A., Michaud, P., 2006. A case for (partially) TAgged GEometric history length branch prediction. J. Instruction Level Parallel. 8, 1–23.

Shao, Y.S., Brooks, D., 2015. Research infrastructures for hardware accelerators. Synth. Lect. Comput. Architect. 10 (4), 1–99.

Sharangpani, H., Arora, K., 2000. Itanium processor microarchitecture. IEEE Micro 20 (5), 24–43.

Shurkin, J., 1984. Engines of the Mind: A History of the Computer. W.W. Norton, New York.

Shustek, L.J., 1978. Analysis and Performance of Computer Instruction Sets (Ph.D. dissertation). Stanford University, Palo Alto, CA.

Silicon Graphics, 1996. MIPS V Instruction Set. http://www.sgi.com/MIPS/arch/ISA5/#MIPSV_indx.

Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., 2016. Mastering the game of Go with deep neural networks and tree search. Nature 529(7587).

Singh, J.P., Hennessy, J.L., Gupta, A., 1993. Scaling parallel programs for multiprocessors: methodology and examples. In: Computer, 2. 7, pp. 22–33.

Singh, A., Ong, J., Agarwal, A., Anderson, G., Armistead, A., Bannon, R., Boving, S., Desai, G., Felderman, B., Germano, P., Kanagala, A., Provost, J., Simmons, J., Eiichi Tanda, E., Wanderer, J., Hölzle, U., Stuart, S., Vahdat, A., 2015. Jupiter rising: a decade of CLOS topologies and centralized control in Google's datacenter network. ACM SIGCOMM Comput. Commun. Rev. 45 (4), 183–197.

Sinharoy, B., Koala, R.N., Tendler, J.M., Eickemeyer, R.J., Joyner, J.B., 2005. POWER5 system microarchitecture. IBM J. Res. Dev. 49 (4–5), 505–521.

Sites, R., 1979. Instruction Ordering for the CRAY-1 Computer, Tech. Rep. 78-CS-023. Dept. of Computer Science, University of California, San Diego.

Sites, R.L. (Ed.), 1992. Alpha Architecture Reference Manual. Digital Press, Burlington, MA.

Sites, R.L., Witek, R. (Eds.), 1995. Alpha Architecture Reference Manual, second ed. Digital Press, Newton, MA.

Skadron, K., Clark, D.W., 1997. Design issues and tradeoffs for write buffers. In: Proceedings of Third International Symposium on High-Performance Computer Architecture, February 1–5, 1997, San Antonio, TX, pp. 144–155.

Skadron, K., Ahuja, P.S., Martonosi, M., Clark, D.W., 1999. Branch prediction, instruction-window size, and cache size: performance tradeoffs and simulation techniques. IEEE Trans. Comput. 48(11).

Slater, R., 1987. Portraits in Silicon. MIT Press, Cambridge, MA.

Slotnick, D.L., Borck, W.C., McReynolds, R.C., 1962. The Solomon computer. In: Proceedings of AFIPS Fall Joint Computer Conference, December 4–6, 1962, Philadelphia, PA, pp. 97–107.

Smith, B.J., 1978. A pipelined, shared resource MIMD computer. In: Proceedings of International Conference on Parallel Processing (ICPP), August, Bellaire, MI, pp. 6–8.

Smith, B.J., 1981a. Architecture and applications of the HEP multiprocessor system. Real Time Signal Process. IV 298, 241–248.

Smith, J.E., 1981b. A study of branch prediction strategies. In: Proceedings of Eighth Annual International Symposium on Computer Architecture (ISCA), May 12–14, 1981, Minneapolis, MN, pp. 135–148.

Smith, A.J., 1982a. Cache memories. Comput. Surv., 14, 3, pp. 473–530.

Smith, J.E., 1982b. Decoupled access/execute computer architectures. In: Proceedings of the 11th International Symposium on Computer Architecture.

Smith, J.E., 1984. Decoupled access/execute computer architectures. ACM Trans. Comput. Syst. 2 (4), 289–308.
Smith, J.E., 1988. Characterizing computer performance with a single number. Commun. ACM 31 (10), 1202–1206.
Smith, J.E., 1989. Dynamic instruction scheduling and the Astronautics ZS-1. Computer 22 (7), 21–35.
Smith, J.E., Goodman, J.R., 1983. A study of instruction cache organizations and replacement policies. In: Proceedings of 10th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1982, Stockholm, Sweden, pp. 132–137.
Smith, A., Lee, J., 1984. Branch prediction strategies and branch-target buffer design. Computer 17 (1), 6–22.
Smith, J.E., Pleszkun, A.R., 1988. Implementing precise interrupts in pipelined processors. IEEE Trans. Comput. 37 (5), 562–573. (This paper is based on an earlier paper that appeared in Proceedings of the 12th Annual International Symposium on Computer Architecture (ISCA), June 17–19, 1985, Boston, MA.
Smith, J.E., Dermer, G.E., Vanderwarn, B.D., Klinger, S.D., Rozewski, C.M., Fowler, D.L., Scidmore, K.R., Laudon, J.P., 1987. The ZS-1 central processor. In: Proceedings of Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 5–8, 1987, Palo Alto, CA, pp. 199–204.
Smith, M.D., Johnson, M., Horowitz, M.A., 1989. Limits on multiple instruction issue. In: Proceedings of Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 3–6, 1989, Boston, pp. 290–302.
Smith, M.D., Horowitz, M., Lam, M.S., 1992. Efficient superscalar performance through boosting. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston, pp. 248–259.
Smotherman, M., 1989. A sequencing-based taxonomy of I/O systems and review of historical machines. Comput. Architect. News 17 (5), 5–15. Reprinted in Computer Architecture Readings, Hill, M.D., Jouppi, N.P., Sohi, G.S. (Eds.), 1999. Morgan Kaufmann, San Francisco, pp. 451–461.
Sodani, A., Sohi, G., 1997. Dynamic instruction reuse. In: Proceedings of 24th Annual International Symposium on Computer Architecture (ISCA), June 2–4, 1997, Denver, CO.
Sohi, G.S., 1990. Instruction issue logic for high-performance, interruptible, multiple functional unit, pipelined computers. IEEE Trans. Comput. 39 (3), 349–359.
Sohi, G.S., Vajapeyam, S., 1989. Tradeoffs in instruction format design for horizontal architectures. In: Proceedings of Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 3–6, 1989, Boston, pp. 15–25.
Sony/Toshiba, 1999. 'Emotion Engine' in PS2 ("IPU is basically an MPEG2 decoder…"). http://www.cpu-collection.de/?l0=co&l1=Sony&l2=Emotion+Engine http://arstechnica.com/gadgets/2000/02/ee/3/.
Soundararajan, V., Heinrich, M., Verghese, B., Gharachorloo, K., Gupta, A., Hennessy, J.L., 1998. Flexible use of memory for replication/migration in cache-coherent DSM multiprocessors. In: Proceedings of 25th Annual International Symposium on Computer Architecture (ISCA), July 3–14, 1998, Barcelona, Spain, pp. 342–355.
SPEC, 1989. SPEC Benchmark Suite Release 1.0 (October 2).
SPEC, 1994. SPEC Newsletter (June).

Sporer, M., Moss, F.H., Mathais, C.J., 1988. An introduction to the architecture of the Stellar Graphics supercomputer. In: Proceedings of IEEE COMPCON, February 29–March 4, 1988, San Francisco, p. 464.

Spurgeon, C., 2001. Charles Spurgeon's Ethernet Web Site. www.host.ots.utexas.edu/ethernet/ethernet-home.html.

Steinberg, D., 2015. Full-Chip Simulations, Keys to Success. In: Proceedings of the Synopsys Users Group (SNUG) Silicon Valley 2015.

Stenström, P., Joe, T., Gupta, A., 1992. Comparative performance evaluation of cache-coherent NUMA and COMA architectures. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 80–91.

Sterling, T., 2001. Beowulf PC Cluster Computing with Windows and Beowulf PC Cluster Computing with Linux. MIT Press, Cambridge, MA.

Stern, N., 1980. Who invented the first electronic digital computer? Ann. Hist. Comput. 2 (4), 375–376.

Stevens, W.R., 1994–1996. TCP/IP Illustrated (three volumes). Addison-Wesley, Reading, MA.

Stokes, J., 2000. Sound and Vision: A Technical Overview of the Emotion Engine. arstechnica.com/reviews/1q00/playstation2/ee-1.html.

Stone, H., 1991. High Performance Computers. Addison-Wesley, New York.

Strauss, W., 1998. DSP Strategies 2002. www.usadata.com/market_research/spr_05/spr_r127-005.htm.

Strecker, W.D., 1976. Cache memories for the PDP-11? In: Proceedings of Third Annual International Symposium on Computer Architecture (ISCA), January 19–21, 1976, Tampa, FL, pp. 155–158.

Strecker, W.D., 1978. VAX-11/780: a virtual address extension of the PDP-11 family. In: Proceedings of AFIPS National Computer Conference, June 5–8, 1978, Anaheim, CA. vol. 47, pp. 967–980.

Sugumar, R.A., Abraham, S.G., 1993. Efficient simulation of caches under optimal replacement with applications to miss characterization. In: Proceedings of ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems, May 17–21, 1993, Santa Clara, CA, pp. 24–35.

Sun Microsystems, 1989. The SPARC Architectural Manual, Version 8, Part No. 8001399-09. Sun Microsystems, Santa Clara, CA.

Sussenguth, E., 1999. IBM's ACS-1 machine. IEEE Comput. 22, 11.

Swan, R.J., Bechtolsheim, A., Lai, K.W., Ousterhout, J.K., 1977a. The implementation of the Cm* multi-microprocessor. In: Proceedings of AFIPS National Computing Conference, June 13–16, 1977, Dallas, TX, pp. 645–654.

Swan, R.J., Fuller, S.H., Siewiorek, D.P., 1977b. Cm*—a modular, multi-microprocessor. In: Proceedings of AFIPS National Computing Conference, June 13–16, 1977, Dallas, TX, pp. 637–644.

Swartzlander, E. (Ed.), 1990. Computer Arithmetic. IEEE Computer Society Press, Los Alamitos, CA.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A., 2015. Going deeper with convolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

Takagi, N., Yasuura, H., Yajima, S., 1985. High-speed VLSI multiplication algorithm with a redundant binary addition tree. IEEE Trans. Comput. C-34 (9), 789–796.

Talagala, N., 2000. Characterizing Large Storage Systems: Error Behavior and Performance Benchmarks (Ph.D. dissertation). Computer Science Division, University of California, Berkeley.

Talagala, N., Patterson, D., 1999. An Analysis of Error Behavior in a Large Storage System, Tech. Report UCB//CSD-99-1042. Computer Science Division, University of California, Berkeley.

Talagala, N., Arpaci-Dusseau, R., Patterson, D., 2000a. Micro-Benchmark Based Extraction of Local and Global Disk Characteristics, CSD-99-1063. Computer Science Division, University of California, Berkeley.

Talagala, N., Asami, S., Patterson, D., Futernick, R., Hart, D., 2000b. The art of massive storage: a case study of a Web image archive. Computer 33 (11), 22–28.

Tamir, Y., Frazier, G., 1992. Dynamically-allocated multi-queue buffers for VLSI communication switches. IEEE Trans. Comput. 41 (6), 725–734.

Tanenbaum, A.S., 1978. Implications of structured programming for machine architecture. Commun. ACM 21 (3), 237–246.

Tanenbaum, A.S., 1988. Computer Networks, second ed. Prentice Hall, Englewood Cliffs, NJ.

Tang, C.K., 1976. Cache design in the tightly coupled multiprocessor system. In: Proceedings of AFIPS National Computer Conference, June 7–10, 1976, New York, pp. 749–753.

Tanqueray, D., 2002. The Cray X1 and supercomputer road map. In: Proceedings of 13th Daresbury Machine Evaluation Workshop, December 11–12, 2002, Daresbury Laboratories, Daresbury, Cheshire, UK.

Tarjan, D., Thoziyoor, S., Jouppi, N., 2005. HPL Technical Report on CACTI 4.0. www.hpl.hp.com/techeports/2006/HPL=2006+86.html.

Taylor, G.S., 1981. Compatible hardware for division and square root. In: Proceedings of 5th IEEE Symposium on Computer Arithmetic, May 18–19, 1981, University of Michigan, Ann Arbor, MI, pp. 127–134.

Taylor, G.S., 1985. Radix 16 SRT dividers with overlapped quotient selection stages. In: Proceedings of Seventh IEEE Symposium on Computer Arithmetic, June 4–6, 1985, University of Illinois, Urbana, IL, pp. 64–71.

Taylor, G., Hilfinger, P., Larus, J., Patterson, D., Zorn, B., 1986. Evaluation of the SPUR LISP architecture. In: Proceedings of 13th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1986, Tokyo.

Taylor, M.B., Lee, W., Amarasinghe, S.P., Agarwal, A., 2005. Scalar operand networks. IEEE Trans. Parallel Distrib. Syst. 16 (2), 145–162.

Tendler, J.M., Dodson, J.S., Fields Jr., J.S., Le, H., Sinharoy, B., 2002. Power4 system microarchitecture. IBM J. Res. Dev. 46 (1), 5–26.

TensorFlow Tutorials, 2016. https://www.tensorflow.org/versions/r0.12/tutorials/index.html.

Texas Instruments, 2000. History of Innovation: 1980s. www.ti.com/corp/docs/company/history/1980s.shtml.

Tezzaron Semiconductor, 2004. Soft Errors in Electronic Memory, White Paper. Tezzaron Semiconductor, Naperville, IL http://www.tezzaron.com/about/papers/soft_errors_1_1_secure.pdf.

Thacker, C.P., McCreight, E.M., Lampson, B.W., Sproull, R.F., Boggs, D.R., 1982. Alto: a personal computer. In: Siewiorek, D.P., Bell, C.G., Newell, A. (Eds.), Computer Structures: Principles and Examples. McGraw-Hill, New York, pp. 549–572.

Thadhani, A.J., 1981. Interactive user productivity. IBM Syst. J. 20 (4), 407–423.

Thekkath, R., Singh, A.P., Singh, J.P., John, S., Hennessy, J.L., 1997. An evaluation of a commercial CC-NUMA architecture—the CONVEX Exemplar SPP1200. In: Proceedings of 11th International Parallel Processing Symposium (IPPS), April 1–7, 1997, Geneva, Switzerland.

Thorlin, J.F., 1967. Code generation for PIE (parallel instruction execution) computers. In: Proceedings of Spring Joint Computer Conference, April 18–20, 1967, Atlantic City, NJ, p. 27.

Thornton, J.E., 1964. Parallel operation in the Control Data 6600. In: Proceedings of AFIPS Fall Joint Computer Conference, Part II, October 27–29, 1964, San Francisco. 26, pp. 33–40.

Thornton, J.E., 1970. Design of a Computer, the Control Data 6600. Scott Foresman, Glenview, IL.

Tjaden, G.S., Flynn, M.J., 1970. Detection and parallel execution of independent instructions. IEEE Trans. Comput. C-19 (10), 889–895.

Tomasulo, R.M., 1967. An efficient algorithm for exploiting multiple arithmetic units. IBM J. Res. Dev. 11 (1), 25–33.

Torrellas, J., Gupta, A., Hennessy, J., 1992. Characterizing the caching and synchronization performance of a multiprocessor operating system. In: Proceedings of Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 12–15, 1992, Boston (SIGPLAN Notices 27:9 (September), pp. 162–174.

Touma, W.R., 1993. The Dynamics of the Computer Industry: Modeling the Supply of Workstations and Their Components. Kluwer Academic, Boston.

Tuck, N., Tullsen, D., 2003. Initial observations of the simultaneous multithreading Pentium 4 processor. In: Proceedings of 12th International Conference on Parallel Architectures and Compilation Techniques (PACT'03), September 27–October 1, 2003, New Orleans, LA, pp. 26–34.

Tullsen, D.M., Eggers, S.J., Levy, H.M., 1995. Simultaneous multithreading: Maximizing on-chip parallelism. In: Proceedings of 22nd Annual International Symposium on Computer Architecture (ISCA), June 22–24, 1995, Santa Margherita, Italy, pp. 392–403.

Tullsen, D.M., Eggers, S.J., Emer, J.S., Levy, H.M., Lo, J.L., Stamm, R.L., 1996. Exploiting choice: instruction fetch and issue on an implementable simultaneous multithreading processor. In: Proceedings of 23rd Annual International Symposium on Computer Architecture (ISCA), May 22–24, 1996, Philadelphia, PA, pp. 191–202.

Tung, L., 2016. Google Translate: 'This landmark update is our biggest single leap in 10 years', ZDNet. http://www.zdnet.com/article/google-translate-this-landmark-update-is-our-biggest-single-leap-in-10years/.

Ungar, D., Blau, R., Foley, P., Samples, D., Patterson, D., 1984. Architecture of SOAR: Smalltalk on a RISC. In: Proceedings of 11th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1984, Ann Arbor, MI, pp. 188–197.

Unger, S.H., 1958. A computer oriented towards spatial problems. Proc. Inst. Radio Eng. 46 (10), 1744–1750.

Vahdat, A., Al-Fares, M., Farrington, N., Niranjan Mysore, R., Porter, G., Radhakrishnan, S., 2010. Scale-out networking in the data center. IEEE Micro 30 (4), 29–41.

Vaidya, A.S., Sivasubramaniam, A., Das, C.R., 1997. Performance benefits of virtual channels and adaptive routing: an application-driven study. In: Proceedings of ACM/IEEE Conference on Supercomputing, November 16–21, 1997, San Jose, CA.

Vajapeyam, S., 1991. Instruction-Level Characterization of the Cray Y-MP Processor (Ph.D. thesis). Computer Sciences Department, University of Wisconsin-Madison.

van Eijndhoven, J.T.J., Sijstermans, F.W., Vissers, K.A., Pol, E.J.D., Tromp, M.I.A., Struik, P., Bloks, R.H.J., van der Wolf, P., Pimentel, A.D., Vranken, H.P.E., 1999. Trimedia CPU64 architecture. In: Proceedings of IEEE International Conference on Computer Design: VLSI in Computers and Processors (ICCD'99), October 10–13, 1999, Austin, TX, pp. 586–592.

Van Vleck, T., 2005. The IBM 360/67 and CP/CMS. http://www.multicians.org/thvv/360-67.html.

Vanhoucke, V., Senior, A., Mao, M.Z., 2011. Improving the speed of neural networks on CPUs. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/37631.pdf.

von Eicken, T., Culler, D.E., Goldstein, S.C., Schauser, K.E., 1992. Active messages: a mechanism for integrated communication and computation. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia.

Waingold, E., Taylor, M., Srikrishna, D., Sarkar, V., Lee, W., Lee, V., Kim, J., Frank, M., Finch, P., Barua, R., Babb, J., Amarasinghe, S., Agarwal, A., 1997. Baring it all to software: raw machines. IEEE Comput. 30, 86–93.

Wakerly, J., 1989. Microcomputer Architecture and Programming. Wiley, New York.

Wall, D.W., 1991. Limits of instruction-level parallelism. In: Proceedings of Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 8–11, 1991, Palo Alto, CA, pp. 248–259.

Wall, D.W., 1993. Limits of Instruction-Level Parallelism, Research Rep. 93/6, Western Research Laboratory. Digital Equipment Corp., Palo Alto, CA.

Walrand, J., 1991. Communication Networks: A First Course. Aksen Associates/Irwin, Homewood, IL.

Wang, W.-H., Baer, J.-L., Levy, H.M., 1989. Organization and performance of a two-level virtual-real cache hierarchy. In: Proceedings of 16th Annual International Symposium on Computer Architecture (ISCA), May 28–June 1, 1989, Jerusalem, pp. 140–148.

Watanabe, T., 1987. Architecture and performance of the NEC supercomputer SX system. Parallel Comput. 5, 247–255.

Waters, F. (Ed.), 1986. IBM RT Personal Computer Technology, SA 23-1057. IBM, Austin, TX.

Watson, W.J., 1972. The TI ASC—a highly modular and flexible super processor architecture. In: Proceedings of AFIPS Fall Joint Computer Conference, December 5–7, 1972, Anaheim, CA, pp. 221–228.

Weaver, D.L., Germond, T., 1994. The SPARC Architectural Manual, Version 9. Prentice Hall, Englewood Cliffs, NJ.

Weicker, R.P., 1984. Dhrystone: a synthetic systems programming benchmark. Commun. ACM 27 (10), 1013–1030.

Weiss, S., Smith, J.E., 1984. Instruction issue logic for pipelined supercomputers. In: Proceedings of 11th Annual International Symposium on Computer Architecture (ISCA), June 5–7, 1984, Ann Arbor, MI, pp. 110–118.

Weiss, S., Smith, J.E., 1987. A study of scalar compilation techniques for pipelined supercomputers. In: Proceedings of Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), October 5–8, 1987, Palo Alto, CA, pp. 105–109.

Weiss, S., Smith, J.E., 1994. Power and PowerPC. Morgan Kaufmann, San Francisco.

Wendel, D., Kalla, R., Friedrich, J., Kahle, J., Leenstra, J., Lichtenau, C., Sinharoy, B., Starke, W., Zyuban, V., 2010. The Power7 processor SoC. In: Proceedings of International Conference on IC Design and Technology, June 2–4, 2010, Grenoble, France, pp. 71–73.

Weste, N., Eshraghian, K., 1993. Principles of CMOS VLSI Design: A Systems Perspective, 2nd ed. Addison-Wesley, Reading, MA.

Wiecek, C., 1982. A case study of the VAX 11 instruction set usage for compiler execution. In: Proceedings of Symposium on Architectural Support for Programming Languages and Operating Systems (ASPLOS), March 1–3, 1982, Palo Alto, CA, pp. 177–184.

Wilkes, M., 1965. Slave memories and dynamic storage allocation. IEEE Trans. Electron. Comput. EC-14 (2), 270–271.

Wilkes, M.V., 1982. Hardware support for memory protection: capability implementations. In: Proceedings of Symposium on Architectural Support for Programming Languages and Operating Systems (ASPLOS), March 1–3, 1982, Palo Alto, CA, pp. 107–116.

Wilkes, M.V., 1985. Memoirs of a Computer Pioneer. MIT Press, Cambridge, MA.
Wilkes, M.V., 1995. Computing Perspectives. Morgan Kaufmann, San Francisco.
Wilkes, M.V., Wheeler, D.J., Gill, S., 1951. The Preparation of Programs for an Electronic Digital Computer. Addison-Wesley, Cambridge, MA.
Williams, T.E., Horowitz, M., Alverson, R.L., Yang, T.S., 1987. A self-timed chip for division. In: Losleben, P. (Ed.), 1987 Stanford Conference on Advanced Research in VLSI. MIT Press, Cambridge, MA.
Williams, S., Waterman, A., Patterson, D., 2009. Roofline: an insightful visual performance model for multicore architectures. Commun. ACM 52 (4), 65–76.
Wilson Jr., A.W., 1987. Hierarchical cache/bus architecture for shared-memory multiprocessors. In: Proceedings of 14th Annual International Symposium on Computer Architecture (ISCA), June 2–5, 1987, Pittsburgh, PA, pp. 244–252.
Wilson, R.P., Lam, M.S., 1995. Efficient context-sensitive pointer analysis for C programs. In: Proceedings of ACM SIGPLAN'95 Conference on Programming Language Design and Implementation, June 18–21, 1995, La Jolla, CA, pp. 1–12.
Wolfe, A., Shen, J.P., 1991. A variable instruction stream extension to the VLIW architecture. In: Proceedings of Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 8–11, 1991, Palo Alto, CA, pp. 2–14.
Wood, D.A., Hill, M.D., 1995. Cost-effective parallel computing. IEEE Comput. 28 (2), 69–72.
Wu, Y., Schuster, M., Chen, Z., Le, Q., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Kaiser, Ł., Gouws, S., Kato, Y., Kudo, T., Kazawa, H., Stevens, K., Kurian, G., Patil, N., Wang, W., Young, C., Smith, J., Riesa, J., Rudnick, A., Vinyals, O., Corrado, G., Hughes, M., Dean, J., 2016. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. http://arxiv.org/abs/1609.08144.
Wulf, W., 1981. Compilers and computer architecture. Computer 14 (7), 41–47.
Wulf, W., Bell, C.G., 1972. C.mmp—a multi-mini-processor. In: Proceedings of AFIPS Fall Joint Computer Conference, December 5–7, 1972, Anaheim, CA, pp. 765–777.
Wulf, W., Harbison, S.P., 1978. Reflections in a pool of processors—an experience report on C.mmp/Hydra. In: Proceedings of AFIPS National Computing Conference, June 5–8, 1978, Anaheim, CA, pp. 939–951.
Wulf, W.A., McKee, S.A., 1995. Hitting the memory wall: implications of the obvious. ACM SIGARCH Comput. Architect. News 23 (1), 20–24.
Wulf, W.A., Levin, R., Harbison, S.P., 1981. Hydra/C.mmp: An Experimental Computer System. McGraw-Hill, New York.
Yamamoto, W., Serrano, M.J., Talcott, A.R., Wood, R.C., Nemirosky, M., 1994. Performance estimation of multistreamed, superscalar processors. In: Proceedings of 27th Annual Hawaii International Conference on System Sciences, January 4–7, 1994, Maui, pp. 195–204.
Yang, Y., Mason, G., 1991. Nonblocking broadcast switching networks. IEEE Trans. Comput. 40 (9), 1005–1015.
Yeager, K., 1996. The MIPS R10000 superscalar microprocessor. IEEE Micro 16 (2), 28–40.
Yeh, T., Patt, Y.N., 1993a. Alternative implementations of two-level adaptive branch prediction. In: Proceedings of 19th Annual International Symposium on Computer Architecture (ISCA), May 19–21, 1992, Gold Coast, Australia, pp. 124–134.
Yeh, T., Patt, Y.N., 1993b. A comparison of dynamic branch predictors that use two levels of branch history. In: Proceedings of 20th Annual International Symposium on Computer Architecture (ISCA), May 16–19, 1993, San Diego, CA, pp. 257–266.

Index

Page references in bold represent figures, tables and boxes.

A

Absolute addressing mode, K-34
Accelerated Strategic Computing Initiative (ASCI)
  ASCI Red, F-104–105
  ASCI White, F-71, F-105
Access 1/Access 2 stages, TI 320C55 DSP, E-7
Access bit, B-52
Access time. *See also* Average memory access time (AMAT)
  DRAM/magnetic disk, **D-3**
  DSM, 372–373
  memory hierarchy design, 85
  slowdown causes, B-3, **B-3**
  SMPs, 371
Access time gap, D-3
Accumulator, 557–558
  architecture, A-3
  extended, A-3
Acknowledgment, packets, F-17
ACM. *See* Association for Computing Machinery (ACM)
ACS project, M-29–30
Activation hardware, 557–558
Ada language, integer division/remainder, **J-12**
Adaptive routing
  definition, F-47–48
  vs. deterministic routing, **F-53–56**
  network fault tolerance, F-98
  and overhead, F-97
Adders
  carry-lookahead, J-37–41
  chip comparison, J-61
  full, J-2–3, **J-3**
  half, J-2–3
  integer division speedup, J-54–57
  integer multiplication speedup
    even/odd array, **J-52**
    many adders, J-50–54, **J-50**
    multipass array multiplier, **J-51**
    signed-digit addition table, **J-54**
    single adder, J-47–49, **J-48–49**
    Wallace tree, **J-53**
  radix-2 division, **J-55**
  radix-4 division, **J-56**
  radix-4 SRT division, **J-57**
  ripple-carry, J-3, **J-3**
  time/space requirements, **J-44**
Addition operations
  chip comparison, J-61
  floating point
    denormals, J-26–27
    overview, J-21–25
    rules, **J-24**
    speedup, J-25–26
  integer, speedup
    carry-lookahead, J-37–41
    carry-lookahead circuit, **J-38**
    carry-lookahead tree, **J-40**
    carry-lookahead tree adder, **J-41**
    carry-select adder, J-43–44, **J-43–44**
    carry-skip adder, J-41–43, **J-42**
    overview, J-37
  ripply-carry addition, J-2–3, **J-3**
Address aliasing prediction, 239–240
Address fault, B-42
Addressing modes
  absolute, K-34
  based indexed addressing, K-34
  base plus scaled indexed, K-34
  for control flow instructions, A-17–18
  data addressing modes, K-32–35
  displacement, A-11–12
  and instruction formats, K-6–9
  instruction set architecture, 13
  memory addressing, A-8–11, **A-10**
  register indirect, K-34
  RISC-V, A-36
Address offset, B-55–56
Address space
  global, B-52
  local, B-52
  memory hierarchy, B-57
  shared memory, 373
  virtual memory, B-12, B-40, **B-41**, B-44, B-55
Address specifier, A-21, K-54
Address stage, TI 320C55 DSP, E-7
Address trace, B-4
Address translation, B-42
  AMD64 paged virtual memory, B-55
  during indexing, B-36–40, 83
  Opteron data TLB, **B-47**
  translation lookaside buffers, B-37, B-46, **B-47**
  virtual memory, B-46, **B-47**, 120
Advanced load address table (ALAT)
  IA-64 ISA, H-40
  vector sparse matrices, G-12–13
Advanced loads, IA-64 ISA, H-40

Advanced mobile phone service (AMPS), cell phones, E-25
Advanced Research Project Agency (ARPA), F-102–103
Advanced RISC Machine (ARM), 12
  architecture, K-22
  GPU computing history, M-53
Advanced Simulation and Computing (ASC) program, F-106
Advanced Switching Interconnect (ASI), F-107
Advanced Switching SAN, F-71
Advanced Technology Attachment (ATA) disks
  Berkeley's Tertiary Disk project, D-12–13
  disk power, D-5
  disk storage, D-4
  historical background, M-88
  RAID 6, D-8–9
Advanced vector extensions (AVX), 282, 305, **306**
Affine, loop-level parallelism dependences, H-6
After rounding rule, J-36
Aggregate bandwidth
  definition, F-13
  effective bandwidth calculations, F-18–19
  shared- *vs.* switched-media networks, **F-23**, F-25
  switched-media networks, F-24–25
  switch microarchitecture, F-56
AI. *See* Artificial intelligence (AI)
Aiken, Howard, M-3–4
ALAT. *See* Advanced load address table (ALAT)
Alewife machine, M-62
ALGOL, M-17–18
Aliases, address translation, B-38
Allen, Fran, M-29–30
Alliant processors, G-26
Alloy cache, 115
Alloyed predictors, 184
Alpha 21164
  cache hierarchy, characteristics, **395**
  L1 caches, **395**
AlphaServer, 395–396
AlphaServer 4100, 395
AltaVista search, cluster history, M-63, M-74–75
ALUs. *See* Arithmetic-logical units (ALUs)
AMAT. *See* Average memory access time (AMAT)
Amazon
  Dynamo key-value storage system, 485–486
  Elastic Computer Cloud, 491–492
  Simple Storage Service, 491–492
  warehouse-scale computers, 10
Amazon Elastic Computer Cloud (EC2), utility computing, M-75–76
Amazon Web Services (AWS)
  availability zones, 497–501, **497**
  cloud computing, 491–497
  EC2 computer unit, **493–494**
  growth, **500**
  guarantee of service, 492
  low cost, 492
  reliance on open source software, 492
  virtual machines, 491
  Xen virtual machine, 126
Amdahl, Gene, M-29–30
Amdahl's Law, 5
  computer design principles, 49–52
  computer system power consumption case study, 69–71
  execution time, 50
  multicore scaling, 436, 438, 442
  parallel processing calculations, 373–377
  pitfall, 61
  software overhead, F-96
  speedup, 374–375
  VMIPS on Linpack, G-18
AMD Athlon 64, Itanium 2 comparison, **H-43**
AMD Fusion, M-53
AMD K-5, M-35
AMD Opteron, 387–388
  address translation, B-38
  data cache example, B-12–15, **B-13**
  implementation, 391
  microprocessor, **27**
  misses per instruction, **B-15**
  NetApp FAS6000 filer, D-42
  paged virtual memory example, B-54–57
  vs. Pentium protection, B-57
  processors, 403
  TLB during address translation, **B-47**
  tournament predictors, 185–187
AMD processors
  GPU computing history, M-53
  power consumption, F-89
  recent advances, M-35
  RISC history, M-23
Amortization of overhead, D-64–67
Ample parallelism, 467
Andreessen, Marc, F-102
Andrew benchmark, 399
Annual failure rate, 62
Antenna, radio receiver, **E-23**
Antialiasing, B-38
Antidependence
  compiler history, M-32
  definition, 172
  finding, H-7–8
  register renaming, 196
Apollo DN 10000, M-32
Application layer, **F-84**
Applied Minds, M-76
Arbitration algorithm
  collision detection, F-23–24
  commercial interconnection networks, **F-57**
  interconnection networks, F-21–22, F-49–51
  network impact, F-52–53
  SAN characteristics, F-76–77
  switched-media networks, F-24–25
  switch microarchitecture, F-56
    pipelining, F-65–66
  system area network, F-104–105
Architecturally visible registers, 234
Architectural Support for Compilers and Operating Systems (ASPLOS), M-12
Architecture. *See also* Computer architecture; Instruction set architecture (ISA)
  and compiler writer, A-30–31
  microarchitecture, 266–273
Areal density, D-2
Argument pointer, K-57
Arithmetic intensity, 307–308

Arithmetic-logical units (ALUs)
data forwarding, **C-36–37**
data hazards stall minimization, C-14–17
DSP media extensions, E-10
effective address cycle, C-5
IA-64 instructions, **H-35**
integer division, J-54
integer multiplication, J-48
integer operations, C-46–48
integer shifting over zeros, J-45
latency, C-46–48
load interlocks, **C-35**
micro-op fusion, 254
MIPS R4000 pipeline, C-59
multicycle implementation, C-29
operation, C-27–28
pipeline branch issues, C-35–36
RISC classic pipeline, C-8
RISC instruction set, C-5
RISC pipeline, C-31–32, C-35
TX-2, M-50
ARM. *See* Advanced RISC Machine (ARM)
ARM AMBA, OCNs, F-3
ARM Cortex-A53
characteristics, **259**
clock cycles per instruction, 251–252, **252**
data miss rate, **132**
memory hierarchy design, 129–131, **130**
misprediction rate, **250**
multiple-issue processors, 247–252
pipeline performance, **249**, 250–252
virtual address, physical and data blocks, **131**
ARMv8, **K-4**, **K-9**, 13, **K-15**, K-16, K-22
ARPANET, F-102
Array
FFT kernel, I-7
ocean application, I-9–10
recurrences, H-12
Array multiplier
example, **J-50**
integers, **J-50**
multipass system, **J-51**
Artificial intelligence (AI), 546
ASC Purple, F-71, F-105
ASPLOS. *See* Architectural Support for Compilers and Operating Systems (ASPLOS)
Assembly language, 2
Association for Computing Machinery (ACM), M-3
Associativity. *See also* Set associativity
Opteron data cache, B-13–14, **B-13**
sizes and, **B-10**
Astronautics ZS-1, M-31
Asynchronous events, exception, C-39
Asynchronous I/O, D-35–36
Asynchronous Transfer Mode (ATM)
interconnection networks, F-102–103
LAN, F-93–94
packet format, **F-79**
total time statistics, **F-94**
VOQs, F-60–61
WANs, F-4, F-84–85, F-102–103
ATA disks. *See* Advanced Technology Attachment (ATA) disks
Atanasoff Berry Computer (ABC), M-5
Atanasoff, John, M-5
ATI Radeon 9700, M-51–52
Atlas computer, M-9
ATM system, TP benchmarks, D-18
Atom 230, 258, **259**
Atomic exchange, 413
Atomic instructions, barrier synchronization, **I-14**
Atomic operations, 386
Attributes field, B-52
Autoincrement deferred addressing, K-52–53
Autonet, F-49
Autonomous instruction fetch units, 127
Availability
computer systems, D-43
I/O system design/evaluation, D-36–37
Average instruction execution time, M-6
Average memory access time (AMAT)
block size calculations, B-26–28, **B-28**
cache optimizations, B-22, B-26–28
cache performance, B-15–17, **B-22**
memory hierarchy design, 82
miss rate, B-29–30, **B-30**
out-of-order computer, B-21
and processor performance, B-17–20
using miss rates, **B-30**
Average reception factor
centralized switched networks, F-33
multi-device interconnection networks, F-26–27
AWS. *See* Amazon Web Services (AWS)

B

Back-off time, shared-media networks, F-103
Backpressure, congestion management, F-69
Backpropagation, 548
Backside bus, 377
Balanced systems, D-64–67
Balanced tree, MINs with nonblicking, F-34
Bandwidth. *See also* Cache bandwidth; Throughput
arbitration, F-49–50
bisection, F-39–40, F-93, 478
and cache miss, B-2
communication mechanism, I-3
compute, 350
congestion management, F-68
Cray Research T3D, F-91
definition, F-13
disparity, **F-29**
FP arithmetic, J-62
gap, disk storage, D-3
instruction fetch, 228–232, **229–230**
latency and effective, F-25–30
log-log plot, **21**
memory, 350, 356
network performance and topology, F-41
over latency, 20
point-to-point links and switches, D-34
shared- *vs.* switched-media networks, **F-23**, F-25
snooping, 389–390
two-device networks, F-13–20
for vector load/store units, 298–299

Banerjee, Uptal, M-32
Bank busy time, vector memory systems, G-9
Banked memory, 346
  vector architectures, **G-10**
Barcelona Supercomputer Center, **F-80**
Barnes
  characteristics, I-8–9
  distributed-memory multiprocessor, **I-32**
  symmetric shared-memory multiprocessors, I-21–22, **I-23**, I-25–26
Barnes-Hut *n*-body algorithm, I-8–9
Barriers
  Cray X1, G-23
  fetch-and-increment, I-20–21
  large-scale multiprocessor synchronization, I-20–21
  large-scale multiprocessor, synchronization, I-13–16, **I-14**, **I-16**, **I-19**, I-20
Based indexed addressing mode, K-34
Base field, B-52
Base plus scaled indexed addressing mode, K-34–35
Base station, E-22–23
Batches, DNNs, 556
Batch processing workloads, 467
Bay Area Research Network (BARRNet), **F-83**
BBN Butterfly, M-61
BBN Monarch, M-61
Before rounding rule, J-36
Benchmarks. *See also* Thread Block; *specific benchmark*
  desktop benchmarks, 41–43
  distribution of data accesses by, **A-14**
  EEMBC, E-12, **E-12**
  embedded applications
    basic considerations, E-12
    power consumption and efficiency, E-13, **E-13–14**
  fallacy, 61
  performance measurement, 40–45
  response time restrictions, **D-18**
  sorting case study, D-64–67
  suite, 41
Beneŝ topology, F-33, **F-34**
Berkeley's Tertiary Disk project
  failures of components, **D-12**
  overview, D-12–13
  system log, **D-43**
Berners-Lee, Tim, F-102
Bertram, Jack, M-29–30
Best-case lower bounds, F-26
Best-case upper bounds, F-26
Between instructions exception, C-39, C-45
Biased exponent, J-15–16
Bidirectional multistage interconnection networks, F-33–34
Bidirectional rings, F-36
Big Endian
  byte order, A-7
  interconnection networks, F-12
BINAC, M-5
Binary code compatibility, embedded systems, E-15
Binary-coded decimal, A-14
Binary-to-decimal conversion, FP precisions, J-33–34
Bing search engine
  negative impact, **486**
  WSCs, 485
Bisection bandwidth, 478
  as network cost constraint, F-93
  network performance and topology, F-93
  NEWS communication, F-42
  topology, F-39
Bisection traffic fraction, F-41–42
Bit error rate (BER), wireless networks, E-21
Bit rot, case study, D-61–64
Bit selection, B-8
Black box network
  basic concept, F-5–6
  effective bandwidth, F-18
  performance, F-13
  switched-media networks, F-24–25
  switched network topologies, F-41
Block. *See also* Cache block
  addressing, B-7–9
  cache optimization, 107–109
  centralized switched networks, F-33
  definition, B-2
  disk array deconstruction, D-51–54
  disk deconstruction case study, D-48–50
  factor, 108
  global code scheduling, H-15–16
  head-of-line, F-59–60
  identification
    memory hierarchy, B-8–9
    virtual memory, B-44–45
  LU kernel, I-8
  memory hierarchy, 81
  multithreading, M-35–36
  network performance and topology, F-41
  offset, B-8–9
    block identification, B-8–9
    cache optimization, B-38
    Opteron data cache, **B-13**, B-14
  placement
    memory hierarchy, B-7–8, **B-7**
    virtual memory, B-44
  RAID performance prediction, D-57–59
  replacement
    memory hierarchy, B-9–10
    virtual memory, B-45
  size, miss rate and, B-26–28, **B-27**
  TI TMS320C55 DSP, E-8
Blocked floating point arithmetic, DSP, E-6
Block servers, *vs.* filers, D-34–35
Block transfer engine (BLT), F-91
Boggs, David, F-103
BOMB, M-4
Booth recoding, J-8–10, **J-9**, J-17
  integer multiplication, **J-49**
Bose-Einstein formula, 34
Bounds checking, B-52
Branch(es)
  completion cycle, C-28
  delayed, C-20, **C-20**
  folding, 231
  history table, C-23
  RISC instruction set, C-5
  VAX, K-57
  WCET, E-4

Branch byte, K-57
Branch hazards, C-18–22
penalty reduction, C-19–20
pipeline issues, C-35–37
scheme performance, C-21–22, **C-22**
simple scheme examples, C-21–22
Branch penalty
branch-target buffers, 230–231, **231**
prediction schemes, **C-21**
reduction, C-19–20
Branch prediction
accuracy, **C-25–26**
buffers, C-23–25, **C-24–26**
correlation, 182–184
cost reduction, C-22, 182–191
dynamic, C-23–25
early schemes, M-29
instruction-level parallelism
correlating branch predictors, 182–184
Intel Core i7, 190–191
specialized branch prediction, 232–234
tagged hybrid predictors, 188–190, **188**, **190**
tournament predictors, 184–188, **186**
integrated, 233
static, C-22, **C-23**
trace scheduling, H-19
Branch registers, IA-64, H-34
Branch stalls, C-61, **C-64**
Branch-target address
branch hazards, **C-38**
pipeline branch issues, C-35–36
RISC instruction set, C-5
Branch-target buffers
branch penalty, 230–231, **231**
handling an instruction with, **230**
instruction fetch bandwidth, 228–232, **229–230**
program counter, 228–229, **229**
Branch-target cache, 228
Branch word, K-57
Brewer, Eric, M-74–75
Bridges, **F-82**, F-83
Bubbles, F-47–48, **F-54**
Buckets, D-26
Buffered crossbar switch, F-66
Buffered wormhole switching, F-52
Buffers. *See also* Branch-target buffers
branch-prediction, C-23–25, **C-24–26**
DSM multiprocessor cache coherence, I-38–40
integrated instruction fetch units, 234
Intel Core i7, **256**
interconnection networks, F-10–11
network interface functions, **F-7**
organizations, F-59–61
translation lookaside, B-37, B-46, **B-47**
write, B-11, B-14
Bundles
example calculations, H-35–36
IA-64, H-34–35, **H-37**
Itanium 2, H-41
Burks, Arthur, M-3
Burroughs B5000, M-17–18
Bus-based coherent multiprocessors, M-59–60
Buses
barrier synchronization, **I-16**
I/O bus replacements, **D-34**
large-scale multiprocessor synchronization, I-12–13
NEWS communication, F-42
scientific workloads on symmetric shared-memory multiprocessors, **I-25**
Sony PlayStation 2 Emotion Engine, E-18
vs. switched networks, F-2
switch microarchitecture, F-56
Bypassing, C-14
SAN example, F-77
Byte displacement addressing, K-52
Byte/word/long displacement deferred addressing, K-52–53

C

Cache(s). *See also* Memory hierarchy
AMD Opteron example, B-12–15, **B-13**, **B-15**
benefits, 350
block frames and memory, **B-7**
concept, M-11
definition, B-2
embedded systems, E-4
Itanium 2, H-42–43
parameters, **B-42**
Sony PlayStation 2 Emotion Engine, E-18
vector processors, G-25
and virtual memory, B-42–43, **B-42**, B-48–49, **B-48**
Cache bandwidth
block addressing, **100**
increasing, 94
multibanked caches, 99–100
nonblocking caches, 100–104
pitfall, 143
Cache block
cache coherence protocol, 382–383, **384–385**
definition, B-2
miss rate reduction, B-26–28
shared state, 386
symmetric shared-memory multiprocessors, I-22, I-25–26, **I-25**
write strategy, B-10–12
Cache coherence
atomic operations, 386
of cached data, 128–129
Cray X1, G-22
definition, 377–379
directory-based (*see* Directory-based cache coherence)
enforcement, 379–380
example protocol, 383–387, **384**
extensions, 388
implementing locks using, 414–417, **416**
large-scale multiprocessors, I-34–36
deadlock and buffering, I-38–40
directory controller, I-40–41
DSM implementation, I-36–37
large-scale multiprocessors history, M-61
mechanism, **384**
multiprocessor, 377–379, 387
nonatomic operations, 386
problem, 377, **378**
program order, 378–379

Cache coherence *(Continued)*
snooping (*see* Snooping cache coherence)
state diagram, **385**, **387**
Cache hit, B-2
example calculation, B-5
Opteron data cache, B-14
Cache miss
block replacement, **B-10**
definition, B-2
distributed-memory multiprocessor, **I-32**
interconnection network, F-91–92
large-scale multiprocessors, I-34–35
WCET, E-4
Cache-only memory architecture (COMA), M-61–62
Cache optimization, B-22–40
advancement, **117**
cache misses, 112–113
case study, 148–164
compiler-controlled prefetching, 111–114
compiler optimizations, 107–109
critical word first, 104–105
early restart, 104–105
energy consumption, **97**
fallacy, 142
floating-point programs, 101–102
hardware prefetching, 109–111
HBM packaging, 114–117
hit time reduction, B-36–40, 95–98
impact, **B-40**, 148–150
miss categories, B-23–25
miss penalty reduction
read misses *vs.* writes, B-35–36
via multilevel caches, B-30–35
miss rate reduction
via associativity, B-28–30
via block size, B-26–28, **B-27**
via cache size, B-28
multibanked caches, 99–100
nonblocking caches, 100–104
pipelined access, 99–100
power reduction, 95–98
way prediction, 98–99
write buffer merging, 105–106, **106**
Cache organization
block placement, B-7–8, **B-7**
Opteron data cache, B-12–15, **B-13**
Cache performance, B-3–6, B-15–16
average memory access time, B-17–20
cache optimizations impact on, **B-40**
equations, **B-22**
example calculation, B-16–17
miss penalty, B-20–21
out-of-order execution, B-20–21
Cache prefetch, 111
Cache size, B-13
miss rate *vs.*, **B-24–25**, B-28, **B-33**, **B-37**
scientific workloads
distributed-memory multiprocessors, **I-29–31**
symmetric shared-memory multiprocessors, I-22–24, **I-24**
virtually addressed, **B-37**
Caching locks, 415
CACTI
energy consumption, **97**
first-level caches, 95–98, **96**
Callee/caller saving, A-19–20
Call gate, **B-53**, B-54
Canonical form, B-55
Capabilities, protection schemes, M-9
Capacity misses
cache size, **B-24**
definition, B-23
memory hierarchy design, 81
scientific workloads on symmetric shared-memory multiprocessors, I-22, I-24, **I-24**
Capital expenditures (CAPEX), 36, 486–490
Carrier sensing, F-23–24
Carrier signal, wireless networks, E-21
Carry-in, carry-skip adder, J-41–42
Carry-lookahead adder (CLA)
chip comparison, J-61
circuit, **J-38**
early computer arithmetic, J-63
example calculations, J-39
integer addition speedup, J-37–41
with ripple-carry adder, **J-42**
tree, **J-40–41**
Carry-out
carry-lookahead circuit, **J-38**
floating-point addition speedup, J-25
Carry-propagate adder (CPA)
integer multiplication, J-48, J-51
multipass array multiplier, **J-51**
Carry-save adder (CSA)
integer division, J-54–55
integer multiplication, J-47–48, **J-48**
Carry-select adder
characteristics, J-43–44
chip comparison, J-61
example, **J-43–44**
Carry-skip adder (CSA)
characteristics, J-41–43
example, **J-42**, **J-44**
Case statements, A-17
Catapult
board design, **568**
CNNs on, 570–572, **571–572**
evaluating, 601–602
guidelines, 577–579
implementation and architecture, 568–569
search acceleration on, 573–574
software, 569
version 1 deployment, 574
version 2 deployment, 575–577, **576–577**
C/C++ language
dependence analysis, H-6
GPU computing history, M-52–53
integer division/remainder, **J-12**
CDB. *See* Common data bus (CDB)
Cedar project, M-61
Cell, Barnes-Hut *n*-body algorithm, I-9
Cell phones
block diagram, **E-23**
embedded system case study
characteristics, E-22–24
Nokia circuit board, **E-24**
overview, E-20
radio receiver, **E-23**

standards and evolution, E-25
wireless networks, E-21–22
Flash memory, D-3–4
Nokia circuit board, **E-24**
wireless communication challenges, **E-21**
wireless networks, E-21–22
Centralized shared-memory multiprocessor, 371, 377
cache coherence protocol, 377–379, **378**, **384**
enforcement, 379–380
example protocol, 383–387, **384**
extensions, 388
state diagram, **385**, **387**
invalidate protocol implementation, 382–383
local memory, 377
SMP and snooping limitations, 389–392
snooping coherence protocols, 380, **381**
example protocol, 383–387, **384**
implementation, 392–393
invalidate protocol, **381**
limitations, 389–392
maintenance, 380–381
structure, **372**
Centralized switched networks, F-31–35, **F-31**
Centrally buffered switch, F-56
Central processing unit (CPU)
average memory access time, B-18–20
DNN and GPUs *vs.*, 595–602
early pipelined versions, M-27–28
execution time, B-3, B-5, **B-22**
GPU computing history, M-53
performance measurement history, M-6
Sony PlayStation 2 Emotion Engine, E-17–18
time, 39
TI TMS320C55 DSP, E-8
vector memory systems, **G-10**
Cerf, Vint, F-102
CFM. *See* Current frame pointer (CFM)
Chaining
convoys, DAXPY code, **G-16**
vector processor performance, G-11–12, **G-12**
Channels, cell phones, E-24
Charge-coupled device (CCD), Sanyo VPC-SX500 digital camera, E-19
Checksum
dirty bits, D-61–64
packet format, **F-7**
Chime, 291
vector chaining, G-11–12
vector execution time, G-4
vector performance, G-2–4
Chip-crossing wire delay, F-3, F-74, F-108
Chip fabrication cost, 67–68
Chipkill, 94
Choke packets, F-69–70
Chunk
disk array deconstruction, D-51–54
Shear algorithm, D-51–54
CIFS. *See* Common Internet File System (CIFS)
Circuit switching, F-51
Circulating water system (CWS), **483**
CISC. *See* Complex Instruction Set Computer (CISC)
CLA. *See* Carry-lookahead adder (CLA)
Clean block, B-11
Clock cycle
floating-point operations, **C-50**
pipeline scheduling, 177–179
RISC classic pipeline, C-6
RISC exception, C-42–43
RISC implementation, **C-30**
switch microarchitecture pipelining, F-66
vector architectures, G-4
Clock cycles per instruction (CPI), 53, 559
ARM Cortex-A53, 251–252, **252**
branch scheme, C-21–22, **C-22**
cache behavior impact, B-19–20
cache hit calculation, B-5
calculation, 375–376
clock rate, 261
data hazards requiring stalls, C-17
instruction-level parallelism, 168–169
Intel Core i7 6700, 256, **257**
microprocessor advances, M-35
pipelined processor, 168–169
pipeline with stalls, C-11
pipelining concept, C-3
processor performance equation, 52
RISC history, M-22
SPEC92 benchmarks, **C-64**
SPECCPUint2006 benchmarks, 256, **257**
stalls, **C-64**
Clock cycle time, 53
and associativity, B-29–30
cache optimization, B-19–20
cache performance, B-3
pipeline performance, C-11
pipelining, C-3
RISC implementation, **C-30**
shared- *vs.* switched-media networks, F-25
Clock rate, 261, **261**
microprocessor advances, M-35
Clock skew, C-8–10
Clos network, F-33, **F-34**, 510–511, **510–511**
Cloud computing
advantages, 490
AWS (*see* Amazon Web Services (AWS))
economies of scale, 491
fallacy, 514
providers, 518
utility computing history, M-75–76
Clusters, 9–10, 369, 478
characteristics, **I-45**
containers, M-76
Cray X1, G-22
history background, M-62–65
IBM Blue Gene/L, I-41–44, **I-43–44**
interconnection network domains, F-3
large-scale multiprocessors, I-6
large-scale multiprocessor trends, M-63–64
power consumption, F-89
utility computing, M-75–76
as WSC forerunners, M-74–75
Cm*, M-57

C.mmp, M-57
CMOS
  cell phone, E-24
  first vector computers, M-48
  ripply-carry addition, J-3
  scaling, 442–443
  vector processors, G-25–27
CNN. *See* Convolutional neural network (CNN)
Coarse-grained multithreading
  definition, 243–244
  superscalar processor, 245
Cocke, John, M-20, M-29–31
Code division multiple access (CDMA), cell phones, E-25
Code scheduling
  example, **H-16**
  parallelism, H-15–23
  superblock scheduling, H-21–23, **H-22**
  trace scheduling, H-19–21, **H-20**
Coefficient of variance, D-27
Coerced exception, C-39
Coherence. *See* Cache coherence
Coherence misses, I-22, 82, 393
Cold aisles, 506, **507**
Cold-start misses, B-23
Collision detection, shared-media networks, F-23–24
Collision misses, B-23
Collision, shared-media networks, F-23–24
Collocation sites, interconnection networks, F-89
COLOSSUS, M-4
Column access strobe (CAS), 85–86
Column major order, 107
COMA. *See* Cache-only memory architecture (COMA)
Combining tree, large-scale multiprocessor synchronization, I-18
Command queue depth, *vs.* disk throughput, **D-4**
Commercial interconnection networks
  congestion management, F-68–70
  connectivity, F-67
  cross-company interoperability, F-67–68
  DECstation 5000 reboots, **F-73**
  fault tolerance, F-70–72
Commit stage, 211
Commoditization, and cost, 30–31
Commodity cluster, characteristics, **I-45**
Common data bus (CDB), 197, 201
  performance, 207
  reservation stations and register tags, **202**
  write result, 211
Common Internet File System (CIFS), D-35
  NetApp FAS6000 filer, D-41–42
Communication bandwidth, I-3
Communication latency, I-3
  hiding, I-4
Communication mechanism, 375–376
  internetworking, F-85–89
  large-scale multiprocessors
    advantages, I-4–6
    metrics, I-3–4
  network interfaces, F-7–8
  NEWS communication, **F-42–44**
Communication protocol, F-8
Compare-select-store unit (CSSU), TI TMS320C55 DSP, E-8
Compiler(s)
  constants, A-31
  definition, C-65
  interaction, A-27–30
  for multimedia instructions, A-31–32
  phase, 399
  primitives, A-30
  regularity, A-30
  role, A-24–33
  structure, A-25–26
  trade-offs, A-30
  writer, A-30–31
Compiler-controlled prefetching, 111–114
Compiler optimization
  for caches, 107–109, 148–164
  instruction-level parallelism, 176–182
  memory consistency, 422
Compiler scheduling, hardware support, M-32–33
Compiler speculation, hardware support
  example calculations, H-29
  memory references, H-32
  overview, H-27
  preserving exception behavior, H-28–32
Compiler techniques
  Cray X1, G-21–22
  dependence analysis, H-7–8
  global code scheduling, H-17–18
  vectorization, G-12–14
  vector sparse matrices, G-12
Complex Instruction Set Computer (CISC), K-51
  RISC history, M-23
Compulsory misses, B-23
  cache size, **B-24**
  memory hierarchy design, 81
Computation-to-communication ratios
  parallel programs, I-10–12
  scaling, **I-11**
Compute bandwidth, 350
Compute-optimized processors, F-92
Computer architecture
  definition, 11–12, M-18–19
  floating-point addition, rule, **J-24**
  functional requirements, 17–18, **18**
  goals, 17–18
  high-level language, M-19–20
  instruction set architecture, 12–17
  limits of energy, 28–29
  of warehouse-scale computers, 477–482
Computer arithmetic
  chip comparison, J-57–61, **J-58–60**
  floating point
    denormals, J-14–15
    exceptions, J-34–35
    fused multiply-add, J-32–33
    IEEE 754, **J-16**
    iterative division, J-27–31
    and memory bandwidth, J-62
    number representation, J-15–16
    overview, J-13–14
    precisions, J-33–34
    remainder, J-31–32
    special values, J-14–15, **J-16**
    underflow, J-36–37, J-62
  floating-point addition
    denormals, J-26–27
    overview, J-21–25

rules, **J-24**
speedup, J-25–26
floating-point multiplication
denormals, J-20–21
examples, **J-19**
overview, J-17–20
rounding, **J-18**, J-19
integer addition speedup
carry-lookahead, J-37–41
carry-lookahead circuit, **J-38**
carry-lookahead tree, **J-40**
carry-lookahead tree adder, **J-41**
carry-select adder, J-43–44, **J-43–44**
carry-skip adder, J-41–43, **J-42**
overview, J-37
integer arithmetic
language comparison, **J-12**
overflow, **J-11**
Radix-2 multiplication/division, J-4–7, **J-4**
restoring/nonrestoring division, J-5, **J-6**
ripply-carry addition, J-2–3
signed numbers, J-7–10
systems issues, J-10–13
integer division
radix-2 division, **J-55**
radix-4 division, **J-56**
radix-4 SRT division, **J-57**
with single adder, J-54–57
SRT division, J-45–47, **J-46**, J-55–57
integer-FP conversions, J-62
integer multiplication
array multiplier, **J-50**
Booth recoding, **J-49**
even/odd array, **J-52**
many adders, J-50–54, **J-50**
multipass array multiplier, **J-51**
signed-digit addition table, **J-54**
with single adder, J-47–49, **J-48–49**
Wallace tree, **J-53**
integer multiplication/division, shifting over zeros, J-45
overview, J-2
rounding modes, J-14, J-17–20, **J-18**, **J-20**
Computer chip fabrication, Cray X1E, G-24
Computer classes
clusters, 9–10
desktop computing, 8
embedded computers, 6–7
internet of things, 6–7
parallel architectures, 10–11
parallelism, 10–11
personal mobile device, 7–8
servers, 8–9
and system characteristics, **E-4**
warehouse-scale computers, 9–10
Computer clusters, 470
Computer design principles
Amdahl's law, 49–52
common case, 49
locality, 48–49
parallelism, 48
processor performance equation, 52–55
Computer room air-conditioning (CRAC), 482
Computer technology, improvements, 2–6
Compute tiles, OCNs, F-3
Compute Unified Device Architecture (CUDA), 311
CUDA Thread, 311
GPU programming, 320
SIMD instructions, 325–326
GPU computing history, M-52–53
Computing efficiently, at low utilization, 468
Conditional branching
global code scheduling, H-16, **H-16**
graphics processing units, 323–326
options, A-18–19, **A-19**
static branch prediction, C-22
Conditional instructions
example calculations, H-23–24
exposing parallelism, H-23–27
limitations, H-26–27
Condition codes, K-11, C-44, K-57
Conflict misses
cache optimizations, B-23
cache size, **B-24**
memory hierarchy design, 82
Congestion control, F-68, F-70
Congestion management, F-68–70
Connectedness, F-30, **F-48**
Connection Machine CM-5, F-96
Connection Multiprocessor 2, M-46, M-56
Consistency. *See* Memory consistency
Constellation, characteristics, **I-45**
Containers, cluster history, M-76
Contention delay, F-26
Context switching, B-49, 119
Control bits, messages, F-7
Control Data Corporation (CDC)
CDC 6600, C-66–67
computer architecture definition, M-19
early computer arithmetic, J-64–65
first dynamic scheduling, M-28–29
multiple-issue processor development, M-28–29
multithreading history, M-35
RISC history, M-28–29
first vector computers, M-47
STAR-100, first vector computers, M-47
STAR processor, G-26
Control dependences
conditional instructions, H-24
global code scheduling, H-16
hardware-based speculation, 208
instruction-level parallelism, 174–176
maintenance, 175
Control flow instructions, 14
addressing modes for, A-17–18
classes, **A-17**
compilers
role, A-24–33
structure, A-25–26, **A-25**
conditional branch options, A-18–19, **A-19**
conditional instructions, H-27
procedure invocation options, A-19–20
RISC-V, A-39–40
types, A-16
Control hazards, C-11
Controllers, historical background, M-88–89
Convex Exemplar, M-62
Convex processors, G-26

Convolutional neural network (CNN), 550–552, **551**
on Catapult, 570–572, **571**
processing element of, **572**
Convolution, DSP, E-5
Convoy, 290–292
chained, DAXPY code, **G-16**
DAXPY on VMIPS, G-20–21
strip-mined loop, G-5
vector starting times, G-4
Conway, Lynn, M-29–30
Cooling systems, **483**
Cooling towers, 508
Copper wiring
Ethernet, F-82
interconnection networks, F-9–10
Copy propagation, H-10
Core, 17
Core i7, 346–353
Core plus ASIC, embedded systems, E-3
Cortex-A53
ARM, 129–131, **130**
performance, 132
Cosmic Cube, M-60–61
Cost
branch prediction, C-22
disk storage, D-2
DRAM/magnetic disk, **D-3**
interconnecting node calculations, **F-32–33**
Internet Archive Cluster, D-38–40
I/O system design/evaluation, D-36
magnetic storage history, M-85–86
SIMD supercomputer development, M-45
Cost-performance, 467
DSAs, 600–601, **601**
extensive pipelining, C-70
IBM eServer p5 multiprocessor, 440, **441**
sorting case study, D-64–67
Cost trends
cost-sensitive designs, 29
integrated circuit, 31–35
manufacturing *vs.* operation, 36
vs. price, 35
time, volume, and commoditization, 30–31
Counter register, K-25
CPA. *See* Carry-propagate adder (CPA)
CPI. *See* Clock cycles per instruction (CPI)
CP-67 program, M-10
CPU. *See* Central processing unit (CPU)
Cray-1
first vector computers, M-47–48
pipeline depths, G-4
RISC history, M-20
vector performance measures, G-16
Cray-2
DRAM, G-25
first vector computers, M-47–48
tailgating, G-20–21
Cray-3, G-27
Cray-4, G-27
Cray C90
first vector computers, M-48
vector performance calculations, G-8
Cray J90, M-48
Cray Research programmers, 303
Cray Research T3D, F-91
Cray, Seymour, G-25, G-27, M-47–48
Cray supercomputers, early computer arithmetic, J-63–64
Cray T90, 299
Cray T3D, M-61, F-104–105
Cray T3E, M-49, M-61, F-71, F-98–99
Cray X, 370
Cray X1, M-64
cluster history, M-64
first vector computers, M-49
MSP module, **G-22**, G-23–24
overview, G-21–23
Cray X1E, G-24, F-91, F-95
Cray X2, first vector computers, M-49
Cray X-MP, M-47–48
Cray XT3, M-59, M-63
Cray XT3 SeaStar, F-67
Cray Y-MP
first vector computers, M-48
parallel processing debates, M-58
Create vector index instruction (CVI), G-13
Credit-based control flow, F-10, F-18, F-69, F-75
CRISP, M-29
Critical path
global code scheduling, H-16
trace scheduling, H-19–21, **H-20**
Critical word first, cache optimization, 104–105
Crossbars, F-31, **F-31**
Convex Exemplar, M-62
Crossbar switch
centralized switched networks, F-31
interconnecting node calculations, **F-32–33**
Cross-company interoperability, F-67–68
Crusoe, M-32–33
Cryptanalysis, M-4
CSA. *See* Carry-save adder (CSA)
CUDA. *See* Compute Unified Device Architecture (CUDA)
Current frame pointer (CFM), H-33–34
Custom cluster
characteristics, **I-45**
IBM Blue Gene/L, I-41–44, **I-43–44**
Cut-through packet switching, F-51–53
CVI. *See* Create vector index instruction (CVI)
CYBER 205, M-47
vector processor history, G-26–27
Cycle time. *See also* Clock cycle time
CPI calculation, 375–376
memory hierarchy design, 85
Cyclic redundancy check (CRC)
IBM Blue Gene/L 3D torus network, F-76
network interface, F-8
Cydrome, M-31–33

D

DAG. *See* Directed acyclic graph (DAG)
Dark silicon, 28
DASH multiprocessor, M-61
Data addressing modes, K-32–35
Data cache
cache performance, B-16–17
TLB, B-46
Data cache miss
applications *vs.* OS, **B-59**
Opteron, B-12–15, **B-13**

sizes and associativities, **B-10**
writes, **B-10**
Datacenters, containers, M-76
Data dependences
conditional instructions, H-24
definition, 170–172
example calculations, H-3–4
instruction-level parallelism, 170–172
maintenance, 175
Data fetch (DF), C-58–59
Data flow
control dependence, 174–176
execution, hardware-based speculation, 209
global code scheduling, H-17–18
limit, M-34
Data hazards
definition, C-11
dynamic scheduling, 191–201
instruction set complications, C-45
microarchitectural techniques case study, 266–273
pipelined execution of instructions, C-13
program order, 173–174
stall minimization by forwarding, C-14–15, **C-15–16**
stall requirements, C-16–17
types, C-12
Data integration (DI), 44
Data-level parallelism (DLP), 5, 10–11
computer design principles, 48
cross-cutting issues
banked memory and graphics memory, 346
energy and DLP, 345
strided accesses and TLB misses, 346
energy and, 345
graphics processing units
conditional branching, 323–326
multimedia SIMD computers *vs.*, 335
NVIDIA computational structures, 313–320
NVIDIA GPU instruction set architecture, 320–323
NVIDIA GPU memory structures, 326–328, **327**
Pascal GPU architecture, 328–331
programming, 310–313
quick guide, **314**
vector architectures *vs.*, 331–334, **332**
loop-level parallelism
analysis, 337–339
CUDA/NVIDIA term, **337–338**
dependent computations, 344–345
finding dependences, 339–344
SIMD Multimedia Extensions, 304–310
vector architecture, 282
execution time, 290–293
vs. graphics processing units, 331–334
memory banks, 298–299
multidimensional arrays, 299–301
multiple lanes, 293–294
predicate registers, 296–298
processor example, 288–290
programming, 302–304
RV64V extension, 283–287, **284**
sparse matrices, 301–302
vector-length registers, 294–296
WSCs, 467
Data link layer
definition, **F-84**
interconnection networks, F-10
Data parallelism, M-56
Data-race-free, 419
Data races, 419
Data transfers
cache miss rate calculations, B-16
RISC-V, A-36
Data trunks, C-70
Data types, dependence analysis, H-10
Dauber, Phil, M-29–30
DAXPY loop
chained convoys, **G-16**
on enhanced VMIPS, G-19–21
vector performance measures, G-16
VMIPS, G-19–21
calculations, G-18
on Linpack, G-18
peak performance, G-17
D-cache, way prediction, 98–99
DDR. *See* Double data rate (DDR)
DDR3 memory systems, 153–155
Deadlock, F-45–46, 386
avoidance, F-46
large-scale multiprocessor cache coherence, I-34–35, I-38–40
recovery, F-46
Dead time, vector pipeline, G-8, **G-8**
DEC Alpha processor, K-3
Decoder, radio receiver, **E-23**
Decode stage, TI 320C55 DSP, E-7
DEC PDP-11, address space, B-57–58
DECstation 5000, **F-73**
DEC VAX
address space, B-57–58
cluster history, M-62, M-74
computer architecture definition, M-19
early computer arithmetic, J-63–64
early pipelined CPUs, M-28
failures, D-13–15
integer overflow, **J-11**
RISC history, M-20
DEC VAX-11/780, M-6–7, M-11, M-19
DEC VAX 8700
vs. MIPS M2000, **M-22**
RISC history, M-22
Dedicated link network
black box network, F-5–6
effective bandwidth, F-18
example, **F-6**
Deep neural networks (DNNs)
acceleration, 606–613
activation, 546
applications, **547**, **595**
batches, 556
convolutional neural network, 550–552
CPUs and GPUs *vs.*, 595–602
multilayer perceptron, 549–550
neurons of, 546–547
performance summary, 603
quantization, 556
recurrent neural network, 553–555
training set sizes/time, **548**

Deep neural networks (DNNs) *(Continued)*
training *vs.* inference, 547–549
weights/parameters, 546
Defect tolerance, 67–68
Delayed branch
behavior, **C-20**
compiler history, M-33
definition, C-20
Dell PowerEdge servers, 55–58, **56**
Dell Poweredge Thunderbird, **F-80**
Demand access, memory hierarchy design, 138
Demodulator, radio receiver, **E-23**
Dennard scaling, 4–5, 58, 368–369, 442
Denormals, J-14–15, J-20–21
floating point addition, J-26–27
floating-point underflow, J-36
Dense matrix multiplication, LU kernel, I-8
Density-optimized processors, *vs.* SPEC-optimized, F-89
Dependability
benchmark examples, D-21–23
definition, D-10–11
disk operators, D-13–15
integrated circuits, 36–38
Internet Archive Cluster, D-38–40
in memory systems, 93–94
via redundancy, 467
Dependence analysis
basic approach, H-5
example calculations, H-7
limitations, H-8–9
Dependence distance, loop-carried dependences, H-6
Dependences
control, 174–176
data, 170–172
finding, H-6–10
loop-level parallelism, H-3
name, 172–173
sparse matrices, G-12–13
types, 170–171
Dependent computations, H-10–12, 344–345
Descriptor privilege level (DPL), **B-53**
Descriptor tables, B-52
Design faults, D-11
Desktop benchmarks, 41–43
Desktop computers
interconnection networks, **F-72**
multimedia support, **E-11**
RAID history, M-87
RISC architectures survey for, K-3–29
system characteristics, **E-4**
Desktop computing, A-2, 8
Desktop/server RISC architectures, instruction formats for, **K-8**
Destination offset, IA-32 segment, **B-53**
Deterministic routing algorithm, F-46–47
DF. *See* Data fetch (DF)
Dies, 31
embedded systems, E-15
Intel Core i7 microprocessor, **32**
RISC-V, **33**
yield, 33–34
Digital Alpha
conditional instructions, H-27
Digital Alpha 21064, M-48
processors
MAX, multimedia support, **E-11**
recent advances, M-35
synchronization history, M-64–65
Digital Equipment Vax, 2
Digital Linear Tape, M-85
Digital signal processor (DSP)
cell phones, E-23–24, **E-23**
definition, E-3
desktop multimedia support, **E-11**
embedded RISCs, K-28
examples and characteristics, **E-6**
media extensions, E-10–11
overview, E-5–7
TI TMS320C55, **E-6–7**, E-7–8
TI TMS320C6x, E-8–10
TI TMS320C64x, **E-9**
TI TMS320C6x instruction packet, **E-10**
Dimension-order routing (DOR), F-46–47
Direct attached disks, D-35
Directed acyclic graph (DAG), 582
Direct-mapped cache, B-7, **B-8**
address translation, B-38
early work, M-11
memory hierarchy, **B-48**, 81
Direct memory access (DMA)
historical background, M-89
network interface functions, F-7
Sanyo VPC-SX500 digital camera, E-19
Sony PlayStation 2 Emotion Engine, E-18
TI TMS320C55 DSP, E-8
zero-copy protocols, F-95
Direct networks, F-35, **F-37**, F-96
Directory-based cache coherence, 380, 391–392
case study, 451–452
home node, 406
large-scale multiprocessors history, M-61
local node, 406
operations, 406
protocol example, 408–412
remote node, 406–407
state transition diagram, 408, **409–410**
Directory-based multiprocessor
characteristics, **I-31**
scientific workloads, I-26, I-29
synchronization, I-16, I-19–20
Directory controller, cache coherence, I-40–41
Directory protocol, 404
DirectX 9, M-51–52
DirectX 10 generation, M-52–53
Dirty bit, B-11, B-46, D-61–64
Dirty block, B-11, B-36
Discrete cosine transform, DSP, E-5
Disk arrays
deconstruction case study, D-51–54
RAID 6, D-8–9
RAID levels, D-6–10
Disk layout, RAID performance prediction, D-57–59
Disk power, D-5
Disk storage, D-2–10, D-48–50
Disk system
performance milestones, **22**
subsystem, failure rates of, 51–52
workload measurements, 400
Dispatch stage, 266–273
Displacement addressing, K-52
Displacement-style addressing mode, A-11–12

Display lists, Sony PlayStation 2 Emotion Engine, E-17
Distributed routing, F-49
Distributed shared memory (DSM), 371, 373
  access time, 372–373
  architecture, **373**
  characteristics, **I-45**
  directory-based cache coherence, 404–412, **405**
  disadvantages, 372–373
  multicore processor, **373**, **405**, **452**
Distributed shared-memory multiprocessors
  cache coherence implementation, I-36–37
  scientific application performance, I-26–32, **I-28–32**
Distributed switched networks, F-35–40
Divide operations
  chip comparison, J-61
  floating-point iterative, J-27–31
  integer shifting over zeros, J-45
  integers, speedup
    radix-2 division, **J-55**
    radix-4 division, **J-56**
    radix-4 SRT division, **J-57**
    with single adder, J-54–57
    SRT division, J-45–47, **J-46**, J-55–57
  language comparison, **J-12**
  n-bit unsigned integers, **J-4**
  Radix-2, J-4–7, **J-4**
  restoring/nonrestoring, J-5, **J-6**
  SRT division, J-45–47, **J-46**
DLP. *See* Data-level parallelism (DLP)
DLX, integer arithmetic, J-11–12
DNNs. *See* Deep neural networks (DNNs)
Domain-specific architectures (DSAs), 5
  architecture renaissance, 605–606
  cost-performance, 600–601
  CPUs and GPUs *vs.* DNN accelerators, 595–602
  custom chip, 602
  deep neural networks
    activation, 546
    applications, **547**
    batches, 556
    convolutional neural network, 550–552
    multilayer perceptron, 549–550
    neurons of, 546–547
    quantization, 556
    recurrent neural network, 553–555
    training set sizes/time, **548**
    training *vs.* inference, 547–549
    weights/parameters, 546
  designing, 604
  guidelines for, 543–544, **543**
  heterogeneity, 592–594
  Intel Crest, 579
  ISPs, 580–582
  Microsoft Catapult
    board design, **568**
    CNNs on, 570–572, **571–572**
    evaluating, 601–602
    guidelines, 577–579
    implementation and architecture, 568–569
    search acceleration on, 573–574
    software, 569
    version 1 deployment, 574
    version 2 deployment, 575–577, **576–577**
  open instruction set, 594
  performance counters, 603
  performance/watt, 600–601
  Pixel Visual Core
    architecture philosophy, 583–584
    evaluating, 601–602
    example, 588
    floor plan, **592**
    Halo, 584–585
    implementation, 590–591
    instruction set architecture, 587–588
    line buffers in, 590
    processing element, 588–589
    processor, 585–587
    software, 582
    two-dimensional array, **586**
    two-dimensional line buffers, 589–590
  response time, 596–600
  rooflines, 596–600
  system on a chip, 592–594
  systolic array, **561**
  TCO, 600–601
  tensor processing unit
    architecture, 557–558
    block diagram, **558**
    case study, 606–617
    die, **562**
    guidelines, 566–567
    implementation, 560–563
    improving, 564–566
    instruction set architecture, 559
    microarchitecture, 559–560
    origin, 557
    printed circuit board, **563**
    software, 563
    TensorFlow program, **564**
  throughput, 596–600
Double data rate (DDR), 87, 399
  IBM Blue Gene/L, I-43
  InfiniBand, F-81
Double-extended floating-point arithmetic, J-33–34
Double failures, RAID reconstruction, D-55–57
Double-precision floating point, **C-63**, 329
  chip comparison, **J-58**
  DSP media extensions, E-10
Double rounding
  FP precisions, J-34
  FP underflow, J-36–37
Double words, A-7, **A-8**, **A-14**, K-35, 300
DPL. *See* Descriptor privilege level (DPL)
DRAM. *See* Dynamic random-access memory (DRAM)
DRDRAM, Sony PlayStation 2, E-16
Driver domains, Xen virtual machine, 126
DSAs. *See* Domain-specific architectures (DSAs)
DSM. *See* Distributed shared memory (DSM)
DSP. *See* Digital signal processor (DSP)
Dual inline memory modules (DIMMs), F-74, 89
Dynamically allocatable multi-queues (DAMQs), F-56
Dynamically shared libraries, A-18

Dynamic branch prediction, C-23–25
Dynamic energy, 25
Dynamic network reconfiguration, F-71–73
Dynamic power, 80
Dynamic programming feature (DPF), 577
Dynamic random-access memory (DRAM)
  arithmetic operations and energy cost, **29**
  clock rates, bandwidth, and names, **89**
  cost *vs.* access time, **D-3**
  Cray X1, G-22
  dependability, 516
  die stacking, **91**
  disk storage, D-3
  embedded benchmarks, E-13
  errors and faults, D-11
  first vector computers, M-47–49
  IBM Blue Gene/L, I-43–44
  internal organization, **86**
  magnetic storage history, M-86
  memory hierarchy design, 85–87
  memory performance improvement, 87–90
  PlayStation 2, **E-16**, E-17
  price pressures, 34
  semiconductor, 19
  stacked/embedded, 91
  timing parameters, 153–155
  vector memory systems, G-9–10
  vector processor, G-9–10, G-25
Dynamic register typing, 287
Dynamic scheduling
  advantages, 191–192
  data hazards, 191–201
  definition, C-65–66
  first use, M-28–29
  out-of-order execution, 193–194
  with scoreboard, C-66–70, **C-68**
  Tomasulo's algorithm, 195–204
    loop-based example, 204–208, **206**
    steps, **205**
  unoptimized code, C-70
Dynamic voltage-frequency scaling (DVFS), 27

E

Early restart, cache optimization, 104–105
Earth Simulator, M-48–49, M-63
ECC. *See* Error-Correcting Code (ECC)
Eckert, J. Presper, M-2–5, M-20
Eckert-Mauchly Computer Corporation, M-5, M-57
ECL minicomputer, M-20
EEMBC. *See* Electronic Design News Embedded Microprocessor Benchmark Consortium (EEMBC)
Effective address, A-8–9
  RISC classic pipeline, C-8
  RISC instruction set, C-5
  simple RISC implementation, C-27
  TLB, B-49
Effective bandwidth
  definition, F-13
  example calculations, F-18–19
  vs. packet size, **F-19**
  two-device networks, F-13–20
Efficiency factor, F-53, F-55–56
Eight-way set associativity
  cache optimization, B-28–29
  conflict misses, B-23
  data cache misses, **B-10**
Elapsed time, 39
Electronically erasable programmable read-only memory (EEPROM), 92
Electronic Design News Embedded Microprocessor Benchmark Consortium (EEMBC), 41
  benchmark classes, E-12
  kernel suites, **E-12**
  power consumption and efficiency metrics, E-13, **E-13–14**
Electronic Discrete Variable Automatic Computer (EDVAC), M-2–3
Electronic Numerical Integrator and Calculator (ENIAC), M-2–3, M-85
Embedded applications, A-2
Embedded computer, 6–7
  RISC architectures survey for, K-3–29
Embedded DRAM, 91
Embedded multiprocessors, characteristics, E-14–15
Embedded systems
  benchmarks
    basic considerations, E-12
    power consumption and efficiency, E-13, **E-13–14**
  cell phone case study
    block diagram, **E-23**
    characteristics, E-22–24
    Nokia circuit board, **E-24**
    overview, E-20
    radio receiver, **E-23**
    standards and evolution, E-25
    wireless networks, E-21–22
  characteristics, **E-4**
  digital signal processor
    cell phones, E-23–24, **E-23**
    definition, E-3
    desktop multimedia support, **E-11**
    examples and characteristics, **E-6**
    media extensions, E-10–11
    overview, E-5–7
    TI TMS320C55, **E-6–7**, E-7–8
    TI TMS320C6x, E-8–10
    TI TMS320C64x, **E-9**
    TI TMS320C6x instruction packet, **E-10**
  EEMBC benchmark suite, **E-12**
  overview, E-2
  performance, **E-13–14**
  real-time processing, E-3–4
  Sanyo digital cameras, SOC, **E-20**
  Sanyo VPC-SX500 digital camera case study, E-19
  Sony PlayStation 2 case study, E-15–18
    block diagram, **E-16**
    organization, **E-18**
EMC, M-87–88
Emotion Engine
  organization modes, **E-18**
  Sony PlayStation 2 case study, E-15–18
empowerTel Networks, MXP processor, E-14–15

Enclaves, instruction set extensions, 125
Encore Multimax, M-59–60
End-to-end flow control, F-69
Energy
  and DLP, 345
  limits of, 28–29
  within microprocessor, 25–28
  proportionality, 503
  systems perspective, 23–24
Energy efficiency, 434–437, 467. *See also* Power consumption
  embedded benchmarks, E-13
Engineering Research Associates (ERA), M-4–5
ENIAC. *See* Electronic Numerical Integrator and Calculator (ENIAC)
Enigma coding machine, M-4
Entry time, transactions, D-16, **D-17**
Environmental faults, storage systems, D-11
EPIC approach
  historical background, M-33
  IA-64, H-33
E-24 RF. *See* Register fetch (RF)
Error correcting codes (ECCs), 93–94
  disk storage, D-11
  hardware dependability, D-15
  RAID 2, D-6
Error handling, interconnection networks, F-9–12
Errors, definition, D-10
Escape resource set, F-47–48
ETA processor, G-26–27
Ethernet, 478
  and bandwidth, **F-82**, F-93
  LANs, F-4, F-82–84, F-103–104
  packet format, **F-79**
  shared-media networks, F-23–24
  shared- *vs.* switched-media networks, **F-23**
  switch *vs.* NIC, **F-90**
  system area networks, F-76–77
  total time statistics, **F-94**
  WAN, F-84–85
Eugene, Miya, M-65
European Center for Particle Research (CERN), F-102
Even/odd array
  example, **J-52**
  integer multiplication, J-51–52
EVEN-ODD scheme development, D-10
Exception
  arithmetic-logical units, C-5
  categories, **C-40**
  control dependence, 174–175
  floating-point, C-41–42
  floating-point arithmetic, J-34–35
  imprecise, 194
  memory protection, 175
  precise, C-41–44
  preservation via hardward support, H-28–32
  RISC V, C-42–43, **C-42**
  stopping/restarting, C-41–42
  types and requirements, C-38–41, **C-40**
  unexpected sequences, C-70
Execute step
  Itanium 2, H-42
  TI 320C55 DSP, E-7
Execution, C-69, 198, 211
Execution address cycle (EX)
  data hazards requiring stalls, **C-18**
  data hazards stall minimization, C-14
  dynamic scheduling pipelines, C-66
  exception stopping/restarting, C-41
  floating point pipeline, C-46
  longer latency pipelines, C-51
  MIPS R4000 pipeline, C-58–59
  pipeline branch issues, C-35–36
  RISC exception, **C-42–43**, C-43
  RISC instruction set, C-5, **C-6**
  RISC pipeline, C-32–35
  simple RISC implementation, C-27
Execution time, 39
  Amdahl's law, 50
  application/OS misses, **B-59**
  cache performance, B-3
  central processing unit, B-3, B-5, **B-22**
  components, 400
  multiprocessor, 438
  multiprogrammed parallel "make" workload, **400**
  pipelining performance, C-3, C-8–10
  second-level cache size, B-32, **B-34**
  and stall time, B-21
  vector length, **G-7**
Expand-down field, **B-53**
Explicit parallelism, H-34–37
Explicit unit-stride, 333
Exponential back-off
  large-scale multiprocessor synchronization, I-17–18
  spin lock, **I-17**
Exponential distribution, D-27
Extended accumulator, A-3, K-30
Extended stack architecture, K-30

F

Fabrication cost, 67–68
Fabrication yield, 67–68
Failure. *See also* Mean time between failures (MTBF); Mean time to failure (MTTF)
  Berkeley's Tertiary Disk project, D-12
  definition, D-10
  dependability, 37–38
  dirty bits, D-61–64
  RAID
    reconstruction, D-55–57
    row-diagonal parity, **D-9**
  rates of disk subsystem, 51–52
  storage system, D-6–10
    components, D-43
  Tertiary Disk, D-13
Failures in time (FIT), 37
False sharing, 393–394, **398**
Fast Fourier transformation (FFT)
  characteristics, I-7
  distributed-memory multiprocessor, **I-32**
  example calculations, I-27–29
  symmetric shared-memory multiprocessors, I-22, **I-23**, I-25–26
Fat trees, F-34, **F-38**
Fault, 111
  definition, D-10
  dependability benchmarks, D-21
  programming mistakes, D-11
  Tandem Computers, D-13
Fault detection, 64
Fault-induced deadlock, F-45–46

Fault tolerance, F-70–72, F-98
dependability benchmarks, D-21
RAID, **D-7**
Fault-tolerant routing, F-70–71, F-98–99
FC. *See* Fibre Channel (FC)
FC-AL. *See* Fibre Channel Arbitrated Loop (FC-AL)
Feature Extraction, 573, **574**
Feature functional unit (FFU), 576
Feature maps, two-dimensional, 550
Feature size, 21
FEC. *See* Forward error correction (FEC)
Federal Communications Commission (FCC), D-15
FENCE in RISC V, 420–422
Fetch-and-increment, 413–414
large-scale multiprocessor synchronization, I-20–21
sense-reversing barrier, **I-21**
Fetch stage, TI 320C55 DSP, E-7
FFT. *See* Fast Fourier transformation (FFT)
Fibre Channel (FC), F-106
file system benchmarking, **D-20**
NetApp FAS6000 filer, D-41–42
Fibre Channel Arbitrated Loop (FC-AL), M-88, F-106
block servers *vs.* filers, D-35
Fibre Channel Switched (FC-SW), F-106
Filers
vs. block servers, D-34–35
NetApp FAS6000 filer, D-41–43
servers, SPEC benchmarking, D-20–21
Filters, radio receiver, **E-23**
Fine-grained multithreading, 243–244
Fingerprint, storage system, D-48
Finite-state machine, F-49, F-56–58
Firmware, network interfaces, F-7–8
First-in first-out (FIFO), B-9, **B-10**, 197
definition, D-26
First-level caches
cache optimization, B-30–35
hit time/power reduction, 95–98
interconnection network, F-74
Itanium 2, **H-41**
memory hierarchy, B-48–49, **B-48**
parameter ranges, **B-42**
First-reference misses, B-23
Fixed-field decoding, C-5
Fixed length, 14
Fixed-point arithmetic, DSP, E-5–6
Flash memory
disk storage, D-3–4
embedded benchmarks, E-13
memory hierarchy design, 92–93
technology trends, 19
FLASH multiprocessor, M-62
Flexible chaining, 290–291
vector processor, G-11
Flex point, 579
Floating-point (FP) operations, K-38–40
addition
denormals, J-26–27
overview, J-21–25
rules, **J-24**
speedup, J-25–26
chip comparison, **J-58**
CPI, **C-64**
data dependences, 171
denormals, J-14–15, J-20–21, J-26–27
double-precision, **C-63**
DSP media extensions, E-10–11
early computer arithmetic, J-64–65
exceptions, J-34–35, C-41–42
fused multiply-add, J-32–33
IEEE 754, **J-16**
integer conversions, J-62
Itanium 2, **H-41**
iterative division, J-27–31
latency, **C-61**, C-63, **177**
and memory bandwidth, J-62
micro-op fusion, 254
MIPS R4000 pipeline, C-60–61, **C-60**
mispeculation, **239**
multiplication
denormals, J-20–21
examples, **J-19**
overview, J-17–20
rounding, **J-18**, J-19
multiplication precision, J-21
multiply and add operation, **C-62**
number representation, J-15–16
overflow, **J-11**
overview, J-13–14
performance, 308
pipeline scheduling, 178
precisions, J-33–34
programs, 101–102
register file, **C-50**
remainder, J-31–32
result stalls, C-61, **C-64**
RISC exception, C-43
RISC multicycle operations, C-45–55
RISC pipeline, C-45–55, **C-47–48**, **C-57**
RISC-V, A-40–41
special values, J-14–15, **J-16**
square root, 51
static branch prediction, **C-23**
structural stalls, C-61, **C-64**
Tomasulo's algorithm, **198**
underflow, J-36–37, J-62
vector chaining, G-11
Floating-point registers (FPRs)
IA-64, H-34
IBM Blue Gene/L, I-42
Floating Point Systems AP-120B, M-30
Floppy disks, M-85–86
Flow-balanced state, **D-24**
Flow control
and arbitration, F-22
interconnection networks, F-9–12
Fluent, F-80–81
Flush, branch penalty reduction, C-19
Forget gate, 553
Form factor, interconnection networks, F-9–10
FORTRAN
compiler vectorization, G-14, **G-15**
dependence analysis, H-6
integer division/remainder, **J-12**
performance measurement history, M-6
Forward error correction (FEC), E-5–7
Forwarding, C-14
arithmetic-logical units, **C-36–37**
data hazards stall minimization, C-14–15, **C-15–16**
load instruction, **C-17**
longer latency pipelines, C-49–52
table, F-56–58
Forward path, cell phones, E-24
Fourier-Motzkin algorithm, M-32
Fourier transform, DSP, E-5

Four-way set associativity, B-23
FPGA, 568–569
  Catapult, 567
  Feature Extraction, **574**
FP operations. *See* Floating-point (FP) operations
Fragmentation problem, 114
Frame pointer, K-57
Free-form expressions, 573–574
Freeze, branch penalty reduction, C-19
Frequency modulation (FM), wireless neworks, E-21
Front-end stage, Itanium 2, H-42
FU. *See* Functional unit (FU)
Fujitsu Primergy BX3000 blade server, F-89
Fujitsu SPARC64 X+, 389, 426, 429
  feature, **427**
  performance, 429–431, **432**
Fujitsu VP100, M-48
Fujitsu VP200, M-48
Full access
  dimension-order routing, F-47–48
  interconnection network topology, F-30
Full adders, J-2–3, **J-3**
Full-duplex mode, F-23
Fully associative cache, B-7–9, B-12, 81
Fully connected layer, 549
Fully connected topology, F-35–36, **F-35–36**
Functional hazards, 266–273
Functional unit (FU), C-46
  execution slots, superscalar processors, 244–245, **244**
  Itanium 2, H-41–43
  latency, **C-47**, 177
  OCNs, F-3
Function pointers, A-18
Fused multiply-add, floating point, J-32–33
Future file approach, C-54

G

Gates, 553
Gateways, Ethernet, F-83
Gather-scatter, A-31–32, 301–302, 352
  sparse matrices, G-13–14
GE 645, M-9–10
GeForce 8800, M-52
General-Purpose Computing on GPUs (GPGPU), M-52
General-purpose electronic computers, M-2–4
General-purpose registers (GPRs)
  architectures, A-3
  IA-64, H-38
Geometric mean, 46
Gibson mix, M-6
Global address space, B-52
Global code scheduling
  example, **H-16**
  parallelism, H-15–23
  superblock scheduling, H-21–23, **H-22**
  trace scheduling, H-19–21, **H-20**
Global common subexpression elimination, A-26
Global data area, A-29
Global Environment for Network Innovation (GENI), F-102
Global miss rate, B-31
Global optimizations, A-26
Global Positioning System, CDMA, E-25
Global predictors, 184–188
Global scheduling algorithms, 219–220
Global system for mobile communication (GSM), cell phones, E-25
Goldschmidt's division algorithm, J-29–30
Goldstine, Herman, M-2–3
Google
  clusters history, M-63
  containers, M-76
Google App Engine, M-75–76
Google Clusters, 94
  power consumption, F-89
Google File System (GFS), 474
Google Translate, 4, 7, 40–45
Google WSCs
  airflow, 506
  availability zones, **498**
  cooling, 506–508
  generators, **505**
  networking, 510–511
  network switches, **502**
  network traffic, **501**
  on-site substation, **504**
  power distribution, 504–506
  power utilization efficiency of, **485**
  racks, 509–510, **509**, **512**
  servers, **505**, 512–513, **513**
  switch gear, **505**
  transformers, **505**
Gordon Bell Prize, M-58
GPGPU. *See* General-Purpose Computing on GPUs (GPGPU)
GPRs. *See* General-purpose registers (GPRs)
Gradual underflow, J-15, J-36
Grain size, 370
Grant phase, arbitration, F-49–50
Graph coloring, A-27
Graphical Processor Units (GPUs)
  computing history, M-52–53
  historical background, M-50–51
  scalable, M-51
Graphics data RAMs (GDRAMs), 90
Graphics-intensive benchmarks, 41
Graphics memory, 346
Graphics pipelines, M-51–52
Graphics processing unit (GPU), 10
  conditional branching, 323–326
  DNN and CPUs *vs.*, 595–602
  embedded *vs.* server, 346–353
  fallacy, 353
  multimedia SIMD and MIMD *vs.*, 347–353
  multimedia SIMD computers *vs.*, 335
  NVIDIA computational structures, 313–320
  NVIDIA GPU instruction set architecture, 320–323
  NVIDIA GPU memory structures, 326–328, **327**
  Pascal GPU architecture, 328–331
  programming, 310–313
  quick guide, **314**
  vector architectures *vs.*, 331–334, **332**
  vector kernel implementation, 357–359
Graphics synchronous DRAMs (GSDRAMs), 90
Graphics Synthesizer, Sony PlayStation 2, E-16–18, **E-16**

Greatest common divisor (GCD), 342–343
test, loop-level parallelism dependences, H-7
Grid computing, M-75
Grid mapping, 315, **316**
Grid topology, F-36–38
Gshare predictors
tagged hybrid *vs.*, **190**
2-bit predictor, 184, **186**, 262
Guest domains, 126
Guest virtual machine, 121

H

Half adders, J-2–3
Half-duplex mode, F-23
Half-precision floating-point arithmetic, 329
Halo, 584–585
HAMR, 19
Handshaking, interconnection networks, F-10
Hard cores, Cortex-A53, 130
Hard errors, memory hierarchy design, 93
Hard real-time systems, definition, E-3–4
Hardware, 17
compiler scheduling support, M-32–33
compiler speculation support
memory references, H-32
overview, H-27
preserving exception behavior, H-28–32
designing, 17–18
for exposing parallelism, H-23–27
faults, D-11
interconnection networks, F-8
pipeline hazard detection, **C-34**
Hardware-based speculation, 208–217
data flow execution, 209
definition, 208
disadvantage, 241
instruction execution step, 211–212
key ideas, 208
reorder buffer, 209–212, 214–215
vs. software speculation, 240–241
write result, 217
Hardware prefetching, 109–111, 148–164
Hardware primitives, 412–414
large-scale multiprocessor synchronization, I-18–21
Harvard architecture, M-3–4
Hazards. *See also* Data hazards
control hazards, C-11
data (*see* Data hazards)
definition, C-10–11
detection, hardware, **C-34**
functional, 266–273
instruction set complications, C-45
longer latency pipelines, C-49–52
read after write, C-12–14
structural (*see* Structural hazards)
write after read, C-12
write after write, C-12
Header
messages, F-7
packet format, **F-7**
switch microarchitecture pipelining, F-64
TCP/IP, F-87–89
Head-of-line (HOL) blocking, F-59–61, **F-60**
Heap, A-29
HEP processor, M-35
Heterogeneity, DSAs, 592–594
Heterogeneous architecture, 282
Hewlett-Packard AlphaServer, F-104–105
Hewlett-Packard PA-RISC
EPIC approach, M-33
floating-point precisions, J-33
MAX2, multimedia support, **E-11**
Hewlett-Packard RISC microprocessors, G-26
Hewlett Packard server, WSCs, 476
Hewlett-Packard's PA-RISC, K-3
Hidden layers, 546–547
High bandwidth memory (HBM), 346
cache optimization, 114–117
memory hierarchy design, 91
Pascal GPU architecture, 329
Higher-radix division, J-54–55
Higher-radix multiplication, integer, J-48
High-level language computer architecture (HLLCA), M-20
High-level optimizations, A-26
Highly parallel memory systems, 150–153
High-order functions, A-18
High-performance computing (HPC), 466
vector processor history, G-27–28
High Performance Fortran (HPF)—programs, 422
High-speed chip-to-chip interconnect, 329
Hillis, Danny, M-46, M-56, M-58–59, M-76
Histogram, D-26
History file approach, C-54
Hitachi S810, M-48
Hit time, B-15–16
address translation, 83
first-level caches, 95–98
latency, **115**
memory hierarchy design, 82
reducing, 94
reduction, B-36–40
way prediction, 98–99
HLLCA. *See* High-level language computer architecture (HLLCA)
Home node, 406
Hop count, F-30
Hops, F-36, **F-40**
Host
NVIDIA GPU memory structures, 327
virtual machine, 121
Host channel adapters (HCAs), F-90
historical background, M-89
Hot aisles, 506, **506**
Hot swapping, F-71–73
HPC. *See* High-performance computing (HPC)
HP Precision Architecture, integer arithmetic, J-11–12
HP ProLiant BL10e G2 blade server, F-89
HPSm, M-31
Hybrid predictors, 184
Hypercube networks, F-44, F-96
HyperTransport, F-67
NetApp FAS6000 filer, D-42
Hypervisor. *See* Virtual machine monitor (VMM)

I
IAS machine, M-3, M-5
IBM
  BlueGene, 370
  Chipkill, 94
  cluster history, M-62, M-74
  computer history, M-5
  early VM work, M-10
  IBM 360, address space, B-58
  IBM 370 architecture, 124
  magnetic storage, M-85–86
  multiple-issue processor development, M-29–30
  RAID history, M-87
IBM 360
  architects, M-10
  computer architecture definition, M-18
  I/O bus history, M-89
  memory hierarchy development, M-9–10
  parallel processing debates, M-58
IBM 360/85, M-11, M-29
IBM 360/91
  early computer arithmetic, J-63
  history, M-29
  speculation concept origins, M-31
IBM 370
  early computer arithmetic, J-63–64
  integer overflow, **J-11**
  vector processor history, G-27
IBM 370/158, M-6–7
IBM 650, M-5
IBM 701, M-5
IBM 702, M-5
IBM 704, M-5, M-27–28
IBM 705, M-5
IBM 801, M-20
IBM 3081, M-61
IBM 7030, M-27–28
IBM 360/370 architecture, K-69–70
  branches and special loads and stores—RX format, K-72
  branches and status setting R-R instructions, K-71
  branches/logical and floating-point instructions—RX format, K-71
  definition, K-69
  360 detailed measurements, K-70–74
  historical perspective and references, K-75
  integer/logical and floating-point R-R instructions, K-70
  measurements, K-70–74
  RS and SI format instructions, K-72
  SS format instructions, K-73
IBM AS/400, M-87
IBM Blue Gene/L, F-4, **I-44**
  cluster history, M-64
  computing node, I-42–44, **I-43**
  as custom cluster, I-41–44, **I-43–44**
  deterministic *vs.* adaptive routing, **F-53–56**
  parallel processing debates, M-59
  system area network, F-76–77
  3D torus network, F-39
IBM 3840 cartridge, M-85
IBM 9840 cartridge, M-85
IBM CoreConnect
  cross-company interoperability, F-68
  OCNs, F-3
IBM eServer p5 multiprocessor
  benchmarks, 440
  cost-performance, 440, **441**
IBM Federation network interfaces, F-18
IBM Power 1, M-31
IBM Power 2, M-31
IBM Power 4
  multithreading history, M-36
  recent advances, M-35
IBM Power 5, 424
  Itanium 2 comparison, **H-43**
  multithreading history, M-36
IBM Power 8, 371, 389–390, 426
  design, 429
  feature, **427**
  on-chip organizations, **428**
  performance, 431–432, **432**
IBM Power processors
  branch-prediction buffer, **C-25**
  characteristics, **265**
IBM Pulsar processor, M-35–36
IBM RP3, M-61
IBM RS/6000, M-58
IBM RT-PC, M-21
IBM SAGE, M-89
IBM Stretch, M-6
IBM 3090 Vector Facility, G-27
IBM zSeries, G-27
IC. *See* Instruction count (IC)
I-cache, way prediction, 98–99
ID. *See* Instruction decode (ID)
Ideal pipeline CPI, 169
IDE disks, Berkeley's Tertiary Disk project, D-12
Idle Control Register (ICR), TI TMS320C55 DSP, E-8
Idle domains, TI TMS320C55 DSP, E-8
IEEE arithmetic
  floating point, J-13–14
    addition, J-21–27
    exceptions, J-34–35
    multiplication, J-17–21
    remainder, J-31–32
    underflow, J-36–37
  historical background, J-63–65
  iterative division, J-30
  NaN, J-14
  rounding modes, **J-20**
  single-precision numbers, J-15
  $-x$ *vs.* $0 - x$, J-62
IEEE 754 floating-point standard, **J-16**
IEEE 1394, Sony PlayStation 2 Emotion Engine case study, E-15
IEEE standard 802.3 (Ethernet), F-82
  LAN history, F-82
IF cycle. *See* Instruction fetch (IF) cycle
Illiac IV, M-45, M-55, F-104
ILP. *See* Instruction-level parallelism (ILP)
Image processing units (IPUs), 580–582
Image signal processors (ISPs), 580–582
  hardwired predecessors of, 580–581
  interconnection of, **582**
Immediate addressing, K-52
IMPACT, M-33
Implicit unit stride, 333
Imprecise exceptions, 194
Inclusion, 383
  drawback, B-35
  implementation, 423–424
  L1 caches, 423–424

Inclusion *(Continued)*
L2 caches, 423–424
L3 caches, 424
memory hierarchy history, M-12
multilevel, B-34, 423
property, 78
Indexed addressing, K-34, K-53
Indexes
address translation during, B-36–40
Opteron data cache, B-13
recurrences, H-12
Index field, B-8–9
Index vector, 301–302
Indirect addressing, K-52
Indirect jumps, branch prediction, 232–234
Indirect networks, F-32–33
Inexact exception
floating-point arithmetic, J-35
floating-point underflow, J-36
InfiniBand, F-62, F-68, F-77–81, **F-79**
cluster history, M-64
Infinite population model, D-30
Initiation rate, 290
Inktomi, M-63, M-74–75
In-order commit, speculation concept origins, M-31
In-order execution
average memory access time, B-18
cache miss, B-2
IBM Power processors, **265**
Input buffered switch, F-56
Input gate, 553
Input-output buffered switch, F-56, **F-58**, **F-65**
Instruction cache
AMD Opteron example, B-15, **B-15**
application/OS misses, **B-59**
branch prediction, C-24
TI TMS320C55 DSP, E-8
Instruction commit, C-43–44, 209, 235
Instruction count (IC), B-4, 53
cache performance, B-15–16
processor performance equation, 52
RISC history, M-23
Instruction decode (ID)
branch hazards, C-18
data hazards requiring stalls, **C-18**
dynamic scheduling, C-66, 193
longer latency pipelines, C-50–51
MIPS R4000 pipeline, C-56
pipeline branch issues, C-35–36
RISC classic pipeline, C-8
RISC instruction set, C-5, **C-6**
RISC pipeline, C-32–34, **C-35**
simple RISC implementation, C-27
Instruction delivery, 228–240
stage, Itanium 2, H-42
Instruction fetch (IF), 253
bandwidth, 228–232, **229–230**
cycle
ARM Cortex-A53, 249–250
branch hazards, C-18
branch-prediction buffer, C-24
data hazards requiring stalls, **C-18**
exception stopping/restarting, C-41
MIPS R4000 pipeline, C-56
RISC exception, **C-42–43**, C-43
RISC instruction set, C-4, **C-6**
RISC pipeline, C-31–33
simple RISC implementation, C-27
units, integrated, 233–234
Instruction formats
addressing modes and, K-6–9
high-level language computer architecture, M-20
IA-64 ISA, H-34–38, **H-39**
Instruction groups, IA-64, H-34
Instruction issue, C-33–34
Itanium 2, H-41–43
Instruction-level parallelism (ILP), 5, 10, 368, 370
aggressive compiler-based approaches, 168
approaches, 168
branch prediction
correlating branch predictors, 182–184
Intel Core i7, 190–191
specialized, 232–234
tagged hybrid predictors, 188–190, **188**, **190**
tournament predictors, 184–188, **186**
branch-prediction buffer, C-25, **C-25**
clock cycles per instruction, 168–169
compiler scheduling, M-32
compiler techniques, 176–182
concepts, 169–170
control dependences, 174–176
data dependences, 170–172
data flow limit, M-34
data hazards, 173–174
definition, 168
dynamic scheduling, 222–227
advantages, 191–192
data hazards, 191–201
out-of-order execution, 193–194
Tomasulo's algorithm, 195–208, **205–206**
early studies, M-33–34
exploitation methods, H-21–23
exploitation of, 2
exploitation statically, H-2
exposing with hardware support, H-23–27
IA-64, H-32
loop unrolling, 177–182
microarchitectural techniques case study, 266–273
multiple-issue processors, M-31, 218–227
advantages, 221–222
challenges, 182, 221–222
characteristics, **219**
dynamically scheduled processor, 222, 224
EPIC approach, 221
microarchitectural techniques case study, 266–273
with speculation, **223**
superscalar, 218, 223
VLIW approach, 218–222, **220**
multithreading history, M-36
name dependences, 172–173
pipeline scheduling, 177–182
scaling, 442
speculation, 222–227
address aliasing prediction, 239–240
advanced techniques, 228–240
advantages, 237–238
challenge of issues per clock, 236–237

control dependence, 175–176
disadvantages, 238
and energy efficiency, 238–239
exception handling, 199
execution, 241
hardware *vs.* software, 240–241
microarchitectural techniques case study, 266–273
multiple branches, 238
register renaming *vs.* ROB, 234–236
static scheduling, 218–222
TI 320C6x DSP, E-8
Instruction path length, 52
Instruction prefetch, 234
Instruction register (IR)
RISC pipeline, C-31–32
simple RISC implementation, C-27
Instruction set architecture (ISA), 12–17. *See also* Intel 80x86 processors; Reduced Instruction Set Computer (RISC)
byte-addressed computers, **A-8**
changes, A-46–47
classes, **A-4**
classifying, A-3–6
class of, 12
complications, C-43–45
computer architecture definition, M-18–19
Cray X1, G-21–22
cross-cutting issues, 126–127
encoding, 14, A-21–24, **A-22**
fallacies and pitfalls, A-42–55
first vector computers, M-49
general-purpose register computers, **A-6**
high-level language computer architecture, M-20
IA-64
instruction formats, H-34–37, **H-39**
instructions, **H-35–37**
instruction set basics, H-38
overview, H-32–40
predication and speculation, H-38–40
register model, H-33–34
memory addressing, A-7–13
memory and total operands, **A-5**
MIPS
RISC history, M-20–23, **M-22**
stack architectures, M-17–18
operands, **A-4**, A-13–15
operations in, A-15–16
optimizations impact on performance, A-27
performance and energy efficiency, 258
Pixel Visual Core, 587–588
register allocation, A-27
RISC-V, A-34
TPU, 559
virtual machine, 120–121
for virtual machine, 122–123
Instruction set, extension, 124–125
Instructions per clock (IPC), 52, 169
Integer arithmetic
addition speedup
carry-lookahead, J-37–41
carry-lookahead circuit, **J-38**
carry-lookahead tree, **J-40**
carry-lookahead tree adder, **J-41**
carry-select adder, J-43–44, **J-43–44**
carry-skip adder, J-41–43, **J-42**
overview, J-37
division
radix-2 division, **J-55**
radix-4 division, **J-56**
radix-4 SRT division, **J-57**
with single adder, J-54–57
SRT division, J-45–47, **J-46**, J-55–57
FP conversions, J-62
language comparison, **J-12**
multiplication
array multiplier, **J-50**
Booth recoding, **J-49**
even/odd array, **J-52**
many adders, J-50–54, **J-50**
multipass array multiplier, **J-51**
signed-digit addition table, **J-54**
with single adder, J-47–49, **J-48–49**
Wallace tree, **J-53**
multiplication/division, shifting over zeros, J-45
overflow, **J-11**
Radix-2 multiplication/division, J-4–7, **J-4**
restoring/nonrestoring division, J-5, **J-6**
ripply-carry addition, J-2–3
signed numbers, J-7–10
SRT division, J-45–47, **J-46**
systems issues, J-10–13
Integer operations
ARM Cortex-A53, **249**
data dependences, 171
Itanium 2, **H-41**
mispeculation, **239**
RISC pipeline, C-45–55
stalls, C-55
static branch prediction, C-22, **C-23**
Integer registers, IA-64, H-33–34
Integrated branch prediction, 233
Integrated circuits
basics, cell phones, E-24, **E-24**
cost of, 31–35
dependability, 36–38
logic technology, 19
power and energy in, 23–29
Intel 80286, M-9–10
Intel Core i7, 100
branch prediction, 190–191
buffers and queues, **256**
hardware prefetching, 110
Intel Core i7 920
characteristics, **259**
clock cycles per instruction, 256, **257**
misprediction rate, **192**
relative performance and energy efficiency, **260**
Intel Core i7 6700, 55–56
clock cycles per instruction, 256, **257**
memory hierarchy design, 133–142, **134–135**
misprediction rate, **192**
multiple-issue processors, 247, 252–257
performance, 138–142, 255–257
pipeline structure, **254**
Intel Core i7 microprocessor
die, **32**
fallacy, 61
floorplan, **32**

Intel Core i7 920 multicore computer, **309**
Intel Crest, 579, **580**
Intel 8087, floating point remainder, J-31
Intel Haswell CPU Roofline, **599**
Intel i7, 388, **395**
Intel i7 920
  performance and energy efficiency, 434–437
  simultaneous multithreading, 246
Intel i860, M-30, M-32, M-50, M-60–61
Intel IA-32 architecture
  call gate, **B-53**, B-54
  descriptor table, B-52
  instruction set complications, C-45
  OCNs, F-3
  segment descriptor, B-52, **B-53**
  segmented virtual memory, B-51–54
Intel IA-64 architecture
  compiler scheduling history, M-32–33
  conditional instructions, H-27
  explicit parallelism, H-34–37
  historical background, M-33
  ISA
    instruction formats, H-34–37, **H-39**
    instructions, **H-35–37**
    instruction set basics, H-38
    overview, H-32–40
    predication and speculation, H-38–40
  Itanium 2 processor
    instruction latency, **H-41**
    overview, H-40–41
    performance, H-43, **H-43**
  parallelism exploitation statically, H-2
  register model, H-33–34
  RISC history, M-23
  software pipelining, H-14–15
  synchronization history, M-64–65
Intel iPSC 860, M-60–61
Intel Itanium, 168
  instruction-level parallelism, 261–262
  sparse matrices, G-13
  speculation, 241
Intel Itanium 2
  IA-64
    functional units and instruction issue, H-41–43
    instruction latency, **H-41**
    overview, H-40–41
    performance, H-43, **H-43**
Intellectual property, DSAs, 593
Intelligent devices, historical background, M-88
Intel Paragon, M-60–61, F-96
Intel Pentium 4, **110**, 261–262
  Extreme, M-35
  Itanium 2 comparison, **H-43**
  multithreading history, M-36
Intel Pentium II, M-35
Intel Pentium III, power consumption, F-89
Intel Pentium M, F-89
Intel Pentium MMX, multimedia support, **E-11**
Intel Pentium Pro, M-35
Intel Pentium processors
  early computer arithmetic, J-65
  vs. Opteron memory protection, B-57
  segmented virtual memory, B-51–54
Intel processor, **261**
  instruction set extensions, 125
  multiple processors, 5
  power consumption, F-89
Intel Teraflops processors, OCNs, F-3
Intel Thunder Tiger 4 QsNet^{II}, F-67
Intel 80x86
  comparative operation measurements, K-45–48
  floating-point operations, K-38–40
  instruction encoding, K-40–43
  integer operations, K-35–37
  measurements of instruction set usage, K-44–48
  operand addressing, K-44–45
  processors
    address space, B-58
    integer overflow, **J-11**
    memory hierarchy development, M-9–10
    protection structure, B-50
  registers and data addressing modes, K-32–35
  SPECint92 programs, **K-44**, **K-46**, **K-48–50**
Intel x86, conditional instructions, H-27
Intel Xeon, F-80, **354**, 387
Intel Xeon E7, 426
Interactive workloads, 467
Interarrival times, D-30
Interconnection networks
  adaptive routing, F-97
  adaptive routing and fault tolerance, F-98
  arbitration, F-49–51
  basic characteristics, F-2, **F-21**
  bisection bandwidth, F-93
  commercial
    congestion management, F-68–70
    connectivity, F-67
    cross-company interoperability, F-67–68
    DECstation 5000 reboots, **F-73**
    fault tolerance, F-70–72
  communication bandwidth, I-3
  compute-optimized processors *vs.* receiver overhead, F-92
  definition, F-2
  density- *vs.* SPEC-optimized processors, F-89
  device example, **F-3**
  direct *vs.* high-dimensional, F-96
  domains, F-3–4, **F-3**
  Ethernet, F-82–84
  examples, F-73–85
  HOL blocking, F-59–61
  IBM Blue Gene/L, I-43
  InfiniBand, F-77–81
  LAN, F-82–84
  link bandwidth, F-94
  memory hierarchy interface, F-91–92
  mesh network routing, **F-47**
  MIN *vs.* direct network costs, F-96
  multi-device connections
    basic considerations, F-20–21
    effective bandwidth *vs.* nodes, **F-29**
    latency *vs.* nodes, **F-28**
    performance characterization, F-25–30

shared-media networks, F-23–24
shared- *vs.* switched-media networks, **F-23**, F-25
switched-media networks, F-24–25
topology, routing, arbitration, switching, F-21–22
OCN, **F-27–29**
protection, F-91
routing, F-22, F-44–56
routing/arbitration/switching impact, F-21–22
SAN characteristics, **F-27–29**
software overhead, F-96
storage area networks, F-106–108
switching, F-51–52
switch microarchitecture, F-56–66
switch *vs.* NIC, **F-90**
system area networks, F-104–106
system/storage area network, F-77–81
top-level architecture, **F-75**
topology, F-30–44
two-device interconnections
basic considerations, F-6
effective bandwidth *vs.* packet size, **F-19**
example, **F-6**
interface functions, F-6–9
performance, F-13–20
structure and functions, F-9–12
virtual channels and throughput, F-47–48
WAN, F-84–85
wormhole switching performance, F-52
zero-copy protocols, F-95
Intermittent faults, D-11
Internal fragmentation, B-47
International Computer Architecture Symposium (ISCA), M-12
International Mobile Telephony 2000 (IMT-2000), cell phone standards, E-25
International Technology Roadmap for Semiconductors (ITRS), 58–59, **59**
Internet Archive Cluster, D-36–41
containers, M-76
Internet of Things (IoT), 6–7
Internet Protocol (IP), F-85–89
cores, OCNs, F-3
routers, VOQs, F-31, F-103
Internetworking, F-2, **F-84**, F-85–89
Interprocedural analysis, H-10
Interprocessor communication, large-scale multiprocessors, I-3–6
Interrupt Enable (IE) flag, 127
Invalidate protocol, 380
example, 385, **385**
implementation, 382–383
snooping coherence, **381**
Invalid exception, floating-point arithmetic, J-35
Inverted page table, B-44–45
I/O bandwidth, D-15–16
I/O benchmarks, response time restrictions, D-18
I/O-bound, 121, 123–124
I/O bus
historical background, M-88–89
point-to-point replacement, **D-34**
Sony PlayStation 2 Emotion Engine case study, E-15
I/O cache coherency, 128–129
I/O devices
address translation, B-38
historical background, M-88–89
performance, D-15–23
SANs, F-4
shared-media networks, F-23
switched networks, F-2
switch *vs.* NIC, F-90
write strategy, B-11
I/O interfaces, storage area network history, F-107
I/O network, F-67
I/O processor (IOP)
first dynamic scheduling, M-28–29
Sony PlayStation 2 Emotion Engine case study, E-15
I/O subsystems
design, D-59–61
interconnection network speed, F-92
vs. NIC, F-95
zero-copy protocols, F-95
I/O systems
asynchronous, D-35–36
as black box, **D-24**
dirty bits, D-61–64
multithreading history, M-35
queing theory, D-23
queue calculations, D-29
random variable distribution, D-26
IP block, DSAs, 593
IPC. *See* Instructions per clock (IPC)
IPoIB, F-81
IR. *See* Instruction register (IR)
ISA. *See* Instruction set architecture (ISA)
iSCSI
NetApp FAS6000 filer, D-41–42
storage area network, F-106–107
ISPs. *See* Image signal processors (ISPs)
Issue logic, 236
Issue stage
ID pipe stage, 194
instruction step, 197, 211
Iterative division, floating point, J-27–31

J

Java benchmark, 435–437, **435**, **437**
Java language, dependence analysis, H-10
Java Virtual Machine (JVM), early stack architectures, M-18
Johnson, Reynold B., M-85
Jumps, VAX, K-57
Just-in-time (JIT), M-18

K

Kahle, Brewster, M-76
Kahn, Robert, F-102
k-ary *n*-cubes, F-38–39
Kendall Square Research KSR-1, M-61–62
Kernels
EEMBC benchmarks, **E-12**
FFT, I-7
FORTRAN, compiler vectorization, **G-15**
LU, I-8
process, 40
DAG of, 582
Driver, 563
segmented virtual memory, B-51

Kernels *(Continued)*
throughput computing, 350
vector kernel implementation, 357–359
virtual memory, 119

L

LabVIEW, embedded benchmarks, E-13
Lampson, Butler, F-103
Large-scale multiprocessors
cache coherence implementation, I-34–35
deadlock and buffering, I-38–40
directory controller, I-40–41
DSM multiprocessor, I-36–37
characteristics, **I-45**
cluster history, M-63–64
example calculations, I-12–13
historical background, M-60–62
IBM Blue Gene/L, I-41–44, **I-43–44**
interprocessor communication, I-3–6
for parallel programming, I-2
scientific applications, I-6–12
distributed-memory multiprocessors, I-26–32, **I-28–32**
parallel processing, I-33–34
symmetric shared-memory multiprocessor, I-21–26, **I-23–26**
space and relation of classes, **I-46**
synchronization
mechanisms, I-17–21
performance, I-12–16
Latency, 20. *See also* Response time
ALU, C-46–48
bandwidth, F-25–30
barrier synchronization, I-16
and cache miss, B-2
cluster history, M-74
communication mechanism, I-3
definition, D-15–16, C-46–48
deterministic *vs.* adaptive routing, **F-53–56**
distributed-memory multiprocessors, I-30, **I-32**
Flash memory, D-3
FP operations, **C-61**, C-63, **177**
functional units, **C-47**
hazards and forwarding, C-49–52
interconnection networks, F-13–20
Itanium 2 instructions, **H-41**
microarchitectural techniques case study, 266–273
OCNs *vs.* SANs, **F-28**
out-of-order execution, B-20–21
packets, F-13, **F-13**
performance trends, 20, **21**
snooping cache coherence, 447, **448**
Sony PlayStation 2 Emotion Engine, E-17
throughput *vs.* response time, D-16
utility computing, M-75
vector memory systems, G-9
vector start-up, **G-8**
Latency-hiding techniques, 418
L1 cache. *See also* First-level caches
address translation, B-46
Alpha 21164, **395**
ARM Cortex-A53, 251–252
data cache size, **402**
first-level caches, 95–98
inclusion, 423–424
Intel i7, **395**
memory hierarchy, **B-39**, B-48–49, **B-48**
miss rate, **402**
Opteron memory, **B-57**
L2 cache. *See also* Second-level caches
Alpha 21164, **395**
ARM Cortex-A53, 251–252
cache optimization, **B-34**, B-35
IBM Blue Gene/L, I-42
inclusion, 423–424
Intel i7, **395**
memory hierarchy, **B-39**, B-48–49, **B-48**, **B-57**
memory system, 241
L3 cache. *See also* Third-level caches
Alpha 21164, **395**
IBM Blue Gene/L, I-42
IBM Power8, 371
IBM Power processors, **265**
inclusion, 424
Intel i7, **395**
memory access cycle shift, 396–397, **397**
miss rate, 397–399, **398**
snoop bandwidth, 390
Learning curve, 30
Least common ancestor (LCA), F-48–49
Least recently used (LRU)
AMD Opteron data cache, B-14
block replacement, B-9–10, **B-10**
memory hierarchy history, M-11
virtual memory block replacement, B-45
Limit field, B-52
Linear speedup, multiprocessor, 438–439, **440**
Line Buffer Pool (LBP), 590
Line locking, embedded systems, E-4
Line, memory hierarchy, 81
Link injection bandwidth
calculation, F-17
interconnection networks, F-26–27
Link pipelining, F-16–17
Link reception bandwidth, calculation, F-17
Link register, K-25
Linpack benchmark
cluster history, M-64
parallel processing debates, M-59
VMIPS performance, G-17–19
Linux operating system, RAID benchmarks, **D-22**
Liquid crystal display (LCD), Sanyo VPC-SX500 digital camera, E-19
LISP, K-21–22
RISC history, M-20–21
Little Endian
byte, A-7
interconnection networks, F-12
Little's Law, 328
definition, D-24
server utilization calculation, D-29
Livelock, network routing, F-45–46
Liveness, control dependences, 176
Livermore Fortran Kernels, M-6
Load instruction
control dependence, 175
data hazards requiring stalls, C-17, **C-17**
RISC instruction set, C-5
Load interlock, C-33–34, **C-35**
Load memory data (LMD), C-28–29
Load reserved, 413–414, 416
Loads instruction, 199

Load stalls, C-61, **C-64**
Load-store architecture, A-3
Load-store instruction set architecture, C-5, 12, C-28
  RISC history, M-20
Local address space, B-52
Local area networks (LANs)
  characteristics, F-4
  cross-company interoperability, F-67–68
  effective bandwidth, F-18–19
  Ethernet as, F-82
  fault tolerance calculations, **F-72**
  InfiniBand, F-77–78
  interconnection network domain relationship, F-4, **F-5**
  latency and effective bandwidth, F-27–29
  offload engines, F-8
  packet latency, F-13–16, **F-13**
  shared-media networks, F-23
  time of flight, F-14
Local memory
  centralized shared-memory architectures, 377
  multiprocessor architecture, 371–373
  NVIDIA GPU memory structures, 326–327
Local miss rate, B-31
Local node, 406
Local optimizations, A-26
Local predictors, 184–188
Local scheduling techniques, 219–220
Local state, 377
Location counts, WSCs, 468
Locks
  caching, 415
  large-scale multiprocessor synchronization, I-18–21
  spin, 414–416
  using coherence, 414–417, **416**
Lockup-free cache, 100–104
Logical units, storage systems, D-34
Logical volumes, D-34
Long displacement addressing, K-52
Long Instruction Word (LIW)
  EPIC approach, M-33
  multiple-issue processor development, M-30, M-32
Long mode, K-32
Long short-term memory (LSTM) cells, 553, **554–555**
Loop branches prediction, 232–234
Loop-carried dependence, 289, 312, 337–339
  dependence distance, H-6
  example calculations, H-4–5
  loop-level parallelism, H-3
  recurrence form, H-5
Loop interchange, cache optimization, 107
Loop-level parallelism, H-2–12
  analysis, 337–339
  CUDA/NVIDIA term, **337–338**
  definition, 169–170
  dependent computations, 344–345
  detection and enhancement, H-2–12
    dependence analysis, H-6–10
    dependent computation elimination, H-10–12
  finding dependences, 339–344
  history, M-32–33
  SIMD, 170
Loop stream detection, 254
Loop unrolling
  limitation, 181
  and scheduling, 181–182
  software pipelining, H-12–15, **H-13**, **H-15**
Lossless networks
  definition, F-12
  switch buffer organizations, F-59–60
Lossy networks, definition, F-12
LRU. *See* Least recently used (LRU)
LU kernel
  characteristics, I-8
  distributed-memory multiprocessor, **I-32**
  symmetric shared-memory multiprocessors, I-22, **I-23**, I-25–26

M

MAC. *See* Multiply-accumulate (MAC)
Machine language programmer, M-18
Machine learning, 546, 573
Machine memory, virtual machine, 123
Macro-op fusion, 253
Magnetic disk technology, 19
Magnetic storage
  access time, D-3
  cost *vs.* access time, **D-3**
  historical background, M-85–86
Mail servers, D-20–21
Main memory, B-2–3, 377, 400
  block identification, B-44–45
  block placement, B-44
  cache function, B-2
  vector processor, G-25
  vs. virtual memory, B-41
  write strategy, B-45–46
MapReduce
  AWS, 495, **495**
  WSCs, 471–476, **472**
Mark-I, M-3–4, M-6
Mark-II, M-3–4
Mark-III, M-3–4
Mark-IV, M-3–4
MasPar, M-46, M-56
Massively parallel processors (MPPs)
  characteristics, **I-45**
  cluster history, M-62–63, M-74
  system area network, F-104
Matrix300 kernel, prediction buffer, **C-25**
Matrix multiplication, LU kernel, I-8
Matrix multiply unit, 557–558, **558**, **562**
Mauchly, John, M-2–3, M-5, M-20
Maximum transfer unit, network interfaces, F-8
McCreight, Ed, F-103
McFarling's gshare predictor, 184
MCP operating system, M-17–18
Mean time between failures (MTBF), 37
Mean time to failure (MTTF)
  computer system power consumption case study, 69–71
  dependability, 37–38
    benchmarks, D-21
  disk arrays, D-6
  fallacy, 62
  I/O subsystem design, D-59–61
  RAID, M-86–87
  RAID reconstruction, D-55–57
  TB-80 cluster, D-41
  WSCs, 468

Mean time to repair (MTTR), 37
dependability benchmarks, D-21
disk arrays, D-6
RAID 6, D-8–9
RAID reconstruction, D-55–56
Mean time until data loss (MTDL), D-55–57
Media extensions, DSPs, E-10–11
Media, interconnection networks, F-9–12
Mellanox MHEA28-XT, F-80
Memory access
ALUs, data forwarding, **C-36–37**
cache hit calculation, B-5
Cray Research T3D, F-91, **F-91**
cross-cutting issues, 127–128
data hazards stall minimization, C-14, **C-16**
distributed-memory multiprocessor, **I-32**
exception stopping/restarting, C-41
instruction set complications, C-43–44
integrated instruction fetch units, 234
longer latency pipelines, C-51
MIPS R4000 pipeline, C-59
multicycle FP operations, C-52
RISC classic pipeline, C-8
RISC exception, **C-42–43**, C-43
RISC instruction set, C-5, **C-6**
RISC pipeline, C-32–35, **C-36**
simple RISC implementation, C-28
vector architectures, **G-10**
Memory addressing, 13
addressing modes, A-8–11, **A-10**
compiler-based speculation, H-32
displacement addressing mode, A-11–12
immediate/literal, A-12, **A-12**
interpreting, A-7–8
vector architectures, **G-10**
Memory bandwidth, 350, 356
Memory banks. *See also* Banked memory
example, 301
vector architecture, 298–299
vector systems, G-9–11
Memory bus (M-bus), 377
interconnection networks, F-91–92
Memory consistency, 377–378, 417–422
case study, 456–458
compiler optimization, 422
development of models, M-64–65
programmer's view, 418–419
relaxed consistency models, 419–422, **421**
sequential consistency, 417
speculation to hide latency, 422–423
Memory-constrained scaling, I-33–34
Memory hierarchy, F-91–92
address space, B-57
block identification, B-8–9
block placement, B-7–8, **B-7**
block replacement, B-9, **B-10**
cache optimizations, B-22–40, **B-40**
hit time reduction, B-36–40
miss categories, B-23–25
miss penalty reduction, B-30–36
miss rate reduction, B-26–30, **B-27**
cache performance, B-3–6, B-15–16, **B-40**
average memory access time, B-17–20
equations, **B-22**
example calculation, B-16–17
miss penalty, B-20–21
out-of-order execution, B-20–21
development, M-9–12
levels in slow down, **B-3**
Opteron data cache example, B-12–15, **B-13**, **B-15**
Opteron L1/L2, **B-57**
OS and page size, B-58
questions, B-6–12
terminology, B-2
virtual address to L2 cache, **B-39**
virtual memory, B-2–3, B-40–49
address space, B-12, **B-41**, B-44, B-55
address translation, B-46, **B-47**
caches and, B-42–43, **B-42**, B-48–49, **B-48**
classes, B-43
Intel Pentium *vs.* AMD Opteron, B-57
paged example, B-54–57
page size selection, B-46–47
parameter ranges, **B-42**
Pentium *vs.* Opteron protection, B-57
protection, B-49–50
questions, B-44–46
segmented example, B-51–54
write strategy, B-45–46
WSCs, 479–482, **479**
Memory hierarchy design
ARM Cortex-A53, 129–131, **130**
basics of, 81–84
cache optimization
advancement, **117**
cache misses, 112–113
compiler-controlled prefetching, 111–114
compiler optimizations, 107–109
critical word first, 104–105
early restart, 104–105
energy consumption, **97**
floating-point programs, 101–102
hardware prefetching, 109–111
HBM packaging, 114–117
hit time/power reduction, 95–98
multibanked caches, 99–100
nonblocking caches, 100–104
pipelined access, 99–100
way prediction, 98–99
write buffer merging, 105–106, **106**
Cortex-A53 performance, 132
C program evaluation, **151**
cross-cutting issues
autonomous instruction fetch units, 127
case study, 150–153
coherency of cached data, 128–129
protection, virtualization, and instruction set architecture, 126–127
special instruction caches, 128
speculation and memory access, 127–128
Intel Core i7 6700, 133–142, **134**
in personal mobile device, **79**
pitfall, 143

technology and optimizations
dependability, 93–94
DRAM technology, 85–87
Flash memory, 92–93
GDRAMs, 90
phase-change memory technology, 93
SRAM technology, 85
stacked/embedded DRAMs, 91
synchronous DRAM, 87–90
virtual machine
hardware management, 121
impact on virtual memory, 123–124
instruction set architecture for, 122–123
protection via, 120–122
software management, 121
virtual machine monitor
instruction set extension, 124–125
requirements, 122
Xen virtual machine, 126
virtual memory
instruction set extension, 124–125
protection via, 119–120
virtual machines impact on, 123–124
Memory latency, 85
Memoryless, D-27–28
Memory mapping, B-52
Memory-memory architecture, A-3
Memory protection
exception, 175
Pentium *vs.* Opteron, B-57
processes, B-49–50
safe calls, B-54
segmented virtual memory, B-51–54
Memory stall cycles
average memory access time, B-18
definition, B-3–4, **B-22**
miss rate calculation, B-6
out-of-order execution, B-20
Memory system, 377–378
coherency, 378
Intel Core i7 6700 pipeline structure, **254**
multiprocessor architecture, 369, 371–373
page size changes, B-58
speculative execution, 241
vector architectures, G-9–11
vector chaining, G-11
virtual (*see* Virtual memory)
Memristor, 93
Mesh interface unit (MIU), F-74
Mesh network, F-43, **F-47**
Mesh topology, F-74
MESIF protocol, 388
MESI protocol, 388, 449
Message ID, packet header, F-8, F-17
Message-passing communication
advantages, I-5–6
historical background, M-60–61
Message Passing Interface (MPI)
function, F-8
InfiniBand, F-80–81
lack in shared-memory multiprocessors, I-5
Message-passing protocols, 373
Messages
adaptive routing, **F-64**
interconnection networks, F-6–9
zero-copy protocols, F-95
MFLOPS. *See* Millions of floating-point operations per second (MFLOPS)
Microarchitecture, 17, **272**
case study, 266–273
Cray X1, G-21–22
OCNs, F-3
TPU, 559–560
Microbenchmarks
disk array deconstruction, D-51–54
disk deconstruction, D-48–50
Microinstructions, complications, C-45
MicroMIPS64, K-3
16-bit instruction, **K-7**, **K-10**, **K-17**
register encodings, **K-7**
Micro-op decode, 253
Micro-op fusion, ALU, 254
Microprocessor
AMD Opteron, **27**
clock rate, **26**
design for typical case, 27
do nothing well, 27
energy and power within, 25–28
growth rate, 2, **3**
inside disks, D-4
performance milestones, **22**
recent advances, M-35
VAX 11/780, **3**
Microprocessor without Interlocked Pipeline Stages (MIPS)
early pipelined CPUs, M-28
multiple-issue processor, M-30
performance measurement history, M-6–7
RISC history, M-20
Microsoft
availability zones, **499**
containers, M-76
Microsoft Azure, M-75–76
Microsoft Catapult
Bing search engine, 573
board design, **568**
CNNs on, 570–572, **571–572**
evaluating, 601–602
guidelines, 577–579
implementation and architecture, 568–569
search acceleration on, 573–574
software, 569
version 1 deployment, 574
version 2 deployment, 575–577, **576–577**
Microsoft's DirectX 8, M-51–52
Microsoft Windows, RAID benchmarks, **D-22**
Microsoft XBox, M-51–52
Migration, 379
Million instructions per second (MIPS)
conditional instructions, H-27
embedded systems, E-15
MIPS16, **K-4**, K-5
MIPS M2000, M-22, **M-22**
MIPS M2000 *vs.* VAX 8700, **M-22**
MIPS64 R6, K-19, **K-19**
MIPS R2000, M-21
MIPS R10000, 423
MIPS R4000 pipeline, C-55–64
floating-point operations, C-60–61, **C-60**
performance, C-61–64
RISC instruction set, C-3–4
Sony PlayStation 2 Emotion Engine, E-17

Millions of floating-point operations per second (MFLOPS)
early performance measures, M-7
parallel processing debates, M-58–59
SIMD computer history, M-56
SIMD supercomputer development, M-46
vector performance measures, G-15–16
Minibatches, DNNs, 556
Minicomputers, 4
MIPS. *See* Million instructions per second (MIPS)
MIPS R3000
integer division/remainder, J-11–12
integer overflow, **J-11**
MIPS R3010
arithmetic functions, J-57–61
chip comparison, **J-58**
chip layout, **J-59–60**
floating-point exceptions, J-35
MIPS R4000, early pipelined CPUs, M-28
Misprediction rate
ARM Cortex-A53, **250**
branch-prediction buffers, **C-25**
profile-based predictor, **C-23**
SPEC89 benchmark, **C-26**
SPEC89 *vs.* predictor size, **187**
static branch prediction, C-22, **C-23**
tagged hybrid *vs.* gshare predictors, **190**
Misses per instruction
advantage, B-6
application/OS statistics, **B-59**
and block size, 397–399, **398**
cache performance, B-5–6
memory hierarchy design, 82
Miss penalty, B-20–21
cache optimization, B-30–36, 105–106, **106**
compiler-controlled prefetching, 111–114
critical word first, 104–105
early restart, 104–105
hardware prefetching, 109–111
memory hierarchy design, 82
multilevel caches reducing, 83
reducing, 95
reduction via multilevel caches, B-30–35
write buffers, 83
Miss rate
AMD Opteron example, B-15
average memory access time, B-29–30, **B-30**
bigger caches reducing, 83
cache optimization
and associativity, B-28–30
and block size, B-26–28, **B-27**
and cache size, **B-24–25**, B-28, **B-33**, **B-37**
and virtually addressed cache size, **B-37**
cache performance, B-16–17
and cache size, **B-24–25**, B-28, **B-33**, **B-37**
compiler-controlled prefetching, 111–114
data, B-16
and data cache size, **402**
early IBM computers, M-11
example calculations, B-6
formula, B-16
global, B-31
hardware prefetching, 109–111
higher associativity reducing, 83
instruction, B-16
larger blocks reducing, 82
L1 caches, **402**
L3 caches, 397–399, **398**
local, B-31
measurement, B-4–5
memory hierarchy, 81
memory stall clock cycles, B-4
reducing, 95
scientific workloads
distributed-memory multiprocessors, **I-28–32**
symmetric shared-memory multiprocessors, I-22, **I-23–25**
total and distribution, **B-25**
unified, B-16
Miss Status Handling Registers (MSHRs), 104
MIT Raw, **F-78**
Mixed cache, B-15
Mixer, radio receiver, **E-23**
M/M/1 model, D-30, D-32, D-57
M/M/2 model, D-57
Modified, shared, invalid (MSI) protocol, 388
Modified state, 383–384, 406
large-scale multiprocessor cache coherence, I-35–36
Modula-3, integer division/remainder, **J-12**
Module availability, 37
Module reliability, 37
MOESI, 388
Moore's Law, 19
interconnection networks, F-74
pitfall, 58
point-to-point links and switches, D-34
RISC history, M-23
semiconductor manufacturing, 4
switch size, F-29
transistors, 5, 540
Motion JPEG encoder, Sanyo VPC-SX500 digital camera, E-19
Motorola 68882, floating-point precisions, J-33
Motorola 68000, memory protection, M-10
Move address, K-55–56
MPEG
multimedia SIMD extensions history, M-50
Sanyo VPC-SX500 digital camera, E-19
Sony PlayStation 2 Emotion Engine, E-17
MPPs. *See* Massively parallel processors (MPPs)
MTTF. *See* Mean time to failure (MTTF)
MTTR. *See* Mean time to repair (MTTR)
Multibanked caches, 99–100
Multichip modules, OCNs, F-3
Multicomputers, 370
cluster history, M-65
definition, M-59
historical background, M-64–65
Multicore processor, 17, 369, 371–372, 382, 408
approaches, 389

architecture, **430**
coherence, 387
Cray X1E, G-24
development, 404
DSM, **373**, **405**, **452**
Intel i7 performance and energy efficiency, 434–437
on multiprogrammed workload, 426–432
OCN, F-101
performance, 426–437, **432**
point-to-point, **446**
scaling, **432**, 442–444
single chip, 382, **391**, 446–451
and SMT, 436–437
Multics protection software, M-9–10
Multicycle operations, RISC pipeline, C-45–49
FP pipeline performance, C-55
hazards and forwarding, C-49–52
maintaining precise exceptions, C-53–55
Multidimensional arrays, 299–301
Multiflow processor, M-31–33
Multigrid methods, ocean application, I-9–10
Multilayer perceptrons (MLPs), 549–550
Multilevel caches
centralized shared-memory architectures, 377
memory hierarchy history, M-12
miss penalty reduction, B-30–35, **B-33**
write process, B-11
Multilevel exclusion, B-35
Multilevel inclusion, B-34, 423
memory hierarchy history, M-12
Multimedia applications, desktop processor support, **E-11**
Multimedia Extensions (MMX), K-31, M-49–50
Multimedia instruction sets, 10
Multimedia SIMD extensions, M-49–50
DSPs, E-11
Multimode fiber, interconnection networks, F-9–10
Multipass array multiplier, **J-51**
Multiple instruction streams, multiple data streams (MIMD), 11, 282, 369–370, 438–439
early computers, M-57
first vector computers, M-49
multimedia SIMD and GPU *vs.*, 347–353
Multiple instruction streams, single data stream (MISD), 11
Multiple-issue processors
advantages, 221–222
challenges, 182, 221–222
characteristics, **219**
dynamically scheduled processor, 222, 224
early development, M-29–32
EPIC approach, 221
instruction-level parallelism, 218–227
microarchitectural techniques case study, 266–273
with speculation, **223**
superscalar, 218, 223
VLIW approach, 218–222, **220**
Multiple lanes technique
vector architecture, 293–294, **294**
vector performance, G-7–9
Multiple-precision addition, J-13
Multiply-accumulate (MAC), 589
DSP, E-5
TI TMS320C55 DSP, E-8
Multiply operations
chip comparison, J-61
floating point
denormals, J-20–21
examples, **J-19**
multiplication, J-17–21
overview, J-17–20
precision, J-21
rounding, **J-18**, J-19
integer arithmetic
array multiplier, **J-50**
Booth recoding, **J-49**
even/odd array, **J-52**
issues, J-11
many adders, J-50–54, **J-50**
multipass array multiplier, **J-51**
n-bit unsigned integers, **J-4**
Radix-2, J-4–7
signed-digit addition table, **J-54**
with single adder, J-47–49, **J-48–49**
Wallace tree, **J-53**
integer shifting over zeros, J-45
Multiprocessor
application, 373–374
architecture issues and approach, 370–373
bus-based coherent, M-59–60
cache coherence, 377–379
cluster history, M-62–65
coining of term, M-59
definition, 369
early computers, M-57
early machines, M-57
embedded systems, E-14–15
execution time, 438
factors, 368
fallacy, 60
Intel, 5
large-scale (*see* Large-scale multiprocessors)
linear speedup, 438–439, **440**
parallel processing challenges, 373–377
parallel processing debates, M-57–59
performance gains, 424–426
processor performance, 371–372
recent advances and developments, M-59–60
shared-memory (*see* Shared-memory multiprocessors (SMPs))
SIMD computers, M-55–57
synchronization and consistency models, M-64–65
Xeon E7 MP scalability, 433–434
Multiprogramming, 369
virtual memory, B-49, 119
workload, 399–404, 426–432
Multistage interconnection networks (MINs)
bidirectional, F-34
crossbar switch calculations, **F-32–33**
vs. direct network costs, F-96
topology, F-32
Multistage switch fabrics, F-31
Multi-Streaming Processor (MSP)
Cray X1, G-21–24, **G-22**
Cray X1E, G-24
first vector computers, M-49
Multithreaded SIMD Processor, 311, 315, **317**

Multithreading, 369
hardware approaches, 243–244
historical background, M-35–36
instruction-level parallelism, 242–247
parallel benchmarks, **247**
performance gains, 424–426
simultaneous (*see* Simultaneous multithreading (SMT))
speedup from, **248**
superscalar processors, 245–247
MVAPICH, F-81, **F-81**
M-way set associative, B-8
MXP processor, components, E-14–15
Myrinet SAN, F-49, M-63, M-74, F-90, **F-90**

N

NAK. *See* Negative acknowledge (NAK)
Name dependences
instruction-level parallelism, 172–173
register renaming, 196
Nameplate power rating, 482
NaN. *See* Not a Number (NaN)
NASA Ames Research Center, M-46, M-65
NAS parallel benchmarks, vector processor history, G-27–28
National Science Foundation, F-102
Natural parallelism embedded systems, E-15
n-bit adder, carry-lookahead, **J-38**
n-bit number representation, J-7–10
n-bit unsigned integers division, **J-4**
N-body algorithms, Barnes application, I-8–9
NBS DYSEAC, M-89
N-cube topology, F-36–38
NEC SX/2, M-48
NEC SX/5, M-48–49
NEC SX/6, M-48–49
NEC SX-8, M-48–49
NEC SX-9, M-49
first vector computers, M-49
NEC VR 4122, embedded benchmarks, E-13
Negative acknowledge (NAK)
cache coherence, I-39
directory controller, I-41
DSM multiprocessor cache coherence, I-37
Negative-first routing, F-48
Nested page tables, 146
Netscape, F-102
Network Appliance (NetApp), D-9, D-41–43
Network-attached storage (NAS), M-88, 478
Network bandwidth, interconnection network, F-18
Network-Based Computer Laboratory (Ohio State), F-80–81
Network buffers, network interfaces, F-8
Network fabric, F-24–25
Network File System (NFS)
benchmarking, D-20, **D-20**
block servers *vs.* filers, D-35
interconnection networks, F-93–94
TCP/IP, F-86
Networking
Google WSCs, 510–511
performance milestones, **22**
Network injection bandwidth
interconnection network, F-19–20
multi-device interconnection networks, F-26
Network interface
fault tolerance, F-67
functions, F-6
message composition/processing, F-6–9
Network interface card (NIC)
functions, F-8
I/O subsystem, F-95
vs. switches, F-90, **F-90**
zero-copy protocols, F-95
Network I/O, 467
Network layer, **F-84**
Network nodes, F-23, F-35, **F-36**
distributed switched networks, F-35
Network of Workstations, M-63, M-74–75
Network on chip (NoC), F-3
Network ports, F-30
Network protocol layer, F-10
Network reception bandwidth, F-19–20
Network reconfiguration, F-70–71
Network technology, 20
personal computers, F-2
Newton's iteration, J-27–30, **J-28**
Nicely, Thomas, J-65
Nodes
communication bandwidth, I-3
direct network topology, **F-37**
distributed switched networks, F-35–40
IBM Blue Gene/L, I-42–44, **I-43**
IBM Blue Gene/L 3D torus network, F-76
network topology performance and costs, **F-40**
points-to analysis, H-9
Nokia cell phone, circuit board, E-24, **E-24**
Nonatomic operations, cache coherence, 386
Nonbinding prefetch, 111
Nonblocking caches
cache bandwidth, 100–104
case study, 148–164
effectiveness, **102**
implementing, 103–104
memory hierarchy history, M-11–12
Nonblocking crossbar, F-33
Nonfaulting prefetches, 111
Nonoptimal replacement algorithm, M-11
Non-overlapped latency, B-20
Nonrecurring engineering (NRE) costs, 542
Nonrestoring division, J-5, **J-6**
Nonuniform cache access (NUCA), 371, 390, 426
Nonuniform memory access (NUMA), 372–373, 391, 426–429
large-scale multiprocessors history, M-61
Non-unit strides, 300
vector processor, G-25
North-last routing, F-48
Not a Number (NaN), J-14–16, J-21, J-34–35
Notifications, interconnection networks, F-10
NOW project, M-74–75
No-write allocate, B-11–12
NSFNET, F-102

NTSC/PAL encoder, Sanyo VPC-SX500 digital camera, E-19
NUMA. *See* Nonuniform memory access (NUMA)
NVIDIA GeForce, M-51–52
NVIDIA K80 GPU die Roofline, **599**
NVIDIA system
  computational structures, 313–320
  GPU computational structures, 313–320
  GPU computing history, M-52–53
  GPU instruction set architecture, 320–323
  GPU memory structures, 326–328, **327**
  graphics pipeline history, M-51–52
  instruction set architecture, 320–323
  NVIDIA P100, **354**
  scalable GPUs, M-51
  Tegra Parker system *vs.* Core i7, 346–353
N-way set associative, B-8
  conflict misses, B-23
  memory hierarchy, 81
  TLB, B-49
NYU Ultracomputer, M-61

O

Occupancy, communication bandwidth, I-3
Ocean application
  characteristics, I-9–10
  distributed-memory multiprocessor, I-30, **I-32**
  example calculations, I-11–12
  miss rates, **I-28**
  symmetric shared-memory multiprocessors, **I-23**
Offline reconstruction, RAID, D-55
Offload engines
  network interfaces, F-8
  TCP/IP reliance, F-100
Offset, B-8–9
  address, B-55–56
  block identification, B-8–9
  cache optimization, B-38
  destination, IA-32 segment, **B-53**
  IA-32 segment, **B-53**
  Opteron data cache, **B-13**, B-14
  page, B-38
  sign-extended, C-5
  virtual memory, B-43–44, B-46, B-55–56
  word, C-28
OLTP. *See* Online transaction processing (OLTP)
Omega, **F-31**, F-32–33
On-chip memory, embedded systems, E-4
On-chip networks (OCNs)
  basic considerations, F-3
  commercial implementations, F-73–74
  commercial interconnection networks, F-63
  cross-company interoperability, F-68
  effective bandwidth, F-19
  interconnection network domain relationship, F-4, **F-5**
  latency *vs.* nodes, **F-28**
  packet latency, **F-13**, F-14–16
  time of flight, F-14
  topology, F-30
  wormhole switching, F-52
One's complement, J-7–8
One-way set associativity, conflict misses, B-23
Online reconstruction, RAID, D-55
Online transaction processing (OLTP), 44, 395–396, **396**, 401
  storage system benchmarks, D-18
Opcode, A-21
OpenGL, M-51–53
Open instruction set, DSAs, 594
Open Systems Interconnect (OSI)
  Ethernet, F-86
  layers, **F-84**
Operand delivery stage, Itanium 2, H-42
Operands
  DSP, E-6
  instruction set architecture, **A-4**, A-13–15
  read, C-68
    dynamic scheduling pipelines, C-66
    ID pipe stage, 194
  TMS320C55 DSP, **E-6**
  type and size, A-13–15
Operating systems (general)
  address translation, B-38
  communication performance, F-8
  disk access scheduling, D-44
  memory protection performance, B-58
  miss statistics, **B-59**
  and page size, B-58
  segmented virtual memory, B-54
  storage systems, D-35–36
  vendor-independent, 2
  workload, 399–404
Operational costs count, 468
Operational expenditures (OPEX), 36, 486–490, **488**
Operation faults, D-11
Operations, 14 *See also specific types of operations*
  atomic, 386
  in instruction set, A-15–16, **A-15**
Operator dependability, disks, D-13–15
Opteron Data Cache, B-11
Optical media, interconnection networks, F-9–10
Oracle database, miss statistics, **B-59**
Ordering, and deadlock, F-47–48
Organizations, 17
  block placement, B-7–8, **B-7**
  buffer, F-59–61
  data dependences, 172
  designing, 17–18
  dynamic random-access memory (DRAM), **86**
  on-chip
    IBM Power8, **428**
    Xeon E7, **428**
  Opteron data cache, B-12–15, **B-13**
  Sony PlayStation Emotion Engine, **E-18**
  SPEC benchmarks, 433–434
Out-of-order completion
  definition, C-53
  dynamic scheduling pipelines, C-66
Out-of-order execution
  and cache miss, B-2
  dynamic scheduling, 193–194
  dynamic scheduling pipelines, C-66
  memory hierarchy, B-2
  microarchitectural techniques case study, 266–273

Out-of-order execution *(Continued)*
miss penalty, B-20–21
Tomasulo's scheme, 208
Out-of-order processors, memory hierarchy history, M-12
Output buffered switch, F-56, **F-58**, F-61, **F-65**
Output dependence, 173
compiler history, M-32
finding, H-7–8
Output gate, 553
Overclocking, 28
Overflow, integer arithmetic, J-8, J-10–11, **J-11**
Overhead
Amdahl's law, F-96
calculating, **F-94**
communication latency, I-4
interconnection networks, **F-27–29**
OCNs *vs.* SANs, **F-28**
processor, **G-4**
software, F-96
sorting case study, D-64–67
time of flight, F-14
Overlapping triplets, integer multiplication, J-49
Oversubscription, 478

P

Packed decimal, A-14
Packets
ATM, **F-79**
bidirectional rings, F-36
centralized switched networks, F-33
discarding, F-69
effective bandwidth *vs.* packet size, **F-19**
format example, **F-7**
InfiniBand, **F-79**
latency issues, F-13, **F-13**
lossless *vs.* lossy networks, F-12
network interfaces, F-8
network routing, F-22
switching, F-51
switch microarchitecture, F-56–59
pipelining, F-64–66
TI TMS320C6x DSP, **E-10**
topology, F-21–22
transport, interconnection networks, F-9–12
Page(s)
coloring, B-38
definition, B-43
vs. segments, **B-43**
Paged segments, B-43–44, **B-43**
Paged virtual memory, B-54–57
Page fault
definition, B-2–3, B-42, B-45
exception stopping/restarting, C-41
Page offset
definition, B-38
virtual memory, B-43–44, B-46
Page size, B-56
operating systems and, B-58
selection, B-46–47
virtual memory, B-46–47
Page table entry (PTE)
definition, B-44, 136
fields in, B-52
Page tables
AMD64 paged virtual memory, B-55
descriptor tables as, B-52
main memory block, B-44–45
nested, 146
protection processes, B-50
segmented virtual memory, B-51–52
shadow, 123
size, B-47
virtual address to physical address, **B-45**
Paging support, 330
Paired single operations, DSP media extensions, E-10–11
Palt, definition, B-2–3
Papadopolous, Greg, M-76
Parallel architectures, classes of, 10–11
Parallelism
challenges, 373–377
classes of, 10–11
computer design principles, 48
dependence analysis, H-8–9
Ethernet, F-74, F-82–83
exploitation statically, H-2
exposing with hardware support, H-23–27
global code scheduling, H-15–23, **H-16**
IA-64 instruction format, H-34–37
ILP (*see* Instruction-level parallelism (ILP))
request-level, 369
software pipelining, H-12–15
superblock scheduling, H-21–23, **H-22**
taking advantage of, 48
thread-level (*see* Thread-level parallelism (TLP))
trace scheduling, H-19–21, **H-20**
Parallel memory systems, highly, 150–153
Parallel processing, 369
Parallel processors
areas of debate, M-57–59
bus-based coherent multiprocessors, M-59–60
cluster history, M-62–65
large-scale multiprocessors history, M-60–62
recent advances and developments, M-59–60
scientific applications, I-33–34
SIMD computers history, M-55–57
synchronization and consistency models, M-64–65
virtual memory history, M-65
Parallel programming
computation communication, I-10–12
with large-scale multiprocessors, I-2
Parallel Thread Execution (PTX), 320–323, **322**
Paravirtualization, 126
PA-RISC, K-3
Parity, dirty bits, D-61–64
PARSEC benchmark
simultaneous multithreading, 246
without SMT, 435–437, **435**, **437**
Partial disk failure, dirty bits, D-61–64
Partial store order (PSO), 420, **421**, 457
Partitioned add operation, DSP media extensions, E-10
Partitioning, 480–482
Pascal GPU architecture, 328–331, **329**
full-chip block diagram, **318**
SIMD Processor, **330**
Pascal programs, integer division/remainder, **J-12**

Pattern, disk array deconstruction, D-51
Payload
messages, F-6
packet format, **F-7**
p-bits, J-21–23, J-25, J-36–37
PC. *See* Program counter (PC)
PCI bus, historical background, M-88
PCI-Express (PCIe), storage area network, F-29
PCI-X, M-88
storage area network, F-29
PCI-X 2.0, F-67
PCMCIA slot, Sony PlayStation 2 Emotion Engine case study, E-15
PC-relative addressing, A-9, K-52
PDP-11, M-10–11, M-19, M-57, M-88
Peak performance
Cray X1E, G-24
DAXPY on VMIPS, G-21
fallacy, 63, **63**
in vector architectures, 355
VMIPS on DAXPY, G-17
Peer-to-peer, wireless networks, E-22
Pegasus, M-17
PennySort competition, D-66
Pentium, K-31–32
Perfect Club benchmarks
vectorization, 303, **303**
vector processor history, G-27–28
Perfect-shuffle exchange, F-32
Performability, RAID reconstruction, D-55–57
Performance. *See also* Cost-performance; Peak performance
benchmarks, 40–45
branch scheme, C-21–22, **C-22**
cache (*see* Cache performance)
common data bus, 207
deep neural networks, 603
dirty bits, D-61–64
disk array deconstruction, D-51–54
disk deconstruction, D-48–50
DSAs, 600–601, **601**
embedded computers, **E-13–14**
Fujitsu SPARC64 X+, 429–431, **432**
IBM Power8, 431–432, **432**
instruction set architecture, 258
Intel Core i7 920, 434–437
Intel Core i7 6700, 138–142, 255–257
Internet Archive Cluster, D-36–41
interprocessor communication, I-3
I/O devices, D-15–23
I/O subsystem design, D-59–61
I/O system design/evaluation, D-36–41
Itanium 2, H-43, **H-43**
large-scale multiprocessors, scientific application
distributed-memory multiprocessors, I-26–32, **I-28–32**
parallel processors, I-33–34
symmetric shared-memory multiprocessor, I-21–26, **I-23–26**
synchronization, I-12–16
latency, 20, **21**
measuring, 39–47
microprocessor, **22**
multicore processor, 426–437, **432**
quantitative measures, M-6–7
reporting, 39–47
sorting case study, D-64–67
summarizing, 39–47
symmetric shared-memory multiprocessor, I-21–26, **I-23–26**
trends, 20
vector processor, G-2–9
chaining, G-11–12, **G-12**
DAXPY on VMIPS, G-19–21
sparse matrices, G-12–14
start-up and multiple lanes, G-7–9
unchaining, **G-12**
VMIPS on Linpack, G-17–19
Permanent failure, commercial interconnection networks, F-70
Permanent faults, D-11, 93
Personal computers, 4, 6
LANs, F-4
networks, F-2
PCIe, F-67
Personal mobile device (PMD), A-2, 7–8
image-processing unit for, 542
memory hierarchy in, **79**
PetaBox GB2000, Internet Archive Cluster, D-37
Phase-change memory (PCM)
memory hierarchy design, 93
Xpoint memory chips, 93
Phase-ordering problem, A-26
Physical address
AMD Opteron data cache, B-12–13
AMD64 paged virtual memory, B-55
safe calls, B-54
translation, B-36–40
virtual address, **B-45**
virtual memory, B-42, B-51
Physical cache, B-36–37
Physical channels, F-47–48
Physical layer, **F-84**
Physical memory
centralized shared-memory multiprocessor, **372**
directory-based cache coherence, 380
memory hierarchy, B-40–42
virtual machine, 123
Physical register
instruction, **237**
register renaming, 235
SIMD instructions, 320
uses of, 234–235
Physical transfer units (phits), F-64
Physical volumes, D-35
PicoJoules, **541**
PID. *See* Process-identifier tag (PID)
Pin-out bandwidth, topology, F-39
Pipelined circuit switching, F-51
Pipelined CPUs, early versions, M-27–28
Pipeline delays, C-44
Pipeline interlock, C-17
Pipeline latches, C-30–31
Pipeline organization, data dependences, 172
Pipeline registers
data hazards stall minimization, C-14
definition, C-30–32
pipelining performance, C-8–10
Pipeline scheduling
instruction-level parallelism, 177–182
microarchitectural techniques case study, 266–273

Pipeline stall cycles
ARM Cortex-A53, 250–251
branch scheme performance, C-21
Pipelining
branch cost reduction, C-22
branch hazards, C-18–22
branch issues, C-35–37
branch penalty reduction, C-19–20
branch-prediction buffers, C-23–25, **C-24–26**
branch scheme performance, C-21–22, **C-22**
cache access, 99–100
classic stages for RISC processor, C-6–8, **C-7**
compiler scheduling, M-32
computer design principles, 48
concept, C-2–3
data hazards, C-12–17
definition, C-11
instruction set complications, C-45
pipelined execution of instructions, C-13
stall minimization by forwarding, C-14–15, **C-15–16**
stall requirements, C-16–17
types, C-12
definition, C-2
detection of hazard, **C-34**
example, **C-7**
exception
arithmetic-logical units, C-5
categories, **C-40**
floating-point, C-41–42
precise, C-41–44
RISC V, C-42–43, **C-42**
stopping/restarting, C-41–42
types and requirements, C-38–41, **C-40**
unexpected sequences, C-70
floating-point addition speedup, J-25
graphics pipeline history, M-51–52
hazards, C-10–25
instruction set complications, C-43–45
interconnection networks, F-12
MIPS R4000, C-55–64
performance issues, C-8–10
performance with stalls, C-11–12
predicted-not-taken scheme, C-19–20, **C-19**
RISC V, C-30–33
classic pipeline stages, C-6–8
control, C-33–35
exception, C-42–43, **C-42**
FP pipeline, C-45–55, **C-57**
instruction set, C-3–4, C-65
instruction set complications, C-43–45
integer pipeline to handle multicycle operations, C-45–55
multicycle FP operations, C-45–55
pipeline control, C-33–35
simple implementation, C-4–6, **C-6**, C-26–29, **C-30**
simple implementation, C-26–37
speedup from, C-11–12
static branch prediction, C-22, **C-23**
switch microarchitecture, F-64–66
Pipe segment, C-3
Pipe stage
definition, C-3
exception stopping/restarting, C-41
performance issues, C-8–10
program counter, **C-31–32**
register additions, **C-31**
RISC processor, C-8
Pixel Visual Core
architecture philosophy, 583–584
evaluating, 601–602
example, 588
floor plan, **592**
Halo, 584–585
implementation, 590–591
instruction set architecture, 587–588
line buffers in, 590
processing element, 588–589
processor, 585–587
programmer view of, **589**
software, 582
two-dimensional array, **586**
two-dimensional line buffers, 589–590
PLA, early computer arithmetic, J-65
Points-to analysis, H-9
Point-to-point links
bus replacement, **D-34**
Ethernet, F-30
storage systems, D-34
switched-media networks, F-24–25
Poison bits, compiler-based speculation, H-28, H-30–31
Poisson distribution
basic equation, D-28
random variables, D-26–34
Poisson, Siméon, D-28
Polycyclic scheduling, M-32
Portable computers, interconnection networks, F-89
Portable mobile devices (PMDs), 580–581
Port number, network interfaces, F-7–8
Position independence, A-17
Power, 442–443
first-level caches, 95–98
within microprocessor, 25–28
systems perspective, 23–24
Power 3, K-25
additional instructions, **K-24**
branch registers, K-23–25
Power consumption. *See also* Energy efficiency
case study, 69–71
embedded benchmarks, E-13
interconnection networks, F-61, F-89
simultaneous multithreading, 246
speculation, 238–239
TI TMS320C55 DSP, E-8
Power gating, 28
PowerPC, K-6, K-11, K-25
cluster history, M-64
conditional instructions, H-27
IBM Blue Gene/L, I-41–42
RISC history, M-21
PowerPC AltiVec, multimedia support, **E-11**
Power-performance
Dell PowerEdge servers, 55–58, **56**
of servers, **57**
Power utilization effectiveness (PUE)
pitfall, 515
WSCs, 483, **484–485**
Precise exceptions
definition, C-41–44
maintaining, C-53–55

Precisions, floating-point arithmetic, J-33–34
Predicated instructions
example calculations, H-25
exposing parallelism, H-23–27
IA-64, H-38–40
Predicate registers, 296–298
IA-64, H-34
Predication, TI TMS320C6x DSP, E-10
Predicted-not-taken scheme, C-19–20, **C-19**
Prediction. *See also* Misprediction rate
branch
accuracy, **C-25–26**
cost reduction, C-22
dynamic, C-23–25
instruction-level parallelism, 182–191
static, C-22, **C-23**
branch-prediction buffers, C-23–25, **C-24–26**
return address, 232–234
size, **187**
2-bit scheme, **C-24**, 182, 184, **185**
Prediction by Partial Matching (PPM), 188
Prefetching, 376
instruction, 234
Itanium 2, H-42
memory hierarchy design, 138
software and hardware, 148–164
Prefix, Intel 80x86 integer operations, K-35
Presentation layer, **F-84**
Present bit, B-52
Price, cost *vs.*, 35
Price-performance, 8
Primitives, synchronization, M-64–65
Principle of locality
coining of term, M-11
computer design principles, 48–49
definition, B-2
scientific workloads on symmetric shared-memory multiprocessors, I-25–26
Private data, 377
Private memory, NVIDIA GPU memory structures, 326
Procedure calls
IA-64 register model, H-33–34
VAX, K-57
Procedure invocation options, A-19–20
Process-complexity factor, 34
Process concept, B-49, 119
Process-identifier tag (PID), B-37–38, **B-37**
Processing element (PE) array, 570, **572**, 580, 584–585, 587, 592
Processor consistency, 420
Processor cycle
cache performance, B-3
definition, C-3
Processor-dependent optimizations, A-26
Processor-intensive benchmarks, 41
Processor performance
average memory access time and, B-17–20
equation, 52–55
multiprocessors, 371–372
Process switch
definition, **B-37**, B-49
virtual memory, B-49, 119
Producer-server model, D-15–16, **D-16**
Profile-based predictor, misprediction rate, **C-23**
Program counter (PC), A-17
branch hazards, C-18
branch-target buffers, 228–229, **229**
dynamic branch prediction, C-23–24
exception stopping/restarting, C-41–42
pipeline branch issues, C-35–36
pipe stage, **C-31–32**
precise exceptions, C-54
RISC V instruction set, C-4
simple RISC implementation, C-27–29
tagged hybrid predictors, 188–189
Programmer's view, memory consistency, 418–419
Programming models, warehouse-scale computers, 471–476
Program order
cache coherence, 378–379
control dependence, 174–175
data hazards, 173–174
definition, 173
Protection schemes
cross-cutting issues, 126–127
development, M-9–12
network interfaces, F-8
Pentium *vs.* Opteron, B-57
processes, B-50
safe calls, B-54
segmented virtual memory, B-51–54
virtual memory, B-41
Protocol deadlock, routing, F-45–46
Protocol stack, F-86–87
PSO. *See* Partial store order (PSO)
PTE. *See* Page table entry (PTE)
Pulse amplitude modulation (PAM-4), 59

Q

QCDOD, M-64
QPI. *See* QuickPath Interconnect (QPI)
QsNetII, F-67, **F-80**
Quadrics SAN, **F-80**
Quality of service (QoS)
dependability benchmarks, D-21
WAN, F-102–103
Quantitative performance measures, development, M-6–7
Quantization, DNNs, 556
Queue
definition, D-25
discipline, D-26
Intel Core i7, **256**
waiting time calculations, D-26
Queuing locks, large-scale multiprocessor synchronization, I-18–21
Queuing theory, D-23–34
QuickPath Interconnect (QPI), 426–429

R

Race-to-halt, 28
Radio frequency amplifier, radio receiver, **E-23**
Radio receiver, components, **E-23**
Radio waves, wireless networks, E-21
Radix-8 multiplication, J-49
Radix-2 multiplication/division, J-4–7, **J-4**, **J-55**
Radix-4 multiplication/division, J-49, **J-49**, J-56, **J-56–58**, J-61

RAID. *See* Redundant array of inexpensive disks (RAID)
Random access memory (RAM), F-56
Random Access Method of Accounting Control (RAMAC), M-85, M-88–89
Random replacement, B-9–10, **B-10**
Random variables, distribution, D-26–34
Ranking, 573
Ransomware, 491
Ray casting (RC), 350
Read after write (RAW), C-12–14
  check for, C-52
  first vector computers, M-47–48
  instruction set complications, C-44
  program order, 173
  RISC pipeline control, C-34
  stalls, C-49, **C-50**, C-51
  TI TMS320C55 DSP, E-8
  Tomasulo's algorithm, 195, 217
Read miss, 382–383, 410
  cache coherence, **384–385**, 386, 388
  directory-based cache coherence protocol, 410–411
  memory stall clock cycles, B-4
  miss penalty reduction, B-35–36
  Opteron data cache, B-14
Read operand, C-68
  dynamic scheduling pipelines, C-66
  ID pipe stage, 194
Real addressing mode, K-31
Real memory, 123, 126
Real mode, **K-36**
Real numbers, K-39
Real-time constraints, E-2
Real-time performance, 7–8
  requirement, definition, E-3–4
Real-time processing, embedded systems, E-3–4
Rearrangeably nonblocking, F-33
Receiving overhead, F-28, **F-28**, F-41, F-67
Receiving overhead, communication latency, I-3
Reconfiguration deadlock, F-45–46
Recovery time, vector processor, G-8
Rectified linear unit (ReLU), 546
Recurrences
  basic approach, H-11
  loop-carried dependences, H-5
Recurrent neural network (RNN), 553–555
Red-black Gauss-Seidel, I-9–10
Reduced Instruction Set Computer (RISC), 413–414, 423
  cache performance, B-6
  compiler history, M-33
  correlating predictors, 183
  development, 2
  early pipelined CPUs, M-28
  FENCE in, 420–422
  historical background, M-20–23, **M-22**
  multimedia SIMD extensions history, M-50
  pipeline scheduling and loop unrolling, 177–178
  reduced code size in, A-23–24
  RISC-I, M-20–21
  RISC-II, M-20–21
  Sanyo VPC-SX500 digital camera, E-19
  vector processor history, G-26
Reduced Instruction Set Computer (RISC) architectures, A-33–42
  addressing modes and instruction formats, K-6–9
  ARM architecture, K-22
  16-bit instructions, K-3, **K-4**, K-5, **K-10**
  compare and conditional branch, K-11–16
  conditional branches, **K-17**
  control instructions, **K-18**
  data transfer instructions, **K-18**
  digital signal-processing extensions, K-28
  extensions beyond RV64G, K-18–19
  for lower-end applications, K-3
  MIPS64 R6, K-19
  multimedia and graphics operations, K-25–27
  Power3, K-23–25
  RISC-V integer ISA, **K-13**
  RV64GC core 16-bit instructions, K-16–17
  RV64G core instructions, K-11
  RV64G instruction, K-5, **K-10**, **K-12**
  SIMD extensions, K-25–27
  SPARC v.9, K-20–22
  survey of, K-3–29
Reduced Instruction Set Computer (RISC) V, 12–17, **13**, 413–414, 423
  addressing modes, A-36
  control flow instructions, A-39–40
  data types for, A-35–36
  dies, **33**
  FENCE in, 420–422
  floating point instructions for, **16**
  floating-point operations, A-40–41
  instruction format, A-36–37
  instruction set architecture formats, **16**
  instruction set organization, A-34
  load and store instructions, **A-38**
  operations, A-37–39
  pipelining, C-30–33
    classic pipeline stages, C-6–8
    control, C-33–35
    exception, C-42–43, **C-42**
    FP pipeline, C-45–55, **C-57**
    instruction set, C-3–4, C-65
    instruction set complications, C-43–45
    integer pipeline to handle multicycle operations, C-45–55
    multicycle FP operations, C-45–55
    pipeline control, C-33–35
    simple implementation, C-4–6, **C-6**, C-26–29, **C-30**
  registers for, A-34–35
  scalar architecture, RV64V, **284**
  SIMD Processor, 306–307
  SPECint2006 programs, **A-42**
  subset of instructions in, **15**
  Tomasulo's algorithm
    floating-point operation, **198**
    instruction set, 195
Reductions, 344–345
Redundancy
  chip fabrication cost case study, 67–68

computer system power consumption case study, 69–71
index checks, B-9
simple RISC implementation, C-29
Redundant array of inexpensive disks (RAID)
dependability benchmarks, D-21–23
disk array deconstruction case study, D-51–54
disk deconstruction case study, D-48–50
hardware dependability, D-15
historical background, M-86–88
I/O subsystem design, D-59–61
logical units, D-35
NetApp FAS6000 filer, D-41–43
overview, D-6–8
performance prediction, D-57–59
RAID 0, D-6
RAID 1, D-6, M-87
RAID 2, D-6, M-87
RAID 3, D-6, M-87
RAID 4, D-7, M-87
RAID 5, D-8, M-87
RAID 6, D-8–10
RAID 10, D-8
row-diagonal parity, D-9–10, **D-9**, D-41–42
Redundant multiplication, integers, J-47
Reference bit, B-45, B-52
Regional explicit congestion notification (RECN), F-70
Register(s)
DSP examples, **E-6**
IA-64, H-33–34
instructions and hazards, **C-13**
network interface functions, F-7
pipe stage, **C-31**
tag, **202**
Register addressing, K-52
Register allocation, A-26–27
Register deferred addressing, K-52–53
Register fetch (RF)
cycle, C-5
MIPS R4000 pipeline, C-56
simple RISC implementation, C-27
Register file, C-27, 200–201
data hazards, C-16–17
floating-point operations, **C-50**
OCNs, F-3
precise exceptions, C-54
RISC instruction set, C-5, C-7–8
simple RISC implementation, C-29
Register indirect addressing mode, K-34
Register management software-pipelined loops, H-14
Register-memory
architecture, A-3
ISAs, 12
Register prefetch, 111
Register pressure, 182
Register renaming
antidependence, 196
deallocating registers, 235
definition, 173, 195–196
expected output, **269**
initial state table, **270**
microarchitectural techniques case study, 266–273
name dependences, 196
vs. reorder buffers, 234–236
reservation stations, 196–197, 199–200
sample code, **269–270**
Register stack engine, IA-64, H-34
Register Transfer Level (RTL) code, 569
Regularity, bidirectional MINs, F-33–34
Reinforcement learning (RL), 549
Relaxed consistency models, 419–422, **421**
Release consistency (RC), 420–422, **421**, 457
Reliability
commercial interconnection networks, **F-37**
I/O subsystem design, D-59–61
storage systems, D-44
Relocation, virtual memory, B-41–42
Remainder, floating point, J-31–32
Remington-Rand, M-5
Remote direct memory access (RDMA), F-80
Remote node, 406–407
Renaming map, 235
Reorder buffer (ROB)
compiler-based speculation, H-31–32
hardware-based speculation, 209–212, 214–215
issue with, 236
register renaming *vs.*, 234–236
Replication
definition, 377, 379
virtual memory, B-49
Reply, messages, F-6
Reproducibility, 45
Request
messages, F-6
switch microarchitecture, F-58–59
Requested protection level, B-54
Request-level parallelism (RLP)
definition, 5, 10–11, 369
WSCs, 467
Request phase, F-49–50
Request-reply deadlock, F-45–46
Reservation stations
common data bus, **202**
fields, 199–200
register renaming, 196–197, 199–200
Reserved register, 414
Resource sparing, F-70–71
Response time. *See also* Latency
definition, 20, 39
DNN applications, 596–600
I/O benchmarks, **D-18**
producer-server model, **D-16**
vs. throughput, D-16–18, **D-17**
Restartable pipeline
definition, C-40–41
exception, C-41–42
Restorations, dependability, 37
Restoring division, J-5, **J-6**
Resume event, exception, C-40
Return address, predictors, 232–234
Returns, cache coherence, 378–379
Reverse path, cell phones, E-24
RF. *See* Register fetch (RF)
Rings, F-43
protection processes, B-50
Ripple-carry adder, J-2–3, **J-3**
carry-lookahead adder with, **J-42**
chip comparison, J-61
Ripply-carry addition, J-2–3

RISC. *See* Reduced Instruction Set Computer (RISC)
ROB. *See* Reorder buffer (ROB)
Role code, 569, **570**
Roofline model, **349**
  CPUs *vs.* GPUs, **355**
  DNN applications, 596–600, **597**
Round digit, J-18
Rounding modes, J-14, J-17–20, **J-18**, **J-20**
  FP precisions, J-34
  fused multiply-add, J-33
Round-robin (RR), F-49
Routers, **F-64**, F-83
Routing algorithm, F-21–22, F-45–49
Row access strobe (RAS), memory hierarchy design, 85–86
Row-diagonal parity, D-9–10, **D-9**, D-41–42
Row major order, blocking, 107
RV64c, 16-bit instruction formats, **K-7**
RV32E, **K-4**
RV64GC, K-3
  ALU instructions in, **K-17**
  16-bit instructions, **K-10**
  core 16-bit instructions, K-16–17
  register encodings, **K-7**
RV64G core instructions, K-11
RV64G, extensions beyond, K-18–19
RV64G instruction, **K-12**
RV64V extension
  data sizes, **287**
  vector architecture, 283–287, **284**
  vector instructions, **286**
RV64V instruction set, 293

S

Sanyo digital cameras, SOC, **E-20**
Sanyo VPC-SX500 digital camera, embedded system case study, E-19
SASI, M-88
SATA disks. *See* Serial Advanced Technology Attachment (SATA) disks
Saturating arithmetic, DSP media extensions, E-11
Scalability
  computer design principles, 48
  server systems, 9
Scalable GPUs, M-51
Scalar lane (SCL), 586–588
Scalar processors, 310, 326, **332**, 334. *See also* Superscalar processors
  early pipelined CPUs, M-28
  vs. vector, G-19
Scalar registers
  Cray X1, G-21–22
  set of, 285
Scaled speedup. *See* Weak scaling
Scaling
  CMOS, 442–443
  computation-to-communication ratios, **I-11**
  instruction-level parallelism, 442
  multicore processor, **432**, 442–444
  scientific applications on parallel processing, I-34
  SPECintRate benchmarks, 429–431, **431**
  strong, 439
  weak, 439
Scan Line Interleave (SLI), M-51
Scatter store, 301–302
Schorr, Herb, M-29–30
Scientific applications
  Barnes, I-8–9
  characteristics, I-6–12
  cluster history, M-62–63
  distributed-memory multiprocessors, I-26–32, **I-28–32**
  FFT kernel, I-7
  LU kernel, I-8
  ocean, I-9–10
  parallel processors, I-33–34
  parallel program computation/communication, I-10–12, **I-11**
  parallel programming, I-2
  symmetric shared-memory multiprocessor, I-21–26, **I-23–26**
Scoreboarding
  definition, 194–195
  dynamic scheduling with, C-66–70, **C-68**
SCSI. *See* Small Computer System Interface (SCSI)
SDRWAVE, J-62
Second-level caches
  cache optimization, B-30–35, **B-34**
  execution time, B-32, **B-34**
  interconnection network, F-74
  Itanium 2, **H-41**
  memory hierarchy, B-48–49, **B-48**
  miss rate calculations, B-30–35, **B-34**
Secure Virtual Machine (SVM), 146
Seek distance, D-46, **D-46–47**
Seek time, storage disks, D-45–46, **D-46**
Segment descriptor, B-52, **B-53**
Segmented virtual memory
  bounds checking, B-52
  Intel Pentium processors, B-51–54
  memory mapping, B-52
  safe calls, B-54
  sharing and protection, B-52–53
Segments
  definition, B-43
  pages *vs.*, **B-43**
Self-correction, Newton's algorithm, J-28–30
Self-draining pipelines, M-30
Self-routing, MINs, F-48–49
Semiconductor
  DRAM, 19
  flash, 19
  ITRS, 58–59, **59**
  manufacturing, 4
Sending overhead
  OCNs *vs.* SANs, **F-27–29**
  time of flight, F-14
Sending overhead, communication latency, I-3
Sense-reversing barrier
  code example, I-15, **I-21**
  large-scale multiprocessor, synchronization, I-14–16
Sequency number, packet header, F-8
Sequential consistency (SC), 417, **421**, 457
  implementation, 418, 423
  programmer's view, 418–419
Sequential interleaving, 100
Sequent Symmetry, M-59–60
Serial Advanced Technology Attachment (SATA) disks
  NetApp FAS6000 filer, D-42
  power consumption, D-5

RAID 6, D-8–9
vs. SAS drives, D-5, **D-5**
Serial Attach SCSI (SAS) drive
historical background, M-88
power consumption, D-5
vs. SATA drives, **D-5**
Serialization
barrier synchronization, I-16
cache coherence, 378–381
definition, 380–382, 413
DSM multiprocessor cache coherence, I-37
Serpentine recording, M-85
Serve-longest-queue (SLQ) scheme arbitration, F-49
Server(s), A-2, 8–9.
See also Warehouse-scale computer (WSC)
benchmarks, 43–45
CPU utilization, **475**
definition, D-25
Google WSCs, 512–513, **513**
single-server model, **D-25**
system characteristics, **E-4**
Server computer, RISC architectures survey for, K-3–29
Serverless Computing, 496
ServerNet interconnection network, F-70–71
Server utilization, D-25, D-28–29
Service accomplishment, 36
Service interruption, 36
Service level agreements (SLAs), 36–37
Service level objectives (SLOs), 36, 485–486
Session layer, **F-84**
Set associativity, 81
AMD Opteron data cache, B-13
cache block, B-8, **B-8**
cache misses, **B-10**
Set, definition, B-8
Settle time, D-46
SFS benchmark, NFS, D-20–21
Shadow page table, 123
Sharding, 480–482
Shared data, 377
Shared-media networks, F-23–25
Shared memory, 373, 379, 406
address space, 373
distributed (*see* Distributed shared memory (DSM))
Shared-memory communication, large-scale multiprocessors, I-4–5
Shared-memory multiprocessors (SMPs), 371, 373
access time, 371
definition, M-64
history background, M-61
snooping coherence protocols, 380
Shared state
cache block, 386
definition, 383–384
Sharing addition, segmented virtual memory, B-52–53
Shear algorithms, disk array, D-51–54
Sheet Generator (SHG), 585
Shell code, 569, **570**
Shifting over zeros, integer multiplication/division, J-45–47
SiFive, **33**
Signals, definition, E-2
Signal-to-noise ratio (SNR), wireless networks, E-21
Signed-digit representation
example, **J-54**
integer multiplication, J-53
Signed number arithmetic, J-7–10
Sign-extended offset, RISC, C-5
Significand, J-15
Sign magnitude, J-7–8
Silicon Graphics Altix, M-64
Silicon Graphics Challenge, M-60
Silicon Graphics 4D/240, M-59–60
Silicon Graphics Origin, M-62, M-64
Silicon Graphics systems (SGI), vector processor history, G-27
Simultaneous multithreading (SMT), 424–426, **425**, 435
definition, 244
historical background, M-35–36
implementations, 245
Java and PARSEC benchmark without, 435–437, **435**, **437**
multicore processor and, 436–437
superscalar processors, 245–247
Single chip multicore processor, 382, **391**, 446–451
Single-event upsets (SEUs), 569
Single-extended precision floating-point arithmetic, J-33–34
Single instruction multiple data (SIMD), 11, 170, 282
historical overview, M-55–57
instruction
DSP media extensions, E-10
IBM Blue Gene/L, I-42
Sony PlayStation 2, **E-16**
Intel Core i7 920 multicore computer, **309**
loop-level parallelism, 170
multimedia extensions
256-bit-wide operations, **304**
data-level parallelism, 304–310
GPU and MIMD *vs.*, 347–353
vs. GPUs, 335, **335**
Intel Core i7 920 multicore computer, **309**
NEC SX-9 vector processor, **309**
programming, 307
RISC-V, 306–307
roofline visual performance model, 307–310
NEC SX-9 vector processor, **309**
processors
multithreaded, 311, 315, **317**
Pascal GPU architecture, **330**
RISC V, 306–307
supercomputer development, M-45–46
system area network history, F-104
thread instructions, 315–317, **316**, **319**
thread schedule, 315–317
TI 320C6x DSP, E-9
Single instruction, multiple thread (SIMT), 311
Single instruction stream, single data stream (SISD), 11, M-56
SIMD, M-46
Single-precision floating point
arithmetic, J-33–34
representation, J-15
Single-precision floating-point arithmetic, 329
Single-Streaming Processor (SSP)
Cray X1, G-21–24
Cray X1E, G-24
Skippy algorithm, D-49, **D-50**

Small Computer System Interface (SCSI)
Berkeley's Tertiary Disk project, D-4
dependability benchmarks, D-21
disk storage, D-4
historical background, M-88
I/O subsystem design, D-59–61
RAID reconstruction, D-56
storage area network history, F-106–107
Small form factor (SFF) disk, M-86
Smalltalk, K-21–22
Smart interface cards, *vs.* smart switches, F-90
Smart switches, *vs.* smart interface cards, F-90
SMPs. *See* Shared-memory multiprocessors (SMPs)
SMT. *See* Simultaneous multithreading (SMT)
Snooping bandwidth, 389–390
Snooping cache coherence, 380, **381**
example protocol, 383–387, **384**
implementation, 392–393
invalidate protocol, **381**
large-scale multiprocessors, I-34–35, M-61
latencies, 447, **448**
limitations, 389–392
maintenance, 380–381
sample types, **M-60**
SoC. *See* System-on-chip (SoC)
Soft cores, 130
Soft errors, 93
Soft real-time, 7–8
definition, E-3–4
Software as a service (SaaS)
growth of, 9
WSCs, 467
Software guard extensions (SGX), 125
Software pipelining
example calculations, H-13–14
loops, execution pattern, **H-15**
technique, H-12–15, **H-13**
Software prefetching, 148–164
Software speculation
definition, 176
hardware-based *vs.*, 240–241
Software technology
large-scale multiprocessor, I-6
synchronization, I-17–18
network interfaces, F-7–8
Solaris, RAID benchmarks, D-21, **D-22**, D-23
Sonic Smart Interconnect, OCNs, F-3
Sony PlayStation 2
block diagram, **E-16**
embedded multiprocessors, E-14–15
Emotion Engine case study, E-15–18
Emotion Engine organization, **E-18**
Sort procedure, VAX
code example, K-62–64
full procedure, K-65
register allocation, K-62
register preservation, K-64–65
Source routing, F-49
SPARC "annulling" branch, K-18–19
SPARCLE processor, M-35–36
SPARC v.9
additional instructions, **K-22**
fast traps, K-20–21
integer arithmetic, **K-21**
LISP, K-21–22
misaligned trap, **K-21**
register windows, K-20
Smalltalk, K-21–22
SPARC VIS, K-25–26, **K-27**
SPARC64 X+, 389, 426, 429
feature, **427**
performance, 429–431, **432**
Sparse matrices, vector architecture, G-12–14, 301–302
Spatial locality, B-26
coining of term, M-11
computer design principles, 49
definition, B-2
SPEC benchmark
active benchmarks, **44**
correlating predictors, 182
desktop performance, 41–43, **42**
early performance measures, M-7
organization, 433–434
server performance, 43–45
static branch prediction, C-22, **C-23**
storage systems, D-20–21
vector processor history, G-27–28
SPEC89 benchmark, 41
branch-prediction buffer, C-24–25, **C-25**
misprediction rate, **187**
mispredictions rate, **C-26**
tournament predictors, 187
SPEC92 benchmarks
CPI, **C-64**
stalls, C-61–62
SPEC95 benchmarks
procedure returns, 232
return address buffer, 232, **233**
SPEC2000 benchmarks
compulsory miss rate, B-23
perl benchmark, **144**
speculation, 238–239
SPEC2006Cint execution times, **47**
SPECCPU2006 benchmark
Intel Core i7 920/6700, **192**
nonblocking caches, 101–102
virtual machine, 121
SPEC CPU95 benchmark, return address buffer, 232, **233**
SPECCPUint2006 benchmark, clock cycles per instruction, 256, **257**
SPECfp benchmark
Intel Core i7, 253
interconnection network, F-91–92
Itanium 2, **H-43**
stalls, C-55, **C-56–57**
SPECfpRate benchmark
cost-performance, 440, **441**
speedup, 440, **440**
SPEChpc96 benchmark, G-27–28
Special instruction caches, 128
Special-purpose machines
historical background, M-4–5
SIMD computer history, M-56–57
Special-purpose register computer, A-3
Special values, floating point, J-14–15, **J-16**
SPECInt2006 benchmark
ARM Cortex-A53, 132, **132–133**, **250**
L1 data cache miss rate, **139**
SPECINT92 benchmark, nonblocking caches, 101–102
SPECINT benchmarks
interconnection network, F-91–92
Itanium 2, **H-43**
SPECint95 benchmarks, F-92
SPECintRate benchmarks
cost-performance, 440, **441**

performance scaling, 429–431, **431**
speedup, 440, **440**
SPEC Mail benchmark, D-20–21
SPEC-optimized processors, *vs.* density-optimized, F-89
SPECpower benchmark, WSCs, 475–476
SPECRate benchmark, 439
for memory-intensive benchmarks, 116, **116**
server performance, 43
SPECRatios, 46–47
SPEC SFS benchmarks, D-20
Speculation
address aliasing prediction, 239–240
advanced techniques, 228–240
advantages, 237–238
challenge of issues per clock, 236–237
concept origins, M-31
control dependence, 175–176
cross-cutting issues, 127–128
disadvantages, 238
and energy efficiency, 238–239
exception handling, 199
execution, 241
hardware-based, 208–217
data flow execution, 209
definition, 208
disadvantage, 241
instruction execution step, 211–212
key ideas, 208
reorder buffer, 209–212, 214–215
vs. software speculation, 240–241
write result, 217
IA-64, H-38–40
ILP studies, M-33–34
memory reference, hardware support, H-32
microarchitectural techniques case study, 266–273
multiple branches, 238
register renaming *vs.* ROB, 234–236
software, 176, 240–241
SPEC Web benchmarks, D-20–21
Speedup
Amdahl's Law, 374–375
computer design principles, 49–52
floating-point addition, J-25–26
integer addition
carry-lookahead, J-37–41
carry-lookahead circuit, **J-38**
carry-lookahead tree, **J-40**
carry-lookahead tree adder, **J-41**
carry-select adder, J-43–44, **J-43–44**
carry-skip adder, J-41–43, **J-42**
overview, J-37
integer division
radix-2 division, **J-55**
radix-4 division, **J-56**
radix-4 SRT division, **J-57**
with single adder, J-54–57
SRT division, J-45–47, **J-46**, J-55–57
integer multiplication
array multiplier, **J-50**
Booth recoding, **J-49**
even/odd array, **J-52**
many adders, J-50–54, **J-50**
multipass array multiplier, **J-51**
signed-digit addition table, **J-54**
with single adder, J-47–49, **J-48–49**
Wallace tree, **J-53**
integer multiplication/division, shifting over zeros, J-45
integer SRT division, J-45–47, **J-46**
linear, 438–439, **440**
from multithreading, **248**
pipeline with stalls, C-11–12
SPECfpRate benchmarks, 440, **440**
SPECintRate benchmarks, 440, **440**
switch buffer organizations, F-59–60
TPC-C benchmarks, 440, **440**
Sperry-Rand, M-4–5
Spin locks, 414–416
large-scale multiprocessor synchronization
barrier synchronization, I-16
exponential back-off, **I-17**
SPRAM, Sony PlayStation 2 Emotion Engine organization, **E-18**
Sprowl, Bob, F-103
Squared coefficient of variance, D-27
SRAM. *See* Static random-access memory (SRAM)
SRT division
chip comparison, J-61
complications, J-45–47
early computer arithmetic, J-65
example, **J-46**
historical background, J-63
integers, with adder, J-55–57
radix-4, J-56–57, **J-57**
Stack, A-3, A-28
architecture, historical background, M-17–18
Stacked DRAM, 91
Stack frame, K-57
Stack pointer, K-57
Stale copy, cache coherency, 128
Stall
control dependences, 176
cycles
average memory access time, B-18
branch scheme performance, C-21
definition, B-3–4, B-6, **B-22**
miss rate calculation, B-6
out-of-order execution, B-20
data hazards minimization, **C-13**, C-14–15
data hazards requiring, C-16–17
longer latency pipelines, C-49, **C-50**
pipelining performance with, C-11–12
RAW, C-49, **C-50**, C-51
SPEC92 benchmarks, C-61–62
SPECfp benchmarks, C-55, **C-56–57**
Standardization, commercial interconnection networks, F-67–68
Standard Performance Evaluation Corporation (SPEC), 41
Start-up time
DAXPY on VMIPS, G-20–21
definition, 292
page size selection, B-47
vector architectures, G-4, **G-4**, **G-8**
vector convoys, **G-4**

Start-up time *(Continued)*
vector performance, G-2–4, G-16
vector processor, G-7–9, G-25
VMIPS, **G-5**
Statically based exploitation, ILP, H-2
Static power, 80
Static random-access memory (SRAM)
arithmetic operations and energy cost, **29**
memory hierarchy design, 85
price pressures, 34
vector memory systems, G-9–10
vector processor, G-9–10, G-25
Static scheduling
definition, C-65
instruction-level parallelism, 218–222
unoptimized code, C-70
Stencil computation, 550
Sticky bit, J-18
Stochastic gradient descent, 548
Storage area networks, F-77–81, F-106–108
dependability benchmarks, D-21–23
Storage systems
Amazon, Dynamo key-value, 485–486
asynchronous I/O and operating systems, D-35–36
Berkeley's Tertiary Disk project, D-12–13
block servers *vs.* filers, D-34–35
bus replacement, D-34
component failure, D-43
computer system availability, D-43
dependability benchmarks, D-21–23
dirty bits, D-61–64
disk array deconstruction, D-51–54
disk arrays, D-6–10
disk deconstruction, D-48–50
disk power, D-5
disk seeks, D-45
disk storage, D-2–10
file system benchmarking, D-20–21
Internet Archive Cluster (*see* Internet Archive Cluster)
I/O performance, D-15–23
I/O system design/evaluation, D-59–61
mail server benchmarking, D-20–21
NetApp FAS6000 filer, D-41–43
operator dependability, D-13–15
OS-scheduled disk access, D-44
point-to-point links, D-34
queuing theory, D-23–34
RAID performance prediction, D-57–59
RAID reconstruction case study, D-55–57
real faults and failures, D-10–15
reliability, D-15–23
response time restrictions for benchmarks, **D-18**
seek distance comparison, **D-47**
seek time *vs.* distance, **D-46**
server utilization calculation, D-28–29
sorting case study, D-64–67
Tandem Computers, D-13
throughput *vs.* response time, D-16–18
TP benchmarks, D-18–20
transactions components, **D-17**
web server benchmarking, D-20–21
WSCs, 478
Store-and-forward packet switching, F-51
Store conditional
advantage, 414, 416
definition, 413–414
Store instructions, C-5–6, 199. *See also* Load-store instruction set architecture
Store unit
bandwidth for, 298–299
definition, 285
Streaming SIMD Extensions (SSE), 305
Stride, 300
vector memory systems, G-10–11
Strided accesses, 346
Strided addressing, A-31–32. *See also* Unit stride addressing
Striping, D-51
disk arrays, D-6
RAID, D-8
Strip-mined vector loop
convoys, G-5
DAXPY on VMIPS, G-20–21
Strip mining, 180, 296, **297**
DAXPY on VMIPS, G-20–21
Strong scaling, 439
Structural hazards
check for, C-52
definition, C-11
Subblocking, cache optimization, 114
Subset property, 423
Sun Microsystems, B-38
Sun Microsystems Enterprise, M-60
Sun Microsystems Niagara (T1/T2), multithreading history, M-35
Sun Microsystems SPARC
conditional instructions, H-27
integer arithmetic, J-11–12
integer overflow, **J-11**
RISC history, M-21
synchronization history, M-64–65
Sun Microsystems SPARCCenter, M-60
Sun Microsystems SPARCstation-2, F-92
Sun Microsystems SPARCstation-20, F-92
Sun Microsystems SPARC V8, floating-point precisions, J-33
Sun Microsystems SPARC VIS, multimedia support, **E-11**
Sun Microsystems UltraSPARC, M-63, M-74
Sun Microsystems UltraSPARC T1 processor, **F-78**
Sun Modular Datacenter, M-76
SUN servers, 94
Sun Ultra 5, **47**
Superblock scheduling
basic process, H-21–23, **H-22**
compiler history, M-33
Supercomputers, 10
clusters, **F-80**
commercial interconnection networks, **F-37**
direct network topology, **F-37**
SAN characteristics, F-30–31
SIMD, development, M-45–46
Superpipelining, C-55
Superscalar processors, 223
announcement, M-35
coarse-grained multithreading, 245

coining of term, M-31, M-34
dynamically scheduled, M-36, 224
functional unit execution slots, 244–245, **244**
ILP, M-33–34
recent advances, M-35
simultaneous multithreading, 245–247
Supervised learning, 547–548
Supervisor process, virtual memory, 119
Sussenguth, Ed, M-29–30
Sutherland, Ivan, M-35
Swap procedure, VAX, K-57
code example, K-59–60
full procedure, K-61
register allocation for, K-59
register preservation, K-60–61
Switched-media networks, F-2, F-24–25
Switched networks
centralized, F-31–35
distributed, F-35–40
Switches
context, B-49
early LANs and WANs, F-29
interconnecting node calculations, **F-32–33**
vs. NIC, **F-90**
process switch, B-49
statements, A-17
storage systems, D-34
switched-media networks, F-24–25
Switch fabric, switched-media networks, F-24–25
Switching, F-21–22, F-44–56
Switch microarchitecture, basic microarchitecture
Switch ports, F-30
Syllable, IA-64, H-35
Symbolic loop unrolling, software pipelining, H-12–15, **H-13**
Symmetric multiprocessors (SMP), F-106
characteristics, **I-45**
first vector computers, M-49
Symmetric shared-memory multiprocessors, 371
limitations, 389–392
performance, 393–404
commercial workload, 394–399
multiprogramming and OS workload, 399–404
scientific workloads, I-21–26, **I-23–26**
Synapse N+1, M-59–60
Synchronization, 352, 412
Cray X1, G-23
fetch-and-increment, 413–414
hardware primitives, 412–414
historical background, M-64–65
large-scale multiprocessors
barrier synchronization, I-13–16, **I-14**, **I-16**, **I-19**, I-20
hardware primitives, I-18–21
performance challenges, I-12–16
sense-reversing barrier, **I-21**
software implementations, I-17–18
tree-based barrier, **I-19**
locks using coherence, 414–417, **416**
message-passing communication, I-5
Synchronous dynamic random-access memory (SDRAM)
capacity and access times, **88**
IBM Blue Gene/L, I-42–43
memory hierarchy design, 87–90
power consumption reduction, 89–90
Synchronous events, exception, C-39
Synchronous I/O, definition, D-35
Synonyms, address translation, B-38
Synthetic benchmarks, 40
System area networks, F-76–77, **F-80**, F-104–106
System call, virtual memory, 119
System interface controller (SIF), F-74
System-on-chip (SoC)
cell phone, E-24
cost trends, 31
cross-company interoperability, F-23
DSAs, 592–594
embedded systems, E-3
Sanyo digital cameras, **E-20**
Sanyo VPC-SX500 digital camera, E-19
System response time, D-16
Systems software, 503
System/storage area networks (SANs)
characteristics, F-3
communication protocols, F-8
congestion management, F-68–70
cross-company interoperability, F-67–68
effective bandwidth, F-19
fat trees, F-34–35
fault tolerance, F-71
InfiniBand, F-77–81
interconnection network domain relationship, F-4, **F-5**
latency and effective bandwidth, F-29–30
packet latency, **F-13**, F-14–16
time of flight, F-14
System virtual machines, 120–121
Systolic array, 560

T

Tag, 383
AMD Opteron data cache, B-13–14
memory hierarchy, 81
registers, **202**
virtual memory fast address translation, B-46
write strategy, B-10
Tag check
MIPS R4000 pipeline, C-58–59
write strategy, B-10
Tag field, B-8–9
Tagged hybrid predictors, 188–190, **188**, **190**
Tail duplication, superblock scheduling, H-21
Tailgating, G-20–21
Tail latency, 473
Tail tolerant systems, 486
Tandem Computers, D-13
cluster history, M-62, M-74, M-87
Target address
branch hazards, C-18–19
branch penalty reduction, C-19–20
branch-target buffers, 231
pipeline branch issues, C-35–36
RISC instruction set, C-5
Target channel adapters (TCAs), F-90
Target instructions
branch-target buffers, 231
GPU conditional branching, 323
Task-level parallelism (TLP), 10
TB-80 VME rack, **D-38**, D-41

Technology trends
bandwidth over latency, 20
implementation technologies, 19–20
scaling of transistor performance and wires, 21–23
Temporal locality, B-26
coining of term, M-11
computer design principles, 49
definition, B-2
Tensor processing unit (TPU)
architecture, 557–558
block diagram, **558**
case study, 606–617
die, **562**
factors limiting, **598**
guidelines, 566–567
implementation, 560–563
improving, 564–566
instruction set architecture, 559
microarchitecture, 559–560
origin, 557
printed circuit board, **563**
software, 563
TensorFlow program, **564**
TERA processor, M-35
Terminate event, exception, C-40
Tertiary Disk project, D-12–13
Tesla, M-52
Test-and-set operation, synchronization, 413
Texas Instruments 8847
arithmetic functions, J-57–62
chip comparison, **J-58**
chip layout, **J-59–60**
Texas Instruments ASC, first vector computers, M-47
TFLOPS, parallel processing debates, M-58
Thacker, Chuck, F-103
Thermal design power (TDP), 24
Thin-film transistor (TFT), Sanyo VPC-SX500 digital camera, E-19
Thinking Machines, M-46, M-56, M-87
Thinking Multiprocessors CM-5, M-60–61
Think time, D-16
Third-level caches, 166, 262
interconnection network, F-91–92
Thrash, B-25–26
Thread Block, 311, 315
Thread Block Scheduler, 315, **316**
Thread-level parallelism (TLP), 5, 10–11, 369
centralized shared-memory multiprocessor, 371, 377
basic schemes for enforcing coherence, 379–380
cache coherence protocol, 377–379, **378**, 383–387, **384**
extensions to coherence protocol, 388
implementation techniques, 382–383
SMP and snooping limitations, 389–392
snooping coherence protocols, 380–381, **381**, 392–393
structure, **372**
definition, 242
directory-based cache coherence, 380
case study, 451–452
protocol example, 408–412
distributed shared memory, 371, 373
access time, 372–373
architecture, **373**
directory-based cache coherence, 404–412, **405**
disadvantages, 372–373
embedded systems, E-15
memory consistency, 379, 417–422
case study, 456–458
compiler optimization, 422
programmer's view, 418–419
relaxed consistency models, 419–422, **421**
speculation to hide latency, 422–423
multicore processor, 369, 371–372, 382, 387, 408
approaches, 389
architecture, **430**
coherence, 387
development, 404
DSM, **373**, **405**, **452**
Intel i7 920 performance and energy efficiency, 434–437
on multiprogrammed workload, 426–432
performance, 426–437, **432**
scalability in Xeon E7 with different workloads, 433–434
scaling, **432**, 442–444
single chip, 382, **391**, 446–451
and SMT, 436–437
multiprocessor architecture, 370–373
vs. multithreading, M-36
parallel processing challenges, 373–377
single-chip multicore processor, 446–451
synchronization, 412
hardware primitives, 412–414
locks using coherence, 414–417, **416**
Thread of SIMD instructions
GPU programming, 315–317, **316**
scheduling, **319**
Three-dimensional space, direct networks, F-39
Throttling packets, F-10
Throughput, 20, 39. *See also* Bandwidth
computing kernel, 350, **351**
definition, C-3, F-13
disk storage, **D-4**
DNN applications, 596–600
producer-server model, D-15–16, **D-16**
vs. response time, D-16–18
routing comparison, **F-54**
uniprocessor, 242–247
Thumb-2, K-3
16-bit instructions, **K-7**, **K-10**
register encodings, **K-7**
Tilera TILE-Gx processors, OCNs, F-3
Time, and cost, 30–31
Time-constrained scaling, I-33–34
Time division multiple access (TDMA), cell phones, E-25
Time of flight
communication latency, I-3
interconnection networks, F-14
Time-sharing, B-49–50
Timing independent, M-18

TI TMS320C55 DSP
architecture, **E-7**
characteristics, E-7–8
data operands, **E-6**
TI TMS320C6x DSP
architecture, **E-9**
characteristics, E-8–10
instruction packet, **E-10**
TLB. *See* Translation lookaside buffer (TLB)
TLP. *See* Thread-level parallelism (TLP)
Tomasulo's algorithm
advantages, 201
definition, 194–195
dynamic scheduling, 195–201
RAW, 217
RISC-V floating-point unit, **198**
steps in, **216**
TOP500, M-59
Top of Rack (ToR) switch, 477–478
Topology, F-21–22, F-30–44
Torus networks, **F-53–56**, F-76–77
Total cost of ownership (TCO), 577
case study, 519–521
DSAs, 600–601, **601**
resource allocation, 521–522
Total store ordering (TSO), 420, **421**
Tournament predictors, 184–188, **186**
advantage, 185–187
branch address, **186**
early schemes, M-29
local/global predictors, 184–188, **186**
Toy programs, 40
TPC-C benchmarks
definition, 44, 439
speedup, 440, **440**
TPC-C, file system benchmarking, D-18–20
TPU. *See* Tensor processing unit (TPU)
Trace compaction, H-19
Trace scheduling, H-19–21, **H-20**
Trace selection, definition, H-19
Traffic intensity, queuing theory, D-26
Trailer
messages, F-6
packet format, **F-7**
Transaction components, D-16, I-38–39
Transaction-processing (TP)
benchmarks, server performance, 43–44
storage system benchmarks, D-18–20
Transaction Processing Council (TPC), 43–45
benchmarks overview, D-18–20
Transfers, A-16. *See also* Data transfers
Transforms, DSP, E-5
Transient failure, F-70
Transient faults, D-11, 93
Transistor performance, scaling, 21–23
Translation lookaside buffer (TLB)
address translation, B-37, B-46, **B-47**
ARM Cortex-A53, 251–252
coining of term, M-9
interconnection network protection, F-91
misses, 346
Opteron, **B-47**, B-56–57
speculation, 237–238
Transmission Control Protocol (TCP), congestion management, F-69
Transmission Control Protocol/Internet Protocol (TCP/IP), F-86
ATM, F-102–103
headers, **F-88**
internetworking, F-85–89
reliance on, F-99
WAN, F-102
Transmission speed, interconnection network performance, F-13
Transmission time, F-14
communication latency, I-3
Transport latency
time of flight, F-14
topology, F-25–26
Transport layer, **F-84**
Transputer, F-105
Trap-handling routines, C-54
Tree-based barrier, large-scale multiprocessor synchronization, **I-19**
Tree height reduction, H-11
Trellis codes, definition, E-6–7
TRIPS Edge processor, F-67
Trojan horses, B-51–53
True dependence, finding, H-7–8
True sharing misses, 393–394, 397, **398**
TSMC, Stratton, F-3
TSO. *See* Total store ordering (TSO)
TSS operating system, M-9
Turbo mode in 2008, 28
Turing, Alan, M-4, M-20
Turn Model routing algorithm, F-48
Two-dimensional line buffer, 589–590
Two-level predictors, 183, 191
Two's complement, J-7–8
Two-way set associativity, B-8
average memory access time, B-19
conflict misses, B-23
Opteron data cache, B-13–14, **B-13**
2:1 cache rule of thumb, B-29
TX-2, M-35, M-50

U

Ultrix, DECstation 5000 reboots, **F-73**
UMA. *See* Uniform memory access (UMA)
Unbiased exponent, J-15
Uncached state, 406
Underflow
floating-point arithmetic, J-36–37, J-62
gradual, J-15, J-36
Unicasting, shared-media networks, F-24
Unified buffer, 558
Unified cache
AMD Opteron example, B-15, **B-15**
miss rate, B-16
Unified virtual memory, 330
Uniform memory access (UMA), 371
Uninterruptible power supply (UPS), 504
Uniprocessor, 377–378
cache coherence mechanism, **384**, 386
throughput, 242–247
Unit stride addressing, A-31–32
UNIVAC I, M-5, M-17
UNIX systems, B-38
block servers *vs.* filers, D-34
floating point remainder, J-32
miss statistics, **B-59**
seek distance comparison, **D-47**

UNIX systems *(Continued)*
vector processor history, G-26
workload, 399
Unoptimized code, C-70
Unpacked decimal, A-14, J-16
Unshielded twisted pair (UTP), F-104
Up*/down* routing, F-49
USB, Sony PlayStation 2 Emotion Engine case study, E-15
Use bit, B-45–46, B-52
User-level communication, F-8
User maskable events, exception, C-39
User nonmaskable events, exception, C-39
User requested events, exception, C-39
User Space Driver, 563
Utility computing, M-75–76
Utilization
I/O system calculations, D-26
queuing theory, D-25

V

Valid bit, B-8, 383, 393–394
address translation, B-46
AMD Opteron data cache, B-14
page table entry, B-52
Value prediction, 228, 234
VAPI, InfiniBand, F-81
Variable length, 14
Variables, random, distribution, D-26–34
VAX architecture
fallacies and pitfalls, K-65–67
instructions encoding, K-54–55
operands and addressing modes, K-51–54
operations, K-56–57
sort, K-62–65
swap, K-59–61
Vector architecture, 10, A-31, 282
computer development, M-45–46
execution time, 290–293
fallacy, 356
vs. graphics processing units, 331–334
memory banks, 298–299
memory systems, G-9–11
multidimensional arrays, 299–301
multiple lanes, 293–294
pitfall, 355–356
predicate registers, 296–298
processor example, 288–290
programming, 302–304
RV64V extension, 283–287, **284**
sparse matrices, 301–302
start-up latency and dead time, **G-8**
vector-length registers, 294–296
vector-register characteristics, **G-3**
Vector array, 585
Vector element, 289
Vector functional units, 285
Vector instruction
definition, 289
instruction-level parallelism, 170
Vectorized code, 289
Vectorizing compilers
effectiveness, G-14
FORTRAN test kernels, **G-15**
sparse matrices, G-12–13
Vector kernel implementation, case study, 357–359
Vector-length register (VLR), 294–296
performance, G-4–5
Vector load
bandwidth for, 298–299
definition, 285
Vector-mask control, 297
Vector-mask registers
Cray X1, G-21–22
Vector processor
Cray X1, G-21–24, **G-22–23**
Cray X1E, G-24
DAXPY on VMIPS, G-17, G-19–21
definition, 370
DSP media extensions, E-10
execution time, **G-7**
historical background, G-26–28
measures, G-15–16
NEC SX-9 vector processor, **309**
overview, G-25–26
performance, G-2–9
chaining, G-11–12, **G-12**
DAXPY on VMIPS, G-17
sparse matrices, G-12–14
start-up and multiple lanes, G-7–9
unchaining, **G-12**
vs. scalar processor, G-19
Sony PlayStation 2 Emotion Engine, E-17–18
start-up overhead, **G-4**
vector kernel implementation, 357–359
VMIPS on DAXPY, G-17, G-19–21
VMIPS on Linpack, G-17–19
Vector registers, 284
Very-large-scale integration (VLSI)
early computer arithmetic, J-63
interconnection network topology, F-30
RISC history, M-21
Wallace tree, J-52–53
Very long instruction word (VLIW)
compiler history, M-32
EPIC approach, M-33
IA-64, H-33
instruction set, 587, **587**
multiple issue processors, 218–222, **220**, **271**
multiple-issue processors, M-30
multithreading history, M-36
TI 320C6x DSP, E-8–10
VGA controller, M-51
VI interface, M-63, M-74
Virtual address
AMD Opteron data cache, B-12–13
memory hierarchy, **B-39**
miss rate *vs.* cache size, **B-37**
Opteron mapping, **B-55**
Opteron memory management, B-54–57, **B-55**
and page size, B-58
physical address, **B-45**
translation, B-36–40
virtual memory, **B-41**, B-42, B-44
Virtual cache, B-36–38
Virtual channels (VCs), F-47–48
HOL blocking, **F-60**
switching, F-52
switch microarchitecture pipelining, F-66
system area network, F-105–106
and throughput, F-97
Virtual cut-through switching, F-52
Virtual functions, A-17
Virtual instruction set architecture (VISA), 587–588
Virtualization
Intel 80x86 instruction, **145**
memory hierarchy design, 126–127

Virtual Machine Control State (VMCS), 146
Virtual machine monitor (VMM), 121
instruction set extension, 124–125
laissez faire attitude, 145
pitfall, 145
requirements, 122
Xen virtual machine, 126
Virtual machines (VMs), 491
early IBM work, M-10
hardware management, 121
impact on virtual memory, 123–124
instruction set architecture for, 122–123
protection via, 120–122
software management, 121
Virtual memory, B-2–3, B-40–49
address space, B-12, **B-41**, B-44, B-55
address translation, B-46, **B-47**
caches and, B-42–43, **B-42**, B-48–49, **B-48**
classes, B-43
instruction set extension, 124–125
Intel Pentium *vs.* AMD Opteron, B-57
paged example, B-54–57
page size selection, B-46–47
parameter ranges, **B-42**
Pentium *vs.* Opteron protection, B-57
protection, B-49–50, 119–120
questions, B-44–46
segmented example, B-51–54
virtual machine, 123–124
Virtual output queues (VOQs), F-60–61
VLIW. *See* Very Long Instruction Word (VLIW)
VLR. *See* Vector-length register (VLR)
VLSI. *See* Very-large-scale integration (VLSI)
VME rack, D-37–38, **D-38**
VMIPS
DAXPY, G-18–21
enhanced, DAXPY performance, G-19–21
peak performance on DAXPY, G-17
performance, G-4
on Linpack, G-17–19
sparse matrices, G-13
start-up penalties, **G-5**
vector execution time, G-6–7
vector performance measures, G-16
Voltage regulator controller (VRC), F-74
Volume, and cost, 30–31
Von Neumann computer, M-2–3
Von Neumann, John, M-2–5
Voodoo2, M-51

W

Wafer
definition, 31
RISC-V dies, **33**
yield, 34
Waiting line, D-25
Wallace tree
example, J-52–53, **J-53**
historical background, J-63
Wall-clock time, 39
scientific applications on parallel processors, I-33
WAR. *See* Write after read (WAR)
Warehouse-scale computer (WSC), 4–5, 9–10, 369–370, 466
active *vs.* inactive low power modes, 516
average memory latency, 480
capital costs, 516
case study, **487**
cloud computing
advantages, 490
AWS (*see* Amazon Web Services (AWS))
economies of scale, 491
fallacy, 514
cluster history, M-74–75
computer architecture of, 477–482
cost, 486–490
cost-performance, 515, 517
cost trends, 36
efficiency
and cost, 482–490
energy, 503
measuring, 483–486
fault tolerance, 516
Google
cooling, 506–508
networking, 510–511
power distribution, 504–506
racks, 509–510
servers, 512–513
hierarchy of switches, **477**
Layer 3 network, **481**
low-power servers, 519–521
memory hierarchy, 479–482
microsecond delays, 517
opportunities/problems, 468
performance, 514
power utilization effectiveness, 483, **484**
preventing, 501–503
programming models and workloads, 471–476
resource allocation, 521–522
server cost and power, 519–521
storage, 478
total cost of ownership, 519–521
Warp, M-32
Water-side economization, 508
Wavelength division multiplexing (WDM), F-103
WAW. *See* Write after write (WAW)
Way prediction, hit time, 98–99
WB cycle. *See* Write-back (WB) cycle
WCET. *See* Worst-case execution time (WCET)
Weak ordering, 420, **421**
Weak scaling, 439
Web servers
benchmarking, D-21
WAN, F-102
Weighted arithmetic mean time, D-27
Weight FIFO, 558
Weight memory, 558
Weitek 3364
arithmetic functions, J-57–61
chip comparison, **J-58**
chip layout, **J-59–60**
West-first routing, F-48
Wet-bulb temperature, 508
Whirlwind project, M-4
Wide area networks (WANs)
ATM, F-4
characteristics, F-4
cross-company interoperability, F-68
effective bandwidth, F-19
fault tolerance, F-71–73
historical overview, F-102–103
InfiniBand, F-77–78

Wide area networks (WANs) *(Continued)*
interconnection network domain relationship, F-4, **F-5**
latency and effective bandwidth, **F-27–29**
offload engines, F-8
packet latency, **F-13**, F-14–16
switching, F-51
time of flight, F-14
Wilkes, Maurice, M-3
Winchester disk design, M-86
Window, F-69
TCP/IP headers, **F-88**
Wireless networks
basic challenges, **E-21**
and cell phones, E-21–22
Wires, scaling of, 21–23
Within instructions exception, C-39
instruction set complications, C-45
stopping/restarting exception, C-41
Word(s)
AMD Opteron data cache, B-14–15
double, A-7, **A-8**, **A-14**, **A-44**, 300
DSP, E-6
half, A-8, **A-8**, **A-14**, **A-44**
Word count, **B-53**
Word displacement addressing, K-52
Word offset, C-28
Working set effect, I-24
Workload, 39–40
commercial, 394–399
measurements, 400
multiprogramming and OS, 399–404
phases, 399–400
RAID performance prediction, D-57–59
scalability in Xeon E7 with, 433–434
symmetric shared-memory multiprocessors, I-21–26, **I-23–26**
warehouse-scale computers, 471–476
Wormhole switching, F-52, F-97, F-105
Worst-case execution time (WCET), E-4
Write after read (WAR)
dynamic scheduling, 193
hazard, C-12, C-69
multiple-issue processors, M-30
program order, 174
register renaming, 196
TI TMS320C55 DSP, E-8
Tomasulo's algorithm, 195, 207
Write after write (WAW), C-12
check for, C-52
dynamic scheduling, 193
longer latency pipelines, C-49, C-51
multiple-issue processors, M-30
program order, 173
register renaming, 196
Tomasulo's algorithm, 195
Write allocate, B-11–12
Write-back cache, B-11–12
cache coherence protocol, 385, **385**
directory-based cache coherence protocol, 411
memory hierarchy, 81
snooping coherence, 380–384, **381**
uniprocessor, 386
Write-back (WB) cycle
data hazards stall minimization, C-13–14
MIPS R4000 pipeline, C-58–59
multicycle FP operations, C-52
RISC classic pipeline, C-8
RISC exception, C-43
RISC instruction set, C-5, **C-6**
RISC pipeline, C-33
RISC pipeline control, C-35, **C-36**
simple RISC implementation, C-29
Write broadcast protocol, 381
Write buffer, B-11, B-14, 382
memory hierarchy, 81
merging, 105–106, **106**
Write hit
cache coherence, **384–385**, 386, **387**
definition, B-11
Write invalidate protocol, 380
example, 385, **385**
implementation, 382–383
snooping coherence, **381**
Write miss, 411, 418
AMD Opteron data cache, B-12
cache coherence, **384–385**, 385, **387**
directory-based cache coherence protocol, 411
memory stall clock cycles, B-4
miss penalty reduction, B-35–36
operation, 408
Opteron data cache, B-12, B-14
options, B-11
Write result, 199
dynamic scheduling with scoreboard, C-69
hardware-based speculation, 217
instruction step, 211
Write serialization
cache coherence, 378–379
definition, 380–382, 413
Write stall, B-11
Write strategy
memory hierarchy, B-45–46
virtual memory, B-45–46
Write-through cache, B-11–12
average memory access time, B-16–17
coherence protocol, **378**, 382–384
memory hierarchy, 81
Write update protocol, 381

X

XALANCBMK benchmarks, 138
XBox, M-51–52
Xen virtual machine, 126
Xeon E7, 389, 426–429
feature, **427**
on-chip organizations, **428**
performance, 431, **432**
QuickPath Interconnect, 429
scalability, 433–434, **434**
Xerox Palo Alto Research Center, F-103
XIMD architecture, M-36
Xon/Xoff, interconnection networks, F-10–11, F-18

Z

Zero-copy protocols, F-8
Zero-load latency, F-75
Z-80 microcontroller, cell phones, E-24
Zuse, Konrad, M-4–5

Translation between GPU terms in book and official NVIDIA and OpenCL terms.

Type	More Descriptive Name used in this Book	Official CUDA/ NVIDIA Term	Book Definition and OpenCL Terms	Official CUDA/NVIDIA Definition
Program Abstractions	Vectorizable Loop	Grid	A vectorizable loop, executed on the GPU, made up of 1 or more "Thread Blocks" (or bodies of vectorized loop) that can execute in parallel. OpenCL name is "index range."	A Grid is an array of Thread Blocks that can execute concurrently, sequentially, or a mixture.
	Body of Vectorized Loop	Thread Block	A vectorized loop executed on a "Streaming Multiprocessor" (multithreaded SIMD processor), made up of 1 or more "Warps" (or threads of SIMD instructions). These "Warps" (SIMD Threads) can communicate via "Shared Memory" (Local Memory). OpenCL calls a thread block a "work group."	A Thread Block is an array of CUDA threads that execute concurrently together and can cooperate and communicate via Shared Memory and barrier synchronization. A Thread Block has a Thread Block ID within its Grid.
	Sequence of SIMD Lane Operations	CUDA Thread	A vertical cut of a "Warp" (or thread of SIMD instructions) corresponding to one element executed by one "Thread Processor" (or SIMD lane). Result is stored depending on mask. OpenCL calls a CUDA thread a "work item."	A CUDA Thread is a lightweight thread that executes a sequential program and can cooperate with other CUDA threads executing in the same Thread Block. A CUDA thread has a thread ID within its Thread Block.
Machine Object	A Thread of SIMD Instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a "Streaming Multiprocessor" (multithreaded SIMD processor). Results stored depending on a per element mask.	A Warp is a set of parallel CUDA Threads (e.g., 32) that execute the same instruction together in a multithreaded SIMT/SIMD processor.
	SIMD Instruction	PTX Instruction	A single SIMD instruction executed across the "Thread Processors" (SIMD lanes).	A PTX instruction specifies an instruction executed by a CUDA Thread.
Processing Hardware	Multithreaded SIMD Processor	Streaming Multiprocessor	Multithreaded SIMD processor that executes "Warps" (thread of SIMD instructions), independent of other SIMD processors. OpenCL calls it a "Compute Unit." However, CUDA programmer writes program for one lane rather than for a "vector" of multiple SIMD lanes.	A Streaming Multiprocessor (SM) is a multithreaded SIMT/SIMD processor that executes Warps of CUDA Threads. A SIMT program specifies the execution of one CUDA thread, rather than a vector of multiple SIMD lanes.
	Thread Block Scheduler	Giga Thread Engine	Assigns multiple "Thread Blocks" (or body of vectorized loop) to "Streaming Multiprocessors" (multithreaded SIMD processors).	Distributes and schedules Thread Blocks of a Grid to Streaming Multiprocessors as resources become available.
	SIMD Thread Scheduler	Warp Scheduler	Hardware unit that schedules and issues "Warps" (threads of SIMD instructions) when they are ready to execute; includes a scoreboard to track "Warp" (SIMD thread) execution.	A Warp Scheduler in a Streaming Multiprocessor schedules Warps for execution when their next instruction is ready to execute.
	SIMD Lane	Thread Processor	Hardware SIMD Lane that executes the operations in a "Warp" (thread of SIMD instructions) on a single element. Results stored depending on mask. OpenCL calls it a "Processing Element."	A Thread Processor is a datapath and register file portion of a Streaming Multiprocessor that executes operations for one or more lanes of a Warp.
Memory Hardware	GPU Memory	Global Memory	DRAM memory accessible by all "Streaming Multiprocessors" (or multithreaded SIMD processors) in a GPU. OpenCL calls it "Global Memory."	Global Memory is accessible by all CUDA Threads in any Thread Block in any Grid. Implemented as a region of DRAM, and may be cached.
	Private Memory	Local Memory	Portion of DRAM memory private to each "Thread Processor" (SIMD lane). OpenCL calls it "Private Memory."	Private "thread-local" memory for a CUDA Thread. Implemented as a cached region of DRAM.
	Local Memory	Shared Memory	Fast local SRAM for one "Streaming Multiprocessor" (multithreaded SIMD processor), unavailable to other Streaming Multiprocessors. OpenCL calls it "Local Memory."	Fast SRAM memory shared by the CUDA Threads composing a Thread Block, and private to that Thread Block. Used for communication among CUDA Threads in a Thread Block at barrier synchronization points.
	SIMD Lane Registers	Registers	Registers in a single "Thread Processor" (SIMD lane) allocated across full "Thread Block" (or body of vectorized loop).	Private registers for a CUDA Thread. Implemented as multithreaded register file for certain lanes of several warps for each thread processor.

RV64G Instruction Subset

Mnemonic	Function
Data transfer	*Move data to/from GPRs and FPRs*
`lb,lbu,lh,lhu,lw,lwu`	Load byte, half word, or word to lower portion of GPR with/without sign extension
`ld,sd`	Load or store a double word to GPR
`sb,sh,sw`	Store a byte, half word, or word from lowest portion of GPR to memory
`fld,flw,fsd,fsw`	Load or store a double word or word to/from the FPRs
ALU Operations	*Register-register and register immediate ALU operations*
`add,addi,addw,addiw`	Add, add immediate, add word, or add word immediate. Word version affects lower 32 bits.
`and,andi,or,ori,xor,xori`	AND, AND immediate, or OR immediate, exclusive OR, exclusive OR immediate
`auipc`	Add upper immediate to PC; puts sum of a shifted immediate and PC in a register
`lui`	Loads an immediate value into the upper portion of a word.
`mul,mulw,mulh,mulhsu, mulhu`	Multiply, multiply word, multiply halfword, multiply upper half, signed and unsigned. Word affects lower 32 bits.
`div,diw,divu`	Divide, divide word, divide unsigned.
`rem,remw,remu,remuw`	Remainder, remainder word, remainder unsigned.
`sll,slli,srl,srli,sra,srai`	Shift left /right logical, right arithmetic, immediate and with shift amount in a GPR.
`sllw,sllwi,srlw,srlwi, sraw, sraiw`	Word shifts: affect only the lower 32-bits or a GPR.
`slt,slti,sltiu,sltu`	Set Less then: if first operand less than the second, set destination to 1 else 0; immediate form and signed/unsigned.
`sub,subi,subw,subwi`	Subtract, subtract immediate. Word version affects lower 32 bits.
Control Transfer	*Branches, jumps, procedure calls*
`beq,bge,bgeu,blt,bltu,bne`	Compare two registers if condition is true branch to PC + offset
`jal,jalr`	Jump, Jump to register contents. The address of the next instruction is saved in designated register. Unconditional jump without link by setting destination register to x0.
Floating Point Operations	*Floating point instructions operating of FPRs.*
`fadd.*, fsub.*, fmul.*, fdiv.* , fsrt.*`	FP add, subtract, multiply, divide, and square root; single (.s) and double (.d) precision versions.
`fmadd.*, fmsub.*, fmnadd.*, fmnsub.*`	Multiply-add, multiply-subtract, negte multiply-add, negate multiply-subtract; single (.s) and double (.d) precision versions.
`fsgnj.*, sgnjn.*, fsgnjx.*`	Copy sign, inverse sign, or XOR of sign to first operand; single (.s) and double (.d) precision versions.
`fmin.*, fmax.*`	Minimum and maximum of two values; single (.s) and double (.d) precision versions.
`feq.*, flt.*, fle.*`	Floating point compares; single (.s) and double (.d) precision versions.
`fclass.*`	Classify type of FP value; single (.s) and double (.d) precision versions.
`fmv.*.x,fmv.x.*`	Move from/to GPRs; single (.s) and double (.d) precision versions.
`fcvt.d.s, fcvt.s.d`	Convert SP to DP or DP to SP
`fcvt.*.w, fcvt.*.wu, fcvt.*.i, fct.*.lu`	Convert from word or double word, signed or unsigned to DP or DP.
`fcvt.w.*, fcvt.wu.*, fcvt.i.*, fcvt.lu.*`	Convert to word or double word, signed or unsigned.

COMPUTER ARCHITECTURE

Sixth Edition

A Quantitative Approach

John L. Hennessy | David A. Patterson

Foreword by Norman P. Jouppi

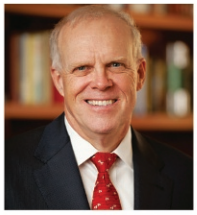
"This sixth edition comes at a critical time: Moore's Law is fading just as deep learning demands unprecedented compute cycles. The new chapter on domain-specific architectures documents a number of promising approaches and prophesies a rebirth in computer architecture. Like the scholars of the European Renaissance, computer architects must understand our own history, and then combine the lessons of that history with new techniques to remake the world."
—Cliff Young, Google

Computer Architecture: A Quantitative Approach, Sixth Edition has been considered essential reading by instructors, students, and practitioners of computer design for nearly 30 years. The sixth edition of this classic textbook is fully revised with the latest developments in processor and system architecture. It now features examples from the RISC-V ("RISC Five") instruction set architecture, a modern RISC instruction set developed and designed to be a free and openly adoptable standard. It also includes a new chapter on domain-specific architectures and an updated chapter on warehouse-scale computing that features the first public information on Google's newest WSC. True to its original mission of demystifying computer architecture, this edition continues the longstanding tradition of focusing on areas where the most exciting computing innovation is happening, while always keeping an emphasis on good engineering design.

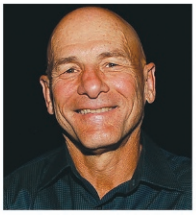
Features

- Includes a new chapter on domain-specific architectures, explaining how they are the only path forward for improved performance and energy efficiency given the end of Moore's Law and Dennard scaling
- Features the introduction of four DSAs from industry: Google Tensor Processing Unit, Google Pixel Visual Core, Intel Nervana Neural Network Processor, and Microsoft Catapult
- Features extensive updates to the chapter on warehouse-scale computing, with the first public information on the newest Google WSC
- Offers updates to other chapters including new material dealing with the use of stacked DRAM; data on the performance of new NVIDIA Pascal GPU vs. new AVX-512 Intel Skylake CPU; and extensive additions to content covering multicore architecture and organization

About the Authors



John L. Hennessy
Professor of Electrical Engineering and Computer Science
President Emeritus
Stanford University



David A. Patterson
Distinguished Engineer, Google
Pardee Chair of Computer Science, Emeritus
University of California at Berkeley

Computer Systems and Design
Computer Engineering

ISBN 978-0-12-811905-1

9 780128 119051

MORGAN KAUFMANN PUBLISHERS

AN IMPRINT OF ELSEVIER

elsevier.com/books-and-journals